

Eiichi Horigome

Table of contents

1		15
2		16
1	1	17
3	1-1 R	18
3.1	1.1 R	18
	3.1.1 R	18
	3.1.2	20
	3.1.3	22
	3.1.4	24
	3.1.5	25
	3.1.6	26
3.2	1.2 tidyverse	29
	3.2.1	29
	3.2.2 dplyr	31
	3.2.3 stringr	36
	3.2.4 tidyr	38
	3.2.5 ggplot2	40
3.3	1.3	45
	3.3.1 purrr	45
	3.3.2 nest	49
	3.3.3 future furrr	50
3.4	1.4	54
	3.4.1 dbplyr	54
	3.4.2 Arrow Parquet	54
	3.4.3 Arrow Dataset	55
	3.4.4 Spark	56
	3.4.5	57
	3.4.6	58
	3.4.7	58
	3.4.8 purrr	59
	3.4.9	60

3.5		61
3.5.1		61
4	I-2	63
4.1	2.1	63
4.2	2.2	63
	4.2.1 2.2.1	63
	4.2.2 2.2.2	64
	4.2.3 2.2.3	66
4.3	2.3	67
	4.3.1 2.3.1	67
	4.3.2 2.3.2	68
	4.3.3 2.3.3	70
4.4	2.4 ROE	73
	4.4.1 2.4.1 ROE	73
	4.4.2 2.4.2 ROE	76
	4.4.3 2.4.3	78
4.5	2.5	80
	4.5.1 2.5.1 ROE	80
	4.5.2 2.5.2 PM ATO	82
4.6	2.6	86
	4.6.1 2.6.1	86
	4.6.2 2.6.2	87
4.7	2.7	89
5	I-3	90
5.1	3.1	90
5.2	3.2	90
	5.2.1 3.2.1	90
	5.2.2 3.2.2	92
	5.2.3 3.2.3	93
5.3	3.3	96
	5.3.1 3.3.1	96
	5.3.2 3.3.2	98
	5.3.3 3.3.3	100
	5.3.4 3.3.4	102
5.4	3.4	108
	5.4.1 3.4.1	108
	5.4.2 3.4.2	110
	5.4.3 3.4.3	111
	5.4.4 3.4.4	113
5.5	3.5	114
	5.5.1 3.5.1	114

	5.5.2	3.5.2		116
	5.5.3	3.5.3		118
	5.5.4	3.5.4		118
	5.5.5	3.5.5		119
5.6	3.6			121
	5.6.1	3.6.1	t	121
	5.6.2	3.6.2		122
	5.6.3	3.6.3		126
5.7	3.7			128
5.8	3.8			130
6	I-4			131
6.1	4.1			131
6.2	4.2			131
	6.2.1	4.2.1	3	131
	6.2.2	4.2.2		135
	6.2.3	4.2.3		136
6.3	4.3			137
	6.3.1	4.3.1		137
	6.3.2	4.3.2		138
	6.3.3	4.3.3		143
6.4	4.4			146
	6.4.1	4.4.1	10	146
	6.4.2	4.4.2	10	149
6.5	4.5	CAPM		152
	6.5.1	4.5.1		152
	6.5.2	4.5.2		156
6.6	4.6	2×3	SMB HML	158
	6.6.1	4.6.1	Size-BE/ME	158
	6.6.2	4.6.2	SMB HML	162
	6.6.3	4.6.3		165
6.7	4.7	FF3	CAPM	167
	6.7.1	4.7.1	FF3	167
	6.7.2	4.7.2		172
6.8	4.8			176
6.9	4.9			177
7	I-5			179
7.1	5.1			179
	7.1.1	5.1.1		179
	7.1.2	5.1.2		180
	7.1.3	5.1.3		180

7.2	5.2 FF3	181
	7.2.1 5.2.1 FF3	181
	7.2.2 5.2.2	181
	7.2.3 5.2.3	182
	7.2.4 5.2.4	183
7.3	5.3	184
	7.3.1 5.3.1	184
	7.3.2 5.3.2	185
	7.3.3 5.3.3	187
	7.3.4 5.3.4	188
7.4	5.4	189
	7.4.1 5.4.1	189
	7.4.2 5.4.2	190
	7.4.3 5.4.3	191
	7.4.4 5.4.4	192
	7.4.5 5.4.5	193
7.5	5.5	193
	7.5.1 5.5.1	193
	7.5.2 5.5.2	194
	7.5.3 5.5.3	194
	7.5.4 5.5.4	195
7.6	5.6	195
	7.6.1 5.6.1	195
	7.6.2 5.6.2	196
	7.6.3 5.6.3	196
7.7	5.7	197
8	I-6	198
8.1	6.1	198
	8.1.1 6.1.1	198
	8.1.2 6.1.2	199
	8.1.3 6.1.3	199
8.2	6.2	201
	8.2.1 6.2.1	201
	8.2.2 6.2.2	202
	8.2.3 6.2.3	203
	8.2.4 6.2.4	203
	8.2.5 6.2.5	204
	8.2.6 6.2.6	206
	8.2.7 6.2.7	206
	8.2.8 6.2.8	207
8.3	6.3	208
	8.3.1 6.3.1	208

	8.3.2	6.3.2		210
	8.3.3	6.3.3		211
	8.3.4	6.3.4		212
	8.3.5	6.3.5		213
8.4	6.4			215
	8.4.1	6.4.1		215
	8.4.2	6.4.2	0	216
	8.4.3	6.4.3	0	217
	8.4.4	6.4.4		219
	8.4.5	6.4.5		220
	8.4.6	6.4.6		221
8.5	6.5			222
	8.5.1	6.5.1		222
	8.5.2	6.5.2		224
	8.5.3	6.5.3		225
	8.5.4	6.5.4		226
	8.5.5	6.5.5		227
	8.5.6	6.5.6		228
8.6	6.6			229
	8.6.1	6.6.1	AR CAR	229
	8.6.2	6.6.2	AAR CAAR	231
	8.6.3	6.6.3	CAAR	234
	8.6.4	6.6.4	AAR	236
	8.6.5	6.6.5	0 AAR	238
	8.6.6	6.6.6	CAAR	239
	8.6.7	6.6.7	AR	240
8.7	6.7			242
	8.7.1	6.7.1		242
	8.7.2	6.7.2	CAR	243
	8.7.3	6.7.3	CAR	245
	8.7.4	6.7.4	CAR	247
	8.7.5	6.7.5	CAR	248
	8.7.6	6.7.6	CAR	249
	8.7.7	6.7.7	CAR	251
8.8	6.8		CAR	252
	8.8.1	6.8.1		252
	8.8.2	6.8.2	CAR	253
	8.8.3	6.8.3		254
8.9	6.9			256
	8.9.1	6.9.1		256
	8.9.2	6.9.2		257
	8.9.3	6.9.3	Parquet	258
	8.9.4	6.9.4		259

8.10	6.10	260
9	I-7	261
9.1	7.1	261
9.2	7.2	262
	9.2.1	262
	9.2.2	263
	9.2.3	263
	9.2.4	264
	9.2.5	264
9.3	7.3 tidymodels	267
	9.3.1	268
	9.3.2	268
9.4	7.4	269
9.5	7.5	272
	9.5.1	272
	9.5.2	273
9.6	7.6	274
9.7	7.7 8	277
10	I-8	281
10.1	8.1	281
10.2	8.2	282
	10.2.1 7	282
	10.2.2	283
	10.2.3	283
10.3	8.3	285
	10.3.1	285
	10.3.2	285
	10.3.3	285
	10.3.4 VaR	285
	10.3.5	285
10.4	8.4	286
	10.4.1	286
	10.4.2	287
	10.4.3 nloptr	287
10.5	8.5	293
10.6	8.6	297
10.7	8.7	299

II	II	301
11	II-1	302
11.1	1.1	302
11.2	1.2	306
11.2.1	1.2.1	306
11.2.2	1.2.2	307
11.3	1.3	309
11.3.1	1.3.1	311
11.3.2	1.3.2	312
11.3.3	1.3.3	313
11.3.4	1.3.4	314
11.4	1.4	315
11.4.1	1.4.1	317
11.5	1.5	320
12	II-2 CTD BPV	322
12.1	2.1	322
12.1.1	2.1.1	322
12.1.2	2.1.2	323
12.1.3	2.1.3	324
12.2	2.2	329
12.2.1	2.2.1	329
12.2.2	2.2.2	334
12.3	2.3	335
12.3.1	2.3.1	335
12.3.2	2.3.2	337
12.4	2.4	340
12.4.1	2.4.1 Implied Term Structure	340
12.4.2	2.4.2	342
12.4.3	2.4.3	345
12.5	2.5	350
12.5.1	2.5.1	350
12.5.2	2.5.2	352
12.5.3	2.5.3	353
12.5.4	2.5.4	355
12.5.5	2.5.5	356
12.6	2.6 CTD BPV	358
12.6.1	2.6.1	358
12.6.2	2.6.2	358
12.6.3	2.6.3	359
12.6.4	2.6.4	359
12.6.5	2.6.5 CTD BPV	362

12.7	2.7	364
13	II-3 OIS	365
13.1	3.1 IBOR OIS	365
	13.1.1 3.1.1 IBOR	365
	13.1.2 3.1.2 OIS	365
13.2	3.2 TONA OIS	367
	13.2.1 3.2.1 TONA	367
	13.2.2 3.2.2 TONA OIS	368
	13.2.3 3.2.3 JPY OIS	370
	13.2.4 3.2.4	371
13.3	3.3 Fixing	372
	13.3.1 3.3.1 OIS	372
	13.3.2 3.3.2 Fixing	374
13.4	3.4 SOFR	376
	13.4.1 3.4.1 SOFR OIS	376
	13.4.2 3.4.2 SOFR OIS	377
	13.4.3 3.4.3	378
	13.4.4 3.4.4	380
13.5	3.5 1 EONIA OIS	382
	13.5.1 3.5.1 EONIA	382
	13.5.2 3.5.2 EONIA	383
	13.5.3 3.5.3	384
	13.5.4 3.5.4	385
	13.5.5 3.5.5 5 OIS	386
	13.5.6 3.5.6 OIS	387
	13.5.7 3.5.7	389
	13.5.8 3.5.8	390
13.6	3.6 2	391
	13.6.1 3.6.1	392
	13.6.2 3.6.2 EONIA	393
	13.6.3 3.6.3 OIS	395
	13.6.4 3.6.4	397
	13.6.5 3.6.5	399
	13.6.6 3.6.6 1	399
13.7	3.7	400
13.8		400
14	II-4	401
14.1	4.1 Black–Scholes–Merton Black-76	401
	14.1.1 4.1.1 Black–Scholes–Merton	401
	14.1.2 4.1.2 Black-76	402
	14.1.3 4.1.3 Black-76	402

14.2	4.2 Black-76	403
14.2.1	4.2.1 Black Greeks	403
14.2.2	4.2.2 Black European option	405
14.3	4.3 Black–Scholes–Merton European option	406
14.3.1	4.3.1 Black–Scholes–Merton	406
14.3.2	4.3.2 European put Tree	407
14.4	4.4 Monte Carlo	409
14.4.1	4.4.1 Monte Carlo	409
14.4.2	4.4.2 Monte Carlo	411
14.4.3	4.4.3 Black 1	413
14.5	4.5 American put	415
14.6	4.6 Monte Carlo Longstaff–Schwartz	417
14.6.1	4.6.1 Sobol	417
14.6.2	4.6.2 Longstaff–Schwartz	419
14.7	4.7	422
15	II-5 Norma	424
15.1	5.1 Normal	424
15.1.1	5.1.1	424
15.1.2	5.1.2 Bachelier	425
15.1.3	5.1.3 Normal volatility	425
15.2	5.2 SOFR	426
15.2.1	5.2.1	426
15.2.2	5.2.2 Greeks	428
15.3	5.3	429
15.3.1	5.3.1	429
15.3.2	5.3.2 Normal	431
15.3.3	5.3.3	432
15.4	5.4 Caplet Cap	434
15.5	5.5 Swaption	437
15.5.1	5.5.1	437
15.5.2	5.5.2 Bachelier Swaption	439
15.6	5.6 Black volatility Normal volatility	441
15.7	5.7	442
16	II-6 SABR	444
16.1	6.1 SABR	444
16.1.1	6.1.1 SABR	444
16.1.2	6.1.2 Alpha	444
16.1.3	6.1.3 Beta	445
16.1.4	6.1.4 Volatility of volatility	445
16.1.5	6.1.5 Rho	445
16.1.6	6.1.6 Hagan	445

16.2	6.2 Black SABR	445
16.2.1	6.2.1	445
16.2.2	6.2.2	447
16.2.3	6.2.3	450
16.3	6.3 SABR	452
16.4	6.4 Shifted SABR	456
16.5	6.5 Shifted SABR Normal	457
16.5.1	6.5.1	457
16.5.2	6.5.2	459
16.5.3	6.5.3 Normal	461
16.6	6.6	464
16.6.1	6.6.1 Strike	464
16.6.2	6.6.2 Black SABR Shifted SABR Normal	466
16.7	6.7	466
17	II-7 Early Termination	468
17.1	7.1 Hull–White	469
17.1.1	7.1.1 Hull–White 1-factor	469
17.1.2	7.1.2	469
17.1.3	7.1.3 Bullet Bond	470
17.1.4	7.1.4 Hull–White	472
17.2	7.2 Callable Bond Tree	474
17.2.1	7.2.1	474
17.2.2	7.2.2 Callable Bond Tree	475
17.3	7.3 Hull–White	476
17.4	7.4 Bermudan Receiver Swaption	478
17.5	7.5 Vasicek Hull–White	480
17.5.1	7.5.1 Vasicek	480
17.5.2	7.5.2 Vasicek Hull–White	481
17.6	7.6	485
17.6.1	7.6.1 Hull–White	485
17.6.2	7.6.2 1Y×5Y ATM Swap	485
17.6.3	7.6.3 One-helper calibration	487
17.6.4	7.6.4 European Swaption Engine	490
17.6.5	7.6.5 Tree step	492
17.7	7.7	495
18	II-8 CDS	497
18.1	8.1 CDS	498
18.1.1	8.1.1	498
18.1.2	8.1.2 Recovery Rate Loss Given Default	499
18.1.3	8.1.3 CDS Spread Hazard Rate	499

18.2	8.2	CDS	500
18.2.1	8.2.1	1 CDS	500
18.2.2	8.2.2	Hazard Rate	501
18.2.3	8.2.3	CDS Schedule	504
18.2.4	8.2.4	MidPoint CDS Engine	506
18.2.5	8.2.5	CDS	508
18.3	8.3	Premium Leg Protection Leg	509
18.3.1	8.3.1	Premium Leg	509
18.3.2	8.3.2	Protection Leg	510
18.3.3	8.3.3	Risky PV01 Fair Spread	510
18.4	8.4	Hazard Rate	512
18.4.1	8.4.1	Zero-NPV CDS Hazard Rate	512
18.4.2	8.4.2	Spread Recovery Rate	515
18.5	8.5	CDS2015 ISDA	516
18.5.1	8.5.1	CDS CDS2015	516
18.5.2	8.5.2	CDS2015 Schedule	518
18.5.3	8.5.3	ISDA	519
18.5.4	8.5.4	ISDA Hazard Rate	520
18.6	8.6	ISDA Engine	522
18.6.1	8.6.1	ISDA CDS Engine	522
18.6.2	8.6.2	Accrual rebate NPV	523
18.6.3	8.6.3	Fair Spread Fair Upfront	525
18.6.4	8.6.4	ISDA	526
18.6.5	8.6.5	MidPoint ISDA Engine	528
18.6.6	8.6.6	MidPoint Engine ISDA Engine	529
18.7	8.7		531
19	II-9	PRDC	533
19.1	9.1	PRDC	534
19.1.1	9.1.1		534
19.1.2	9.1.2	Coupon Schedule	536
19.1.3	9.1.3	Domestic Foreign Discount Curve	538
19.2	9.2	FX Coupon Payoff	540
19.2.1	9.2.1	FX Process	540
19.2.2	9.2.2	Coupon Payoff	541
19.3	9.3	Monte Carlo PRDC	545
19.3.1	9.3.1	Coupon FX	545
19.3.2	9.3.2	FX	547
19.3.3	9.3.3	PRDC	549
19.3.4	9.3.4	Coupon	552
19.4	9.4	Coupon	554
19.4.1	9.4.1	Coupon	554
19.4.2	9.4.2	Coupon Cap Floor	556

19.4.3	9.4.3 Coupon	557
19.5	9.5 FX Monte Carlo	559
19.5.1	9.5.1 FX Spot	559
19.5.2	9.5.2 FX Volatility	561
19.5.3	9.5.3 Monte Carlo	564
19.6	9.6 Part II	565
19.6.1	9.6.1 PRDC	565
19.6.2	9.6.2	565
19.6.3	9.6.3 FX volatility	566
19.6.4	9.6.4	566
19.6.5	9.6.5	566
19.6.6	9.6.6 Part II	566
19.7	9.7	568
III	III	570
20	III-1	571
20.1	1.1	571
	20.1.1	572
20.2	1.2	572
20.3	1.3 future	574
20.4	1.4 Arrow	576
20.5	1.5	578
21	III-2	581
21.1	2.1	581
21.2	2.2	583
21.3	2.3	586
21.4	2.4	588
21.5	2.5	591
22	III-3 PRDC	593
22.1	3.1	593
	22.1.1	594
	22.1.2	595
22.2	3.2 PRDC	597
	22.2.1	601
22.3	3.3	602
	22.3.1	604
	22.3.2 future_map()	607
22.4	3.4	610
	22.4.1 FX Spot	615

22.4.2	FX Volatility	617
22.4.3		618
22.5	3.5 Arrow	619

1

2

2008

20

PC

I

II

III II

R tidyverse R

III II

Part I

I

3 I-1 R

R Base R tidyverse dplyr stringr tidyr purrr ggplot2
Base R Posit [Base R Cheatsheet](#) purrr::map() |>

3.1 1.1 R

3.1.1 R

3.1.1.1

R <-

```
risk_free_rate <- 0.01
future_cash_flow <- 100

present_value <- future_cash_flow / (1 + risk_free_rate)

present_value
#> [1] 99.0099
```

snake_case

3.1.1.2

```
2 + 3
#> [1] 5
10 - 4
#> [1] 6
6 * 7
#> [1] 42
100 / 4
#> [1] 25
```

```

2^5
#> [1] 32

sqrt(16)
#> [1] 4
log(exp(1))
#> [1] 1
abs(-3)
#> [1] 3
round(pi, digits = 3)
#> [1] 3.142
signif(pi, digits = 4)
#> [1] 3.142

```

3.1.1.3

```

numeric_value <- 10.5
integer_value <- 10L
character_value <- "Finance"
logical_value <- TRUE

class(numeric_value)
#> [1] "numeric"
class(integer_value)
#> [1] "integer"
class(character_value)
#> [1] "character"
class(logical_value)
#> [1] "logical"

typeof(numeric_value)
#> [1] "double"
str(numeric_value)
#>  num 10.5

```

3.1.1.4

NA is.na()

```

returns_with_na <- c(0.02, NA, -0.01, 0.03)

is.na(returns_with_na)
#> [1] FALSE TRUE FALSE FALSE
sum(is.na(returns_with_na))
#> [1] 1

mean(returns_with_na, na.rm = TRUE)
#> [1] 0.01333333

```

3.1.2

3.1.2.1

```

interest_rates <- c(0.01, 0.015, 0.02)
year_sequence <- 1:5
fine_grid <- seq(from = 0, to = 1, by = 0.25)
repeated_values <- rep(100, times = 5)

interest_rates
#> [1] 0.010 0.015 0.020
year_sequence
#> [1] 1 2 3 4 5
fine_grid
#> [1] 0.00 0.25 0.50 0.75 1.00
repeated_values
#> [1] 100 100 100 100 100

```

3.1.2.2

R 1

```

interest_rates[1]
#> [1] 0.01
interest_rates[2:3]
#> [1] 0.015 0.020
interest_rates[-1]
#> [1] 0.015 0.020

```

```
interest_rates[interest_rates >= 0.015]
#> [1] 0.015 0.020
```

3.1.2.3

```
portfolio_weights <- c(stock = 0.6, bond = 0.3, cash = 0.1)

portfolio_weights
#> stock bond cash
#> 0.6 0.3 0.1
portfolio_weights["bond"]
#> bond
#> 0.3
sum(portfolio_weights)
#> [1] 1
```

3.1.2.4

R

```
discount_rates <- seq(0.01, 0.05, by = 0.01)
cash_flow <- 100

present_values <- cash_flow / (1 + discount_rates)

tibble(discount_rates, present_values)
#> # A tibble: 5 x 2
#>   discount_rates present_values
#>   <dbl>          <dbl>
#> 1 0.01          99.010
#> 2 0.02          98.039
#> 3 0.03          97.087
#> 4 0.04          96.154
#> 5 0.05          95.238
```

3.1.3

3.1.3.1

```
return_matrix <- matrix(
  c(0.01, 0.02, -0.01, 0.03, 0.00, 0.01),
  nrow = 3,
  ncol = 2,
  byrow = TRUE
)

colnames(return_matrix) <- c("stock_a", "stock_b")
rownames(return_matrix) <- c("month_1", "month_2", "month_3")

return_matrix
#>      stock_a stock_b
#> month_1    0.01    0.02
#> month_2   -0.01    0.03
#> month_3    0.00    0.01
return_matrix[, "stock_a"]
#> month_1 month_2 month_3
#>    0.01  -0.01    0.00
colMeans(return_matrix)
#> stock_a stock_b
#>    0.00    0.02
```

3.1.3.2

```
analysis_result <- list(
  model_name = "simple return model",
  coefficients = c(alpha = 0.001, beta = 1.05),
  observations = 60L,
  converged = TRUE
)

analysis_result$model_name
#> [1] "simple return model"
analysis_result$coefficients
#> alpha  beta
```

```
#> 0.001 1.050
analysis_result[["observations"]]
#> [1] 60
```

3.1.3.3

```
firm_data_base <- data.frame(
  firm_id = 1:3,
  firm_name = c("Firm A", "Firm B", "Firm C"),
  roe = c(0.08, 0.12, 0.15)
)

firm_data_base
#>   firm_id firm_name  roe
#> 1      1    Firm A 0.08
#> 2      2    Firm B 0.12
#> 3      3    Firm C 0.15
str(firm_data_base)
#> 'data.frame':    3 obs. of  3 variables:
#>  $ firm_id  : int  1 2 3
#>  $ firm_name: chr  "Firm A" "Firm B" "Firm C"
#>  $ roe      : num  0.08 0.12 0.15
```

tidyverse tibble

```
firm_data <- tibble(
  firm_id = 1:3,
  firm_name = c("Firm A", "Firm B", "Firm C"),
  roe = c(0.08, 0.12, 0.15)
)

firm_data
#> # A tibble: 3 x 3
#>   firm_id firm_name  roe
#>   <int> <chr>      <dbl>
#> 1     1 Firm A      0.08
#> 2     2 Firm B      0.12
#> 3     3 Firm C      0.15
```

tribble()

```
project_data <- tribble(
  ~project, ~initial_cost, ~cash_flow,
  "A",      100,      120,
  "B",      150,      175,
  "C",      200,      215
)

project_data
#> # A tibble: 3 x 3
#>   project initial_cost cash_flow
#>   <chr>      <dbl>      <dbl>
#> 1 A          100        120
#> 2 B          150        175
#> 3 C          200        215
```

3.1.4

3.1.4.1

```
npv <- 5.2

investment_decision <- if (npv >= 0) {
  "    "
} else {
  "    "
}

investment_decision
#> [1] "    "
```

case_when()

```
credit_scores <- tibble(score = c(85, 72, 58, 91))

credit_scores |>
  mutate(
    rating = case_when(
      score >= 80 ~ "A",
      score >= 65 ~ "B",
      TRUE ~ "C"
    )
  )
```



```

    )
  )
#> # A tibble: 4 x 2
#>   score rating
#>   <dbl> <chr>
#> 1    85 A
#> 2    72 B
#> 3    58 C
#> 4    91 A

```

3.1.4.2

```

calculate_present_value <- function(cash_flow, discount_rate, years = 1) {
  cash_flow / (1 + discount_rate)^years
}

calculate_present_value(cash_flow = 100, discount_rate = 0.02, years = 3)
#> [1] 94.23223

```

```

calculate_present_value(
  cash_flow = c(100, 100, 100),
  discount_rate = 0.02,
  years = 1:3
)
#> [1] 98.03922 96.11688 94.23223

```

3.1.5

3.1.5.1

```

sample_returns <- c(0.01, 0.03, -0.02, 0.00, 0.04, 0.02)

mean(sample_returns)
#> [1] 0.01333333
median(sample_returns)
#> [1] 0.015

```

```

min(sample_returns)
#> [1] -0.02
max(sample_returns)
#> [1] 0.04
range(sample_returns)
#> [1] -0.02 0.04
var(sample_returns)
#> [1] 0.0004666667
sd(sample_returns)
#> [1] 0.02160247
quantile(sample_returns)
#>      0%      25%      50%      75%     100%
#> -0.0200 0.0025 0.0150 0.0275 0.0400

```

3.1.5.2

```

stock_a <- c(0.01, 0.03, -0.02, 0.02, 0.04)
stock_b <- c(0.00, 0.02, -0.01, 0.01, 0.03)

cov(stock_a, stock_b)
#> [1] 0.00035
cor(stock_a, stock_b)
#> [1] 0.9615239

```

3.1.5.3

```

firm_returns <- c(0.08, 0.12, -0.03, 0.05)

rank(firm_returns)
#> [1] 3 4 1 2
rank(-firm_returns)
#> [1] 2 1 4 3
order(firm_returns, decreasing = TRUE)
#> [1] 2 1 4 3

```

3.1.6

R

r
d
p
q

3.1.6.1

```
set_random_seed_LFL(1234)

uniform_random <- runif(n = 10, min = 0, max = 1)

uniform_random
#> [1] 0.113703411 0.622299405 0.609274733 0.623379442 0.860915384 0.640310605
#> [7] 0.009495756 0.232550506 0.666083758 0.514251141
dunif(0.5, min = 0, max = 1)
#> [1] 1
punif(0.5, min = 0, max = 1)
#> [1] 0.5
qunif(0.95, min = 0, max = 1)
#> [1] 0.95
```

3.1.6.2

```
set_random_seed_LFL(1234)

normal_random <- rnorm(n = 10, mean = 0, sd = 1)

normal_random
#> [1] -1.2070657 0.2774292 1.0844412 -2.3456977 0.4291247 0.5060559
#> [7] -0.5747400 -0.5466319 -0.5644520 -0.8900378
dnorm(0, mean = 0, sd = 1)
#> [1] 0.3989423
pnorm(1.96, mean = 0, sd = 1)
#> [1] 0.9750021
qnorm(0.975, mean = 0, sd = 1)
#> [1] 1.959964
```

`dnorm()` `pnorm()` `qnorm()`

```
probability_below <- pnorm(-0.02, mean = 0.01, sd = 0.03)
value_at_risk_95 <- qnorm(0.05, mean = 0.01, sd = 0.03)

probability_below
#> [1] 0.1586553
value_at_risk_95
#> [1] -0.03934561
```

3.1.6.3

```
set_random_seed_LFL(1234)

binomial_random <- rbinom(n = 10, size = 20, prob = 0.6)

binomial_random
#> [1] 15 11 11 11 10 11 17 14 11 12
dbinom(12, size = 20, prob = 0.6)
#> [1] 0.1797058
pbinom(12, size = 20, prob = 0.6)
#> [1] 0.5841071
qbinom(0.95, size = 20, prob = 0.6)
#> [1] 16
```

3.1.6.4

```
set_random_seed_LFL(1234)

poisson_random <- rpois(n = 10, lambda = 3)

poisson_random
#> [1] 1 3 3 3 5 3 0 2 4 3
dpois(2, lambda = 3)
#> [1] 0.2240418
ppois(2, lambda = 3)
#> [1] 0.4231901
qpois(0.95, lambda = 3)
#> [1] 6
```

3.1.6.5

```
distribution_functions <- tribble(
  ~distribution, ~random_function, ~density_function, ~cumulative_function, ~quantile_function,
  " ", "runif()", "dunif()", "punif()", "qunif()",
  " ", "rnorm()", "dnorm()", "pnorm()", "qnorm()",
  " ", "rbinom()", "dbinom()", "pbinom()", "qbinom()",
  " ", "rpois()", "dpois()", "ppois()", "qpois()"
)

distribution_functions
#> # A tibble: 4 x 5
#>   distribution random_function density_function cumulative_function
#>   <chr>         <chr>         <chr>         <chr>
#> 1 runif()      dunif()      punif()
#> 2 rnorm()      dnorm()      pnorm()
#> 3 rbinom()     dbinom()     pbinom()
#> 4 rpois()      dpois()     ppois()
#>   quantile_function
#>   <chr>
#> 1 qunif()
#> 2 qnorm()
#> 3 qbinom()
#> 4 qpois()
```

3.2 1.2 tidyverse

3.2.1

data	raw	derived	R/utility_LFL.R
------	-----	---------	-----------------

```
tbl_daily_stock_return <-
  read_parquet_file_LFL(
    file.path("raw", "market", "daily_stock_returns.parquet")
  )

vec_required_columns <- c("date", "firm1", "firm2")
vec_missing_columns <- setdiff(vec_required_columns, names(tbl_daily_stock_return))

if (length(vec_missing_columns) > 0) {
```

```

    stop("      : ", paste(vec_missing_columns, collapse = ", "))
  }

show_table_LFL(tbl_daily_stock_return)
#> # A tibble: 21 x 3
#>   date          firm1      firm2
#>   * <date>         <dbl>     <dbl>
#> 1 2020-04-01 -0.039482  0.076963
#> 2 2020-04-02  0.0059787 -0.0072488
#> 3 2020-04-03  0.055787  -0.017304
#> 4 2020-04-06  0.041935  0.0021690
#> 5 2020-04-07 -0.020193  0.075545
#> 6 2020-04-08 -0.0022947 -0.096212
#> 7 2020-04-09  0.081707  0.078843
#> 8 2020-04-10  0.034561  0.021536
#> 9 2020-04-13  0.032601  0.13520
#> 10 2020-04-14 -0.037663  0.037979
#> # i 11 more rows

```

3.2.1.1

```

show_data_structure_LFL(tbl_daily_stock_return)
#> Rows: 21
#> Columns: 3
#> $ date <date> 2020-04-01, 2020-04-02, 2020-04-03, 2020-04-06, 2020-04-07, 202~
#> $ firm1 <dbl> -0.039482142, 0.005978689, 0.055786605, 0.041934669, -0.02019279~
#> $ firm2 <dbl> 0.0769625974, -0.0072488484, -0.0173040800, 0.0021689573, 0.0755~
summary(tbl_daily_stock_return)
#>      date          firm1      firm2
#> Min.   :2020-04-01   Min.   : -0.072071   Min.   : -0.0962117
#> 1st Qu.:2020-04-08   1st Qu.: -0.027029   1st Qu.: -0.0196147
#> Median :2020-04-15   Median : 0.005979   Median : -0.0003056
#> Mean   :2020-04-15   Mean   : 0.014161   Mean   : 0.0080253
#> 3rd Qu.:2020-04-22   3rd Qu.: 0.044812   3rd Qu.: 0.0379793
#> Max.   :2020-04-30   Max.   : 0.171297   Max.   : 0.1352008
nrow(tbl_daily_stock_return)
#> [1] 21
ncol(tbl_daily_stock_return)
#> [1] 3

```

```
names(tbl_daily_stock_return)
#> [1] "date" "firm1" "firm2"
```

3.2.2 dplyr

3.2.2.1

```
tbl_daily_stock_return |>
  select(date, firm1)
#> # A tibble: 21 x 2
#>   date          firm1
#>   <date>         <dbl>
#> 1 2020-04-01 -0.039482
#> 2 2020-04-02  0.0059787
#> 3 2020-04-03  0.055787
#> 4 2020-04-06  0.041935
#> 5 2020-04-07 -0.020193
#> 6 2020-04-08 -0.0022947
#> 7 2020-04-09  0.081707
#> 8 2020-04-10  0.034561
#> 9 2020-04-13  0.032601
#> 10 2020-04-14 -0.037663
#> # i 11 more rows
```

3.2.2.2

```
tbl_daily_stock_return |>
  filter(firm1 > 0)
#> # A tibble: 12 x 3
#>   date          firm1      firm2
#>   <date>         <dbl>    <dbl>
#> 1 2020-04-02  0.0059787 -0.0072488
#> 2 2020-04-03  0.055787  -0.017304
#> 3 2020-04-06  0.041935   0.0021690
#> 4 2020-04-09  0.081707   0.078843
#> 5 2020-04-10  0.034561   0.021536
#> 6 2020-04-13  0.032601   0.13520
#> 7 2020-04-15  0.0098798  0.011427
```

```
#> 8 2020-04-17 0.17130 0.039463
#> 9 2020-04-20 0.046325 0.037274
#> 10 2020-04-22 0.051543 -0.053818
#> 11 2020-04-27 0.0045375 -0.019615
#> 12 2020-04-30 0.044812 -0.013410
```

```
tbl_daily_stock_return |>
  filter(firm1 > 0, firm2 > 0)
#> # A tibble: 7 x 3
#>   date          firm1      firm2
#>   <date>         <dbl>     <dbl>
#> 1 2020-04-06 0.041935 0.0021690
#> 2 2020-04-09 0.081707 0.078843
#> 3 2020-04-10 0.034561 0.021536
#> 4 2020-04-13 0.032601 0.13520
#> 5 2020-04-15 0.0098798 0.011427
#> 6 2020-04-17 0.17130 0.039463
#> 7 2020-04-20 0.046325 0.037274
```

3.2.2.3

```
return_data <- tbl_daily_stock_return |>
  mutate(
    spread = firm1 - firm2,
    firm1_gross_return = 1 + firm1,
    firm2_gross_return = 1 + firm2,
    market_direction = case_when(
      firm1 > 0 & firm2 > 0 ~ "both_positive",
      firm1 < 0 & firm2 < 0 ~ "both_negative",
      TRUE ~ "mixed"
    )
  )

return_data
#> # A tibble: 21 x 7
#>   date          firm1      firm2      spread firm1_gross_return
#>   <date>         <dbl>     <dbl>     <dbl>         <dbl>
#> 1 2020-04-01 -0.039482  0.076963 -0.11644         0.96052
```



```

#> 2 2020-04-02 0.0059787 -0.0072488 0.013228 1.0060
#> 3 2020-04-03 0.055787 -0.017304 0.073091 1.0558
#> 4 2020-04-06 0.041935 0.0021690 0.039766 1.0419
#> 5 2020-04-07 -0.020193 0.075545 -0.095738 0.97981
#> 6 2020-04-08 -0.0022947 -0.096212 0.093917 0.99771
#> 7 2020-04-09 0.081707 0.078843 0.0028641 1.0817
#> 8 2020-04-10 0.034561 0.021536 0.013025 1.0346
#> 9 2020-04-13 0.032601 0.13520 -0.10260 1.0326
#> 10 2020-04-14 -0.037663 0.037979 -0.075642 0.96234
#> firm2_gross_return market_direction
#> <dbl> <chr>
#> 1 1.0770 mixed
#> 2 0.99275 mixed
#> 3 0.98270 mixed
#> 4 1.0022 both_positive
#> 5 1.0755 mixed
#> 6 0.90379 both_negative
#> 7 1.0788 both_positive
#> 8 1.0215 both_positive
#> 9 1.1352 both_positive
#> 10 1.0380 mixed
#> # i 11 more rows

```

3.2.2.4

```

return_data |>
  arrange(firm1)
#> # A tibble: 21 x 7
#>   date          firm1      firm2      spread firm1_gross_return
#>   <date>         <dbl>      <dbl>      <dbl>         <dbl>
#> 1 2020-04-28 -0.072071 -0.046873 -0.025197 0.92793
#> 2 2020-04-24 -0.041378 -0.0041105 -0.037268 0.95862
#> 3 2020-04-01 -0.039482 0.076963 -0.11644 0.96052
#> 4 2020-04-14 -0.037663 0.037979 -0.075642 0.96234
#> 5 2020-04-23 -0.028875 -0.035484 0.0066092 0.97112
#> 6 2020-04-16 -0.027029 -0.053488 0.026460 0.97297
#> 7 2020-04-07 -0.020193 0.075545 -0.095738 0.97981
#> 8 2020-04-21 -0.014593 -0.00030564 -0.014288 0.98541
#> 9 2020-04-08 -0.0022947 -0.096212 0.093917 0.99771
#> 10 2020-04-27 0.0045375 -0.019615 0.024152 1.0045

```

```

#>   firm2_gross_return market_direction
#>   <dbl> <chr>
#> 1      0.95313 both_negative
#> 2      0.99589 both_negative
#> 3      1.0770  mixed
#> 4      1.0380  mixed
#> 5      0.96452 both_negative
#> 6      0.94651 both_negative
#> 7      1.0755  mixed
#> 8      0.99969 both_negative
#> 9      0.90379 both_negative
#> 10     0.98039 mixed
#> # i 11 more rows

return_data |>
  arrange(desc(firm1))
#> # A tibble: 21 x 7
#>   date          firm1      firm2      spread firm1_gross_return
#>   <date>         <dbl>      <dbl>      <dbl>         <dbl>
#> 1 2020-04-17 0.17130    0.039463 0.13183      1.1713
#> 2 2020-04-09 0.081707   0.078843 0.0028641    1.0817
#> 3 2020-04-03 0.055787  -0.017304 0.073091    1.0558
#> 4 2020-04-22 0.051543  -0.053818 0.10536     1.0515
#> 5 2020-04-20 0.046325   0.037274 0.0090516    1.0463
#> 6 2020-04-30 0.044812  -0.013410 0.058222    1.0448
#> 7 2020-04-06 0.041935   0.0021690 0.039766    1.0419
#> 8 2020-04-10 0.034561   0.021536 0.013025    1.0346
#> 9 2020-04-13 0.032601   0.13520  -0.10260    1.0326
#> 10 2020-04-15 0.0098798 0.011427  -0.0015473 1.0099
#>   firm2_gross_return market_direction
#>   <dbl> <chr>
#> 1      1.0395 both_positive
#> 2      1.0788 both_positive
#> 3      0.98270 mixed
#> 4      0.94618 mixed
#> 5      1.0373 both_positive
#> 6      0.98659 mixed
#> 7      1.0022 both_positive
#> 8      1.0215 both_positive
#> 9      1.1352 both_positive
#> 10     1.0114 both_positive
#> # i 11 more rows

```

3.2.2.5

```
return_data |>
  summarise(
    observations = n(),
    mean_firm1 = mean(firm1),
    sd_firm1 = sd(firm1),
    mean_firm2 = mean(firm2),
    sd_firm2 = sd(firm2),
    correlation = cor(firm1, firm2)
  )
#> # A tibble: 1 x 6
#>   observations mean_firm1 sd_firm1 mean_firm2 sd_firm2 correlation
#>   <int>         <dbl>    <dbl>      <dbl>    <dbl>         <dbl>
#> 1         21    0.014161 0.053840  0.0080253 0.054390     0.23349
```

3.2.2.6

```
return_data |>
  group_by(market_direction) |>
  summarise(
    observations = n(),
    mean_firm1 = mean(firm1),
    mean_firm2 = mean(firm2),
    .groups = "drop"
  ) |>
  arrange(desc(observations))
#> # A tibble: 3 x 4
#>   market_direction observations mean_firm1 mean_firm2
#>   <chr>                <int>      <dbl>      <dbl>
#> 1 mixed                  8  0.0081649  0.0098864
#> 2 both_positive          7  0.059758   0.046559
#> 3 both_negative          6 -0.031040 -0.039412
```

3.2.2.7

```
return_data |>
  count(market_direction, sort = TRUE)
```

```

#> # A tibble: 3 x 2
#>   market_direction     n
#>   <chr>             <int>
#> 1 mixed              8
#> 2 both_positive      7
#> 3 both_negative      6

```

3.2.2.8

```

firm_master <- tribble(
  ~firm, ~firm_name, ~industry,
  "firm1", "Firm A", "Machinery",
  "firm2", "Firm B", "Chemicals"
)

return_summary <- return_data |>
  summarise(
    firm1 = mean(firm1),
    firm2 = mean(firm2)
  ) |>
  pivot_longer(
    cols = everything(),
    names_to = "firm",
    values_to = "mean_return"
  )

return_summary |>
  left_join(firm_master, by = "firm")
#> # A tibble: 2 x 4
#>   firm mean_return firm_name industry
#>   <chr>      <dbl> <chr>      <chr>
#> 1 firm1  0.014161 Firm A      Machinery
#> 2 firm2  0.0080253 Firm B      Chemicals

```

3.2.3 stringr

```
str_ stringr
```

```

company_names <- tibble(
  raw_name = c(
    " Alpha Holdings Co., Ltd. ",
    "Beta Financial Group",
    "Gamma Industries"
  )
)

company_names <- company_names |>
  mutate(
    clean_name = str_squish(raw_name),
    lower_name = str_to_lower(clean_name),
    name_length = str_length(clean_name),
    contains_group = str_detect(clean_name, regex("group", ignore_case = TRUE)),
    first_word = str_extract(clean_name, "^[A-Za-z]+")
  )

company_names
#> # A tibble: 3 x 6
#>   raw_name                clean_name
#>   <chr>                  <chr>
#> 1 " Alpha Holdings Co., Ltd. " Alpha Holdings Co., Ltd.
#> 2 "Beta Financial Group"      Beta Financial Group
#> 3 "Gamma Industries"         Gamma Industries
#>   lower_name                name_length contains_group first_word
#>   <chr>                    <int> <lgl>          <chr>
#> 1 alpha holdings co., ltd.      24 FALSE        Alpha
#> 2 beta financial group          20 TRUE         Beta
#> 3 gamma industries             16 FALSE        Gamma

```

3.2.3.1

```

company_names |>
  mutate(
    short_name = str_remove(clean_name, regex(" co\\.\\. , ltd\\.\\. $", ignore_case = TRUE)),
    display_name = str_c(first_word, " / ", name_length, " characters")
  ) |>
  select(clean_name, short_name, display_name)
#> # A tibble: 3 x 3
#>   clean_name                short_name                display_name

```

```
#>   <chr>                <chr>                <chr>
#> 1 Alpha Holdings Co., Ltd. Alpha Holdings      Alpha / 24 characters
#> 2 Beta Financial Group   Beta Financial Group Beta / 20 characters
#> 3 Gamma Industries       Gamma Industries   Gamma / 16 characters
```

3.2.4 tidyr

3.2.4.1 wide long

```
firm1 firm2      pivot_longer()

return_long <- tbl_daily_stock_return |>
  pivot_longer(
    cols = c(firm1, firm2),
    names_to = "firm",
    values_to = "return"
  )

return_long
#> # A tibble: 42 x 3
#>   date      firm      return
#>   <date>    <chr>    <dbl>
#> 1 2020-04-01 firm1 -0.039482
#> 2 2020-04-01 firm2  0.076963
#> 3 2020-04-02 firm1  0.0059787
#> 4 2020-04-02 firm2 -0.0072488
#> 5 2020-04-03 firm1  0.055787
#> 6 2020-04-03 firm2 -0.017304
#> 7 2020-04-06 firm1  0.041935
#> 8 2020-04-06 firm2  0.0021690
#> 9 2020-04-07 firm1 -0.020193
#> 10 2020-04-07 firm2  0.075545
#> # i 32 more rows
```

3.2.4.2 long wide

```
return_long |>
  pivot_wider(
    names_from = firm,
```

```

    values_from = return
  )
#> # A tibble: 21 x 3
#>   date      firm1      firm2
#>   <date>      <dbl>      <dbl>
#> 1 2020-04-01 -0.039482  0.076963
#> 2 2020-04-02  0.0059787 -0.0072488
#> 3 2020-04-03  0.055787  -0.017304
#> 4 2020-04-06  0.041935  0.0021690
#> 5 2020-04-07 -0.020193  0.075545
#> 6 2020-04-08 -0.0022947 -0.096212
#> 7 2020-04-09  0.081707  0.078843
#> 8 2020-04-10  0.034561  0.021536
#> 9 2020-04-13  0.032601  0.13520
#> 10 2020-04-14 -0.037663  0.037979
#> # i 11 more rows

```

3.2.4.3

```

security_codes <- tibble(
  security_id = c("JP-1001", "JP-1002", "US-2001")
)

security_codes |>
  separate_wider_delim(
    security_id,
    delim = "-",
    names = c("country", "code")
  )
#> # A tibble: 3 x 2
#>   country code
#>   <chr>   <chr>
#> 1 JP     1001
#> 2 JP     1002
#> 3 US     2001

```

3.2.4.4

```

missing_data <- tibble(
  firm = c("A", "B", "C"),
  return = c(0.02, NA, -0.01)
)

missing_data |>
  replace_na(list(return = 0))
#> # A tibble: 3 x 2
#>   firm  return
#>   <chr>  <dbl>
#> 1 A      0.02
#> 2 B      0
#> 3 C     -0.01

missing_data |>
  drop_na()
#> # A tibble: 2 x 2
#>   firm  return
#>   <chr>  <dbl>
#> 1 A      0.02
#> 2 C     -0.01

```

3.2.5 ggplot2

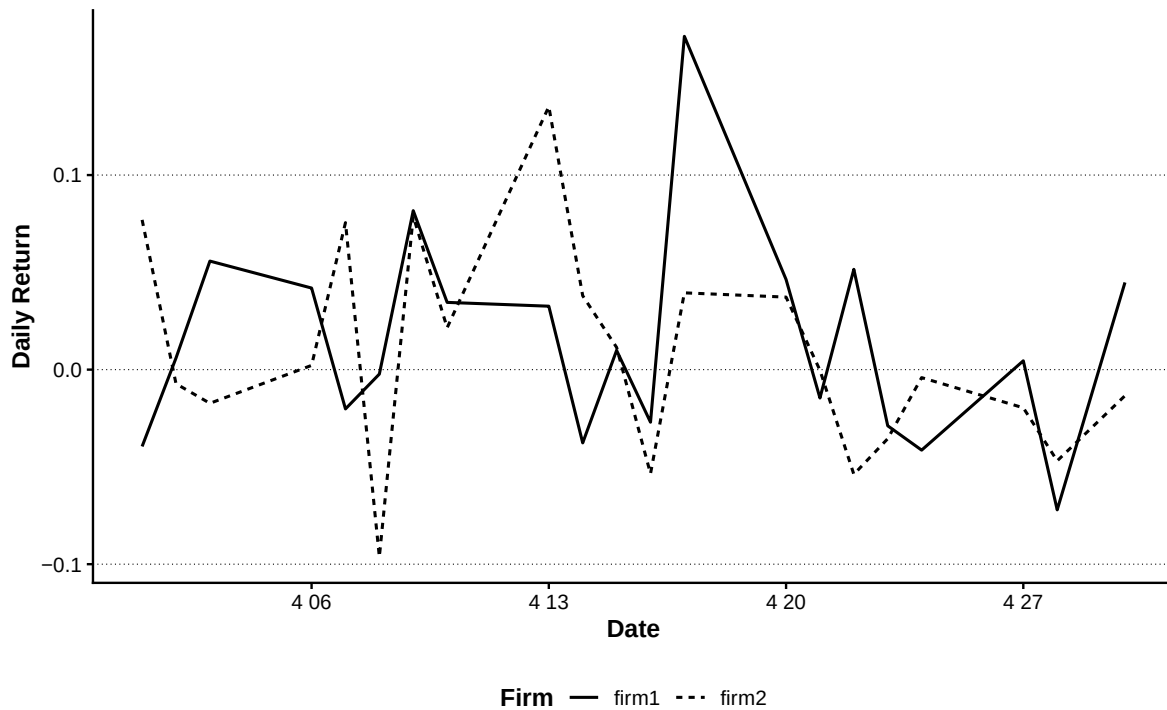
ggplot2 ggplot() aes() geom_*()

3.2.5.1

```

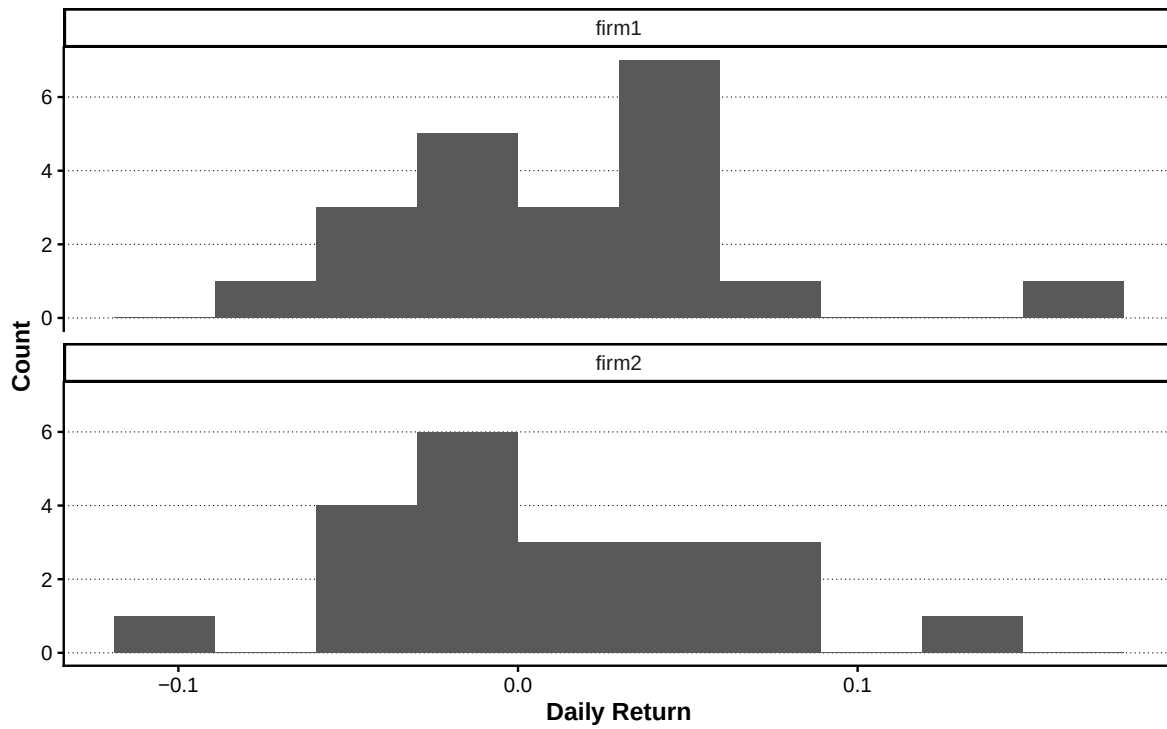
(return_long |>
  ggplot(aes(x = date, y = return, linetype = firm)) +
  geom_line(linewidth = 0.7) +
  labs(
    x = "Date",
    y = "Daily Return",
    linetype = "Firm"
  )) |> add_lagrange_theme_LFL()

```

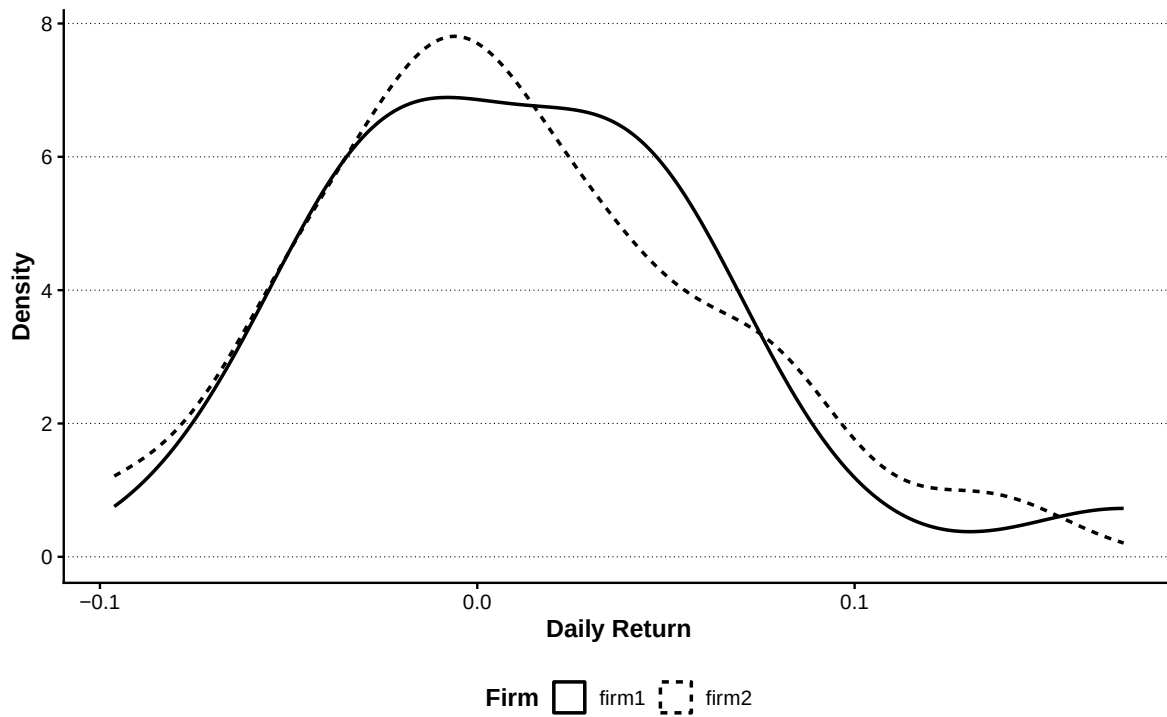
3.2.5.2

```
(return_long |>
  ggplot(aes(x = return)) +
  geom_histogram(bins = 10, boundary = 0) +
  facet_wrap(vars(firm), ncol = 1) +
  labs(
    x = "Daily Return",
    y = "Count"
  ) |>
  add_lagrange_theme_LFL())
```



3.2.5.3

```
(return_long |>
  ggplot(aes(x = return, linetype = firm)) +
  geom_density(linewidth = 0.8) +
  labs(
    x = "Daily Return",
    y = "Density",
    linetype = "Firm"
  ) |>
  add_lagrange_theme_LFL())
```



3.2.5.4

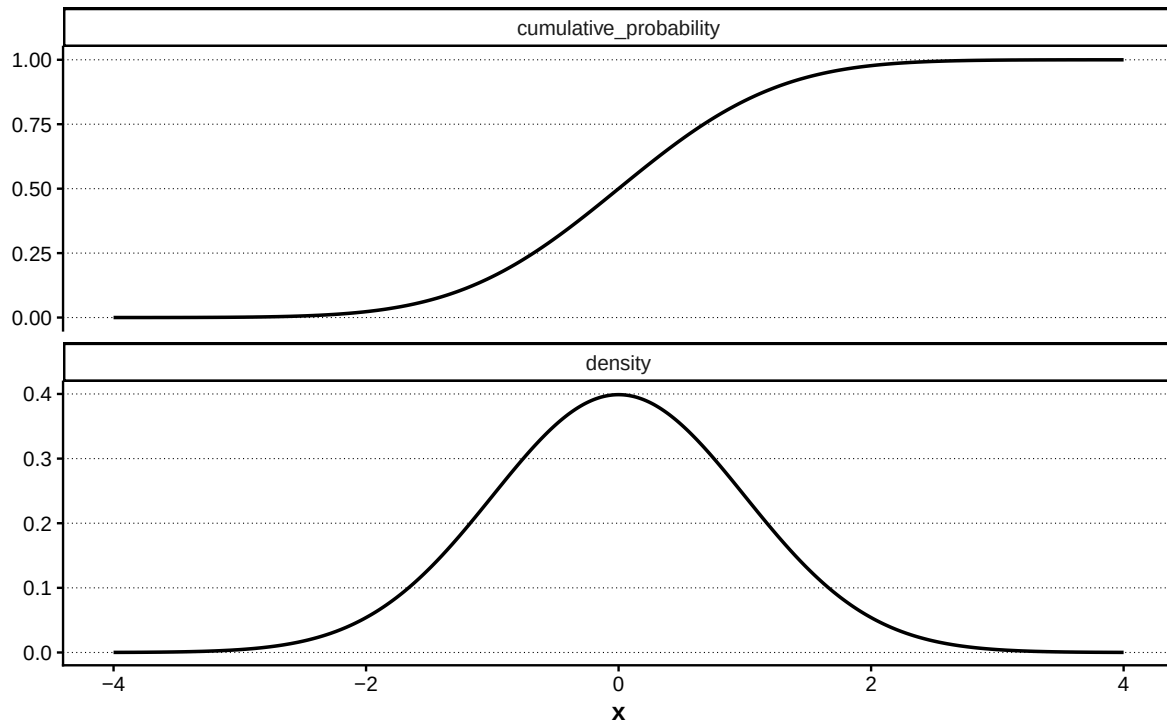
```
normal_curve <- tibble(
  x = seq(-4, 4, length.out = 401),
  density = dnorm(x),
  cumulative_probability = pnorm(x)
)

(normal_curve |>
  pivot_longer(
    cols = c(density, cumulative_probability),
    names_to = "function_type",
    values_to = "value"
  ) |>
  ggplot(aes(x = x, y = value)) +
  geom_line(linewidth = 0.8) +
  facet_wrap(vars(function_type), scales = "free_y", ncol = 1) +
  labs(
    x = "x",
```

```

y = NULL
)) |>
add_lagrange_theme_LFL()

```



3.2.5.5

```

simulated_distributions <- tibble(
  distribution = c("normal", "uniform", "exponential"),
  values = list(rnorm(10000), runif(10000), rexp(10000))
) |>
tidyr::unnest(values)

p <- simulated_distributions |>
ggplot(aes(x = values)) +
  geom_histogram(bins = 30) +
  facet_wrap(vars(distribution), scales = "free") +
  labs(
    x = "Simulated Value",

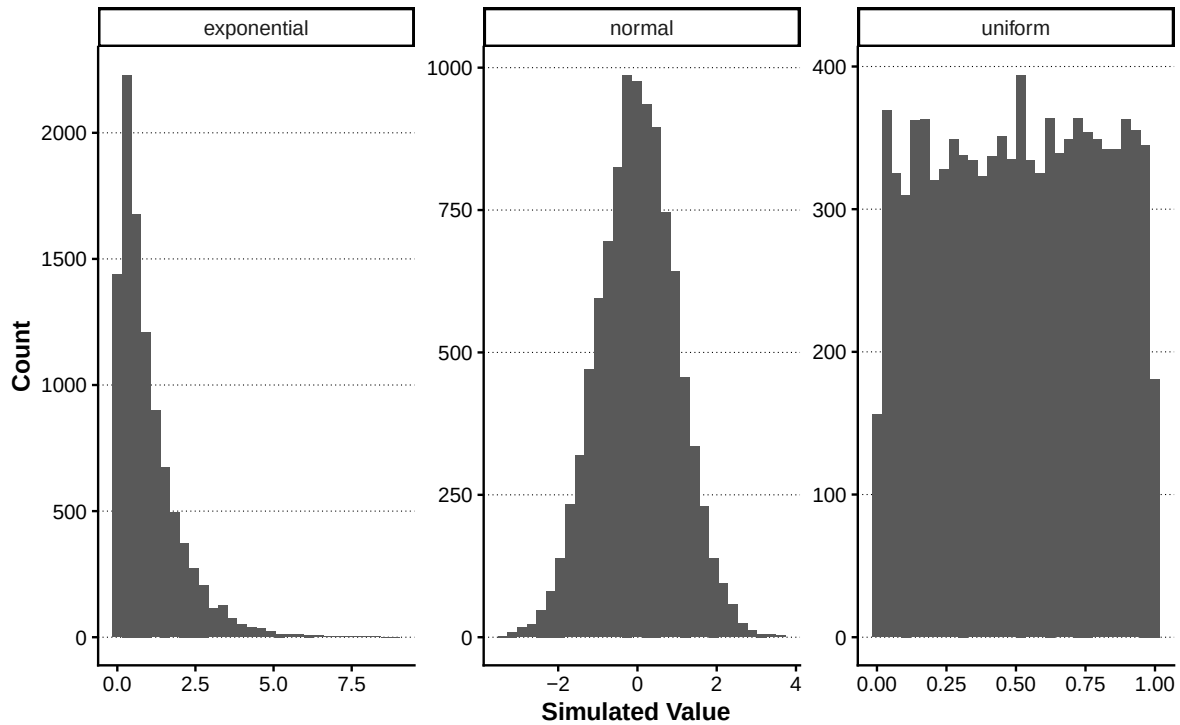
```

```

  y = "Count"
)

add_lagrange_theme_LFL(p)

```



3.3 1.3

3.3.1 purrr

3.3.1.1 map() map

map() map_dbl()

```

discount_rates <- c(0.01, 0.02, 0.03, 0.04)

discount_rates |>
  map(\(rate) calculate_present_value(100, rate))
#> [[1]]
#> [1] 99.0099

```

```

#>
#> [[2]]
#> [1] 98.03922
#>
#> [[3]]
#> [1] 97.08738
#>
#> [[4]]
#> [1] 96.15385

```

```

present_value_table <- tibble(
  discount_rate = discount_rates,
  present_value = discount_rates |>
    map_dbl(\(rate) calculate_present_value(100, rate))
)

```

```

present_value_table
#> # A tibble: 4 x 2
#>   discount_rate present_value
#>       <dbl>         <dbl>
#> 1      0.01      99.010
#> 2      0.02      98.039
#> 3      0.03      97.087
#> 4      0.04      96.154

```

3.3.1.2 map2()

```

cash_flows <- c(100, 120, 150)
years <- c(1, 2, 3)

map2_dbl(
  cash_flows,
  years,
  \(cash_flow, year) calculate_present_value(cash_flow, 0.02, year)
)
#> [1] 98.03922 115.34025 141.34835

```

3.3.1.3 pmap()

```
cash_flow_scenarios <- tribble(
  ~cash_flow, ~discount_rate, ~years,
    100,      0.01,      1,
    120,      0.02,      2,
    150,      0.03,      3
)

cash_flow_scenarios |>
  mutate(
    present_value = pmap_dbl(
      list(cash_flow, discount_rate, years),
      \(cash_flow, discount_rate, years) {
        calculate_present_value(cash_flow, discount_rate, years)
      }
    )
  )

#> # A tibble: 3 x 4
#>   cash_flow discount_rate years present_value
#>   <dbl>      <dbl> <dbl>      <dbl>
#> 1    100      0.01     1      99.010
#> 2    120      0.02     2     115.34
#> 3    150      0.03     3     137.27
```

3.3.1.4

```
return_columns <- c("firm1", "firm2")

return_columns |>
  map_dfr(
    \(column_name) {
      values <- tbl_daily_stock_return[[column_name]]

      tibble(
        firm = column_name,
        observations = length(values),
        mean = mean(values),
        standard_deviation = sd(values),
        minimum = min(values),

```

```

        maximum = max(values)
      )
    }
  )
#> # A tibble: 2 x 6
#>   firm observations      mean standard_deviation  minimum maximum
#>   <chr>         <int>    <dbl>          <dbl>      <dbl>   <dbl>
#> 1 firm1           21 0.014161          0.053840 -0.072071 0.17130
#> 2 firm2           21 0.0080253          0.054390 -0.096212 0.13520

```

3.3.1.5

```

set_random_seed_LFL(1234)

distribution_specs <- tribble(
  ~distribution, ~generator,
  "uniform",     \ (n) runif(n, min = -1, max = 1),
  "normal",      \ (n) rnorm(n, mean = 0, sd = 1),
  "poisson",     \ (n) rpois(n, lambda = 3)
)

simulated_distributions <- distribution_specs |>
  mutate(values = map(generator, \ (generator) generator(1000))) |>
  select(-generator) |>
  unnest_longer(values)

simulated_distributions
#> # A tibble: 3,000 x 2
#>   distribution values
#>   <chr>         <dbl>
#> 1 uniform     -0.77259
#> 2 uniform      0.24460
#> 3 uniform      0.21855
#> 4 uniform      0.24676
#> 5 uniform      0.72183
#> 6 uniform      0.28062
#> 7 uniform     -0.98101
#> 8 uniform     -0.53490
#> 9 uniform      0.33217

```



```
#> 10 uniform      0.028502
#> # i 2,990 more rows
```

3.3.1.6 walk()

walk()

```
c("firm1", "firm2") |>
  walk(\(firm) print(str_c("  : ", firm)))
#> [1] "  : firm1"
#> [1] "  : firm2"
```

3.3.2 nest

tidyr::nest() group_by() nest()

```
tbl_return_long <- tbl_daily_stock_return |>
  pivot_longer(
    cols = c(firm1, firm2),
    names_to = "firm",
    values_to = "return"
  )

tbl_nested <- tbl_return_long |>
  nest(tbl_data = c(date, return), .by = firm)

tbl_nested
#> # A tibble: 2 x 2
#>   firm tbl_data
#>   <chr> <list>
#> 1 firm1 <tibble [21 x 2]>
#> 2 firm2 <tibble [21 x 2]>
```

tbl_data tibble

```
tbl_nested$tbl_data[[1]]
#> # A tibble: 21 x 2
#>   date      return
#>   <date>      <dbl>
#> 1 2020-04-01 -0.039482
```

```
#> 2 2020-04-02 0.0059787
#> 3 2020-04-03 0.055787
#> 4 2020-04-06 0.041935
#> 5 2020-04-07 -0.020193
#> 6 2020-04-08 -0.0022947
#> 7 2020-04-09 0.081707
#> 8 2020-04-10 0.034561
#> 9 2020-04-13 0.032601
#> 10 2020-04-14 -0.037663
#> # i 11 more rows
```

nest() map()

```
tbl_nested_summary <- tbl_nested |>
  mutate(
    tbl_summary = map(
      tbl_data,
      \(tbl_data) tibble(
        num_mean_return = mean(tbl_data$return),
        num_volatility = sd(tbl_data$return)
      )
    )
  ) |>
  select(firm, tbl_summary) |>
  unnest(tbl_summary)

tbl_nested_summary
#> # A tibble: 2 x 3
#>   firm num_mean_return num_volatility
#>   <chr>          <dbl>          <dbl>
#> 1 firm1         0.014161         0.053840
#> 2 firm2         0.0080253        0.054390
```

3.3.3 future furr

purrr::map()	future furr	map()	R	Windows plan(multisession)
1000		4		

```
compute_pi_path <- function(int_path_id, int_points = 100000L) {
  set_random_seed_LFL(100000L + int_path_id)
```

```

vec_x <- runif(int_points, min = -1, max = 1)
vec_y <- runif(int_points, min = -1, max = 1)

tibble(
  int_path_id = int_path_id,
  num_pi = 4 * mean(vec_x^2 + vec_y^2 <= 1)
)
}

```

1000 CPU 8

```

int_paths <- 1000L
int_workers <- min(8L, future::availableCores())

tbl_chunks <- tibble(int_path_id = seq_len(int_paths)) |>
  mutate(
    int_chunk_id = rep(seq_len(int_workers), length.out = int_paths)
  ) |>
  nest(tbl_path_ids = int_path_id, .by = int_chunk_id)

tbl_chunks
#> # A tibble: 8 x 2
#>   int_chunk_id tbl_path_ids
#>   <int> <list>
#> 1         1 1 <tibble [125 x 1]>
#> 2         2 2 <tibble [125 x 1]>
#> 3         3 3 <tibble [125 x 1]>
#> 4         4 4 <tibble [125 x 1]>
#> 5         5 5 <tibble [125 x 1]>
#> 6         6 6 <tibble [125 x 1]>
#> 7         7 7 <tibble [125 x 1]>
#> 8         8 8 <tibble [125 x 1]>

```

```

compute_pi_chunk <- function(tbl_path_ids, int_points = 100000L) {
  tbl_path_ids$int_path_id |>
    map_dfr(compute_pi_path, int_points = int_points)
}

```

map()

```

future::plan(future::sequential)

num_time_map <- system.time({
  tbl_result_map <- tbl_chunks |>
    mutate(
      tbl_result = map(tbl_path_ids, compute_pi_chunk)
    ) |>
    select(int_chunk_id, tbl_result) |>
    unnest(tbl_result)
})[["elapsed"]]

```

future_map()

```

future::plan(future::multisession, workers = int_workers)

num_time_future_map <- system.time({
  tbl_result_future <- tbl_chunks |>
    mutate(
      tbl_result = future_map(
        tbl_path_ids,
        compute_pi_chunk,
        .options = furrr::furrr_options(seed = TRUE)
      )
    ) |>
    select(int_chunk_id, tbl_result) |>
    unnest(tbl_result)
})[["elapsed"]]

future::plan(future::sequential)

```

```

tbl_pi_benchmark <- tribble(
  ~chr_method, ~num_pi, ~num_elapsed_seconds,
  "map", mean(tbl_result_map$num_pi), num_time_map,
  str_c("future_map (", int_workers, " workers)", mean(tbl_result_future$num_pi), num_time_
) |>
  mutate(
    num_absolute_error = abs(num_pi - pi),
    num_speedup = first(num_elapsed_seconds) / num_elapsed_seconds
  )

```

```
tbl_pi_benchmark
#> # A tibble: 2 x 5
#>   chr_method      num_pi num_elapsed_seconds num_absolute_error
#>   <chr>          <dbl>          <dbl>          <dbl>
#> 1 map           3.1414           6.16           0.00016693
#> 2 future_map (8 workers) 3.1415           3.5            0.000094774
#>   num_speedup
#>   <dbl>
#> 1      1
#> 2    1.76
```

```
p <- tbl_pi_benchmark |>
  ggplot(aes(x = reorder(chr_method, num_elapsed_seconds), y = num_elapsed_seconds)) +
  geom_col() +
  coord_flip() +
  labs(x = NULL, y = "Elapsed time (seconds)")
  add_lagrange_theme_LFL(p)
```

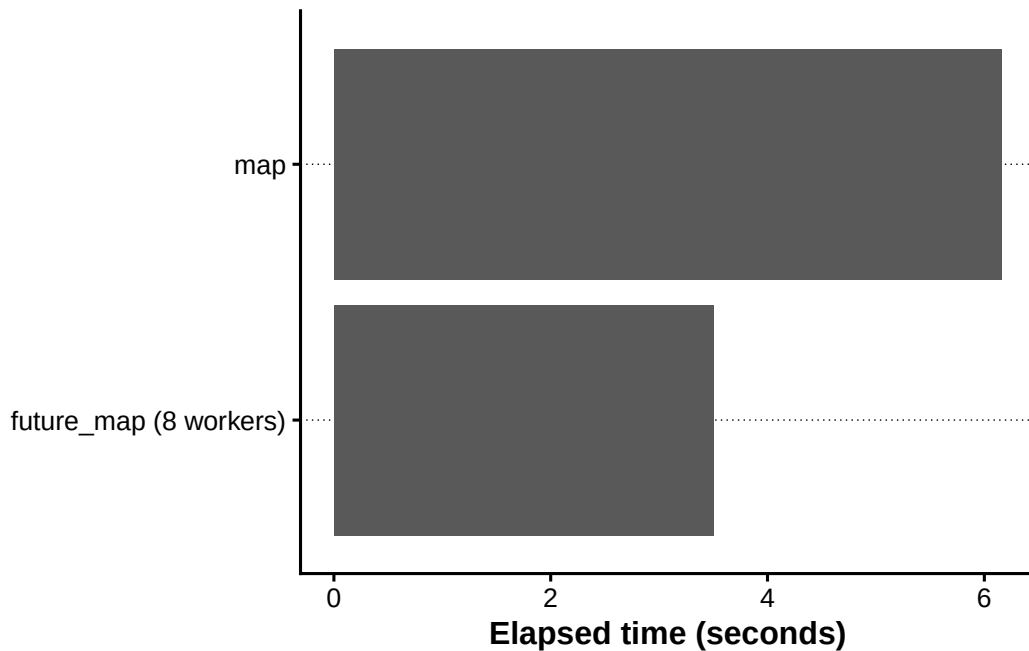


Figure 3.1: map future_map

map()

future_map()

3.4 1.4

R dplyr dbplyr Parquet Dataset Arrow

3.4.1 dbplyr

dbplyr dplyr SQL filter() select() mutate() summarise() collect() R
SQLite
dbplyr R SQL show_query() SQL
collect() R

3.4.2 Arrow Parquet

Apache Arrow R Parquet Dataset Python C++

```
set_random_seed_LFL(20260726)

tbl_arrow_price <- crossing(
  date = seq.Date(as.Date("2025-01-01"), by = "day", length.out = 365),
  chr_symbol = str_c("STOCK_", str_pad(1:20, width = 2, pad = "0"))
) |>
  arrange(chr_symbol, date) |>
  mutate(
    num_return = rnorm(n(), mean = 0.0002, sd = 0.015),
    num_price = 100 * exp(cumsum(num_return)),
    .by = chr_symbol
  )

tbl_arrow_price
#> # A tibble: 7,300 x 4
#>   date      chr_symbol num_return num_price
#>   <date>    <chr>         <dbl>    <dbl>
#> 1 2025-01-01 STOCK_01    -0.014847    98.526
#> 2 2025-01-02 STOCK_01     0.00017318   98.543
#> 3 2025-01-03 STOCK_01     0.0027754    98.817
#> 4 2025-01-04 STOCK_01     0.010886    99.899
#> 5 2025-01-05 STOCK_01     0.016885   101.60
#> 6 2025-01-06 STOCK_01     0.017499   103.39
#> 7 2025-01-07 STOCK_01     0.046333   108.30
```

```
#> 8 2025-01-08 STOCK_01 -0.017287 106.44
#> 9 2025-01-09 STOCK_01 0.0033907 106.80
#> 10 2025-01-10 STOCK_01 -0.0078836 105.96
#> # i 7,290 more rows
```

```
path_parquet <- file.path(tempdir(), "financial_prices.parquet")
```

```
arrow::write_parquet(tbl_arrow_price, sink = path_parquet)
```

```
path_parquet
```

```
#> [1] "C:\\Users\\besuk\\AppData\\Local\\Temp\\RtmpKqN3Pb/financial_prices.parquet"
```

```
arrow::read_parquet(path_parquet) |>
```

```
  slice_head(n = 10)
```

```
#> # A tibble: 10 x 4
```

#>	date	chr_symbol	num_return	num_price
#>	<date>	<chr>	<dbl>	<dbl>
#> 1	2025-01-01	STOCK_01	-0.014847	98.526
#> 2	2025-01-02	STOCK_01	0.00017318	98.543
#> 3	2025-01-03	STOCK_01	0.0027754	98.817
#> 4	2025-01-04	STOCK_01	0.010886	99.899
#> 5	2025-01-05	STOCK_01	0.016885	101.60
#> 6	2025-01-06	STOCK_01	0.017499	103.39
#> 7	2025-01-07	STOCK_01	0.046333	108.30
#> 8	2025-01-08	STOCK_01	-0.017287	106.44
#> 9	2025-01-09	STOCK_01	0.0033907	106.80
#> 10	2025-01-10	STOCK_01	-0.0078836	105.96

3.4.3 Arrow Dataset

Parquet Dataset

```
path_dataset <- file.path(tempdir(), "financial_dataset")
```

```
arrow::write_dataset(
  tbl_arrow_price,
  path = path_dataset,
  format = "parquet",
  partitioning = "chr_symbol",
  existing_data_behavior = "overwrite"
)
```

```
dataset_finance <- arrow::open_dataset(path_dataset)
```

```
dataset_finance
```

```
#> FileSystemDataset with 20 Parquet files
```

```
#> 4 columns
```

```
#> date: date32[day]
```

```
#> num_return: double
```

```
#> num_price: double
```

```
#> chr_symbol: string
```

```
Dataset      dplyr              collect()
```

```
tbl_arrow_selected <- dataset_finance |>
```

```
  filter(
```

```
    chr_symbol == "STOCK_01",
```

```
    date >= as.Date("2025-07-01")
```

```
  ) |>
```

```
  select(date, chr_symbol, num_price, num_return) |>
```

```
  collect()
```

```
tbl_arrow_selected
```

```
#> # A tibble: 184 x 4
```

```
#>   date      chr_symbol num_price num_return
```

```
#>   <date>    <chr>      <dbl>    <dbl>
```

```
#> 1 2025-07-01 STOCK_01    92.811  0.0061451
```

```
#> 2 2025-07-02 STOCK_01    95.183  0.025240
```

```
#> 3 2025-07-03 STOCK_01    97.579  0.024861
```

```
#> 4 2025-07-04 STOCK_01    97.833  0.0025990
```

```
#> 5 2025-07-05 STOCK_01    97.612 -0.0022665
```

```
#> 6 2025-07-06 STOCK_01    98.638  0.010456
```

```
#> 7 2025-07-07 STOCK_01    98.926  0.0029172
```

```
#> 8 2025-07-08 STOCK_01    98.611 -0.0031869
```

```
#> 9 2025-07-09 STOCK_01   100.28   0.016826
```

```
#> 10 2025-07-10 STOCK_01    99.207 -0.010806
```

```
#> # i 174 more rows
```

3.4.4 Spark

Arrow Spark

Arrow

Parquet Dataset

1 PC

Spark

1 PC Arrow dbplyr 1 CPU Spark Arrow dbplyr Spark
##

data/raw/market/daily_stock_returns.parquet

3.4.5

```
analysis_data <- tbl_daily_stock_return |>
  pivot_longer(
    cols = c(firm1, firm2),
    names_to = "firm",
    values_to = "return"
  ) |>
  mutate(
    gross_return = 1 + return,
    direction = case_when(
      return > 0 ~ "positive",
      return < 0 ~ "negative",
      TRUE ~ "zero"
    )
  ) |>
  arrange(firm, date)

analysis_data
#> # A tibble: 42 x 5
#>   date      firm      return gross_return direction
#>   <date>   <chr>      <dbl>      <dbl> <chr>
#> 1 2020-04-01 firm1 -0.039482    0.96052 negative
#> 2 2020-04-02 firm1  0.0059787    1.0060 positive
#> 3 2020-04-03 firm1  0.055787    1.0558 positive
#> 4 2020-04-06 firm1  0.041935    1.0419 positive
#> 5 2020-04-07 firm1 -0.020193    0.97981 negative
#> 6 2020-04-08 firm1 -0.0022947    0.99771 negative
#> 7 2020-04-09 firm1  0.081707    1.0817 positive
#> 8 2020-04-10 firm1  0.034561    1.0346 positive
#> 9 2020-04-13 firm1  0.032601    1.0326 positive
#> 10 2020-04-14 firm1 -0.037663    0.96234 negative
#> # i 32 more rows
```

3.4.6

```
firm_summary <- analysis_data |>
  group_by(firm) |>
  summarise(
    observations = n(),
    mean_return = mean(return),
    standard_deviation = sd(return),
    minimum_return = min(return),
    maximum_return = max(return),
    positive_ratio = mean(return > 0),
    .groups = "drop"
  )
```

```
firm_summary
#> # A tibble: 2 x 7
#>   firm observations mean_return standard_deviation minimum_return
#>   <chr>          <int>      <dbl>          <dbl>          <dbl>
#> 1 firm1             21    0.014161          0.053840        -0.072071
#> 2 firm2             21    0.0080253          0.054390        -0.096212
#>   maximum_return positive_ratio
#>   <dbl>          <dbl>
#> 1    0.17130        0.57143
#> 2    0.13520        0.47619
```

3.4.7

```
cumulative_return_data <- analysis_data |>
  group_by(firm) |>
  mutate(
    cumulative_gross_return = cumprod(gross_return),
    cumulative_return = cumulative_gross_return - 1
  ) |>
  ungroup()
```

```
cumulative_return_data
#> # A tibble: 42 x 7
#>   date      firm      return gross_return direction cumulative_gross_return
#>   <date>   <chr>    <dbl>      <dbl> <chr>          <dbl>
#> 1 2020-04-01 firm1 -0.039482    0.96052 negative      0.96052
```

```

#> 2 2020-04-02 firm1 0.0059787 1.0060 positive 0.96626
#> 3 2020-04-03 firm1 0.055787 1.0558 positive 1.0202
#> 4 2020-04-06 firm1 0.041935 1.0419 positive 1.0629
#> 5 2020-04-07 firm1 -0.020193 0.97981 negative 1.0415
#> 6 2020-04-08 firm1 -0.0022947 0.99771 negative 1.0391
#> 7 2020-04-09 firm1 0.081707 1.0817 positive 1.1240
#> 8 2020-04-10 firm1 0.034561 1.0346 positive 1.1628
#> 9 2020-04-13 firm1 0.032601 1.0326 positive 1.2007
#> 10 2020-04-14 firm1 -0.037663 0.96234 negative 1.1555
#> cumulative_return
#> <dbl>
#> 1 -0.039482
#> 2 -0.033740
#> 3 0.020165
#> 4 0.062945
#> 5 0.041481
#> 6 0.039091
#> 7 0.12399
#> 8 0.16284
#> 9 0.20075
#> 10 0.15553
#> # i 32 more rows

```

3.4.8 purrr

```

firm_names <- unique(analysis_data$firm)

firm_names |>
  map_dfr(
    \(firm_name) {
      firm_returns <- analysis_data |>
        filter(firm == firm_name) |>
        pull(return)

      tibble(
        firm = firm_name,
        mean_return = mean(firm_returns),
        volatility = sd(firm_returns),
        lower_5_percent = quantile(firm_returns, 0.05),
        upper_95_percent = quantile(firm_returns, 0.95)
      )
    }
  )

```

```

    )
  }
)
#> # A tibble: 2 x 5
#>   firm mean_return volatility lower_5_percent upper_95_percent
#>   <chr>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1 firm1    0.014161    0.053840    -0.041378    0.081707
#> 2 firm2    0.0080253   0.054390    -0.053818    0.078843

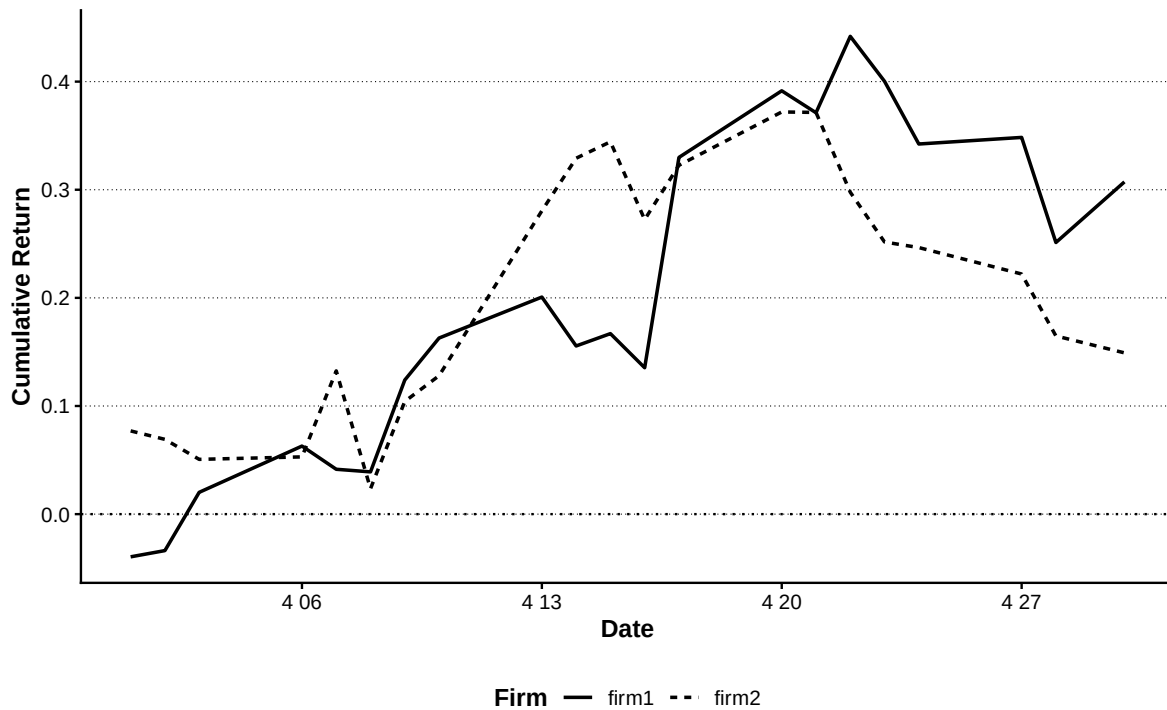
```

3.4.9

```

p <- cumulative_return_data |>
  ggplot(aes(x = date, y = cumulative_return, linetype = firm)) +
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = 0, linetype = "dotted") +
  labs(
    x = "Date",
    y = "Cumulative Return",
    linetype = "Firm"
  )
p |> add_lagrange_theme_LFL()

```



3.5

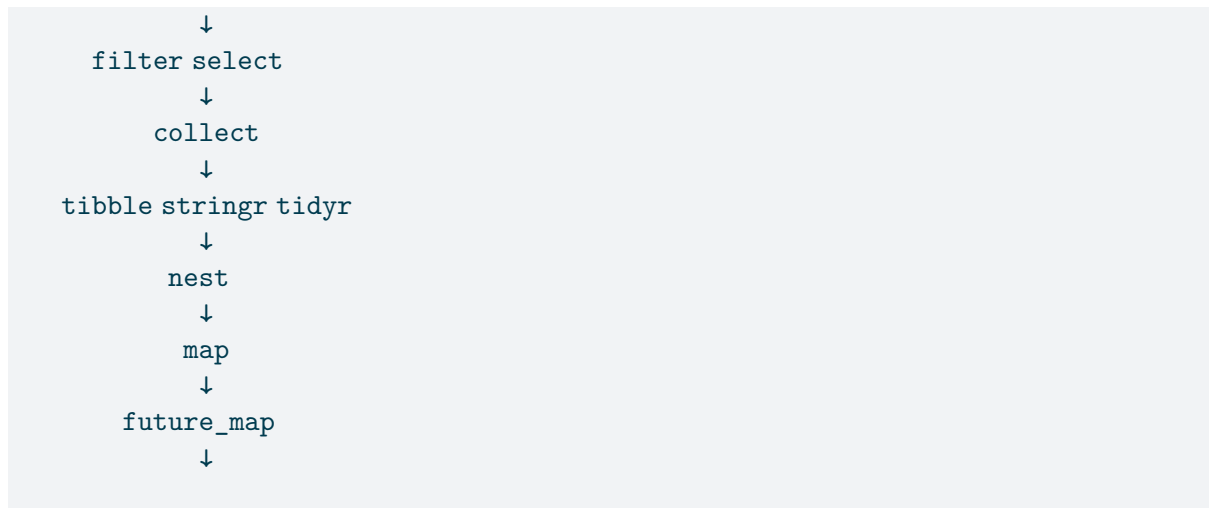
Base R

tidyverse

- snake_case
- purrr::map()
- dplyr stringr tidyr
- |>
- ggplot2
-

3.5.1

CSV Parquet
 ↓
 dbplyr Arrow



dbplyr Arrow

collect() tibble

nest()

map()

future_map()

4 I-2

4.1 2.1

4.2 2.2

4.2.1 2.2.1

```
financial_data_raw <-  
  read_parquet_file_LFL(  
    file.path(  
      "raw",  
      "fundamentals",  
      "financial_statements.parquet"  
    )  
  )  
  
show_table_LFL(financial_data_raw, n = 10)  
#> # A tibble: 7,920 x 11  
#>   year firm_ID industry_ID sales    OX    NFE    X    OA    FA  
#> * <dbl>   <dbl>         <dbl> <dbl> <dbl>   <dbl> <dbl>   <dbl> <dbl>  
#> 1  2015         1           1 5261.4 437.49 NA      286.64 13006. 3543.4  
#> 2  2016         1           1 5949.0 564.14 50.667  513.48 13866. 4642.2  
#> 3  2017         1           1 6505.1 691.18 29.543  661.64 13953. 7744.0  
#> 4  2018         1           1 6846.4 751.29 86.487  664.8  18818. 7284.7  
#> 5  2019         1           1 7572.2 958.53 298.05  660.48 18190  9735.1  
#> 6  2020         1           1 7537.6 778.37 -65.459 843.83 20463. 10274.  
#> 7  2015         2           1 3505.8  45.82  5.7511  40.07  2977.8 2258.3  
#> 8  2016         2           1 3491.3  51.25  1.8765  49.37  3184.8 1881.8  
#> 9  2017         2           1 3945.7  83.43  7.5279  75.9   3392.2 2425.2  
#> 10 2018         2           1 4139.3  93.4   6.8166  86.58  3569.3 2331.0  
#>      OL    FO
```

```

#> * <dbl> <dbl>
#> 1 4373.0 2480.7
#> 2 4534.2 3959.7
#> 3 5111.2 6159.0
#> 4 5137.3 10124.
#> 5 5488.0 11362.
#> 6 5371.4 13772.
#> 7 1840.4 2340.9
#> 8 1769.2 2215.9
#> 9 1955.3 2727.4
#> 10 2022.1 2694.5
#> # i 7,910 more rows

```

4.2.2 2.2.2

```

show_data_structure_LFL(financial_data_raw)
#> Rows: 7,920
#> Columns: 11
#> $ year <dbl> 2015, 2016, 2017, 2018, 2019, 2020, 2015, 2016, 2017, 2018~
#> $ firm_ID <dbl> 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 4, 4~
#> $ industry_ID <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ sales <dbl> 5261.40, 5948.96, 6505.06, 6846.38, 7572.24, 7537.63, 3505~
#> $ OX <dbl> 437.49, 564.14, 691.18, 751.29, 958.53, 778.37, 45.82, 51.~
#> $ NFE <dbl> NA, 50.667498, 29.543157, 86.486500, 298.049774, -65.45877~
#> $ X <dbl> 286.64, 513.48, 661.64, 664.80, 660.48, 843.83, 40.07, 49.~
#> $ OA <dbl> 13005.55, 13865.58, 13952.58, 18818.48, 18190.00, 20462.86~
#> $ FA <dbl> 3543.43, 4642.16, 7743.99, 7284.72, 9735.13, 10274.25, 225~
#> $ OL <dbl> 4372.96, 4534.22, 5111.22, 5137.28, 5487.96, 5371.38, 1840~
#> $ FO <dbl> 2480.72, 3959.70, 6159.02, 10123.91, 11362.22, 13772.15, 2~

financial_data_overview <- financial_data_raw |>
  summarise(
    observations = n(),
    variables = ncol(financial_data_raw),
    first_year = min(year, na.rm = TRUE),
    last_year = max(year, na.rm = TRUE),
    firms = n_distinct(firm_ID),
    industries = n_distinct(industry_ID)
  )

```



```
show_table_LFL(financial_data_overview)
#> # A tibble: 1 x 6
#>   observations variables first_year last_year firms industries
#>   <int>      <int>      <dbl>    <dbl> <int>      <int>
#> 1      7920        11      2015      2020  1515        10
```

```
missing_summary <- financial_data_raw |>
  summarise(
    across(
      everything(),
      list(
        missing = \(x) sum(is.na(x)),
        missing_rate = \(x) mean(is.na(x))
      ),
      .names = "{.col}__{.fn}"
    )
  ) |>
  pivot_longer(
    cols = everything(),
    names_to = c("variable", ".value"),
    names_sep = "__"
  ) |>
  arrange(desc(missing_rate), variable)

show_table_LFL(missing_summary, n = nrow(missing_summary))
#> # A tibble: 11 x 3
#>   variable      missing missing_rate
#>   <chr>      <int>      <dbl>
#> 1 NFE          1  0.00012626
#> 2 FA           0  0
#> 3 FO           0  0
#> 4 OA           0  0
#> 5 OL           0  0
#> 6 OX           0  0
#> 7 X            0  0
#> 8 firm_ID      0  0
#> 9 industry_ID  0  0
#> 10 sales       0  0
#> 11 year        0  0
```

4.2.3 2.2.3

ID ID ID

```
financial_data <- financial_data_raw |>
  mutate(
    firm_ID = as.factor(firm_ID),
    industry_ID = as.factor(industry_ID),
    year = as.integer(year)
  ) |>
  arrange(firm_ID, year)

show_data_structure_LFL(financial_data)
#> Rows: 7,920
#> Columns: 11
#> $ year      <int> 2015, 2016, 2017, 2018, 2019, 2020, 2015, 2016, 2017, 2018~
#> $ firm_ID    <fct> 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 4, 4~
#> $ industry_ID <fct> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ sales      <dbl> 5261.40, 5948.96, 6505.06, 6846.38, 7572.24, 7537.63, 3505~
#> $ OX         <dbl> 437.49, 564.14, 691.18, 751.29, 958.53, 778.37, 45.82, 51.~
#> $ NFE        <dbl> NA, 50.667498, 29.543157, 86.486500, 298.049774, -65.45877~
#> $ X          <dbl> 286.64, 513.48, 661.64, 664.80, 660.48, 843.83, 40.07, 49.~
#> $ OA         <dbl> 13005.55, 13865.58, 13952.58, 18818.48, 18190.00, 20462.86~
#> $ FA         <dbl> 3543.43, 4642.16, 7743.99, 7284.72, 9735.13, 10274.25, 225~
#> $ OL         <dbl> 4372.96, 4534.22, 5111.22, 5137.28, 5487.96, 5371.38, 1840~
#> $ FO         <dbl> 2480.72, 3959.70, 6159.02, 10123.91, 11362.22, 13772.15, 2~
```

```
panel_key_duplicates <- financial_data |>
  count(
    firm_ID,
    year,
    name = "observations"
  ) |>
  filter(observations > 1)

show_table_LFL(panel_key_duplicates)
#> # A tibble: 0 x 3
#> # i 3 variables: firm_ID <fct>, year <int>, observations <int>
```

```

if (nrow(panel_key_duplicates) > 0) {
  stop(
    "firm_ID year      ",
    call. = FALSE
  )
}

```

4.3 2.3

4.3.1 2.3.1

```

firms_by_year <- financial_data |>
  group_by(year) |>
  summarise(
    firms = n_distinct(firm_ID),
    observations = n(),
    .groups = "drop"
  ) |>
  arrange(year)

show_table_LFL(firms_by_year, n = nrow(firms_by_year))
#> # A tibble: 6 x 3
#>   year firms observations
#>   <int> <int>         <int>
#> 1  2015  1266           1266
#> 2  2016  1293           1293
#> 3  2017  1319           1319
#> 4  2018  1323           1323
#> 5  2019  1356           1356
#> 6  2020  1363           1363

```

```

p_firms_by_year <- firms_by_year |>
  ggplot(aes(x = year, y = firms, colour = "Firms")) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = firms_by_year$year) +
  guides(colour = "none") +
  labs(

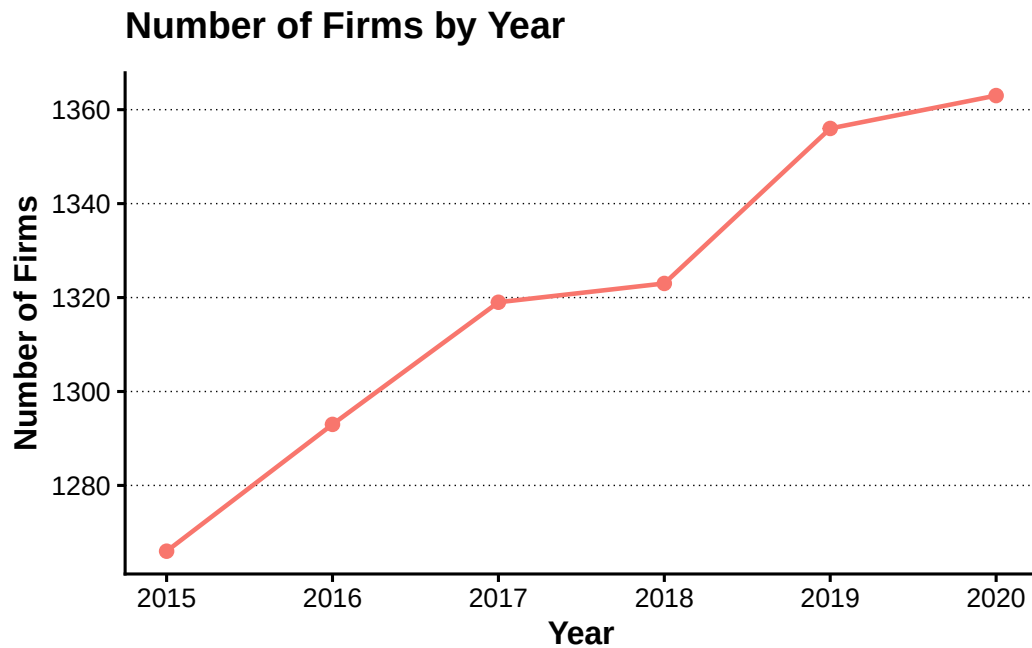
```

```

x = "Year",
y = "Number of Firms",
title = "Number of Firms by Year"
)

add_lagrange_theme_LFL(p_firms_by_year)

```



4.3.2 2.3.2

```

latest_year <-
  max(financial_data$year, na.rm = TRUE)

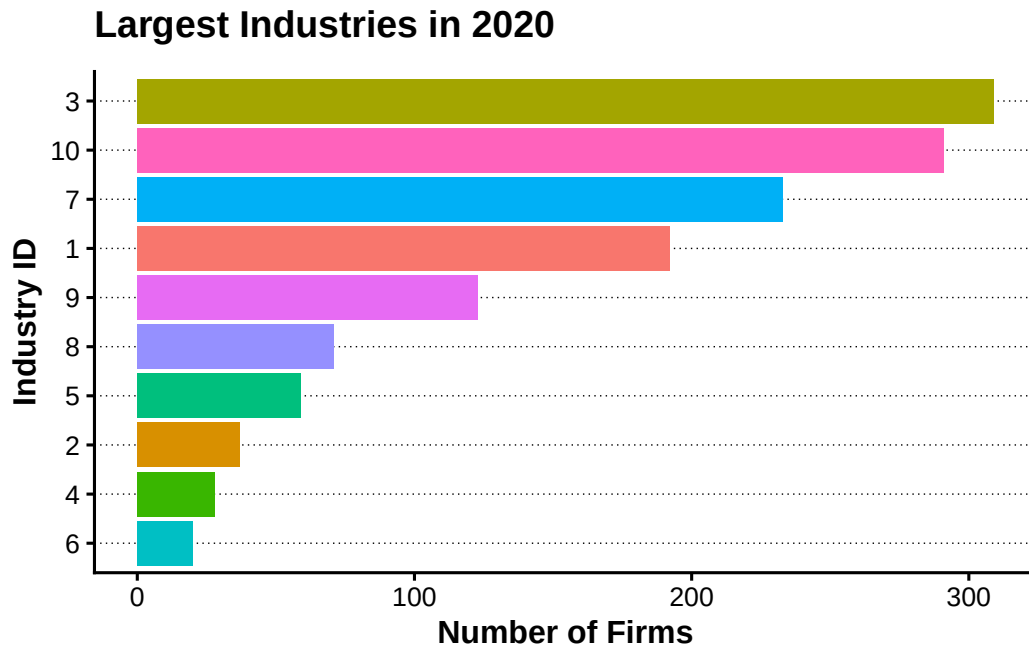
firms_by_industry <- financial_data |>
  filter(year == latest_year) |>
  count(
    industry_ID,
    name = "firms",
    sort = TRUE
  )

```

```
show_table_LFL(firms_by_industry, n = 20)
#> # A tibble: 10 x 2
#>   industry_ID firms
#>   <fct>      <int>
#> 1 3          309
#> 2 10         291
#> 3 7          233
#> 4 1          192
#> 5 9          123
#> 6 8           71
#> 7 5           59
#> 8 2           37
#> 9 4           28
#> 10 6          20
```

```
p_firms_by_industry <- firms_by_industry |>
  slice_head(n = 15) |>
  ggplot(
    aes(
      x = reorder(industry_ID, firms),
      y = firms,
      fill = industry_ID
    )
  ) +
  geom_col(show.legend = FALSE) +
  coord_flip() +
  labs(
    x = "Industry ID",
    y = "Number of Firms",
    title = paste("Largest Industries in", latest_year)
  )

add_lagrange_theme_LFL(p_firms_by_industry)
```



4.3.3 2.3.3

```
sales_summary <- financial_data |>
  summarise(
    observations = sum(!is.na(sales)),
    minimum = min(sales, na.rm = TRUE),
    first_quartile = quantile(sales, 0.25, na.rm = TRUE),
    median = median(sales, na.rm = TRUE),
    mean = mean(sales, na.rm = TRUE),
    third_quartile = quantile(sales, 0.75, na.rm = TRUE),
    maximum = max(sales, na.rm = TRUE)
  )

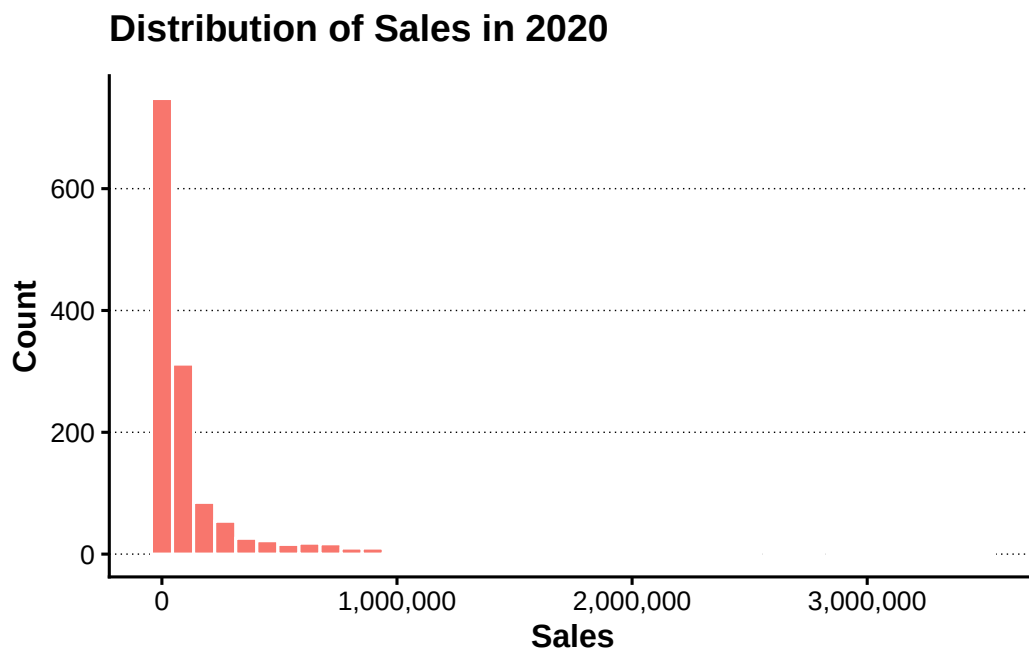
show_table_LFL(sales_summary)
#> # A tibble: 1 x 7
#>   observations minimum first_quartile median    mean third_quartile maximum
#>   <int>      <dbl>      <dbl>  <dbl>  <dbl>      <dbl>      <dbl>
#> 1       7920   205.34     16103. 40431. 166007.    118314. 3496433
```

```

p_sales_distribution <- financial_data |>
  filter(
    year == latest_year,
    !is.na(sales),
    sales >= 0
  ) |>
  ggplot(aes(x = sales, fill = "Sales")) +
  geom_histogram(
    bins = 40,
    colour = "white",
    show.legend = FALSE
  ) +
  scale_x_continuous(labels = scales::label_comma()) +
  labs(
    x = "Sales",
    y = "Count",
    title = paste("Distribution of Sales in", latest_year)
  )

add_lagrange_theme_LFL(p_sales_distribution)

```

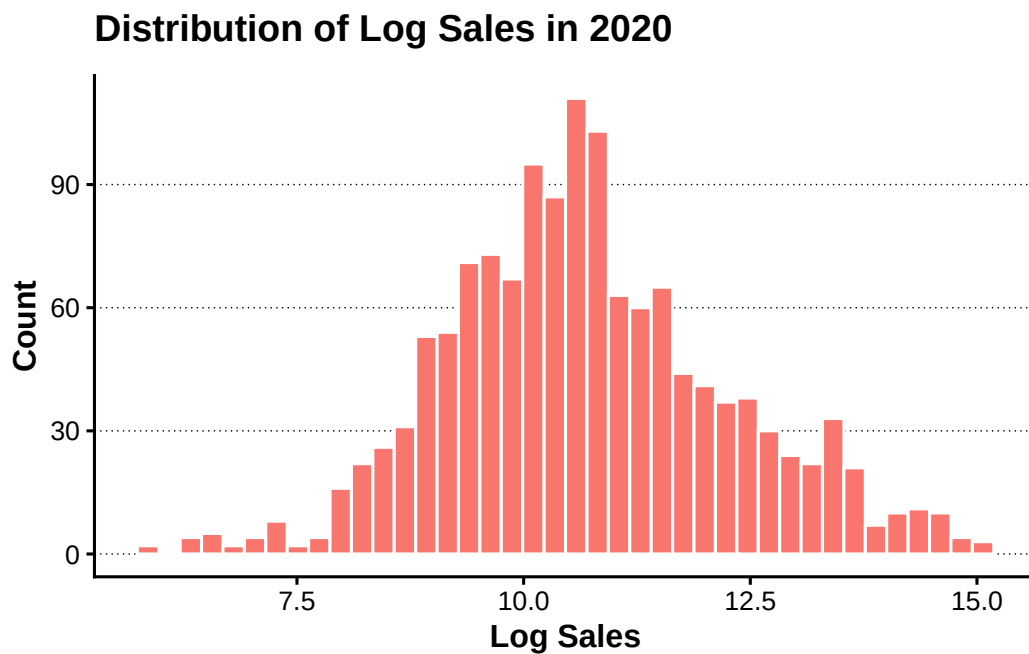


```

p_log_sales_distribution <- financial_data |>
  filter(
    year == latest_year,
    !is.na(sales),
    sales > 0
  ) |>
  ggplot(aes(x = log(sales), fill = "Log Sales")) +
  geom_histogram(
    bins = 40,
    colour = "white",
    show.legend = FALSE
  ) +
  labs(
    x = "Log Sales",
    y = "Count",
    title = paste("Distribution of Log Sales in", latest_year)
  )

add_lagrange_theme_LFL(p_log_sales_distribution)

```



4.4 2.4 ROE

4.4.1 2.4.1 ROE

$$BE_t = (OA_t - OL_t) - (FO_t - FA_t)$$

ROE

$$ROE_t = \frac{X_t}{BE_{t-1}}$$

```
divide_safely_LFL <- function(numerator, denominator) {  
  result <- numerator / denominator  
  replace(result, !is.finite(result), NA_real_)  
}
```

```
financial_data <- financial_data |>  
  mutate(  
    NOA = OA - OL,  
    NFO = FO - FA,  
    BE = NOA - NFO  
  ) |>  
  group_by(firm_ID) |>  
  arrange(year, .by_group = TRUE) |>  
  mutate(  
    lagged_BE = lag(BE),  
    ROE = divide_safely_LFL(X, lagged_BE)  
  ) |>  
  ungroup()
```

```
financial_data |>  
  select(  
    firm_ID,  
    industry_ID,  
    year,  
    X,  
    BE,  
    lagged_BE,
```

```

      ROE
    ) |>
    slice_head(n = 10) |>
    show_table_LFL(n = 10)
#> # A tibble: 10 x 7
#>   firm_ID industry_ID year      X      BE lagged_BE      ROE
#>   <fct>    <fct>      <int> <dbl>    <dbl>    <dbl>    <dbl>
#> 1 1      1      2015 286.64  9695.3      NA      NA
#> 2 1      1      2016 513.48 10014.    9695.3  0.052962
#> 3 1      1      2017 661.64 10426.    10014.  0.066073
#> 4 1      1      2018 664.8  10842.    10426.  0.063762
#> 5 1      1      2019 660.48 11075.    10842.  0.060919
#> 6 1      1      2020 843.83 11594.    11075.  0.076193
#> 7 2      1      2015  40.07  1054.9      NA      NA
#> 8 2      1      2016  49.37  1081.6    1054.9  0.046800
#> 9 2      1      2017  75.9   1134.8    1081.6  0.070174
#> 10 2     1      2018  86.58  1183.7    1134.8  0.076295

```

ROE

```

roe_summary <- financial_data |>
  summarise(
    observations = sum(!is.na(ROE)),
    mean = mean(ROE, na.rm = TRUE),
    median = median(ROE, na.rm = TRUE),
    standard_deviation = sd(ROE, na.rm = TRUE),
    first_percentile = quantile(ROE, 0.01, na.rm = TRUE),
    ninety_ninth_percentile = quantile(ROE, 0.99, na.rm = TRUE)
  )

show_table_LFL(roe_summary)
#> # A tibble: 1 x 6
#>   observations      mean      median standard_deviation first_percentile
#>   <int>      <dbl>      <dbl>          <dbl>          <dbl>
#> 1      6405 0.077713 0.072828          0.066274        -0.079541
#>   ninety_ninth_percentile
#>   <dbl>
#> 1          0.26731

```

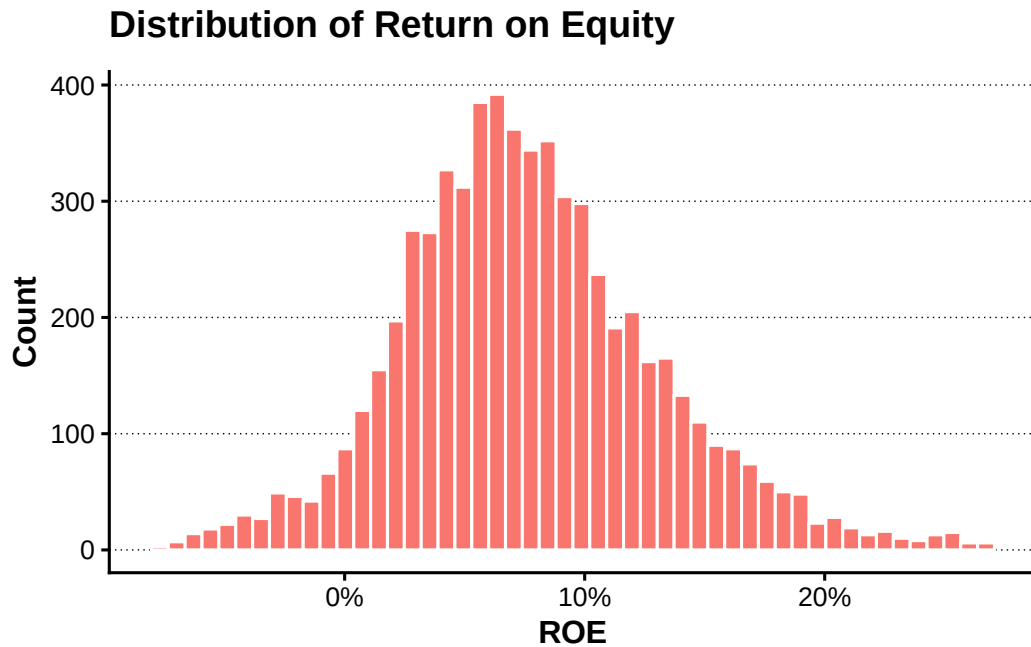
```

roe_limits <- quantile(
  financial_data$ROE,
  probs = c(0.01, 0.99),
  na.rm = TRUE
)

p_roe_distribution <- financial_data |>
  filter(
    !is.na(ROE),
    between(ROE, roe_limits[[1]], roe_limits[[2]])
  ) |>
  ggplot(aes(x = ROE, fill = "ROE")) +
  geom_histogram(
    bins = 50,
    colour = "white",
    show.legend = FALSE
  ) +
  scale_x_continuous(labels = scales::label_percent()) +
  labs(
    x = "ROE",
    y = "Count",
    title = "Distribution of Return on Equity"
  )

add_lagrange_theme_LFL(p_roe_distribution)

```



4.4.2 2.4.2 ROE

```
industry_roe_summary <- financial_data |>
  group_by(industry_ID) |>
  summarise(
    firms = n_distinct(firm_ID),
    observations = sum(!is.na(ROE)),
    mean_ROE = mean(ROE, na.rm = TRUE),
    median_ROE = median(ROE, na.rm = TRUE),
    .groups = "drop"
  ) |>
  filter(observations > 0) |>
  arrange(desc(mean_ROE))

show_table_LFL(industry_roe_summary, n = 20)
#> # A tibble: 10 x 5
#>   industry_ID firms observations mean_ROE median_ROE
#>   <fct>      <int>      <int>    <dbl>    <dbl>
#> 1 2          40        186 0.10762  0.10283
#> 2 6          34        127 0.095142 0.091981
#> 3 8          80        349 0.093711 0.084671
```

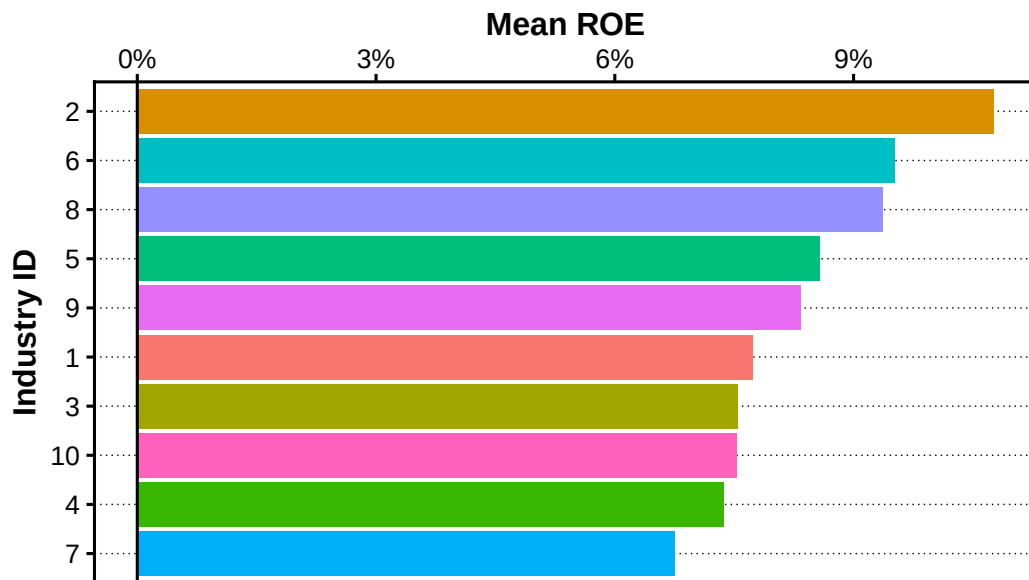
#>	4	5	67	259	0.085703	0.084851
#>	5	9	136	531	0.083403	0.079794
#>	6	1	225	918	0.077285	0.075118
#>	7	3	327	1433	0.075489	0.070116
#>	8	10	314	1388	0.075273	0.070490
#>	9	4	34	138	0.073719	0.072702
#>	10	7	258	1076	0.067555	0.060409

ROE 50

```
p_industry_roe <- industry_roe_summary |>
  filter(observations >= 50) |>
  slice_max(
    order_by = mean_ROE,
    n = 15,
    with_ties = FALSE
  ) |>
  ggplot(
    aes(
      x = reorder(industry_ID, mean_ROE),
      y = mean_ROE,
      fill = industry_ID
    )
  ) +
  geom_col(show.legend = FALSE) +
  coord_flip() +
  geom_hline(yintercept = 0) +
  labs(
    x = "Industry ID",
    y = "Mean ROE",
    title = "Industries with High Mean ROE"
  )

add_lagrange_theme_LFL(p_industry_roe) +
  scale_y_continuous(
    position = "right",
    labels = scales::label_percent()
  )
```

Industries with High Mean ROE



4.4.3 2.4.3

ROE dense_rank()

```
latest_roe_ranking <- financial_data |>
  filter(
    year == latest_year,
    !is.na(ROE)
  ) |>
  group_by(industry_ID) |>
  mutate(
    ROE_rank = dense_rank(desc(ROE))
  ) |>
  ungroup() |>
  arrange(industry_ID, ROE_rank)

latest_roe_ranking |>
  select(
    industry_ID,
    ROE_rank,
    firm_ID,
    ROE
  )
```

```

) |>
slice_head(n = 20) |>
show_table_LFL(n = 20)
#> # A tibble: 20 x 4
#>   industry_ID ROE_rank firm_ID    ROE
#>   <fct>      <int> <fct>    <dbl>
#> 1 1          1      8      0.38826
#> 2 1          2     90      0.33395
#> 3 1          3    225      0.30061
#> 4 1          4     81      0.23113
#> 5 1          5    171      0.21847
#> 6 1          6    138      0.20680
#> 7 1          7     66      0.20335
#> 8 1          8    153      0.19822
#> 9 1          9    181      0.19080
#> 10 1         10     65      0.18881
#> 11 1         11     92      0.17455
#> 12 1         12    195      0.15685
#> 13 1         13     13      0.15474
#> 14 1         14     23      0.15316
#> 15 1         15    148      0.14986
#> 16 1         16    214      0.14639
#> 17 1         17    115      0.14620
#> 18 1         18    201      0.14286
#> 19 1         19     50      0.14212
#> 20 1         20    143      0.14211

```

ROE

```

top_firm_by_industry <- latest_roe_ranking |>
  group_by(industry_ID) |>
  slice_max(
    order_by = ROE,
    n = 1,
    with_ties = FALSE
  ) |>
  ungroup() |>
  select(
    industry_ID,
    firm_ID,
    ROE
  )

```

```
show_table_LFL(top_firm_by_industry, n = 20)
#> # A tibble: 10 x 3
#>   industry_ID firm_ID    ROE
#>   <fct>      <fct>    <dbl>
#> 1 1      8      0.38826
#> 2 2     242     0.37500
#> 3 3     475     0.49754
#> 4 4     619     0.14913
#> 5 5     661     0.26731
#> 6 6     719     0.14220
#> 7 7     929     0.56418
#> 8 8    1042     0.25600
#> 9 9    1167     0.23462
#> 10 10   1380     0.24979
```

4.5 2.5

4.5.1 2.5.1 ROE

ROE

$$ROE_t = RNOA_t + FLEV_{t-1}(RNOA_t - NBC_t)$$

$$RNOA_t = \frac{OX_t}{NOA_{t-1}}$$

$$PM_t = \frac{OX_t}{Sales_t}$$

$$ATO_t = \frac{Sales_t}{NOA_{t-1}}$$

$$FLEV_{t-1} = \frac{NFO_{t-1}}{BE_{t-1}}$$

$$NBC_t = \frac{NFE_t}{NFO_{t-1}}$$


```

financial_data_dupont <- financial_data |>
  group_by(firm_ID) |>
  arrange(year, .by_group = TRUE) |>
  mutate(
    lagged_NOA = lag(NOA),
    lagged_NFO = lag(NFO),
    RNOA = divide_safely_LFL(OX, lagged_NOA),
    PM = divide_safely_LFL(OX, sales),
    ATO = divide_safely_LFL(sales, lagged_NOA),
    lagged_FLEV = divide_safely_LFL(lagged_NFO, lagged_BE),
    NBC = divide_safely_LFL(NFE, lagged_NFO),
    ROE_DuPont = RNOA + lagged_FLEV * (RNOA - NBC),
    DuPont_gap = ROE - ROE_DuPont
  ) |>
  ungroup()

financial_data_dupont |>
  select(
    firm_ID,
    year,
    ROE,
    RNOA,
    PM,
    ATO,
    lagged_FLEV,
    NBC,
    ROE_DuPont,
    DuPont_gap
  ) |>
  slice_head(n = 10) |>
  show_table_LFL(n = 10)
#> # A tibble: 10 x 10
#>   firm_ID year      ROE      RNOA      PM      ATO lagged_FLEV      NBC
#>   <fct>   <int>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1 1      2015 NA      NA      0.083151 NA      NA      NA
#> 2 1      2016 0.052962 0.065350 0.094830 0.68913 -0.10961 -0.047678
#> 3 1      2017 0.066073 0.074071 0.10625  0.69712 -0.068152 -0.043289
#> 4 1      2018 0.063762 0.084974 0.10974  0.77436 -0.15202 -0.054567
#> 5 1      2019 0.060919 0.070062 0.12658  0.55348  0.26187  0.10498
#> 6 1      2020 0.076193 0.061279 0.10326  0.59342  0.14692 -0.040231
#> 7 2      2015 NA      NA      0.013070 NA      NA      NA
#> 8 2      2016 0.046800 0.045056 0.014679 3.0693  0.078263 0.022729

```

```

#> 9 2      2017 0.070174 0.058935 0.021145 2.7872      0.30884 0.022536
#> 10 2     2018 0.076295 0.064999 0.022564 2.8807      0.26625 0.022561
#>   ROE_DuPont DuPont_gap
#>   <dbl>      <dbl>
#> 1  NA      NA
#> 2  0.052961 7.7332e-7
#> 3  0.066072 3.1527e-7
#> 4  0.063762 -3.3567e-7
#> 5  0.060919 -2.0862e-8
#> 6  0.076193 1.1089e-7
#> 7  NA      NA
#> 8  0.046803 -3.2789e-6
#> 9  0.070176 -1.9587e-6
#> 10 0.076298 -2.9722e-6

```

ROE ROE

```

dupont_validation <- financial_data_dupont |>
  summarise(
    comparable_observations = sum(!is.na(DuPont_gap)),
    mean_absolute_gap = mean(abs(DuPont_gap), na.rm = TRUE),
    maximum_absolute_gap = max(abs(DuPont_gap), na.rm = TRUE)
  )

show_table_LFL(dupont_validation)
#> # A tibble: 1 x 3
#>   comparable_observations mean_absolute_gap maximum_absolute_gap
#>   <int>                <dbl>                <dbl>
#> 1      6405          0.00000036675          0.000020751

```

4.5.2 2.5.2 PM ATO

PM ATO

```

industry_pm_ato <- financial_data_dupont |>
  group_by(industry_ID) |>
  summarise(
    observations = sum(complete.cases(PM, ATO)),
    median_PM = median(PM, na.rm = TRUE),
    median_ATO = median(ATO, na.rm = TRUE),
    median_RNOA = median(RNOA, na.rm = TRUE),

```

```

    .groups = "drop"
  ) |>
  filter(
    observations >= 50,
    is.finite(median_PM),
    is.finite(median_ATO)
  )

show_table_LFL(industry_pm_ato, n = 20)
#> # A tibble: 10 x 5
#>   industry_ID observations median_PM median_ATO median_RNOA
#>   <fct>          <int>      <dbl>      <dbl>      <dbl>
#> 1 1          918  0.025331    3.9378    0.083905
#> 2 2          186  0.037769    2.4106    0.10883
#> 3 3         1433  0.048140    1.8342    0.089713
#> 4 4          138  0.040086    1.7797    0.075919
#> 5 5          259  0.034620    2.5099    0.097564
#> 6 6          127  0.048194    1.2994    0.074109
#> 7 7         1076  0.031837    2.5163    0.080634
#> 8 8          349  0.074662    1.0662    0.063506
#> 9 9          531  0.042643    2.1790    0.098533
#> 10 10        1388  0.044615    1.9604    0.090718

```

```

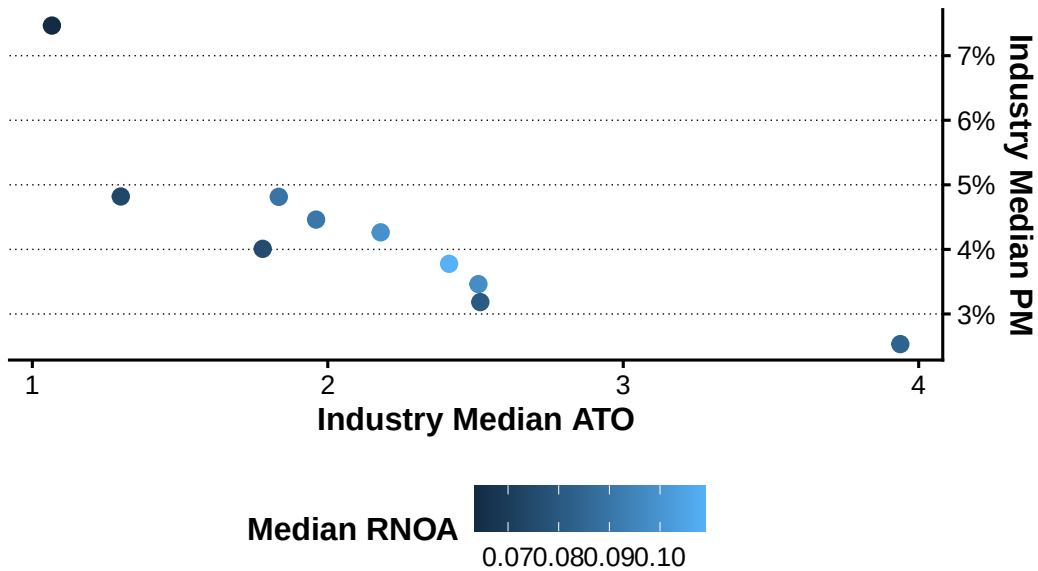
p_pm_ato_tradeoff <- industry_pm_ato |>
  ggplot(
    aes(
      x = median_ATO,
      y = median_PM,
      colour = median_RNOA
    )
  ) +
  geom_point(size = 2.5) +
  labs(
    x = "Industry Median ATO",
    y = "Industry Median PM",
    colour = "Median RNOA",
    title = "Profit Margin and Asset Turnover by Industry"
  )

add_lagrange_theme_LFL(p_pm_ato_tradeoff) +
  scale_y_continuous(
    position = "right",

```

```
labels = scales::label_percent()
)
```

Profit Margin and Asset Turnover by Industry



PM

```
major_industries <- firms_by_industry |>
  slice_head(n = 8) |>
  pull(industry_ID)

pm_limits <- quantile(
  financial_data_dupont$PM,
  probs = c(0.01, 0.99),
  na.rm = TRUE
)

p_pm_by_industry <- financial_data_dupont |>
  filter(
    year == latest_year,
    industry_ID %in% major_industries,
    !is.na(PM)
  ) |>
  ggplot(
    aes(
```

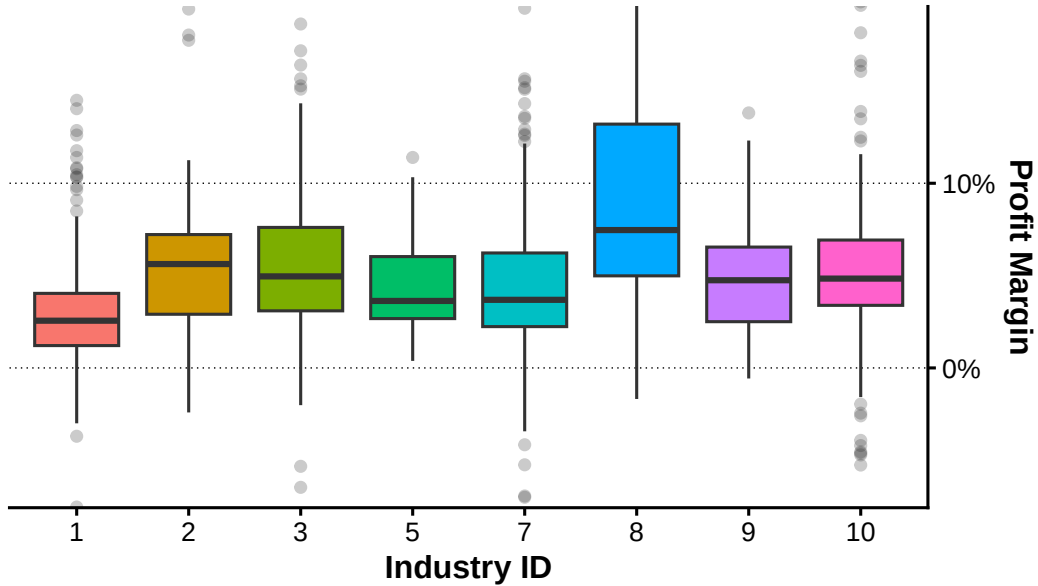
```

    x = industry_ID,
    y = PM,
    fill = industry_ID
  )
) +
geom_boxplot(
  outlier.alpha = 0.25,
  show.legend = FALSE
) +
coord_cartesian(ylim = pm_limits) +
labs(
  x = "Industry ID",
  y = "Profit Margin",
  title = paste(
    "Profit Margin by Industry in",
    latest_year
  )
)

add_lagrange_theme_LFL(p_pm_by_industry) +
scale_y_continuous(
  position = "right",
  labels = scales::label_percent()
)

```

Profit Margin by Industry in 2020



4.6 2.6

4.6.1 2.6.1

```
purrr::map_dfr()
```

```
financial_metrics <- c(  
  "sales",  
  "BE",  
  "ROE",  
  "RNOA",  
  "PM",  
  "ATO"  
)  
  
metric_summary <- financial_metrics |>  
  map_dfr(  
    \(metric_name) {  
      values <- financial_data_dupont[[metric_name]]  
      finite_values <- values[is.finite(values)]
```

```

    tibble(
      metric = metric_name,
      observations = length(finite_values),
      mean = mean(finite_values),
      median = median(finite_values),
      standard_deviation = sd(finite_values),
      minimum = min(finite_values),
      maximum = max(finite_values)
    )
  }
)

show_table_LFL(metric_summary, n = nrow(metric_summary))
#> # A tibble: 6 x 7
#>   metric observations      mean      median standard_deviation  minimum
#>   <chr>          <int>    <dbl>      <dbl>          <dbl>      <dbl>
#> 1 sales           7920 166007.    40431.          381980.    205.34
#> 2 BE              7920 111517.    26882.          420624.    136.93
#> 3 ROE             6405   0.077713  0.072828        0.066274  -0.70192
#> 4 RNOA            6405   0.11141  0.085868        0.12622  -0.82036
#> 5 PM              7920   0.045534  0.040555        0.042223  -0.20079
#> 6 ATO             6405   2.7051    2.1024          1.8684    0.25825
#>   maximum
#>   <dbl>
#> 1 3.4964e+6
#> 2 2.5448e+7
#> 3 1.4055e+0
#> 4 1.8283e+0
#> 5 3.8289e-1
#> 6 9.8170e+0

```

4.6.2 2.6.2

3 ROE data/derived/fundamentals/financial_characteristic

```

financial_output_path <-
  write_derived_parquet_file_LFL(
    financial_data_dupont,
    file.path(
      "fundamentals",
      "financial_characteristics.parquet"
    )
  )

```

```

    )
  )

financial_output_path
#> [1] "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/fundamentals/financial_characteristics.parquet"
file.info(financial_output_path)[, c("size", "mtime")]
#>
#> C:/AnalyticFin/Projects/LagrangeFinance/data/derived/fundamentals/financial_characteristics.parquet
#>
#> C:/AnalyticFin/Projects/LagrangeFinance/data/derived/fundamentals/financial_characteristics.parquet

```

Parquet

```

financial_output_check <-
  read_derived_parquet_file_LFL(
    file.path(
      "fundamentals",
      "financial_characteristics.parquet"
    )
  )

stopifnot(
  nrow(financial_output_check) ==
    nrow(financial_data_dupont)
)

stopifnot(
  identical(
    names(financial_output_check),
    names(financial_data_dupont)
  )
)

output_summary <- tibble(
  output_file = financial_output_path,
  rows = nrow(financial_output_check),
  columns = ncol(financial_output_check)
)

show_table_LFL(output_summary)
#> # A tibble: 1 x 3
#>   output_file

```



```

#>   <chr>
#> 1 C:/AnalyticFin/Projects/LagrangeFinance/data/derived/fundamentals/financial_c~
#>   rows columns
#>   <int>   <int>
#> 1  7920     25

```

4.7 2.7

	ROE	ROE RNOA	3	data/derived/fun
3				

5 I-3

2

4

5.1 3.1

5.2 3.2

5.2.1 3.2.1

```
stock_data_raw <-  
read_parquet_file_LFL(  
  file.path(  
    "raw",  
    "market",  
    "stock_prices_and_returns.parquet"  
  )  
)  
  
show_table_LFL(stock_data_raw, n = 10)  
#> # A tibble: 95,040 x 9  
#>   year month month_ID firm_ID stock_price  DPS shares_outstanding  
#> * <dbl> <dbl>   <dbl>   <dbl>   <dbl> <dbl>           <dbl>  
#> 1  2015     1         1       1       954     0       2422000  
#> 2  2015     2         2       1       960     0       2422000  
#> 3  2015     3         3       1      1113     0       2422000  
#> 4  2015     4         4       1      1081     0       2422000  
#> 5  2015     5         5       1      1317     0       2422000  
#> 6  2015     6         6       1      1366    29       2422000  
#> 7  2015     7         7       1      1353     0       2422000  
#> 8  2015     8         8       1      1209     0       2422000  
#> 9  2015     9         9       1      1291     0       2422000  
#> 10 2015    10        10       1      1407     0       2422000  
#>   adjustment_coefficient      R_F
```

```

#> *                <dbl>      <dbl>
#> 1                1 0.00065068
#> 2                1 0.00058341
#> 3                1 0.00061144
#> 4                1 0.00068482
#> 5                1 0.00073677
#> 6                1 0.00069540
#> 7                1 0.00049354
#> 8                1 0.00047106
#> 9                1 0.00055242
#> 10               1 0.00063803
#> # i 95,030 more rows

```

```

required_stock_columns <- c(
  "year",
  "month",
  "month_ID",
  "firm_ID",
  "stock_price",
  "shares_outstanding",
  "DPS",
  "adjustment_coefficient",
  "R_F"
)

missing_stock_columns <-
  setdiff(required_stock_columns, names(stock_data_raw))

if (length(missing_stock_columns) > 0) {
  stop(
    "          : ",
    paste(missing_stock_columns, collapse = ", "),
    call. = FALSE
  )
}

```

5.2.2 3.2.2

```
show_data_structure_LFL(stock_data_raw)
#> Rows: 95,040
#> Columns: 9
#> $ year          <dbl> 2015, 2015, 2015, 2015, 2015, 2015, 2015, 2015, ~
#> $ month         <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 1, 2, 3, ~
#> $ month_ID      <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, ~
#> $ firm_ID       <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ stock_price   <dbl> 954, 960, 1113, 1081, 1317, 1366, 1353, 1209, 1~
#> $ DPS           <dbl> 0, 0, 0, 0, 0, 29, 0, 0, 0, 0, 0, 29, 0, 0, 0, ~
#> $ shares_outstanding <dbl> 2422000, 2422000, 2422000, 2422000, 2422000, 24~
#> $ adjustment_coefficient <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ R_F           <dbl> 0.00065068255, 0.00058340985, 0.00061144232, 0.~

stock_data_overview <- stock_data_raw |>
  summarise(
    observations = n(),
    variables = ncol(stock_data_raw),
    firms = n_distinct(firm_ID),
    first_year = min(year, na.rm = TRUE),
    last_year = max(year, na.rm = TRUE),
    first_month_ID = min(month_ID, na.rm = TRUE),
    last_month_ID = max(month_ID, na.rm = TRUE)
  )

show_table_LFL(stock_data_overview)
#> # A tibble: 1 x 7
#>   observations variables firms first_year last_year first_month_ID last_month_ID
#>   <int>         <int> <int>    <dbl>    <dbl>         <dbl>         <dbl>
#> 1     95040           9  1515    2015    2020             1             72

stock_missing_summary <- stock_data_raw |>
  summarise(
    across(
      everything(),
      list(
        missing = \(x) sum(is.na(x)),
        missing_rate = \(x) mean(is.na(x))
      ),
      .names = "{.col}__{.fn}"
    )
  )
```

```

) |>
pivot_longer(
  cols = everything(),
  names_to = c("variable", ".value"),
  names_sep = "__"
) |>
arrange(desc(missing_rate), variable)

show_table_LFL(
  stock_missing_summary,
  n = nrow(stock_missing_summary)
)
#> # A tibble: 9 x 3
#>   variable      missing missing_rate
#>   <chr>         <int>         <dbl>
#> 1 DPS              0             0
#> 2 R_F              0             0
#> 3 adjustment_coefficient 0             0
#> 4 firm_ID          0             0
#> 5 month            0             0
#> 6 month_ID         0             0
#> 7 shares_outstanding 0             0
#> 8 stock_price      0             0
#> 9 year            0             0

```

5.2.3 3.2.3

ID ID

```

stock_data <- stock_data_raw |>
mutate(
  firm_ID = as.integer(firm_ID),
  year = as.integer(year),
  month = as.integer(month),
  month_ID = as.integer(month_ID),
  month_date = lubridate::make_date(year, month, 1)
) |>
arrange(firm_ID, month_ID)

show_data_structure_LFL(stock_data)
#> Rows: 95,040

```

```
#> Columns: 10
#> $ year          <int> 2015, 2015, 2015, 2015, 2015, 2015, 2015, 2015, ~
#> $ month         <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 1, 2, 3, ~
#> $ month_ID      <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, ~
#> $ firm_ID       <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ stock_price   <dbl> 954, 960, 1113, 1081, 1317, 1366, 1353, 1209, 1~
#> $ DPS           <dbl> 0, 0, 0, 0, 0, 29, 0, 0, 0, 0, 0, 29, 0, 0, 0, ~
#> $ shares_outstanding <dbl> 2422000, 2422000, 2422000, 2422000, 2422000, 24~
#> $ adjustment_coefficient <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ R_F           <dbl> 0.00065068255, 0.00058340985, 0.00061144232, 0.~
#> $ month_date    <date> 2015-01-01, 2015-02-01, 2015-03-01, 2015-04-01~
```

firm_ID month_ID

```
stock_key_duplicates <- stock_data |>
  count(
    firm_ID,
    month_ID,
    name = "observations"
  ) |>
  filter(observations > 1)

show_table_LFL(stock_key_duplicates)
#> # A tibble: 0 x 3
#> # i 3 variables: firm_ID <int>, month_ID <int>, observations <int>
```

```
if (nrow(stock_key_duplicates) > 0) {
  stop(
    "firm_ID month_ID      ",
    call. = FALSE
  )
}
```

ID

```
month_calendar_check <- stock_data |>
  distinct(
    month_ID,
    year,
    month,
    month_date
```

```

) |>
count(
  month_ID,
  name = "calendar_rows"
) |>
filter(calendar_rows > 1)

show_table_LFL(month_calendar_check)
#> # A tibble: 0 x 2
#> # i 2 variables: month_ID <int>, calendar_rows <int>

if (nrow(month_calendar_check) > 0) {
  stop(
    " month_ID          ",
    call. = FALSE
  )
}

```

```

stock_observations_by_year <- stock_data |>
group_by(year) |>
summarise(
  firms = n_distinct(firm_ID),
  observations = n(),
  first_month_ID = min(month_ID),
  last_month_ID = max(month_ID),
  .groups = "drop"
) |>
arrange(year)

show_table_LFL(
  stock_observations_by_year,
  n = nrow(stock_observations_by_year)
)
#> # A tibble: 6 x 5
#>   year firms observations first_month_ID last_month_ID
#>   <int> <int>         <int>         <int>         <int>
#> 1  2015  1266         15192             1             12
#> 2  2016  1293         15516            13             24
#> 3  2017  1319         15828            25             36
#> 4  2018  1323         15876            37             48

```

#>	5	2019	1356	16272	49	60
#>	6	2020	1363	16356	61	72

5.3 3.3

5.3.1 3.3.1

1

```
dividend_observations <- stock_data |>
  filter(
    !is.na(DPS),
    DPS != 0
  ) |>
  select(
    firm_ID,
    year,
    month,
    month_ID,
    stock_price,
    DPS,
    adjustment_coefficient
  ) |>
  arrange(firm_ID, month_ID)

adjustment_observations <- stock_data |>
  filter(
    !is.na(adjustment_coefficient),
    !near(adjustment_coefficient, 1)
  ) |>
  select(
    firm_ID,
    year,
    month,
    month_ID,
    stock_price,
    DPS,
    adjustment_coefficient
  ) |>
  arrange(firm_ID, month_ID)
```



```

corporate_action_summary <- stock_data |>
  summarise(
    observations = n(),
    dividend_observations =
      sum(!is.na(DPS) & DPS != 0),
    adjusted_observations =
      sum(
        !is.na(adjustment_coefficient) &
        !near(adjustment_coefficient, 1)
      )
  )

show_table_LFL(dividend_observations, n = 20)
#> # A tibble: 14,250 x 7
#>   firm_ID year month month_ID stock_price  DPS adjustment_coefficient
#>   <int> <int> <int>   <int>   <dbl> <dbl>             <dbl>
#> 1     1     1  2015     6       6    1366     29             1
#> 2     1     1  2015    12      12    1477     29             1
#> 3     1     1  2016     6      18    1696     40             1
#> 4     1     1  2016    12      24    2842     40             1
#> 5     1     1  2017     6      30    4317     43             1
#> 6     1     1  2017    12      36    3915     43             1
#> 7     1     1  2018     6      42    2986     43             1
#> 8     1     1  2018    12      48    2992     43             1
#> 9     1     1  2019     6      54    3608     53             1
#> 10    1     1  2019    12      60    4803     53             1
#> 11    1     1  2020     6      66    4698     56             1
#> 12    1     1  2020    12      72    3341     56             1
#> 13    2     2  2015     6       6    2167      3             1
#> 14    2     2  2015    12      12    2613      3             1
#> 15    2     2  2016     6      18    2395      7             1
#> 16    2     2  2016    12      24    3576      7             1
#> 17    2     2  2017     6      30    4966      7             1
#> 18    2     2  2017    12      36    5852      7             1
#> 19    2     2  2018     6      42    4415     12             1
#> 20    2     2  2018    12      48    4542     12             1
#> # i 14,230 more rows
show_table_LFL(adjustment_observations, n = 20)
#> # A tibble: 267 x 7
#>   firm_ID year month month_ID stock_price  DPS adjustment_coefficient
#>   <int> <int> <int>   <int>   <dbl> <dbl>             <dbl>
#> 1     1     1  2017     4      28    4082      0             1.2

```

```

#> 2      9 2015      4      4      2655      0      1.2
#> 3     10 2015      2      2       757      0      1.2
#> 4     21 2019     12     60      3008     19      1.1
#> 5     27 2019      9     57     13750      0      0.1
#> 6     42 2016      5     17     47324      0      0.1
#> 7     48 2019      2     50     13232      0      0.1
#> 8     53 2019      4     52      1754      0      1.1
#> 9     54 2019      2     50      4015      0      1.1
#> 10    60 2019     12     60      1749      0      1.1
#> 11    66 2019     11     59     10140      0      0.1
#> 12    67 2018      1     37      2020      0      1.1
#> 13    72 2016      5     17      1876      0      1.1
#> 14    74 2017      6     30      1402     11      2
#> 15    80 2018      3     39      1393      0      1.2
#> 16    84 2017      4     28     10697      0      0.1
#> 17    90 2020      2     62    153396      0      0.1
#> 18    96 2015      4      4       805      0      1.2
#> 19   103 2020      6     66     44488     468      0.1
#> 20   104 2016      1     13      9191      0      0.1
#> # i 247 more rows
show_table_LFL(corporate_action_summary)
#> # A tibble: 1 x 3
#>   observations dividend_observations adjusted_observations
#>   <int>          <int>          <int>
#> 1     95040        14250          267

```

5.3.2 3.3.2

$$i \quad t \quad ME_{i,t} \quad P_{i,t} \quad N_{i,t}$$

$$ME_{i,t} = P_{i,t} N_{i,t}$$

```

stock_data <- stock_data |>
  mutate(
    ME = stock_price * shares_outstanding,
    ME = if_else(
      is.finite(ME) & ME > 0,
      ME,
      NA_real_
    )
  )

```

```

stock_data |>
  select(
    firm_ID,
    month_date,
    stock_price,
    shares_outstanding,
    ME
  ) |>
  slice_head(n = 10) |>
  show_table_LFL(n = 10)
#> # A tibble: 10 x 5
#>   firm_ID month_date stock_price shares_outstanding      ME
#>   <int> <date>      <dbl>          <dbl>      <dbl>
#> 1     1  2015-01-01      954          2422000 2310588000
#> 2     1  2015-02-01      960          2422000 2325120000
#> 3     1  2015-03-01     1113          2422000 2695686000
#> 4     1  2015-04-01     1081          2422000 2618182000
#> 5     1  2015-05-01     1317          2422000 3189774000
#> 6     1  2015-06-01     1366          2422000 3308452000
#> 7     1  2015-07-01     1353          2422000 3276966000
#> 8     1  2015-08-01     1209          2422000 2928198000
#> 9     1  2015-09-01     1291          2422000 3126802000
#> 10    1  2015-10-01     1407          2422000 3407754000

```

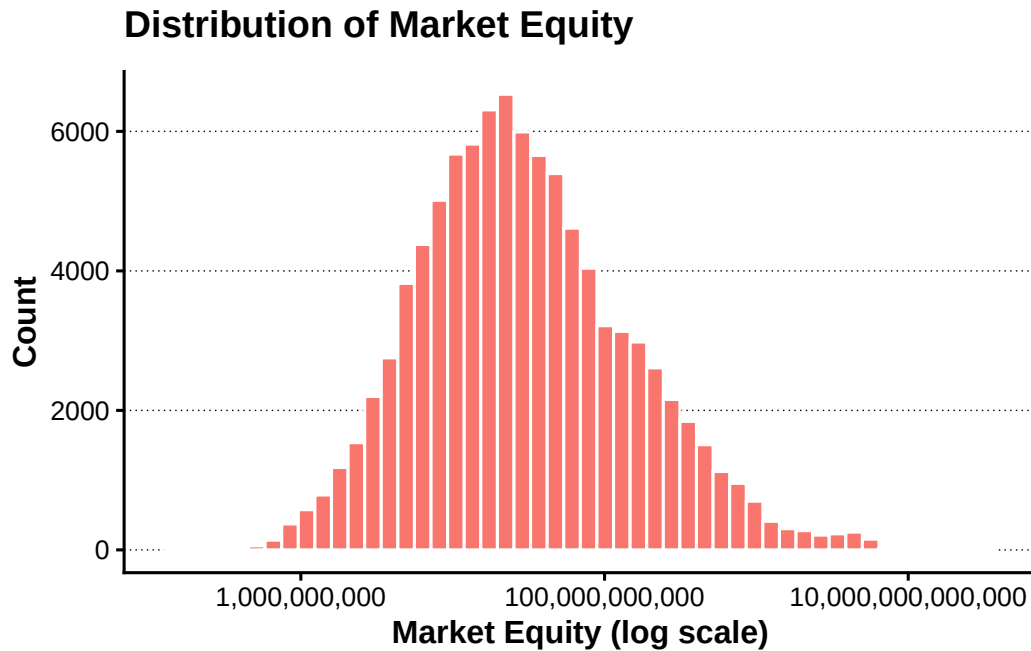
```

p_market_equity <- stock_data |>
  filter(
    is.finite(ME),
    ME > 0
  ) |>
  ggplot(aes(x = ME, fill = "Market Equity")) +
  geom_histogram(
    bins = 50,
    colour = "white",
    show.legend = FALSE
  ) +
  scale_x_log10(
    labels = scales::label_comma()
  ) +
  labs(
    x = "Market Equity (log scale)",
    y = "Count",
    title = "Distribution of Market Equity"
  )

```

)

```
add_lagrange_theme_LFL(p_market_equity)
```



5.3.3 3.3.3

ID

```
stock_data <- stock_data |>
  group_by(firm_ID) |>
  arrange(month_ID, .by_group = TRUE) |>
  mutate(
    lagged_stock_price = lag(stock_price),
    lagged_month_ID = lag(month_ID),
    consecutive_month =
      month_ID - lagged_month_ID == 1L,
    lagged_stock_price = if_else(
      consecutive_month,
      lagged_stock_price,
      NA_real_
    )
  )
```

```
) |>
ungroup()
```

$R_{i,t}$

$$R_{i,t} = \frac{(P_{i,t} + D_{i,t})A_{i,t} - P_{i,t-1}}{P_{i,t-1}}$$

```
stock_data <- stock_data |>
  mutate(
    R = case_when(
      is.na(lagged_stock_price) ~ NA_real_,
      lagged_stock_price <= 0 ~ NA_real_,
      is.na(stock_price) ~ NA_real_,
      is.na(DPS) ~ NA_real_,
      is.na(adjustment_coefficient) ~ NA_real_,
      TRUE ~ (
        (stock_price + DPS) *
          adjustment_coefficient -
          lagged_stock_price
      ) / lagged_stock_price
    ),
    R = if_else(
      is.finite(R),
      R,
      NA_real_
    )
  )

stock_data |>
  select(
    firm_ID,
    month_date,
    lagged_stock_price,
    stock_price,
    DPS,
    adjustment_coefficient,
    R
  ) |>
  filter(!is.na(R)) |>
  slice_head(n = 15) |>
```

```

show_table_LFL(n = 15)
#> # A tibble: 15 x 7
#>   firm_ID month_date lagged_stock_price stock_price  DPS
#>   <int> <date>          <dbl>         <dbl> <dbl>
#> 1     1 2015-02-01          954          960     0
#> 2     1 2015-03-01          960         1113     0
#> 3     1 2015-04-01         1113         1081     0
#> 4     1 2015-05-01         1081         1317     0
#> 5     1 2015-06-01         1317         1366    29
#> 6     1 2015-07-01         1366         1353     0
#> 7     1 2015-08-01         1353         1209     0
#> 8     1 2015-09-01         1209         1291     0
#> 9     1 2015-10-01         1291         1407     0
#> 10    1 2015-11-01         1407         1562     0
#> 11    1 2015-12-01         1562         1477    29
#> 12    1 2016-01-01         1477         1561     0
#> 13    1 2016-02-01         1561         1487     0
#> 14    1 2016-03-01         1487         1526     0
#> 15    1 2016-04-01         1526         1542     0
#>   adjustment_coefficient      R
#>   <dbl>         <dbl>
#> 1           1 0.0062893
#> 2           1 0.15937
#> 3           1 -0.028751
#> 4           1 0.21832
#> 5           1 0.059226
#> 6           1 -0.0095168
#> 7           1 -0.10643
#> 8           1 0.067825
#> 9           1 0.089853
#> 10          1 0.11016
#> 11          1 -0.035851
#> 12          1 0.056872
#> 13          1 -0.047406
#> 14          1 0.026227
#> 15          1 0.010485

```

5.3.4 3.3.4

```

risk_free_rate_by_month <- stock_data |>
  group_by(
    month_ID,
    year,
    month,
    month_date
  ) |>
  summarise(
    observations = n(),
    valid_rates = sum(is.finite(R_F)),
    distinct_rates =
      n_distinct(R_F[is.finite(R_F)]),
    R_F_month = {
      values <- R_F[is.finite(R_F)]

      if (length(values) == 0) {
        NA_real_
      } else {
        median(values)
      }
    },
    .groups = "drop"
  ) |>
  arrange(month_ID)

risk_free_rate_diagnostics <-
  risk_free_rate_by_month |>
  filter(distinct_rates > 1)

show_table_LFL(
  risk_free_rate_diagnostics,
  n = nrow(risk_free_rate_diagnostics)
)

#> # A tibble: 72 x 8
#>   month_ID year month month_date observations valid_rates distinct_rates
#>   <int> <int> <int> <date>          <int>      <int>      <int>
#> 1     1    2015     1 2015-01-01         1266      1266         83
#> 2     2    2015     2 2015-02-01         1266      1266         83
#> 3     3    2015     3 2015-03-01         1266      1266         83
#> 4     4    2015     4 2015-04-01         1266      1266         83
#> 5     5    2015     5 2015-05-01         1266      1266         83
#> 6     6    2015     6 2015-06-01         1266      1266         83

```

#> 7	7	2015	7 2015-07-01	1266	1266	83
#> 8	8	2015	8 2015-08-01	1266	1266	83
#> 9	9	2015	9 2015-09-01	1266	1266	83
#> 10	10	2015	10 2015-10-01	1266	1266	83
#> 11	11	2015	11 2015-11-01	1266	1266	83
#> 12	12	2015	12 2015-12-01	1266	1266	83
#> 13	13	2016	1 2016-01-01	1293	1293	83
#> 14	14	2016	2 2016-02-01	1293	1293	83
#> 15	15	2016	3 2016-03-01	1293	1293	83
#> 16	16	2016	4 2016-04-01	1293	1293	83
#> 17	17	2016	5 2016-05-01	1293	1293	83
#> 18	18	2016	6 2016-06-01	1293	1293	83
#> 19	19	2016	7 2016-07-01	1293	1293	83
#> 20	20	2016	8 2016-08-01	1293	1293	83
#> 21	21	2016	9 2016-09-01	1293	1293	83
#> 22	22	2016	10 2016-10-01	1293	1293	83
#> 23	23	2016	11 2016-11-01	1293	1293	83
#> 24	24	2016	12 2016-12-01	1293	1293	83
#> 25	25	2017	1 2017-01-01	1319	1319	83
#> 26	26	2017	2 2017-02-01	1319	1319	83
#> 27	27	2017	3 2017-03-01	1319	1319	83
#> 28	28	2017	4 2017-04-01	1319	1319	83
#> 29	29	2017	5 2017-05-01	1319	1319	83
#> 30	30	2017	6 2017-06-01	1319	1319	83
#> 31	31	2017	7 2017-07-01	1319	1319	83
#> 32	32	2017	8 2017-08-01	1319	1319	83
#> 33	33	2017	9 2017-09-01	1319	1319	83
#> 34	34	2017	10 2017-10-01	1319	1319	83
#> 35	35	2017	11 2017-11-01	1319	1319	83
#> 36	36	2017	12 2017-12-01	1319	1319	83
#> 37	37	2018	1 2018-01-01	1323	1323	83
#> 38	38	2018	2 2018-02-01	1323	1323	83
#> 39	39	2018	3 2018-03-01	1323	1323	83
#> 40	40	2018	4 2018-04-01	1323	1323	83
#> 41	41	2018	5 2018-05-01	1323	1323	83
#> 42	42	2018	6 2018-06-01	1323	1323	83
#> 43	43	2018	7 2018-07-01	1323	1323	83
#> 44	44	2018	8 2018-08-01	1323	1323	83
#> 45	45	2018	9 2018-09-01	1323	1323	83
#> 46	46	2018	10 2018-10-01	1323	1323	83
#> 47	47	2018	11 2018-11-01	1323	1323	83
#> 48	48	2018	12 2018-12-01	1323	1323	83


```

#> 49      49  2019      1 2019-01-01      1356      1356      83
#> 50      50  2019      2 2019-02-01      1356      1356      83
#> 51      51  2019      3 2019-03-01      1356      1356      83
#> 52      52  2019      4 2019-04-01      1356      1356      83
#> 53      53  2019      5 2019-05-01      1356      1356      83
#> 54      54  2019      6 2019-06-01      1356      1356      83
#> 55      55  2019      7 2019-07-01      1356      1356      83
#> 56      56  2019      8 2019-08-01      1356      1356      83
#> 57      57  2019      9 2019-09-01      1356      1356      83
#> 58      58  2019     10 2019-10-01      1356      1356      83
#> 59      59  2019     11 2019-11-01      1356      1356      83
#> 60      60  2019     12 2019-12-01      1356      1356      83
#> 61      61  2020      1 2020-01-01      1363      1363      83
#> 62      62  2020      2 2020-02-01      1363      1363      83
#> 63      63  2020      3 2020-03-01      1363      1363      83
#> 64      64  2020      4 2020-04-01      1363      1363      83
#> 65      65  2020      5 2020-05-01      1363      1363      83
#> 66      66  2020      6 2020-06-01      1363      1363      83
#> 67      67  2020      7 2020-07-01      1363      1363      83
#> 68      68  2020      8 2020-08-01      1363      1363      83
#> 69      69  2020      9 2020-09-01      1363      1363      83
#> 70      70  2020     10 2020-10-01      1363      1363      83
#> 71      71  2020     11 2020-11-01      1363      1363      83
#> 72      72  2020     12 2020-12-01      1363      1363      83
#>      R_F_month
#>      <dbl>
#> 1 0.000082029
#> 2 0.000082029
#> 3 0.000082029
#> 4 0.000082029
#> 5 0.000082029
#> 6 0.000082029
#> 7 0.000082029
#> 8 0.000082029
#> 9 0.000082029
#> 10 0.000082029
#> 11 0.000082029
#> 12 0.000082029
#> 13 0.000082029
#> 14 0.000075959
#> 15 0.000075959
#> 16 0.000082029

```

```
#> 17 0.000082029
#> 18 0.000082029
#> 19 0.000082029
#> 20 0.000082029
#> 21 0.000082029
#> 22 0.000075959
#> 23 0.000075959
#> 24 0.000075959
#> 25 0.000075959
#> 26 0.000075959
#> 27 0.000075959
#> 28 0.000082029
#> 29 0.000075959
#> 30 0.000075959
#> 31 0.000075959
#> 32 0.000075959
#> 33 0.000075959
#> 34 0.000075959
#> 35 0.000075959
#> 36 0.000075959
#> 37 0.000075959
#> 38 0.000075959
#> 39 0.000082029
#> 40 0.000082029
#> 41 0.000082029
#> 42 0.000075959
#> 43 0.000075959
#> 44 0.000082029
#> 45 0.000082029
#> 46 0.000082029
#> 47 0.000082029
#> 48 0.000082029
#> 49 0.000075959
#> 50 0.000078994
#> 51 0.000082029
#> 52 0.000082029
#> 53 0.000082029
#> 54 0.000075959
#> 55 0.000082029
#> 56 0.000082029
#> 57 0.000082029
#> 58 0.000075959
```

```

#> 59 0.000075959
#> 60 0.000075959
#> 61 0.000082029
#> 62 0.000082029
#> 63 0.000082029
#> 64 0.000075959
#> 65 0.000075959
#> 66 0.000075959
#> 67 0.000075959
#> 68 0.000075959
#> 69 0.000075959
#> 70 0.000082029
#> 71 0.000082029
#> 72 0.000082029

stock_data <- stock_data |>
  select(-R_F) |>
  left_join(
    risk_free_rate_by_month |>
      select(
        month_ID,
        R_F = R_F_month
      ),
    by = "month_ID"
  )

missing_monthly_risk_free_rate <- stock_data |>
  distinct(month_ID, R_F) |>
  filter(!is.finite(R_F))

if (nrow(missing_monthly_risk_free_rate) > 0) {
  stop(
    "      month_ID      ",
    call. = FALSE
  )
}

```

$$R_{i,t}^e$$

$$R_{i,t}^e = R_{i,t} - R_{F,t}$$

```

stock_data <- stock_data |>
  mutate(
    Re = R - R_F,
    Re = if_else(
      is.finite(Re),
      Re,
      NA_real_
    )
  )

```

5.4 3.4

5.4.1 3.4.1

```

calculate_skewness_LFL <- function(x) {
  values <- x[is.finite(x)]

  if (
    length(values) < 3 ||
    near(sd(values), 0)
  ) {
    return(NA_real_)
  }

  mean(
    ((values - mean(values)) /
      sd(values))^3
  )
}

calculate_kurtosis_LFL <- function(x) {
  values <- x[is.finite(x)]

  if (
    length(values) < 4 ||
    near(sd(values), 0)
  ) {
    return(NA_real_)
  }
}

```

```

mean(
  ((values - mean(values)) /
    sd(values))^4
)
}

```

```

monthly_return_summary <- stock_data |>
  select(R, Re) |>
  pivot_longer(
    cols = everything(),
    names_to = "return_type",
    values_to = "return"
  ) |>
  filter(is.finite(return)) |>
  group_by(return_type) |>
  summarise(
    observations = n(),
    mean = mean(return),
    median = median(return),
    standard_deviation = sd(return),
    minimum = min(return),
    first_quartile = quantile(return, 0.25),
    third_quartile = quantile(return, 0.75),
    maximum = max(return),
    skewness = calculate_skewness_LFL(return),
    kurtosis = calculate_kurtosis_LFL(return),
    .groups = "drop"
  )

```

```
show_table_LFL(monthly_return_summary)
```

```

#> # A tibble: 2 x 11
#>   return_type observations      mean  median standard_deviation  minimum
#>   <chr>          <int>    <dbl>   <dbl>          <dbl>    <dbl>
#> 1 R              93525 0.015859 0.010332      0.091131 -0.38028
#> 2 Re             93525 0.015779 0.010252      0.091131 -0.38036
#>   first_quartile third_quartile maximum skewness kurtosis
#>   <dbl>          <dbl>    <dbl>   <dbl>    <dbl>
#> 1    -0.040573     0.064808 0.51498 0.50657  4.2680
#> 2    -0.040652     0.064730 0.51490 0.50656  4.2680

```

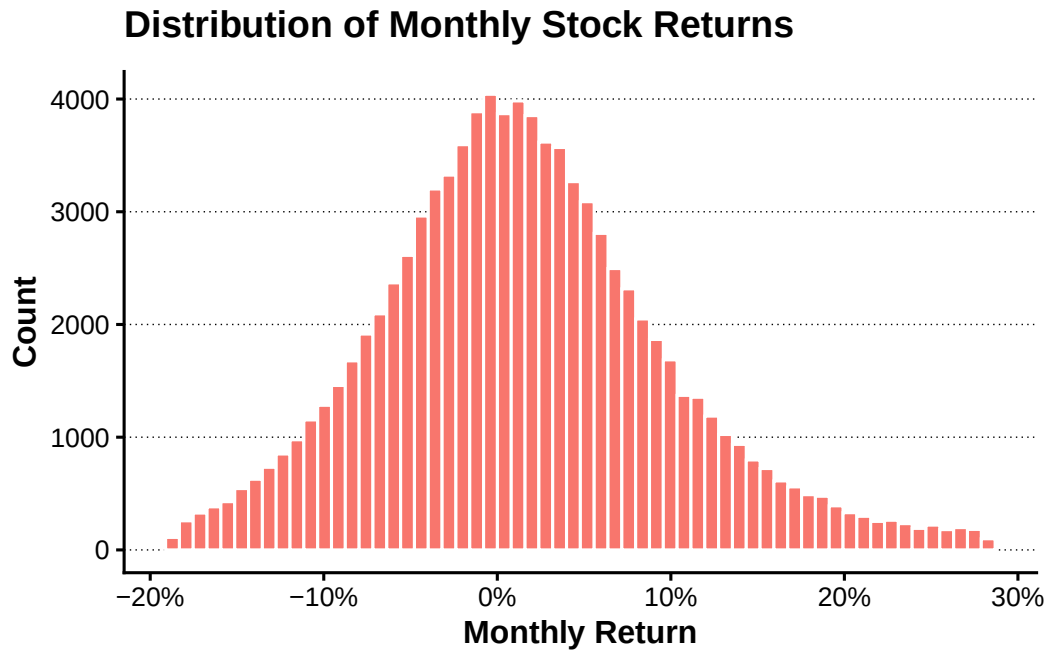
5.4.2 3.4.2

1 99

```
monthly_return_limits <- quantile(
  stock_data$R,
  probs = c(0.01, 0.99),
  na.rm = TRUE
)

p_monthly_return_distribution <- stock_data |>
  filter(
    is.finite(R),
    between(
      R,
      monthly_return_limits[[1]],
      monthly_return_limits[[2]]
    )
  ) |>
  ggplot(aes(x = R, fill = "Monthly Return")) +
  geom_histogram(
    bins = 60,
    colour = "white",
    show.legend = FALSE
  ) +
  scale_x_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Monthly Return",
    y = "Count",
    title = "Distribution of Monthly Stock Returns"
  )

add_lagrange_theme_LFL(
  p_monthly_return_distribution
)
```



5.4.3 3.4.3

6

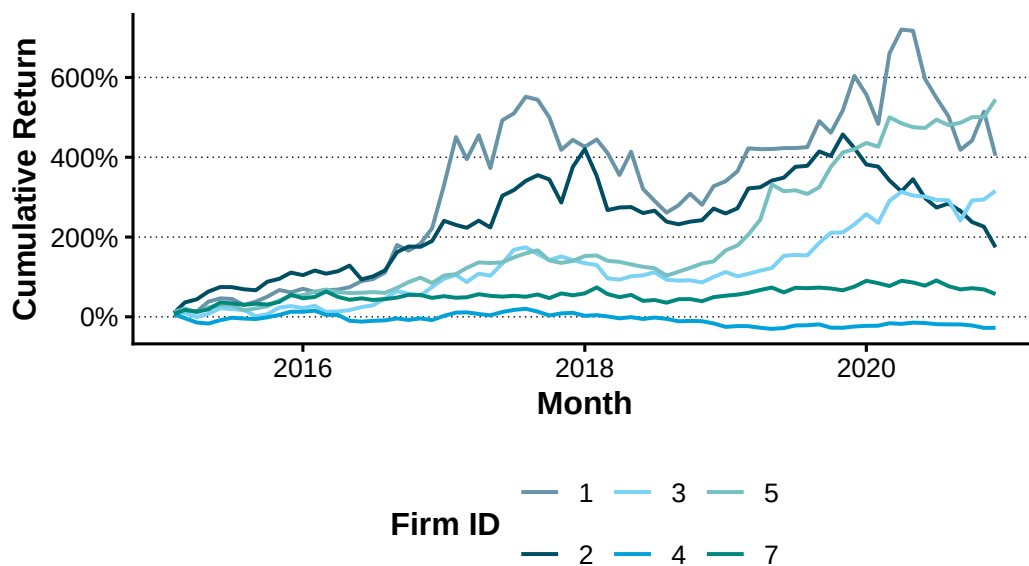
```
sample_firm_ids <- stock_data |>
  filter(is.finite(R)) |>
  count(
    firm_ID,
    sort = TRUE
  ) |>
  slice_head(n = 6) |>
  pull(firm_ID)

cumulative_return_data <- stock_data |>
  filter(
    firm_ID %in% sample_firm_ids,
    is.finite(R)
  ) |>
  group_by(firm_ID) |>
  arrange(month_ID, .by_group = TRUE) |>
  mutate(
    cumulative_return = cumprod(1 + R) - 1
```

```
) |>  
ungroup()
```

```
p_cumulative_return <- cumulative_return_data |>  
  ggplot(  
    aes(  
      x = month_date,  
      y = cumulative_return,  
      colour = factor(firm_ID)  
    )  
  ) +  
  geom_line(linewidth = 0.7) +  
  scale_y_continuous(  
    labels = scales::label_percent()  
  ) +  
  labs(  
    x = "Month",  
    y = "Cumulative Return",  
    colour = "Firm ID",  
    title = "Cumulative Returns of Selected Firms"  
  )  
  
add_lagrange_theme_LFL(p_cumulative_return) +  
  ggthemes::scale_colour_economist()
```


Cumulative Returns of Selected Firms



5.4.4 3.4.4

```
firm_return_summary <- sample_firm_ids |>
  map_dfr(
    \(firm_id) {
      firm_returns <- stock_data |>
        filter(
          firm_ID == firm_id,
          is.finite(R)
        ) |>
        pull(R)

      tibble(
        firm_ID = firm_id,
        observations = length(firm_returns),
        mean_return = mean(firm_returns),
        volatility = sd(firm_returns),
        minimum_return = min(firm_returns),
        maximum_return = max(firm_returns)
      )
    }
  )
```

```
)

show_table_LFL(firm_return_summary)
#> # A tibble: 6 x 6
#>   firm_ID observations mean_return volatility minimum_return maximum_return
#>   <int>      <int>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1     1         71  0.029240  0.11544    -0.18157    0.33603
#> 2     2         71  0.018125  0.089135   -0.18825    0.24481
#> 3     3         71  0.023028  0.075369   -0.14460    0.16440
#> 4     4         71 -0.0030159  0.054766   -0.13816    0.11475
#> 5     5         71  0.028390  0.061602   -0.097372    0.25720
#> 6     7         71  0.0077876  0.053950   -0.096949    0.14549
```

5.5 3.5

5.5.1 3.5.1

$i \ t$

$$R_{i,t}^{annual} = \prod_{m=1}^{M_{i,t}} (1 + R_{i,t,m}) - 1$$

```
annual_stock_data <- stock_data |>
  group_by(
    firm_ID,
    year
  ) |>
  summarise(
    months = sum(is.finite(R)),
    R = if (
      sum(is.finite(R)) == 0
    ) {
      NA_real_
    } else {
      prod(1 + R[is.finite(R)]) - 1
    },
    Re = if (
      sum(is.finite(Re)) == 0
    ) {
      NA_real_
    }
```

```

} else {
  prod(1 + Re[is.finite(Re)]) - 1
},
ME = {
  values <- ME[is.finite(ME)]

  if (length(values) == 0) {
    NA_real_
  } else {
    dplyr::last(values)
  }
},
.groups = "drop"
) |>
arrange(firm_ID, year)

show_table_LFL(annual_stock_data, n = 20)
#> # A tibble: 7,920 x 6
#>   firm_ID year months      R      Re      ME
#>   <int> <int> <int>   <dbl>   <dbl>   <dbl>
#> 1     1     1  2015     11  0.61213  0.61073 3577294000
#> 2     1     1  2016     12  0.99727  0.99547 6883324000
#> 3     1     1  2017     12  0.68786  0.68637 11376990000
#> 4     1     1  2018     12 -0.21361 -0.21439 8694752000
#> 5     1     1  2019     12  0.64684  0.64533 13957518000
#> 6     1     1  2020     12 -0.28430 -0.28501 9708946000
#> 7     2     2  2015     11  1.1092   1.1074 4086732000
#> 8     2     2  2016     12  0.37523  0.37395 5592864000
#> 9     2     2  2017     12  0.64073  0.63928 9152528000
#> 10    2     2  2018     12 -0.21969 -0.22046 7103688000
#> 11    2     2  2019     12  0.40819  0.40688 9962680000
#> 12    2     2  2020     12 -0.47547 -0.47599 5194044000
#> 13    3     3  2015     11  0.27011  0.26898 70434234000
#> 14    3     3  2016     12  0.37526  0.37398 96064839000
#> 15    3     3  2017     12  0.38682  0.38558 132546843000
#> 16    3     3  2018     12 -0.17493 -0.17573 107515395000
#> 17    3     3  2019     12  0.65869  0.65717 174487833000
#> 18    3     3  2020     12  0.25382  0.25265 214032195000
#> 19    4     4  2015     11  0.12885  0.12784 14518674000
#> 20    4     4  2016     12 -0.18552 -0.18631 11592135000
#> # i 7,900 more rows

```

5.5.2 3.5.2

2 data/derived/fundamentals/financial_characteristics.parquet

```
financial_data <-
  read_derived_parquet_file_LFL(
    file.path(
      "fundamentals",
      "financial_characteristics.parquet"
    )
  ) |>
  mutate(
    firm_ID = as.integer(firm_ID),
    year = as.integer(year)
  )

show_table_LFL(financial_data, n = 10)
```

```
#> # A tibble: 7,920 x 25
#>   year firm_ID industry_ID sales    OX    NFE    X    OA    FA
#>   <int>  <int> <fct>      <dbl> <dbl>   <dbl> <dbl> <dbl> <dbl>
#> 1  2015      1 1          5261.4 437.49 NA      286.64 13006. 3543.4
#> 2  2016      1 1          5949.0 564.14 50.667  513.48 13866. 4642.2
#> 3  2017      1 1          6505.1 691.18 29.543  661.64 13953. 7744.0
#> 4  2018      1 1          6846.4 751.29 86.487  664.8  18818. 7284.7
#> 5  2019      1 1          7572.2 958.53 298.05  660.48 18190  9735.1
#> 6  2020      1 1          7537.6 778.37 -65.459  843.83 20463. 10274.
#> 7  2015      2 1          3505.8 45.82   5.7511  40.07  2977.8 2258.3
#> 8  2016      2 1          3491.3 51.25   1.8765  49.37  3184.8 1881.8
#> 9  2017      2 1          3945.7 83.43   7.5279  75.9   3392.2 2425.2
#> 10 2018      2 1          4139.3 93.4    6.8166  86.58  3569.3 2331.0
#>   OL    FO    NOA    NFO    BE lagged_BE    ROE lagged_NOA
#>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
#> 1 4373.0 2480.7 8632.6 -1062.7 9695.3 NA NA NA
#> 2 4534.2 3959.7 9331.4 -682.46 10014. 9695.3 0.052962 8632.6
#> 3 5111.2 6159.0 8841.4 -1585.0 10426. 10014. 0.066073 9331.4
#> 4 5137.3 10124. 13681. 2839.2 10842. 10426. 0.063762 8841.4
#> 5 5488.0 11362. 12702. 1627.1 11075. 10842. 0.060919 13681.
#> 6 5371.4 13772. 15091. 3497.9 11594. 11075. 0.076193 12702.
#> 7 1840.4 2340.9 1137.5 82.560 1054.9 NA NA NA
#> 8 1769.2 2215.9 1415.6 334.04 1081.6 1054.9 0.046800 1137.5
#> 9 1955.3 2727.4 1436.9 302.14 1134.8 1081.6 0.070174 1415.6
#> 10 2022.1 2694.5 1547.2 363.48 1183.7 1134.8 0.076295 1436.9
#>   lagged_NFO    RNOA    PM    ATO lagged_FLEV    NBC ROE_DuPont
```

```

#>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1      NA      NA      0.083151 NA      NA      NA      NA      NA
#> 2 -1062.7    0.065350 0.094830 0.68913 -0.10961 -0.047678 0.052961
#> 3  -682.46    0.074071 0.10625  0.69712 -0.068152 -0.043289 0.066072
#> 4 -1585.0    0.084974 0.10974  0.77436 -0.15202 -0.054567 0.063762
#> 5  2839.2    0.070062 0.12658  0.55348  0.26187  0.10498  0.060919
#> 6  1627.1    0.061279 0.10326  0.59342  0.14692 -0.040231 0.076193
#> 7      NA      NA      0.013070 NA      NA      NA      NA      NA
#> 8    82.560  0.045056 0.014679 3.0693  0.078263 0.022729 0.046803
#> 9   334.04  0.058935 0.021145 2.7872  0.30884 0.022536 0.070176
#> 10  302.14  0.064999 0.022564 2.8807  0.26625 0.022561 0.076298
#>   DuPont_gap
#>      <dbl>
#> 1  NA
#> 2 7.7332e-7
#> 3 3.1527e-7
#> 4 -3.3567e-7
#> 5 -2.0862e-8
#> 6 1.1089e-7
#> 7  NA
#> 8 -3.2789e-6
#> 9 -1.9587e-6
#> 10 -2.9722e-6
#> # i 7,910 more rows

```

```

financial_key_duplicates <- financial_data |>
  count(
    firm_ID,
    year,
    name = "observations"
  ) |>
  filter(observations > 1)

show_table_LFL(financial_key_duplicates)
#> # A tibble: 0 x 3
#> # i 3 variables: firm_ID <int>, year <int>, observations <int>

if (nrow(financial_key_duplicates) > 0) {
  stop(
    "    firm_ID year    ",

```

```

    call. = FALSE
  )
}

```

5.5.3 3.5.3

```

annual_data <- annual_stock_data |>
  left_join(
    financial_data,
    by = c("firm_ID", "year")
  ) |>
  arrange(firm_ID, year)

stopifnot(
  nrow(annual_data) ==
    nrow(annual_stock_data)
)

annual_data_overview <- annual_data |>
  summarise(
    observations = n(),
    firms = n_distinct(firm_ID),
    first_year = min(year, na.rm = TRUE),
    last_year = max(year, na.rm = TRUE),
    financial_matches =
      sum(!is.na(BE))
  )

show_table_LFL(annual_data_overview)
#> # A tibble: 1 x 5
#>   observations firms first_year last_year financial_matches
#>   <int> <int>    <int>    <int>         <int>
#> 1      7920  1515      2015      2020           7920

```

5.5.4 3.5.4

```

monthly_data <- stock_data |>
  left_join(

```

```

    financial_data,
    by = c("firm_ID", "year")
  ) |>
  arrange(firm_ID, month_ID)

stopifnot(
  nrow(monthly_data) ==
    nrow(stock_data)
)

row_count_check <- tibble(
  stock_rows = nrow(stock_data),
  monthly_rows = nrow(monthly_data),
  annual_rows = nrow(annual_data)
)

show_table_LFL(row_count_check)
#> # A tibble: 1 x 3
#>   stock_rows monthly_rows annual_rows
#>   <int>         <int>         <int>
#> 1     95040         95040         7920

```

5.5.5 3.5.5

```

latest_analysis_year <- annual_data |>
  filter(
    is.finite(sales),
    sales > 0,
    is.finite(X),
    X > 0,
    is.finite(ME),
    ME > 0
  ) |>
  summarise(
    latest_year = max(year)
  ) |>
  pull(latest_year)

bubble_chart_data <- annual_data |>

```

```

filter(
  year == latest_analysis_year,
  is.finite(sales),
  sales > 0,
  is.finite(X),
  X > 0,
  is.finite(ME),
  ME > 0
)

```

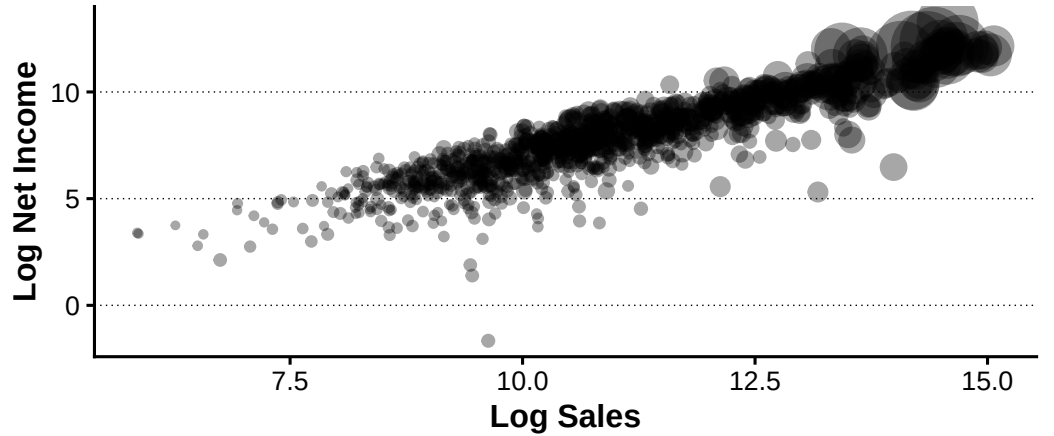
```

p_sales_earnings_market_equity <- bubble_chart_data |>
  ggplot(
    aes(
      x = log(sales),
      y = log(X),
      size = ME
    )
  ) +
  geom_point(alpha = 0.35) +
  scale_size(
    range = c(1, 12),
    name = "Market Equity"
  ) +
  labs(
    x = "Log Sales",
    y = "Log Net Income",
    title = paste(
      "Sales, Earnings and Market Equity in",
      latest_analysis_year
    )
  )

add_lagrange_theme_LFL(
  p_sales_earnings_market_equity
)

```


Sales, Earnings and Market Equity in 2020



1000000000000 2000000000000 3000000000000 4000000000000

5.6 3.6

5.6.1 3.6.1 t

0

```
test_firm_id <- stock_data |>
  filter(is.finite(Re)) |>
  count(
    firm_ID,
    sort = TRUE
  ) |>
  slice_head(n = 1) |>
  pull(firm_ID)

test_firm_returns <- stock_data |>
  filter(
    firm_ID == test_firm_id,
    is.finite(Re)
  ) |>
  pull(Re)
```

```

manual_t_value <- mean(test_firm_returns) /
  (
    sd(test_firm_returns) /
    sqrt(length(test_firm_returns))
  )

t_test_result <- t.test(
  test_firm_returns,
  mu = 0
)

t_test_summary <- broom::tidy(t_test_result) |>
  mutate(
    firm_ID = test_firm_id,
    manual_t_value = manual_t_value,
    .before = 1
  )

show_table_LFL(t_test_summary)
#> # A tibble: 1 x 10
#>   firm_ID manual_t_value estimate statistic p.value parameter  conf.low
#>   <int>         <dbl>     <dbl>     <dbl>   <dbl>     <dbl>    <dbl>
#> 1       1           2.1285 0.029161    2.1285 0.036817      70 0.0018367
#>   conf.high method                alternative
#>   <dbl> <chr>                  <chr>
#> 1 0.056485 One Sample t-test two.sided

```

5.6.2 3.6.2

```

annual_data <- annual_data |>
  group_by(firm_ID) |>
  arrange(year, .by_group = TRUE) |>
  mutate(
    lagged_ME = lag(ME),
    lagged_BEME = lagged_BE / lagged_ME,
    lagged_BEME = if_else(
      is.finite(lagged_BEME) &
      lagged_BEME > 0,
      lagged_BEME,

```

```

      NA_real_
    )
  ) |>
  ungroup()

```

```

regression_observations_by_year <- annual_data |>
  filter(
    is.finite(Re),
    is.finite(lagged_BEME)
  ) |>
  count(
    year,
    name = "observations"
  ) |>
  arrange(year)

show_table_LFL(
  regression_observations_by_year,
  n = nrow(regression_observations_by_year)
)
#> # A tibble: 5 x 2
#>   year observations
#>   <int>         <int>
#> 1  2016         1241
#> 2  2017         1270
#> 3  2018         1268
#> 4  2019         1304
#> 5  2020         1322

```

```

eligible_regression_years <-
  regression_observations_by_year |>
  filter(observations >= 30)

if (nrow(eligible_regression_years) == 0) {
  stop(
    " ",
    call. = FALSE
  )
}

```

```

regression_year <- eligible_regression_years |>
  summarise(year = max(year)) |>
  pull(year)

lm_sample_data <- annual_data |>
  filter(
    year == regression_year,
    is.finite(Re),
    is.finite(lagged_BEME)
  ) |>
  select(
    firm_ID,
    year,
    Re,
    lagged_BEME
  )

```

```

beme_plot_limit <- quantile(
  lm_sample_data$lagged_BEME,
  0.99,
  na.rm = TRUE
)

p_beme_return_regression <- lm_sample_data |>
  ggplot(
    aes(
      x = lagged_BEME,
      y = Re
    )
  ) +
  geom_point(alpha = 0.35) +
  geom_smooth(
    method = "lm",
    se = FALSE
  ) +
  coord_cartesian(
    xlim = c(0, beme_plot_limit)
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(

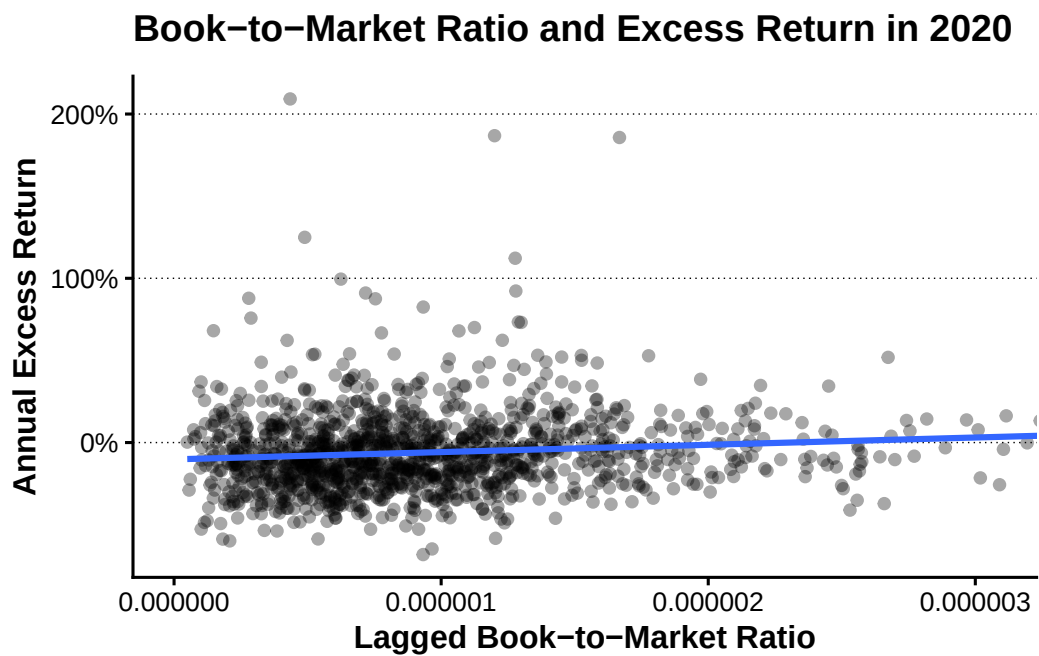
```

```

x = "Lagged Book-to-Market Ratio",
y = "Annual Excess Return",
title = paste(
  "Book-to-Market Ratio and Excess Return in",
  regression_year
)
)

add_lagrange_theme_LFL(
  p_beme_return_regression
)

```



```

lm_result <- lm(
  Re ~ lagged_BEME,
  data = lm_sample_data
)

show_table_LFL(
  broom::tidy(lm_result)
)

#> # A tibble: 2 x 5
#>   term                estimate  std.error statistic    p.value

```

```

#>   <chr>           <dbl>         <dbl>         <dbl>         <dbl>
#> 1 (Intercept)    -0.10237      0.012394      -8.2593 3.5232e-16
#> 2 lagged_BEME 44359.      10973.         4.0426 5.5911e- 5

show_table_LFL(
  broom::glance(lm_result)
)
#> # A tibble: 1 x 12
#>   r.squared adj.r.squared  sigma statistic    p.value    df logLik    AIC
#>   <dbl>      <dbl>      <dbl>    <dbl>    <dbl> <dbl> <dbl> <dbl>
#> 1  0.012230    0.011481  0.24523    16.343 0.000055911    1 -16.667 39.334
#>   BIC deviance df.residual  nobs
#>   <dbl>    <dbl>      <int> <int>
#> 1 54.895   79.379      1320 1322

```

5.6.3 3.6.3

```
nest() purrr::map()
```

```

yearly_regression_models <- annual_data |>
  filter(
    is.finite(Re),
    is.finite(lagged_BEME)
  ) |>
  group_by(year) |>
  filter(n() >= 30) |>
  ungroup() |>
  nest(data = -year) |>
  mutate(
    model = map(
      data,
      \(data) lm(
        Re ~ lagged_BEME,
        data = data
      )
    ),
    coefficients = map(
      model,
      broom::tidy
    ),
    fit = map(

```

```

    model,
    broom::glance
  )
)

```

```

yearly_regression_coefficients <-
  yearly_regression_models |>
  select(year, coefficients) |>
  unnest(coefficients) |>
  arrange(year, term)

```

```

yearly_regression_fit <-
  yearly_regression_models |>
  select(year, fit) |>
  unnest(fit) |>
  select(
    year,
    r.squared,
    adj.r.squared,
    sigma,
    statistic,
    p.value,
    nobs
  ) |>
  arrange(year)

```

```

show_table_LFL(
  yearly_regression_coefficients,
  n = nrow(yearly_regression_coefficients)
)

```

```
#> # A tibble: 10 x 6
```

#>	year	term	estimate	std.error	statistic	p.value
#>	<int>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
#> 1	2016	(Intercept)	0.056439	0.016563	3.4076	6.7644e- 4
#> 2	2016	lagged_BEME	72103.	12275.	5.8738	5.4661e- 9
#> 3	2017	(Intercept)	0.17989	0.015673	11.478	4.4094e-29
#> 4	2017	lagged_BEME	32467.	11890.	2.7306	6.4094e- 3
#> 5	2018	(Intercept)	-0.12679	0.015203	-8.3395	1.9259e-16
#> 6	2018	lagged_BEME	65283.	13067.	4.9959	6.6754e- 7
#> 7	2019	(Intercept)	0.28895	0.026406	10.942	1.0121e-26
#> 8	2019	lagged_BEME	100567.	19057.	5.2771	1.5363e- 7
#> 9	2020	(Intercept)	-0.10237	0.012394	-8.2593	3.5232e-16

```
#> 10 2020 lagged_BEME 44359.      10973.      4.0426 5.5911e- 5

show_table_LFL(
  yearly_regression_fit,
  n = nrow(yearly_regression_fit)
)
#> # A tibble: 5 x 7
#>   year r.squared adj.r.squared  sigma statistic  p.value  nobs
#>   <int>   <dbl>      <dbl>   <dbl>      <dbl>    <dbl> <int>
#> 1  2016 0.027092    0.026306  0.28691    34.501  5.4661e-9  1241
#> 2  2017 0.0058459   0.0050619  0.28492     7.4562  6.4094e-3  1270
#> 3  2018 0.019334    0.018559   0.27721    24.959  6.6754e-7  1268
#> 4  2019 0.020940    0.020188   0.49819    27.847  1.5363e-7  1304
#> 5  2020 0.012230    0.011481   0.24523    16.343  5.5911e-5  1322
```

5.7 3.7

```
4          data/derived/returns/monthly_stock_returns.parquet    data/derived/fundamentals/annual_firm_characteristics.parquet
```

```
output_data <- list(
  monthly = monthly_data,
  annual = annual_data
)

output_files <- c(
  monthly = file.path(
    "returns",
    "monthly_stock_returns.parquet"
  ),
  annual = file.path(
    "fundamentals",
    "annual_firm_characteristics.parquet"
  )
)

output_paths <- output_data |>
  imap_chr(
    \(data, data_name)
      write_derived_parquet_file_LFL(
        data,
        output_files[[data_name]]
      )
  )
```



```

    )
  )

output_paths
#>
#>          "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/returns/monthly_stock_re
#>
#> "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/fundamentals/annual_firm_characteri

```

```

output_verification <- output_data |>
  imap_dfr(
    \(expected_data, data_name) {
      output_check <-
        read_derived_parquet_file_LFL(
          output_files[[data_name]]
        )

      stopifnot(
        nrow(output_check) ==
          nrow(expected_data)
      )

      stopifnot(
        identical(
          names(output_check),
          names(expected_data)
        )
      )

      tibble(
        data = data_name,
        output_file = output_paths[[data_name]],
        rows = nrow(output_check),
        columns = ncol(output_check),
        size_bytes =
          file.info(
            output_paths[[data_name]]
          )$size
      )
    }
  )
}

```

```

)

show_table_LFL(output_verification)
#> # A tibble: 2 x 5
#>   data
#>   <chr>
#> 1 monthly
#> 2 annual
#>   output_file
#>   <chr>
#> 1 C:/AnalyticFin/Projects/LagrangeFinance/data/derived/returns/monthly_stock_re~
#> 2 C:/AnalyticFin/Projects/LagrangeFinance/data/derived/fundamentals/annual_firm~
#>   rows columns size_bytes
#>   <int>   <int>     <dbl>
#> 1  95040     39    4761273
#> 2   7920     31    1698508

```

5.8 3.8

2

data/derived/returns/m

4

6 I-4

3 SMB HML 10 CAPM Fama–
French 3

1. 3
- 2.
- 3.
- 4.
5. 2×3 SMB HML
6. CAPM FF3
7. 5

6.1 4.1

1 3 Parquet

6.2 4.2

6.2.1 4.2.1 3

3 2

```
data/derived/returns/monthly_stock_returns.parquet
data/derived/fundamentals/annual_firm_characteristics.parquet
```

monthly_stock_returns.parquet annual_firm_characteristics.parquet

```

tbl_monthly <-
  read_derived_parquet_file_LFL(
    file.path(
      "returns",
      "monthly_stock_returns.parquet"
    )
  ) |>
mutate(
  firm_ID = as.integer(firm_ID),
  year = as.integer(year),
  month = as.integer(month),
  month_ID = as.integer(month_ID),
  month_date = as.Date(month_date)
) |>
arrange(firm_ID, month_ID)

tbl_annual <-
  read_derived_parquet_file_LFL(
    file.path(
      "fundamentals",
      "annual_firm_characteristics.parquet"
    )
  ) |>
mutate(
  firm_ID = as.integer(firm_ID),
  year = as.integer(year)
) |>
arrange(firm_ID, year)

show_data_structure_LFL(tbl_monthly)
#> Rows: 95,040
#> Columns: 39
#> $ year                <int> 2015, 2015, 2015, 2015, 2015, 2015, 2015, 2015, ~
#> $ month               <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 1, 2, 3, ~
#> $ month_ID            <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, ~
#> $ firm_ID             <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ stock_price         <dbl> 954, 960, 1113, 1081, 1317, 1366, 1353, 1209, 1~
#> $ DPS                 <dbl> 0, 0, 0, 0, 0, 29, 0, 0, 0, 0, 0, 29, 0, 0, 0, ~
#> $ shares_outstanding  <dbl> 2422000, 2422000, 2422000, 2422000, 2422000, 24~
#> $ adjustment_coefficient <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ month_date          <date> 2015-01-01, 2015-02-01, 2015-03-01, 2015-04-01~
#> $ ME                 <dbl> 2310588000, 2325120000, 2695686000, 2618182000, ~

```

```

#> $ lagged_stock_price <dbl> NA, 954, 960, 1113, 1081, 1317, 1366, 1353, 120~
#> $ lagged_month_ID <int> NA, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, ~
#> $ consecutive_month <lgl> NA, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, T~
#> $ R <dbl> NA, 0.006289308, 0.159375000, -0.028751123, 0.2~
#> $ R_F <dbl> 0.00008202927, 0.00008202927, 0.00008202927, 0.~
#> $ Re <dbl> NA, 0.006207279, 0.159292971, -0.028833152, 0.2~
#> $ industry_ID <fct> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ sales <dbl> 5261.40, 5261.40, 5261.40, 5261.40, 5261.40, 52~
#> $ OX <dbl> 437.49, 437.49, 437.49, 437.49, 437.49, 437.49,~
#> $ NFE <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ X <dbl> 286.64, 286.64, 286.64, 286.64, 286.64, 286.64,~
#> $ OA <dbl> 13005.55, 13005.55, 13005.55, 13005.55, 13005.5~
#> $ FA <dbl> 3543.43, 3543.43, 3543.43, 3543.43, 3543.43, 35~
#> $ OL <dbl> 4372.96, 4372.96, 4372.96, 4372.96, 4372.96, 43~
#> $ FO <dbl> 2480.72, 2480.72, 2480.72, 2480.72, 2480.72, 24~
#> $ NOA <dbl> 8632.59, 8632.59, 8632.59, 8632.59, 8632.59, 86~
#> $ NFO <dbl> -1062.71, -1062.71, -1062.71, -1062.71, -1062.7~
#> $ BE <dbl> 9695.30, 9695.30, 9695.30, 9695.30, 9695.30, 96~
#> $ lagged_BE <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ ROE <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ lagged_NOA <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ lagged_NFO <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ RNOA <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ PM <dbl> 0.08315087, 0.08315087, 0.08315087, 0.08315087,~
#> $ ATO <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ lagged_FLEV <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ NBC <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ ROE_DuPont <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ DuPont_gap <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
show_data_structure_LFL(tbl_annual)
#> Rows: 7,920
#> Columns: 31
#> $ firm_ID <int> 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 4, 4~
#> $ year <int> 2015, 2016, 2017, 2018, 2019, 2020, 2015, 2016, 2017, 2018~
#> $ months <int> 11, 12, 12, 12, 12, 12, 11, 12, 12, 12, 12, 12, 11, 12, 12~
#> $ R <dbl> 0.61213017, 0.99727265, 0.68786382, -0.21361287, 0.6468357~
#> $ Re <dbl> 0.61073273, 0.99547052, 0.68636684, -0.21438527, 0.6453315~
#> $ ME <dbl> 3577294000, 6883324000, 11376990000, 8694752000, 139575180~
#> $ industry_ID <fct> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ sales <dbl> 5261.40, 5948.96, 6505.06, 6846.38, 7572.24, 7537.63, 3505~
#> $ OX <dbl> 437.49, 564.14, 691.18, 751.29, 958.53, 778.37, 45.82, 51.~
#> $ NFE <dbl> NA, 50.667498, 29.543157, 86.486500, 298.049774, -65.45877~

```

```

#> $ X <dbl> 286.64, 513.48, 661.64, 664.80, 660.48, 843.83, 40.07, 49.~
#> $ OA <dbl> 13005.55, 13865.58, 13952.58, 18818.48, 18190.00, 20462.86~
#> $ FA <dbl> 3543.43, 4642.16, 7743.99, 7284.72, 9735.13, 10274.25, 225~
#> $ OL <dbl> 4372.96, 4534.22, 5111.22, 5137.28, 5487.96, 5371.38, 1840~
#> $ FO <dbl> 2480.72, 3959.70, 6159.02, 10123.91, 11362.22, 13772.15, 2~
#> $ NOA <dbl> 8632.59, 9331.36, 8841.36, 13681.20, 12702.04, 15091.48, 1~
#> $ NFO <dbl> -1062.71, -682.46, -1584.97, 2839.19, 1627.09, 3497.90, 82~
#> $ BE <dbl> 9695.30, 10013.82, 10426.33, 10842.01, 11074.95, 11593.58,~
#> $ lagged_BE <dbl> NA, 9695.30, 10013.82, 10426.33, 10842.01, 11074.95, NA, 1~
#> $ ROE <dbl> NA, 0.05296174, 0.06607269, 0.06376165, 0.06091859, 0.0761~
#> $ lagged_NOA <dbl> NA, 8632.59, 9331.36, 8841.36, 13681.20, 12702.04, NA, 113~
#> $ lagged_NFO <dbl> NA, -1062.71, -682.46, -1584.97, 2839.19, 1627.09, NA, 82.~
#> $ RNOA <dbl> NA, 0.06535003, 0.07407066, 0.08497448, 0.07006184, 0.0612~
#> $ PM <dbl> 0.083150872, 0.094830021, 0.106252671, 0.109735364, 0.1265~
#> $ ATO <dbl> NA, 0.6891281, 0.6971181, 0.7743582, 0.5534778, 0.5934189,~
#> $ lagged_FLEV <dbl> NA, -0.10961084, -0.06815181, -0.15201610, 0.26186934, 0.1~
#> $ NBC <dbl> NA, -0.047677633, -0.043289214, -0.054566648, 0.104977044,~
#> $ ROE_DuPont <dbl> NA, 0.05296097, 0.06607237, 0.06376199, 0.06091861, 0.0761~
#> $ DuPont_gap <dbl> NA, 0.00000077332412, 0.00000031526680, -0.00000033567247,~
#> $ lagged_ME <dbl> NA, 3577294000, 6883324000, 11376990000, 8694752000, 13957~
#> $ lagged_BEME <dbl> NA, 0.0000027102329, 0.0000014547942, 0.0000009164401, 0.0~

```

```

tbl_input_overview <- tribble(
  ~data, ~observations, ~variables, ~firms, ~first_year, ~last_year,
  "monthly", nrow(tbl_monthly), ncol(tbl_monthly),
  n_distinct(tbl_monthly$firm_ID),
  min(tbl_monthly$year, na.rm = TRUE),
  max(tbl_monthly$year, na.rm = TRUE),
  "annual", nrow(tbl_annual), ncol(tbl_annual),
  n_distinct(tbl_annual$firm_ID),
  min(tbl_annual$year, na.rm = TRUE),
  max(tbl_annual$year, na.rm = TRUE)
)

```

```
show_table_LFL(tbl_input_overview)
```

```

#> # A tibble: 2 x 6
#>   data      observations variables firms first_year last_year
#>   <chr>          <int>      <int> <int>      <int>      <int>
#> 1 monthly      95040         39  1515      2015      2020
#> 2 annual       7920         31  1515      2015      2020

```

6.2.2 4.2.2

```
vec_required_monthly_columns <- c(
  "year", "month", "month_ID", "month_date",
  "firm_ID", "R", "R_F", "Re", "ME"
)

vec_required_annual_columns <- c(
  "year", "firm_ID", "ME", "BE", "lagged_BE"
)

vec_missing_monthly_columns <- setdiff(
  vec_required_monthly_columns,
  names(tbl_monthly)
)

vec_missing_annual_columns <- setdiff(
  vec_required_annual_columns,
  names(tbl_annual)
)

if (length(vec_missing_monthly_columns) > 0) {
  stop(
    "          : ",
    paste(vec_missing_monthly_columns, collapse = ", ")
  )
}

if (length(vec_missing_annual_columns) > 0) {
  stop(
    "          : ",
    paste(vec_missing_annual_columns, collapse = ", ")
  )
}

tbl_monthly_key_duplicates <- tbl_monthly |>
  count(firm_ID, month_ID, name = "observations") |>
  filter(observations > 1)

tbl_annual_key_duplicates <- tbl_annual |>
  count(firm_ID, year, name = "observations") |>
```

```

filter(observations > 1)

if (nrow(tbl_monthly_key_duplicates) > 0) {
  stop("    firm_ID month_ID    ")
}

if (nrow(tbl_annual_key_duplicates) > 0) {
  stop("    firm_ID year    ")
}

show_table_LFL(tbl_monthly_key_duplicates)
#> # A tibble: 0 x 3
#> # i 3 variables: firm_ID <int>, month_ID <int>, observations <int>
show_table_LFL(tbl_annual_key_duplicates)
#> # A tibble: 0 x 3
#> # i 3 variables: firm_ID <int>, year <int>, observations <int>

```

6.2.3 4.2.3

```

compute_median_safe <- function(x) {
  values <- x[is.finite(x)]
  if (length(values) == 0) return(NA_real_)
  median(values)
}

compute_mean_safe <- function(x) {
  values <- x[is.finite(x)]
  if (length(values) == 0) return(NA_real_)
  mean(values)
}

compute_weighted_mean_safe <- function(x, weight) {
  valid <- is.finite(x) & is.finite(weight) & weight > 0
  if (!any(valid)) return(NA_real_)
  sum(x[valid] * weight[valid]) / sum(weight[valid])
}

compute_significance_stars <- function(p_value) {
  case_when(

```



```

    is.na(p_value) ~ "",
    p_value < 0.01 ~ "****",
    p_value < 0.05 ~ "***",
    p_value < 0.10 ~ "**",
    TRUE ~ ""
  )
}

```

6.3 4.3

6.3.1 4.3.1

t $t - 1$

```

if (!"lagged_ME" %in% names(tbl_annual)) {
  tbl_annual <- tbl_annual |>
    group_by(firm_ID) |>
    arrange(year, .by_group = TRUE) |>
    mutate(lagged_ME = lag(ME)) |>
    ungroup()
}

if (!"lagged_BEME" %in% names(tbl_annual)) {
  tbl_annual <- tbl_annual |>
    mutate(lagged_BEME = lagged_BE / lagged_ME)
}

tbl_annual <- tbl_annual |>
  mutate(
    lagged_ME = if_else(
      is.finite(lagged_ME) & lagged_ME > 0,
      lagged_ME,
      NA_real_
    ),
    lagged_BEME = if_else(
      is.finite(lagged_BEME) & lagged_BEME > 0,
      lagged_BEME,
      NA_real_
    )
  ) |>

```

```

arrange(firm_ID, year)

tbl_lagged_variable_availability <- tbl_annual |>
  group_by(year) |>
  summarise(
    firms = n_distinct(firm_ID),
    valid_lagged_ME = sum(is.finite(lagged_ME)),
    valid_lagged_BEME = sum(is.finite(lagged_BEME)),
    valid_for_2x3 = sum(
      is.finite(lagged_ME) & is.finite(lagged_BEME)
    ),
    .groups = "drop"
  )

show_table_LFL(tbl_lagged_variable_availability, n = Inf)
#> # A tibble: 6 x 5
#>   year firms valid_lagged_ME valid_lagged_BEME valid_for_2x3
#>   <int> <int>         <int>         <int>         <int>
#> 1  2015  1266             0             0             0
#> 2  2016  1293          1241          1241          1241
#> 3  2017  1319          1270          1270          1270
#> 4  2018  1323          1268          1268          1268
#> 5  2019  1356          1304          1304          1304
#> 6  2020  1363          1322          1322          1322

```

6.3.2 4.3.2

$t \quad i$

$$w_{i,t}^M = \frac{ME_{i,t-1}}{\sum_j ME_{j,t-1}}$$

```

tbl_market_membership <- tbl_annual |>
  filter(is.finite(lagged_ME), lagged_ME > 0) |>
  group_by(year) |>
  mutate(market_weight = lagged_ME / sum(lagged_ME)) |>
  ungroup() |>
  select(year, firm_ID, lagged_ME, market_weight)

tbl_market_weight_check <- tbl_market_membership |>
  group_by(year) |>

```

```

summarise(
  firms = n_distinct(firm_ID),
  weight_sum = sum(market_weight),
  minimum_weight = min(market_weight),
  maximum_weight = max(market_weight),
  .groups = "drop"
)

if (any(abs(tbl_market_weight_check$weight_sum - 1) > 1e-10)) {
  stop("          1          ")
}

show_table_LFL(tbl_market_weight_check, n = Inf)
#> # A tibble: 5 x 5
#>   year firms weight_sum minimum_weight maximum_weight
#>   <int> <int>      <dbl>          <dbl>          <dbl>
#> 1  2016  1241          1  0.0000019019          0.040134
#> 2  2017  1270          1  0.0000015083          0.032973
#> 3  2018  1268          1  0.0000012609          0.042639
#> 4  2019  1304          1  0.0000021887          0.039396
#> 5  2020  1322          1  0.00000098548          0.056868

```

$$R_{M,t} = \sum_i w_{i,t}^M R_{i,t}$$

```

tbl_market_returns <- tbl_monthly |>
  inner_join(
    tbl_market_membership,
    by = c("year", "firm_ID")
  ) |>
  group_by(month_ID, year, month, month_date) |>
  summarise(
    R_F = compute_median_safe(R_F),
    R_M = compute_weighted_mean_safe(R, lagged_ME),
    eligible_firms = n_distinct(firm_ID),
    valid_return_firms = sum(
      is.finite(R) & is.finite(lagged_ME) & lagged_ME > 0
    ),
    .groups = "drop"
  ) |>

```

```

mutate(
  R_Me = R_M - R_F,
  return_coverage = valid_return_firms / eligible_firms
) |>
arrange(month_ID)

tbl_invalid_market_months <- tbl_market_returns |>
  filter(!is.finite(R_F) | !is.finite(R_M) | !is.finite(R_Me))

if (nrow(tbl_invalid_market_months) > 0) {
  stop("      ")
}

show_table_LFL(tbl_market_returns)
#> # A tibble: 60 x 10
#>   month_ID year month month_date      R_F      R_M eligible_firms
#>   <int> <int> <int> <date>      <dbl>      <dbl>      <int>
#> 1     13  2016     1 2016-01-01 0.000082029  0.045835      1241
#> 2     14  2016     2 2016-02-01 0.000075959  0.028423      1241
#> 3     15  2016     3 2016-03-01 0.000075959 -0.057976      1241
#> 4     16  2016     4 2016-04-01 0.000082029 -0.0010718     1241
#> 5     17  2016     5 2016-05-01 0.000082029 -0.0064160     1241
#> 6     18  2016     6 2016-06-01 0.000082029 -0.037075     1241
#> 7     19  2016     7 2016-07-01 0.000082029  0.029853     1241
#> 8     20  2016     8 2016-08-01 0.000082029  0.053757     1241
#> 9     21  2016     9 2016-09-01 0.000082029  0.023650     1241
#> 10    22  2016    10 2016-10-01 0.000075959 -0.012053     1241
#>   valid_return_firms      R_Me return_coverage
#>           <int>      <dbl>      <dbl>
#> 1           1241  0.045753          1
#> 2           1241  0.028347          1
#> 3           1241 -0.058052          1
#> 4           1241 -0.0011539         1
#> 5           1241 -0.0064980         1
#> 6           1241 -0.037157          1
#> 7           1241  0.029771          1
#> 8           1241  0.053675          1
#> 9           1241  0.023568          1
#> 10          1241 -0.012129          1
#> # i 50 more rows

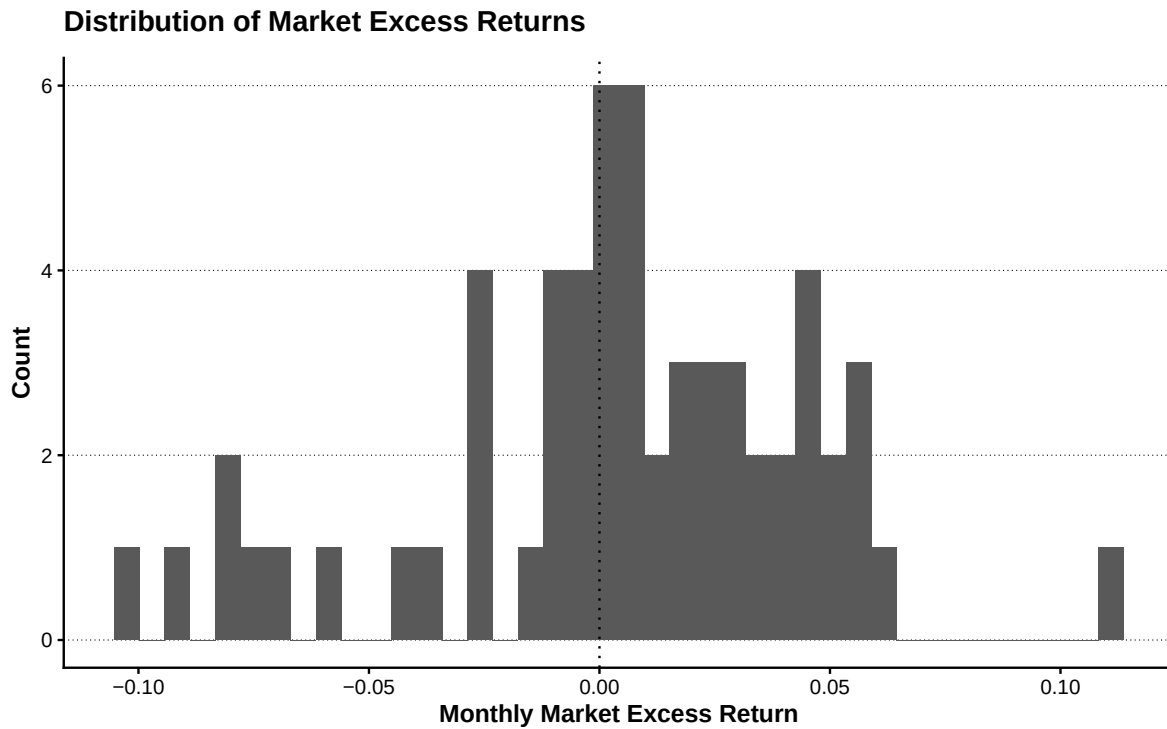
```

```
tbl_market_return_summary <- tbl_market_returns |>
  summarise(
    observations = n(),
    mean_market_return = mean(R_M),
    vec_mean_market_excess_return = mean(R_Me),
    market_volatility = sd(R_M),
    minimum_market_return = min(R_M),
    maximum_market_return = max(R_M),
    minimum_coverage = min(return_coverage)
  )

show_table_LFL(tbl_market_return_summary)
#> # A tibble: 1 x 7
#>   observations mean_market_return vec_mean_market_excess_return
#>   <int>          <dbl>          <dbl>
#> 1         60      0.0041003      0.0040215
#>   market_volatility minimum_market_return maximum_market_return minimum_coverage
#>   <dbl>          <dbl>          <dbl>          <dbl>
#> 1      0.041498      -0.10244      0.11104          1

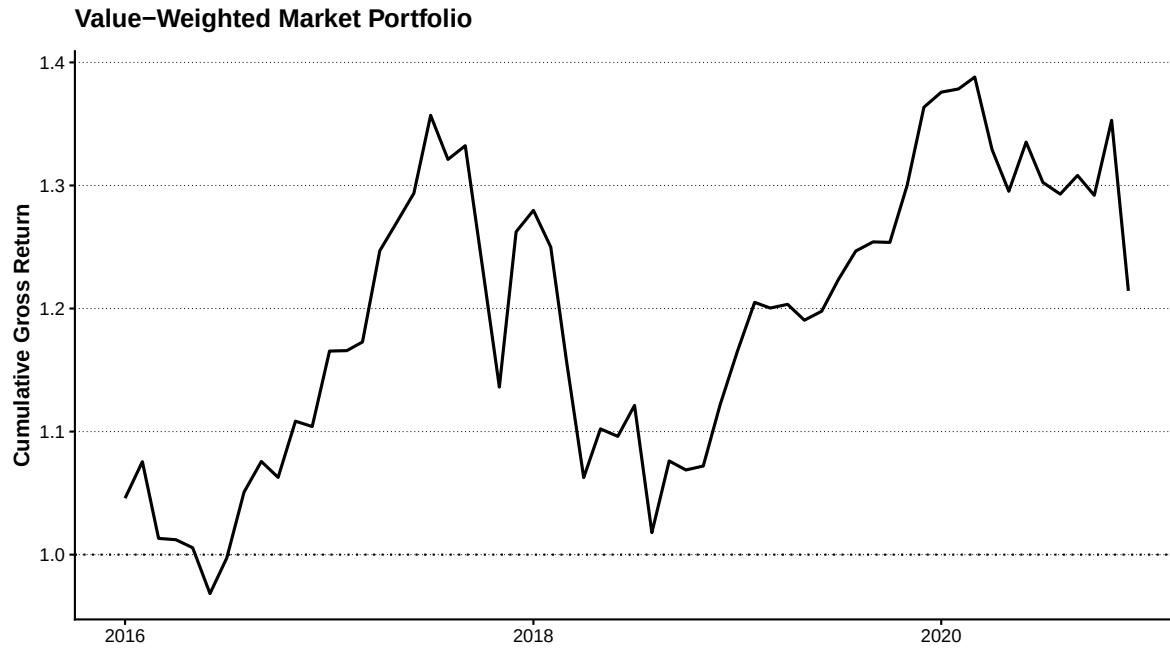
plot_market_excess_return <- tbl_market_returns |>
  ggplot(aes(x = R_Me)) +
  geom_histogram(bins = 40) +
  geom_vline(xintercept = 0, linetype = "dotted") +
  labs(
    x = "Monthly Market Excess Return",
    y = "Count",
    title = "Distribution of Market Excess Returns"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_market_excess_return))
```



```
plot_cumulative_market_return <- tbl_market_returns |>
  arrange(month_ID) |>
  mutate(cumulative_gross_market_return = cumprod(1 + R_M)) |>
  ggplot(aes(x = month_date, y = cumulative_gross_market_return)) +
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = 1, linetype = "dotted") +
  labs(
    x = NULL,
    y = "Cumulative Gross Return",
    title = "Value-Weighted Market Portfolio"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_cumulative_market_return))
```



6.3.3 4.3.3

```
tbl_formation_universe <- tbl_annual |>
  filter(
    is.finite(lagged_ME),
    lagged_ME > 0,
    is.finite(lagged_BEME),
    lagged_BEME > 0
  ) |>
  select(year, firm_ID, lagged_ME, lagged_BEME)

tbl_formation_breakpoints <- tbl_formation_universe |>
  group_by(year) |>
  summarise(
    firms = n_distinct(firm_ID),
    size_median = median(lagged_ME),
    BEME_30 = quantile(lagged_BEME, probs = 0.30, names = FALSE),
    BEME_70 = quantile(lagged_BEME, probs = 0.70, names = FALSE),
    .groups = "drop"
  )
```

```

tbl_formation <- tbl_formation_universe |>
  left_join(tbl_formation_breakpoints, by = "year") |>
  group_by(year) |>
  mutate(
    ME_rank10 = ntile(lagged_ME, 10),
    BEME_rank10 = ntile(lagged_BEME, 10)
  ) |>
  ungroup() |>
  mutate(
    size_group = if_else(lagged_ME <= size_median, "S", "B"),
    BEME_group = case_when(
      lagged_BEME <= BEME_30 ~ "L",
      lagged_BEME <= BEME_70 ~ "N",
      TRUE ~ "H"
    ),
    FF_portfolio_type = factor(
      paste0(size_group, BEME_group),
      levels = c("SL", "SN", "SH", "BL", "BN", "BH")
    )
  ) |>
  select(
    year, firm_ID, lagged_ME, lagged_BEME,
    ME_rank10, BEME_rank10,
    size_group, BEME_group, FF_portfolio_type
  )

tbl_portfolio_membership_counts <- tbl_formation |>
  count(year, FF_portfolio_type, name = "firms") |>
  complete(
    year,
    FF_portfolio_type,
    fill = list(firms = 0)
  ) |>
  arrange(year, FF_portfolio_type)

tbl_empty_portfolios <- tbl_portfolio_membership_counts |>
  filter(firms == 0)

if (nrow(tbl_empty_portfolios) > 0) {
  stop("      2x3      ")
}

```



```

show_table_LFL(tbl_formation_breakpoints, n = Inf)
#> # A tibble: 5 x 5
#>   year firms size_median      BEME_30      BEME_70
#>   <int> <int>      <dbl>      <dbl>      <dbl>
#> 1  2016  1241 24564654000 0.00000076171 0.0000014304
#> 2  2017  1270 25133528500 0.00000070949 0.0000013738
#> 3  2018  1268 28268839000 0.00000061518 0.0000012038
#> 4  2019  1304 23853596500 0.00000073364 0.0000014045
#> 5  2020  1322 31878381000 0.00000057440 0.0000011304
show_table_LFL(tbl_portfolio_membership_counts, n = Inf)
#> # A tibble: 30 x 3
#>   year FF_portfolio_type firms
#>   <int> <fct>      <int>
#> 1  2016 SL          140
#> 2  2016 SN          210
#> 3  2016 SH          271
#> 4  2016 BL          233
#> 5  2016 BN          286
#> 6  2016 BH          101
#> 7  2017 SL          141
#> 8  2017 SN          224
#> 9  2017 SH          270
#> 10 2017 BL          240
#> 11 2017 BN          284
#> 12 2017 BH          111
#> 13 2018 SL          152
#> 14 2018 SN          224
#> 15 2018 SH          258
#> 16 2018 BL          229
#> 17 2018 BN          282
#> 18 2018 BH          123
#> 19 2019 SL          167
#> 20 2019 SN          225
#> 21 2019 SH          260
#> 22 2019 BL          224
#> 23 2019 BN          297
#> 24 2019 BH          131
#> 25 2020 SL          173
#> 26 2020 SN          248
#> 27 2020 SH          240
#> 28 2020 BL          224
#> 29 2020 BN          280

```

6.4 4.4

6.4.1 4.4.1 10

10

```
tbl_size_decile_returns <- tbl_monthly |>
  inner_join(
    tbl_formation |>
      select(year, firm_ID, lagged_ME, ME_rank10),
    by = c("year", "firm_ID")
  ) |>
  group_by(
    month_ID, year, month, month_date, ME_rank10
  ) |>
  summarise(
    R_F = compute_median_safe(R_F),
    R_equal = compute_mean_safe(R),
    R_value = compute_weighted_mean_safe(R, lagged_ME),
    firms = n_distinct(firm_ID),
    valid_return_firms = sum(is.finite(R)),
    .groups = "drop"
  ) |>
  mutate(
    Re_equal = R_equal - R_F,
    Re_value = R_value - R_F
  ) |>
  arrange(month_ID, ME_rank10)

tbl_size_decile_coverage <- tbl_size_decile_returns |>
  group_by(month_ID) |>
  summarise(
    portfolios = n_distinct(ME_rank10),
    .groups = "drop"
  ) |>
  filter(portfolios != 10)

if (nrow(tbl_size_decile_coverage) > 0) {
  stop("10")
}
```

```

}

tbl_size_decile_summary <- tbl_size_decile_returns |>
  group_by(ME_rank10) |>
  summarise(
    months = n(),
    mean_equal_excess_return = mean(Re_equal, na.rm = TRUE),
    mean_value_excess_return = mean(Re_value, na.rm = TRUE),
    value_return_volatility = sd(Re_value, na.rm = TRUE),
    .groups = "drop"
  ) |>
  arrange(ME_rank10)

show_table_LFL(tbl_size_decile_summary, n = Inf)
#> # A tibble: 10 x 5
#>   ME_rank10 months mean_equal_excess_return mean_value_excess_return
#>   <int>   <int>          <dbl>          <dbl>
#> 1         1     60          0.014931          0.015119
#> 2         2     60          0.013581          0.013512
#> 3         3     60          0.015247          0.015316
#> 4         4     60          0.013073          0.012958
#> 5         5     60          0.010989          0.010910
#> 6         6     60          0.010277          0.010388
#> 7         7     60          0.0067439         0.0065837
#> 8         8     60          0.0051647         0.0054430
#> 9         9     60          0.0046434         0.0045278
#> 10        10     60          0.0037215         0.0029811
#>   value_return_volatility
#>   <dbl>
#> 1      0.042085
#> 2      0.040856
#> 3      0.040102
#> 4      0.041626
#> 5      0.041165
#> 6      0.040414
#> 7      0.041338
#> 8      0.041428
#> 9      0.044503
#> 10     0.042314

```

```

plot_size_decile <- tbl_size_decile_summary |>
  select(

```

```

    ME_rank10,
    equal_weighted = mean_equal_excess_return,
    value_weighted = mean_value_excess_return
  ) |>
  pivot_longer(
    cols = c(equal_weighted, value_weighted),
    names_to = "weighting",
    values_to = "mean_excess_return"
  ) |>
  mutate(ME_rank10 = factor(ME_rank10, levels = 1:10)) |>
  ggplot(
    aes(
      x = ME_rank10,
      y = mean_excess_return,
      fill = weighting
    )
  ) +
  geom_col(position = "dodge") +
  geom_hline(yintercept = 0, linetype = "dotted") +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = "Size Decile",
    y = "Mean Monthly Excess Return",
    fill = "Weighting",
    title = "Size-Sorted Portfolio Returns"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_size_decile))

```



6.4.2 4.4.2 10

```
tbl_beme_decile_returns <- tbl_monthly |>
  inner_join(
    tbl_formation |>
      select(year, firm_ID, lagged_ME, BEME_rank10),
    by = c("year", "firm_ID")
  ) |>
  group_by(
    month_ID, year, month, month_date, BEME_rank10
  ) |>
  summarise(
    R_F = compute_median_safe(R_F),
    R_equal = compute_mean_safe(R),
    R_value = compute_weighted_mean_safe(R, lagged_ME),
    firms = n_distinct(firm_ID),
    valid_return_firms = sum(is.finite(R)),
    .groups = "drop"
  ) |>
```

```

mutate(
  Re_equal = R_equal - R_F,
  Re_value = R_value - R_F
) |>
arrange(month_ID, BEME_rank10)

tbl_beme_decile_summary <- tbl_beme_decile_returns |>
  group_by(BEME_rank10) |>
  summarise(
    months = n(),
    mean_equal_excess_return = mean(Re_equal, na.rm = TRUE),
    mean_value_excess_return = mean(Re_value, na.rm = TRUE),
    value_return_volatility = sd(Re_value, na.rm = TRUE),
    .groups = "drop"
  ) |>
  arrange(BEME_rank10)

show_table_LFL(tbl_beme_decile_summary, n = Inf)
#> # A tibble: 10 x 5
#>   BEME_rank10 months mean_equal_excess_return mean_value_excess_return
#>   <int>    <int>          <dbl>                <dbl>
#> 1         1     60          0.0069080            -0.0012722
#> 2         2     60          0.0057059             0.0024925
#> 3         3     60          0.0043601             0.0021706
#> 4         4     60          0.0090970             0.0034693
#> 5         5     60          0.0099654             0.0085384
#> 6         6     60          0.010433              0.0055909
#> 7         7     60          0.0086709             0.0085607
#> 8         8     60          0.012979             0.0088343
#> 9         9     60          0.016017             0.014657
#> 10        10     60          0.014389             0.0097273
#>   value_return_volatility
#>   <dbl>
#> 1      0.052048
#> 2      0.039802
#> 3      0.045255
#> 4      0.045192
#> 5      0.047651
#> 6      0.042627
#> 7      0.044756
#> 8      0.044844
#> 9      0.049615

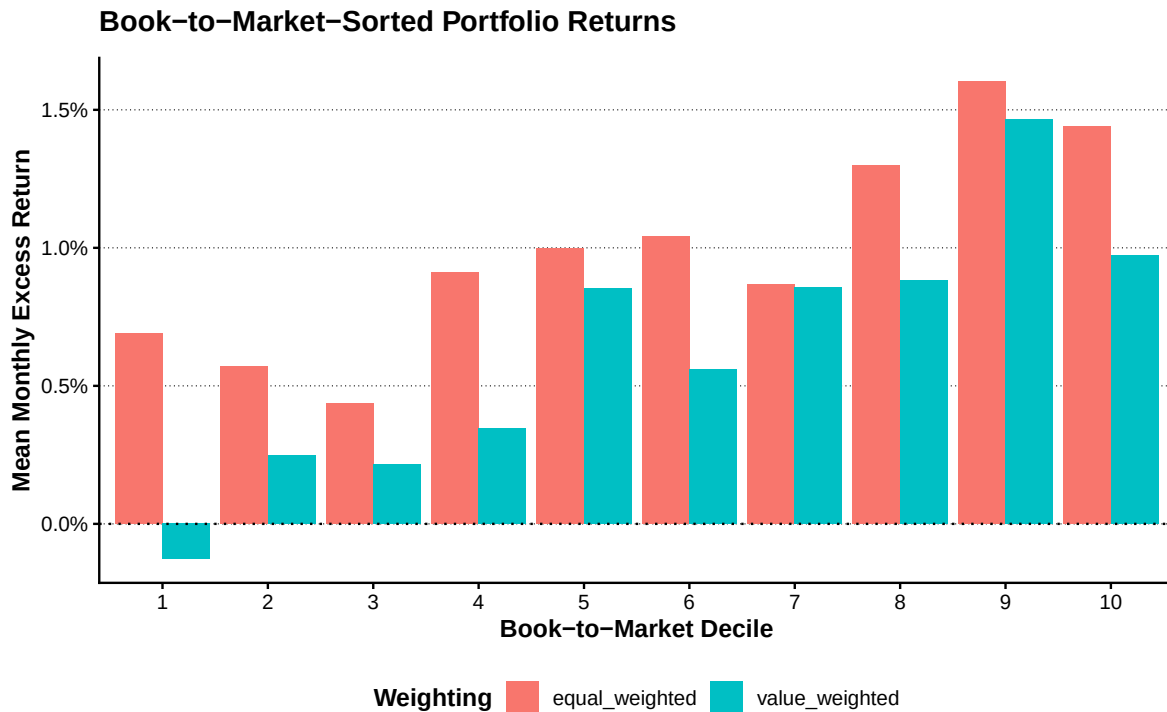
```

#> 10

0.043590

```
plot_beme_decile <- tbl_beme_decile_summary |>
  select(
    BEME_rank10,
    equal_weighted = mean_equal_excess_return,
    value_weighted = mean_value_excess_return
  ) |>
  pivot_longer(
    cols = c(equal_weighted, value_weighted),
    names_to = "weighting",
    values_to = "mean_excess_return"
  ) |>
  mutate(BEME_rank10 = factor(BEME_rank10, levels = 1:10)) |>
  ggplot(
    aes(
      x = BEME_rank10,
      y = mean_excess_return,
      fill = weighting
    )
  ) +
  geom_col(position = "dodge") +
  geom_hline(yintercept = 0, linetype = "dotted") +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = "Book-to-Market Decile",
    y = "Mean Monthly Excess Return",
    fill = "Weighting",
    title = "Book-to-Market-Sorted Portfolio Returns"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_beme_decile))
```



6.5 4.5 CAPM

6.5.1 4.5.1

CAPM p

$$R_{p,t}^e = \alpha_p + \beta_p R_{M,t}^e + \varepsilon_{p,t}$$

```
tbl_capm_data <- tbl_size_decile_returns |>
  select(month_ID, ME_rank10, Re = Re_value) |>
  inner_join(
    tbl_market_returns |>
      select(month_ID, R_Me),
    by = "month_ID"
  ) |>
  filter(is.finite(Re), is.finite(R_Me)) |>
  arrange(ME_rank10, month_ID)

tbl_capm_models <- tbl_capm_data |>
```



```

group_by(ME_rank10) |>
nest() |>
mutate(
  model = map(data, \(data) lm(Re ~ R_Me, data = data)),
  coefficients = map(model, broom::tidy),
  fit = map(model, broom::glance)
)

tbl_capm_results <- tbl_capm_models |>
select(ME_rank10, coefficients) |>
unnest(coefficients) |>
mutate(significance = compute_significance_stars(p.value)) |>
arrange(ME_rank10, term)

tbl_capm_fit <- tbl_capm_models |>
select(ME_rank10, fit) |>
unnest(fit) |>
select(
  ME_rank10, r.squared, adj.r.squared,
  sigma, statistic, p.value, nobs
) |>
arrange(ME_rank10)

tbl_capm_alpha <- tbl_capm_results |>
filter(term == "(Intercept)") |>
transmute(
  ME_rank10,
  tbl_capm_alpha = estimate,
  standard_error = std.error,
  statistic,
  p_value = p.value,
  significance
)

show_table_LFL(tbl_capm_results, n = Inf)
#> # A tibble: 20 x 7
#> # Groups:   ME_rank10 [10]
#>   ME_rank10 term          estimate std.error statistic    p.value
#>   <int> <chr>          <dbl>    <dbl>    <dbl>    <dbl>
#> 1     1 1 (Intercept)  0.012412  0.0041175   3.0144 3.8151e- 3
#> 2     2 1 R_Me        0.67325   0.099584   6.7607 7.3884e- 9
#> 3     3 2 (Intercept)  0.010641  0.0036808   2.8910 5.3962e- 3

```

```

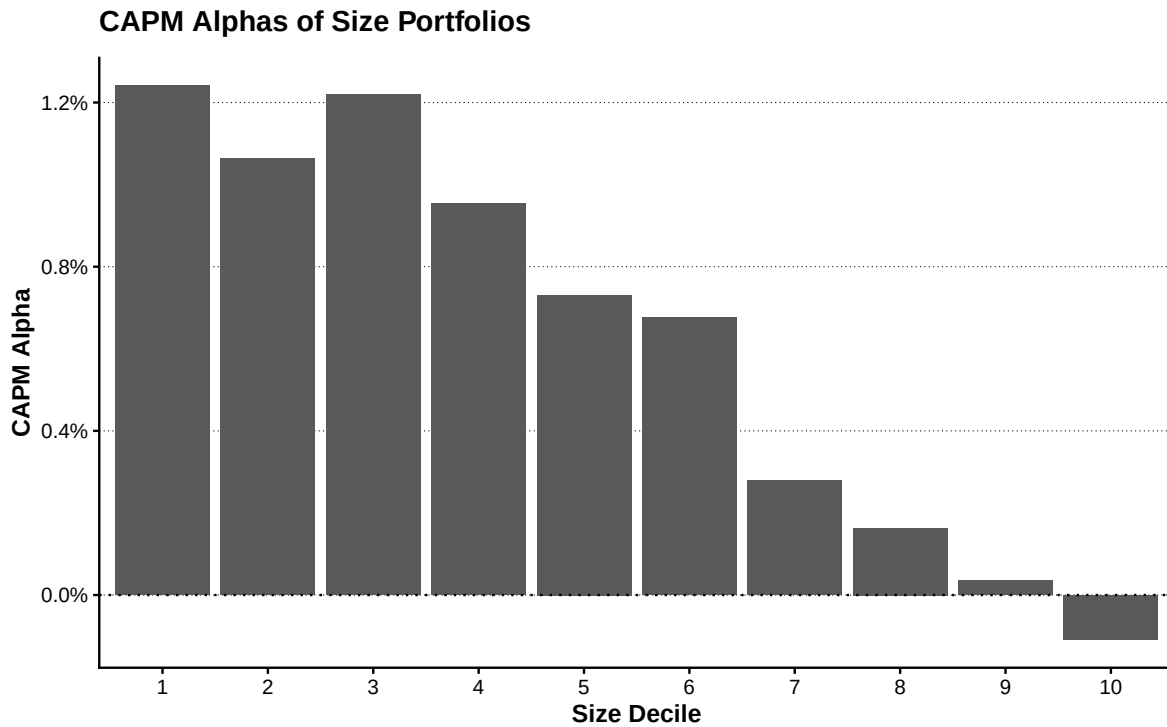
#> 4      2 R_Me      0.71390    0.089021    8.0195 5.6813e-11
#> 5      3 (Intercept) 0.012195    0.0031256    3.9016 2.5116e- 4
#> 6      3 R_Me      0.77614    0.075595   10.267 1.1583e-14
#> 7      4 (Intercept) 0.0095370    0.0028845    3.3062 1.6262e- 3
#> 8      4 R_Me      0.85081    0.069764   12.196 1.2013e-17
#> 9      5 (Intercept) 0.0073101    0.0023212    3.1493 2.5865e- 3
#> 10     5 R_Me      0.89511    0.056138   15.945 7.2596e-23
#> 11     6 (Intercept) 0.0067526    0.0019672    3.4325 1.1093e- 3
#> 12     6 R_Me      0.90394    0.047579   18.999 1.3994e-26
#> 13     7 (Intercept) 0.0027911    0.0017412    1.6030 1.1437e- 1
#> 14     7 R_Me      0.94309    0.042112   22.395 3.1166e-30
#> 15     8 (Intercept) 0.0016328    0.0017077    0.95613 3.4298e- 1
#> 16     8 R_Me      0.94747    0.041302   22.940 8.8761e-31
#> 17     9 (Intercept) 0.00035036    0.0014465    0.24221 8.0947e- 1
#> 18     9 R_Me      1.0388     0.034985   29.693 9.0177e-37
#> 19    10 (Intercept) -0.0010957    0.00059445   -1.8432 7.0415e- 2
#> 20    10 R_Me      1.0138     0.014377   70.512 6.5289e-58
#>      significance
#>      <chr>
#> 1  "***"
#> 2  "***"
#> 3  "***"
#> 4  "***"
#> 5  "***"
#> 6  "***"
#> 7  "***"
#> 8  "***"
#> 9  "***"
#> 10 "***"
#> 11 "***"
#> 12 "***"
#> 13 ""
#> 14 "***"
#> 15 ""
#> 16 "***"
#> 17 ""
#> 18 "***"
#> 19 "*"
#> 20 "***"
show_table_LFL(tbl_capm_fit, n = Inf)
#> # A tibble: 10 x 7
#> # Groups:   ME_rank10 [10]

```

#>	ME_rank10	r.squared	adj.r.squared	sigma	statistic	p.value	nobs	
#>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<int>	
#>	1	1	0.44073	0.43109	0.031743	45.706	7.3884e- 9	60
#>	2	2	0.52580	0.51763	0.028376	64.312	5.6813e-11	60
#>	3	3	0.64507	0.63895	0.024096	105.41	1.1583e-14	60
#>	4	4	0.71944	0.71461	0.022238	148.73	1.2013e-17	60
#>	5	5	0.81424	0.81104	0.017894	254.23	7.2596e-23	60
#>	6	6	0.86156	0.85918	0.015166	360.96	1.3994e-26	60
#>	7	7	0.89634	0.89456	0.013423	501.54	3.1166e-30	60
#>	8	8	0.90073	0.89902	0.013165	526.25	8.8761e-31	60
#>	9	9	0.93827	0.93721	0.011152	881.65	9.0177e-37	60
#>	10	10	0.98847	0.98827	0.0045827	4972.0	6.5289e-58	60

```
plot_capm_alpha <- tbl_capm_alpha |>
  mutate(ME_rank10 = factor(ME_rank10, levels = 1:10)) |>
  ggplot(aes(x = ME_rank10, y = tbl_capm_alpha)) +
  geom_col() +
  geom_hline(yintercept = 0, linetype = "dotted") +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = "Size Decile",
    y = "CAPM Alpha",
    title = "CAPM Alphas of Size Portfolios"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_capm_alpha))
```



6.5.2 4.5.2

```
tbl_capm_beta <- tbl_capm_results |>
  filter(term == "R_Me") |>
  transmute(
    ME_rank10,
    tbl_capm_beta = estimate,
    beta_p_value = p.value
  )

tbl_security_market_line <- tbl_size_decile_summary |>
  select(
    ME_rank10,
    mean_excess_return = mean_value_excess_return
  ) |>
  left_join(tbl_capm_beta, by = "ME_rank10")

vec_mean_market_excess_return <- mean(
  tbl_market_returns$R_Me,
```

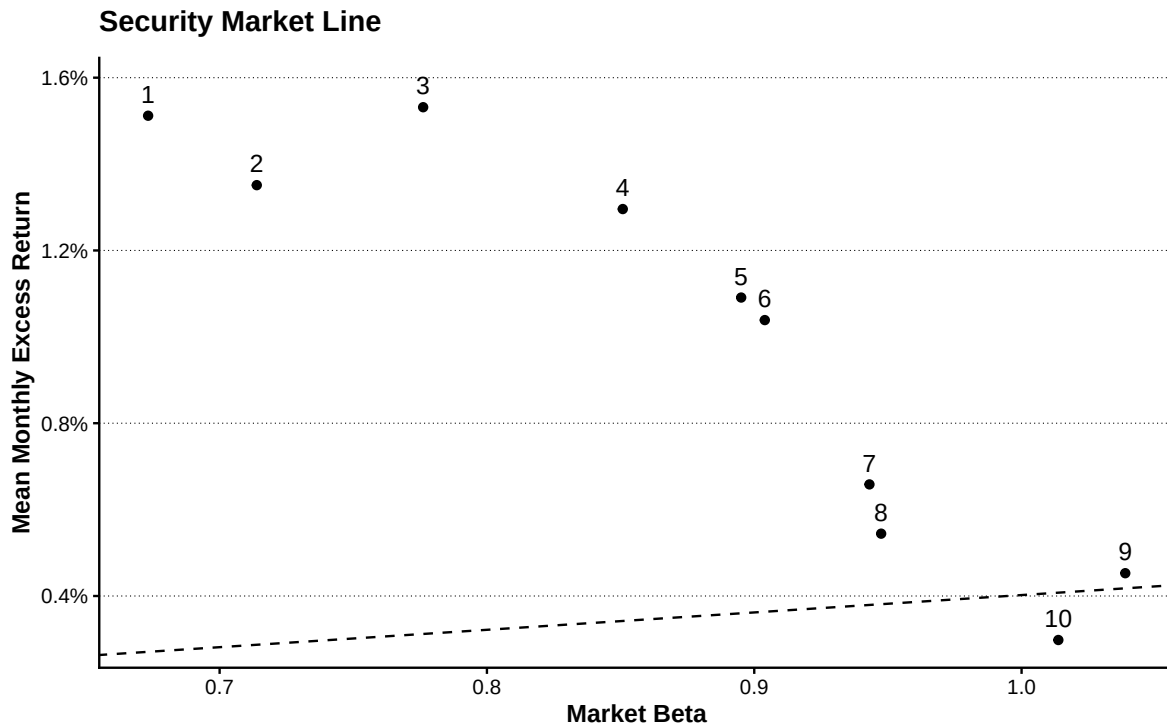
```

    na.rm = TRUE
  )

plot_security_market_line <- tbl_security_market_line |>
  ggplot(
    aes(
      x = tbl_capm_beta,
      y = mean_excess_return
    )
  ) +
  geom_point() +
  geom_text(aes(label = ME_rank10), nudge_y = 0.0005) +
  geom_abline(
    intercept = 0,
    slope = vec_mean_market_excess_return,
    linetype = "dashed"
  ) +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = "Market Beta",
    y = "Mean Monthly Excess Return",
    title = "Security Market Line"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_security_market_line))

```



6.6 4.6 2×3 SMB HML

6.6.1 4.6.1 Size–BE/ME

Small Big 2 Low Neutral High 3

SL	Small	Low
SN	Small	Neutral
SH	Small	High
BL	Big	Low
BN	Big	Neutral
BH	Big	High

```
tbl_ff_portfolio_returns <- tbl_monthly |>
  inner_join(
    tbl_formation |>
      select(
```

```

      year, firm_ID, lagged_ME,
      size_group, BEME_group, FF_portfolio_type
    ),
    by = c("year", "firm_ID")
  ) |>
  filter(!is.na(FF_portfolio_type)) |>
  group_by(
    month_ID, year, month, month_date,
    FF_portfolio_type, size_group, BEME_group
  ) |>
  summarise(
    R_F = compute_median_safe(R_F),
    R = compute_weighted_mean_safe(R, lagged_ME),
    firms = n_distinct(firm_ID),
    valid_return_firms = sum(is.finite(R)),
    .groups = "drop"
  ) |>
  mutate(Re = R - R_F) |>
  arrange(month_ID, FF_portfolio_type)

tbl_ff_portfolio_coverage <- tbl_ff_portfolio_returns |>
  group_by(month_ID) |>
  summarise(
    portfolios = n_distinct(FF_portfolio_type),
    .groups = "drop"
  ) |>
  filter(portfolios != 6)

if (nrow(tbl_ff_portfolio_coverage) > 0) {
  stop("6 Size--BE/ME      ")
}

tbl_ff_portfolio_summary <- tbl_ff_portfolio_returns |>
  group_by(
    FF_portfolio_type,
    size_group,
    BEME_group
  ) |>
  summarise(
    months = n(),
    mean_return = mean(R),
    mean_excess_return = mean(Re),

```

```

    volatility = sd(R),
    .groups = "drop"
  ) |>
  arrange(FF_portfolio_type)

show_table_LFL(tbl_ff_portfolio_summary, n = Inf)
#> # A tibble: 6 x 7
#>   FF_portfolio_type size_group BEME_group months mean_return mean_excess_return
#>   <fct>             <chr>      <chr>      <int>      <dbl>          <dbl>
#> 1 SL               S         L          60  0.010056      0.0099773
#> 2 SN               S         N          60  0.013474      0.013395
#> 3 SH               S         H          60  0.014002      0.013923
#> 4 BL               B         L          60  0.00081235    0.00073351
#> 5 BN               B         N          60  0.0063080     0.0062291
#> 6 BH               B         H          60  0.010766      0.010687
#>   volatility
#>   <dbl>
#> 1  0.044267
#> 2  0.041252
#> 3  0.040505
#> 4  0.043049
#> 5  0.042621
#> 6  0.046846

```

```

plot_ff_portfolio <- tbl_ff_portfolio_summary |>
  ggplot(
    aes(
      x = BEME_group,
      y = mean_excess_return,
      fill = size_group
    )
  ) +
  geom_col(position = "dodge") +
  geom_text(
    aes(label = FF_portfolio_type, group = size_group),
    position = position_dodge(width = 0.9),
    vjust = -0.5
  ) +
  geom_hline(yintercept = 0, linetype = "dotted") +
  scale_y_continuous(
    labels = scales::label_percent(),
    expand = expansion(mult = c(0.05, 0.15))
  )

```

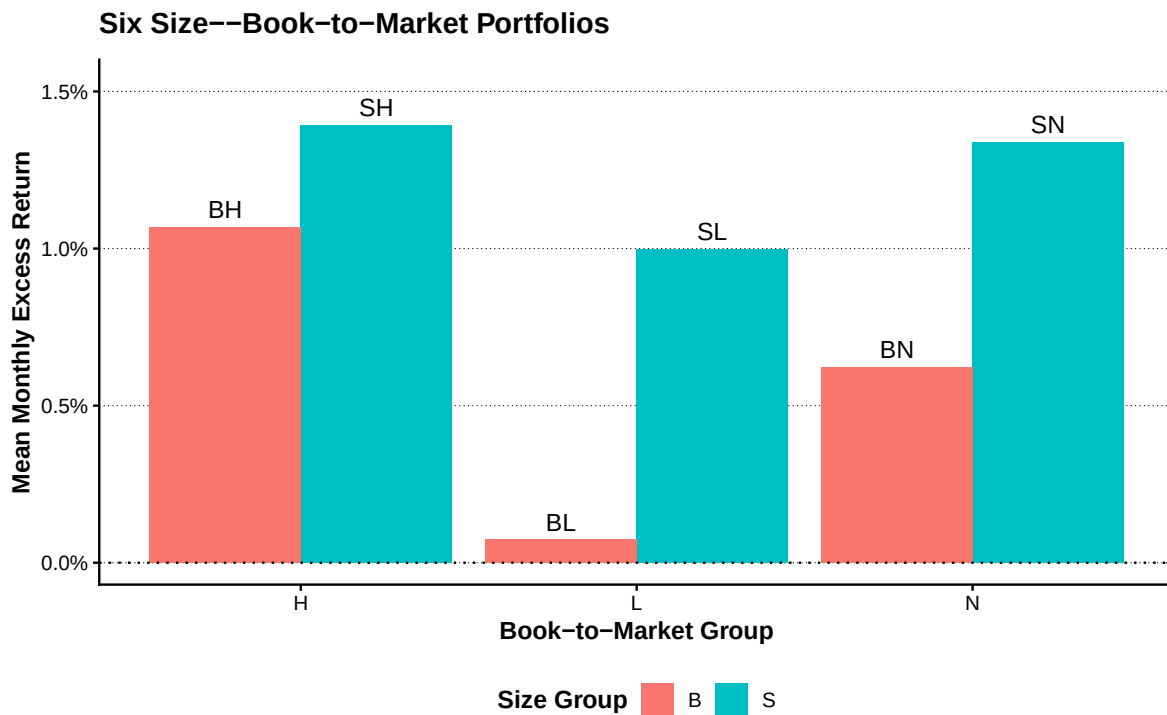


```

) +
labs(
  x = "Book-to-Market Group",
  y = "Mean Monthly Excess Return",
  fill = "Size Group",
  title = "Six Size--Book-to-Market Portfolios"
)

show_plot_LFL(add_lagrange_theme_LFL(plot_ff_portfolio))

```



```

plot_ff_cumulative <- tbl_ff_portfolio_returns |>
  group_by(FF_portfolio_type) |>
  arrange(month_ID, .by_group = TRUE) |>
  mutate(cumulative_gross_return = cumprod(1 + R)) |>
  ungroup() |>
  ggplot(
    aes(
      x = month_date,
      y = cumulative_gross_return,
      linetype = FF_portfolio_type
    )
  )

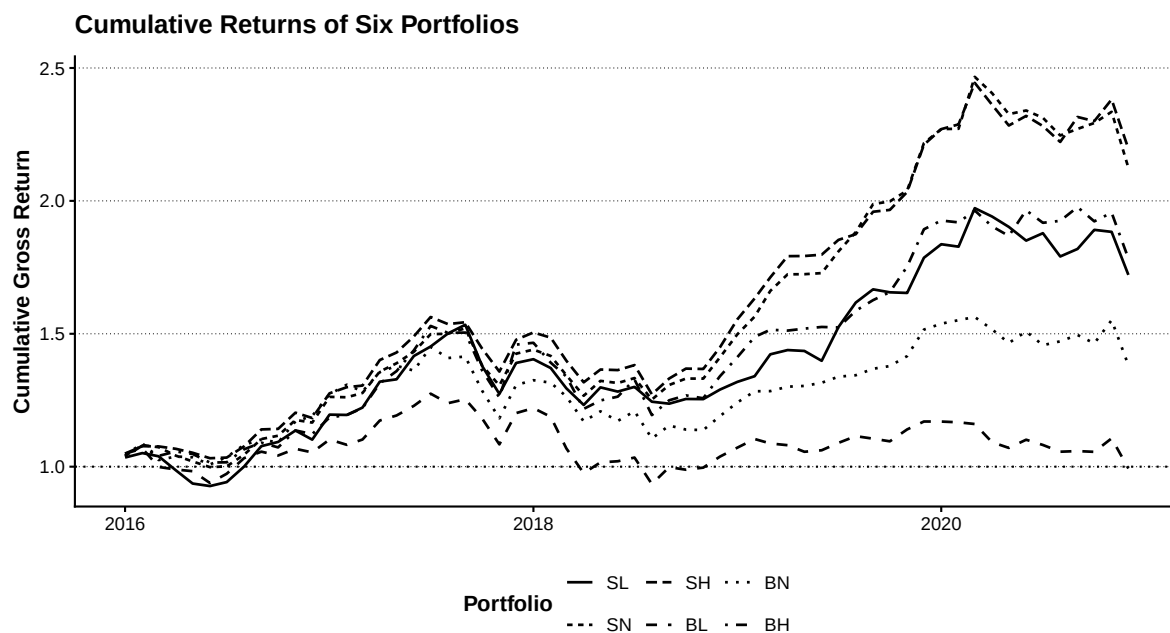
```

```

    )
  ) +
  geom_line(linewidth = 0.7) +
  geom_hline(yintercept = 1, linetype = "dotted") +
  labs(
    x = NULL,
    y = "Cumulative Gross Return",
    linetype = "Portfolio",
    title = "Cumulative Returns of Six Portfolios"
  )
)

show_plot_LFL(add_lagrange_theme_LFL(plot_ff_cumulative))

```



6.6.2 4.6.2 SMB HML

SMB Small

Big

$$SMB_t = \frac{SL_t + SN_t + SH_t}{3} - \frac{BL_t + BN_t + BH_t}{3}$$

HML High

Low

$$HML_t = \frac{SH_t + BH_t}{2} - \frac{SL_t + BL_t}{2}$$

```
tbl_ff_portfolio_wide <- tbl_ff_portfolio_returns |>
  select(
    month_ID, year, month, month_date,
    FF_portfolio_type, R
  ) |>
  pivot_wider(
    names_from = FF_portfolio_type,
    values_from = R
  ) |>
  arrange(month_ID)

tbl_smb_hml <- tbl_ff_portfolio_wide |>
  mutate(
    SMB = (SL + SN + SH) / 3 - (BL + BN + BH) / 3,
    HML = (SH + BH) / 2 - (SL + BL) / 2
  ) |>
  select(
    month_ID, year, month, month_date,
    SL, SN, SH, BL, BN, BH, SMB, HML
  )

tbl_factor <- tbl_market_returns |>
  select(
    month_ID, year, month, month_date,
    R_F, R_M, R_Me,
    market_return_coverage = return_coverage
  ) |>
  inner_join(
    tbl_smb_hml,
    by = c("month_ID", "year", "month", "month_date")
  ) |>
  arrange(month_ID)

tbl_invalid_factor_rows <- tbl_factor |>
  filter(
    !if_all(
      c(R_F, R_M, R_Me, SMB, HML),
      is.finite
    )
  )
```

```

)

if (nrow(tbl_invalid_factor_rows) > 0) {
  stop(" ")
}

show_table_LFL(tbl_factor)
#> # A tibble: 60 x 16
#>   month_ID year month month_date      R_F      R_M      R_Me
#>   <int> <int> <int> <date>      <dbl>      <dbl>      <dbl>
#> 1      13  2016     1 2016-01-01 0.000082029 0.045835 0.045753
#> 2      14  2016     2 2016-02-01 0.000075959 0.028423 0.028347
#> 3      15  2016     3 2016-03-01 0.000075959 -0.057976 -0.058052
#> 4      16  2016     4 2016-04-01 0.000082029 -0.0010718 -0.0011539
#> 5      17  2016     5 2016-05-01 0.000082029 -0.0064160 -0.0064980
#> 6      18  2016     6 2016-06-01 0.000082029 -0.037075 -0.037157
#> 7      19  2016     7 2016-07-01 0.000082029 0.029853 0.029771
#> 8      20  2016     8 2016-08-01 0.000082029 0.053757 0.053675
#> 9      21  2016     9 2016-09-01 0.000082029 0.023650 0.023568
#> 10     22  2016    10 2016-10-01 0.000075959 -0.012053 -0.012129
#>   market_return_coverage      SL      SN      SH      BL      BN
#>   <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1      1      0.034424 0.040738 0.049300 0.048792 0.043035
#> 2      1      0.016158 0.039043 0.026503 0.018793 0.043267
#> 3      1 -0.011443 -0.0075343 -0.00082137 -0.065067 -0.058172
#> 4      1 -0.050894 -0.029672 -0.0084329 -0.0096066 0.013125
#> 5      1 -0.050419 -0.020829 -0.013882 -0.0064587 -0.0031242
#> 6      1 -0.010094 -0.021100 -0.019579 -0.045272 -0.025807
#> 7      1 0.016136 0.0030885 0.0024072 0.037354 0.027623
#> 8      1 0.061823 0.043624 0.040794 0.060068 0.044383
#> 9      1 0.076890 0.056521 0.059240 0.023654 0.018573
#> 10     1 0.014770 0.011308 0.0017272 -0.013614 -0.010340
#>   BH      SMB      HML
#>   <dbl>      <dbl>      <dbl>
#> 1 0.037868 -0.0017441 0.0019763
#> 2 0.039974 -0.0067772 0.015763
#> 3 -0.035989 0.046477 0.019850
#> 4 0.016129 -0.036215 0.034098
#> 5 -0.0090124 -0.022178 0.016992
#> 6 -0.030949 0.017085 0.0024194
#> 7 0.0011262 -0.014824 -0.024978
#> 8 0.048092 -0.0021008 -0.016503

```

```
#> 9  0.023533  0.042297 -0.0088858
#> 10 -0.016441  0.022733 -0.0079348
#> # i 50 more rows
```

6.6.3 4.6.3

```
vec_factor_names <- c("R_Me", "SMB", "HML")

tbl_factor_summary <- vec_factor_names |>
  map_dfr(
    \(factor_name) {
      values <- tbl_factor[[factor_name]]
      finite_values <- values[is.finite(values)]

      tibble(
        factor = factor_name,
        observations = length(finite_values),
        mean = mean(finite_values),
        standard_deviation = sd(finite_values),
        minimum = min(finite_values),
        median = median(finite_values),
        maximum = max(finite_values),
        t_value = mean(finite_values) /
          (sd(finite_values) / sqrt(length(finite_values)))
      )
    }
  )

tbl_factor_correlation <- tbl_factor |>
  select(R_Me, SMB, HML) |>
  cor(use = "complete.obs")

show_table_LFL(tbl_factor_summary, n = Inf)
#> # A tibble: 3 x 8
#>   factor observations      mean standard_deviation  minimum    median  maximum
#>   <chr>          <int>    <dbl>          <dbl>    <dbl>    <dbl>    <dbl>
#> 1 R_Me             60 0.0040215          0.041498 -0.10252 0.0060055 0.11097
#> 2 SMB              60 0.0065488          0.022857 -0.038447 0.0055499 0.069390
#> 3 HML              60 0.0069496          0.018058 -0.043572 0.0089560 0.035442
#>   t_value
```

```

#>      <dbl>
#> 1 0.75064
#> 2 2.2193
#> 3 2.9811
tbl_factor_correlation
#>      R_Me      SMB      HML
#> R_Me  1.0000000 -0.3431883  0.1702499
#> SMB   -0.3431883  1.0000000 -0.3806883
#> HML    0.1702499 -0.3806883  1.0000000

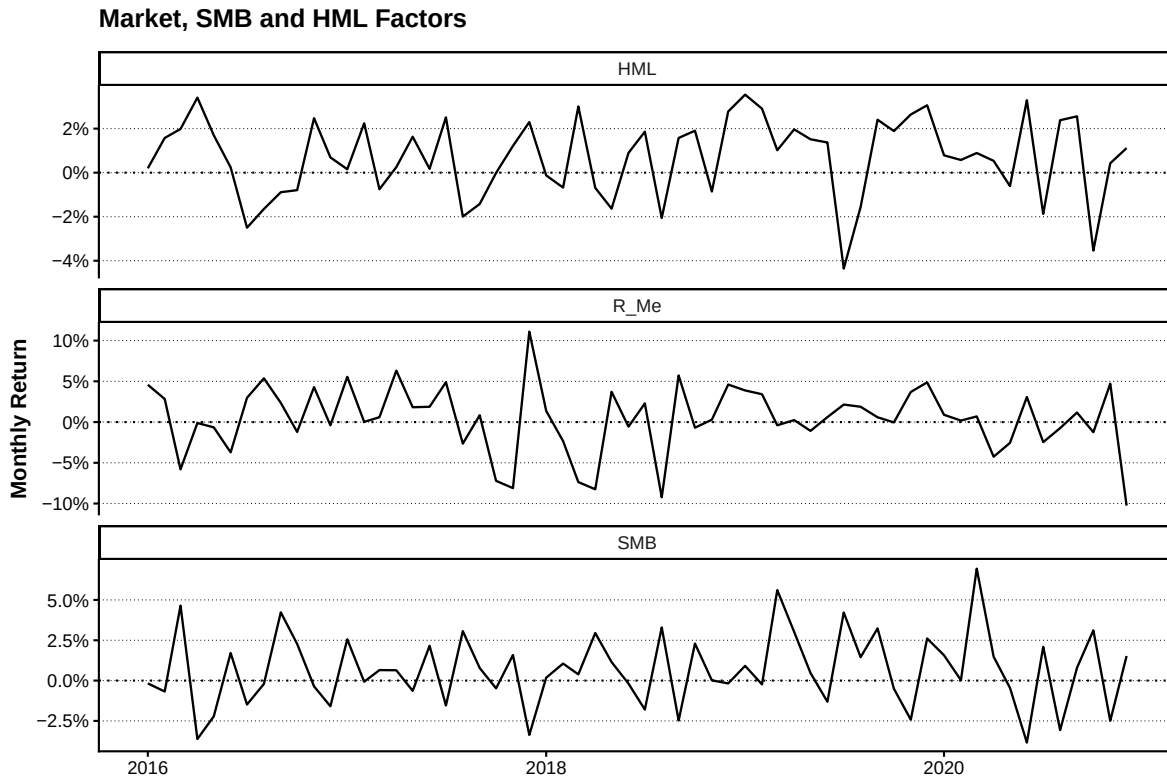
```

```

plot_factor_time_series <- tbl_factor |>
  select(month_date, R_Me, SMB, HML) |>
  pivot_longer(
    cols = c(R_Me, SMB, HML),
    names_to = "factor",
    values_to = "return"
  ) |>
  ggplot(aes(x = month_date, y = return)) +
  geom_line(linewidth = 0.6) +
  geom_hline(yintercept = 0, linetype = "dotted") +
  facet_wrap(
    vars(factor),
    ncol = 1,
    scales = "free_y"
  ) +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = NULL,
    y = "Monthly Return",
    title = "Market, SMB and HML Factors"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_factor_time_series))

```



6.7 4.7 FF3 CAPM

6.7.1 4.7.1 FF3

Fama–French 3

SMB HML

$$R_{p,t}^e = \alpha_p + \beta_{M,p} R_{M,t}^e + \beta_{SMB,p} SMB_t + \beta_{HML,p} HML_t + \varepsilon_{p,t}$$

```
tbl_ff3_data <- tbl_size_decile_returns |>
  select(month_ID, ME_rank10, Re = Re_value) |>
  inner_join(
    tbl_factor |>
      select(month_ID, R_Me, SMB, HML),
    by = "month_ID"
  ) |>
  filter(
    if_all(
```

```

      c(Re, R_Me, SMB, HML),
      is.finite
    )
  ) |>
  arrange(ME_rank10, month_ID)

tbl_ff3_models <- tbl_ff3_data |>
  group_by(ME_rank10) |>
  nest() |>
  mutate(
    model = map(
      data,
      \(data) lm(Re ~ R_Me + SMB + HML, data = data)
    ),
    coefficients = map(model, broom::tidy),
    fit = map(model, broom::glance)
  )

tbl_ff3_results <- tbl_ff3_models |>
  select(ME_rank10, coefficients) |>
  unnest(coefficients) |>
  mutate(significance = compute_significance_stars(p.value)) |>
  arrange(ME_rank10, term)

tbl_ff3_fit <- tbl_ff3_models |>
  select(ME_rank10, fit) |>
  unnest(fit) |>
  select(
    ME_rank10, r.squared, adj.r.squared,
    sigma, statistic, p.value, nobs
  ) |>
  arrange(ME_rank10)

tbl_ff3_alpha <- tbl_ff3_results |>
  filter(term == "(Intercept)") |>
  transmute(
    ME_rank10,
    tbl_ff3_alpha = estimate,
    standard_error = std.error,
    statistic,
    p_value = p.value,
    significance
  )

```


)

```
show_table_LFL(tbl_ff3_results, n = Inf)
```

```
#> # A tibble: 40 x 7
```

```
#> # Groups:   ME_rank10 [10]
```

#>	ME_rank10	term	estimate	std.error	statistic	p.value
#>	<int>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
#> 1	1	(Intercept)	0.0022160	0.0019246	1.1514	2.5446e- 1
#> 2	1	HML	0.037314	0.098061	0.38052	7.0500e- 1
#> 3	1	R_Me	0.92776	0.042009	22.085	2.5525e-29
#> 4	1	SMB	1.3610	0.081276	16.746	1.8226e-23
#> 5	2	(Intercept)	0.0012670	0.0015741	0.80490	4.2428e- 1
#> 6	2	HML	0.048745	0.080200	0.60779	5.4578e- 1
#> 7	2	R_Me	0.94434	0.034357	27.486	3.3917e-34
#> 8	2	SMB	1.2382	0.066472	18.627	1.1450e-25
#> 9	3	(Intercept)	0.0024754	0.0014256	1.7364	8.7992e- 2
#> 10	3	HML	0.26825	0.072634	3.6931	5.0419e- 4
#> 11	3	R_Me	0.96149	0.031116	30.900	7.1830e-37
#> 12	3	SMB	1.0857	0.060201	18.034	5.4337e-25
#> 13	4	(Intercept)	-0.00088852	0.00085856	-1.0349	3.0517e- 1
#> 14	4	HML	0.40182	0.043744	9.1858	9.0449e-13
#> 15	4	R_Me	1.0215	0.018740	54.513	3.2219e-50
#> 16	4	SMB	1.0607	0.036256	29.256	1.2872e-35
#> 17	5	(Intercept)	-0.00088220	0.00091786	-0.96115	3.4061e- 1
#> 18	5	HML	0.31739	0.046765	6.7869	7.7807e- 9
#> 19	5	R_Me	1.0289	0.020034	51.357	8.5258e-49
#> 20	5	SMB	0.83200	0.038760	21.466	1.0638e-28
#> 21	6	(Intercept)	0.0012087	0.0013152	0.91901	3.6203e- 1
#> 22	6	HML	0.15817	0.067009	2.3605	2.1759e- 2
#> 23	6	R_Me	1.0084	0.028706	35.128	7.7326e-40
#> 24	6	SMB	0.61456	0.055539	11.065	1.0274e-15
#> 25	7	(Intercept)	-0.0019158	0.0011588	-1.6532	1.0388e- 1
#> 26	7	HML	0.11287	0.059041	1.9117	6.1037e- 2
#> 27	7	R_Me	1.0370	0.025293	41.001	1.8493e-43
#> 28	7	SMB	0.54127	0.048935	11.061	1.0435e-15
#> 29	8	(Intercept)	0.0011876	0.0014388	0.82540	4.1264e- 1
#> 30	8	HML	-0.26686	0.073308	-3.6403	5.9543e- 4
#> 31	8	R_Me	1.0247	0.031405	32.627	3.9979e-38
#> 32	8	SMB	0.30377	0.060760	4.9995	5.9999e- 6
#> 33	9	(Intercept)	-0.0010787	0.0015851	-0.68056	4.9895e- 1
#> 34	9	HML	-0.013149	0.080760	-0.16281	8.7125e- 1
#> 35	9	R_Me	1.0790	0.034597	31.187	4.4005e-37

```

#> 36      9 SMB      0.20749  0.066936  3.0999  3.0278e- 3
#> 37     10 (Intercept) 0.00017429 0.00042384  0.41123 6.8247e- 1
#> 38     10 HML      -0.0028300 0.021595  -0.13105 8.9620e- 1
#> 39     10 R_Me      0.98160  0.0092510 106.11  2.9778e-66
#> 40     10 SMB      -0.17118  0.017898  -9.5641  2.2416e-13
#>      significance
#>      <chr>
#> 1  ""
#> 2  ""
#> 3  "***"
#> 4  "***"
#> 5  ""
#> 6  ""
#> 7  "***"
#> 8  "***"
#> 9  "*"
#> 10 "***"
#> 11 "***"
#> 12 "***"
#> 13 ""
#> 14 "***"
#> 15 "***"
#> 16 "***"
#> 17 ""
#> 18 "***"
#> 19 "***"
#> 20 "***"
#> 21 ""
#> 22 "**"
#> 23 "***"
#> 24 "***"
#> 25 ""
#> 26 "*"
#> 27 "***"
#> 28 "***"
#> 29 ""
#> 30 "***"
#> 31 "***"
#> 32 "***"
#> 33 ""
#> 34 ""
#> 35 "***"

```

```

#> 36 "***"
#> 37 ""
#> 38 ""
#> 39 "***"
#> 40 "***"
show_table_LFL(tbl_ff3_fit, n = Inf)
#> # A tibble: 10 x 7
#> # Groups:   ME_rank10 [10]
#>   ME_rank10 r.squared adj.r.squared   sigma statistic   p.value nobs
#>   <int>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl> <int>
#> 1         1    0.91540    0.91087 0.012564    201.99 5.3609e-30    60
#> 2         2    0.93996    0.93674 0.010276    292.24 3.6748e-34    60
#> 3         3    0.94888    0.94615 0.0093063    346.51 4.0778e-36    60
#> 4         4    0.98279    0.98187 0.0056047   1066.2 2.3878e-49    60
#> 5         5    0.97989    0.97881 0.0059918    909.61 1.8726e-47    60
#> 6         6    0.95716    0.95487 0.0085857    417.09 2.9086e-38    60
#> 7         7    0.96821    0.96651 0.0075647    568.60 6.8782e-42    60
#> 8         8    0.95121    0.94860 0.0093927    363.94 1.1072e-36    60
#> 9         9    0.94869    0.94594 0.010347    345.12 4.5371e-36    60
#> 10        10    0.99594    0.99572 0.0027668   4581.1 6.5209e-67    60

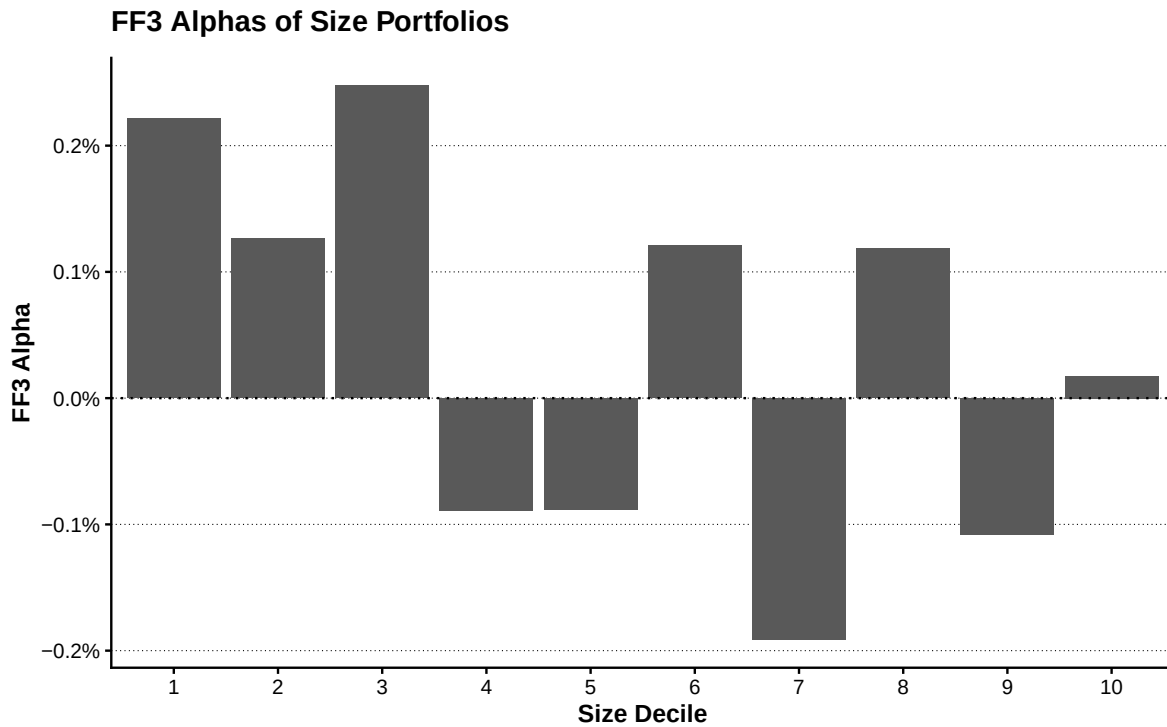
```

```

plot_ff3_alpha <- tbl_ff3_alpha |>
  mutate(ME_rank10 = factor(ME_rank10, levels = 1:10)) |>
  ggplot(aes(x = ME_rank10, y = tbl_ff3_alpha)) +
  geom_col() +
  geom_hline(yintercept = 0, linetype = "dotted") +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = "Size Decile",
    y = "FF3 Alpha",
    title = "FF3 Alphas of Size Portfolios"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_ff3_alpha))

```



6.7.2 4.7.2

```
tbl_model_alpha_comparison <- tbl_capm_alpha |>
  select(ME_rank10, tbl_capm_alpha) |>
  left_join(
    tbl_ff3_alpha |>
      select(ME_rank10, tbl_ff3_alpha),
    by = "ME_rank10"
  ) |>
  pivot_longer(
    cols = c(tbl_capm_alpha, tbl_ff3_alpha),
    names_to = "model",
    values_to = "alpha"
  )

tbl_model_fit_comparison <- tbl_capm_fit |>
  select(
    ME_rank10,
    CAPM_adjusted_R2 = adj.r.squared
  )
```

```

) |>
left_join(
  tbl_ff3_fit |>
    select(
      ME_rank10,
      FF3_adjusted_R2 = adj.r.squared
    ),
  by = "ME_rank10"
)

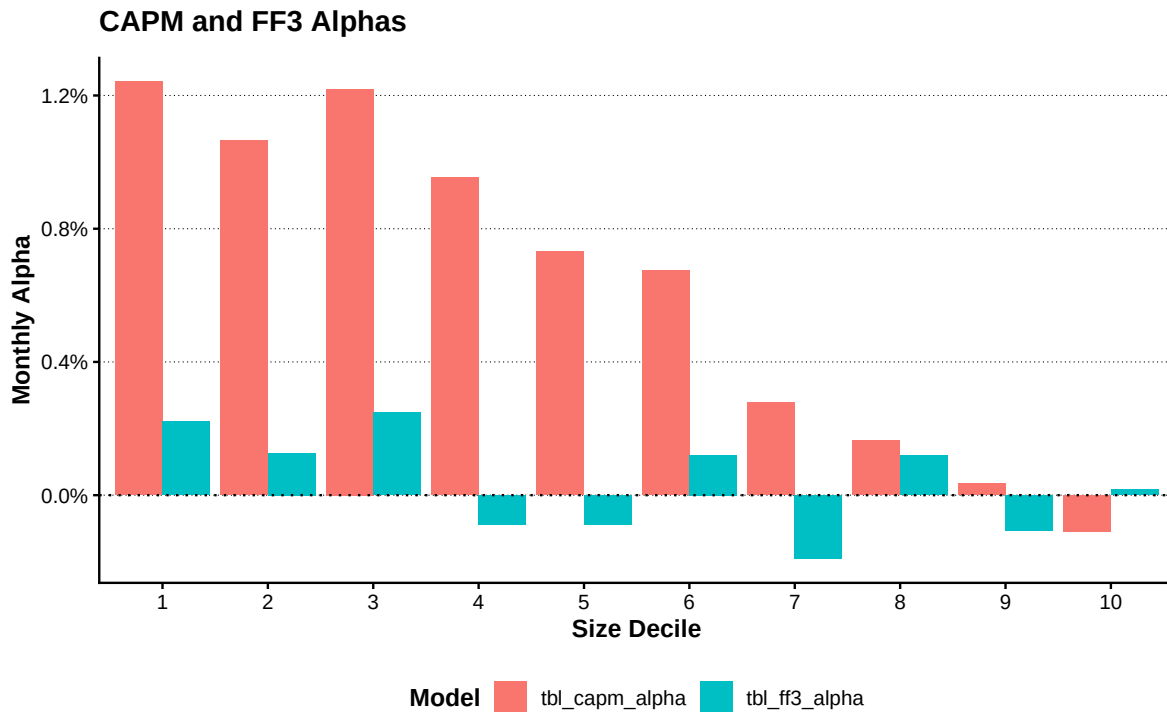
show_table_LFL(tbl_model_alpha_comparison, n = Inf)
#> # A tibble: 20 x 3
#> # Groups:   ME_rank10 [10]
#>   ME_rank10 model          alpha
#>   <int> <chr>          <dbl>
#> 1     1 tbl_capm_alpha 0.012412
#> 2     1 tbl_ff3_alpha 0.0022160
#> 3     2 tbl_capm_alpha 0.010641
#> 4     2 tbl_ff3_alpha 0.0012670
#> 5     3 tbl_capm_alpha 0.012195
#> 6     3 tbl_ff3_alpha 0.0024754
#> 7     4 tbl_capm_alpha 0.0095370
#> 8     4 tbl_ff3_alpha -0.00088852
#> 9     5 tbl_capm_alpha 0.0073101
#> 10    5 tbl_ff3_alpha -0.00088220
#> 11    6 tbl_capm_alpha 0.0067526
#> 12    6 tbl_ff3_alpha 0.0012087
#> 13    7 tbl_capm_alpha 0.0027911
#> 14    7 tbl_ff3_alpha -0.0019158
#> 15    8 tbl_capm_alpha 0.0016328
#> 16    8 tbl_ff3_alpha 0.0011876
#> 17    9 tbl_capm_alpha 0.00035036
#> 18    9 tbl_ff3_alpha -0.0010787
#> 19   10 tbl_capm_alpha -0.0010957
#> 20   10 tbl_ff3_alpha 0.00017429
show_table_LFL(tbl_model_fit_comparison, n = Inf)
#> # A tibble: 10 x 3
#> # Groups:   ME_rank10 [10]
#>   ME_rank10 CAPM_adjusted_R2 FF3_adjusted_R2
#>   <int>          <dbl>          <dbl>
#> 1     1          0.43109          0.91087
#> 2     2          0.51763          0.93674

```

#>	3	3	0.63895	0.94615
#>	4	4	0.71461	0.98187
#>	5	5	0.81104	0.97881
#>	6	6	0.85918	0.95487
#>	7	7	0.89456	0.96651
#>	8	8	0.89902	0.94860
#>	9	9	0.93721	0.94594
#>	10	10	0.98827	0.99572

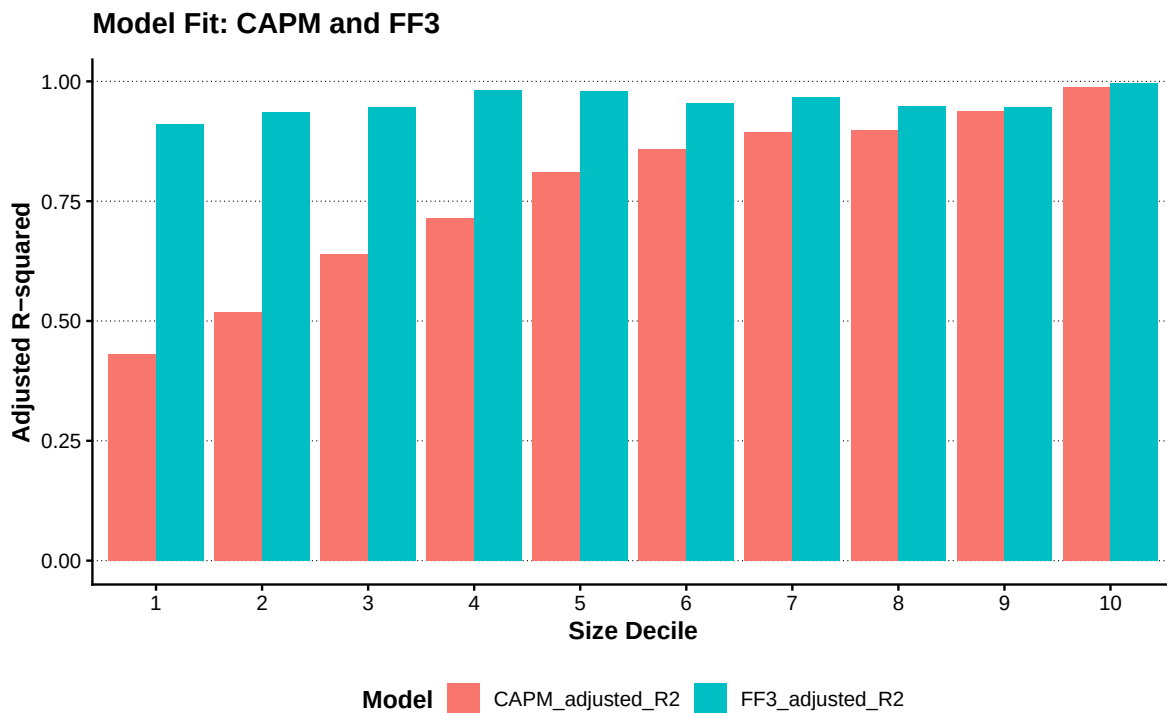
```
plot_model_alpha <- tbl_model_alpha_comparison |>
  mutate(ME_rank10 = factor(ME_rank10, levels = 1:10)) |>
  ggplot(
    aes(
      x = ME_rank10,
      y = alpha,
      fill = model
    )
  ) +
  geom_col(position = "dodge") +
  geom_hline(yintercept = 0, linetype = "dotted") +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = "Size Decile",
    y = "Monthly Alpha",
    fill = "Model",
    title = "CAPM and FF3 Alphas"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_model_alpha))
```



```
plot_model_fit <- tbl_model_fit_comparison |>
  pivot_longer(
    cols = c(CAPM_adjusted_R2, FF3_adjusted_R2),
    names_to = "model",
    values_to = "adjusted_R2"
  ) |>
  mutate(ME_rank10 = factor(ME_rank10, levels = 1:10)) |>
  ggplot(
    aes(
      x = ME_rank10,
      y = adjusted_R2,
      fill = model
    )
  ) +
  geom_col(position = "dodge") +
  labs(
    x = "Size Decile",
    y = "Adjusted R-squared",
    fill = "Model",
    title = "Model Fit: CAPM and FF3"
  )
```

```
show_plot_LFL(add_lagrange_theme_LFL(plot_model_fit))
```



6.8 4.8

```
5 data/derived/factors/fama_french_factors.parquet
```

```
tbl_factor_output <- tbl_factor |>
  select(
    month_ID, year, month, month_date,
    R_F, R_M, R_Me, SMB, HML,
    SL, SN, SH, BL, BN, BH,
    market_return_coverage
  )

vec_factor_output_path <-
  write_derived_parquet_file_LFL(
    tbl_factor_output,
    file.path(
```



```

      "factors",
      "fama_french_factors.parquet"
    )
  )

tbl_factor_output_check <-
  read_derived_parquet_file_LFL(
    file.path(
      "factors",
      "fama_french_factors.parquet"
    )
  )

stopifnot(
  nrow(tbl_factor_output_check) == nrow(tbl_factor_output),
  identical(names(tbl_factor_output_check), names(tbl_factor_output))
)

tbl_factor_output_summary <- tibble(
  output_file = vec_factor_output_path,
  rows = nrow(tbl_factor_output_check),
  columns = ncol(tbl_factor_output_check),
  first_month = min(tbl_factor_output_check$month_ID),
  last_month = max(tbl_factor_output_check$month_ID)
)

show_table_LFL(tbl_factor_output_summary)
#> # A tibble: 1 x 5
#>   output_file
#>   <chr>
#> 1 C:/AnalyticFin/Projects/LagrangeFinance/data/derived/factors/fama_french_fact~
#>   rows columns first_month last_month
#>   <int>   <int>      <int>      <int>
#> 1    60     16         13         72

```

6.9 4.9

		2×3	6	SMB HML
10	CAPM FF3			nest() purrr::map()

```
data/derived/factors/fama_french_factors.parquet
```

5

7 I-5

Fama–French 3

7.1 5.1

7.1.1 5.1.1

utility

```
tbl_monthly_raw <-  
  read_derived_parquet_file_LFL(  
    file.path(  
      "returns",  
      "monthly_stock_returns.parquet"  
    )  
  )  
tbl_annual_raw <-  
  read_derived_parquet_file_LFL(  
    file.path(  
      "fundamentals",  
      "annual_firm_characteristics.parquet"  
    )  
  )  
tbl_factor <-  
  read_derived_parquet_file_LFL(  
    file.path(  
      "factors",  
      "fama_french_factors.parquet"  
    )  
  )
```

7.1.2 5.1.2

2020

36

```
tbl_monthly <- tbl_monthly_raw |>
  group_by(firm_ID) |>
  mutate(
    is_public_2020 = max(month_ID, na.rm = TRUE) == 72,
    n_observations = n()
  ) |>
  ungroup() |>
  filter(
    is_public_2020,
    n_observations >= 36
  ) |>
  select(-n_observations)
```

```
tbl_annual <- tbl_monthly |>
  distinct(year, firm_ID) |>
  left_join(
    tbl_annual_raw,
    by = c("year", "firm_ID")
  )
```

7.1.3 5.1.3

month_ID

R_F

```
tbl_monthly_factor <- tbl_monthly |>
  select(-any_of("R_F")) |>
  left_join(
    tbl_factor,
    by = "month_ID"
  )
```

7.2 5.2 FF3

7.2.1 5.2.1 FF3

$i \quad t \quad 3$

$$R_{i,t} - R_{F,t} = \beta_{i,M} R_{M,t}^e + \beta_{i,SMB} SMB_t + \beta_{i,HML} HML_t + \varepsilon_{i,t}$$

$R_{M,t}^e \quad SMB_t \quad HML_t \quad Re$

```
tbl_ff3_models <- tbl_monthly_factor |>
  select(
    firm_ID,
    Re,
    R_Me,
    SMB,
    HML
  ) |>
  drop_na() |>
  group_by(firm_ID) |>
  nest(tbl_firm = -firm_ID) |>
  mutate(
    list_model = map(
      tbl_firm,
      \(tbl_data) {
        lm(
          Re ~ 0 + R_Me + SMB + HML,
          data = tbl_data
        )
      }
    ),
    tbl_coefficient = map(
      list_model,
      broom::tidy
    )
  ) |>
  ungroup()
```

7.2.2 5.2.2

```
tbl_ff3_loadings <- tbl_ff3_models |>
  select(
    firm_ID,
    tbl_coefficient
  ) |>
  unnest(tbl_coefficient) |>
  select(
    firm_ID,
    term,
    estimate
  ) |>
  pivot_wider(
    names_from = term,
    values_from = estimate
  ) |>
  rename(
    beta_market = R_Me,
    beta_size = SMB,
    beta_value = HML
  )

show_table_head_LFL(tbl_ff3_loadings)
#> # A tibble: 6 x 4
#>   firm_ID beta_market beta_size beta_value
#>   <int>      <dbl>      <dbl>      <dbl>
#> 1     1        1.6191      2.6384      0.48152
#> 2     2        1.2405      0.50565     -0.52252
#> 3     3        0.69623     1.1439      0.33640
#> 4     4        0.37560     0.50193     -0.43245
#> 5     5        0.58264     1.5325      1.1194
#> 6     7        0.38022     0.92052     -0.29961
```

7.2.3 5.2.3

12

```
vec_factor_names <- c("R_Me", "SMB", "HML")

tbl_factor_risk_premium <- tbl_factor |>
  summarise(
    across(
```

```

    all_of(vec_factor_names),
    \((vec_return) {
      mean(vec_return, na.rm = TRUE) * 12
    }
  )
) |>
pivot_longer(
  cols = everything(),
  names_to = "factor",
  values_to = "annual_risk_premium"
)

vec_factor_risk_premium <- tbl_factor_risk_premium |>
  arrange(match(factor, vec_factor_names)) |>
  pull(annual_risk_premium)

names(vec_factor_risk_premium) <- vec_factor_names

show_table_LFL(tbl_factor_risk_premium)
#> # A tibble: 3 x 2
#>   factor annual_risk_premium
#>   <chr>          <dbl>
#> 1 R_Me          0.048258
#> 2 SMB           0.078586
#> 3 HML           0.083395

```

7.2.4 5.2.4

FF3 i

$$E[R_i - R_F] = \beta_{i,M} E[R_M^e] + \beta_{i,SMB} E[SMB] + \beta_{i,HML} E[HML]$$

```

tbl_cost_of_equity <- tbl_ff3_loadings |>
  mutate(
    cost_of_equity =
      beta_market * vec_factor_risk_premium[["R_Me"]] +
      beta_size * vec_factor_risk_premium[["SMB"]] +
      beta_value * vec_factor_risk_premium[["HML"]]
  )

```

```
show_table_head_LFL(tbl_cost_of_equity)
#> # A tibble: 6 x 5
#>   firm_ID beta_market beta_size beta_value cost_of_equity
#>   <int>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1     1      1.6191      2.6384      0.48152      0.32563
#> 2     2      1.2405      0.50565     -0.52252      0.056027
#> 3     3      0.69623      1.1439      0.33640      0.15155
#> 4     4      0.37560      0.50193     -0.43245      0.021505
#> 5     5      0.58264      1.5325      1.1194      0.24190
#> 6     7      0.38022      0.92052     -0.29961      0.065702
```

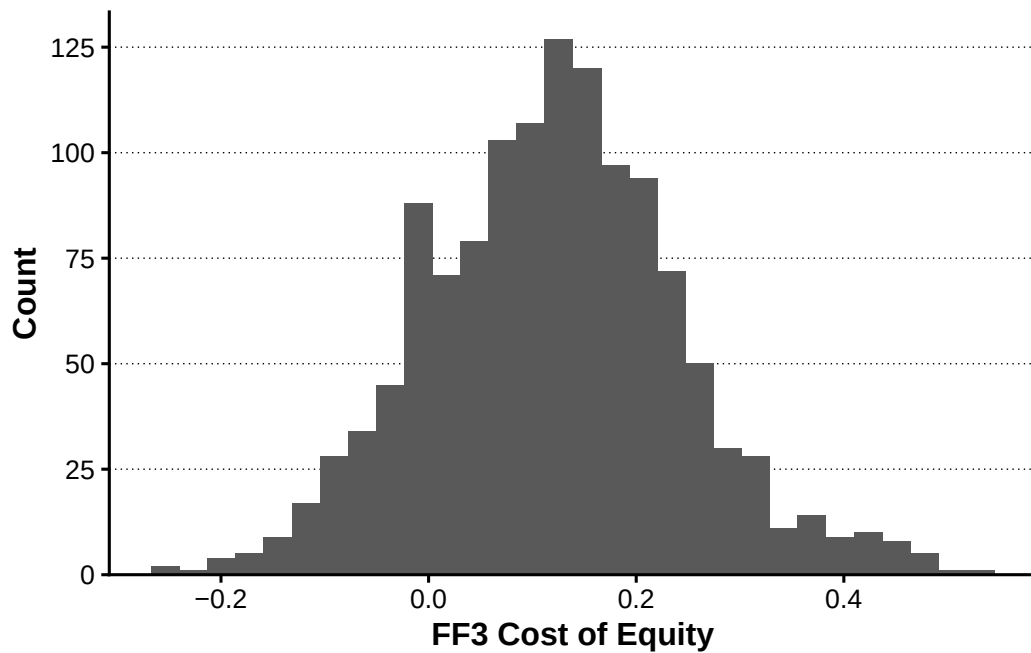
7.3 5.3

7.3.1 5.3.1

```
plot_cost_of_equity_distribution <- tbl_cost_of_equity |>
  ggplot(
    aes(x = cost_of_equity)
  ) +
  geom_histogram(bins = 30) +
  labs(
    x = "FF3 Cost of Equity",
    y = "Count"
  ) +
  scale_y_continuous(
    expand = expansion(mult = c(0, 0.05))
  )

plot_cost_of_equity_distribution <- add_lagrange_theme_LFL(
  plot_cost_of_equity_distribution
)

show_plot_LFL(plot_cost_of_equity_distribution)
```

7.3.2 5.3.2

```
tbl_firm_characteristics_2020 <- tbl_annual |>
  arrange(
    firm_ID,
    year
  ) |>
  group_by(firm_ID) |>
  mutate(
    lagged_ME = lag(ME)
  ) |>
  ungroup() |>
  filter(year == 2020) |>
  mutate(
    size_group = ntile(lagged_ME, 2),
    size_group = factor(
      size_group,
      levels = c(1, 2),
      labels = c("Small", "Large")
    )
  ) |>
```

```

inner_join(
  tbl_cost_of_equity,
  by = "firm_ID"
) |>
drop_na(
  lagged_ME,
  cost_of_equity,
  size_group
)

```

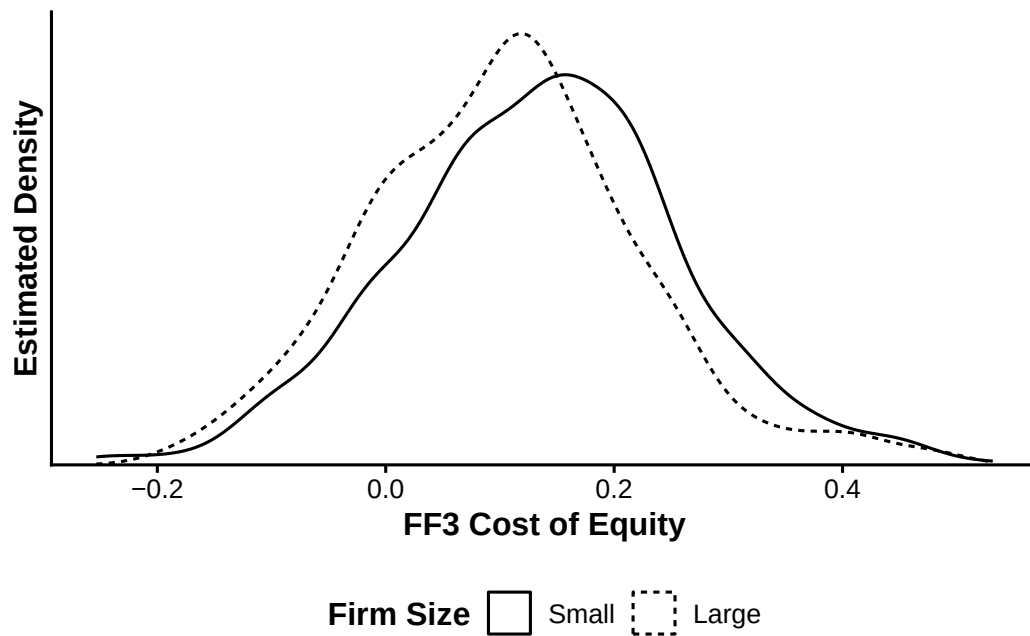
```

plot_cost_of_equity_by_size <- tbl_firm_characteristics_2020 |>
  ggplot(
    aes(
      x = cost_of_equity,
      linetype = size_group
    )
  ) +
  geom_density() +
  labs(
    x = "FF3 Cost of Equity",
    y = "Estimated Density",
    linetype = "Firm Size"
  ) +
  scale_y_continuous(
    expand = expansion(mult = c(0, 0.05)),
    breaks = NULL
  )

plot_cost_of_equity_by_size <- add_lagrange_theme_LFL(
  plot_cost_of_equity_by_size
)

show_plot_LFL(plot_cost_of_equity_by_size)

```



7.3.3 5.3.3

```
tbl_dividend_yield_2020 <- tbl_monthly |>
  filter(year == 2020) |>
  group_by(firm_ID) |>
  summarise(
    annual_dividend = sum(
      DPS * shares_outstanding,
      na.rm = TRUE
    ),
    latest_ME = ME[which.max(month_ID)],
    .groups = "drop"
  ) |>
  mutate(
    dividend_yield = annual_dividend / latest_ME
  ) |>
  filter(
    is.finite(dividend_yield),
    dividend_yield > 0
  )
```

```
show_table_head_LFL(tbl_dividend_yield_2020)
#> # A tibble: 6 x 4
#>   firm_ID annual_dividend latest_ME dividend_yield
#>   <int>      <dbl>      <dbl>      <dbl>
#> 1     1      325472000  9708946000    0.033523
#> 2     2      37536000  5194044000    0.0072267
#> 3     3     4660110000 214032195000    0.021773
#> 4     4     101352000  8652927000    0.011713
#> 5     5     745488000 235170402000    0.0031700
#> 6     7     64848000  15909376000    0.0040761
```

7.3.4 5.3.4

$$r_i \quad y_i \quad g_i$$

$$g_i = \frac{r_i - y_i}{1 + y_i}$$

```
tbl_implied_dividend_growth <- tbl_dividend_yield_2020 |>
  inner_join(
    tbl_cost_of_equity |>
      select(
        firm_ID,
        cost_of_equity
      ),
    by = "firm_ID"
  ) |>
  mutate(
    implied_dividend_growth =
      (cost_of_equity - dividend_yield) /
      (1 + dividend_yield)
  )

show_table_head_LFL(tbl_implied_dividend_growth)
#> # A tibble: 6 x 6
#>   firm_ID annual_dividend latest_ME dividend_yield cost_of_equity
#>   <int>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1     1      325472000  9708946000    0.033523    0.32563
#> 2     2      37536000  5194044000    0.0072267    0.056027
#> 3     3     4660110000 214032195000    0.021773    0.15155
#> 4     4     101352000  8652927000    0.011713    0.021505
```

```

#> 5      5      745488000 235170402000      0.0031700      0.24190
#> 6      7      64848000  15909376000      0.0040761      0.065702
#>   implied_dividend_growth
#>                               <dbl>
#> 1                0.28263
#> 2                0.048450
#> 3                0.12701
#> 4                0.0096791
#> 5                0.23797
#> 6                0.061376

```

7.4 5.4

7.4.1 5.4.1

```

tbl_factor_covariance <- compute_covariance_table_LFL(
  tbl_data = tbl_factor,
  vec_variables = vec_factor_names,
  annualization_factor = 12
)

matrix_factor_covariance <- convert_covariance_table_to_matrix_LFL(
  tbl_factor_covariance
)

show_table_LFL(tbl_factor_covariance)
#> # A tibble: 9 x 3
#>   variable_row variable_column covariance
#>   <chr>         <chr>         <dbl>
#> 1 R_Me         R_Me         0.020665
#> 2 R_Me         SMB         -0.0039063
#> 3 R_Me         HML         0.0015310
#> 4 SMB         R_Me         -0.0039063
#> 5 SMB         SMB         0.0062694
#> 6 SMB         HML         -0.0018855
#> 7 HML         R_Me         0.0015310
#> 8 HML         SMB         -0.0018855
#> 9 HML         HML         0.0039129

```

7.4.2 5.4.2

```
n_portfolio_firms <- 100L

tbl_investment_universe <- tbl_annual |>
  filter(year == 2020) |>
  inner_join(
    tbl_cost_of_equity |>
      select(
        firm_ID,
        cost_of_equity
      ),
    by = "firm_ID"
  ) |>
  slice_max(
    order_by = ME,
    n = n_portfolio_firms,
    with_ties = FALSE
  ) |>
  arrange(desc(ME)) |>
  select(
    firm_ID,
    ME,
    cost_of_equity
  )

vec_investment_firm_ids <- tbl_investment_universe |>
  pull(firm_ID)

n_portfolio_firms <- length(vec_investment_firm_ids)

show_table_head_LFL(tbl_investment_universe)
#> # A tibble: 6 x 3
#>   firm_ID      ME cost_of_equity
#>   <int>    <dbl>         <dbl>
#> 1     790 2.7339e13      0.061424
#> 2     987 5.4441e12     -0.071023
#> 3    1237 5.2764e12      0.091188
#> 4    1057 5.1050e12     -0.058108
#> 5     988 4.9127e12     -0.10059
#> 6    1400 3.7669e12      0.20979
```

7.4.3 5.4.3

```
tbl_idiosyncratic_variance <- tbl_monthly_factor |>
  filter(
    firm_ID %in% vec_investment_firm_ids
  ) |>
  inner_join(
    tbl_ff3_loadings,
    by = "firm_ID"
  ) |>
  mutate(
    fitted_return =
      R_F +
      beta_market * R_Me +
      beta_size * SMB +
      beta_value * HML,
    residual_return = R - fitted_return
  ) |>
  group_by(firm_ID) |>
  summarise(
    idiosyncratic_variance =
      12 * var(
        residual_return,
        na.rm = TRUE
      ),
    .groups = "drop"
  ) |>
  right_join(
    tbl_investment_universe |>
      select(firm_ID),
    by = "firm_ID"
  ) |>
  arrange(
    match(
      firm_ID,
      vec_investment_firm_ids
    )
  )

show_table_head_LFL(tbl_idiosyncratic_variance)
#> # A tibble: 6 x 2
#>   firm_ID idiosyncratic_variance
```

#>	<int>	<dbl>
#> 1	790	0.055478
#> 2	987	0.036155
#> 3	1237	0.053695
#> 4	1057	0.045481
#> 5	988	0.12455
#> 6	1400	0.094833

7.4.4 5.4.4

$$B \quad \Sigma_{\epsilon}$$

$$\Sigma = B\Sigma_F B^T + \Sigma_{\epsilon}$$

```
tbl_portfolio_loadings <- tbl_investment_universe |>
  select(firm_ID) |>
  left_join(
    tbl_ff3_loadings,
    by = "firm_ID"
  )

matrix_factor_loadings <- tbl_portfolio_loadings |>
  select(
    beta_market,
    beta_size,
    beta_value
  ) |>
  as.matrix()

vec_idiosyncratic_variance <- tbl_idiosyncratic_variance |>
  pull(idiosyncratic_variance)

matrix_idiosyncratic_covariance <- diag(
  vec_idiosyncratic_variance
)

matrix_return_covariance <-
  matrix_factor_loadings %*%
  matrix_factor_covariance %*%
  t(matrix_factor_loadings) +
  matrix_idiosyncratic_covariance
```



```
rownames(matrix_return_covariance) <- vec_investment_firm_ids
colnames(matrix_return_covariance) <- vec_investment_firm_ids
```

7.4.5 5.4.5

```
vec_expected_returns <- tbl_investment_universe |>
  pull(cost_of_equity)

names(vec_expected_returns) <- vec_investment_firm_ids

tibble(
  firm_ID = vec_investment_firm_ids,
  expected_return = vec_expected_returns
) |>
  show_table_head_LFL()
#> # A tibble: 6 x 2
#>   firm_ID expected_return
#>   <int>         <dbl>
#> 1     790         0.061424
#> 2     987        -0.071023
#> 3    1237         0.091188
#> 4    1057        -0.058108
#> 5     988        -0.10059
#> 6    1400         0.20979
```

7.5 5.5

7.5.1 5.5.1

$$\mu_p^* \quad w$$

$$\min_w \frac{1}{2} w^T \Sigma w$$

$$w^T \mu = \mu_p^*$$

$$w^T \mathbf{1} = 1$$

7.5.2 5.5.2

```
target_return <- 0.10

list_mean_variance_portfolio <- compute_mean_variance_portfolio_LFL(
  matrix_covariance = matrix_return_covariance,
  vec_expected_returns = vec_expected_returns,
  target_return = target_return
)
```

7.5.3 5.5.3

```
tbl_optimal_portfolio <- create_portfolio_weight_table_LFL(
  list_optimization = list_mean_variance_portfolio,
  vec_asset_ids = vec_investment_firm_ids,
  target_return = target_return
) |>
  arrange(
    desc(abs(weight))
  )

show_table_head_LFL(
  tbl_optimal_portfolio,
  n = 10
)

#> # A tibble: 10 x 3
#>   asset_ID    weight target_return
#>   <int>      <dbl>         <dbl>
#> 1     208 -0.16960           0.1
#> 2    1473  0.075792          0.1
#> 3     727  0.075617          0.1
#> 4    1253  0.073272          0.1
#> 5     987  0.068849          0.1
#> 6     829  0.062825          0.1
#> 7     409 -0.059685          0.1
#> 8      33  0.057845          0.1
```

```
#> 9      765 0.057677      0.1
#> 10     709 0.057313      0.1
```

7.5.4 5.5.4

```
tbl_portfolio_summary <- compute_portfolio_summary_LFL(
  list_optimization = list_mean_variance_portfolio,
  target_return = target_return
) |>
mutate(
  realized_expected_return = sum(
    list_mean_variance_portfolio$solution *
    vec_expected_returns
  ),
  return_constraint_error =
    realized_expected_return -
    target_return,
  weight_constraint_error =
    weight_sum -
    1
)

show_table_LFL(tbl_portfolio_summary)
#> # A tibble: 1 x 8
#>   target_return portfolio_return portfolio_variance portfolio_risk weight_sum
#>   <dbl>          <dbl>          <dbl>          <dbl>    <dbl>
#> 1      0.1          0.1          0.0069899      0.083606      1
#>   realized_expected_return return_constraint_error weight_constraint_error
#>   <dbl>          <dbl>          <dbl>          <dbl>
#> 1      0.1          -2.7756e-17      0
```

7.6 5.6

7.6.1 5.6.1

```
vec_target_returns <- seq(
  from = -0.10,
  to = 0.40,
```

```
length.out = 100
)
```

7.6.2 5.6.2

```
tbl_efficient_frontier <- compute_efficient_frontier_LFL(
  matrix_covariance = matrix_return_covariance,
  vec_expected_returns = vec_expected_returns,
  vec_target_returns = vec_target_returns
)

show_table_head_LFL(
  tbl_efficient_frontier |>
    select(
      target_return,
      portfolio_risk
    )
)

#> # A tibble: 6 x 2
#>   target_return portfolio_risk
#>   <dbl>          <dbl>
#> 1    -0.1          0.079982
#> 2   -0.094949      0.078497
#> 3   -0.089899      0.077068
#> 4   -0.084848      0.075697
#> 5   -0.079798      0.074389
#> 6   -0.074747      0.073145
```

7.6.3 5.6.3

```
plot_efficient_frontier <- tbl_efficient_frontier |>
  ggplot(
    aes(
      x = portfolio_risk,
      y = target_return
    )
  ) +
  geom_line() +
```

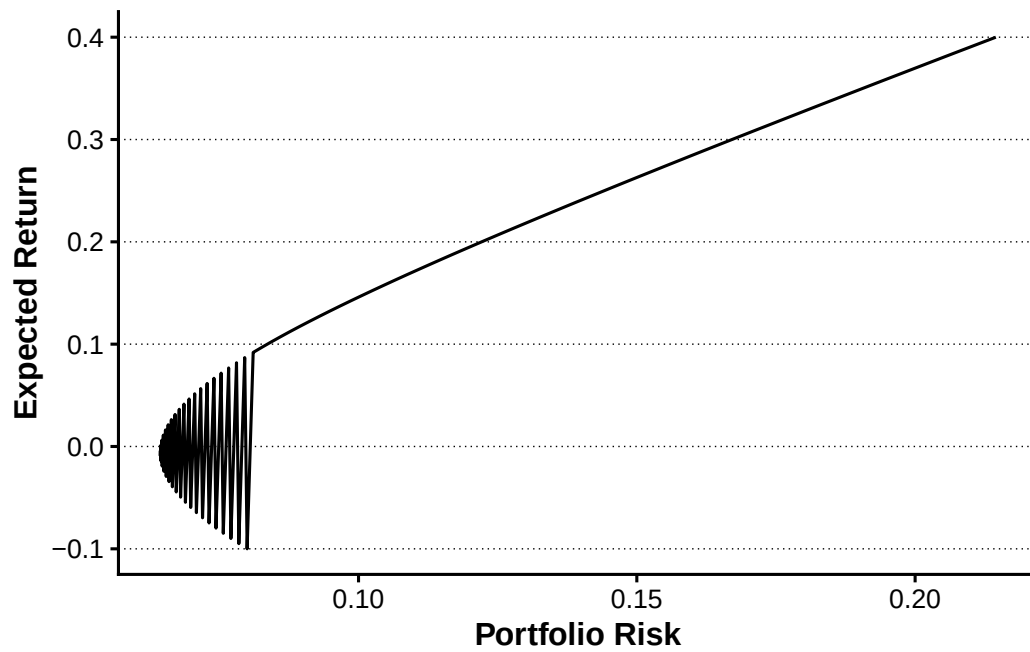
```

labs(
  x = "Portfolio Risk",
  y = "Expected Return"
)

plot_efficient_frontier <- add_lagrange_theme_LFL(
  plot_efficient_frontier
)

show_plot_LFL(plot_efficient_frontier)

```



7.7 5.7

Fama-French 3

8 I-6

- 1.
- 2. 5
- 3. 0
- 4.
- 5.
- 6. AAR CAAR
- 7. CAR
- 8.

8.1 6.1

8.1.1 6.1.1

$i \quad t \qquad R_{i,t} \qquad R_{M,t}$

$R_{i,t} = \alpha_i + \beta_i R_{M,t} + \varepsilon_{i,t}$

$\widehat{R}_{i,t} = \hat{\alpha}_i + \hat{\beta}_i R_{M,t}$

$AR_{i,t}$

$AR_{i,t} = R_{i,t} - \widehat{R}_{i,t}$

$t \qquad CAR_{i,t}$

$CAR_{i,t} = \sum_{\tau=T_1}^t AR_{i,\tau}$

8.1.2 6.1.2

0			
		-130	-31
		-30	-1
			0
		1	30
100	61	80	

8.1.3 6.1.3

```
estimation_start <- -130L
estimation_end <- -31L
event_start <- -30L
event_end <- 30L
minimum_estimation_observations <- 80L

tbl_window_parameter <- tribble(
  ~parameter, ~value,
  "estimation_start", estimation_start,
  "estimation_end", estimation_end,
  "event_start", event_start,
  "event_end", event_end,
  "minimum_estimation_observations", minimum_estimation_observations
)

tbl_window_parameter
#> # A tibble: 5 x 2
#>   parameter          value
#>   <chr>          <int>
#> 1 estimation_start    -130
#> 2 estimation_end      -31
#> 3 event_start        -30
#> 4 event_end           30
#> 5 minimum_estimation_observations 80
```

```

vec_project_candidate <- unique(c(
  Sys.getenv("QUARTO_PROJECT_DIR"),
  getwd(),
  dirname(getwd())
))

vec_project_candidate <- vec_project_candidate[
  nzchar(vec_project_candidate)
]

vec_project_match <- vec_project_candidate[
  file.exists(file.path(vec_project_candidate, "_quarto.yml"))
]

if (length(vec_project_match) == 0L) {
  stop("LagrangeFinance      ")
}

project_dir <- normalizePath(
  vec_project_match[[1]],
  winslash = "/",
  mustWork = TRUE
)

data_dir <- get_data_dir_LFL()

raw_event_data_dir <- file.path(
  data_dir,
  "raw",
  "events"
)

derived_event_data_dir <- file.path(
  data_dir,
  "derived",
  "events"
)

dir.create(
  derived_event_data_dir,
  recursive = TRUE,
  showWarnings = FALSE
)

```



```

)

tibble(
  directory = c(
    "project",
    "raw_events",
    "derived_events"
  ),
  path = c(
    project_dir,
    raw_event_data_dir,
    derived_event_data_dir
  )
)
#> # A tibble: 3 x 2
#>   directory      path
#>   <chr>         <chr>
#> 1 project      C:/AnalyticFin/Projects/LagrangeFinance
#> 2 raw_events   C:/AnalyticFin/Projects/LagrangeFinance/data/raw/events
#> 3 derived_events C:/AnalyticFin/Projects/LagrangeFinance/data/derived/events

```

8.2 6.2

8.2.1 6.2.1

3

```

data/raw/events/firm_returns.parquet
data/raw/events/market_returns.parquet
data/raw/events/earnings_events.parquet

```

Parquet

```

vec_input_file <- c(
  return = file.path(
    "raw",
    "events",
    "firm_returns.parquet"
  ),
  market = file.path(

```

```

    "raw",
    "events",
    "market_returns.parquet"
  ),
  event = file.path(
    "raw",
    "events",
    "earnings_events.parquet"
  )
)

vec_input_path <- purrr::map_chr(
  vec_input_file,
  find_data_file_LFL
)

tibble(
  data_name = names(vec_input_path),
  file_name = basename(vec_input_path),
  path = unname(vec_input_path)
)
#> # A tibble: 3 x 3
#>   data_name file_name
#>   <chr>      <chr>
#> 1 return    firm_returns.parquet
#> 2 market    market_returns.parquet
#> 3 event     earnings_events.parquet
#>   path
#>   <chr>
#> 1 C:/AnalyticFin/Projects/LagrangeFinance/data/raw/events/firm_returns.parquet
#> 2 C:/AnalyticFin/Projects/LagrangeFinance/data/raw/events/market_returns.parquet
#> 3 C:/AnalyticFin/Projects/LagrangeFinance/data/raw/events/earnings_events.parquet

```

8.2.2 6.2.2

```

tbl_return_raw <-
  read_parquet_file_LFL(
    vec_input_file[["return"]]
  )

```

```
tbl_market_raw <-
  read_parquet_file_LFL(
    vec_input_file[["market"]]
  )

tbl_event_raw <-
  read_parquet_file_LFL(
    vec_input_file[["event"]]
  )
```

8.2.3 6.2.3

```
validate_required_columns_LFL(
  tbl_return_raw,
  c("date", "firm_ID", "R"),
  "tbl_return_raw"
)

validate_required_columns_LFL(
  tbl_market_raw,
  c("date", "R_M"),
  "tbl_market_raw"
)

validate_required_columns_LFL(
  tbl_event_raw,
  c(
    "event_date",
    "firm_ID",
    "earnings_forecast",
    "realized_earnings",
    "lagged_ME"
  ),
  "tbl_event_raw"
)
```

8.2.4 6.2.4

```
tbl_return <- tbl_return_raw |>
  transmute(
    date = as.Date(date),
    firm_ID = as.integer(firm_ID),
    R = as.numeric(R)
  ) |>
  arrange(
    firm_ID,
    date
  )

tbl_market <- tbl_market_raw |>
  transmute(
    date = as.Date(date),
    R_M = as.numeric(R_M)
  ) |>
  arrange(date)

tbl_event <- tbl_event_raw |>
  transmute(
    event_date = as.Date(event_date),
    firm_ID = as.integer(firm_ID),
    earnings_forecast = as.numeric(
      earnings_forecast
    ),
    realized_earnings = as.numeric(
      realized_earnings
    ),
    lagged_ME = as.numeric(lagged_ME)
  ) |>
  arrange(
    event_date,
    firm_ID
  )
```

8.2.5 6.2.5

```
validate_unique_keys_LFL(
  tbl_return,
  c("firm_ID", "date"),
```

```

    "tbl_return"
  )

  validate_unique_keys_LFL(
    tbl_market,
    "date",
    "tbl_market"
  )

  validate_unique_keys_LFL(
    tbl_event,
    c("firm_ID", "event_date"),
    "tbl_event"
  )

```

```

tbl_missing_summary <- bind_rows(
  summarize_missing_values_LFL(
    tbl_return,
    "return"
  ),
  summarize_missing_values_LFL(
    tbl_market,
    "market"
  ),
  summarize_missing_values_LFL(
    tbl_event,
    "event"
  )
)

```

```
tbl_missing_summary
```

```
#> # A tibble: 10 x 5
```

#>	data_name	observations	missing_ratio	variable	missing_values
#>	<chr>	<int>	<dbl>	<chr>	<int>
#>	1 return	1610400	0	date	0
#>	2 return	1610400	0	firm_ID	0
#>	3 return	1610400	0	R	0
#>	4 market	1220	0	date	0
#>	5 market	1220	0	R_M	0
#>	6 event	3960	0	event_date	0
#>	7 event	3960	0	firm_ID	0
#>	8 event	3960	0	earnings_forecast	0

```
#> 9 event          3960          0 realized_earnings          0
#> 10 event         3960          0 lagged_ME              0
```

8.2.6 6.2.6

```
tbl_input_overview <- tribble(
  ~data_name, ~observations, ~firms, ~first_date, ~last_date,
  "individual returns",
  nrow(tbl_return),
  n_distinct(tbl_return$firm_ID),
  min(tbl_return$date, na.rm = TRUE),
  max(tbl_return$date, na.rm = TRUE),
  "market returns",
  nrow(tbl_market),
  NA_integer_,
  min(tbl_market$date, na.rm = TRUE),
  max(tbl_market$date, na.rm = TRUE),
  "events",
  nrow(tbl_event),
  n_distinct(tbl_event$firm_ID),
  min(tbl_event$event_date, na.rm = TRUE),
  max(tbl_event$event_date, na.rm = TRUE)
)

tbl_input_overview
#> # A tibble: 3 x 5
#>   data_name      observations firms first_date last_date
#>   <chr>          <int> <int> <date>    <date>
#> 1 individual returns    1610400  1320 2016-01-04 2020-12-30
#> 2 market returns        1220    NA 2016-01-04 2020-12-30
#> 3 events                3960  1320 2017-01-04 2020-04-07
```

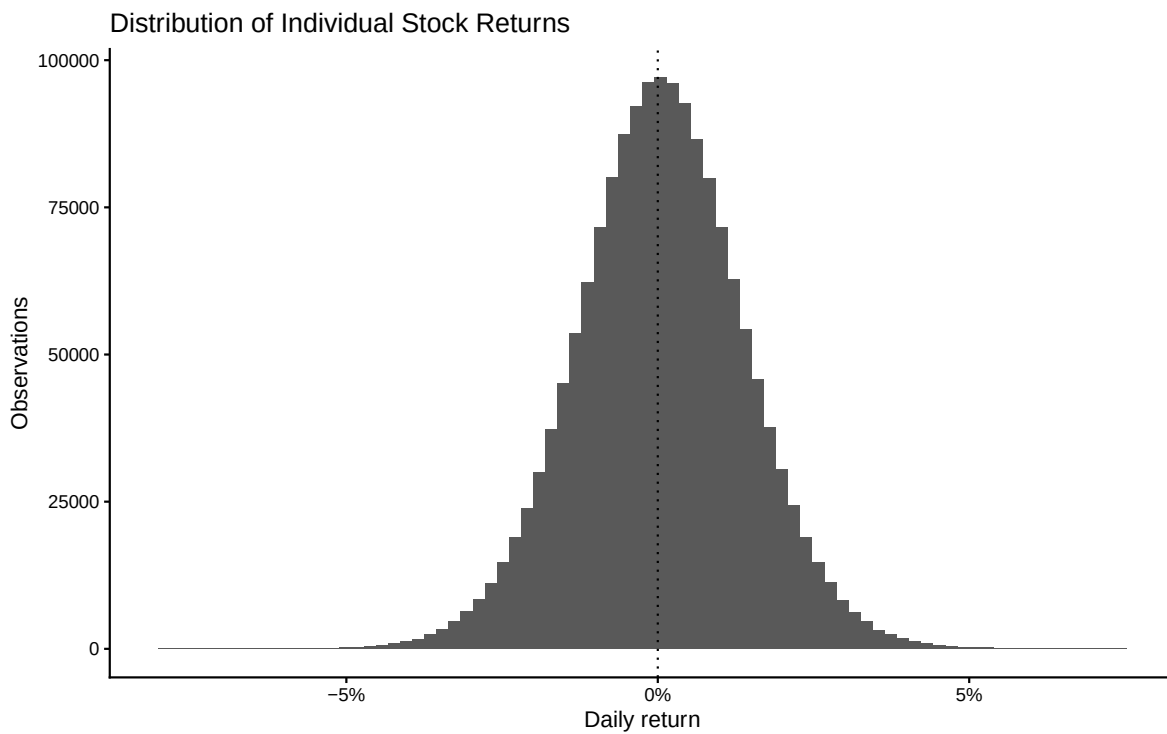
8.2.7 6.2.7

```
plot_return_distribution <- tbl_return |>
  filter(is.finite(R)) |>
  ggplot(aes(x = R)) +
  geom_histogram(bins = 80) +
```

```

geom_vline(
  xintercept = 0,
  linetype = "dotted"
) +
scale_x_continuous(
  labels = scales::label_percent()
) +
labs(
  x = "Daily return",
  y = "Observations",
  title = "Distribution of Individual Stock Returns"
) +
theme_classic()
plot_return_distribution

```



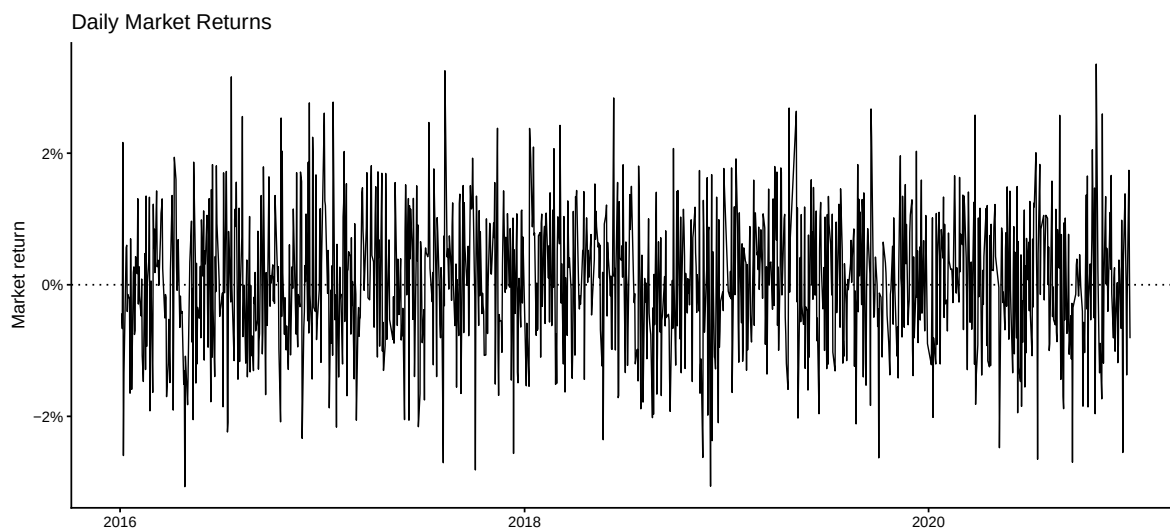
8.2.8 6.2.8

```

plot_market_return_series <- tbl_market |>
  ggplot(aes(x = date, y = R_M)) +
  geom_line(linewidth = 0.4) +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = NULL,
    y = "Market return",
    title = "Daily Market Returns"
  ) +
  theme_classic()

```

plot_market_return_series



8.3 6.3

8.3.1 6.3.1

$$Surprise_i = \frac{RealizedEarnings_i - ForecastEarnings_i}{ME_{i,-1}}$$

```
tbl_event <- compute_earnings_surprise_LFL(
  tbl_event = tbl_event,
  number_of_groups = 5L
)

tbl_event |>
  select(
    event_ID,
    firm_ID,
    event_date,
    event_year,
    earnings_surprise,
    event_strength
  ) |>
  slice_head(n = 20)
#> # A tibble: 20 x 6
#>   event_ID firm_ID event_date event_year earnings_surprise event_strength
#>   <int>   <int> <date>         <dbl>         <dbl> <ord>
#> 1       1     39 2017-01-04     2017      -0.0081327 1
#> 2       2     57 2017-01-04     2017      -0.010049 1
#> 3       3     73 2017-01-04     2017      -0.0020232 4
#> 4       4    109 2017-01-04     2017      -0.0043961 3
#> 5       5    111 2017-01-04     2017      -0.0044873 3
#> 6       6    231 2017-01-04     2017      -0.014399 1
#> 7       7    944 2017-01-04     2017      -0.0068979 2
#> 8       8   1227 2017-01-04     2017      -0.0011248 5
#> 9       9    422 2017-01-05     2017      -0.0048408 3
#> 10      10   1203 2017-01-05     2017      -0.00086568 5
#> 11      11    346 2017-01-06     2017      -0.0058582 2
#> 12      12    590 2017-01-06     2017      -0.010344 1
#> 13      13    644 2017-01-06     2017      -0.0064889 2
#> 14      14   1153 2017-01-06     2017      -0.0021003 4
#> 15      15    191 2017-01-10     2017      -0.0034658 4
#> 16      16    353 2017-01-10     2017      -0.010249 1
#> 17      17    566 2017-01-10     2017      -0.0043876 3
#> 18      18    322 2017-01-11     2017      -0.0046897 3
#> 19      19   1050 2017-01-11     2017      -0.0024991 4
#> 20      20    426 2017-01-12     2017      -0.0010089 5
```

8.3.2 6.3.2

```
tbl_event_strength_summary <- tbl_event |>
  group_by(
    event_year,
    event_strength
  ) |>
  summarise(
    events = n(),
    mean_surprise = mean(
      earnings_surprise,
      na.rm = TRUE
    ),
    median_surprise = median(
      earnings_surprise,
      na.rm = TRUE
    ),
    .groups = "drop"
  ) |>
  arrange(
    event_year,
    event_strength
  )

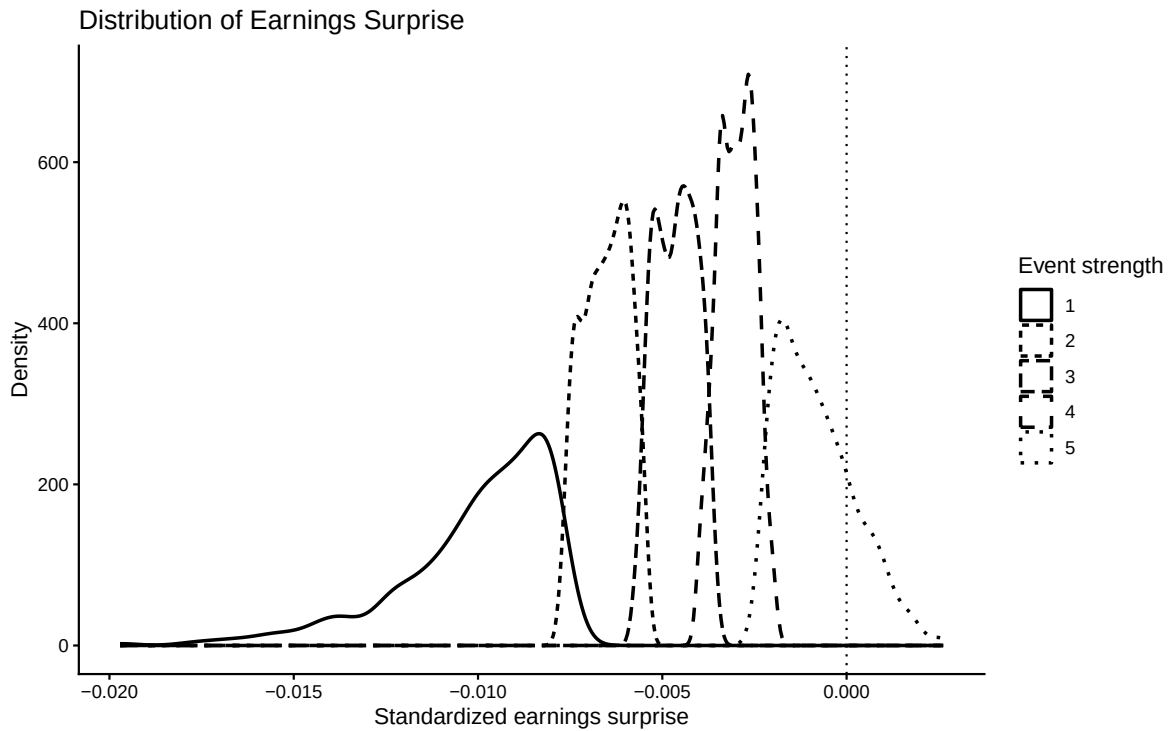
tbl_event_strength_summary
#> # A tibble: 20 x 5
#>   event_year event_strength events mean_surprise median_surprise
#>   <dbl> <ord>          <int>      <dbl>          <dbl>
#> 1     2017 1             214    -0.010271     -0.0098319
#> 2     2017 2             214    -0.0064294     -0.0063866
#> 3     2017 3             214    -0.0044249     -0.0043693
#> 4     2017 4             213    -0.0028679     -0.0028917
#> 5     2017 5             213    -0.00062223    -0.00081805
#> 6     2018 1             263    -0.0099427     -0.0092589
#> 7     2018 2             263    -0.0066866     -0.0066374
#> 8     2018 3             263    -0.0048770     -0.0048670
#> 9     2018 4             263    -0.0031921     -0.0031796
#> 10    2018 5             263    -0.00098501    -0.0011702
#> 11    2019 1             262    -0.0098017     -0.0094002
#> 12    2019 2             262    -0.0064618     -0.0064241
#> 13    2019 3             261    -0.0045936     -0.0045890
#> 14    2019 4             261    -0.0029579     -0.0029594
```

#> 15	2019 5	261	-0.00088716	-0.0010823
#> 16	2020 1	54	-0.010749	-0.010333
#> 17	2020 2	54	-0.0065058	-0.0065211
#> 18	2020 3	54	-0.0043621	-0.0044412
#> 19	2020 4	54	-0.0026712	-0.0026349
#> 20	2020 5	54	-0.00051141	-0.00069626

8.3.3 6.3.3

```
plot_earnings_surprise_distribution <- tbl_event |>
  filter(is.finite(earnings_surprise)) |>
  ggplot(
    aes(
      x = earnings_surprise,
      linetype = event_strength
    )
  ) +
  geom_density(linewidth = 0.8) +
  geom_vline(
    xintercept = 0,
    linetype = "dotted"
  ) +
  labs(
    x = "Standardized earnings surprise",
    y = "Density",
    linetype = "Event strength",
    title = "Distribution of Earnings Surprise"
  ) +
  theme_classic()

plot_earnings_surprise_distribution
```



8.3.4 6.3.4

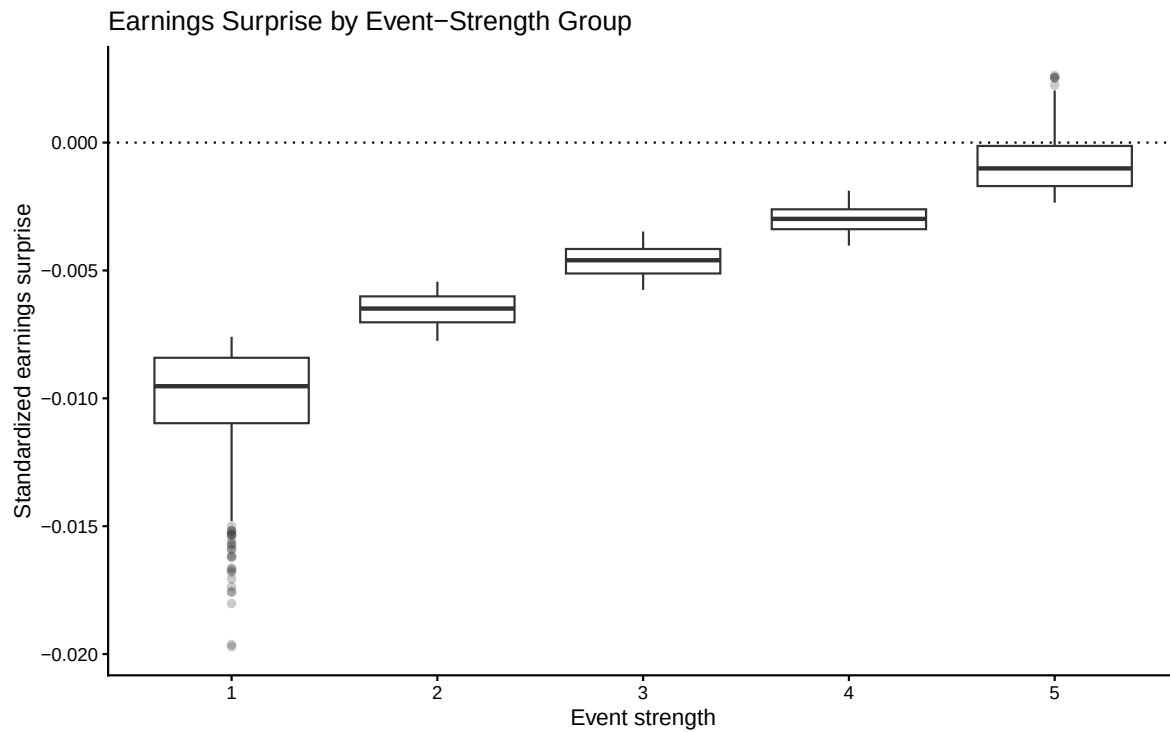
```
plot_surprise_by_strength <- tbl_event |>
  filter(is.finite(earnings_surprise)) |>
  ggplot(
    aes(
      x = event_strength,
      y = earnings_surprise
    )
  ) +
  geom_boxplot(
    outlier.alpha = 0.25
  ) +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  labs(
    x = "Event strength",
```

```

  y = "Standardized earnings surprise",
  title = "Earnings Surprise by Event-Strength Group"
) +
  theme_classic()

plot_surprise_by_strength

```



8.3.5 6.3.5

```

plot_events_by_year <- tbl_event |>
  count(
    event_year,
    event_strength,
    name = "events"
  ) |>
  ggplot(
    aes(
      x = factor(event_year),

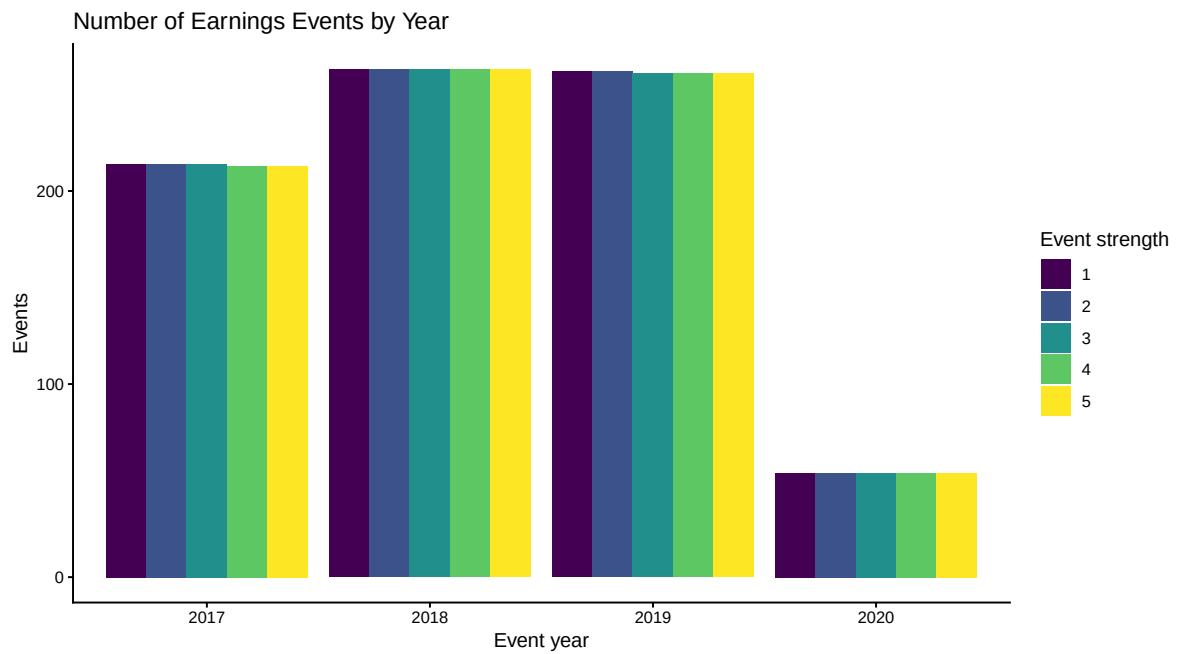
```

```

    y = events,
    fill = event_strength
  )
) +
geom_col(
  position = "dodge"
) +
labs(
  x = "Event year",
  y = "Events",
  fill = "Event strength",
  title = "Number of Earnings Events by Year"
) +
theme_classic()

plot_events_by_year

```



8.4 6.4

8.4.1 6.4.1

```
tbl_trading_calendar <- tbl_market |>
  distinct(date) |>
  arrange(date) |>
  mutate(
    date_ID = row_number()
  ) |>
  select(
    date_ID,
    date
  )

tbl_market_with_ID <- tbl_market |>
  inner_join(
    tbl_trading_calendar,
    by = "date"
  ) |>
  select(
    date_ID,
    date,
    R_M
  )

tbl_return_market <- tbl_return |>
  inner_join(
    tbl_market_with_ID,
    by = "date"
  ) |>
  select(
    firm_ID,
    date_ID,
    R,
    R_M
  ) |>
  arrange(
    firm_ID,
```

```

    date_ID
  )

```

8.4.2 6.4.2 0

0

```

tbl_event <- assign_event_day_zero_LFL(
  tbl_event = tbl_event,
  tbl_trading_calendar = tbl_trading_calendar
)

tbl_event |>
  select(
    event_ID,
    firm_ID,
    event_date,
    event_trade_date,
    event_trade_date_ID,
    event_strength,
    earnings_surprise
  ) |>
  slice_head(n = 20)
#> # A tibble: 20 x 7
#>   event_ID firm_ID event_date event_trade_date event_trade_date_ID
#>   <int>   <int> <date>      <date>                <int>
#> 1       1     39 2017-01-04 2017-01-05                247
#> 2       2     57 2017-01-04 2017-01-05                247
#> 3       3     73 2017-01-04 2017-01-05                247
#> 4       4    109 2017-01-04 2017-01-05                247
#> 5       5    111 2017-01-04 2017-01-05                247
#> 6       6    231 2017-01-04 2017-01-05                247
#> 7       7    944 2017-01-04 2017-01-05                247
#> 8       8   1227 2017-01-04 2017-01-05                247
#> 9       9    422 2017-01-05 2017-01-06                248
#> 10      10   1203 2017-01-05 2017-01-06                248
#> 11      11    346 2017-01-06 2017-01-10                249
#> 12      12    590 2017-01-06 2017-01-10                249
#> 13      13    644 2017-01-06 2017-01-10                249
#> 14      14   1153 2017-01-06 2017-01-10                249
#> 15      15    191 2017-01-10 2017-01-11                250

```



```

#> 16      16      353 2017-01-10 2017-01-11      250
#> 17      17      566 2017-01-10 2017-01-11      250
#> 18      18      322 2017-01-11 2017-01-12      251
#> 19      19     1050 2017-01-11 2017-01-12      251
#> 20      20      426 2017-01-12 2017-01-13      252
#>      event_strength earnings_surprise
#>      <ord>                <dbl>
#> 1 1                -0.0081327
#> 2 1                -0.010049
#> 3 4                -0.0020232
#> 4 3                -0.0043961
#> 5 3                -0.0044873
#> 6 1                -0.014399
#> 7 2                -0.0068979
#> 8 5                -0.0011248
#> 9 3                -0.0048408
#> 10 5               -0.00086568
#> 11 2               -0.0058582
#> 12 1               -0.010344
#> 13 2               -0.0064889
#> 14 4               -0.0021003
#> 15 4               -0.0034658
#> 16 1               -0.010249
#> 17 3               -0.0043876
#> 18 3               -0.0046897
#> 19 4               -0.0024991
#> 20 5               -0.0010089

```

8.4.3 6.4.3 0

```

tbl_event_gap <- tbl_event |>
  mutate(
    calendar_day_gap = as.integer(
      event_trade_date - event_date
    )
  )

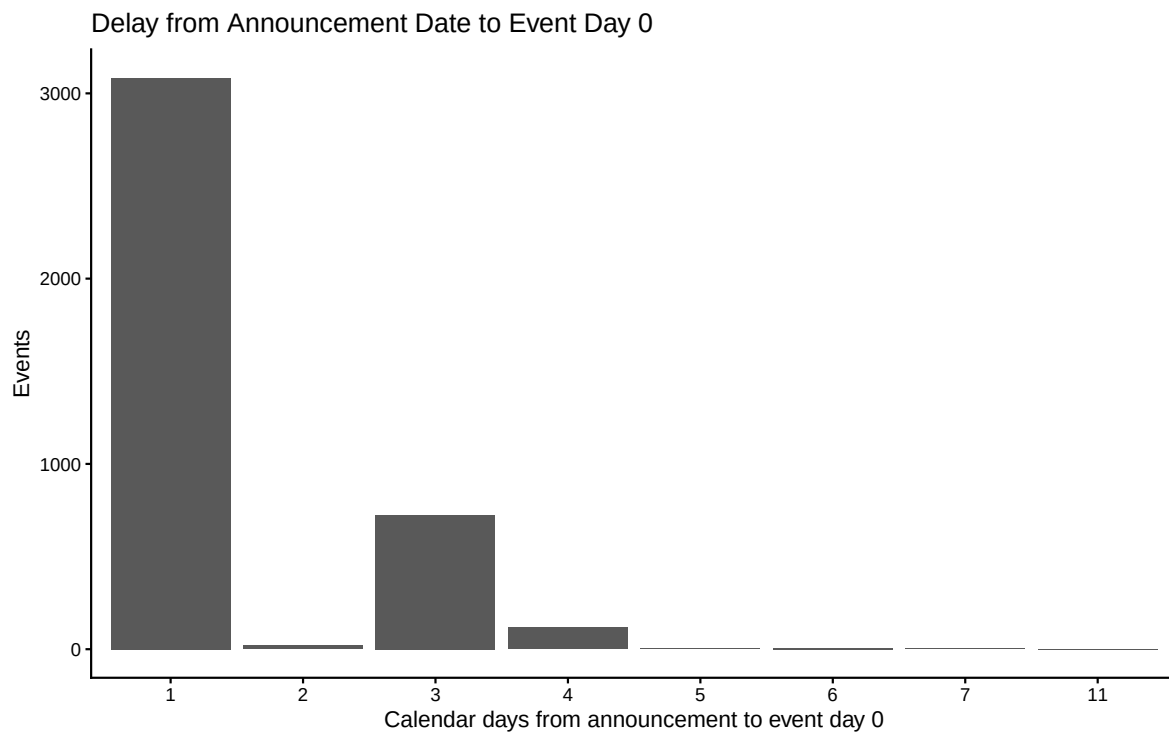
plot_event_date_gap <- tbl_event_gap |>
  count(
    calendar_day_gap,

```

```

    name = "events"
  ) |>
  ggplot(
    aes(
      x = factor(calendar_day_gap),
      y = events
    )
  ) +
  geom_col() +
  labs(
    x = "Calendar days from announcement to event day 0",
    y = "Events",
    title = "Delay from Announcement Date to Event Day 0"
  ) +
  theme_classic()
plot_event_date_gap

```



8.4.4 6.4.4

```
tbl_event_panel <- create_event_study_panel_LFL(
  tbl_event = tbl_event,
  tbl_trading_calendar = tbl_trading_calendar,
  tbl_return_market = tbl_return_market,
  estimation_start = estimation_start,
  event_end = event_end
)

tbl_event_panel |>
  select(
    event_ID,
    firm_ID,
    relative_day,
    date,
    R,
    R_M
  ) |>
  slice_head(n = 20)
#> # A tibble: 20 x 6
#>   event_ID firm_ID relative_day date          R          R_M
#>   <int>    <int>      <int> <date>      <dbl>      <dbl>
#> 1         1        39      -130 2016-06-23  0.0098506  0.018087
#> 2         1        39      -129 2016-06-24  0.012580  0.0092862
#> 3         1        39      -128 2016-06-27  0.015076  0.0031098
#> 4         1        39      -127 2016-06-28  0.015788  0.00067260
#> 5         1        39      -126 2016-06-29 -0.0033987 -0.0048162
#> 6         1        39      -125 2016-06-30 -0.0071048 -0.0038061
#> 7         1        39      -124 2016-07-01  0.014684 -0.0032365
#> 8         1        39      -123 2016-07-04 -0.0069849 -0.0012511
#> 9         1        39      -122 2016-07-05 -0.023656 -0.018547
#> 10        1        39      -121 2016-07-06  0.011629  0.017017
#> 11        1        39      -120 2016-07-07 -0.0064901 -0.0011435
#> 12        1        39      -119 2016-07-08 -0.0062219 -0.0012174
#> 13        1        39      -118 2016-07-11  0.0055502  0.017228
#> 14        1        39      -117 2016-07-12  0.0064255  0.0060624
#> 15        1        39      -116 2016-07-13 -0.027040 -0.022380
#> 16        1        39      -115 2016-07-14 -0.014094 -0.020934
#> 17        1        39      -114 2016-07-15  0.0064607  0.0081064
#> 18        1        39      -113 2016-07-19 -0.010573 -0.0026187
```

#> 19	1	39	-112	2016-07-20	0.024173	0.031602
#> 20	1	39	-111	2016-07-21	-0.00044361	-0.00024792

8.4.5 6.4.5

```

estimation_window_length <-
  estimation_end -
  estimation_start + 1L

event_window_length <-
  event_end -
  event_start + 1L

tbl_event_coverage <- tbl_event_panel |>
  group_by(
    event_ID,
    firm_ID,
    event_strength
  ) |>
  summarise(
    valid_estimation_days = sum(
      between(
        relative_day,
        estimation_start,
        estimation_end
      ) &
      is.finite(R) &
      is.finite(R_M)
    ),
    valid_event_days = sum(
      between(
        relative_day,
        event_start,
        event_end
      ) &
      is.finite(R) &
      is.finite(R_M)
    ),
    .groups = "drop"
  )

```

```
tbl_event_coverage |>
  summarise(
    events = n(),
    complete_estimation_window = sum(
      valid_estimation_days ==
        estimation_window_length
    ),
    usable_estimation_window = sum(
      valid_estimation_days >=
        minimum_estimation_observations
    ),
    complete_event_window = sum(
      valid_event_days ==
        event_window_length
    )
  )
#> # A tibble: 1 x 4
#>   events complete_estimation_window usable_estimation_window
#>   <int>                <int>                <int>
#> 1   3960                3960                3960
#>   complete_event_window
#>               <int>
#> 1                 3960
```

8.4.6 6.4.6

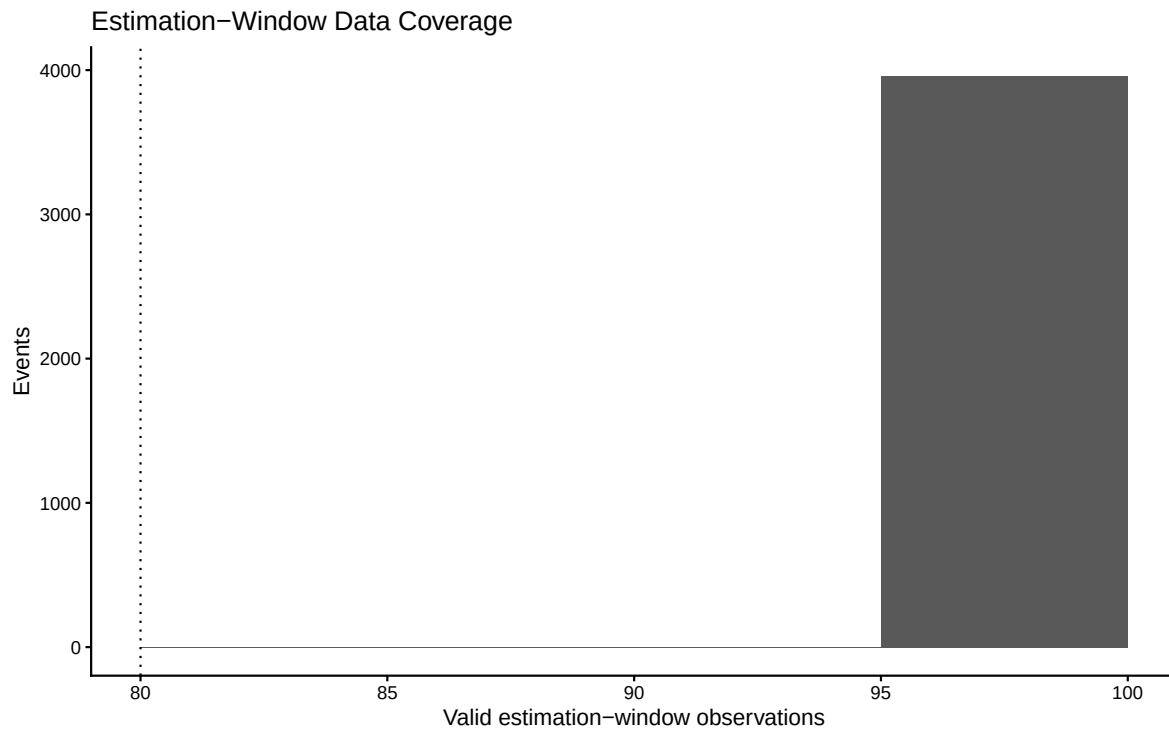
```
plot_estimation_coverage <- tbl_event_coverage |>
  ggplot(
    aes(
      x = valid_estimation_days
    )
  ) +
  geom_histogram(
    binwidth = 5,
    boundary = 0
  ) +
  geom_vline(
    xintercept =
      minimum_estimation_observations,
    linetype = "dotted"
```

```

) +
labs(
  x = "Valid estimation-window observations",
  y = "Events",
  title = "Estimation-Window Data Coverage"
) +
theme_classic()

plot_estimation_coverage

```



8.5 6.5

8.5.1 6.5.1

```

tbl_market_model <- estimate_market_models_LFL(
  tbl_event_panel = tbl_event_panel,
  estimation_start = estimation_start,
  estimation_end = estimation_end,

```

```

minimum_observations =
  minimum_estimation_observations
)

tbl_market_model |>
  slice_head(n = 20)
#> # A tibble: 20 x 13
#>   event_ID firm_ID event_date event_trade_date event_year event_strength
#>   <int>    <int> <date>      <date>          <dbl> <ord>
#> 1         1      39 2017-01-04 2017-01-05          2017 1
#> 2         2      57 2017-01-04 2017-01-05          2017 1
#> 3         3      73 2017-01-04 2017-01-05          2017 4
#> 4         4     109 2017-01-04 2017-01-05          2017 3
#> 5         5     111 2017-01-04 2017-01-05          2017 3
#> 6         6     231 2017-01-04 2017-01-05          2017 1
#> 7         7     944 2017-01-04 2017-01-05          2017 2
#> 8         8    1227 2017-01-04 2017-01-05          2017 5
#> 9         9     422 2017-01-05 2017-01-06          2017 3
#> 10        10    1203 2017-01-05 2017-01-06          2017 5
#> 11        11     346 2017-01-06 2017-01-10          2017 2
#> 12        12     590 2017-01-06 2017-01-10          2017 1
#> 13        13     644 2017-01-06 2017-01-10          2017 2
#> 14        14    1153 2017-01-06 2017-01-10          2017 4
#> 15        15     191 2017-01-10 2017-01-11          2017 4
#> 16        16     353 2017-01-10 2017-01-11          2017 1
#> 17        17     566 2017-01-10 2017-01-11          2017 3
#> 18        18     322 2017-01-11 2017-01-12          2017 3
#> 19        19    1050 2017-01-11 2017-01-12          2017 4
#> 20        20     426 2017-01-12 2017-01-13          2017 5
#>   earnings_surprise      alpha      beta  sigma_AR r_squared adjusted_r_squared
#>   <dbl>          <dbl>    <dbl>    <dbl>    <dbl>          <dbl>
#> 1    -0.0081327   -3.5923e-4 0.75040 0.0079236 0.51009      0.50509
#> 2    -0.010049   -2.3180e-4 1.3879 0.0079819 0.77826      0.77600
#> 3    -0.0020232   -7.9684e-4 1.2040 0.0080940 0.71979      0.71693
#> 4    -0.0043961   -6.2781e-5 1.0279 0.0079145 0.66194      0.65849
#> 5    -0.0044873   -2.7146e-4 1.0319 0.0081593 0.64996      0.64639
#> 6    -0.014399     6.9858e-4 1.2164 0.0078301 0.73696      0.73428
#> 7    -0.0068979   -9.3859e-4 1.2906 0.0078816 0.75685      0.75437
#> 8    -0.0011248     4.6660e-4 1.1543 0.0078297 0.71615      0.71326
#> 9    -0.0048408   -1.9959e-4 0.48469 0.0093315 0.23306      0.22523
#> 10   -0.00086568  -8.3396e-4 0.89616 0.0088518 0.53584      0.53111
#> 11   -0.0058582     6.9132e-4 0.60488 0.0084943 0.36168      0.35517

```

```

#> 12      -0.010344      2.3376e-3  1.1796  0.0070260  0.75900      0.75654
#> 13      -0.0064889      4.3520e-4  1.1550  0.0075195  0.72499      0.72219
#> 14      -0.0021003     -4.3792e-5  0.96253  0.0085477  0.58624      0.58202
#> 15      -0.0034658     -2.6240e-3  0.59313  0.0078355  0.39675      0.39060
#> 16      -0.010249      6.5290e-7  1.0088  0.0082464  0.63202      0.62826
#> 17      -0.0043876      4.9855e-4  0.91843  0.0083511  0.58128      0.57701
#> 18      -0.0046897      5.8502e-4  0.90935  0.0076618  0.62317      0.61932
#> 19      -0.0024991      4.2531e-4  1.1475  0.0080806  0.70304      0.70001
#> 20      -0.0010089     -1.0163e-3  0.91607  0.0082056  0.60440      0.60036
#>      estimation_observations
#>      <int>
#> 1      100
#> 2      100
#> 3      100
#> 4      100
#> 5      100
#> 6      100
#> 7      100
#> 8      100
#> 9      100
#> 10     100
#> 11     100
#> 12     100
#> 13     100
#> 14     100
#> 15     100
#> 16     100
#> 17     100
#> 18     100
#> 19     100
#> 20     100

```

8.5.2 6.5.2

```

tbl_market_model_summary <- tbl_market_model |>
  summarise(
    estimated_events = n(),
    mean_alpha = mean(alpha),
    mean_beta = mean(beta),
    median_beta = median(beta),

```



```

    minimum_beta = min(beta),
    maximum_beta = max(beta),
    mean_r_squared = mean(r_squared),
    median_r_squared = median(r_squared),
    mean_sigma_AR = mean(sigma_AR),
    minimum_observations =
      min(estimation_observations),
    maximum_observations =
      max(estimation_observations)
  )
)

tbl_market_model_summary
#> # A tibble: 1 x 11
#>   estimated_events mean_alpha mean_beta median_beta minimum_beta maximum_beta
#>   <int>          <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1      3960 -0.000017343  0.99751  1.0048  -0.0083393  2.1776
#>   mean_r_squared median_r_squared mean_sigma_AR minimum_observations
#>   <dbl>          <dbl>    <dbl>
#> 1      0.59659      0.62721  0.0079758
#>   maximum_observations
#>   <int>
#> 1      100

```

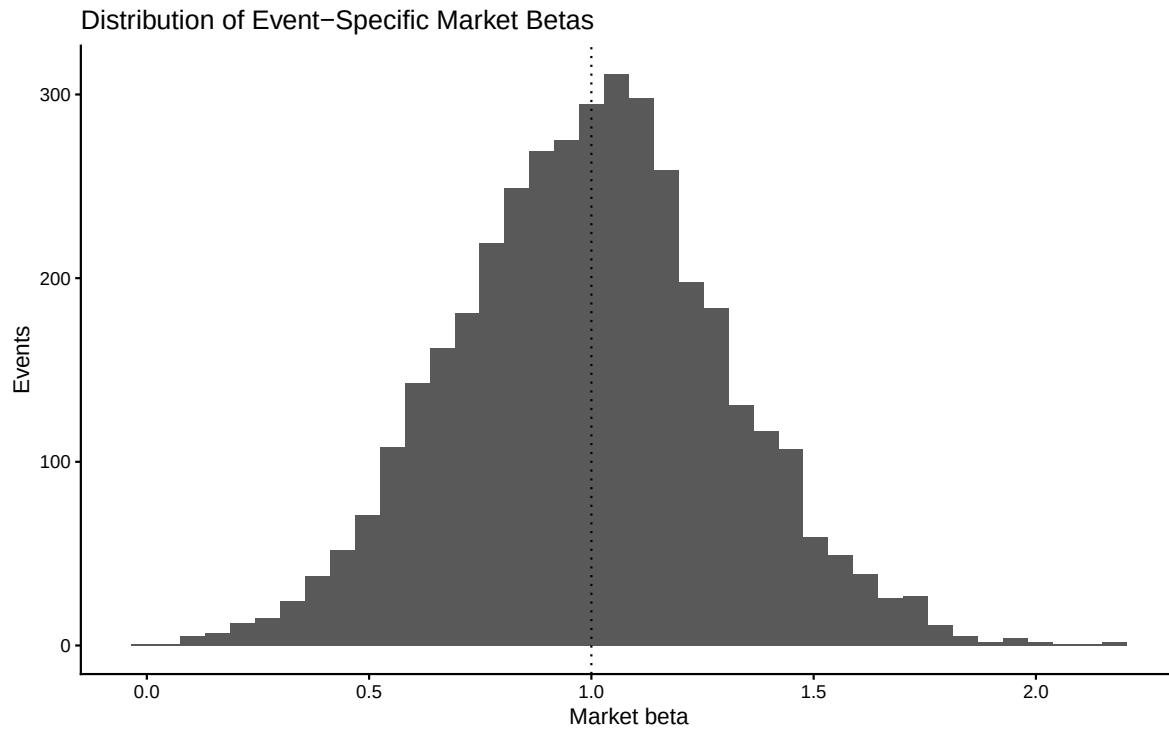
8.5.3 6.5.3

```

plot_market_beta <- tbl_market_model |>
  ggplot(aes(x = beta)) +
  geom_histogram(bins = 40) +
  geom_vline(
    xintercept = 1,
    linetype = "dotted"
  ) +
  labs(
    x = "Market beta",
    y = "Events",
    title = "Distribution of Event-Specific Market Betas"
  ) +
  theme_classic()

plot_market_beta

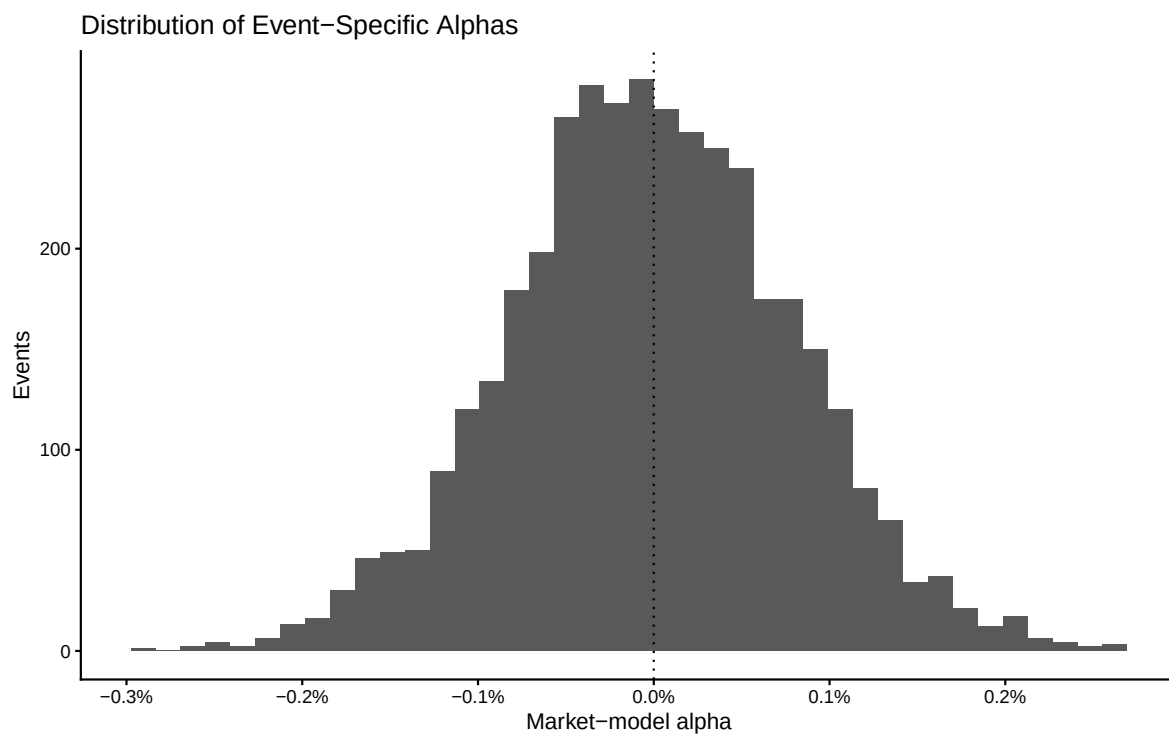
```



8.5.4 6.5.4

```
plot_market_alpha <- tbl_market_model |>
  ggplot(aes(x = alpha)) +
  geom_histogram(bins = 40) +
  geom_vline(
    xintercept = 0,
    linetype = "dotted"
  ) +
  scale_x_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Market-model alpha",
    y = "Events",
    title = "Distribution of Event-Specific Alphas"
  ) +
  theme_classic()
```

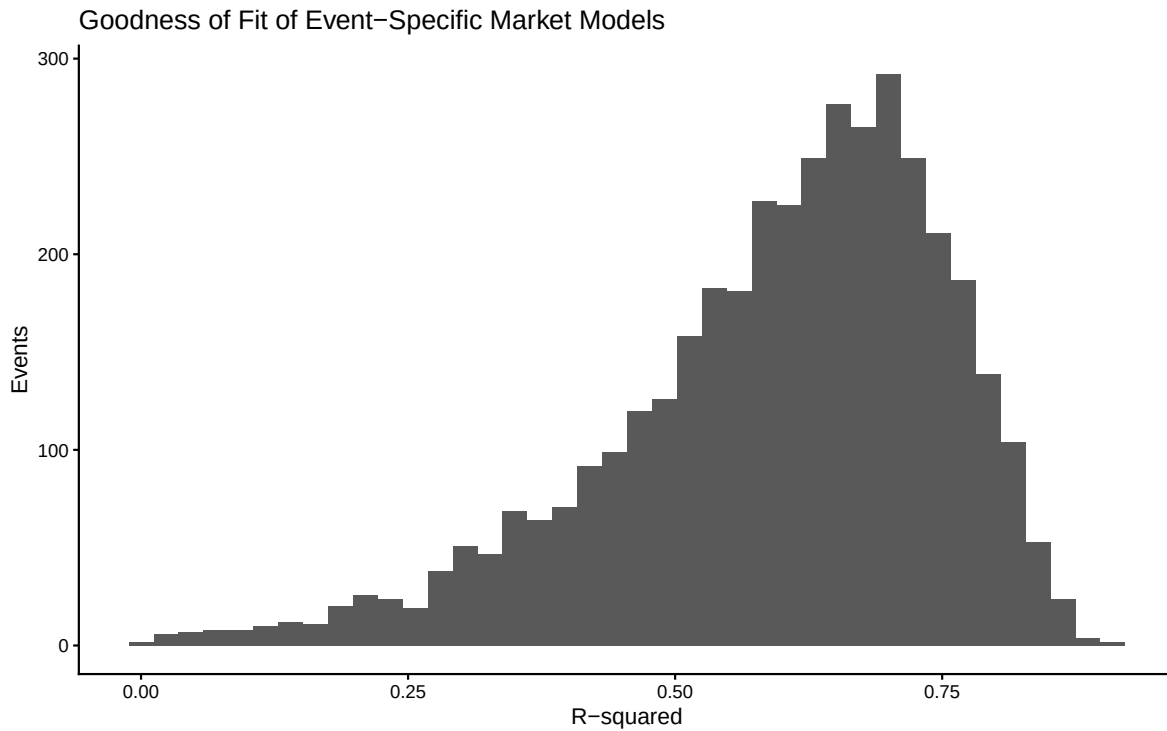
```
plot_market_alpha
```



8.5.5 6.5.5

```
plot_market_model_r_squared <- tbl_market_model |>
  ggplot(aes(x = r_squared)) +
  geom_histogram(bins = 40) +
  labs(
    x = "R-squared",
    y = "Events",
    title = "Goodness of Fit of Event-Specific Market Models"
  ) +
  theme_classic()

plot_market_model_r_squared
```



8.5.6 6.5.6

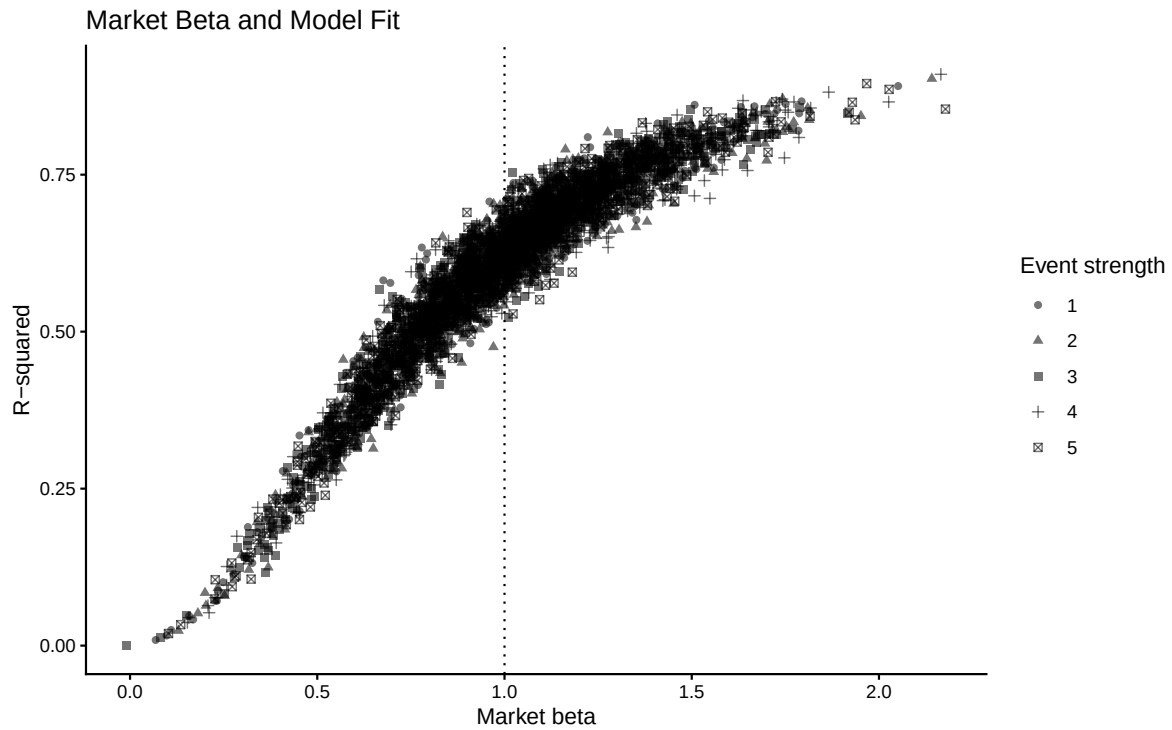
```
plot_beta_r_squared <- tbl_market_model |>
  ggplot(
    aes(
      x = beta,
      y = r_squared,
      shape = event_strength
    )
  ) +
  geom_point(alpha = 0.55) +
  geom_vline(
    xintercept = 1,
    linetype = "dotted"
  ) +
  labs(
    x = "Market beta",
    y = "R-squared",
    shape = "Event strength",
  )
```

```

    title = "Market Beta and Model Fit"
  ) +
  theme_classic()

plot_beta_r_squared

```



8.6 6.6

8.6.1 6.6.1 AR CAR

```

tbl_abnormal_return <- compute_abnormal_returns_LFL(
  tbl_event_panel = tbl_event_panel,
  tbl_market_model = tbl_market_model,
  event_start = event_start,
  event_end = event_end,
  require_complete_window = TRUE
)

```

```
tbl_abnormal_return |>
  select(
    event_ID,
    firm_ID,
    event_strength,
    relative_day,
    date,
    R,
    R_M,
    normal_return,
    AR,
    CAR
  ) |>
  slice_head(n = 20)
#> # A tibble: 20 x 10
#>   event_ID firm_ID event_strength relative_day date          R
#>   <int>   <int> <ord>          <int> <date>      <dbl>
#> 1       1       39 1          -30 2016-11-18 -0.0013155
#> 2       1       39 1          -29 2016-11-21  0.0037198
#> 3       1       39 1          -28 2016-11-22  0.021021
#> 4       1       39 1          -27 2016-11-24  0.0030893
#> 5       1       39 1          -26 2016-11-25 -0.0062645
#> 6       1       39 1          -25 2016-11-28 -0.0040561
#> 7       1       39 1          -24 2016-11-29  0.012325
#> 8       1       39 1          -23 2016-11-30 -0.0096488
#> 9       1       39 1          -22 2016-12-01  0.014048
#> 10      1       39 1          -21 2016-12-02  0.012192
#> 11      1       39 1          -20 2016-12-05  0.0097831
#> 12      1       39 1          -19 2016-12-06  0.026133
#> 13      1       39 1          -18 2016-12-07 -0.0044072
#> 14      1       39 1          -17 2016-12-08  0.015023
#> 15      1       39 1          -16 2016-12-09  0.0074484
#> 16      1       39 1          -15 2016-12-12  0.011444
#> 17      1       39 1          -14 2016-12-13 -0.011043
#> 18      1       39 1          -13 2016-12-14  0.019488
#> 19      1       39 1          -12 2016-12-15  0.0054953
#> 20      1       39 1          -11 2016-12-16 -0.0066719
#>           R_M normal_return          AR          CAR
#>           <dbl>          <dbl>          <dbl>          <dbl>
#> 1 -0.0014725 -0.0014642  0.00014876  0.00014876
#> 2 -0.0033404 -0.0028659  0.0065857  0.0067344
#> 3  0.017138  0.012501  0.0085199  0.015254
```

```
#> 4  0.015701    0.011423   -0.0083334   0.0069209
#> 5 -0.023346   -0.017878    0.011614    0.018535
#> 6 -0.0065350  -0.0052632    0.0012071    0.019742
#> 7 -0.0041437  -0.0034686    0.015794    0.035536
#> 8  0.0029259   0.0018364   -0.011485    0.024050
#> 9  0.00047329 -0.0000040664  0.014052    0.038102
#> 10 0.010758    0.0077138    0.0044779    0.042580
#> 11 -0.0054821  -0.0044730    0.014256    0.056836
#> 12 0.018634    0.013624    0.012509    0.069345
#> 13 -0.0073350  -0.0058634    0.0014562    0.070801
#> 14 0.027649    0.020389   -0.0053659    0.065435
#> 15 -0.0014362  -0.0014370    0.0088854    0.074321
#> 16 -0.0064222  -0.0051784    0.016623    0.090943
#> 17 -0.014330   -0.011113    0.000069896  0.091013
#> 18 0.022428    0.016471    0.0030171    0.094030
#> 19 0.018578    0.013581   -0.0080862    0.085944
#> 20 -0.0034403  -0.0029408   -0.0037311    0.082213
```

8.6.2 6.6.2 AAR CAAR

g N_g

$$AAR_{g,t} = \frac{1}{N_g} \sum_{i \in g} AR_{i,t}$$

$$CAAR_{g,t} = \frac{1}{N_g} \sum_{i \in g} CAR_{i,t}$$

```
significance_stars <- function(p_value) {
  case_when(
    is.na(p_value) ~ "",
    p_value < 0.01 ~ "***",
    p_value < 0.05 ~ "**",
    p_value < 0.10 ~ "*",
    TRUE ~ ""
  )
}
```

```

tbl_daily_statistics <- tbl_abnormal_return |>
  group_by(
    relative_day,
    event_strength
  ) |>
  summarise(
    events = n_distinct(event_ID),
    AAR = mean(AR),
    CAAR = mean(CAR),
    standard_deviation_AR = sd(AR),
    standard_deviation_CAR = sd(CAR),
    .groups = "drop"
  ) |>
  mutate(
    standard_error_AAR =
      standard_deviation_AR /
      sqrt(events),
    standard_error_CAAR =
      standard_deviation_CAR /
      sqrt(events),
    t_AAR = if_else(
      is.finite(standard_error_AAR) &
        standard_error_AAR > 0,
      AAR / standard_error_AAR,
      NA_real_
    ),
    t_CAAR = if_else(
      is.finite(standard_error_CAAR) &
        standard_error_CAAR > 0,
      CAAR / standard_error_CAAR,
      NA_real_
    ),
    p_AAR = if_else(
      is.finite(t_AAR),
      2 * pt(
        -abs(t_AAR),
        df = events - 1L
      ),
      NA_real_
    ),
    p_CAAR = if_else(
      is.finite(t_CAAR),

```



```

    2 * pt(
      -abs(t_CAAR),
      df = events - 1L
    ),
    NA_real_
  ),
  significance_AAR =
    significance_stars(p_AAR),
  significance_CAAR =
    significance_stars(p_CAAR)
) |>
arrange(
  event_strength,
  relative_day
)

tbl_daily_statistics
#> # A tibble: 305 x 15
#>   relative_day event_strength events      AAR      CAAR
#>   <int> <ord>      <int>    <dbl>    <dbl>
#> 1      -30 1          793 -0.00033279 -0.00033279
#> 2      -29 1          793  0.00034601  0.000013216
#> 3      -28 1          793 -0.000030469 -0.000017253
#> 4      -27 1          793  0.00024231  0.00022506
#> 5      -26 1          793  0.00042018  0.00064524
#> 6      -25 1          793  0.00020818  0.00085342
#> 7      -24 1          793  0.00053366  0.0013871
#> 8      -23 1          793 -0.000028460  0.0013586
#> 9      -22 1          793 -0.00014238  0.0012162
#> 10     -21 1          793 -0.00019155  0.0010247
#>   standard_deviation_AR standard_deviation_CAR standard_error_AAR
#>   <dbl>                <dbl>                <dbl>
#> 1      0.0079974        0.0079974        0.00028400
#> 2      0.0081183        0.011119         0.00028829
#> 3      0.0076763        0.013469         0.00027259
#> 4      0.0079520        0.015165         0.00028238
#> 5      0.0082483        0.017336         0.00029291
#> 6      0.0080764        0.019248         0.00028680
#> 7      0.0080576        0.020949         0.00028613
#> 8      0.0081628        0.022817         0.00028987
#> 9      0.0079747        0.024791         0.00028319
#> 10     0.0083095        0.026730         0.00029508

```

```

#>   standard_error_CAAR    t_AAR    t_CAAR    p_AAR    p_CAAR significance_AAR
#>   <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <chr>
#> 1      0.00028400 -1.1718   -1.1718   0.24163   0.24163   ""
#> 2      0.00039484  1.2002    0.033472  0.23042   0.97331   ""
#> 3      0.00047829 -0.11178  -0.036072  0.91103   0.97123   ""
#> 4      0.00053853  0.85808   0.41790   0.39111   0.67613   ""
#> 5      0.00061562  1.4345    1.0481    0.15181   0.29490   ""
#> 6      0.00068350  0.72588   1.2486    0.46812   0.21218   ""
#> 7      0.00074393  1.8651    1.8645    0.062543  0.062617  "*"
#> 8      0.00081025 -0.098181  1.6768    0.92181   0.093980  ""
#> 9      0.00088037 -0.50277   1.3815    0.61526   0.16751   ""
#> 10     0.00094920 -0.64913   1.0795    0.51644   0.28068   ""
#>   significance_CAAR
#>   <chr>
#> 1 ""
#> 2 ""
#> 3 ""
#> 4 ""
#> 5 ""
#> 6 ""
#> 7 "*"
#> 8 "*"
#> 9 ""
#> 10 ""
#> # i 295 more rows

```

8.6.3 6.6.3 CAAR

```

plot_caar_by_strength <- tbl_daily_statistics |>
  ggplot(
    aes(
      x = relative_day,
      y = CAAR,
      linetype = event_strength
    )
  ) +
  annotate(
    "rect",
    xmin = -1,
    xmax = 1,

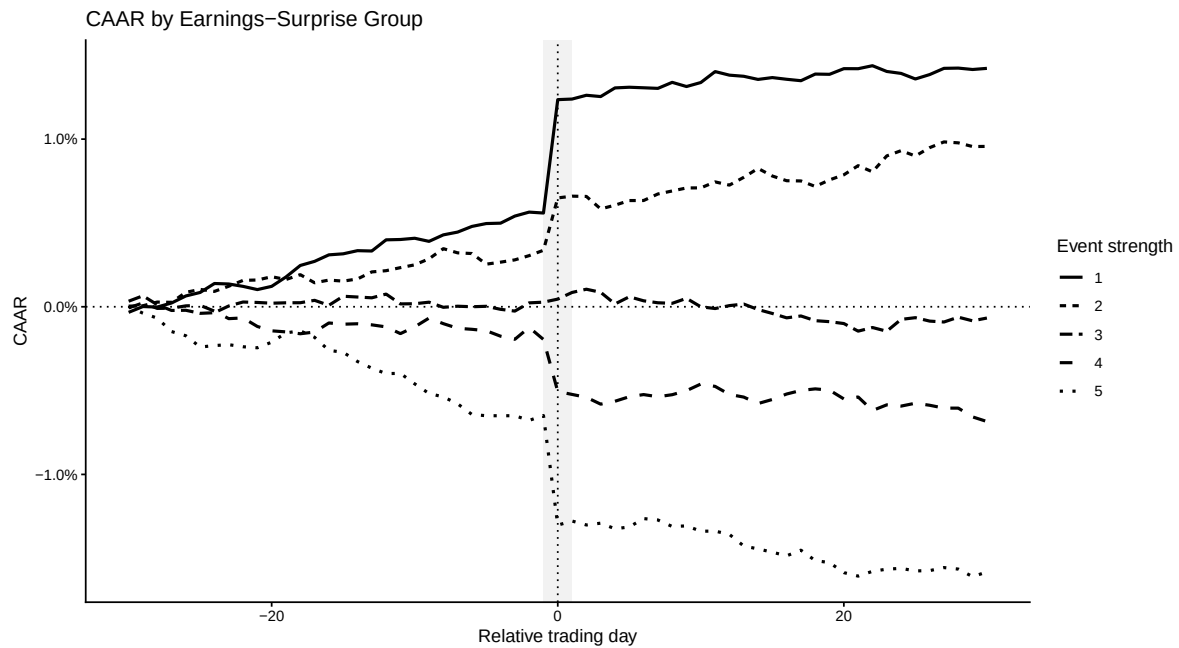
```

```

    ymin = -Inf,
    ymax = Inf,
    alpha = 0.08
) +
geom_line(linewidth = 0.9) +
geom_hline(
  yintercept = 0,
  linetype = "dotted"
) +
geom_vline(
  xintercept = 0,
  linetype = "dotted"
) +
scale_y_continuous(
  labels = scales::label_percent()
) +
labs(
  x = "Relative trading day",
  y = "CAAR",
  linetype = "Event strength",
  title = "CAAR by Earnings-Surprise Group"
) +
theme_classic()

plot_caar_by_strength

```



8.6.4 6.6.4 AAR

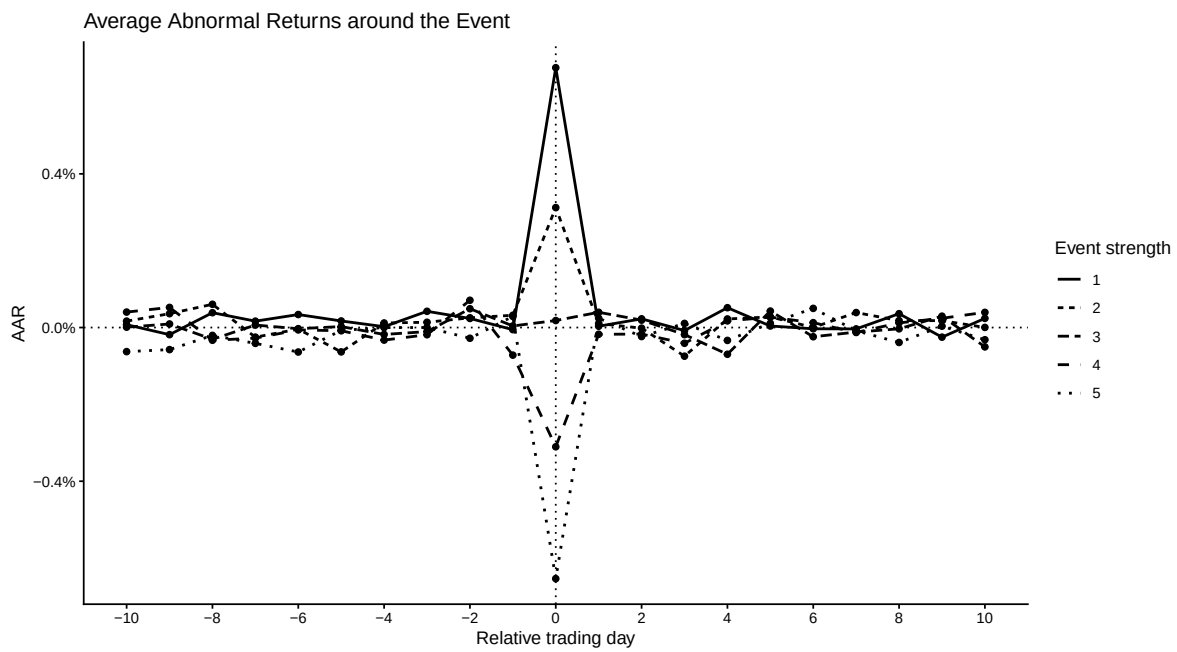
```
plot_aar_around_event <- tbl_daily_statistics |>
  filter(
    between(
      relative_day,
      -10L,
      10L
    )
  ) |>
  ggplot(
    aes(
      x = relative_day,
      y = AAR,
      linetype = event_strength
    )
  ) +
  geom_line(linewidth = 0.8) +
  geom_point() +
  geom_vline(
    xintercept = 0,
    linetype = "dotted"
  )
```

```

) +
geom_hline(
  yintercept = 0,
  linetype = "dotted"
) +
scale_x_continuous(
  breaks = seq(-10, 10, 2)
) +
scale_y_continuous(
  labels = scales::label_percent()
) +
labs(
  x = "Relative trading day",
  y = "AAR",
  linetype = "Event strength",
  title = "Average Abnormal Returns around the Event"
) +
theme_classic()

plot_aar_around_event

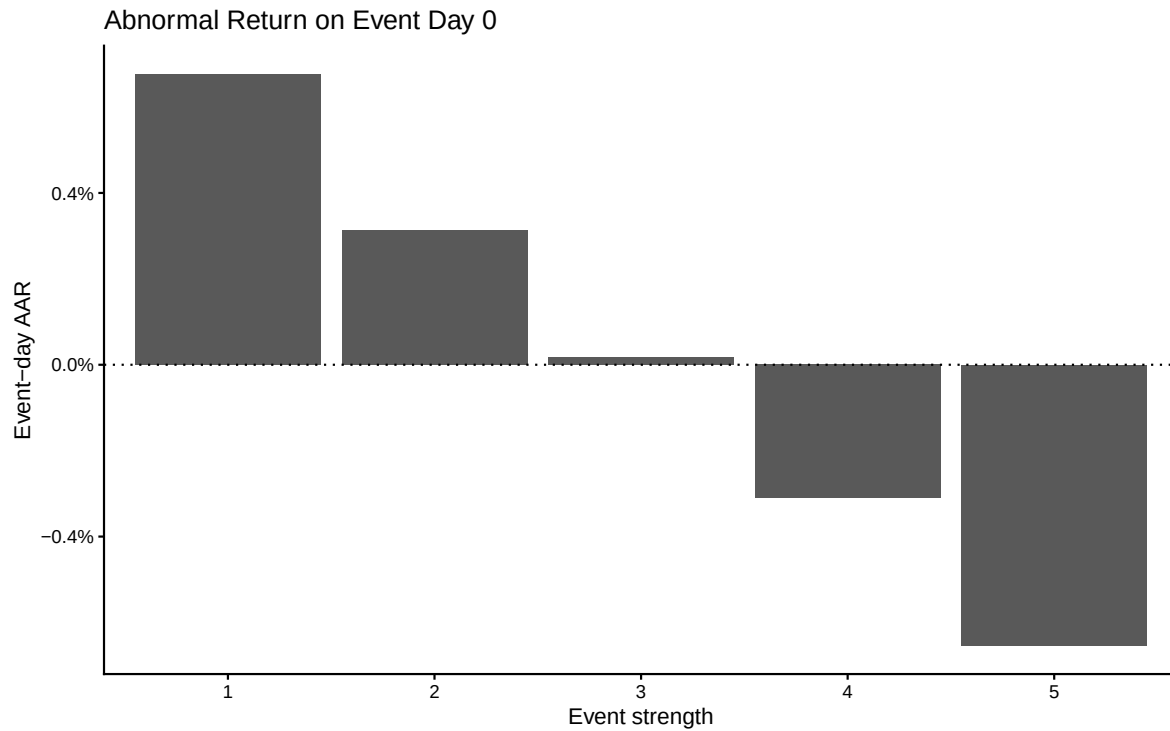
```



8.6.5 6.6.5 0 AAR

```
plot_event_day_aar <- tbl_daily_statistics |>
  filter(relative_day == 0L) |>
  ggplot(
    aes(
      x = event_strength,
      y = AAR
    )
  ) +
  geom_col() +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Event strength",
    y = "Event-day AAR",
    title = "Abnormal Return on Event Day 0"
  ) +
  theme_classic()

plot_event_day_aar
```



8.6.6 6.6.6 CAAR

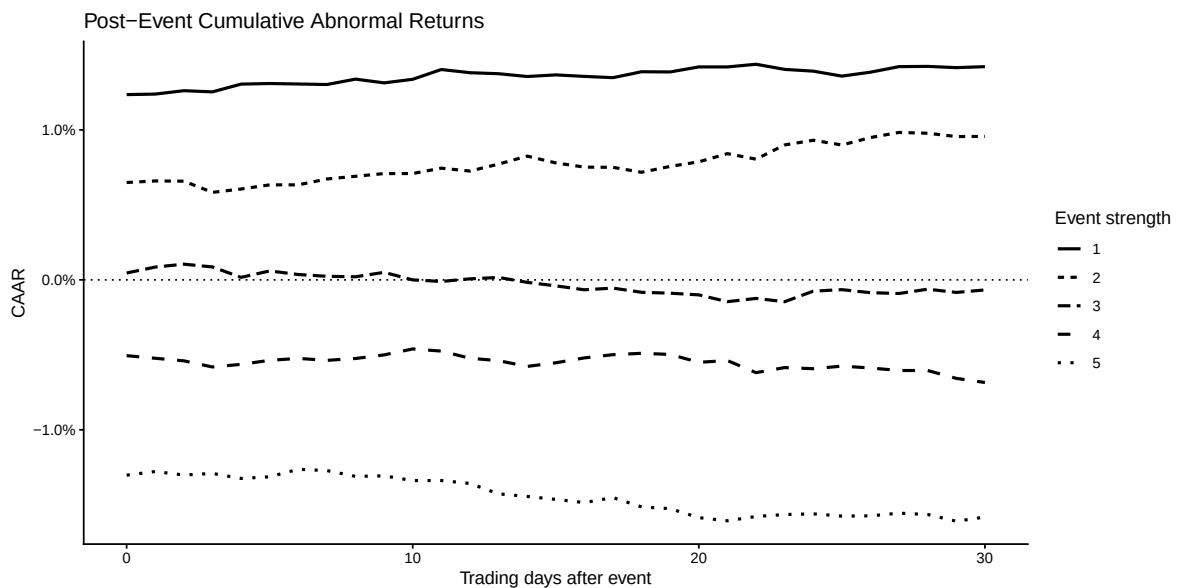
```
plot_post_event_caar <- tbl_daily_statistics |>
  filter(
    between(
      relative_day,
      0L,
      30L
    )
  ) |>
  ggplot(
    aes(
      x = relative_day,
      y = CAAR,
      linetype = event_strength
    )
  ) +
  geom_line(linewidth = 0.9) +
  geom_hline(
```

```

    yintercept = 0,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Trading days after event",
    y = "CAAR",
    linetype = "Event strength",
    title = "Post-Event Cumulative Abnormal Returns"
  ) +
  theme_classic()

plot_post_event_caar

```



8.6.7 6.6.7 AR

```

plot_ar_distribution_by_strength <- tbl_abnormal_return |>
  filter(
    between(
      relative_day,

```

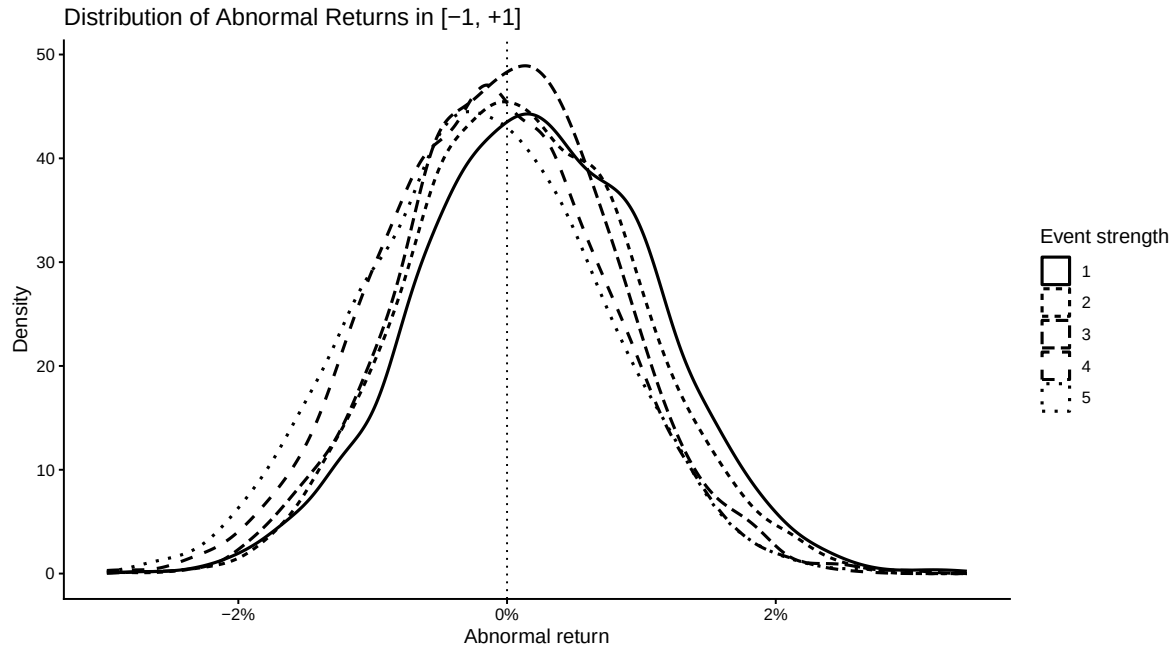


```

    -1L,
    1L
  )
) |>
ggplot(
  aes(
    x = AR,
    linetype = event_strength
  )
) +
geom_density(linewidth = 0.8) +
geom_vline(
  xintercept = 0,
  linetype = "dotted"
) +
scale_x_continuous(
  labels = scales::label_percent()
) +
labs(
  x = "Abnormal return",
  y = "Density",
  linetype = "Event strength",
  title = "Distribution of Abnormal Returns in [-1, +1]"
) +
theme_classic()

plot_ar_distribution_by_strength

```



8.7 6.7

8.7.1 6.7.1

```
tbl_summary_window <- tribble(
  ~window_ID, ~window_label, ~start_day, ~end_day,
  "m1_p1", "[-1, +1]", -1L, 1L,
  "0_p1", "[0, +1]", 0L, 1L,
  "m5_p5", "[-5, +5]", -5L, 5L,
  "m10_p10", "[-10, +10]", -10L, 10L,
  "m30_p30", "[-30, +30]", -30L, 30L
)
```

```
tbl_summary_window
#> # A tibble: 5 x 4
#>   window_ID window_label start_day end_day
#>   <chr>      <chr>         <int>  <int>
#> 1 m1_p1     [-1, +1]           -1      1
#> 2 0_p1      [0, +1]            0      1
#> 3 m5_p5     [-5, +5]          -5      5
```

```
#> 4 m10_p10 [-10, +10] -10 10
#> 5 m30_p30 [-30, +30] -30 30
```

8.7.2 6.7.2 CAR

```
tbl_event_window_return <-
  compute_event_window_returns_LFL(
    tbl_abnormal_return = tbl_abnormal_return,
    tbl_window = tbl_summary_window
  )

tbl_event_window_return |>
  slice_head(n = 20)
#> # A tibble: 20 x 14
#>   event_ID firm_ID event_date event_trade_date event_year event_strength
#>   <int>   <int> <date>      <date>          <dbl> <ord>
#> 1     1     1     39 2017-01-04 2017-01-05      2017 1
#> 2     2     2     57 2017-01-04 2017-01-05      2017 1
#> 3     3     3     73 2017-01-04 2017-01-05      2017 4
#> 4     4     4    109 2017-01-04 2017-01-05      2017 3
#> 5     5     5    111 2017-01-04 2017-01-05      2017 3
#> 6     6     6    231 2017-01-04 2017-01-05      2017 1
#> 7     7     7    944 2017-01-04 2017-01-05      2017 2
#> 8     8     8   1227 2017-01-04 2017-01-05      2017 5
#> 9     9     9    422 2017-01-05 2017-01-06      2017 3
#> 10    10    10   1203 2017-01-05 2017-01-06      2017 5
#> 11    11    11    346 2017-01-06 2017-01-10      2017 2
#> 12    12    12    590 2017-01-06 2017-01-10      2017 1
#> 13    13    13    644 2017-01-06 2017-01-10      2017 2
#> 14    14    14   1153 2017-01-06 2017-01-10      2017 4
#> 15    15    15    191 2017-01-10 2017-01-11      2017 4
#> 16    16    16    353 2017-01-10 2017-01-11      2017 1
#> 17    17    17    566 2017-01-10 2017-01-11      2017 3
#> 18    18    18    322 2017-01-11 2017-01-12      2017 3
#> 19    19    19   1050 2017-01-11 2017-01-12      2017 4
#> 20    20    20    426 2017-01-12 2017-01-13      2017 5
#>   earnings_surprise window_ID window_label start_day end_day expected_days
#>   <dbl> <chr>      <chr>          <int>   <int>      <int>
#> 1   -0.0081327 0_p1      [0, +1]          0     1          2
#> 2   -0.010049 0_p1      [0, +1]          0     1          2
```

```

#> 3      -0.0020232  0_p1      [0, +1]      0      1      2
#> 4      -0.0043961  0_p1      [0, +1]      0      1      2
#> 5      -0.0044873  0_p1      [0, +1]      0      1      2
#> 6      -0.014399   0_p1      [0, +1]      0      1      2
#> 7      -0.0068979  0_p1      [0, +1]      0      1      2
#> 8      -0.0011248  0_p1      [0, +1]      0      1      2
#> 9      -0.0048408  0_p1      [0, +1]      0      1      2
#> 10     -0.00086568 0_p1      [0, +1]      0      1      2
#> 11     -0.0058582  0_p1      [0, +1]      0      1      2
#> 12     -0.010344   0_p1      [0, +1]      0      1      2
#> 13     -0.0064889  0_p1      [0, +1]      0      1      2
#> 14     -0.0021003  0_p1      [0, +1]      0      1      2
#> 15     -0.0034658  0_p1      [0, +1]      0      1      2
#> 16     -0.010249   0_p1      [0, +1]      0      1      2
#> 17     -0.0043876  0_p1      [0, +1]      0      1      2
#> 18     -0.0046897  0_p1      [0, +1]      0      1      2
#> 19     -0.0024991  0_p1      [0, +1]      0      1      2
#> 20     -0.0010089  0_p1      [0, +1]      0      1      2
#>      CAR_window observed_days
#>      <dbl>      <int>
#> 1 -0.0065537      2
#> 2 -0.0015993      2
#> 3 -0.0081648      2
#> 4  0.00033339      2
#> 5 -0.0011367      2
#> 6  0.034678        2
#> 7  0.0043073        2
#> 8 -0.015256        2
#> 9 -0.011574        2
#> 10 0.0037716        2
#> 11 0.023263         2
#> 12 0.019180         2
#> 13 -0.00048559      2
#> 14 0.000067580      2
#> 15 0.013548         2
#> 16 0.0086222        2
#> 17 -0.016704        2
#> 18 -0.011378        2
#> 19 -0.013969        2
#> 20 -0.0089423       2

```

8.7.3 6.7.3 CAR

```
tbl_event_window_summary <- tbl_event_window_return |>
  group_by(
    window_ID,
    window_label,
    start_day,
    end_day,
    event_strength
  ) |>
  summarise(
    events = n_distinct(event_ID),
    mean_CAR = mean(CAR_window),
    median_CAR = median(CAR_window),
    standard_deviation_CAR =
      sd(CAR_window),
    positive_CAR_ratio =
      mean(CAR_window > 0),
    .groups = "drop"
  ) |>
  mutate(
    standard_error_CAR =
      standard_deviation_CAR /
      sqrt(events),
    t_value = if_else(
      is.finite(standard_error_CAR) &
        standard_error_CAR > 0,
      mean_CAR /
        standard_error_CAR,
      NA_real_
    ),
    p_value = if_else(
      is.finite(t_value),
      2 * pt(
        -abs(t_value),
        df = events - 1L
      ),
      NA_real_
    ),
    significance =
      significance_stars(p_value)
  ) |>
```

```

arrange(
  window_ID,
  event_strength
)

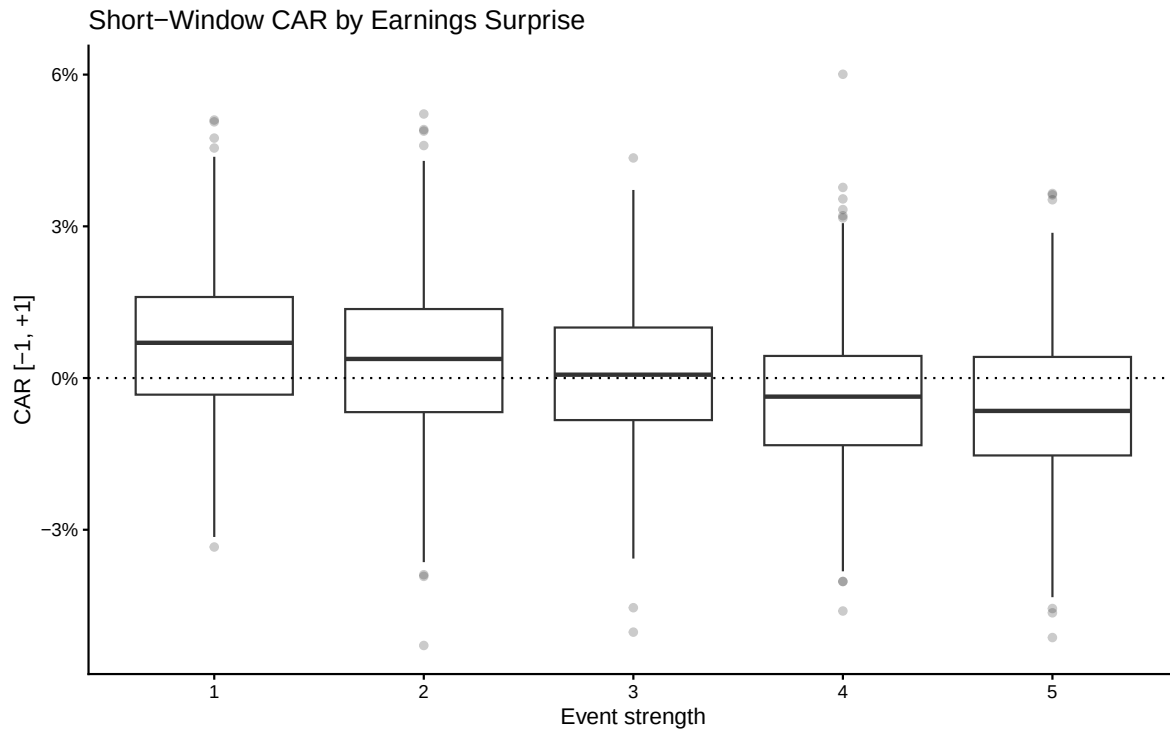
tbl_event_window_summary
#> # A tibble: 25 x 14
#>   window_ID window_label start_day end_day event_strength events mean_CAR
#>   <chr>      <chr>      <int>  <int> <ord>      <int>    <dbl>
#> 1 0_p1      [0, +1]         0      1 1          793  0.0067977
#> 2 0_p1      [0, +1]         0      1 2          793  0.0032266
#> 3 0_p1      [0, +1]         0      1 3          792  0.00057929
#> 4 0_p1      [0, +1]         0      1 4          791 -0.0032795
#> 5 0_p1      [0, +1]         0      1 5          791 -0.0062931
#> 6 m10_p10  [-10, +10]        -10     10 1          793  0.0093638
#> 7 m10_p10  [-10, +10]        -10     10 2          793  0.0047579
#> 8 m10_p10  [-10, +10]        -10     10 3          792 -0.00017347
#> 9 m10_p10  [-10, +10]        -10     10 4          791 -0.0030068
#> 10 m10_p10 [-10, +10]        -10     10 5          791 -0.0094161
#>   median_CAR standard_deviation_CAR positive_CAR_ratio standard_error_CAR
#>   <dbl>          <dbl>          <dbl>          <dbl>
#> 1  0.0070450      0.011741      0.72383      0.00041695
#> 2  0.0037860      0.011942      0.60025      0.00042406
#> 3  0.00060199     0.011135      0.52146      0.00039566
#> 4 -0.0030695     0.011171      0.39064      0.00039718
#> 5 -0.0060704     0.011668      0.29709      0.00041485
#> 6  0.0086618     0.040511      0.59016      0.0014386
#> 7  0.0049804     0.042086      0.55485      0.0014945
#> 8  0.00040740    0.040214      0.50758      0.0014289
#> 9 -0.0031404     0.039484      0.47914      0.0014039
#> 10 -0.011686     0.039978      0.39697      0.0014215
#>   t_value      p_value significance
#>   <dbl>      <dbl> <chr>
#> 1  16.303    9.6026e-52 "****"
#> 2   7.6087   7.8622e-14 "****"
#> 3   1.4641   1.4356e- 1 ""
#> 4  -8.2568   6.2656e-16 "****"
#> 5 -15.170    8.3233e-46 "****"
#> 6   6.5091   1.3410e-10 "****"
#> 7   3.1836   1.5114e- 3 "****"
#> 8  -0.12140 9.0341e- 1 ""
#> 9  -2.1417   3.2520e- 2 "***"

```

```
#> 10  -6.6243  6.4515e-11  "***"  
#> # i 15 more rows
```

8.7.4 6.7.4 CAR

```
plot_short_window_car <- tbl_event_window_return |>  
  filter(window_ID == "m1_p1") |>  
  ggplot(  
    aes(  
      x = event_strength,  
      y = CAR_window  
    )  
  ) +  
  geom_boxplot(  
    outlier.alpha = 0.25  
  ) +  
  geom_hline(  
    yintercept = 0,  
    linetype = "dotted"  
  ) +  
  scale_y_continuous(  
    labels = scales::label_percent()  
  ) +  
  labs(  
    x = "Event strength",  
    y = "CAR [-1, +1]",  
    title = "Short-Window CAR by Earnings Surprise"  
  ) +  
  theme_classic()  
  
plot_short_window_car
```



8.7.5 6.7.5 CAR

```
plot_window_mean_car <- tbl_event_window_summary |>
  ggplot(
    aes(
      x = event_strength,
      y = mean_CAR,
      fill = window_label
    )
  ) +
  geom_col(
    position = "dodge"
  ) +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  )
```

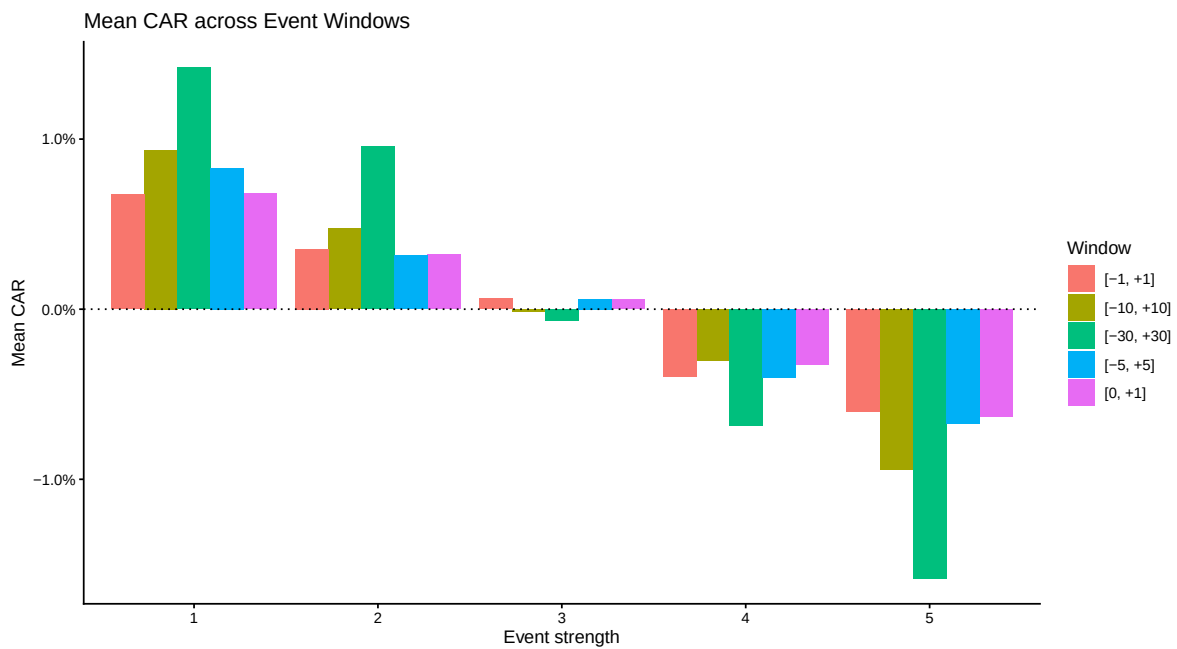


```

) +
labs(
  x = "Event strength",
  y = "Mean CAR",
  fill = "Window",
  title = "Mean CAR across Event Windows"
) +
theme_classic()

plot_window_mean_car

```



8.7.6 6.7.6 CAR

```

plot_positive_car_ratio <- tbl_event_window_summary |>
ggplot(
  aes(
    x = event_strength,
    y = positive_CAR_ratio,
    fill = window_label
  )
) +

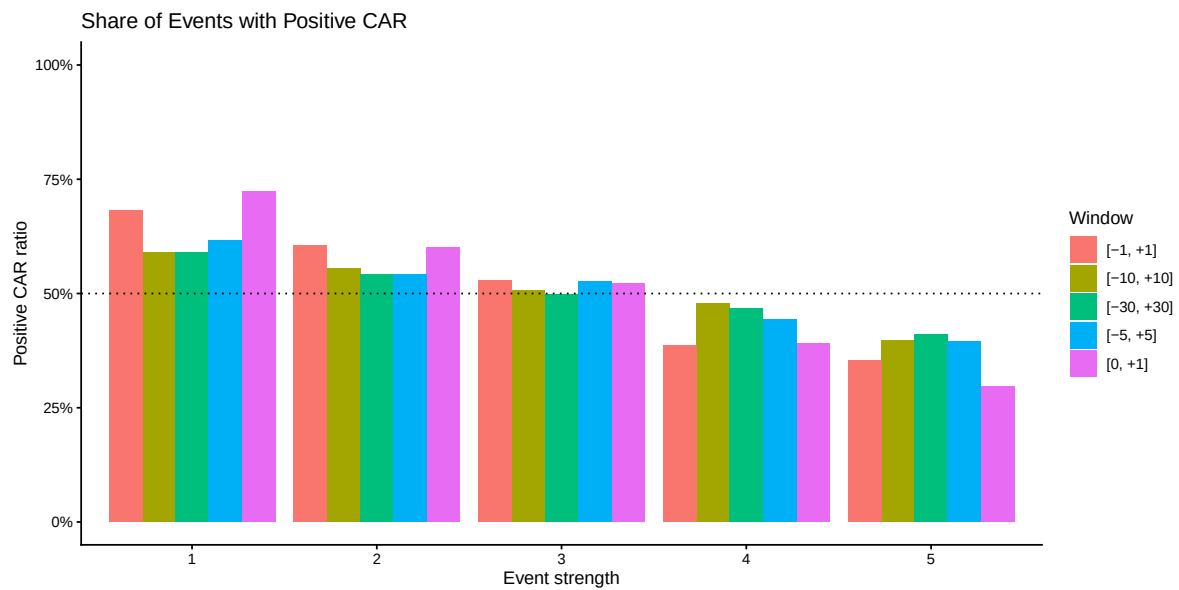
```

```

geom_col(
  position = "dodge"
) +
geom_hline(
  yintercept = 0.5,
  linetype = "dotted"
) +
scale_y_continuous(
  labels = scales::label_percent(),
  limits = c(0, 1)
) +
labs(
  x = "Event strength",
  y = "Positive CAR ratio",
  fill = "Window",
  title = "Share of Events with Positive CAR"
) +
theme_classic()

plot_positive_car_ratio

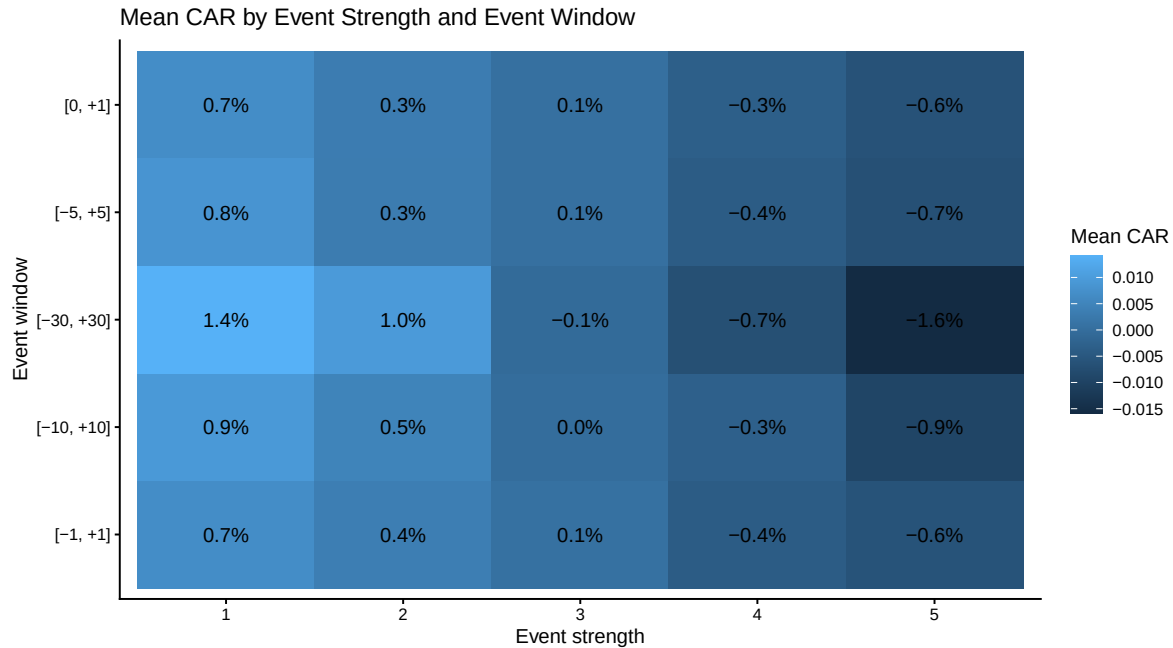
```



8.7.7 6.7.7 CAR

```
plot_window_car_heatmap <- tbl_event_window_summary |>
  ggplot(
    aes(
      x = event_strength,
      y = window_label,
      fill = mean_CAR
    )
  ) +
  geom_tile() +
  geom_text(
    aes(
      label = scales::percent(
        mean_CAR,
        accuracy = 0.1
      )
    )
  ) +
  labs(
    x = "Event strength",
    y = "Event window",
    fill = "Mean CAR",
    title = "Mean CAR by Event Strength and Event Window"
  ) +
  theme_classic()

plot_window_car_heatmap
```



8.8 6.8 CAR

8.8.1 6.8.1

```
tbl_surprise_regression <- tbl_event_window_return |>
  filter(
    window_ID == "m1_p1",
    is.finite(earnings_surprise),
    is.finite(CAR_window)
  )

model_surprise_car <- lm(
  CAR_window ~ earnings_surprise,
  data = tbl_surprise_regression
)

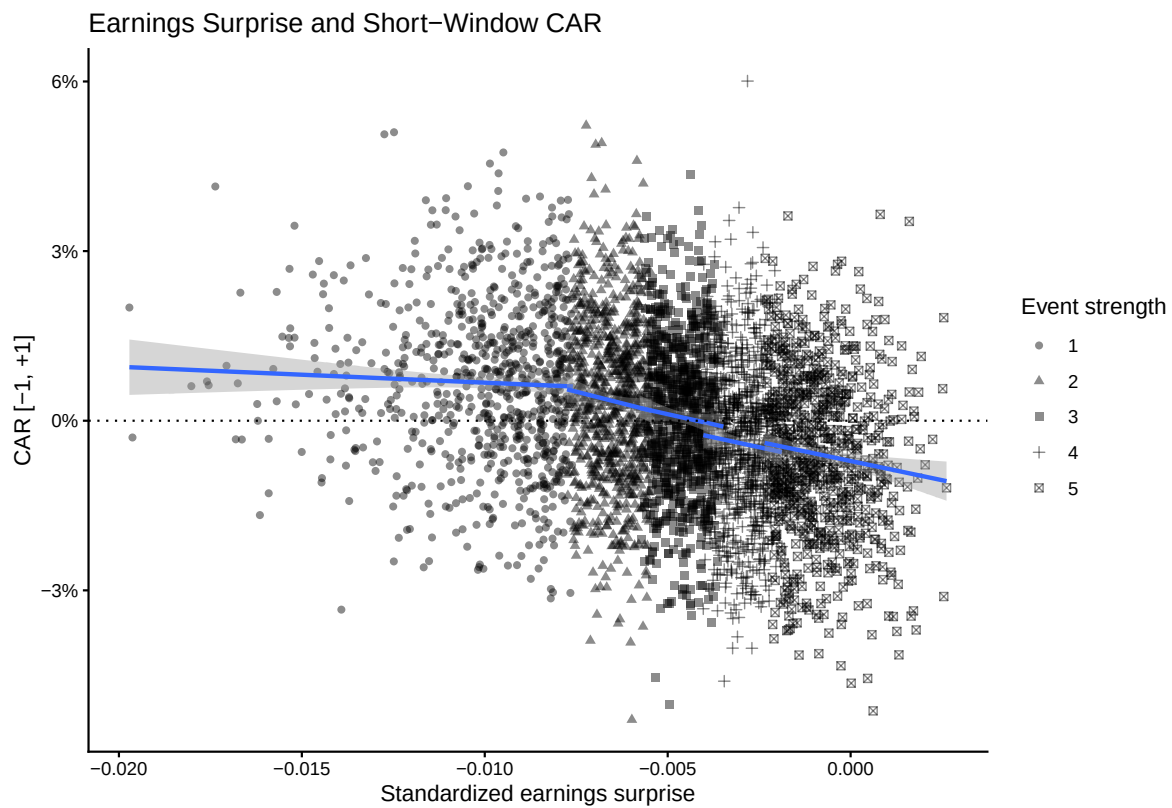
broom::tidy(model_surprise_car)
#> # A tibble: 2 x 5
#>   term                estimate std.error statistic    p.value
#>   <chr>                <dbl>    <dbl>    <dbl>    <dbl>
#> 1 (Intercept)        -0.0066774 0.00041568  -16.064 2.6646e-56
```

```
#> 2 earnings_surprise -1.3714 0.069096 -19.848 1.1933e-83
broom::glance(model_surprise_car)
#> # A tibble: 1 x 12
#>   r.squared adj.r.squared sigma statistic p.value df logLik AIC
#>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
#> 1 0.090519 0.090289 0.014511 393.93 1.1933e-83 1 11144. -22282.
#>   BIC deviance df.residual nobs
#>   <dbl> <dbl> <int> <int>
#> 1 -22263. 0.83344 3958 3960
```

8.8.2 6.8.2 CAR

```
plot_surprise_car <- tbl_surprise_regression |>
  ggplot(
    aes(
      x = earnings_surprise,
      y = CAR_window,
      shape = event_strength
    )
  ) +
  geom_point(alpha = 0.45) +
  geom_smooth(
    method = "lm",
    se = TRUE
  ) +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Standardized earnings surprise",
    y = "CAR [-1, +1]",
    shape = "Event strength",
    title = "Earnings Surprise and Short-Window CAR"
  ) +
  theme_classic()
```

```
plot_surprise_car
```



8.8.3 6.8.3

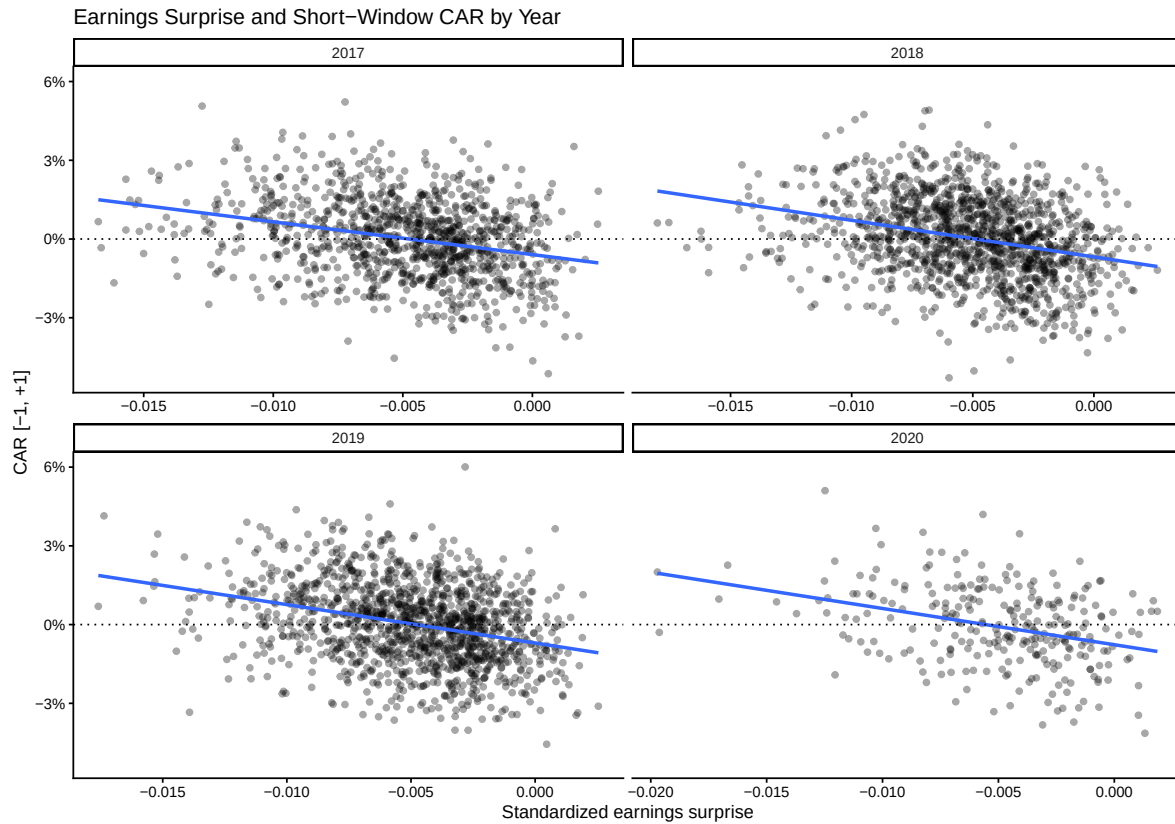
```
plot_surprise_car_by_year <- tbl_surprise_regression |>
  ggplot(
    aes(
      x = earnings_surprise,
      y = CAR_window
    )
  ) +
  geom_point(alpha = 0.35) +
  geom_smooth(
    method = "lm",
    se = FALSE
  ) +
```

```

facet_wrap(
  vars(event_year),
  scales = "free_x"
) +
geom_hline(
  yintercept = 0,
  linetype = "dotted"
) +
scale_y_continuous(
  labels = scales::label_percent()
) +
labs(
  x = "Standardized earnings surprise",
  y = "CAR [-1, +1]",
  title = "Earnings Surprise and Short-Window CAR by Year"
) +
theme_classic()

plot_surprise_car_by_year

```



8.9 6.9

8.9.1 6.9.1

```
tbl_event_window_wide <- tbl_event_window_return |>
  select(
    event_ID,
    window_ID,
    CAR_window
  ) |>
  pivot_wider(
    names_from = window_ID,
    values_from = CAR_window,
    names_prefix = "CAR_"
  )
```



```
tbl_event_output <- tbl_market_model |>
  semi_join(
    tbl_abnormal_return |>
      distinct(event_ID),
    by = "event_ID"
  ) |>
  left_join(
    tbl_event_window_wide,
    by = "event_ID"
  ) |>
  arrange(event_ID)
```

8.9.2 6.9.2

```
tbl_event_daily_output <- tbl_abnormal_return |>
  select(
    event_ID,
    firm_ID,
    event_date,
    event_trade_date,
    event_year,
    event_strength,
    earnings_surprise,
    relative_day,
    date,
    R,
    R_M,
    normal_return,
    AR,
    CAR,
    alpha,
    beta,
    sigma_AR,
    r_squared,
    estimation_observations
  )

tbl_event_summary_output <- tbl_daily_statistics |>
  arrange(
    event_strength,
```

```
    relative_day
  )
}
```

8.9.3 6.9.3 Parquet

```
list_output <- list(
  event_results = tbl_event_output,
  event_daily = tbl_event_daily_output,
  event_summary = tbl_event_summary_output
)

vec_output_file <- c(
  event_results = file.path(
    "events",
    "event_study_results.parquet"
  ),
  event_daily = file.path(
    "events",
    "event_study_daily.parquet"
  ),
  event_summary = file.path(
    "events",
    "event_study_summary.parquet"
  )
)

vec_output_path <- purrr::imap_chr(
  list_output,
  \(tbl_output, output_name)
  write_derived_parquet_file_LFL(
    tbl_output,
    vec_output_file[[output_name]]
  )
)

tibble(
  output_name = names(vec_output_path),
  path = unname(vec_output_path)
)
#> # A tibble: 3 x 2
```

```

#>   output_name
#>   <chr>
#> 1 event_results
#> 2 event_daily
#> 3 event_summary
#>   path
#>   <chr>
#> 1 C:/AnalyticFin/Projects/LagrangeFinance/data/derived/events/event_study_resul~
#> 2 C:/AnalyticFin/Projects/LagrangeFinance/data/derived/events/event_study_daily~
#> 3 C:/AnalyticFin/Projects/LagrangeFinance/data/derived/events/event_study_summa~

```

8.9.4 6.9.4

```

tbl_output_verification <- purrr::imap_dfr(
  list_output,
  \(tbl_expected, output_name) {
    output_file <- vec_output_file[[output_name]]
    output_path <- vec_output_path[[output_name]]

    tbl_check <-
      read_derived_parquet_file_LFL(
        output_file
      )

    stopifnot(
      nrow(tbl_check) ==
        nrow(tbl_expected)
    )

    stopifnot(
      identical(
        names(tbl_check),
        names(tbl_expected)
      )
    )

    tibble(
      output_name = output_name,
      output_file = output_path,
      rows = nrow(tbl_check),

```

```

        columns = ncol(tbl_check),
        size_bytes =
            file.info(output_path)$size
    )
}
)

tbl_output_verification
#> # A tibble: 3 x 5
#>   output_name
#>   <chr>
#> 1 event_results
#> 2 event_daily
#> 3 event_summary
#>   output_file
#>   <chr>
#> 1 C:/AnalyticFin/Projects/LagrangeFinance/data/derived/events/event_study_resul~
#> 2 C:/AnalyticFin/Projects/LagrangeFinance/data/derived/events/event_study_daily~
#> 3 C:/AnalyticFin/Projects/LagrangeFinance/data/derived/events/event_study_summa~
#>   rows columns size_bytes
#>   <int>   <int>     <dbl>
#> 1   3960     18    467503
#> 2 241560     19    9222492
#> 3    305     15     33564

```

8.10 6.10

```

0
130 31 100 30 30 61
CAR CAR AAR CAAR CAR -
purr 0 AR CAR utility_LFL.R
3

```

```

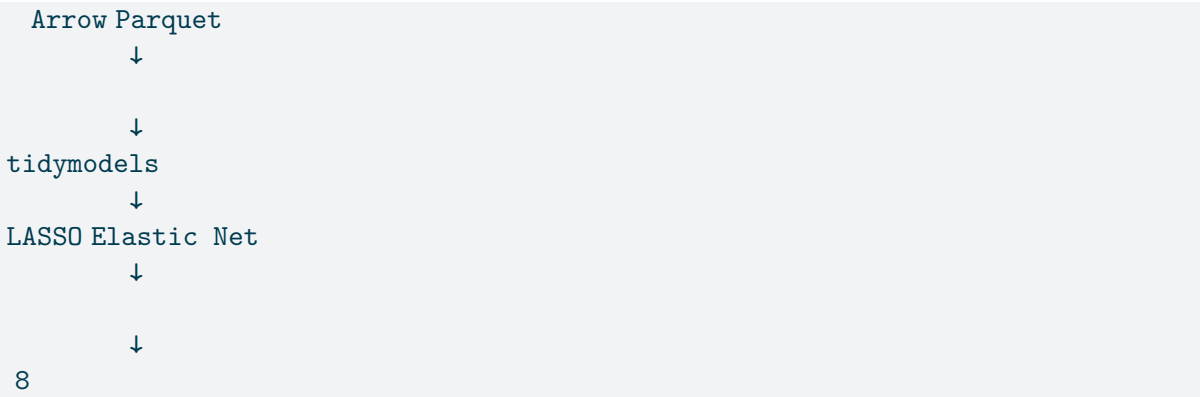
data/derived/events/event_study_results.parquet
data/derived/events/event_study_daily.parquet
data/derived/events/event_study_summary.parquet

```

9 I-7

2 6

tidymodels



9.1 7.1

4 SMB HML 5

LASSO 0

$$\min_{\beta} \left[\frac{1}{N} \sum_{i=1}^N (y_i - x_i^\top \beta)^2 + \lambda \sum_{j=1}^p |\beta_j| \right]$$

0

9.2 7.2

9.2.1

```
tbl_financial <- read_derived_parquet_file_LFL(  
  file.path(  
    "fundamentals",  
    "annual_firm_characteristics.parquet"  
  )  
)  
  
tbl_monthly <- read_derived_parquet_file_LFL(  
  file.path(  
    "returns",  
    "monthly_stock_returns.parquet"  
  )  
)  
  
tbl_factor <- read_derived_parquet_file_LFL(  
  file.path(  
    "factors",  
    "fama_french_factors.parquet"  
  )  
)
```

```
tbl_input_overview <- tribble(  
  ~data, ~observations, ~variables,  
  "annual_firm_characteristics",  
  nrow(tbl_financial),  
  ncol(tbl_financial),  
  "monthly_stock_returns",  
  nrow(tbl_monthly),  
  ncol(tbl_monthly),  
  "fama_french_factors",  
  nrow(tbl_factor),  
  ncol(tbl_factor)  
)  
  
show_table_LFL(tbl_input_overview)  
#> # A tibble: 3 x 3  
#>   data observations variables
```

#>	<chr>	<int>	<int>
#> 1	annual_firm_characteristics	7920	31
#> 2	monthly_stock_returns	95040	39
#> 3	fama_french_factors	60	16

9.2.2

$t \quad t + 1$

$$y_{i,t} = R_{i,t+1}^e$$

```
tbl_target <- tbl_financial |>
  mutate(
    firm_ID = as.integer(firm_ID),
    year = as.integer(year)
  ) |>
  arrange(firm_ID, year) |>
  group_by(firm_ID) |>
  mutate(
    forward_excess_return = lead(Re),
    prediction_year = year + 1L
  ) |>
  ungroup()
```

9.2.3

```
tbl_price_features <- tbl_monthly |>
  mutate(
    firm_ID = as.integer(firm_ID),
    year = as.integer(year)
  ) |>
  filter(is.finite(R)) |>
  group_by(firm_ID, year) |>
  summarise(
    momentum = prod(1 + R) - 1,
    volatility = sd(R),
    downside_risk = sqrt(mean(pmin(R, 0)^2)),
    average_market_equity = mean(ME, na.rm = TRUE),
```

```
.groups = "drop"  
)
```

9.2.4

```
tbl_market_features <- tbl_factor |>  
  group_by(year) |>  
  summarise(  
    market_premium = prod(1 + R_Me) - 1,  
    smb_return = prod(1 + SMB) - 1,  
    hml_return = prod(1 + HML) - 1,  
    market_volatility = sd(R_Me),  
    .groups = "drop"  
  )
```

9.2.5

```
vec_candidate_features <- c(  
  "ROE",  
  "BE",  
  "ME",  
  "lagged_BE",  
  "lagged_ME",  
  "lagged_BEME",  
  "sales",  
  "X",  
  "NOA",  
  "NFO"  
)  
  
vec_available_features <- intersect(  
  vec_candidate_features,  
  names(tbl_target)  
)  
  
tbl_ml <- tbl_target |>  
  select(  
    firm_ID,
```



```

    year,
    prediction_year,
    forward_excess_return,
    any_of("industry_ID"),
    all_of(vec_available_features)
  ) |>
left_join(
  tbl_price_features,
  by = c("firm_ID", "year")
) |>
left_join(
  tbl_market_features,
  by = "year"
) |>
mutate(
  across(
    where(is.numeric),
    \(x) replace(x, !is.finite(x), NA_real_)
  )
) |>
filter(!is.na(forward_excess_return)) |>
arrange(year, firm_ID)

show_table_LFL(tbl_ml, n = 20)
#> # A tibble: 6,405 x 23
#>   firm_ID year prediction_year forward_excess_return industry_ID ROE
#>   <dbl> <dbl>          <dbl>          <dbl> <fct>      <dbl>
#> 1     1   2015          2016           0.99547  1         NA
#> 2     2   2015          2016           0.37395  1         NA
#> 3     3   2015          2016           0.37398  1         NA
#> 4     4   2015          2016          -0.18631  1         NA
#> 5     5   2015          2016           0.20702  1         NA
#> 6     6   2015          2016           0.11198  1         NA
#> 7     7   2015          2016          -0.052504  1         NA
#> 8     9   2015          2016           0.43471  1         NA
#> 9    10   2015          2016           0.15483  1         NA
#> 10   11   2015          2016           0.29474  1         NA
#> 11   12   2015          2016           0.15874  1         NA
#> 12   13   2015          2016           0.15519  1         NA
#> 13   14   2015          2016          -0.15663  1         NA
#> 14   15   2015          2016          -0.20041  1         NA
#> 15   16   2015          2016           0.19678  1         NA

```

#> 16	17	2015	2016	0.63234	1	NA	
#> 17	18	2015	2016	0.0057293	1	NA	
#> 18	20	2015	2016	0.19451	1	NA	
#> 19	21	2015	2016	1.0695	1	NA	
#> 20	22	2015	2016	0.16118	1	NA	
#>	BE	ME	lagged_BE	lagged_ME	lagged_BEME	sales	X
#>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
#> 1	9695.3	3577294000	NA	NA	NA	5261.4	286.64
#> 2	1054.9	4086732000	NA	NA	NA	3505.8	40.07
#> 3	93620.	70434234000	NA	NA	NA	440790.	-8434.5
#> 4	12287.	14518674000	NA	NA	NA	39725.	443.93
#> 5	146587.	59856474000	NA	NA	NA	420762.	12054.
#> 6	7571.9	5428920000	NA	NA	NA	26111.	-2506.0
#> 7	12970.	16125536000	NA	NA	NA	38979.	322.75
#> 8	12151.	3983774000	NA	NA	NA	47104.	355.68
#> 9	14306.	5485326000	NA	NA	NA	36601.	680.84
#> 10	159546.	195685659000	NA	NA	NA	412145.	15778.
#> 11	489.68	935820000	NA	NA	NA	4033.1	29.26
#> 12	98895.	157028040000	NA	NA	NA	484365.	24489.
#> 13	3888.3	12791475000	NA	NA	NA	25210.	71.29
#> 14	5282.7	7087944000	NA	NA	NA	51123.	571.86
#> 15	25353	27627851000	NA	NA	NA	44856.	2753.2
#> 16	212026.	111711138000	NA	NA	NA	216941.	7316.5
#> 17	14701.	42414944000	NA	NA	NA	118060.	-1774.8
#> 18	14702.	19779450000	NA	NA	NA	48057.	2097.2
#> 19	145766.	74118912000	NA	NA	NA	185845.	4349.8
#> 20	10149.	9317574000	NA	NA	NA	18495.	1332.1
#>	NOA	NFO	momentum	volatility	downside_risk	average_market_equity	
#>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	
#> 1	8632.6	-1062.7	0.61213	0.094533	0.035071	3112490182.	
#> 2	1137.5	82.560	1.1092	0.078195	0.010238	3232077091.	
#> 3	38481.	-55139.	0.27011	0.087622	0.049993	54901722636.	
#> 4	11646.	-640.48	0.12885	0.070908	0.044877	12609110182.	
#> 5	106449.	-40138.	0.52732	0.076101	0.039111	51142171091.	
#> 6	5769.0	-1802.9	-0.12046	0.11701	0.059670	5240739000	
#> 7	14788.	1818.4	0.55478	0.063699	0.015603	13334124364.	
#> 8	15335.	3184.8	0.34029	0.054433	0.033304	3651620727.	
#> 9	16853.	2546.7	0.41757	0.037882	0.0036665	4823362727.	
#> 10	84414.	-75133.	0.64006	0.078345	0.039664	172147588364.	
#> 11	703.98	214.3	0.97104	0.072857	0.0065920	705073091.	
#> 12	79762.	-19133.	0.38819	0.054400	0.022389	144948960000	
#> 13	2918.9	-969.36	0.49441	0.090479	0.044151	10387280455.	

```

#> 14  5208.5    -74.220  0.23658  0.077891  0.042909      6111087273.
#> 15  19292.    -6060.5  0.95127  0.091483  0.023616      21716063636.
#> 16 173620.   -38405.   0.31718  0.046885  0.016005     103602747818.
#> 17  16798.    2097.4   0.24503  0.033376  0.014007      38183010000
#> 18  15808.    1105.9   0.92749  0.095839  0.023590      16527104909.
#> 19 190563.   44797.   0.25693  0.051748  0.020822      68086841455.
#> 20   6712.4   -3436.8   0.31695  0.050601  0.018874      8430738545.
#>   market_premium smb_return hml_return market_volatility
#>   <dbl>         <dbl>         <dbl>         <dbl>
#> 1      NA      NA      NA      NA
#> 2      NA      NA      NA      NA
#> 3      NA      NA      NA      NA
#> 4      NA      NA      NA      NA
#> 5      NA      NA      NA      NA
#> 6      NA      NA      NA      NA
#> 7      NA      NA      NA      NA
#> 8      NA      NA      NA      NA
#> 9      NA      NA      NA      NA
#> 10     NA      NA      NA      NA
#> 11     NA      NA      NA      NA
#> 12     NA      NA      NA      NA
#> 13     NA      NA      NA      NA
#> 14     NA      NA      NA      NA
#> 15     NA      NA      NA      NA
#> 16     NA      NA      NA      NA
#> 17     NA      NA      NA      NA
#> 18     NA      NA      NA      NA
#> 19     NA      NA      NA      NA
#> 20     NA      NA      NA      NA
#> # i 6,385 more rows

```

9.3 7.3 tidymodels

tidymodels workflow()

```

recipe
  ↓
model specification
  ↓
workflow
  ↓

```

```
resampling
  ↓
tuning
  ↓
prediction
```

9.3.1

```
factor_recipe <- recipes::recipe(
  forward_excess_return ~ .,
  data = tbl_ml
) |>
  recipes::update_role(
    firm_ID,
    year,
    prediction_year,
    new_role = "id"
  ) |>
  recipes::step_unknown(
    recipes::all_nominal_predictors()
  ) |>
  recipes::step_novel(
    recipes::all_nominal_predictors()
  ) |>
  recipes::step_dummy(
    recipes::all_nominal_predictors()
  ) |>
  recipes::step_impute_median(
    recipes::all_numeric_predictors()
  ) |>
  recipes::step_zv(
    recipes::all_predictors()
  ) |>
  recipes::step_normalize(
    recipes::all_numeric_predictors()
  )
```

9.3.2

```

vec_analysis_years <- sort(unique(tbl_ml$year))

split_year <- vec_analysis_years[
  max(
    2L,
    floor(length(vec_analysis_years) * 0.7)
  )
]

tbl_training <- tbl_ml |>
  filter(year <= split_year)

tbl_testing <- tbl_ml |>
  filter(year > split_year)

tbl_split_overview <- tribble(
  ~sample, ~observations, ~first_year, ~last_year,
  "training",
  nrow(tbl_training),
  min(tbl_training$year),
  max(tbl_training$year),
  "testing",
  nrow(tbl_testing),
  min(tbl_testing$year),
  max(tbl_testing$year)
)

show_table_LFL(tbl_split_overview)
#> # A tibble: 2 x 4
#>   sample observations first_year last_year
#>   <chr>          <int>      <dbl>    <dbl>
#> 1 training         3779      2015      2017
#> 2 testing         2626      2018      2019

```

9.4 7.4

LASSO Ridge Elastic Net `glmnet`

- mixture = 1 LASSO
- mixture = 0 Ridge
- $0 < \text{mixture} < 1$ Elastic Net

```

elastic_net_spec <- parsnip::linear_reg(
  penalty = tune::tune(),
  mixture = tune::tune()
) |>
  parsnip::set_engine("glmnet")

elastic_net_workflow <- workflows::workflow() |>
  workflows::add_recipe(factor_recipe) |>
  workflows::add_model(elastic_net_spec)

```

```

tbl_parameter_grid <- tidyr::crossing(
  penalty = c(0.0001, 0.001, 0.01, 0.1),
  mixture = c(0, 0.5, 1)
)

```

```

set.seed(1234)

validation_folds <- rsample::vfold_cv(
  tbl_training,
  v = min(5L, nrow(tbl_training))
)

```

```

factor_tuning <- tune::tune_grid(
  elastic_net_workflow,
  resamples = validation_folds,
  grid = tbl_parameter_grid,
  metrics = yardstick::metric_set(
    yardstick::rmse,
    yardstick::mae
  )
)

```

```

best_parameters <- tune::select_best(
  factor_tuning,
  metric = "rmse"
)

show_table_LFL(best_parameters)
#> # A tibble: 1 x 3

```

```

#>   penalty mixture .config
#>   <dbl>   <dbl> <chr>
#> 1    0.001     0.5 pre0_mod05_post0

final_factor_workflow <- tune::finalize_workflow(
  elastic_net_workflow,
  best_parameters
)

final_factor_fit <- parsnip::fit(
  final_factor_workflow,
  data = tbl_training
)

```

```

tbl_selected_factors <- final_factor_fit |>
  workflows::extract_fit_parsnip() |>
  broom::tidy() |>
  filter(
    term != "(Intercept)",
    estimate != 0
  ) |>
  arrange(desc(abs(estimate)))

show_table_LFL(tbl_selected_factors, n = 30)
#> # A tibble: 22 x 3
#>   term                estimate penalty
#>   <chr>                <dbl>   <dbl>
#> 1 volatility          -0.052925   0.001
#> 2 lagged_BEME           0.040087   0.001
#> 3 momentum             0.035791   0.001
#> 4 market_premium       -0.027679   0.001
#> 5 smb_return            -0.026595   0.001
#> 6 hml_return            0.025744   0.001
#> 7 market_volatility    -0.025094   0.001
#> 8 industry_ID_X7        0.022350   0.001
#> 9 ME                   -0.019009   0.001
#> 10 industry_ID_X10      0.017595   0.001
#> 11 industry_ID_X5       -0.015146   0.001
#> 12 lagged_ME           0.015131   0.001
#> 13 X                   -0.011249   0.001

```

```

#> 14 ROE -0.0099735 0.001
#> 15 industry_ID_X2 0.0094625 0.001
#> 16 industry_ID_X6 -0.0092099 0.001
#> 17 industry_ID_X3 0.0088236 0.001
#> 18 industry_ID_X9 -0.0058770 0.001
#> 19 NOA 0.0049012 0.001
#> 20 lagged_BE -0.0034932 0.001
#> 21 BE 0.0023084 0.001
#> 22 industry_ID_X8 0.0015864 0.001

```

9.5 7.5

9.5.1

```

tbl_test_predictions <- predict(
  final_factor_fit,
  new_data = tbl_testing
) |>
  bind_cols(
    tbl_testing |>
      select(
        firm_ID,
        year,
        prediction_year,
        forward_excess_return
      )
  ) |>
  rename(
    predicted_return = .pred,
    realized_return = forward_excess_return
  )

```

```

tbl_prediction_metrics <- yardstick::metric_set(
  yardstick::rmse,
  yardstick::mae,
  yardstick::rsq
)(
  tbl_test_predictions,
  truth = realized_return,

```



```

    estimate = predicted_return
  )

show_table_LFL(tbl_prediction_metrics)
#> # A tibble: 3 x 3
#>   .metric .estimator .estimate
#>   <chr>    <chr>      <dbl>
#> 1 rmse    standard    3.9625
#> 2 mae     standard    3.0002
#> 3 rsq     standard    0.25561

```

9.5.2

```

final_factor_fit_all <- parsnip::fit(
  final_factor_workflow,
  data = tbl_ml
)

tbl_security_scores <- predict(
  final_factor_fit_all,
  new_data = tbl_ml
) |>
  bind_cols(
    tbl_ml |>
      select(
        firm_ID,
        year,
        prediction_year,
        forward_excess_return
      )
  ) |>
  rename(
    expected_return = .pred,
    realized_return = forward_excess_return
  ) |>
  group_by(prediction_year) |>
  mutate(
    score_rank = percent_rank(expected_return),
    score_decile = ntile(expected_return, 10L)
  )

```

```
) |>
ungroup()
```

9.6 7.6

```
tbl_decile_returns <- tbl_security_scores |>
  group_by(
    prediction_year,
    score_decile
  ) |>
  summarise(
    securities = n(),
    average_expected_return = mean(
      expected_return,
      na.rm = TRUE
    ),
    average_realized_return = mean(
      realized_return,
      na.rm = TRUE
    ),
    .groups = "drop"
  )

show_table_LFL(tbl_decile_returns, n = Inf)
#> # A tibble: 50 x 5
#>   prediction_year score_decile securities average_expected_return
#>   <dbl>          <int>      <int>          <dbl>
#> 1      2016           1        125          0.040431
#> 2      2016           2        124          0.079922
#> 3      2016           3        124          0.10226
#> 4      2016           4        124          0.11709
#> 5      2016           5        124          0.13332
#> 6      2016           6        124          0.14901
#> 7      2016           7        124          0.16426
#> 8      2016           8        124          0.17963
#> 9      2016           9        124          0.19541
#> 10     2016          10        124          0.22457
#> 11     2017           1        127          0.053063
```

```

#> 12      2017      2      127      0.13216
#> 13      2017      3      127      0.16286
#> 14      2017      4      127      0.18964
#> 15      2017      5      127      0.21151
#> 16      2017      6      127      0.23091
#> 17      2017      7      127      0.25076
#> 18      2017      8      127      0.27422
#> 19      2017      9      127      0.29802
#> 20      2017     10      127      0.35038
#> 21      2018      1      127     -0.21248
#> 22      2018      2      127     -0.13342
#> 23      2018      3      127     -0.10167
#> 24      2018      4      127     -0.081166
#> 25      2018      5      127     -0.062174
#> 26      2018      6      127     -0.046061
#> 27      2018      7      127     -0.026265
#> 28      2018      8      127     -0.0056155
#> 29      2018      9      126      0.019222
#> 30      2018     10      126      0.074212
#> 31      2019      1      131      0.27616
#> 32      2019      2      131      0.33718
#> 33      2019      3      131      0.36518
#> 34      2019      4      131      0.38434
#> 35      2019      5      130      0.40089
#> 36      2019      6      130      0.41739
#> 37      2019      7      130      0.43302
#> 38      2019      8      130      0.45139
#> 39      2019      9      130      0.47472
#> 40      2019     10      130      0.52847
#> 41      2020      1      133     -0.20674
#> 42      2020      2      133     -0.14097
#> 43      2020      3      132     -0.11078
#> 44      2020      4      132     -0.088086
#> 45      2020      5      132     -0.066190
#> 46      2020      6      132     -0.045460
#> 47      2020      7      132     -0.025661
#> 48      2020      8      132     -0.0046676
#> 49      2020      9      132      0.022774
#> 50      2020     10      132      0.075941
#>      average_realized_return
#>                                <dbl>
#> 1              0.0093272

```

#> 2	0.10518
#> 3	0.018788
#> 4	0.098786
#> 5	0.11719
#> 6	0.14958
#> 7	0.21376
#> 8	0.22106
#> 9	0.26803
#> 10	0.21087
#> 11	0.041990
#> 12	0.14854
#> 13	0.16197
#> 14	0.19607
#> 15	0.24228
#> 16	0.26056
#> 17	0.26490
#> 18	0.28203
#> 19	0.25886
#> 20	0.30981
#> 21	-0.18027
#> 22	-0.15977
#> 23	-0.12313
#> 24	-0.067214
#> 25	-0.086579
#> 26	-0.089470
#> 27	0.0077331
#> 28	-0.035701
#> 29	0.0080498
#> 30	0.11281
#> 31	0.17750
#> 32	0.24310
#> 33	0.38461
#> 34	0.41635
#> 35	0.36221
#> 36	0.36378
#> 37	0.45805
#> 38	0.50355
#> 39	0.52715
#> 40	0.64451
#> 41	-0.17070
#> 42	-0.12237
#> 43	-0.070044

```
#> 44          -0.085112
#> 45          -0.079495
#> 46          -0.053790
#> 47          -0.039142
#> 48          -0.011216
#> 49           0.0052943
#> 50           0.024575
```

10 10

```
tbl_long_short <- tbl_decile_returns |>
  filter(score_decile %in% c(1L, 10L)) |>
  select(
    prediction_year,
    score_decile,
    average_realized_return
  ) |>
  pivot_wider(
    names_from = score_decile,
    values_from = average_realized_return,
    names_prefix = "decile_"
  ) |>
  mutate(
    long_short_return =
      decile_10 - decile_1
  )

show_table_LFL(tbl_long_short, n = Inf)
#> # A tibble: 5 x 4
#>   prediction_year  decile_1 decile_10 long_short_return
#>   <dbl>         <dbl>    <dbl>         <dbl>
#> 1      2016  0.0093272  0.21087         0.20154
#> 2      2017  0.041990  0.30981         0.26782
#> 3      2018 -0.18027   0.11281         0.29308
#> 4      2019  0.17750   0.64451         0.46701
#> 5      2020 -0.17070   0.024575        0.19527
```

9.7 7.7 8

8

```
tbl_expected_returns <- tbl_security_scores |>
  transmute(
    rebalance_year = prediction_year,
    firm_ID,
    expected_return,
    score_rank,
    score_decile
  )

tbl_asset_return_history <- tbl_monthly |>
  transmute(
    firm_ID = as.integer(firm_ID),
    month_ID = as.integer(month_ID),
    month_date = as.Date(month_date),
    return = R
  ) |>
  filter(is.finite(return))
tbl_investment_universe <- tbl_expected_returns |>
  left_join(
    tbl_financial |>
      transmute(
        rebalance_year = as.integer(year) + 1L,
        firm_ID = as.integer(firm_ID),
        market_equity = ME
      ),
    by = c(
      "rebalance_year", "firm_ID"
    ) |>
    filter(
      is.finite(expected_return),
      is.finite(market_equity),
      market_equity > 0
    )
  )
```

```
tbl_chapter_07_outputs <- list(
  expected_returns = tbl_expected_returns,
  security_scores = tbl_security_scores,
  factor_selection = tbl_selected_factors,
  asset_return_history = tbl_asset_return_history,
  investment_universe = tbl_investment_universe
)
```

```

vec_chapter_07_files <- c(
  expected_returns = file.path(
    "portfolios",
    "expected_returns.parquet"
  ),
  security_scores = file.path(
    "factors",
    "security_scores.parquet"
  ),
  factor_selection = file.path(
    "factors",
    "factor_selection.parquet"
  ),
  asset_return_history = file.path(
    "portfolios",
    "asset_return_history.parquet"
  ),
  investment_universe = file.path(
    "portfolios",
    "investment_universe.parquet"
  )
)

vec_chapter_07_paths <- tbl_chapter_07_outputs |>
  imap_chr(
    \(data, name) {
      write_derived_parquet_file_LFL(
        data,
        vec_chapter_07_files[[name]]
      )
    }
  )

vec_chapter_07_paths
#>
#>      "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/portfolios/expected_returns.parquet"
#>      "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/portfolios/security_scores.parquet"
#>      "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/factors/security_scores.parquet"
#>      "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/factors/factor_selection.parquet"
#>      "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/portfolios/asset_return_history.parquet"
#>      "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/portfolios/investment_universe.parquet"

```

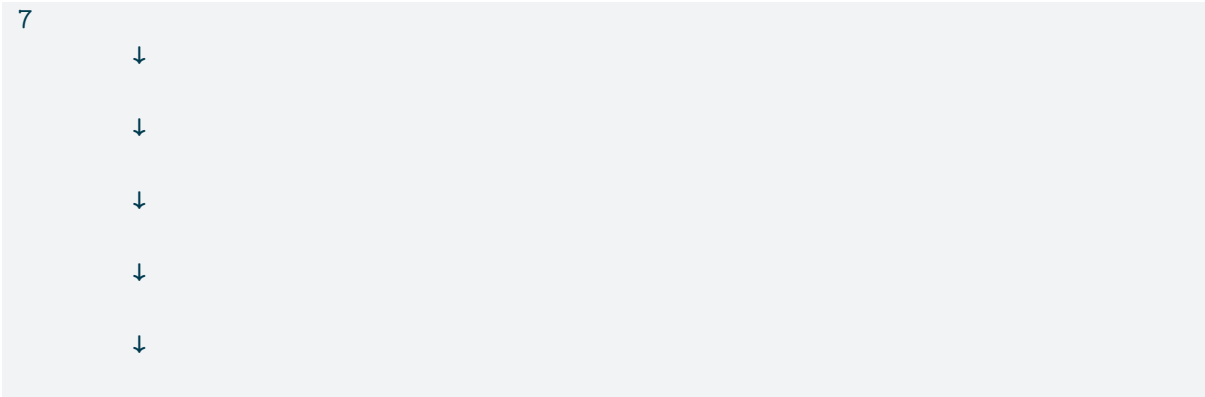
```
#> investment_uni  
#> "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/portfolios/investment_universe.par
```

tidymodels

8

10 I-8

7
5 VaR



10.1 8.1

N w μ Σ

$$\mu_p = w^\top \mu$$

$$\sigma_p^2 = w^\top \Sigma w$$

$$\max_w \left[w^\top \mu - \frac{\gamma}{2} w^\top \Sigma w \right]$$

γ

10.2 8.2

10.2.1 7

```
tbl_expected_returns <- read_derived_parquet_file_LFL(
  file.path(
    "portfolios",
    "expected_returns.parquet"
  )
)

tbl_return_history <- read_derived_parquet_file_LFL(
  file.path(
    "portfolios",
    "asset_return_history.parquet"
  )
)

tbl_investment_universe <- read_derived_parquet_file_LFL(
  file.path(
    "portfolios",
    "investment_universe.parquet"
  )
)
```

```
tbl_input_overview <- tribble(
  ~data, ~observations, ~variables,
  "expected_returns",
  nrow(tbl_expected_returns),
  ncol(tbl_expected_returns),
  "asset_return_history",
  nrow(tbl_return_history),
  ncol(tbl_return_history),
  "investment_universe",
  nrow(tbl_investment_universe),
)
```

```

    ncol(tbl_investment_universe)
  )

show_table_LFL(tbl_input_overview)
#> # A tibble: 3 x 3
#>   data                observations variables
#>   <chr>                <int>      <int>
#> 1 expected_returns      6405         5
#> 2 asset_return_history  93525        4
#> 3 investment_universe   6405         6

```

10.2.2

1

```

rebalance_year <- tbl_investment_universe |>
  summarise(
    rebalance_year =
      min(rebalance_year, na.rm = TRUE)
  ) |>
  pull(rebalance_year)

tbl_universe_one_year <- tbl_investment_universe |>
  filter(rebalance_year == !!rebalance_year) |>
  arrange(desc(market_equity)) |>
  slice_head(n = 20)

```

20

10.2.3

```

vec_firm_ids <- tbl_universe_one_year |>
  pull(firm_ID)

tbl_return_wide <- tbl_return_history |>
  filter(firm_ID %in% vec_firm_ids) |>
  select(
    month_ID,
    firm_ID,

```

```

      return
    ) |>
    pivot_wider(
      names_from = firm_ID,
      values_from = return
    ) |>
    arrange(month_ID)

mat_return <- tbl_return_wide |>
  select(-month_ID) |>
  drop_na() |>
  as.matrix()

mat_covariance <- cov(mat_return)

vec_expected_return <- tbl_universe_one_year |>
  arrange(firm_ID) |>
  pull(expected_return)

```

```

vec_covariance_firm_ids <- colnames(mat_return) |>
  as.integer()

tbl_universe_one_year <- tbl_universe_one_year |>
  filter(firm_ID %in% vec_covariance_firm_ids) |>
  arrange(match(firm_ID, vec_covariance_firm_ids))

vec_expected_return <- tbl_universe_one_year |>
  pull(expected_return)

mat_covariance <- mat_covariance[
  as.character(tbl_universe_one_year$firm_ID),
  as.character(tbl_universe_one_year$firm_ID),
  drop = FALSE
]

```

10.3 8.3

10.3.1

$$\sum_{i=1}^N w_i = 1$$

10.3.2

$$w_i \geq 0$$

10.3.3

$$w_i \leq w_{\max}$$

$$1 \qquad 20\%$$

10.3.4 VaR

$$\text{VaR}$$

$$VaR_c = z_c \sigma_p - \mu_p$$

$$z_c \sqrt{w^\top \Sigma w} - w^\top \mu \leq VaR_{\max}$$

10.3.5

$$w_{t-1}$$

$$Turnover_t = \frac{1}{2} \sum_{i=1}^N |w_{i,t} - w_{i,t-1}|$$

$$\text{VaR}$$

10.4 8.4

10.4.1

```
portfolio_return_LFL <- function(  
  weight,  
  expected_return  
) {  
  sum(weight * expected_return)  
}  
  
portfolio_variance_LFL <- function(  
  weight,  
  covariance_matrix  
) {  
  as.numeric(  
    t(weight) %*%  
      covariance_matrix %*%  
      weight  
  )  
}  
  
portfolio_volatility_LFL <- function(  
  weight,  
  covariance_matrix  
) {  
  sqrt(  
    portfolio_variance_LFL(  
      weight,  
      covariance_matrix  
    )  
  )  
}  
  
portfolio_var_LFL <- function(  
  weight,  
  expected_return,  
  covariance_matrix,  
  confidence_level = 0.95  
) {  
  qnorm(confidence_level) *
```

```

    portfolio_volatility_LFL(
      weight,
      covariance_matrix
    ) -
    portfolio_return_LFL(
      weight,
      expected_return
    )
  }

```

10.4.2

```

maximum_weight <- 0.20
maximum_var <- 0.10
confidence_level <- 0.95
risk_aversion <- 5

```

10.4.3 nloptr

```

assets <- length(vec_expected_return)

objective_function <- function(weight) {
  -portfolio_return_LFL(
    weight,
    vec_expected_return
  ) +
  risk_aversion / 2 *
  portfolio_variance_LFL(
    weight,
    mat_covariance
  )
}

objective_function <- function(weight) {
  objective_value <-
  -portfolio_return_LFL(
    weight,
    vec_expected_return
  )
}

```

```

    ) +
    risk_aversion / 2 *
    portfolio_variance_LFL(
      weight,
      mat_covariance
    )

objective_gradient <-
  -vec_expected_return +
  risk_aversion *
  as.numeric(
    mat_covariance %*% weight
  )

list(
  objective = objective_value,
  gradient = objective_gradient
)
}

equality_constraint <- function(weight) {
  list(
    constraints = sum(weight) - 1,
    jacobian = matrix(
      rep(1, length(weight)),
      nrow = 1
    )
  )
}

inequality_constraint <- function(weight) {
  portfolio_volatility <-
    portfolio_volatility_LFL(
      weight,
      mat_covariance
    )

  constraint_value <-
    qnorm(confidence_level) *
    portfolio_volatility -
    portfolio_return_LFL(
      weight,

```



```

      vec_expected_return
    ) -
    maximum_var

volatility_gradient <- if (
  portfolio_volatility > 1e-12
) {
  as.numeric(
    mat_covariance %*% weight
  ) /
    portfolio_volatility
} else {
  rep(0, length(weight))
}

constraint_gradient <-
  qnorm(confidence_level) *
  volatility_gradient -
  vec_expected_return

list(
  constraints = constraint_value,
  jacobian = matrix(
    constraint_gradient,
    nrow = 1
  )
)
}

equality_constraint <- function(weight) {
  sum(weight) - 1
}

inequality_constraint <- function(weight) {
  portfolio_var_LFL(
    weight,
    vec_expected_return,
    mat_covariance,
    confidence_level
  ) - maximum_var
}

objective_function <- function(weight) {

```

```

objective_value <-
  -portfolio_return_LFL(
    weight,
    vec_expected_return
  ) +
  risk_aversion / 2 *
  portfolio_variance_LFL(
    weight,
    mat_covariance
  )

objective_gradient <-
  -vec_expected_return +
  risk_aversion *
  as.numeric(
    mat_covariance %*% weight
  )

list(
  objective = objective_value,
  gradient = objective_gradient
)
}

equality_constraint <- function(weight) {
  list(
    constraints = sum(weight) - 1,
    jacobian = rep(1, length(weight))
  )
}

inequality_constraint <- function(weight) {
  portfolio_volatility <-
    portfolio_volatility_LFL(
      weight,
      mat_covariance
    )

  constraint_value <-
    qnorm(confidence_level) *
    portfolio_volatility -
    portfolio_return_LFL(

```

```

    weight,
    vec_expected_return
  ) -
  maximum_var

volatility_gradient <- if (
  portfolio_volatility > 1e-12
) {
  as.numeric(
    mat_covariance %*% weight
  ) /
    portfolio_volatility
} else {
  rep(0, length(weight))
}

constraint_gradient <-
  qnorm(confidence_level) *
  volatility_gradient -
  vec_expected_return

list(
  constraints = constraint_value,
  jacobian = constraint_gradient
)
}

assets <- length(vec_expected_return)

if (assets * maximum_weight < 1) {
  stop(
    " maximum_weight          ",
    call. = FALSE
  )
}

```

```

optimization_result <- nloptr::nloptr(
  x0 = rep(1 / assets, assets),
  eval_f = objective_function,
  lb = rep(0, assets),
  ub = rep(maximum_weight, assets),
  eval_g_eq = equality_constraint,

```

```

eval_g_ineq = inequality_constraint,
opts = list(
  algorithm = "NLOPT_LD_SLSQP",
  xtol_rel = 1e-8,
  maxeval = 1000,
  print_level = 0
)
)

tbl_optimized_weights <- tbl_universe_one_year |>
  select(
    rebalance_year,
    firm_ID,
    expected_return
  ) |>
  mutate(
    weight = optimization_result$solution
  ) |>
  arrange(desc(weight))

show_table_LFL(tbl_optimized_weights, n = Inf)
#> # A tibble: 20 x 4
#>   rebalance_year firm_ID expected_return weight
#>   <dbl>      <dbl>          <dbl>    <dbl>
#> 1      2016      1253      0.20697  2e- 1
#> 2      2016       472      0.15848  2e- 1
#> 3      2016      1205      0.14965  2e- 1
#> 4      2016      1400      0.18788  2.0000e- 1
#> 5      2016      1408      0.17726  2.0000e- 1
#> 6      2016       721      0.14213  8.8991e-16
#> 7      2016      1057      0.051368 1.3889e-16
#> 8      2016      1237      0.12760  1.1713e-16
#> 9      2016       456      0.015991 7.5341e-17
#> 10     2016       725      0.096705 2.5315e-17
#> 11     2016       331      0.14022  0
#> 12     2016       392      0.14504  0
#> 13     2016       401      0.053347  0
#> 14     2016       409      0.13995  0
#> 15     2016       579      0.060645  0
#> 16     2016       710      0.11519  0
#> 17     2016       987      0.097476  0
#> 18     2016       988     -0.0079991 0

```

```
#> 19      2016      996      0.072768  0
#> 20      2016     1023      0.095368  0
```

```
tbl_constraint_check <- tribble(
  ~constraint, ~value, ~limit,
  "weight_sum",
  sum(tbl_optimized_weights$weight),
  1,
  "minimum_weight",
  min(tbl_optimized_weights$weight),
  0,
  "maximum_weight",
  max(tbl_optimized_weights$weight),
  maximum_weight,
  "portfolio_var",
  portfolio_var_LFL(
    tbl_optimized_weights$weight,
    vec_expected_return,
    mat_covariance,
    confidence_level
  ),
  maximum_var
)
```

```
show_table_LFL(tbl_constraint_check)
#> # A tibble: 4 x 3
#>   constraint      value limit
#>   <chr>          <dbl> <dbl>
#> 1 weight_sum      1.0000     1
#> 2 minimum_weight  0          0
#> 3 maximum_weight  0.2         0.2
#> 4 portfolio_var   0.0052375    0.1
```

10.5 8.5

```

rebalance_month_ID <- tbl_return_history |>
  mutate(
    year = lubridate::year(month_date)
  ) |>
  filter(year == rebalance_year) |>
  summarise(
    first_month_ID =
      min(month_ID, na.rm = TRUE)
  ) |>
  pull(first_month_ID)

```

```

tbl_backtest_returns <- tbl_return_history |>
  filter(
    firm_ID %in% tbl_optimized_weights$firm_ID,
    month_ID >= rebalance_month_ID,
    month_ID < rebalance_month_ID + 12L
  ) |>
  left_join(
    tbl_optimized_weights |>
      select(
        firm_ID,
        weight
      ),
    by = "firm_ID"
  ) |>
  group_by(
    month_ID,
    month_date
  ) |>
  summarise(
    portfolio_return = sum(
      weight * return,
      na.rm = TRUE
    ),
    .groups = "drop"
  ) |>
  arrange(month_ID)

```

```
tbl_equal_weight_returns <- tbl_return_history |>
  filter(
    firm_ID %in% tbl_optimized_weights$firm_ID,
    month_ID >= rebalance_month_ID,
    month_ID < rebalance_month_ID + 12L
  ) |>
  group_by(
    month_ID,
    month_date
  ) |>
  summarise(
    equal_weight_return =
      mean(return, na.rm = TRUE),
    .groups = "drop"
  )

tbl_backtest_returns <- tbl_backtest_returns |>
  left_join(
    tbl_equal_weight_returns,
    by = c(
      "month_ID",
      "month_date"
    )
  )
)
```

```
tbl_backtest_plot <- tbl_backtest_returns |>
  mutate(
    optimized = cumprod(
      1 + portfolio_return
    ) - 1,
    equal_weight = cumprod(
      1 + equal_weight_return
    ) - 1
  ) |>
  select(
    month_date,
    optimized,
    equal_weight
  ) |>
  pivot_longer(
    cols = c(
      optimized,

```

```

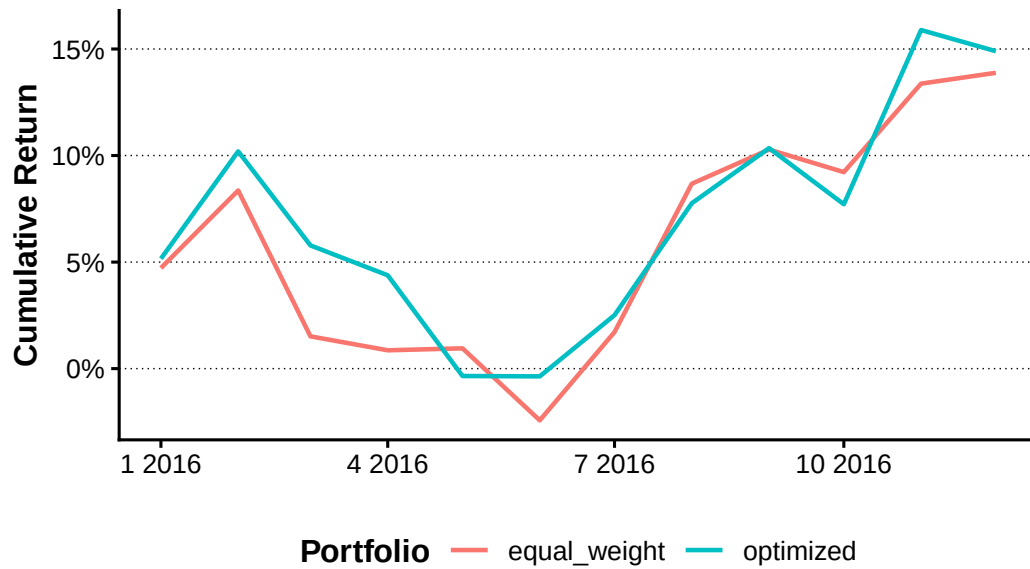
    equal_weight
  ),
  names_to = "portfolio",
  values_to = "cumulative_return"
)

plot_backtest <- tbl_backtest_plot |>
  ggplot(
    aes(
      x = month_date,
      y = cumulative_return,
      colour = portfolio
    )
  ) +
  geom_line(linewidth = 0.8) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = NULL,
    y = "Cumulative Return",
    colour = "Portfolio",
    title = "Optimized and Equal-Weight Portfolios"
  )

show_plot_LFL(
  add_lagrange_theme_LFL(
    plot_backtest
  )
)

```


Optimized and Equal-Weight Portfolios



10.6 8.6

```
annualized_return_LFL <- function(  
  x,  
  periods = 12  
) {  
  prod(1 + x)^(periods / length(x)) - 1  
}  
  
annualized_volatility_LFL <- function(  
  x,  
  periods = 12  
) {  
  sd(x) * sqrt(periods)  
}  
  
maximum_drawdown_LFL <- function(x) {  
  wealth <- cumprod(1 + x)  
  drawdown <- wealth / cummax(wealth) - 1  
  min(drawdown)  
}
```

```
tbl_performance_summary <- tribble(
  ~portfolio, ~annualized_return,
  ~annualized_volatility, ~sharpe_ratio,
  ~maximum_drawdown,
  "optimized",
  annualized_return_LFL(
    tbl_backtest_returns$portfolio_return
  ),
  annualized_volatility_LFL(
    tbl_backtest_returns$portfolio_return
  ),
  annualized_return_LFL(
    tbl_backtest_returns$portfolio_return
  ) /
  annualized_volatility_LFL(
    tbl_backtest_returns$portfolio_return
  ),
  maximum_drawdown_LFL(
    tbl_backtest_returns$portfolio_return
  ),
  "equal_weight",
  annualized_return_LFL(
    tbl_backtest_returns$equal_weight_return
  ),
  annualized_volatility_LFL(
    tbl_backtest_returns$equal_weight_return
  ),
  annualized_return_LFL(
    tbl_backtest_returns$equal_weight_return
  ) /
  annualized_volatility_LFL(
    tbl_backtest_returns$equal_weight_return
  ),
  maximum_drawdown_LFL(
    tbl_backtest_returns$equal_weight_return
  )
)

show_table_LFL(tbl_performance_summary)
#> # A tibble: 2 x 5
#>   portfolio    annualized_return annualized_volatility sharpe_ratio
```

```

#>   <chr>                <dbl>                <dbl>                <dbl>
#> 1 optimized            0.14896                0.13773                1.0816
#> 2 equal_weight        0.13883                0.12901                1.0761
#>   maximum_drawdown
#>           <dbl>
#> 1          -0.095817
#> 2          -0.099548

```

10.7 8.7

```

tbl_chapter_08_outputs <- list(
  optimized_weights =
    tbl_optimized_weights,
  backtest_returns =
    tbl_backtest_returns,
  performance_summary =
    tbl_performance_summary
)

vec_chapter_08_files <- c(
  optimized_weights = file.path(
    "portfolios",
    "optimized_weights.parquet"
  ),
  backtest_returns = file.path(
    "portfolios",
    "backtest_returns.parquet"
  ),
  performance_summary = file.path(
    "portfolios",
    "performance_summary.parquet"
  )
)

vec_chapter_08_paths <- tbl_chapter_08_outputs |>
  imap_chr(
    \(data, name) {
      write_derived_parquet_file_LFL(

```

```

        data,
        vec_chapter_08_files[[name]]
    )
}
)

vec_chapter_08_paths
#> optimized_weights.parquet
#> "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/portfolios/optimized_weights.parquet"
#> backtest_returns.parquet
#> "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/portfolios/backtest_returns.parquet"
#> performance_summary.parquet
#> "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/portfolios/performance_summary.parquet"

```

7

VaR

Part II

II

11 II-1

100 1 100

11.1 1.1

	OIS		DF
	DF	1	DF
EONIA		DF	

```
eonia_benchmark <- GaussQuant::eonia_curve_benchmark_GQL()

eonia_curve_tbl <- eonia_benchmark$final_validation$quote_repricing |>
  dplyr::select(
    pillar_date,
    discount_factor,
    zero_rate,
    one_day_forward_rate
  ) |>
  dplyr::filter(!is.na(.data$pillar_date)) |>
  dplyr::distinct(.data$pillar_date, .keep_all = TRUE) |>
  dplyr::arrange(.data$pillar_date)

spot_curve_plot <- eonia_curve_tbl |>
  dplyr::filter(!is.na(.data$zero_rate)) |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = .data$pillar_date,
      y = 100 * .data$zero_rate
    )
  ) +
  ggplot2::geom_line(linewidth = 0.7) +
  ggplot2::geom_point(size = 1.5) +
```

```

ggplot2::scale_x_date(
  date_breaks = "5 years",
  date_labels = "%Y"
) +
ggplot2::labs(
  title = "EONIA      ",
  x = " ",
  y = "      % "
) +
ggplot2::theme_minimal()

discount_curve_plot <- eonia_curve_tbl |>
  dplyr::filter(!is.na(.data$discount_factor)) |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = .data$pillar_date,
      y = .data$discount_factor
    )
  ) +
  ggplot2::geom_line(linewidth = 0.7) +
  ggplot2::geom_point(size = 1.5) +
  ggplot2::scale_x_date(
    date_breaks = "5 years",
    date_labels = "%Y"
  ) +
  ggplot2::labs(
    title = "EONIA      ",
    x = " ",
    y = "      "
  ) +
  ggplot2::theme_minimal()

forward_curve_plot <- eonia_curve_tbl |>
  dplyr::filter(!is.na(.data$one_day_forward_rate)) |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = .data$pillar_date,
      y = 100 * .data$one_day_forward_rate
    )
  ) +
  ggplot2::geom_line(linewidth = 0.7) +
  ggplot2::geom_point(size = 1.5) +

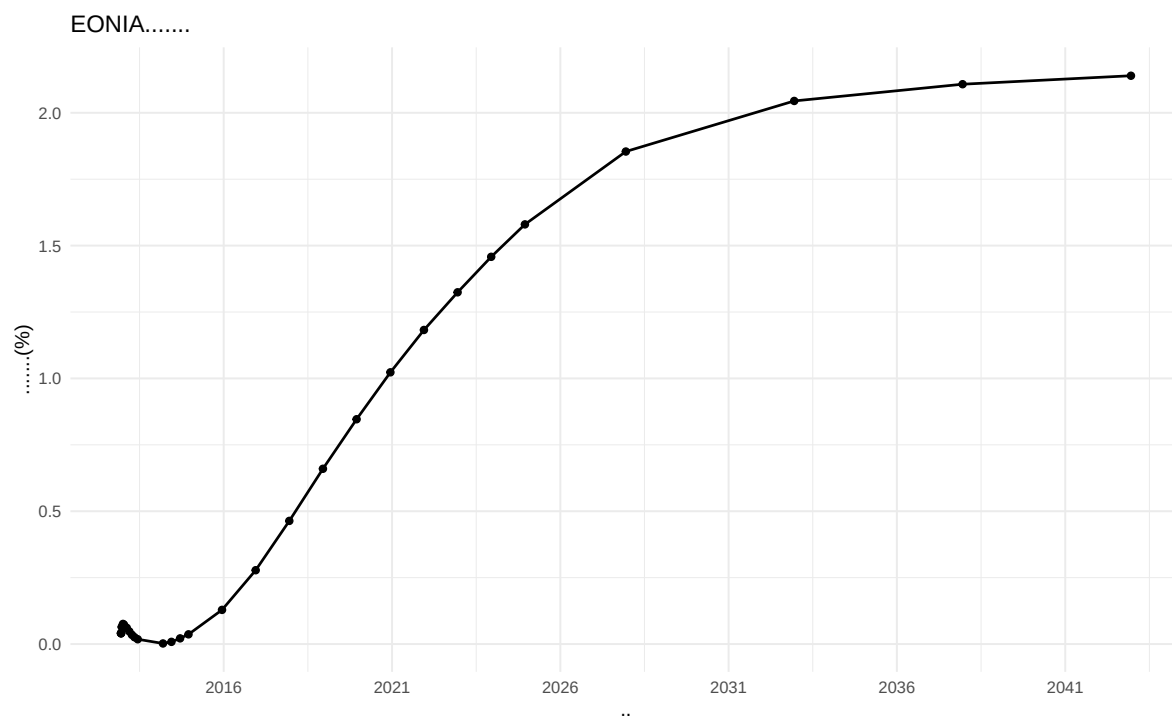
```

```

ggplot2::scale_x_date(
  date_breaks = "5 years",
  date_labels = "%Y"
) +
ggplot2::labs(
  title = "EONIA 1",
  x = "",
  y = "% "
) +
ggplot2::theme_minimal()

print(spot_curve_plot)

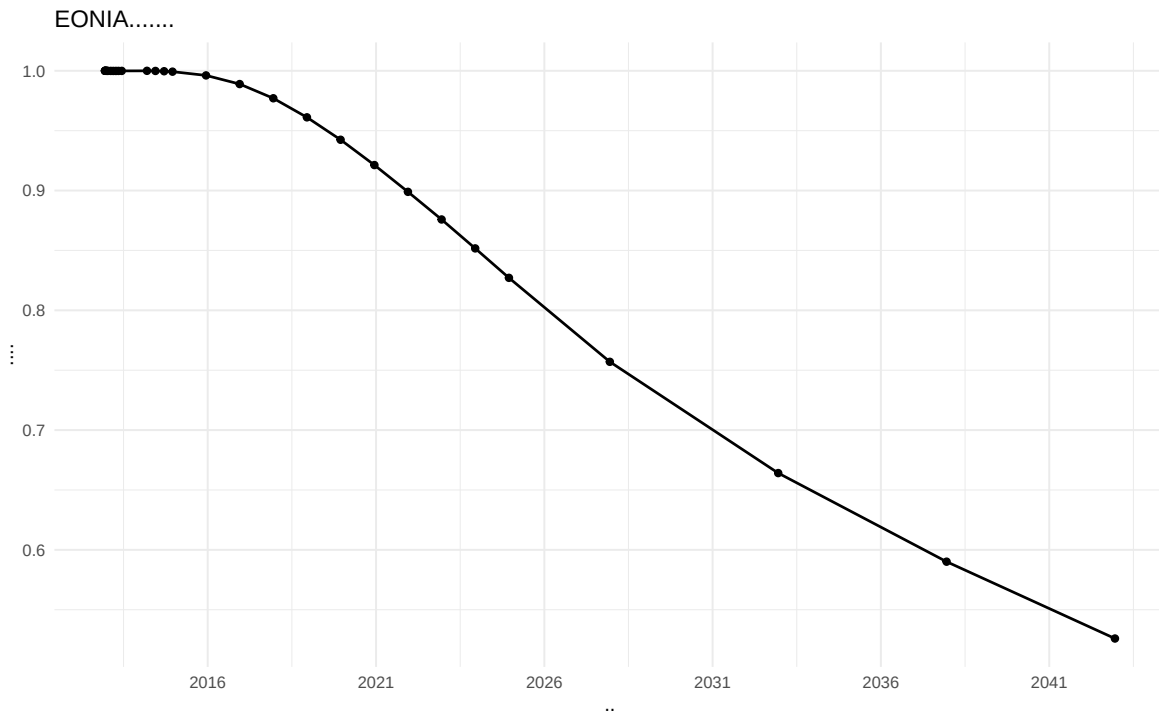
```



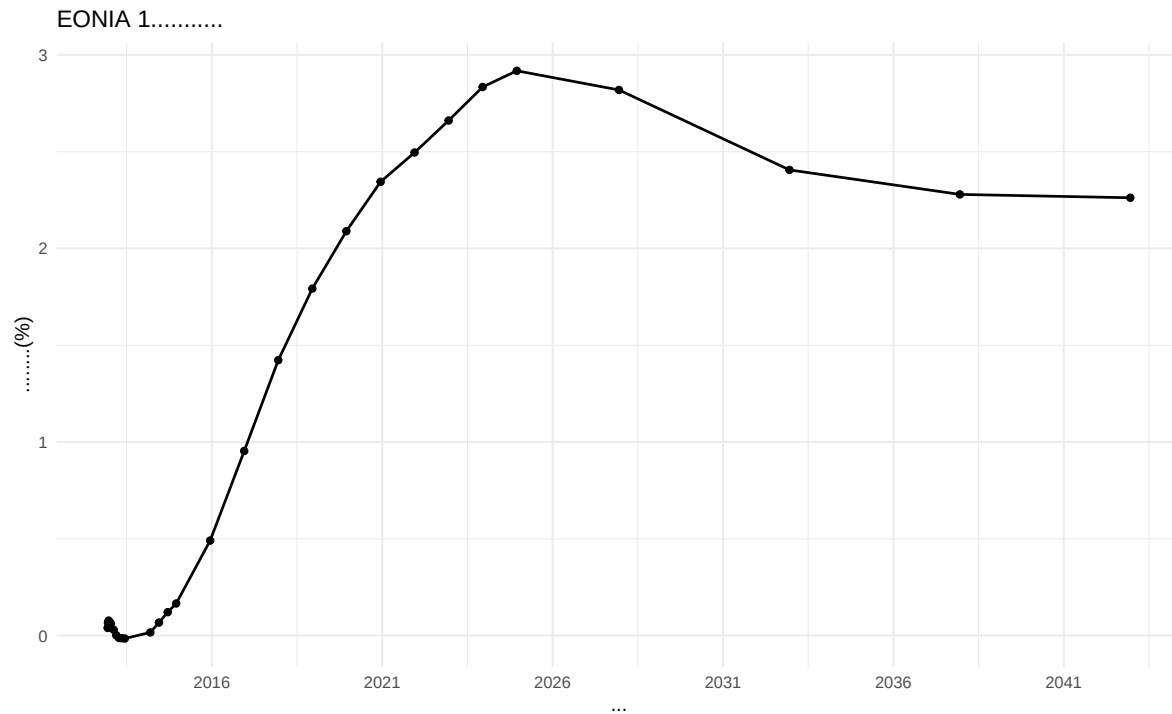
```

print(discount_curve_plot)

```

```
print(forward_curve_plot)
```



11.2 1.2

11.2.1 1.2.1

Day Count Convention

- **Actual/365 Fixed** 365
- **Actual/360** 360
- **Actual/Actual ICMA**
- **Actual/Actual ISDA** 365 366
- **30/360 Bond Basis** 1 30 1 360
- **30E/360** 31 30 1 30 1 360

30/360	1	180	90	30	Actual			
Day count convention					Actual/Actual	ICMA 30/360	Bond	
Basis					Actual/360	Actual/365		
Fixed								
	Actual/Actual ICMA		reference period					
			Following		Modified	Following		Preced-
ing	Unadjusted							

11.2.2 1.2.2

N	T							
					$T = \frac{N}{365}$			
	1		2 29		1	2 29		365
	8		7					
100	C	R	P	T	Y			
					$Y = \frac{C + \frac{R - P}{T}}{P} \times 100$			
	7				4		3	
		30/360 Bond Basis			Unadjusted			
	2	30/360 Bond Basis			2			

```

jp_calendar <- QuantLib::Japan()

dc_a365 <- QuantLib::Actual365Fixed()
dc_a360 <- QuantLib::Actual360()
dc_30 <- GaussQuant::day_counter_GQL(
  "Thirty360",
  convention = "BondBasis"
)

compd_compounded <- QuantLib::Compounding_Compounded_get()
compd_simple <- QuantLib::Compounding_Simple_get()

freq_annual <- QuantLib::Frequency_Annual_get()
freq_semiannual <- QuantLib::Frequency_Semiannual_get()

trade_date_chr <- "2022-08-19"
issue_date_chr <- "2022-07-28"
maturity_date_chr <- "2025-07-28"

trade_date <- GaussQuant::date_GQL(trade_date_chr)
issue_date_ql <- GaussQuant::date_GQL(issue_date_chr)
maturity_date_ql <- GaussQuant::date_GQL(maturity_date_chr)

settlement_days <- 2L

GaussQuant::set_eval_date_GQL(trade_date)

settlement_date <- GaussQuant::advance_days_GQL(
  calendar_obj = jp_calendar,
  date_obj = trade_date,
  n_days = settlement_days
)

settlement_date_chr <- as.character(settlement_date)
settlement_date_ql <- GaussQuant::date_GQL(settlement_date)

setup_tbl <- tibble::tibble(
  item = c(
    "trade_date",
    "settlement_date",
    "issue_date",
    "maturity_date",

```

```

    "settlement_days",
    "day_counter",
    "compounding",
    "frequency"
  ),
  value = c(
    trade_date_chr,
    settlement_date_chr,
    issue_date_chr,
    maturity_date_chr,
    as.character(settlement_days),
    "Thirty360(BondBasis)",
    "Compounded",
    "Semiannual"
  )
)

GaussQuant::show_tbl_GQH(setup_tbl, "Bond setup dates and conventions")
#> # A tibble: 8 x 2
#>   item      value
#>   <chr>    <chr>
#> 1 trade_date 2022-08-19
#> 2 settlement_date 2022-08-23
#> 3 issue_date 2022-07-28
#> 4 maturity_date 2025-07-28
#> 5 settlement_days 2
#> 6 day_counter  Thirty360(BondBasis)
#> 7 compounding  Compounded
#> 8 frequency    Semiannual

```

2

11.3 1.3

100	0.37%	2025 7 28	6	Unadjusted
-----	-------	-----------	---	------------

```

face_amount <- 100
coupon_rate <- 0.00370
clean_price_input <- 97.0

```

```

bond_obj <- GaussQuant::fixed_rate_bond_GQL(
  issue_date = issue_date_chr,
  maturity_date = maturity_date_chr,
  face_amount = face_amount,
  coupon_rate = coupon_rate,
  settlement_days = settlement_days,
  calendar = jp_calendar,
  accrual_day_counter = dc_30,
  payment_convention = "Unadjusted",
  schedule_convention = "Unadjusted",
  maturity_convention = "Unadjusted",
  date_generation = "Backward",
  end_of_month = FALSE,
  schedule_frequency = GaussQuant::period_months_GQL(6)
)

bond_schedule <- QuantLib::Schedule(
  issue_date_ql,
  maturity_date_ql,
  GaussQuant::period_months_GQL(6),
  jp_calendar,
  "Unadjusted",
  "Unadjusted",
  "Backward",
  FALSE
)

schedule_tbl <- GaussQuant::schedule_dates_GQL(bond_schedule)

bond_setup_tbl <- tibble::tibble(
  metric = c(
    "bond_class",
    "face_amount",
    "coupon_rate",
    "clean_price_input"
  ),
  value = list(
    paste(class(bond_obj), collapse = ", "),
    face_amount,
    coupon_rate,
    clean_price_input
  )
)

```

```

)

GaussQuant::show_tbl_GQH(GaussQuant::metric_tbl_display_GQH(bond_setup_tbl), "Fixed-rate bond")
#> # A tibble: 4 x 2
#>   metric      value
#>   <chr>      <chr>
#> 1 bond_class    _p_FixedRateBond
#> 2 face_amount  100.00000000
#> 3 coupon_rate   0.00370000
#> 4 clean_price_input 97.00000000
GaussQuant::show_tbl_GQH(schedule_tbl, "Bond schedule", n = 20)
#> # A tibble: 7 x 1
#>   schedule_date
#>   <date>
#> 1 2022-07-28
#> 2 2023-01-28
#> 3 2023-07-28
#> 4 2024-01-28
#> 5 2024-07-28
#> 6 2025-01-28
#> 7 2025-07-28

```

11.3.1 1.3.1

97

```

price_to_yield <- GaussQuant::bond_yield_from_clean_safe_GQL(
  bond = bond_obj,
  clean_price = clean_price_input,
  day_counter = dc_30,
  compounding = compd_compounded,
  frequency = freq_semiannual
)

yield_tbl <- tibble::tibble(
  metric = c(
    "clean_price_input",
    "yield_from_clean_price",
    "yield_percent"
  )
)

```

```

),
value = list(
  clean_price_input,
  price_to_yield,
  100 * price_to_yield
)
)

GaussQuant::show_tbl_GQH(GaussQuant::metric_tbl_display_GQH(yield_tbl), "Clean price -> yield")
#> # A tibble: 3 x 2
#>   metric      value
#>   <chr>      <chr>
#> 1 clean_price_input 97.00000000
#> 2 yield_from_clean_price 0.01418724
#> 3 yield_percent      1.41872370

```

11.3.2 1.3.2

$$\text{Dirty Price} = \text{Clean Price} + \text{Accrued Interest}$$

```

price_tbl <- GaussQuant::bond_price_measures_GQL(
  bond = bond_obj,
  yield = price_to_yield,
  day_counter = dc_30,
  compounding = cmpd_compounded,
  frequency = freq_semiannual,
  settlement_date = settlement_date_chr
)

price_tbl <- GaussQuant::complete_accrual_metrics_GQL(
  price_tbl = price_tbl,
  bond_schedule = bond_schedule,
  settlement_date = settlement_date,
  day_counter = dc_30
)

```



```

price_tbl_display <- GaussQuant::metric_tbl_display_GQH(price_tbl)

GaussQuant::show_tbl_GQH(price_tbl_display, "Yield -> price summary")
#> # A tibble: 6 x 2
#>   metric          value
#>   <chr>          <chr>
#> 1 settlement_date 2022-08-23
#> 2 previous_coupon_date 2022-07-28
#> 3 accrued_days    25.00000000
#> 4 accrued_amount  0.02569444
#> 5 clean_price     97.00000012
#> 6 dirty_price     97.02569456

dirty_price_from_yield <- GaussQuant::get_metric_num_GQH(price_tbl, "dirty_price")
clean_price_from_yield <- GaussQuant::get_metric_num_GQH(price_tbl, "clean_price")
accrued_amount <- GaussQuant::get_metric_num_GQH(price_tbl, "accrued_amount")

cat(
  "clean price:",
  sprintf("%.8f", clean_price_from_yield),
  "\n",
  "dirty price:",
  sprintf("%.8f", dirty_price_from_yield),
  "\n",
  "accrued:",
  sprintf("%.8f", accrued_amount),
  "\n"
)
#> clean price: 97.00000012
#> dirty price: 97.02569456
#> accrued: 0.02569444

```

97

11.3.3 1.3.3

```

risk_tbl <- GaussQuant::bond_risk_measures_GQL(
  bond = bond_obj,
  yield = price_to_yield,

```

```

    day_counter = dc_30,
    compounding = compd_compounded,
    frequency = freq_semiannual
)

risk_tbl_display <- GaussQuant::metric_tbl_display_GQH(risk_tbl)

GaussQuant::show_tbl_GQH(risk_tbl_display, "Bond risk measures")
#> # A tibble: 4 x 2
#>   metric          value
#>   <chr>          <chr>
#> 1 modified_duration 2.89593215
#> 2 bpv              0.02809798
#> 3 dv01             0.02809798
#> 4 convexity        9.84952310

modified_duration <- GaussQuant::get_metric_num_GQH(risk_tbl, "modified_duration")
convexity_value <- GaussQuant::get_metric_num_GQH(risk_tbl, "convexity")

cat(
  "modified duration:",
  sprintf("%.8f", modified_duration),
  "\n",
  "convexity:",
  sprintf("%.8f", convexity_value),
  "\n"
)
#> modified duration: 2.89593215
#> convexity: 9.84952310

```

11.3.4 1.3.4

Δy

$$\frac{\Delta P}{P} \approx -D_{\text{mod}} \Delta y + \frac{1}{2} C (\Delta y)^2$$

```

handcalc_tbl <- GaussQuant::bond_risk_handcalc_GQL(
  bond = bond_obj,
  yield = price_to_yield,
  dirty_price = dirty_price_from_yield,
  modified_duration = modified_duration,
  convexity = convexity_value,
  day_counter = dc_30,
  compounding = compd_compounded,
  frequency = freq_semiannual
)

handcalc_tbl_display <- GaussQuant::metric_tbl_display_GQH(handcalc_tbl)

GaussQuant::show_tbl_GQH(handcalc_tbl_display, "Risk hand calculations")
#> # A tibble: 5 x 2
#>   metric                value
#>   <chr>                <chr>
#> 1 bpv_hand             -0.02809798
#> 2 modified_duration_hand 0.02895932
#> 3 convexity_hand        0.00098495
#> 4 price_up_100bp        94.26306473
#> 5 price_approx_delta_gamma 94.26367912

```

11.4 1.4

```

schedule_dates_chr <- schedule_tbl[[1]] |>
  as.character()

schedule_dates_chr <- schedule_dates_chr[!is.na(schedule_dates_chr)]

future_dates_chr <- schedule_dates_chr[schedule_dates_chr > settlement_date_chr]

coupon_per_period <- face_amount * coupon_rate / 2

manual_cf_tbl <- tibble::tibble(
  payment_date = future_dates_chr,

```

```

period_no = seq_along(future_dates_chr),
cashflow = coupon_per_period
) |>
dplyr::mutate(
  cashflow = ifelse(
    .data$payment_date == maturity_date_chr,
    .data$cashflow + face_amount,
    .data$cashflow
  ),
  ql_date = purrr::map(payment_date, GaussQuant::date_GQL),
  time = purrr::map_dbl(
    ql_date,
    ~ dc_30$yearFraction(settlement_date_ql, .x)
  ),
  discount_factor = 1 / (1 + price_to_yield / 2)^(2 * time),
  pv = cashflow * discount_factor
) |>
dplyr::select(-ql_date)

manual_dirty_price <- sum(manual_cf_tbl$pv, na.rm = TRUE)
manual_clean_price <- manual_dirty_price - accrued_amount

manual_summary_tbl <- tibble::tibble(
  metric = c(
    "manual_dirty_price",
    "manual_clean_price",
    "wrapper_dirty_price",
    "wrapper_clean_price",
    "dirty_price_difference",
    "clean_price_difference"
  ),
  value = list(
    manual_dirty_price,
    manual_clean_price,
    dirty_price_from_yield,
    clean_price_from_yield,
    manual_dirty_price - dirty_price_from_yield,
    manual_clean_price - clean_price_from_yield
  )
)

GaussQuant::show_tbl_GQH(manual_cf_tbl, "Manual cashflow PV table")

```

```

#> # A tibble: 6 x 6
#>   payment_date period_no cashflow    time discount_factor    pv
#>   <chr>          <int>    <dbl>    <dbl>        <dbl>    <dbl>
#> 1 2023-01-28         1    0.185 0.43056      0.99393 0.18388
#> 2 2023-07-28         2    0.185 0.93056      0.98693 0.18258
#> 3 2024-01-28         3    0.185 1.4306      0.97998 0.18130
#> 4 2024-07-28         4    0.185 1.9306      0.97308 0.18002
#> 5 2025-01-28         5    0.185 2.4306      0.96622 0.17875
#> 6 2025-07-28         6   100.18 2.9306      0.95942 96.119
GaussQuant::show_tbl_GQH(GaussQuant::metric_tbl_display_GQH(manual_summary_tbl), "Manual PV")
#> # A tibble: 6 x 2
#>   metric          value
#>   <chr>          <chr>
#> 1 manual_dirty_price 97.02569456
#> 2 manual_clean_price 97.00000012
#> 3 wrapper_dirty_price 97.02569456
#> 4 wrapper_clean_price 97.00000012
#> 5 dirty_price_difference 0.00000000
#> 6 clean_price_difference 0.00000000

```

11.4.1 1.4.1

```

flat_yield_handle <- GaussQuant::bond_flat_forward_handle_GQL(
  settlement_date = settlement_date_chr,
  rate = price_to_yield,
  day_counter = dc_30,
  compounding = compd_compounded,
  frequency = freq_semiannual
)

flat_engine <- QuantLib::DiscountingBondEngine(flat_yield_handle)

QuantLib::Instrument_setPricingEngine(
  bond_obj,
  flat_engine
)
#> NULL

```



```

zspread_tbl <- tibble::tibble(
  metric = c(
    "base_yield",
    "z_spread_bp",
    "dirty_price_before_spread",
    "dirty_price_after_spread",
    "price_difference"
  ),
  value = list(
    price_to_yield,
    10000 * z_spread,
    flat_dirty_price,
    zspread_dirty_price,
    zspread_dirty_price - flat_dirty_price
  )
)

GaussQuant::show_tbl_GQH(
  GaussQuant::metric_tbl_display_GQH(zspread_tbl),
  "Z-spread pricing"
)
#> # A tibble: 5 x 2
#>   metric          value
#>   <chr>          <chr>
#> 1 base_yield    0.01418724
#> 2 z_spread_bp   50.00000000
#> 3 dirty_price_before_spread 97.02569456
#> 4 dirty_price_after_spread 95.63266391
#> 5 price_difference -1.39303065

```

Z-spread Z-spread
 ### 1.4.3

```

bond_cf_tbl <- GaussQuant::bond_cashflow_table_GQL(
  bond = bond_obj
)

GaussQuant::show_tbl_GQH(bond_cf_tbl, "Bond cashflow table", n = 30)
#> # A tibble: 7 x 2
#>   date          amount

```

```
#>   <chr>           <dbl>
#> 1 2023-01-28     0.18500
#> 2 2023-07-28     0.18500
#> 3 2024-01-28     0.18500
#> 4 2024-07-28     0.18500
#> 5 2025-01-28     0.18500
#> 6 2025-07-28     0.18500
#> 7 2025-07-28    100
```

11.5 1.5

```
summary_tbl <- tibble::tibble(
  metric = c(
    "trade_date",
    "settlement_date",
    "issue_date",
    "maturity_date",
    "clean_price_input",
    "yield_from_clean_price",
    "clean_price_from_yield",
    "dirty_price_from_yield",
    "accrued_amount",
    "modified_duration",
    "convexity"
  ),
  value = c(
    GaussQuant::iso_GQL(trade_date),
    settlement_date_chr,
    issue_date_chr,
    maturity_date_chr,
    sprintf("%.8f", clean_price_input),
    sprintf("%.8f", price_to_yield),
    sprintf("%.8f", clean_price_from_yield),
    sprintf("%.8f", dirty_price_from_yield),
    sprintf("%.8f", accrued_amount),
    sprintf("%.8f", modified_duration),
    sprintf("%.8f", convexity_value)
  )
)
```



```

    )
)

GaussQuant::show_tbl_GQH(summary_tbl, "Bond analysis summary", n = 20)
#> # A tibble: 11 x 2
#>   metric          value
#>   <chr>         <chr>
#> 1 trade_date    2022-08-19
#> 2 settlement_date 2022-08-23
#> 3 issue_date    2022-07-28
#> 4 maturity_date  2025-07-28
#> 5 clean_price_input 97.00000000
#> 6 yield_from_clean_price 0.01418724
#> 7 clean_price_from_yield 97.00000012
#> 8 dirty_price_from_yield 97.02569456
#> 9 accrued_amount  0.02569444
#> 10 modified_duration 2.89593215
#> 11 convexity      9.84952310

cat("\nchapter01 bonds stable rewrite completed successfully.\n")
#>
#> chapter01 bonds stable rewrite completed successfully.

```

12

II-2

CTD BPV

FRA

Implied Term

Structure

12.12.1

0

t

t1

D(0,t)

D(0,t)

z(0,t)

$$D(0,t)=\exp\left[-z(0,t)t\right]$$

$$z(0,t)=-\frac{\log D(0,t)}{t}$$

t1

t2

f(t1,t2)

$$f(t_1,t_2)=-\frac{\log\left[D(0,t_2)/D(0,t_1)\right]}{t_2-t_1}$$

12.1.1.12.1.1

```

evaluation_date_chr <- "2022-08-19"

GaussQuant::set_eval_date_GQL(evaluation_date_chr)

zero_nodes <- tibble::tribble(
  ~tenor, ~date, ~zero_rate,
  "0D",   as.Date("2022-08-19"), 0.0005,
  "1M",   as.Date("2022-09-19"), 0.0006,
  "3M",   as.Date("2022-11-21"), 0.0008,
  "6M",   as.Date("2023-02-20"), 0.0011,
  "1Y",   as.Date("2023-08-21"), 0.0018,
  "2Y",   as.Date("2024-08-19"), 0.0032,
  "3Y",   as.Date("2025-08-19"), 0.0048,
  "5Y",   as.Date("2027-08-19"), 0.0075,
  "10Y",  as.Date("2032-08-19"), 0.0115
)

GaussQuant::show_tbl_GQH(
  zero_nodes,
  "Zero-rate nodes",
  n = 20
)

#> # A tibble: 9 x 3
#>   tenor date      zero_rate
#>   <chr> <date>      <dbl>
#> 1 0D    2022-08-19    0.0005
#> 2 1M    2022-09-19    0.0006
#> 3 3M    2022-11-21    0.0008
#> 4 6M    2023-02-20    0.0011
#> 5 1Y    2023-08-21    0.0018
#> 6 2Y    2024-08-19    0.0032
#> 7 3Y    2025-08-19    0.0048
#> 8 5Y    2027-08-19    0.0075
#> 9 10Y   2032-08-19    0.0115

```

12.1.2 2.1.2

build_zero_curve_GQL() QuantLib ZeroCurve Actual/365 Fixed

```

zero_curve <- GaussQuant::build_zero_curve_GQL(
  date_chr = as.character(zero_nodes$date),

```

```

    zero_rates = zero_nodes$zero_rate,
    day_counter = "Actual365Fixed"
  )

zero_curve_handle <- GaussQuant::yield_curve_handle_GQL(
  zero_curve
)

zero_curve_tbl <- GaussQuant::curve_tbl_GQH(
  curve = zero_curve,
  tenors = c(
    "1D", "1W", "1M", "3M", "6M",
    "1Y", "2Y", "3Y", "5Y", "7Y", "10Y"
  )
)

GaussQuant::show_tbl_GQH(
  zero_curve_tbl,
  "Zero curve",
  n = 20
)
#> # A tibble: 11 x 4
#>   tenor date      discount zero_rate
#>   <chr> <date>      <dbl>     <dbl>
#> 1 1D    2022-08-20  1.00000 0.00050323
#> 2 1W    2022-08-26  0.99999 0.00052258
#> 3 1M    2022-09-19  0.99995 0.00060000
#> 4 3M    2022-11-19  0.99980 0.00079365
#> 5 6M    2023-02-19  0.99945 0.0010967
#> 6 1Y    2023-08-19  0.99821 0.0017923
#> 7 2Y    2024-08-19  0.99361 0.0032000
#> 8 3Y    2025-08-19  0.98569 0.0048000
#> 9 5Y    2027-08-19  0.96317 0.0075
#> 10 7Y    2029-08-19  0.93824 0.0091004
#> 11 10Y   2032-08-19  0.89128 0.0115

```

QuantLib

12.1.3 2.1.3

```

zero_curve_grid <- GaussQuant::curve_grid_tbl_GQL(
  curve = zero_curve,
  n = 240L,
  extrapolate = TRUE
) |>
dplyr::mutate(
  next_time = dplyr::lead(.data$time),
  next_discount = dplyr::lead(.data$discount_factor),
  forward_rate = dplyr::if_else(
    !is.na(.data$next_time) &
    .data$next_time > .data$time &
    .data$discount_factor > 0 &
    .data$next_discount > 0,
    -log(.data$next_discount / .data$discount_factor) /
      (.data$next_time - .data$time),
    NA_real_
  )
)

grid_show_index <- unique(
  round(
    seq(
      from = 1,
      to = nrow(zero_curve_grid),
      length.out = 12
    )
  )
)

GaussQuant::show_tbl_GQH(
  zero_curve_grid |>
  dplyr::slice(grid_show_index) |>
  dplyr::select(
    .data$curve_date,
    .data$time,
    .data$discount_factor,
    .data$zero_rate,
    .data$forward_rate
  ),
  "Zero-curve grid",
  n = 20
)

```

```
#> # A tibble: 12 x 5
#>   curve_date      time discount_factor zero_rate forward_rate
#>   <date>        <dbl>          <dbl>    <dbl>    <dbl>
#> 1 2022-08-19    0            1          0      0.00054839
#> 2 2023-07-21  0.92055        0.99845 0.0016808 0.0030308
#> 3 2024-06-06  1.8            0.99477 0.0029154 0.0055000
#> 4 2025-05-08  2.7205        0.98824 0.0043485 0.0087671
#> 5 2026-04-09  3.6411        0.97960 0.0056618 0.010636
#> 6 2027-03-12  4.5644        0.96896 0.0069082 0.013126
#> 7 2028-01-26  5.4411        0.95819 0.0078503 0.012233
#> 8 2028-12-28  6.3644        0.94681 0.0085881 0.013707
#> 9 2029-11-29  7.2849        0.93433 0.0093238 0.015178
#> 10 2030-10-31  8.2055        0.92077 0.010059 0.016652
#> 11 2031-09-17  9.0849        0.90685 0.010762 0.018057
#> 12 2032-08-19 10.008        0.89128 0.0115    NA
```

```
discount_factor_plot <- zero_curve_grid |>
```

```
  ggplot2::ggplot(
    ggplot2::aes(
      x = .data$curve_date,
      y = .data$discount_factor
    )
  ) +
  ggplot2::geom_line() +
  ggplot2::labs(
    title = "Discount factor curve",
    x = NULL,
    y = "Discount factor"
  ) +
  ggplot2::theme_minimal()
```

```
zero_rate_plot <- zero_curve_grid |>
```

```
  ggplot2::ggplot(
    ggplot2::aes(
      x = .data$curve_date,
      y = 100 * .data$zero_rate
    )
  ) +
  ggplot2::geom_line() +
  ggplot2::labs(
    title = "Continuous zero-rate curve",
    x = NULL,
```

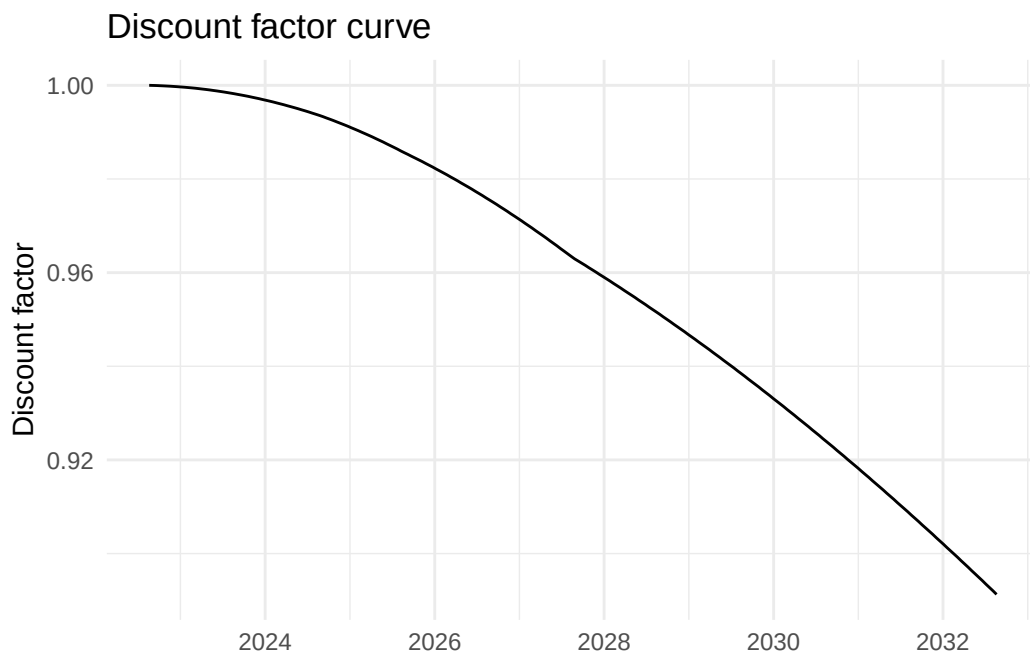
```

    y = "Zero rate (%)"
  ) +
  ggplot2::theme_minimal()

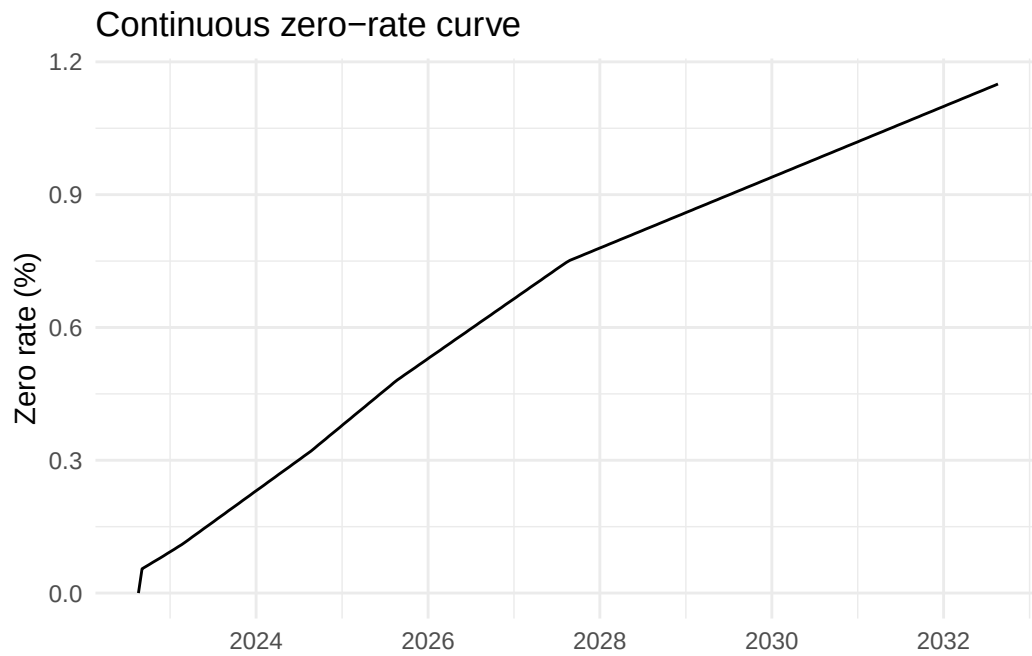
forward_rate_plot <- zero_curve_grid |>
  dplyr::filter(!is.na(.data$forward_rate)) |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = .data$curve_date,
      y = 100 * .data$forward_rate
    )
  ) +
  ggplot2::geom_line() +
  ggplot2::labs(
    title = "One-step forward-rate curve",
    x = NULL,
    y = "Forward rate (%)"
  ) +
  ggplot2::theme_minimal()

print(discount_factor_plot)

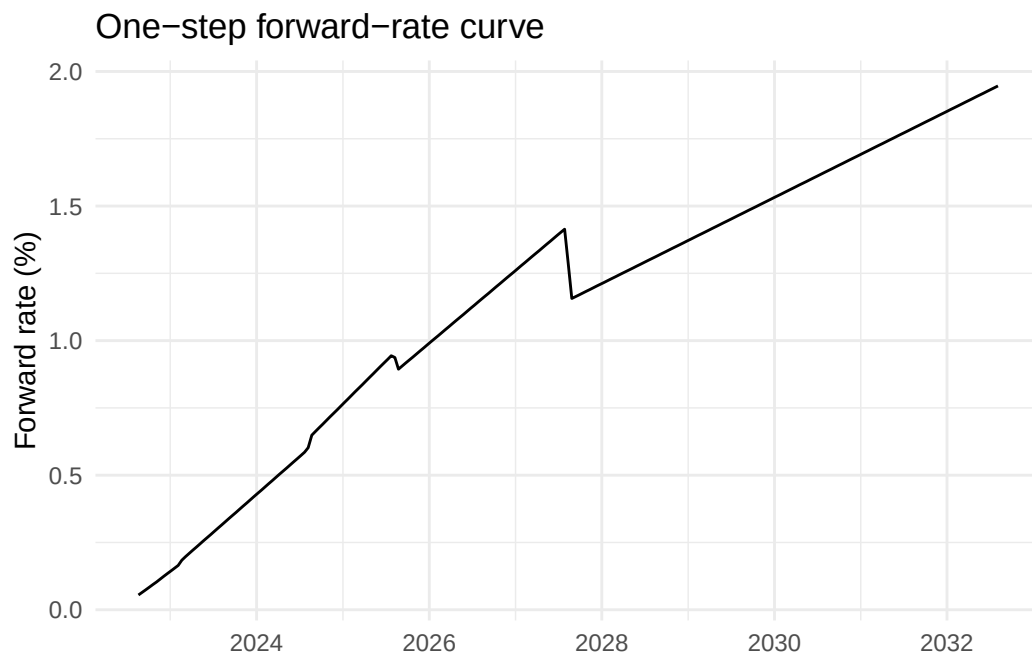
```



```
print(zero_rate_plot)
```



```
print(forward_rate_plot)
```



12.2 2.2

12.2.1 2.2.1

- ZeroCurve
- DiscountCurve

```
discount_curve <- GaussQuant::build_discount_curve_GQL(
  nodes = zero_nodes |>
    dplyr::select(
      .data$date,
      .data$zero_rate
    ),
  day_counter = "Actual365Fixed"
)

discount_curve_handle <- GaussQuant::yield_curve_handle_GQL(
  discount_curve
)

comparison_max_time <- min(
  as.numeric(zero_curve$maxTime()),
  as.numeric(discount_curve$maxTime())
)

comparison_time_grid <- seq(
  from = 0,
  to = comparison_max_time,
  length.out = 300
)

comparison_reference_date <- as.Date(
  GaussQuant::iso_GQL(
    zero_curve$referenceDate()
  )
)
```

```

interpolation_tbl <- tibble::tibble(
  time = comparison_time_grid,
  date = comparison_reference_date +
    round(comparison_time_grid * 365)
) |>
dplyr::mutate(
  ql_date = purrr::map(
    as.character(.data$date),
    GaussQuant::date_GQL
  ),
  zero_curve_discount = purrr::map_dbl(
    .data$ql_date,
    ~ GaussQuant::curve_discount_safe_GQL(
      zero_curve,
      .x
    )
  ),
  discount_curve_discount = purrr::map_dbl(
    .data$ql_date,
    ~ GaussQuant::curve_discount_safe_GQL(
      discount_curve,
      .x
    )
  )
) |>
dplyr::select(
  -.data$ql_date
) |>
tidyr::pivot_longer(
  cols = c(
    zero_curve_discount,
    discount_curve_discount
  ),
  names_to = "interpolation",
  values_to = "discount_factor"
) |>
dplyr::mutate(
  interpolation = dplyr::recode(
    .data$interpolation,
    zero_curve_discount = "ZeroCurve",
    discount_curve_discount = "DiscountCurve"
  )
)

```

```

) |>
dplyr::group_by(
  .data$interpolation
) |>
dplyr::arrange(
  .data$interpolation,
  .data$time
) |>
dplyr::mutate(
  zero_rate = dplyr::if_else(
    .data$time > 0 &
    .data$discount_factor > 0,
    -log(.data$discount_factor) /
    .data$time,
    zero_nodes$zero_rate[[1]]
  ),
  next_time = dplyr::lead(
    .data$time
  ),
  next_discount = dplyr::lead(
    .data$discount_factor
  ),
  forward_rate = dplyr::if_else(
    !is.na(.data$next_time) &
    .data$next_time > .data$time &
    .data$discount_factor > 0 &
    .data$next_discount > 0,
    -log(
      .data$next_discount /
      .data$discount_factor
    ) / (
      .data$next_time -
      .data$time
    ),
    NA_real_
  )
) |>
dplyr::ungroup()

```

```

interpolation_zero_plot <- interpolation_tbl |>
ggplot2::ggplot(
  ggplot2::aes(

```

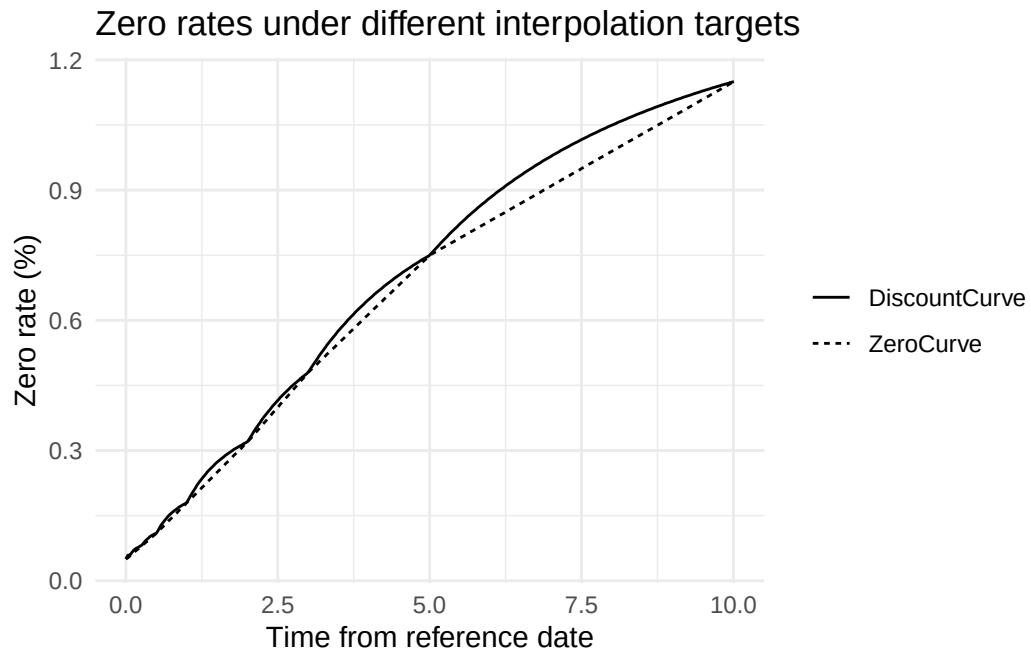
```

      x = .data$time,
      y = 100 * .data$zero_rate,
      linetype = .data$interpolation
    )
  ) +
  ggplot2::geom_line() +
  ggplot2::labs(
    title = "Zero rates under different interpolation targets",
    x = "Time from reference date",
    y = "Zero rate (%)",
    linetype = NULL
  ) +
  ggplot2::theme_minimal()

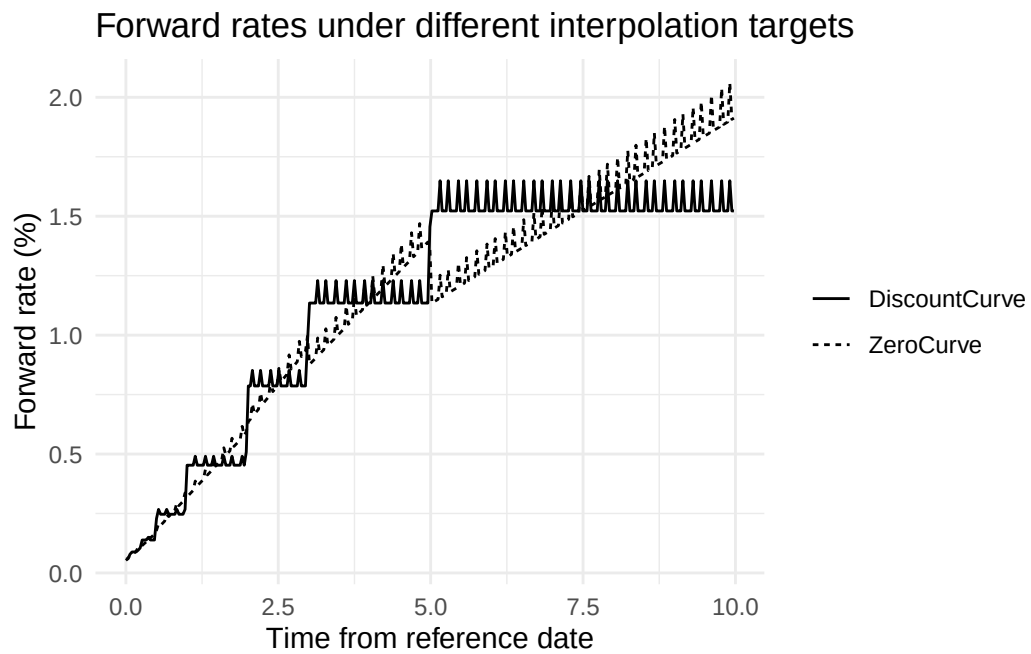
interpolation_forward_plot <- interpolation_tbl |>
  dplyr::filter(!is.na(.data$forward_rate)) |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = .data$time,
      y = 100 * .data$forward_rate,
      linetype = .data$interpolation
    )
  ) +
  ggplot2::geom_line() +
  ggplot2::labs(
    title = "Forward rates under different interpolation targets",
    x = "Time from reference date",
    y = "Forward rate (%)",
    linetype = NULL
  ) +
  ggplot2::theme_minimal()

print(interpolation_zero_plot)

```



```
print(interpolation_forward_plot)
```



12.2.2 2.2.2

1

QuantLib

ZeroCurve

```
reference_date_before <- as.Date(
  GaussQuant::iso_GQL(
    zero_curve$referenceDate()
  )
)

GaussQuant::set_eval_date_GQL(
  "2022-09-19"
)

reference_date_after <- as.Date(
  GaussQuant::iso_GQL(
    zero_curve$referenceDate()
  )
)

reference_date_tbl <- tibble::tribble(
  ~item, ~value,
  "curve_reference_date_before", as.character(reference_date_before),
  "evaluation_date_after_change", "2022-09-19",
  "curve_reference_date_after", as.character(reference_date_after)
)

GaussQuant::show_tbl_GQH(
  reference_date_tbl,
  "Fixed reference date"
)

#> # A tibble: 3 x 2
#>   item                value
#>   <chr>              <chr>
#> 1 curve_reference_date_before 2022-08-19
#> 2 evaluation_date_after_change 2022-09-19
#> 3 curve_reference_date_after 2022-08-19

GaussQuant::set_eval_date_GQL(
  evaluation_date_chr
)
```

12.3 2.3

12.3.1 2.3.1

GaussQuant build_bond_discount_curve_GQL()

PiecewiseFl

```

GaussQuant::set_eval_date_GQL(
  "2008-09-15"
)

bond_curve <- GaussQuant::build_bond_discount_curve_GQL(
  settlement_date = "2008-09-18",
  fixing_days = 3,
  settlement_days = 3
)

bond_curve_tbl <- GaussQuant::curve_tbl_GQH(
  curve = bond_curve,
  tenors = c(
    "1M", "3M", "6M", "1Y",
    "2Y", "3Y", "5Y", "7Y",
    "10Y", "15Y", "20Y"
  )
)

GaussQuant::show_tbl_GQH(
  bond_curve_tbl,
  "Deposit and bond bootstrap curve",
  n = 30
)

#> # A tibble: 11 x 4
#>   tenor date      discount zero_rate
#>   <chr> <date>      <dbl>    <dbl>
#> 1 1M    2008-10-18  0.99921  0.0096148
#> 2 3M    2008-12-18  0.99761  0.0096148
#> 3 6M    2009-03-18  0.99286  0.014471
#> 4 1Y    2009-09-18  0.98097  0.019229
#> 5 2Y    2010-09-18  0.95732  0.021818

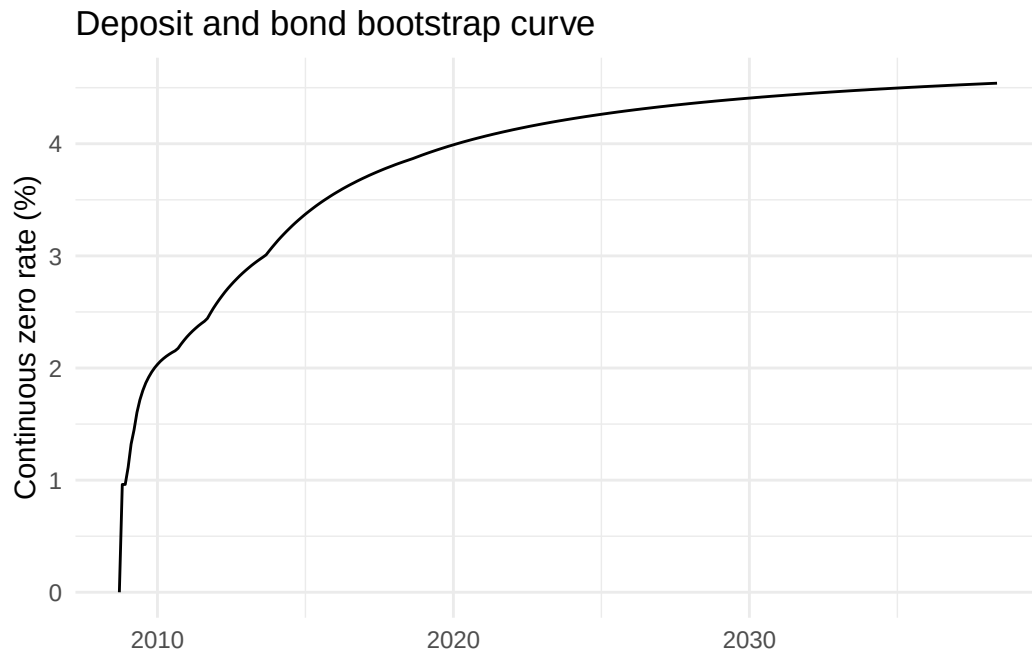
```

```
#> 6 3Y    2011-09-18  0.92904 0.024541
#> 7 5Y    2013-09-18  0.85971 0.030237
#> 8 7Y    2015-09-18  0.78217 0.035102
#> 9 10Y   2018-09-18  0.67867 0.038764
#> 10 15Y  2023-09-18  0.53179 0.042103
#> 11 20Y  2028-09-18  0.41668 0.043772
```

```
bond_curve_grid <- GaussQuant::curve_grid_tbl_GQL(
  curve = bond_curve,
  n = 300L,
  extrapolate = TRUE
)

bond_curve_plot <- bond_curve_grid |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = .data$curve_date,
      y = 100 * .data$zero_rate
    )
  ) +
  ggplot2::geom_line() +
  ggplot2::labs(
    title = "Deposit and bond bootstrap curve",
    x = NULL,
    y = "Continuous zero rate (%)"
  ) +
  ggplot2::theme_minimal()

print(bond_curve_plot)
```

12.3.2 2.3.2

GaussQuant build_swap_curve_GQL()

PiecewiseFlatForward

```
swap_curve <- GaussQuant::build_swap_curve_GQL(
  settlement_date = "2008-09-18",
  fixing_days = 3
)

swap_curve_handle <- GaussQuant::yield_curve_handle_GQL(
  swap_curve
)

swap_curve_tbl <- GaussQuant::curve_tbl_GQH(
  curve = swap_curve,
  tenors = c(
    "1W", "1M", "3M", "6M",
    "1Y", "2Y", "3Y", "5Y",
    "7Y", "10Y", "15Y"
  )
)
```

```

    )
  )

GaussQuant::show_tbl_GQH(
  swap_curve_tbl,
  "Deposit and swap bootstrap curve",
  n = 30
)

#> # A tibble: 11 x 4
#>   tenor date      discount zero_rate
#>   <chr> <date>      <dbl>    <dbl>
#> 1 1W    2008-09-25  0.99916  0.044079
#> 2 1M    2008-10-18  0.99733  0.032579
#> 3 3M    2008-12-18  0.99197  0.032440
#> 4 6M    2009-03-18  0.98327  0.034085
#> 5 1Y    2009-09-18  0.96714  0.033440
#> 6 2Y    2010-09-18  0.94376  0.028954
#> 7 3Y    2011-09-18  0.90902  0.031805
#> 8 5Y    2013-09-18  0.83739  0.035499
#> 9 7Y    2015-09-18  0.76276  0.038692
#> 10 10Y   2018-09-18  0.66310  0.041086
#> 11 15Y   2023-09-18  0.52138  0.043421

```

```

comparison_tenors <- c(
  "1M", "3M", "6M", "1Y",
  "2Y", "3Y", "5Y", "7Y",
  "10Y", "15Y"
)

bootstrap_comparison_tbl <- dplyr::bind_rows(
  GaussQuant::curve_tbl_GQH(
    curve = bond_curve,
    tenors = comparison_tenors
  ) |>
  dplyr::mutate(
    curve = "Deposit + bond"
  ),
  GaussQuant::curve_tbl_GQH(
    curve = swap_curve,
    tenors = comparison_tenors
  ) |>
  dplyr::mutate(

```

```

      curve = "Deposit + swap"
    )
  )

GaussQuant::show_tbl_GQH(
  bootstrap_comparison_tbl,
  "Bootstrap curve comparison",
  n = 30
)

#> # A tibble: 20 x 5
#>   tenor date      discount zero_rate curve
#>   <chr> <date>      <dbl>      <dbl> <chr>
#> 1 1M    2008-10-18  0.99921  0.0096148 Deposit + bond
#> 2 3M    2008-12-18  0.99761  0.0096148 Deposit + bond
#> 3 6M    2009-03-18  0.99286  0.014471  Deposit + bond
#> 4 1Y    2009-09-18  0.98097  0.019229  Deposit + bond
#> 5 2Y    2010-09-18  0.95732  0.021818  Deposit + bond
#> 6 3Y    2011-09-18  0.92904  0.024541  Deposit + bond
#> 7 5Y    2013-09-18  0.85971  0.030237  Deposit + bond
#> 8 7Y    2015-09-18  0.78217  0.035102  Deposit + bond
#> 9 10Y   2018-09-18  0.67867  0.038764  Deposit + bond
#> 10 15Y  2023-09-18  0.53179  0.042103  Deposit + bond
#> 11 1M    2008-10-18  0.99733  0.032579  Deposit + swap
#> 12 3M    2008-12-18  0.99197  0.032440  Deposit + swap
#> 13 6M    2009-03-18  0.98327  0.034085  Deposit + swap
#> 14 1Y    2009-09-18  0.96714  0.033440  Deposit + swap
#> 15 2Y    2010-09-18  0.94376  0.028954  Deposit + swap
#> 16 3Y    2011-09-18  0.90902  0.031805  Deposit + swap
#> 17 5Y    2013-09-18  0.83739  0.035499  Deposit + swap
#> 18 7Y    2015-09-18  0.76276  0.038692  Deposit + swap
#> 19 10Y   2018-09-18  0.66310  0.041086  Deposit + swap
#> 20 15Y  2023-09-18  0.52138  0.043421  Deposit + swap

bootstrap_zero_plot <- bootstrap_comparison_tbl |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = .data$date,
      y = 100 * .data$zero_rate,
      linetype = .data$curve
    )
  ) +
  ggplot2::geom_line() +

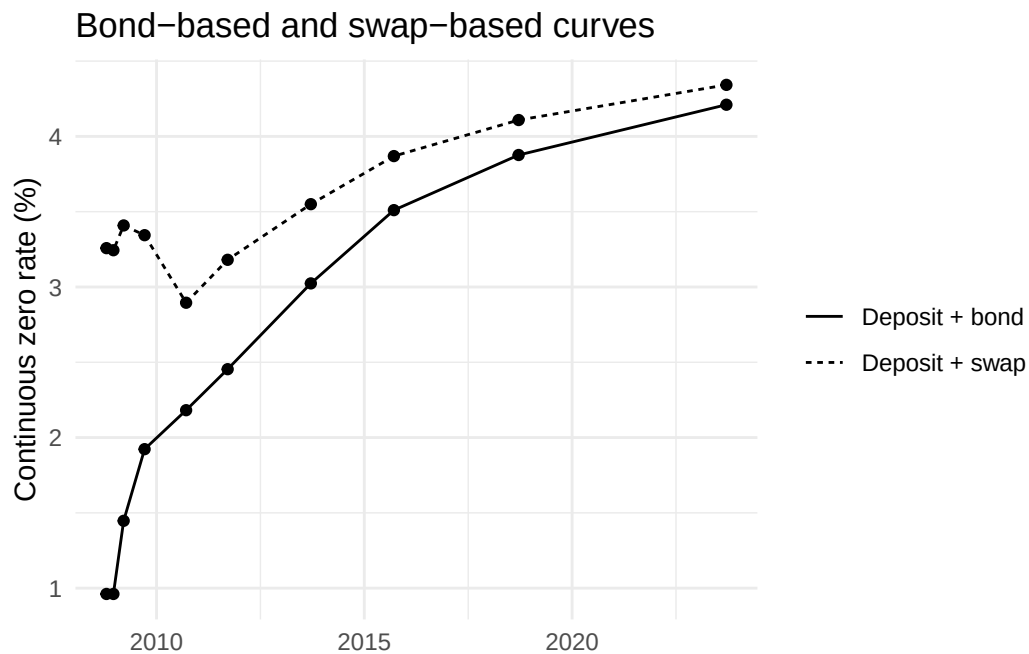
```

```

ggplot2::geom_point() +
ggplot2::labs(
  title = "Bond-based and swap-based curves",
  x = NULL,
  y = "Continuous zero rate (%)",
  linetype = NULL
) +
ggplot2::theme_minimal()

print(bootstrap_zero_plot)

```



FRA

FRA

12.4 2.4

12.4.1 2.4.1 Implied Term Structure

Implied Term Structure
Structure

t_0 t_1 t_2 Implied Term

$$D_{t_1}(t_2) = \frac{D_{t_0}(t_2)}{D_{t_0}(t_1)}$$

```

implied_reference_date <- GaussQuant::date_GQL(
  "2009-09-18"
)

implied_constructor <- get0(
  "ImpliedTermStructure",
  envir = asNamespace("QuantLib"),
  inherits = FALSE
)

implied_curve <- if (is.null(implied_constructor)) {
  NULL
} else {
  tryCatch(
    implied_constructor(
      swap_curve_handle,
      implied_reference_date
    ),
    error = function(e) NULL
  )
}

implied_curve_tbl <- if (is.null(implied_curve)) {
  tibble::tibble(
    status = "ImpliedTermStructure is not available in the installed QuantLib binding."
  )
} else {
  GaussQuant::curve_tbl_GQH(
    curve = implied_curve,
    tenors = c(
      "1M", "3M", "6M", "1Y",
      "2Y", "3Y", "5Y", "10Y"
    )
  )
}

GaussQuant::show_tbl_GQH(
  implied_curve_tbl,
  "Implied term structure",

```

```

  n = 20
)
#> # A tibble: 8 x 4
#>   tenor date      discount zero_rate
#>   <chr> <date>      <dbl>    <dbl>
#> 1 1M    2009-10-18  0.99799  0.024471
#> 2 3M    2009-12-18  0.99392  0.024471
#> 3 6M    2010-03-18  0.98794  0.024471
#> 4 1Y    2010-09-18  0.97583  0.024471
#> 5 2Y    2011-09-18  0.93990  0.030989
#> 6 3Y    2012-09-18  0.90209  0.034338
#> 7 5Y    2014-09-18  0.82636  0.038145
#> 8 10Y   2019-09-18  0.65344  0.042551

```

Implied Term Structure

12.4.2 2.4.2

50

```

parallel_spread <- 50 / 10000

spreaded_curve_bundle <- GaussQuant::make_zero_spreaded_curve_GQL(
  base_curve_handle = swap_curve_handle,
  spread_rate = parallel_spread,
  day_counter = QuantLib::Actual365Fixed(),
  extrapolate = TRUE
)

spreaded_curve <- spreaded_curve_bundle$curve

parallel_shift_tbl <- dplyr::bind_rows(
  GaussQuant::curve_tbl_GQH(
    curve = swap_curve,
    tenors = comparison_tenors
  ) |>
  dplyr::mutate(
    curve = "Base curve"
  ),
  GaussQuant::curve_tbl_GQH(

```

```

    curve = spreaded_curve,
    tenors = comparison_tenors
  ) |>
  dplyr::mutate(
    curve = "Base curve + 50bp"
  )
)

parallel_shift_wide_tbl <- parallel_shift_tbl |>
  dplyr::select(
    .data$tenor,
    .data$date,
    .data$curve,
    .data$discount,
    .data$zero_rate
  ) |>
  tidyr::pivot_wider(
    names_from = curve,
    values_from = c(
      discount,
      zero_rate
    )
  ) |>
  dplyr::mutate(
    observed_zero_shift_bp =
      10000 * (
        .data$`zero_rate_Base curve + 50bp` -
        .data$`zero_rate_Base curve`
      )
  )
)

GaussQuant::show_tbl_GQH(
  parallel_shift_wide_tbl,
  "Parallel curve shift",
  n = 30
)

#> # A tibble: 10 x 7
#>   tenor date      `discount_Base curve` `discount_Base curve + 50bp`
#>   <chr> <date>          <dbl>          <dbl>
#> 1 1M    2008-10-18          0.99733          0.99693
#> 2 3M    2008-12-18          0.99197          0.99074
#> 3 6M    2009-03-18          0.98327          0.98088

```

```

#> 4 1Y      2009-09-18      0.96714      0.96249
#> 5 2Y      2010-09-18      0.94376      0.93494
#> 6 3Y      2011-09-18      0.90902      0.89679
#> 7 5Y      2013-09-18      0.83739      0.82022
#> 8 7Y      2015-09-18      0.76276      0.74293
#> 9 10Y     2018-09-18      0.66310      0.64182
#> 10 15Y    2023-09-18      0.52138      0.50176
#>      `zero_rate_Base curve` `zero_rate_Base curve + 50bp` observed_zero_shift_bp
#>      <dbl>                <dbl>                <dbl>
#> 1          0.032579          0.037565          49.856
#> 2          0.032440          0.037397          49.568
#> 3          0.034085          0.038996          49.104
#> 4          0.033440          0.038264          48.240
#> 5          0.028954          0.033650          46.967
#> 6          0.031805          0.036320          45.144
#> 7          0.035499          0.039642          41.437
#> 8          0.038692          0.042456          37.638
#> 9          0.041086          0.044348          32.617
#> 10         0.043421          0.045978          25.572

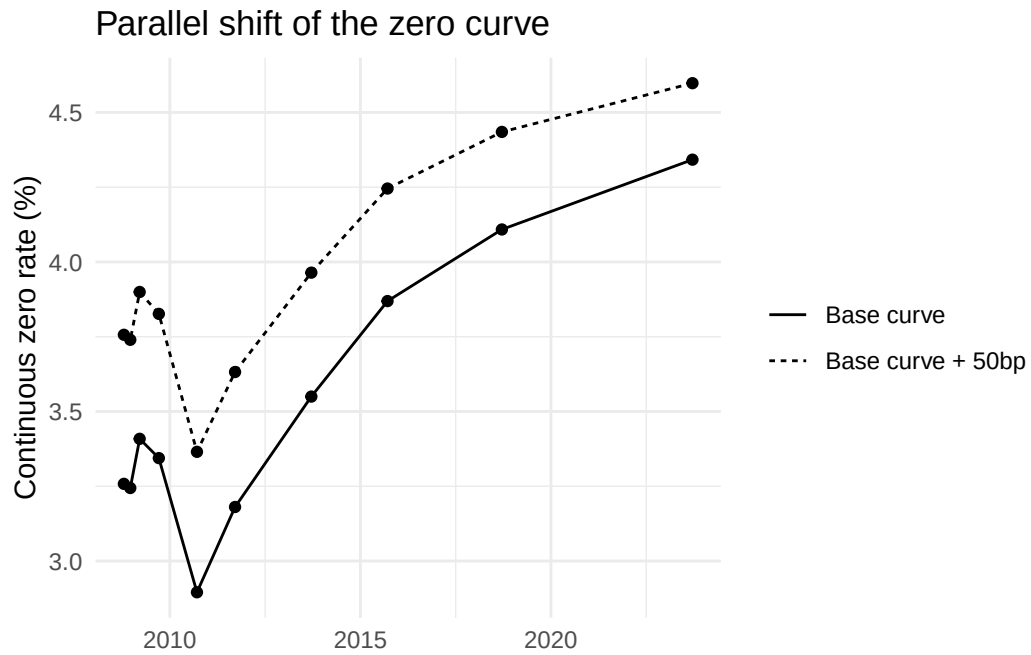
```

```

parallel_shift_plot <- parallel_shift_tbl |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = .data$date,
      y = 100 * .data$zero_rate,
      linetype = .data$curve
    )
  ) +
  ggplot2::geom_line() +
  ggplot2::geom_point() +
  ggplot2::labs(
    title = "Parallel shift of the zero curve",
    x = NULL,
    y = "Continuous zero rate (%)",
    linetype = NULL
  ) +
  ggplot2::theme_minimal()

print(parallel_shift_plot)

```

Key Rate Shift

12.4.3 2.4.3

-
-
-
-
-

```
node_recovery_tbl <- zero_nodes |>
  dplyr::mutate(
    ql_date = purrr::map(
      as.character(.data$date),
      GaussQuant::date_GQL
    ),
    time = as.numeric(
```

```

    .data$date -
      as.Date(evaluation_date_chr)
  ) / 365,
  zero_curve_discount = purrr::map_dbl(
    .data$ql_date,
    ~ GaussQuant::curve_discount_safe_GQL(
      zero_curve,
      .x
    )
  ),
  discount_curve_discount = purrr::map_dbl(
    .data$ql_date,
    ~ GaussQuant::curve_discount_safe_GQL(
      discount_curve,
      .x
    )
  ),
  zero_curve_implied_rate = dplyr::if_else(
    .data$time > 0,
    -log(.data$zero_curve_discount) /
      .data$time,
    .data$zero_rate
  ),
  discount_curve_implied_rate = dplyr::if_else(
    .data$time > 0,
    -log(.data$discount_curve_discount) /
      .data$time,
    .data$zero_rate
  ),
  zero_curve_error =
    .data$zero_curve_implied_rate -
    .data$zero_rate,
  discount_curve_error =
    .data$discount_curve_implied_rate -
    .data$zero_rate
  ) |>
dplyr::select(
  .data$tenor,
  .data$date,
  .data$zero_rate,
  .data$zero_curve_implied_rate,
  .data$discount_curve_implied_rate,

```

```

    .data$zero_curve_error,
    .data$discount_curve_error
  )

GaussQuant::show_tbl_GQH(
  node_recovery_tbl,
  "Node recovery check",
  n = 20
)

#> # A tibble: 9 x 7
#>   tenor date          zero_rate zero_curve_implied_rate discount_curve_implied_rate
#>   <chr> <date>          <dbl>          <dbl>          <dbl>
#> 1 0D    2022-08-19    0.0005          0.0005          0.0005
#> 2 1M    2022-09-19    0.0006          0.00060000     0.00060000
#> 3 3M    2022-11-21    0.0008          0.00080000     0.00080000
#> 4 6M    2023-02-20    0.0011          0.0011000     0.0011000
#> 5 1Y    2023-08-21    0.0018          0.0018000     0.0018000
#> 6 2Y    2024-08-19    0.0032          0.0032000     0.0032000
#> 7 3Y    2025-08-19    0.0048          0.0048000     0.0048000
#> 8 5Y    2027-08-19    0.0075          0.0075         0.0075
#> 9 10Y   2032-08-19    0.0115          0.0115         0.0115
#>   zero_curve_error discount_curve_error
#>           <dbl>           <dbl>
#> 1           0           0
#> 2      2.2757e-16      2.2757e-16
#> 3     -1.8117e-16     -1.8117e-16
#> 4      6.3317e-17      6.3317e-17
#> 5      1.7781e-17      1.7781e-17
#> 6      6.9389e-18      6.9389e-18
#> 7      8.6736e-18      8.6736e-18
#> 8      2.6021e-18      2.6021e-18
#> 9      1.7347e-18      1.7347e-18

```

```

curve_validation_tbl <- dplyr::bind_rows(
  GaussQuant::curve_grid_tbl_GQL(
    zero_curve,
    n = 200L,
    extrapolate = TRUE
  ) |>
  dplyr::mutate(

```

```

      curve = "ZeroCurve"
    ),
    GaussQuant::curve_grid_tbl_GQL(
      discount_curve,
      n = 200L,
      extrapolate = TRUE
    ) |>
    dplyr::mutate(
      curve = "DiscountCurve"
    ),
    GaussQuant::curve_grid_tbl_GQL(
      bond_curve,
      n = 200L,
      extrapolate = TRUE
    ) |>
    dplyr::mutate(
      curve = "Deposit + bond"
    ),
    GaussQuant::curve_grid_tbl_GQL(
      swap_curve,
      n = 200L,
      extrapolate = TRUE
    ) |>
    dplyr::mutate(
      curve = "Deposit + swap"
    ),
    GaussQuant::curve_grid_tbl_GQL(
      spreaded_curve,
      n = 200L,
      extrapolate = TRUE
    ) |>
    dplyr::mutate(
      curve = "Deposit + swap + 50bp"
    )
  ) |>
  dplyr::group_by(.data$curve) |>
  dplyr::summarise(
    first_date = min(
      .data$curve_date,
      na.rm = TRUE
    ),
    last_date = max(

```

```

    .data$curve_date,
    na.rm = TRUE
  ),
  min_discount = min(
    .data$discount_factor,
    na.rm = TRUE
  ),
  max_discount = max(
    .data$discount_factor,
    na.rm = TRUE
  ),
  nonpositive_discount_count = sum(
    .data$discount_factor <= 0,
    na.rm = TRUE
  ),
  missing_discount_count = sum(
    is.na(.data$discount_factor)
  ),
  missing_zero_rate_count = sum(
    is.na(.data$zero_rate)
  ),
  .groups = "drop"
)

GaussQuant::show_tbl_GQH(
  curve_validation_tbl,
  "Curve validation summary",
  n = 20
)

#> # A tibble: 5 x 8
#>   curve          first_date last_date  min_discount max_discount
#>   <chr>          <date>      <date>      <dbl>         <dbl>
#> 1 Deposit + bond 2008-09-18 2038-05-15    0.26019         1
#> 2 Deposit + swap 2008-09-18 2023-09-18    0.52138         1
#> 3 Deposit + swap + 50bp 2008-09-18 2023-09-18    0.50176         1
#> 4 DiscountCurve 2022-08-19 2032-08-19    0.89128         1
#> 5 ZeroCurve      2022-08-19 2032-08-19    0.89128         1
#>   nonpositive_discount_count missing_discount_count missing_zero_rate_count
#>   <int>                <int>                <int>
#> 1         0                  0                  0
#> 2         0                  0                  0
#> 3         0                  0                  0

```

```
#> 4 0 0
#> 5 0 0
```

12.5 2.5

version Factor: CF

Con-

to-Deliver: CTD

Cheapest-

CTD

CTD

CTD

BPV

```
knitr::opts_chunk$set(
  collapse = TRUE,
  comment = "#>",
  warning = FALSE,
  message = FALSE
)

suppressPackageStartupMessages({
  library(tidyverse)
  library(QuantLib)
  library(GaussQuant)
})
```

12.5.1 2.5.1

	Actual/Actual Bond	Actual/360	1
100	32 1 109 + 10 / 32	109-10	

```

cal_us_gov <- QuantLib::UnitedStates("GovernmentBond")

dc_act_act_bond <- GaussQuant::day_counter_GQL("ActualActual_Bond")
dc_act_360 <- GaussQuant::day_counter_GQL("Actual360")

compd_compounded <- QuantLib::Compounding_Compounded_get()
freq_semiannual <- QuantLib::Frequency_Semiannual_get()

settlement_days_t1 <- 1L
trade_date_chr <- "2023-04-20"
repo_end_date_chr <- "2023-07-06"

trade_date <- GaussQuant::date_GQL(trade_date_chr)
repo_end_date_ql <- GaussQuant::date_GQL(repo_end_date_chr)

GaussQuant::set_eval_date_GQL(trade_date)

settle_date <- GaussQuant::advance_days_GQL(
  calendar_obj = cal_us_gov,
  date_obj = trade_date,
  n_days = settlement_days_t1
)

settle_date_chr <- as.character(settle_date)
settle_date_ql <- GaussQuant::date_GQL(settle_date_chr)

setup_tbl <- tibble::tribble(
  ~item, ~value,
  "trade_date", trade_date_chr,
  "spot_settlement_date", settle_date_chr,
  "repo_end_date", repo_end_date_chr,
  "settlement_days", as.character(settlement_days_t1),
  "calendar", "UnitedStates(GovernmentBond)",
  "bond_day_counter", "ActualActual(Bond)",
  "repo_day_counter", "Actual360",
  "compounding", "Compounded",
  "frequency", "Semiannual"
)

GaussQuant::show_tbl_GQH(setup_tbl, "Treasury futures setup")
#> # A tibble: 9 x 2
#>   item                value

```

```

#>   <chr>                <chr>
#> 1 trade_date          2023-04-20
#> 2 spot_settlement_date 2023-04-21
#> 3 repo_end_date       2023-07-06
#> 4 settlement_days      1
#> 5 calendar            UnitedStates(GovernmentBond)
#> 6 bond_day_counter     ActualActual(Bond)
#> 7 repo_day_counter     Actual360
#> 8 compounding          Compounded
#> 9 frequency            Semiannual

```

12.5.2 2.5.2

BPV

4.125% 2

6%

```

bond_bundle <- GaussQuant::us_treasury_bond_GQL(
  effective_date = "2022-09-30",
  maturity_date = "2027-09-30",
  coupon_rate_pct = 4.125,
  face_amount = 100,
  settlement_days = settlement_days_t1,
  calendar = cal_us_gov,
  day_counter = dc_act_act_bond
)

bond_obj <- bond_bundle$bond

GaussQuant::show_tbl_GQH(
  GaussQuant::schedule_dates_GQL(bond_bundle$schedule),
  "US Treasury bond schedule",
  n = 20
)
#> # A tibble: 11 x 1
#>   schedule_date
#>   <date>
#> 1 2022-09-30
#> 2 2023-03-30
#> 3 2023-09-30
#> 4 2024-03-30
#> 5 2024-09-30

```



```

#> 6 2025-03-30
#> 7 2025-09-30
#> 8 2026-03-30
#> 9 2026-09-30
#> 10 2027-03-30
#> 11 2027-09-30

clean_price_at_six_percent <- GaussQuant::bond_clean_price_from_yield_safe_GQL(
  bond = bond_obj,
  yield_rate = 6 / 100,
  day_counter = dc_act_act_bond,
  compounding = compd_compounded,
  frequency = freq_semiannual,
  settlement_date = GaussQuant::date_GQL("2023-07-06")
)

cat(
  "Yield 6%, settle 2023-07-06 clean price:",
  GaussQuant::fmt_num_GQH(clean_price_at_six_percent, 4),
  "\n"
)
#> Yield 6%, settle 2023-07-06 clean price: 93.07

```

12.5.3 2.5.3

CTD

= +

109-10 3.70%

```

bond_yield <- 3.70 / 100
futures_price <- 109 + 10 / 32

accrued_amount <- bond_obj$accruedAmount(settle_date_q1)
clean_price <- GaussQuant::bond_clean_price_from_yield_safe_GQL(
  bond = bond_obj,
  yield_rate = bond_yield,
  day_counter = dc_act_act_bond,
  compounding = compd_compounded,
  frequency = freq_semiannual,

```

```

    settlement_date = settle_date_ql
  )

dirty_price <- clean_price + accrued_amount

price_tbl <- tibble::tibble(
  trade_date = as.Date(trade_date_chr),
  settlement_date = as.Date(settle_date_chr),
  yield = bond_yield,
  accrued_amount = accrued_amount,
  clean_price = clean_price,
  dirty_price = dirty_price,
  futures_price = futures_price
)

GaussQuant::show_tbl_GQH(price_tbl, "Bond price from yield")
#> # A tibble: 1 x 7
#>   trade_date settlement_date yield accrued_amount clean_price dirty_price
#>   <date>      <date>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1 2023-04-20 2023-04-21    0.037      0.24660    101.72    101.97
#>   futures_price
#>      <dbl>
#> 1      109.31

cat(
  "Yield:",
  sprintf("%.8f", bond_yield),
  "\nClean price:",
  sprintf("%.8f", clean_price),
  "\nDirty price:",
  sprintf("%.8f", dirty_price),
  "\nAccrued amount:",
  sprintf("%.8f", accrued_amount),
  "\n"
)
#> Yield: 0.03700000
#> Clean price: 101.72260614
#> Dirty price: 101.96920940
#> Accrued amount: 0.24660326

```

12.5.4 2.5.4

Actual/Actual Bond

QuantLib

```
interest_rate_obj <- QuantLib::InterestRate(
  bond_yield,
  dc_act_act_bond,
  compd_compounded,
  freq_semiannual
)

settle_date_r <- as.Date(settle_date_chr)

bond_cf_tbl <- GaussQuant::bond_cashflow_table_GQL(bond_obj) |>
  dplyr::mutate(
    pay_date = as.Date(.data$date),
    discount_factor = purrr::map_dbl(
      as.character(.data$pay_date),
      function(x) {
        if (as.Date(x) < settle_date_r) {
          return(NA_real_)
        }

        tryCatch(
          interest_rate_obj$discountFactor(
            settle_date_ql,
            GaussQuant::date_GQL(x)
          ),
          error = function(e) NA_real_
        )
      }
    ),
    pv = .data$amount * .data$discount_factor
  ) |>
  dplyr::select(
    .data$pay_date,
    .data$amount,
    .data$discount_factor,
    .data$pv
  )
```

```

cashflow_pv <- sum(bond_cf_tbl$pv, na.rm = TRUE)

cashflow_check_tbl <- tibble::tribble(
  ~metric, ~value,
  "cashflow_present_value", cashflow_pv,
  "dirty_price_from_yield", dirty_price,
  "illustrative_difference", cashflow_pv - dirty_price
)

GaussQuant::show_tbl_GQH(bond_cf_tbl, "Bond cashflow present-value table", n = 20)
#> # A tibble: 11 x 4
#>   pay_date      amount discount_factor      pv
#>   <date>        <dbl>         <dbl>    <dbl>
#> 1 2023-03-30    2.0625          NA        NA
#> 2 2023-10-02    2.0625      0.98484    2.0312
#> 3 2024-04-01    2.0625      0.96695    1.9943
#> 4 2024-09-30    2.0625      0.94939    1.9581
#> 5 2025-03-31    2.0625      0.93214    1.9225
#> 6 2025-09-30    2.0625      0.91521    1.8876
#> 7 2026-03-30    2.0625      0.89859    1.8533
#> 8 2026-09-30    2.0625      0.88227    1.8197
#> 9 2027-03-30    2.0625      0.86624    1.7866
#> 10 2027-09-30   2.0625      0.85051    1.7542
#> 11 2027-09-30  100          0.85051   85.051
GaussQuant::show_tbl_GQH(
  GaussQuant::metric_tbl_display_GQH(cashflow_check_tbl),
  "Cashflow PV illustration"
)
#> # A tibble: 3 x 2
#>   metric          value
#>   <chr>         <chr>
#> 1 cashflow_present_value 102.05828711
#> 2 dirty_price_from_yield 101.96920940
#> 3 illustrative_difference 0.08907771

```

12.5.5 2.5.5

Actual/Actual Bond

$$= 1 \times \text{---}$$

QuantLib

```
previous_coupon_date <- GaussQuant::date_GQL("2023-03-30")
next_coupon_date <- GaussQuant::date_GQL("2023-09-30")

accrued_days_hand <- dc_act_act_bond$dayCount(
  previous_coupon_date,
  settle_date_ql
)

coupon_period_days <- dc_act_act_bond$dayCount(
  previous_coupon_date,
  next_coupon_date
)

coupon_amount_semiannual <- 4.125 / 2

accrued_interest_hand <- coupon_amount_semiannual *
  accrued_days_hand /
  coupon_period_days

accrued_tbl <- tibble::tibble(
  previous_coupon_date = as.Date(GaussQuant::iso_GQL(previous_coupon_date)),
  settlement_date = as.Date(settle_date_chr),
  next_coupon_date = as.Date(GaussQuant::iso_GQL(next_coupon_date)),
  accrued_days = accrued_days_hand,
  coupon_period_days = coupon_period_days,
  coupon_amount = coupon_amount_semiannual,
  accrued_interest_hand = accrued_interest_hand,
  accrued_amount_quantlib = accrued_amount,
  difference = accrued_interest_hand - accrued_amount
)

GaussQuant::show_tbl_GQH(accrued_tbl, "Accrued interest hand calculation")
#> # A tibble: 1 x 9
#>   previous_coupon_date settlement_date next_coupon_date accrued_days
#>   <date>               <date>         <date>           <int>
#> 1 2023-03-30          2023-04-21      2023-09-30           22
#>   coupon_period_days coupon_amount accrued_interest_hand accrued_amount_quantlib
#>   <int>             <dbl>         <dbl>             <dbl>
#> 1          184        2.0625        0.24660          0.24660
#>   difference
#>   <dbl>
```

```
#> 1 5.1348e-15
```

12.6 2.6 CTD BPV

12.6.1 2.6.1

$$= \times +$$

CF Conversion Factor CF

3

```
deliverable_tbl <- tibble::tribble(  
  ~issue_date, ~maturity_date, ~coupon_rate_pct, ~conversion_factor, ~market_yield_pct,  
  "2022-09-30", "2027-09-30", 4.125, 0.9305, 3.70,  
  "2022-08-31", "2027-08-31", 3.125, 0.8953, 3.69,  
  "2023-01-31", "2028-01-31", 3.500, 0.9011, 3.65  
)
```

```
GaussQuant::show_tbl_GQH(deliverable_tbl, "Treasury futures deliverable basket")
```

```
#> # A tibble: 3 x 5
```

```
#>   issue_date maturity_date coupon_rate_pct conversion_factor market_yield_pct  
#>   <chr>      <chr>          <dbl>          <dbl>          <dbl>  
#> 1 2022-09-30 2027-09-30      4.125          0.9305          3.7  
#> 2 2022-08-31 2027-08-31      3.125          0.8953          3.69  
#> 3 2023-01-31 2028-01-31      3.5           0.9011          3.65
```

12.6.2 2.6.2

$$= - \times$$

12.6.3 2.6.3

= -

CTD

12.6.4 2.6.4

BPV

CTD

```
repo_rate <- 5.10 / 100

repo_days <- dc_act_360$dayCount(
  settle_date_ql,
  repo_end_date_ql
)

repo_year_fraction <- dc_act_360$yearFraction(
  settle_date_ql,
  repo_end_date_ql
)

ctd_result <- GaussQuant::bond_futures_ctd_table_GQL(
  deliverable_tbl = deliverable_tbl,
  settlement_date = settle_date_chr,
  futures_price = futures_price,
  repo_end_date = repo_end_date_chr,
  repo_rate = repo_rate,
  settlement_days = settlement_days_t1,
  calendar = cal_us_gov,
  day_counter = dc_act_act_bond,
  compounding = compd_compounded,
  frequency = freq_semiannual,
  repo_day_counter = dc_act_360,
  carry_day_counter = dc_act_360
)
```

```

)

basket_tbl <- ctd_result$basket

gross_basis_tbl <- basket_tbl |>
  dplyr::select(
    .data$issue_date,
    .data$maturity,
    .data$coupon,
    .data$yield_pct,
    .data$bpv,
    .data$clean_price,
    .data$dirty_price,
    .data$conversion_factor,
    .data$gross_basis
  )

net_basis_tbl <- basket_tbl |>
  dplyr::select(
    .data$maturity,
    .data$conversion_factor,
    .data$gross_basis,
    .data$accrued_start,
    .data$accrued_end,
    .data$coupon_income,
    .data$repo_cost,
    .data$carry,
    .data$net_basis,
    .data$forward_price,
    .data$implied_repo,
    .data$ctd
  )

basket_display_tbl <- basket_tbl |>
  dplyr::select(-dplyr::any_of("bond_obj"))

ctd_tbl <- ctd_result$ctd
ctd_display_tbl <- ctd_tbl |>
  dplyr::select(-dplyr::any_of("bond_obj"))

stopifnot(
  nrow(ctd_tbl) == 1L,

```



```

    isTRUE(ctd_tbl$ctd[[1L]])
  )

cat(
  "Futures price:",
  sprintf("%.6f", futures_price),
  "\nRepo rate:",
  sprintf("%.6f", repo_rate),
  "\nRepo days:",
  repo_days,
  "\nRepo year fraction:",
  sprintf("%.8f", repo_year_fraction),
  "\n"
)
#> Futures price: 109.312500
#> Repo rate: 0.051000
#> Repo days: 76
#> Repo year fraction: 0.21111111

GaussQuant::show_tbl_GQH(gross_basis_tbl, "Gross basis table", n = 20)
#> # A tibble: 3 x 9
#>   issue_date maturity    coupon yield_pct    bpv clean_price dirty_price
#>   <chr>      <chr>      <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1 2022-09-30 2027-09-30 0.04125    3.7 0.041018    101.72    101.97
#> 2 2022-08-31 2027-08-31 0.03125    3.69 0.039410     97.740    98.181
#> 3 2023-01-31 2028-01-31 0.035      3.65 0.043340     99.343    100.12
#>   conversion_factor gross_basis
#>             <dbl>    <dbl>
#> 1          0.9305    0.0073249
#> 2          0.8953   -0.12767
#> 3          0.9011    0.84146

GaussQuant::show_tbl_GQH(net_basis_tbl, "Net basis and implied repo table", n = 20)
#> # A tibble: 3 x 12
#>   maturity    conversion_factor gross_basis accrued_start accrued_end
#>   <chr>             <dbl>    <dbl>    <dbl>    <dbl>
#> 1 2027-09-30          0.9305    0.0073249    0.24660    1.0985
#> 2 2027-08-31          0.8953   -0.12767    0.44158    1.0870
#> 3 2028-01-31          0.9011    0.84146    0.77348    1.5083
#>   coupon_income repo_cost    carry net_basis forward_price implied_repo ctd
#>             <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl> <lgl>
#> 1      0.85190    1.0979 -0.24597    0.25329    101.97    0.039234 TRUE
#> 2      0.64538    1.0571 -0.41171    0.28403     98.152    0.037297 FALSE

```

```
#> 3      0.73481      1.0779 -0.34311      1.1846      99.686      -0.0050462 FALSE
GaussQuant::show_tbl_GQH(basket_display_tbl, "Deliverable basket summary", n = 20)
#> # A tibble: 3 x 18
#>   issue_date maturity      coupon yield_pct      bpv clean_price dirty_price
#>   <chr>      <chr>      <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1 2022-09-30 2027-09-30 0.04125      3.7 0.041018      101.72      101.97
#> 2 2022-08-31 2027-08-31 0.03125      3.69 0.039410      97.740      98.181
#> 3 2023-01-31 2028-01-31 0.035      3.65 0.043340      99.343      100.12
#>   conversion_factor gross_basis accrued_start accrued_end coupon_income
#>   <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1      0.9305      0.0073249      0.24660      1.0985      0.85190
#> 2      0.8953     -0.12767      0.44158      1.0870      0.64538
#> 3      0.9011      0.84146      0.77348      1.5083      0.73481
#>   repo_cost      carry net_basis forward_price implied_repo ctd
#>   <dbl>      <dbl>    <dbl>      <dbl>      <dbl> <lg1>
#> 1      1.0979 -0.24597      0.25329      101.97      0.039234 TRUE
#> 2      1.0571 -0.41171      0.28403      98.152      0.037297 FALSE
#> 3      1.0779 -0.34311      1.1846      99.686     -0.0050462 FALSE
GaussQuant::show_tbl_GQH(ctd_display_tbl, "Cheapest-to-deliver bond", n = 20)
#> # A tibble: 1 x 18
#>   issue_date maturity      coupon yield_pct      bpv clean_price dirty_price
#>   <chr>      <chr>      <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1 2022-09-30 2027-09-30 0.04125      3.7 0.041018      101.72      101.97
#>   conversion_factor gross_basis accrued_start accrued_end coupon_income
#>   <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1      0.9305      0.0073249      0.24660      1.0985      0.85190
#>   repo_cost      carry net_basis forward_price implied_repo ctd
#>   <dbl>      <dbl>    <dbl>      <dbl>      <dbl> <lg1>
#> 1      1.0979 -0.24597      0.25329      101.97      0.039234 TRUE
```

ctd TRUE

CTD

12.6.5 2.6.5 CTD BPV

CTD BPV

$$\text{BPV} \approx \frac{\text{CTD} - \text{BPV}}{\text{CTD} - \text{BPV}}$$

D CTD CTD CTD CT-

```

ctd_bond <- ctd_tbl$bond_obj[[1L]]
ctd_conversion_factor <- ctd_tbl$conversion_factor[[1L]]
ctd_yield <- ctd_tbl$yield_pct[[1L]] / 100

ctd_futures_bpv <- GaussQuant::bond_futures_bpv_GQL(
  bond = ctd_bond,
  conversion_factor = ctd_conversion_factor,
  contracts = 1,
  measure = "dv01",
  ytm = ctd_yield,
  day_counter = dc_act_act_bond,
  compounding = compd_compounded,
  frequency = freq_semiannual
)

ctd_summary_tbl <- tibble::tibble(
  metric = c(
    "ctd_maturity",
    "ctd_coupon",
    "ctd_yield_pct",
    "conversion_factor",
    "gross_basis",
    "net_basis",
    "implied_repo",
    "ctd_bond_bpv",
    "ctd_futures_bpv"
  ),
  value = list(
    as.character(ctd_tbl$maturity[[1L]]),
    ctd_tbl$coupon[[1L]],
    ctd_tbl$yield_pct[[1L]],
    ctd_conversion_factor,
    ctd_tbl$gross_basis[[1L]],
    ctd_tbl$net_basis[[1L]],
    ctd_tbl$implied_repo[[1L]],
    ctd_tbl$bpv[[1L]],
    ctd_futures_bpv
  )
)

GaussQuant::show_tbl_GQH(
  GaussQuant::metric_tbl_display_GQH(ctd_summary_tbl),

```

```

"CTD and futures BPV summary"
)
#> # A tibble: 9 x 2
#>   metric      value
#>   <chr>      <chr>
#> 1 ctd_maturity 2027-09-30
#> 2 ctd_coupon   0.04125000
#> 3 ctd_yield_pct 3.70000000
#> 4 conversion_factor 0.93050000
#> 5 gross_basis   0.00732489
#> 6 net_basis     0.25329120
#> 7 implied_repo  0.03923370
#> 8 ctd_bond_bpv  0.04101840
#> 9 ctd_futures_bpv 0.04408210

```

12.7 2.7

Implied Term Structure

CTD

BPV CTD BPV

CTD

13 II-3 OIS

13.1 3.1 IBOR OIS

13.1.1 3.1.1 IBOR

2007 2009 2008 9 LIBOR LI-
BOR
LIBOR LIBOR LIBOR OIS LI-
BOR
BOR OIS LIBOR TIBOR EURI-
BOR
SOFR TONA
LIBOR 2014 IBOR 2017 F-
CA 2021 LIBOR
LIBOR 2021 LIBOR 2023 6 30 LIBOR 2024 9 30 LI-
BOR
LIBOR TONA SOFR OIS LIBOR
ing
IBOR LIBOR OIS
TONA SOFR OIS OIS Fix-
ing

13.1.2 3.1.2 OIS

OIS Overnight Index Swap
 r_i d_i B

$$R_{\text{comp}} = \prod_{i=1}^n \left(1 + r_i \frac{d_i}{B} \right) - 1$$

N

$$I_{\text{overnight}} = NR_{\text{comp}}$$

K α

$$I_{\text{fixed}} = NK\alpha$$

OIS

$$PV_{\text{OIS}} = PV_{\text{overnight}} - PV_{\text{fixed}}$$

OIS

QuantLib

```
curve_tbl_local <- function(curve_obj, n = 200, extrapolate = TRUE) {  
  if (extrapolate) {  
    QuantLib::TermStructure_enableExtrapolation(curve_obj)  
  }  
  
  reference_date <- as.Date(  
    GaussQuant::iso_GQL(curve_obj$referenceDate())  
  )  
  
  curve_times <- seq(  
    from = 0,  
    to = curve_obj$maxTime(),  
    length.out = n  
  )  
  
  tibble::tibble(time = curve_times) |>  
  dplyr::mutate(  
    discount_factor = purrr::map_dbl(  
      time,  
      function(x) curve_obj$discount(x)  
    ),  
    zero_rate = dplyr::if_else(  
      time > 0,  
      -log(discount_factor) / time,  
      0  
    )  
  )  
}
```

```
),  
  date = reference_date + round(time * 365)  
) |>  
dplyr::select(  
  date,  
  time,  
  discount_factor,  
  zero_rate  
)  
}
```

13.2 3.2 TONA OIS

TONA		2022 8 19	TONA	OIS	OIS
	OIS				

13.2.1 3.2.1 TONA

```
trade_date_tona <- "2022-08-19"

GaussQuant::set_eval_date_GQL(trade_date_tona)

tona_quotes <- tibble::tribble(
  ~as_of_date, ~currency, ~instrument, ~kind, ~tenor, ~rate,
  as.Date(trade_date_tona), "JPY", "TONA", "depo", "1D", -0.0000900,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "1W", -0.0001261,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "2W", -0.0001469,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "1M", -0.0001807,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "3M", -0.0001919,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "6M", -0.0001043,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "9M", 0.0000022,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "12M", 0.0001250,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "18M", 0.0003125,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "2Y", 0.0004875,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "3Y", 0.0007375,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "4Y", 0.0009479,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "5Y", 0.0011854,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "7Y", 0.0019146
```

```
)

GaussQuant::show_tbl_GQH(
  tona_quotes,
  "TONA market quotes",
  n = 20
)

#> # A tibble: 14 x 6
#>   as_of_date currency instrument kind tenor      rate
#>   <date>      <chr>    <chr>    <chr> <chr>    <dbl>
#> 1 2022-08-19 JPY      TONA      depo 1D     -0.00009
#> 2 2022-08-19 JPY      TONA      ois 1W     -0.0001261
#> 3 2022-08-19 JPY      TONA      ois 2W     -0.0001469
#> 4 2022-08-19 JPY      TONA      ois 1M     -0.0001807
#> 5 2022-08-19 JPY      TONA      ois 3M     -0.0001919
#> 6 2022-08-19 JPY      TONA      ois 6M     -0.0001043
#> 7 2022-08-19 JPY      TONA      ois 9M      0.0000022
#> 8 2022-08-19 JPY      TONA      ois 12M     0.000125
#> 9 2022-08-19 JPY      TONA      ois 18M     0.0003125
#> 10 2022-08-19 JPY      TONA      ois 2Y      0.0004875
#> 11 2022-08-19 JPY      TONA      ois 3Y      0.0007375
#> 12 2022-08-19 JPY      TONA      ois 4Y      0.0009479
#> 13 2022-08-19 JPY      TONA      ois 5Y      0.0011854
#> 14 2022-08-19 JPY      TONA      ois 7Y      0.0019146
```

13.2.2 3.2.2 TONA OIS

```
tona_curve_envs <- GaussQuant::ir_build_ois_curve_envs_GQH(
  quotes = tona_quotes,
  trade_date = trade_date_tona,
  verbose = TRUE
)

tona_curve_env <- tona_curve_envs[["JPY::TONA"]]
tona_curve <- tona_curve_env$curve

tona_curve_display_tbl <- curve_tbl_local(
  curve_obj = tona_curve,
  n = 200
)
```



```

GaussQuant::show_tbl_GQH(
  tona_curve_display_tbl,
  "TONA OIS curve",
  n = 12
)
#> # A tibble: 12 x 4
#>   date          time discount_factor zero_rate
#>   <date>        <dbl>          <dbl>      <dbl>
#> 1 2022-08-23 0          1          0
#> 2 2022-09-05 0.035203    1.0000 -0.00014504
#> 3 2022-09-18 0.070407    1.0000 -0.00017306
#> 4 2022-10-01 0.10561     1.0000 -0.00018278
#> 5 2022-10-13 0.14081     1.0000 -0.00018668
#> 6 2022-10-26 0.17602     1.0000 -0.00018902
#> 7 2022-11-08 0.21122     1.0000 -0.00019057
#> 8 2022-11-21 0.24642     1.0000 -0.00019169
#> 9 2022-12-04 0.28163     1.0000 -0.00017512
#> 10 2022-12-17 0.31683    1.0000 -0.00015741
#> 11 2022-12-29 0.35203    1.0001 -0.00014325
#> 12 2023-01-11 0.38724    1.0001 -0.00013166

tona_curve_check_tbl <- tibble::tibble(
  reference_date = GaussQuant::iso_GQL(
    tona_curve$referenceDate()
  ),
  discount_2022_08_23 = tona_curve$discount(
    GaussQuant::date_GQL("2022-08-23")
  )
)

GaussQuant::show_tbl_GQH(
  tona_curve_check_tbl,
  "TONA curve checks",
  n = 20
)
#> # A tibble: 1 x 2
#>   reference_date discount_2022_08_23
#>   <chr>          <dbl>
#> 1 2022-08-23    1

```

$P(0, T)$ $T \ 1$

13.2.3 3.2.3 JPY OIS

TONA OIS 2022 8 23 8 30 7 1,000 0.015% OIS
OIS NPV

```
jpy_ois_short <- GaussQuant::trade_ois_swap_py_GQH(  
  curve_env = tona_curve_env,  
  effective = "2022-08-23",  
  maturity = "2022-08-30",  
  notional = 10000000,  
  fixed_rate = -0.00015,  
  pay_receive = "pay",  
  verbose = TRUE  
)  
  
GaussQuant::ir_show_swap_summary_GQH(  
  trade_env = jpy_ois_short,  
  title = "JPY short OIS summary"  
)  
  
jpy_ois_fixed_cf_tbl <- GaussQuant::swap_fixed_leg_table_GQL(  
  jpy_ois_short$swap  
)  
  
jpy_ois_overnight_cf_tbl <- GaussQuant::ois_overnight_leg_table_GQL(  
  jpy_ois_short$swap  
)  
  
GaussQuant::show_tbl_GQH(  
  jpy_ois_fixed_cf_tbl,  
  "TONA fixed leg cashflows",  
  n = 10  
)  
#> # A tibble: 1 x 2  
#>   date      amount  
#>   <chr>      <dbl>  
#> 1 2022-08-30 -28.767  
  
GaussQuant::show_tbl_GQH(  
  jpy_ois_overnight_cf_tbl,  
  "TONA overnight leg cashflows",  
  n = 10  
)
```

```
)
#> # A tibble: 1 x 2
#>   date      amount
#>   <chr>      <dbl>
#> 1 2022-08-30 -24.184
```

13.2.4 3.2.4

OIS $P(0, T_0)$ $P(0, T_1)$ $P(0, T_p)$ α

$$K_{\text{fair}} = \frac{P(0, T_0) - P(0, T_1)}{\alpha P(0, T_p)}$$

```
jpy_effective_df <- tona_curve$discount(
  GaussQuant::date_GQL("2022-08-23")
)

jpy_maturity_df <- tona_curve$discount(
  GaussQuant::date_GQL("2022-08-30")
)

jpy_payment_df <- tona_curve$discount(
  GaussQuant::date_GQL("2022-09-01")
)

jpy_annuity <- (7 / 365) * jpy_payment_df

jpy_swap_rate_hand <- (
  jpy_effective_df - jpy_maturity_df
) / jpy_annuity

jpy_fair_rate_check_tbl <- tibble::tibble(
  effective_discount_factor = jpy_effective_df,
  maturity_discount_factor = jpy_maturity_df,
  payment_discount_factor = jpy_payment_df,
  annuity = jpy_annuity,
  fair_swap_rate_hand = jpy_swap_rate_hand
)

GaussQuant::show_tbl_GQH(
  jpy_fair_rate_check_tbl,
```

```

"TONA fair-rate hand check",
  n = 20
)
#> # A tibble: 1 x 5
#>   effective_discount_factor maturity_discount_factor payment_discount_factor
#>   <dbl> <dbl> <dbl>
#> 1 1 1 1.0000 1.0000
#>   annuity fair_swap_rate_hand
#>   <dbl> <dbl>
#> 1 0.019178 -0.00012610

```

13.3 3.3 Fixing

13.3.1 3.3.1 OIS

OIS

```

jpy_ois_daily <- GaussQuant::trade_ois_swap_py_GQH(
  curve_env = tona_curve_env,
  effective = "2022-08-23",
  maturity = "2022-08-30",
  notional = 10000000,
  fixed_rate = -0.00015,
  pay_receive = "pay",
  fixed_schedule_tenor = "1Y",
  floating_schedule_tenor = "1D",
  pay_lag = 0,
  verbose = TRUE
)

GaussQuant::set_eval_date_GQL(trade_date_tona)

jpy_daily_float_cf_tbl <- GaussQuant::ois_daily_forward_table_GQH(
  swap = jpy_ois_daily$swap,
  curve = tona_curve,
  index = jpy_ois_daily$index,
  eval_date = trade_date_tona,
  notional = 10000000
)

```

```

GaussQuant::show_tbl_GQH(
  jpy_daily_float_cf_tbl,
  "TONA daily forward leg",
  n = 20
)
#> # A tibble: 5 x 9
#>   fixing_date next_date   days fixing_before_eval fixing_on_eval fixing_value
#>   <chr>        <chr>   <int> <lgl>                <lgl>                <dbl>
#> 1 2022-08-23 2022-08-24     1 FALSE                FALSE                -0.000090000
#> 2 2022-08-24 2022-08-25     1 FALSE                FALSE                -0.00013212
#> 3 2022-08-25 2022-08-26     1 FALSE                FALSE                -0.00013212
#> 4 2022-08-26 2022-08-29     3 FALSE                FALSE                -0.00013212
#> 5 2022-08-29 2022-08-30     1 FALSE                FALSE                -0.00013212
#>   forward_rate_from_df applied_rate amount
#>   <dbl>                <dbl>    <dbl>
#> 1 -0.000090000 -0.000090000 -2.4658
#> 2 -0.00013212 -0.00013212 -3.6196
#> 3 -0.00013212 -0.00013212 -3.6196
#> 4 -0.00013212 -0.00013212 -10.859
#> 5 -0.00013212 -0.00013212 -3.6196

```

$$P(0, t_i) \quad P(0, t_{i+1}) \quad \delta_i$$

$$f_i = \frac{P(0, t_i)/P(0, t_{i+1}) - 1}{\delta_i}$$

```

daily_compounding_result <- if (
  nrow(jpy_daily_float_cf_tbl) >= 2 &&
  all(c("days", "rate") %in% names(jpy_daily_float_cf_tbl))
) {
  daily_compounded_return <- (
    1 +
    jpy_daily_float_cf_tbl$days[-1] /
    365 *
    jpy_daily_float_cf_tbl$rate[-1]
  ) |>
  prod() -
  1
  tibble::tibble(

```

```

    compounded_return = daily_compounded_return,
    floating_interest = daily_compounded_return * 10000000,
    equivalent_annual_rate = daily_compounded_return * 365 / 7
  )
} else {
  tibble::tibble(
    compounded_return = NA_real_,
    floating_interest = NA_real_,
    equivalent_annual_rate = NA_real_
  )
}

GaussQuant::show_tbl_GQH(
  daily_compounding_result,
  "TONA daily compounding result",
  n = 20
)

#> # A tibble: 1 x 3
#>   compounded_return floating_interest equivalent_annual_rate
#>   <dbl>             <dbl>             <dbl>
#> 1             NA             NA             NA

```

13.3.2 3.3.2 Fixing

Fixing OIS
 OIS
 • Fixing
 •
 Fixing Lookback Observation Shift Lockout Payment
 Delay
 2022 8 23 24 TONA Fixing 2022 8 25

```

jpy_fixings_tbl <- tibble::tribble(
  ~date, ~currency, ~instrument, ~fixing,
  "2022-08-23", "JPY", "TONA", -0.000090,
  "2022-08-24", "JPY", "TONA", -0.000085
)

GaussQuant::set_eval_date_GQL(trade_date_tona)

```

```

jpy_overnight_before_tbl <- GaussQuant::ois_overnight_leg_table_GQL(
  jpy_ois_short$swap
)

GaussQuant::ir_apply_fixings_GQH(
  index_obj = jpy_ois_short$index,
  fixings_tbl = jpy_fixings_tbl,
  currency = "JPY",
  instrument = "TONA"
)

GaussQuant::set_eval_date_GQL("2022-08-25")

jpy_overnight_after_tbl <- GaussQuant::ois_overnight_leg_table_GQL(
  jpy_ois_short$swap
)

GaussQuant::ir_show_fixing_before_after_GQH(
  before_tbl = jpy_overnight_before_tbl,
  after_tbl = jpy_overnight_after_tbl,
  title = "TONA fixing before / after",
  n = 10
)

#> # A tibble: 1 x 3
#>   row_id amount_before amount_after
#>   <int>         <dbl>         <dbl>
#> 1     1          -24.184          -24.183

GaussQuant::ir_show_swap_summary_GQH(
  trade_env = jpy_ois_short,
  title = "TONA short OIS summary after fixings"
)

jpy_fixing_diag_tbl <- GaussQuant::ir_fixing_table_GQH(
  swap = jpy_ois_daily$swap,
  curve = tona_curve,
  index = jpy_ois_short$index
)

GaussQuant::show_tbl_GQH(
  jpy_fixing_diag_tbl,
  "TONA daily fixing decomposition",

```

```

  n = 20
)
#> # A tibble: 1 x 7
#>       i fixing_date fixing_value pay_date    amount    df      pv
#>   <int> <chr>          <dbl> <chr>      <dbl> <dbl> <dbl>
#> 1     1 2022-08-29   -0.00014154 2022-08-30 -24.183 1.0000 -24.183

```

13.4 3.4 SOFR

SOFR TONA SOFR

OIS TONA

13.4.1 3.4.1 SOFR OIS

```

trade_date_sofr <- "2023-09-26"

GaussQuant::set_eval_date_GQL(trade_date_sofr)

sofr_quotes <- tibble::tribble(
  ~as_of_date, ~currency, ~instrument, ~kind, ~tenor, ~rate,
  as.Date(trade_date_sofr), "USD", "SOFR", "depo", "1D", 0.0531,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "1M", 0.0532,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "3M", 0.0538,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "6M", 0.0546,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "1Y", 0.0545,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "2Y", 0.0501,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "3Y", 0.0467
)

sofr_curve_envs <- GaussQuant::ir_build_ois_curve_envs_GQH(
  quotes = sofr_quotes,
  trade_date = trade_date_sofr,
  verbose = TRUE
)

sofr_curve_env <- sofr_curve_envs[["USD::SOFR"]]
sofr_curve <- sofr_curve_env$curve

```



```
GaussQuant::show_tbl_GQH(
  curve_tbl_local(sofr_curve),
  "SOFR OIS curve",
  n = 12
)
#> # A tibble: 12 x 4
#>   date          time discount_factor zero_rate
#>   <date>        <dbl>          <dbl>      <dbl>
#> 1 2023-09-28 0          1          0
#> 2 2023-10-04 0.015299    0.99919 0.053075
#> 3 2023-10-09 0.030597    0.99838 0.053075
#> 4 2023-10-15 0.045896    0.99757 0.053075
#> 5 2023-10-20 0.061195    0.99676 0.053075
#> 6 2023-10-26 0.076494    0.99595 0.053075
#> 7 2023-11-01 0.091792    0.99514 0.053092
#> 8 2023-11-06 0.10709     0.99432 0.053170
#> 9 2023-11-12 0.12239     0.99351 0.053228
#> 10 2023-11-17 0.13769     0.99269 0.053273
#> 11 2023-11-23 0.15299     0.99188 0.053309
#> 12 2023-11-28 0.16829     0.99106 0.053339
```

13.4.2 3.4.2 SOFR OIS

```
sofr_ois_trade <- GaussQuant::trade_ois_swap_py_GQH(
  curve_env = sofr_curve_env,
  effective = "2023-09-28",
  maturity = "2025-09-28",
  notional = 10000000,
  fixed_rate = 0.05,
  pay_receive = "pay",
  verbose = TRUE
)

GaussQuant::ir_show_swap_summary_GQH(
  trade_env = sofr_ois_trade,
  title = "SOFR OIS summary"
)

sofr_overnight_cf_tbl <- GaussQuant::ois_overnight_leg_table_GQL(
  sofr_ois_trade$swap
```

```
)

GaussQuant::show_tbl_GQH(
  sofr_overnight_cf_tbl,
  "SOFR overnight leg cashflows",
  n = 20
)
#> # A tibble: 2 x 2
#>   date          amount
#>   <chr>         <dbl>
#> 1 2024-09-30 557111.
#> 2 2025-09-29 459522.
```

13.4.3 3.4.3

- Fixing
-

$$F_j \quad \alpha_j \quad D_j \quad N$$

$$PV_{\text{floating}} = N \sum_{j=1}^m \alpha_j F_j D_j$$

$$F_j \quad D_j$$

EURIBOR 6

NPV

```
GaussQuant::set_eval_date_GQL("2023-09-26")

multi_curve_dates <- c(
  "2023-09-26",
  "2024-09-26",
  "2025-09-26",
  "2026-09-28",
  "2028-09-26"
)

forecast_zero_rates <- c(
```

```

0.0450,
0.0470,
0.0460,
0.0445,
0.0420
)

discount_zero_rates <- c(
  0.0400,
  0.0415,
  0.0410,
  0.0400,
  0.0380
)

forecast_curve <- GaussQuant::build_zero_curve_GQL(
  date_chr = multi_curve_dates,
  zero_rates = forecast_zero_rates,
  day_counter = "Actual365Fixed"
)

discount_curve <- GaussQuant::build_zero_curve_GQL(
  date_chr = multi_curve_dates,
  zero_rates = discount_zero_rates,
  day_counter = "Actual365Fixed"
)

QuantLib::TermStructure_enableExtrapolation(forecast_curve)
#> NULL
QuantLib::TermStructure_enableExtrapolation(discount_curve)
#> NULL

forecast_handle <- GaussQuant::yield_curve_handle_GQL(
  forecast_curve
)

discount_handle <- GaussQuant::yield_curve_handle_GQL(
  discount_curve
)

multi_curve_comparison_tbl <- tibble::tibble(
  date = as.Date(multi_curve_dates)

```

```

) |>
dplyr::mutate(
  forecast_discount_factor = purrr::map_dbl(
    as.character(date),
    function(x) {
      forecast_curve$discount(
        GaussQuant::date_GQL(x)
      )
    }
  ),
  ois_discount_factor = purrr::map_dbl(
    as.character(date),
    function(x) {
      discount_curve$discount(
        GaussQuant::date_GQL(x)
      )
    }
  )
)

GaussQuant::show_tbl_GQH(
  multi_curve_comparison_tbl,
  "Forecast and discount curves",
  n = 20
)

#> # A tibble: 5 x 3
#>   date          forecast_discount_factor ois_discount_factor
#>   <date>                <dbl>                <dbl>
#> 1 2023-09-26                1                1
#> 2 2024-09-26            0.95396            0.95924
#> 3 2025-09-26            0.91199            0.92117
#> 4 2026-09-28            0.87471            0.88663
#> 5 2028-09-26            0.81040            0.82679

```

13.4.4 3.4.4

```

single_curve_swap <- GaussQuant::make_vanilla_swap_GQL(
  effective_date = "2023-09-28",
  maturity_date = "2025-09-28",
  fixed_rate = 0.045,

```

```

    notional = 10000000,
    forecast_handle = forecast_handle,
    discount_handle = forecast_handle,
    swap_type = "payer",
    index = "Euribor6M",
    fixed_tenor = GaussQuant::period_years_GQL(1),
    floating_tenor = GaussQuant::period_months_GQL(6),
    calendar = QuantLib::TARGET()
)

multi_curve_swap <- GaussQuant::make_vanilla_swap_GQL(
  effective_date = "2023-09-28",
  maturity_date = "2025-09-28",
  fixed_rate = 0.045,
  notional = 10000000,
  forecast_handle = forecast_handle,
  discount_handle = discount_handle,
  swap_type = "payer",
  index = "Euribor6M",
  fixed_tenor = GaussQuant::period_years_GQL(1),
  floating_tenor = GaussQuant::period_months_GQL(6),
  calendar = QuantLib::TARGET()
)

single_curve_summary <- GaussQuant::swap_summary_GQL(
  single_curve_swap
)

multi_curve_summary <- GaussQuant::swap_summary_GQL(
  multi_curve_swap
)

curve_valuation_comparison_tbl <- dplyr::bind_rows(
  single_curve_summary |>
    dplyr::mutate(valuation_method = "single_curve"),
  multi_curve_summary |>
    dplyr::mutate(valuation_method = "multi_curve")
) |>
  dplyr::relocate(valuation_method)

GaussQuant::show_tbl_GQH(
  curve_valuation_comparison_tbl,

```

```

"Single-curve and multi-curve valuation",
  n = 20
)
#> # A tibble: 2 x 6
#>   valuation_method    npv fixed_leg_npv floating_leg_npv fair_rate fair_spread
#>   <chr>            <dbl>      <dbl>          <dbl>      <dbl>      <dbl>
#> 1 single_curve    40292.    -840562.      880855.    0.047157  -0.0020999
#> 2 multi_curve    39436.    -847103.      886539.    0.047095  -0.0020420

```

PV EURIBOR N-

13.5 3.5 1 EONIA OIS

2012 12 11 EONIA OIS OIS OIS OIS
 EONIA 5 OIS OIS NPV

13.5.1 3.5.1 EONIA

2012 12 11 OIS OIS OIS

```

case_evaluation_date <- "2012-12-11"

GaussQuant::set_eval_date_GQL(
  case_evaluation_date
)

case_eonia_quotes <- GaussQuant::ametrano_bianchetti_eonia_quotes_GQL()

GaussQuant::show_tbl_GQH(
  case_eonia_quotes,
  "EONIA market quotes",
  n = 40
)
#> # A tibble: 30 x 9
#>   quote_id instrument_type rate_pct    rate fixing_days tenor_n tenor_unit
#>   <chr>      <chr>          <dbl>    <dbl>      <int>   <int> <chr>
#> 1 DEP-ON-OD deposit         0.04  0.0004         0       1 Days
#> 2 DEP-ON-1D deposit         0.04  0.0004         1       1 Days

```

```

#> 3 DEP-ON-2D deposit      0.04  0.0004      2      1 Days
#> 4 OIS-1W    spot_ois      0.07  0.0007      2      1 Weeks
#> 5 OIS-2W    spot_ois      0.069 0.00069    2      2 Weeks
#> 6 OIS-3W    spot_ois      0.078 0.00078    2      3 Weeks
#> 7 OIS-1M    spot_ois      0.074 0.00074    2      1 Months
#> 8 OIS-ECB-1 dated_ois      0.046 0.00046    NA     NA <NA>
#> 9 OIS-ECB-2 dated_ois      0.016 0.00016    NA     NA <NA>
#> 10 OIS-ECB-3 dated_ois     -0.007 -0.00007    NA     NA <NA>
#>   start_date end_date
#>   <chr>      <chr>
#> 1 <NA>      <NA>
#> 2 <NA>      <NA>
#> 3 <NA>      <NA>
#> 4 <NA>      <NA>
#> 5 <NA>      <NA>
#> 6 <NA>      <NA>
#> 7 <NA>      <NA>
#> 8 2013-01-16 2013-02-13
#> 9 2013-02-13 2013-03-13
#> 10 2013-03-13 2013-04-10
#> # i 20 more rows

```

```

rate_pct      rate QuantLib
1             Fixing 0 1 2   OIS OIS      OIS

```

13.5.2 3.5.2 EONIA

EONIA

3

```

case_eonia_benchmark <- GaussQuant::eonia_curve_benchmark_GQL(
  quotes = case_eonia_quotes,
  evaluation_date = case_evaluation_date
)

case_eonia_curve <- case_eonia_benchmark$final$curve
case_eonia_handle <- case_eonia_benchmark$final$curve_handle

case_jump_summary <- case_eonia_benchmark$jump_summary

```

```
GaussQuant::show_tbl_GQH(
  case_jump_summary,
  "EONIA year-end jump",
  n = 20
)
#> # A tibble: 1 x 7
#>   original_forward clean_forward      t12 jump_time jump_rate
#>   <dbl>          <dbl>    <dbl>    <dbl>    <dbl>
#> 1      0.00081531    0.00067122 0.038356 0.0054795 0.0010086
#>   jump_discount_factor jump_date
#>   <dbl> <date>
#> 1      0.99999 2012-12-31
```

2012 12 24 2013 1 7 2013 1 3
 2012 12 31 2013 1 2

13.5.3 3.5.3

```
case_benchmark_check_tbl <- tibble::tribble(
  ~metric, ~reference_value_pct, ~r_value_pct,
  "Original two-week forward", 0.082,
  100 * case_jump_summary$original_forward[[1L]],
  "Adjusted two-week forward", 0.067,
  100 * case_jump_summary$clean_forward[[1L]],
  "Year-end jump", 0.101,
  100 * case_jump_summary$jump_rate[[1L]]
) |>
dplyr::mutate(
  difference_pct =
    r_value_pct -
    reference_value_pct
)

GaussQuant::show_tbl_GQH(
  case_benchmark_check_tbl,
  "EONIA benchmark comparison",
  n = 20
)
#> # A tibble: 3 x 4
```



```
#>   metric                reference_value_pct r_value_pct difference_pct
#>   <chr>                  <dbl>          <dbl>          <dbl>
#> 1 Original two-week forward      0.082      0.081531     -0.00046933
#> 2 Adjusted two-week forward      0.067      0.067122      0.00012164
#> 3 Year-end jump                  0.101      0.10086      -0.00013673

stopifnot(
  all(
    abs(case_benchmark_check_tbl$difference_pct) <= 0.0005
  )
)
```

3 0.0005

13.5.4 3.5.4

Rate Helper

```
case_eonia_validation <- case_eonia_benchmark$final_validation

case_eonia_repricing_tbl <- case_eonia_validation$quote_repricing |>
  dplyr::mutate(
    quote_error_bp = 10000 * quote_error
  )

GaussQuant::show_tbl_GQH(
  case_eonia_repricing_tbl,
  "EONIA quote repricing",
  n = 40
)
#> # A tibble: 30 x 10
#>   quote_id instrument_type market_quote implied_quote quote_error pillar_date
#>   <chr>      <chr>          <dbl>          <dbl>          <dbl> <date>
#> 1 DEP-ON-0D deposit            0.0004      0.00040000    2.6205e-15 2012-12-12
#> 2 DEP-ON-1D deposit            0.0004      0.00040000    2.6205e-15 2012-12-13
#> 3 DEP-ON-2D deposit            0.0004      0.00040000    2.6205e-15 2012-12-14
#> 4 OIS-1W    spot_ois            0.0007      0.00070000    4.5565e-13 2012-12-20
#> 5 OIS-2W    spot_ois            0.00069     0.00069000    1.0801e-14 2012-12-27
#> 6 OIS-3W    spot_ois            0.00078     0.00078000   -1.3611e-15 2013-01-03
#> 7 OIS-1M    spot_ois            0.00074     0.00074000    1.8503e-15 2013-01-14
#> 8 OIS-ECB-1 dated_ois          0.00046     0.00046000    6.5201e-12 2013-02-13
```

```

#> 9 OIS-ECB-2 dated_ois 0.00016 0.00016000 8.4594e-12 2013-03-13
#> 10 OIS-ECB-3 dated_ois -0.00007 -0.000070000 9.1291e-12 2013-04-10
#> discount_factor zero_rate one_day_forward_rate quote_error_bp
#> <dbl> <dbl> <dbl> <dbl>
#> 1 1.00000 0.00040556 0.00040000 2.6205e-11
#> 2 1.00000 0.00040556 0.00040000 2.6205e-11
#> 3 1.00000 0.00040556 0.00066137 2.6205e-11
#> 4 0.99998 0.00064213 0.00077743 4.5565e- 9
#> 5 0.99997 0.00066282 0.00065011 1.0801e-10
#> 6 0.99995 0.00075731 0.00068757 -1.3611e-11
#> 7 0.99993 0.00072998 0.00061936 1.8503e-11
#> 8 0.99989 0.00061131 0.00030302 6.5201e- 8
#> 9 0.99988 0.00047463 0.000016127 8.4594e- 8
#> 10 0.99989 0.00034732 -0.00012196 9.1291e- 8
#> # i 20 more rows

stopifnot(
  max(
    abs(case_eonia_repricing_tbl$quote_error),
    na.rm = TRUE
  ) < 1e-8
)

```

OIS OIS OIS

13.5.5 3.5.5 5 OIS

5 OIS

5 OIS

```

case_ois_tenor <- QuantLib::Period(
  5,
  "Years"
)

case_ois_fixed_rate <- case_eonia_quotes |>
  dplyr::filter(
    quote_id == "OIS-5Y"
  ) |>
  dplyr::pull(
    rate
  )

```

```

case_eonia_index <- QuantLib::Eonia(
  case_eonia_handle
)

case_ois_contract_tbl <- tibble::tibble(
  evaluation_date = as.Date(case_evaluation_date),
  index = "EONIA",
  tenor = "5Y",
  settlement_days = 2L,
  contract_fixed_rate = case_ois_fixed_rate
)

GaussQuant::show_tbl_GQH(
  case_ois_contract_tbl,
  "Five-year EONIA OIS contract",
  n = 20
)

#> # A tibble: 1 x 5
#>   evaluation_date index tenor settlement_days contract_fixed_rate
#>   <date>          <chr> <chr>          <int>          <dbl>
#> 1 2012-12-11      EONIA 5Y              2            0.00456

```

13.5.6 3.5.6 OIS

QuantLib MakeOIS OIS

```

case_ois_builder_quantlib <- QuantLib::MakeOIS(
  swapTenor = case_ois_tenor,
  overnightIndex = case_eonia_index,
  fixedRate = case_ois_fixed_rate
)

case_ois_quantlib <- QuantLib::MakeOIS_makeOIS(
  case_ois_builder_quantlib
)

```

EONIA OIS

```

case_ois_gaussquant <- GaussQuant::build_ois_GQL(
  swap_tenor = case_ois_tenor,

```

```

overnight_index = case_eonia_index,
fixed_rate = case_ois_fixed_rate
)

```

OIS NPV

```

case_ois_quantlib_summary <- GaussQuant::swap_summary_GQL(
  case_ois_quantlib
)

case_ois_gaussquant_summary <- GaussQuant::swap_summary_GQL(
  case_ois_gaussquant
)

case_ois_comparison_tbl <- dplyr::bind_rows(
  case_ois_quantlib_summary |>
    dplyr::mutate(
      calculation_route = "QuantLib"
    ),
  case_ois_gaussquant_summary |>
    dplyr::mutate(
      calculation_route = "GaussQuant"
    )
) |>
  dplyr::relocate(
    calculation_route
  )

GaussQuant::show_tbl_GQH(
  case_ois_comparison_tbl,
  "Five-year EONIA OIS valuation",
  n = 20
)

#> # A tibble: 2 x 6
#>   calculation_route      npv fixed_leg_npv floating_leg_npv fair_rate
#>   <chr>              <dbl>      <dbl>          <dbl>      <dbl>
#> 1 QuantLib          -1.6546e-13    -0.022951      0.022951  0.0045600
#> 2 GaussQuant        -1.6546e-13    -0.022951      0.022951  0.0045600
#>   fair_spread
#>   <dbl>
#> 1 3.2876e-14
#> 2 3.2876e-14

```

```

case_ois_metrics <- c(
  "npv",
  "fixed_leg_npv",
  "floating_leg_npv",
  "fair_rate",
  "fair_spread"
)

case_quantlib_values <- unlist(
  case_ois_quantlib_summary |>
    dplyr::select(
      dplyr::all_of(case_ois_metrics)
    ),
  use.names = FALSE
)

case_gaussquant_values <- unlist(
  case_ois_gaussquant_summary |>
    dplyr::select(
      dplyr::all_of(case_ois_metrics)
    ),
  use.names = FALSE
)

stopifnot(
  isTRUE(
    all.equal(
      case_quantlib_values,
      case_gaussquant_values,
      tolerance = 1e-12
    )
  )
)

```

13.5.7 3.5.7

5 OIS

```

case_ois_rate_check_tbl <- tibble::tibble(
  market_fixed_rate = case_ois_fixed_rate,
  calculated_fair_rate =

```

```

    case_ois_gaussquant_summary$fair_rate[[1L]],
    difference_bp = 10000 * (
      case_ois_gaussquant_summary$fair_rate[[1L]] -
      case_ois_fixed_rate
    ),
    npv = case_ois_gaussquant_summary$npv[[1L]]
  )

GaussQuant::show_tbl_GQH(
  case_ois_rate_check_tbl,
  "Five-year EONIA OIS rate check",
  n = 20
)
#> # A tibble: 1 x 4
#>   market_fixed_rate calculated_fair_rate difference_bp      npv
#>   <dbl>             <dbl>          <dbl>      <dbl>
#> 1         0.00456         0.0045600    -3.2876e-10 -1.6546e-13

stopifnot(
  abs(
    case_ois_gaussquant_summary$fair_rate[[1L]] -
    case_ois_fixed_rate
  ) < 1e-8
)

```

5 OIS

NPV

13.5.8 3.5.8

5 OIS

```

case_ois_fixed_leg_tbl <- GaussQuant::leg_to_cashflow_tbl_GQL(
  case_ois_gaussquant$fixedLeg()
)

case_ois_overnight_leg_tbl <- GaussQuant::leg_to_cashflow_tbl_GQL(
  case_ois_gaussquant$overnightLeg()
)

GaussQuant::show_tbl_GQH(
  case_ois_fixed_leg_tbl,

```

```

"Five-year EONIA OIS fixed leg",
  n = 20
)
#> # A tibble: 5 x 2
#>   date          amount
#>   <chr>         <dbl>
#> 1 2013-12-13 0.0046233
#> 2 2014-12-15 0.0046487
#> 3 2015-12-14 0.0046107
#> 4 2016-12-13 0.0046233
#> 5 2017-12-13 0.0046233

GaussQuant::show_tbl_GQH(
  case_ois_overnight_leg_tbl,
  "Five-year EONIA OIS overnight leg",
  n = 20
)
#> # A tibble: 5 x 2
#>   date          amount
#>   <chr>         <dbl>
#> 1 2013-12-13 0.000014593
#> 2 2014-12-15 0.00071766
#> 3 2015-12-14 0.0031409
#> 4 2016-12-13 0.0072966
#> 5 2017-12-13 0.012153

```

13.6 3.6 2

9 2012 12 11 EONIA 5 OIS

OIS 10 EONIA 9 5 OIS

- EONIA 10
- 9
- EONIA 10

9 5 OIS 0.456% NPV

13.6.1 3.6.1

EONIA	10	10	
	OIS	OIS	OIS
			EONIA

```
case_rate_scenario_tbl <- tibble::tribble(
  ~scenario, ~shift_bp,
  "10bp ", -10,
  " ", 0,
  "10bp ", 10
) |>
dplyr::mutate(
  quotes = purrr::map(
    shift_bp,
    function(shift_bp) {
      case_eonia_quotes |>
        dplyr::mutate(
          rate = rate + shift_bp / 10000,
          rate_pct = 100 * rate
        )
    }
  )
)
```

10	0.001	0.1
	1 OIS	5 OIS 30 OIS

```
case_rate_scenario_quote_tbl <- purrr::map2_dfr(
  case_rate_scenario_tbl$scenario,
  case_rate_scenario_tbl$quotes,
  function(scenario, quotes) {
    quotes |>
      dplyr::filter(
        quote_id %in% c(
          "DEP-ON-OD",
          "OIS-1M",
          "OIS-5Y",
          "OIS-30Y"
        )
      ) |>
      dplyr::transmute(
```



```

    scenario = scenario,
    quote_id = quote_id,
    instrument_type = instrument_type,
    rate_pct = rate_pct,
    rate = rate
  )
}
)

GaussQuant::show_tbl_GQH(
  case_rate_scenario_quote_tbl,
  "EONIA market-rate scenarios",
  n = 20
)
#> # A tibble: 12 x 5
#>   scenario quote_id instrument_type rate_pct    rate
#>   <chr>    <chr>    <chr>          <dbl>  <dbl>
#> 1 10bp    DEP-ON-OD deposit        -0.06 -0.0006
#> 2 10bp    OIS-1M    spot_ois        -0.026 -0.00026
#> 3 10bp    OIS-5Y    spot_ois         0.356  0.00356
#> 4 10bp    OIS-30Y   spot_ois         1.938  0.01938
#> 5        DEP-ON-OD deposit         0.04  0.0004
#> 6        OIS-1M    spot_ois         0.074  0.00074
#> 7        OIS-5Y    spot_ois         0.456  0.00456
#> 8        OIS-30Y   spot_ois         2.038  0.02038
#> 9 10bp    DEP-ON-OD deposit         0.14  0.0014
#> 10 10bp   OIS-1M    spot_ois         0.174  0.00174
#> 11 10bp   OIS-5Y    spot_ois         0.556  0.00556
#> 12 10bp   OIS-30Y   spot_ois         2.138  0.02138

```

10 Fixing

13.6.2 3.6.2 EONIA

EONIA

```

case_rate_scenario_tbl <- case_rate_scenario_tbl |>
  dplyr::mutate(
    benchmark = purrr::map(
      quotes,
      function(quotes) {

```

```

    GaussQuant::eonia_curve_benchmark_GQL(
      quotes = quotes,
      evaluation_date = case_evaluation_date
    )
  }
),
curve = purrr::map(
  benchmark,
  ~ .x$final$curve
),
curve_handle = purrr::map(
  benchmark,
  ~ .x$final$curve_handle
),
max_quote_error = purrr::map_dbl(
  benchmark,
  function(benchmark) {
    max(
      abs(
        benchmark$final_validation$
          quote_repricing$
            quote_error
      ),
      na.rm = TRUE
    )
  }
)
)

case_curve_validation_tbl <- case_rate_scenario_tbl |>
  dplyr::select(
    scenario,
    shift_bp,
    max_quote_error
  )

GaussQuant::show_tbl_GQH(
  case_curve_validation_tbl,
  "EONIA scenario-curve validation",
  n = 20
)
#> # A tibble: 3 x 3

```

```

#>   scenario shift_bp max_quote_error
#>   <chr>      <dbl>      <dbl>
#> 1 10bp      -10        9.8407e-12
#> 2           0         9.8573e-12
#> 3 10bp      10         9.8710e-12

stopifnot(
  all(
    case_curve_validation_tbl$max_quote_error < 1e-8
  )
)

```

Rate Helper

13.6.3 3.6.3 OIS

9 5 OIS EONIA

5 0.456%

```

case_rate_scenario_tbl <- case_rate_scenario_tbl |>
  dplyr::mutate(
    eonia_index = purrr::map(
      curve_handle,
      QuantLib::Eonia
    ),
    ois = purrr::map(
      eonia_index,
      function(eonia_index) {
        GaussQuant::build_ois_GQL(
          swap_tenor = case_ois_tenor,
          overnight_index = eonia_index,
          fixed_rate = case_ois_fixed_rate
        )
      }
    ),
    valuation = purrr::map(
      ois,
      GaussQuant::swap_summary_GQL
    )
  )

```

```

case_ois_scenario_result_tbl <- case_rate_scenario_tbl |>
  dplyr::select(
    scenario,
    shift_bp,
    valuation
  ) |>
  tidyr::unnest(
    cols = valuation
  )

case_base_fair_rate <- case_ois_scenario_result_tbl |>
  dplyr::filter(
    scenario == " "
  ) |>
  dplyr::pull(
    fair_rate
  )

case_base_npv <- case_ois_scenario_result_tbl |>
  dplyr::filter(
    scenario == " "
  ) |>
  dplyr::pull(
    npv
  )

case_ois_scenario_result_tbl <- case_ois_scenario_result_tbl |>
  dplyr::mutate(
    contract_fixed_rate = case_ois_fixed_rate,
    fair_rate_change_bp = 10000 * (
      fair_rate -
      case_base_fair_rate
    ),
    payer_valuation_change =
      npv -
      case_base_npv,
    receiver_npv = -npv,
    receiver_valuation_change =
      -(
        npv -
        case_base_npv
      )
  )

```

```

    )
  ) |>
  dplyr::select(
    scenario,
    shift_bp,
    contract_fixed_rate,
    fair_rate,
    fair_rate_change_bp,
    payer_npv = npv,
    payer_valuation_change,
    receiver_npv,
    receiver_valuation_change
  )

GaussQuant::show_tbl_GQH(
  case_ois_scenario_result_tbl,
  "Five-year EONIA OIS scenario valuation",
  n = 20
)

#> # A tibble: 3 x 9
#>   scenario shift_bp contract_fixed_rate fair_rate fair_rate_change_bp
#>   <chr>      <dbl>          <dbl>    <dbl>          <dbl>
#> 1 10bp      -10            0.00456 0.0035600        -10.000
#> 2          0            0.00456 0.0045600           0
#> 3 10bp      10            0.00456 0.0055600         10.0000
#>   payer_npv payer_valuation_change receiver_npv receiver_valuation_change
#>   <dbl>          <dbl>          <dbl>          <dbl>
#> 1 -5.0484e- 3      -0.0050484    5.0484e- 3      0.0050484
#> 2 -1.6546e-13         0          1.6546e-13         0
#> 3  5.0178e- 3       0.0050178   -5.0178e- 3      -0.0050178

```

payer_npv	NPV	receiver_npv	NPV
payer_valuation_change	receiver_valuation_change	NPV	NPV

13.6.4 3.6.4

9	NPV	9
---	-----	---

```

case_base_scenario_tbl <- case_ois_scenario_result_tbl |>
  dplyr::filter(
    scenario == " "
  )

```

```

)

case_base_check_tbl <- tibble::tibble(
  metric = c(
    "NPV",
    "Fair rate"
  ),
  chapter9_value = c(
    case_ois_gaussquant_summary$npv[[1L]],
    case_ois_gaussquant_summary$fair_rate[[1L]]
  ),
  chapter10_value = c(
    case_base_scenario_tbl$payer_npv[[1L]],
    case_base_scenario_tbl$fair_rate[[1L]]
  )
) |>
dplyr::mutate(
  difference =
    chapter10_value -
    chapter9_value
)

GaussQuant::show_tbl_GQH(
  case_base_check_tbl,
  "Base-scenario consistency check",
  n = 20
)

#> # A tibble: 2 x 4
#>   metric      chapter9_value chapter10_value difference
#>   <chr>          <dbl>          <dbl>          <dbl>
#> 1 NPV          -1.6546e-13      -1.6546e-13          0
#> 2 Fair rate     4.5600e- 3       4.5600e- 3          0

stopifnot(
  isTRUE(
    all.equal(
      case_base_check_tbl$chapter9_value,
      case_base_check_tbl$chapter10_value,
      tolerance = 1e-12
    )
  )
)

```

9 10

13.6.5 3.6.5

5 OIS 0.456%
0.456%
NPV

13.6.6 3.6.6 1

10 1 NPV

```
case_npv_down <- case_ois_scenario_result_tbl |>
  dplyr::filter(
    scenario == "10bp "
  ) |>
  dplyr::pull(
    payer_npv
  )

case_npv_up <- case_ois_scenario_result_tbl |>
  dplyr::filter(
    scenario == "10bp "
  ) |>
  dplyr::pull(
    payer_npv
  )

case_ois_bpv_tbl <- tibble::tibble(
  position = "Fixed-rate payer",
  npv_10bp_down = case_npv_down,
  npv_base = case_base_npv,
  npv_10bp_up = case_npv_up,
  average_npv_change_per_bp =
    (
      case_npv_up -
      case_npv_down
    ) / 20
)
```


14 II-4

1973 Black–Scholes–Merton

Black 1976 Black-Scholes-Merton

Black.

76 Black

Black–Scholes–Merton

Black-76

Payoff

Black-76 Black-Scholes-Merton

Black-76

Greeks

Monte Carlo

Payoff

Black–Scholes–Merton

European

Tree

American

Tree

Longstaff–Schwartz

Black

Normal

Black

14.1 4.1 Black–Scholes–Merton Black-76

14.1.1 4.1.1 Black–Scholes–Merton

S_t

$$dS_t = (r - q)S_t dt + \sigma S_t dW_t$$

$$r \qquad q \qquad \sigma \qquad W_t$$

$$T$$

$$F_0 = S_0 e^{(r-q)T}$$

Black–Scholes–Merton

14.1.2 4.1.2 Black-76

Black-76 F_0 European call

$$C = D(0, T) [F_0 N(d_1) - K N(d_2)]$$

European put

$$P = D(0, T) [K N(-d_2) - F_0 N(-d_1)]$$

$$d_1 = \frac{\log(F_0/K) + \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}}$$

$$d_2 = d_1 - \sigma\sqrt{T}$$

K $D(0, T)$ $N(\cdot)$
Black-76

14.1.3 4.1.3 Black-76

107 32 6.75 107.5 5.2% 5%

Black-76

```
black_input_tbl <- tibble::tribble(
  ~input,      ~value,
  "trade_date", "2023-07-21",
  "maturity_date", "2023-08-26",
  "forward_price", as.character(107 + 6.75 / 32),
  "strike_price",  as.character(107.5),
  "volatility",    as.character(0.052),
  "risk_free_rate", as.character(0.05)
)

trade_date <- GaussQuant::date_GQL("2023-07-21")
maturity_date <- GaussQuant::date_GQL("2023-08-26")
forward_price <- 107 + 6.75 / 32
strike_price <- 107.5
```

```

volatility <- 0.052
risk_free_rate <- 0.05
day_counter <- QuantLib::Actual365Fixed()

call_payoff <- QuantLib::PlainVanillaPayoff(
  QuantLib::Option_Call_get(),
  strike_price
)

GaussQuant::set_eval_date_GQL(trade_date)

maturity <- day_counter$yearFraction(
  trade_date,
  maturity_date
)

GaussQuant::show_tbl_GQH(
  black_input_tbl,
  "Black-76 input assumptions",
  n = 20
)
#> # A tibble: 6 x 2
#>   input      value
#>   <chr>      <chr>
#> 1 trade_date 2023-07-21
#> 2 maturity_date 2023-08-26
#> 3 forward_price 107.2109375
#> 4 strike_price 107.5
#> 5 volatility 0.052
#> 6 risk_free_rate 0.05

```

14.2 4.2 Black-76

14.2.1 4.2.1 Black Greeks

BlackCalculator Greeks

QuantLib

$$\sigma\sqrt{T}$$

Greeks

- **Delta**
- **Gamma** Delta
- **Vega**
- **Theta**

Black-76 Delta Delta Spot Delta

```
black_bundle <- GaussQuant::black_process_bundle_GQL(  
  valuation_date = trade_date,  
  forward = forward_price,  
  risk_free_rate = risk_free_rate,  
  volatility = volatility,  
  day_counter = day_counter  
)  
  
discount_factor <- black_bundle$discount_curve$discount(  
  maturity_date  
)  
  
black_calculator_tbl <- GaussQuant::black_calculator_table_GQL(  
  payoff = call_payoff,  
  forward = forward_price,  
  volatility = volatility,  
  maturity = maturity,  
  discount_factor = discount_factor  
)  
  
GaussQuant::show_tbl_GQH(  
  black_calculator_tbl,  
  "Black calculator summary",  
  n = 20  
)  
#> # A tibble: 6 x 2  
#>   metric      value  
#>   <chr>      <dbl>  
#> 1 npv        0.56160  
#> 2 delta       0.43558  
#> 3 gamma       0.22397  
#> 4 vega       13.203
```

```
#> 5 theta -3.4524
#> 6 theta_per_day -0.0094587
```

14.2.2 4.2.2 Black European option

Black Black process European option Pricing Engine
Pricing Engine

```
european_call <- QuantLib::VanillaOption(
  call_payoff,
  QuantLib::EuropeanExercise(maturity_date)
)

analytic_engine <- QuantLib::AnalyticEuropeanEngine(
  black_bundle$process
)

european_call$setPricingEngine(
  analytic_engine
)
#> NULL

analytic_european_tbl <- GaussQuant::option_metric_table_GQL(
  option = european_call,
  process = black_bundle$process
)

analytic_npv <- analytic_european_tbl |>
  dplyr::filter(.data$metric == "npv") |>
  dplyr::pull(.data$value)

GaussQuant::show_tbl_GQH(
  analytic_european_tbl,
  "Analytic European call",
  n = 20
)
#> # A tibble: 7 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 npv      0.56160
#> 2 delta    0.43558
```

```
#> 3 gamma          0.22397
#> 4 vega           13.203
#> 5 theta         -3.4524
#> 6 theta_per_day -0.0094587
#> 7 implied_volatility 0.051993
```

14.3 4.3 Black–Scholes–Merton European option

14.3.1 4.3.1 Black–Scholes–Merton

Black–Scholes–Merton 100 100 1 20% 5% 0%

Black–Scholes–Merton process

```
bsm_input_tbl <- tibble::tribble(
  ~input,      ~value,
  "trade_date", "2024-03-01",
  "maturity",   "1Y",
  "spot_price", as.character(100),
  "strike_price", as.character(100),
  "volatility",  as.character(0.20),
  "risk_free_rate", as.character(0.05),
  "dividend_rate", as.character(0)
)

trade_date_bsm <- GaussQuant::date_GQL("2024-03-01")
calendar_bsm <- QuantLib::TARGET()

maturity_date_bsm <- calendar_bsm$advance(
  trade_date_bsm,
  1L,
  "Years"
)

spot_price_bsm <- 100
strike_price_bsm <- 100
volatility_bsm <- 0.20
risk_free_rate_bsm <- 0.05
dividend_rate_bsm <- 0

GaussQuant::set_eval_date_GQL(
```

```

    trade_date_bsm
  )

bsm_bundle <- GaussQuant::bsm_process_bundle_GQL(
  valuation_date = trade_date_bsm,
  spot = spot_price_bsm,
  risk_free_rate = risk_free_rate_bsm,
  dividend_rate = dividend_rate_bsm,
  volatility = volatility_bsm,
  day_counter = day_counter,
  calendar = calendar_bsm
)

GaussQuant::show_tbl_GQH(
  bsm_input_tbl,
  "Black-Scholes-Merton assumptions",
  n = 20
)
#> # A tibble: 7 x 2
#>   input      value
#>   <chr>      <chr>
#> 1 trade_date 2024-03-01
#> 2 maturity   1Y
#> 3 spot_price 100
#> 4 strike_price 100
#> 5 volatility  0.2
#> 6 risk_free_rate 0.05
#> 7 dividend_rate 0

```

14.3.2 4.3.2 European put Tree

European Black-Scholes-Merton

Tree

25 100 500 Cox-Ross-Rubinstein Tree

```

european_put <- GaussQuant::make_european_option_GQL(
  spot = spot_price_bsm,
  strike = strike_price_bsm,
  maturity_date = maturity_date_bsm,
  option_type = "put",

```

```

valuation_date = trade_date_bsm,
risk_free_rate = risk_free_rate_bsm,
dividend_yield = dividend_rate_bsm,
volatility = volatility_bsm,
day_counter = day_counter,
calendar = calendar_bsm
)

european_engines <- list(
  crr_25 = QuantLib::BinomialCRRVanillaEngine(
    bsm_bundle$process,
    25L
  ),
  analytic = QuantLib::AnalyticEuropeanEngine(
    bsm_bundle$process
  ),
  crr_100 = QuantLib::BinomialCRRVanillaEngine(
    bsm_bundle$process,
    100L
  ),
  crr_500 = QuantLib::BinomialCRRVanillaEngine(
    bsm_bundle$process,
    500L
  )
)

european_comparison_tbl <- purrr::imap_dfr(
  european_engines,
  function(engine, method) {
    european_put$setPricingEngine(engine)

    tibble::tibble(
      method = method,
      npv = european_put$NPV()
    )
  }
)

analytic_put_npv <- european_comparison_tbl |>
  dplyr::filter(.data$method == "analytic") |>
  dplyr::pull(.data$npv)

```



```

european_comparison_tbl <- european_comparison_tbl |>
  dplyr::mutate(
    difference_from_analytic = .data$npv - analytic_put_npv
  )

GaussQuant::show_tbl_GQH(
  european_comparison_tbl,
  "European put engine comparison",
  n = 20
)

#> # A tibble: 4 x 3
#>   method      npv difference_from_analytic
#>   <chr>      <dbl>                <dbl>
#> 1 crr_25    5.6546                0.071974
#> 2 analytic 5.5826                0
#> 3 crr_100   5.5629               -0.019666
#> 4 crr_500   5.5786               -0.0039372

```

14.4 4.4 Monte Carlo

14.4.1 4.4.1 Monte Carlo

Payoff

$$V_0 = D(0, T)E^Q [\max(F_T - K, 0)]$$

Monte Carlo

Payoff

European call

Monte Carlo

Black

```

mc_setting_tbl <- tibble::tribble(
  ~setting, ~value,
  "steps", 12,
  "paths", 20000,
  "seed", 1
)

mc_path_tbl <- GaussQuant::generate_path_table_GQL(
  process = black_bundle$process,
  maturity = maturity,

```

```

n_steps = 12L,
n_paths = 20000L,
seed = 1L,
sequence = "pseudo"
)

mc_result <- GaussQuant::terminal_option_mc_GQL(
  path_tbl = mc_path_tbl,
  strike = strike_price,
  discount_factor = discount_factor,
  option_type = "call"
)

mc_comparison_tbl <- tibble::tribble(
  ~method, ~npv, ~standard_error,
  "Black analytic", analytic_npv, NA_real_,
  "Pseudo-random Monte Carlo", mc_result$summary$npv[[1]], mc_result$summary$standard_error[
) |>
  dplyr::mutate(
    difference_from_analytic = .data$npv - analytic_npv
  )

GaussQuant::show_tbl_GQH(
  mc_setting_tbl,
  "Monte Carlo settings",
  n = 20
)

#> # A tibble: 3 x 2
#>   setting value
#>   <chr>    <dbl>
#> 1 steps      12
#> 2 paths    20000
#> 3 seed         1

GaussQuant::show_tbl_GQH(
  mc_comparison_tbl,
  "Analytic and Monte Carlo comparison",
  n = 20
)

#> # A tibble: 2 x 4
#>   method npv standard_error difference_from_analytic
#>   <chr>   <dbl>         <dbl>             <dbl>

```

```
#> 1 Black analytic          0.56160      NA          0
#> 2 Pseudo-random Monte Carlo 0.55996    0.0065413 -0.0016320
```

Monte Carlo

14.4.2 4.4.2 Monte Carlo

Black process

120

```
path_plot_tbl <- GaussQuant::generate_path_table_GQL(
  process = black_bundle$process,
  maturity = maturity,
  n_steps = 12L,
  n_paths = 120L,
  seed = 1L,
  sequence = "pseudo"
)

path_plot <- ggplot2::ggplot(
  path_plot_tbl,
  ggplot2::aes(
    x = .data$time,
    y = .data$price,
    group = .data$path_id
  )
) +
  ggplot2::geom_line(
    alpha = 0.45,
    linewidth = 0.4
  ) +
  ggplot2::labs(
    title = "Black model Monte Carlo paths",
    x = "Year",
    y = "Forward price"
  ) +
  ggplot2::theme_minimal()

terminal_plot_tbl <- path_plot_tbl |>
  dplyr::group_by(.data$path_id) |>
  dplyr::slice_max(
```

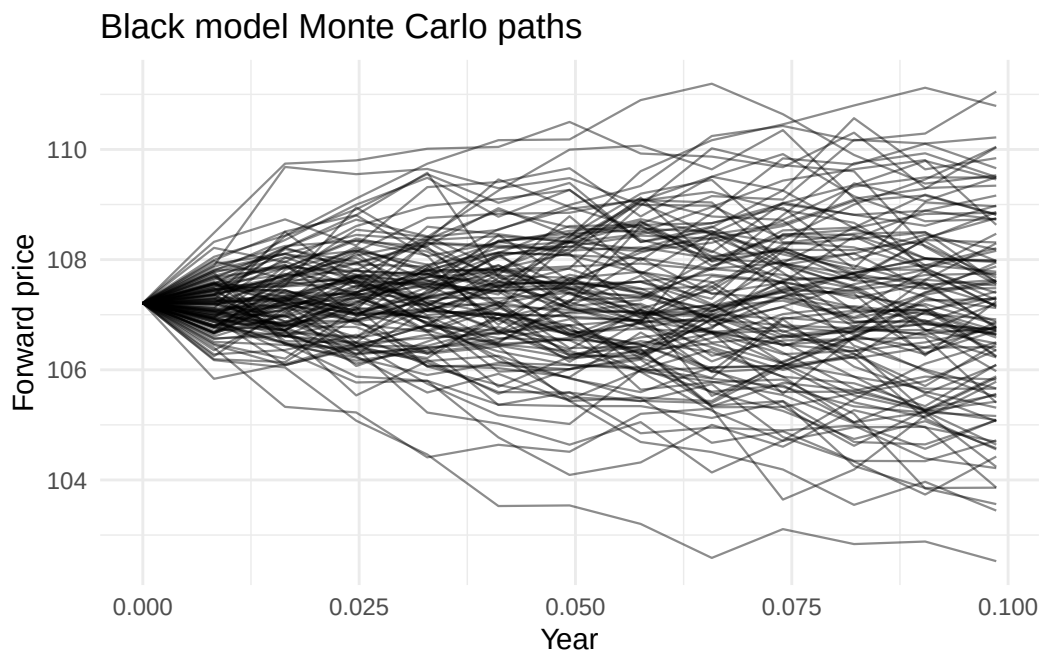
```

    order_by = .data$step,
    n = 1L,
    with_ties = FALSE
  ) |>
  dplyr::ungroup()

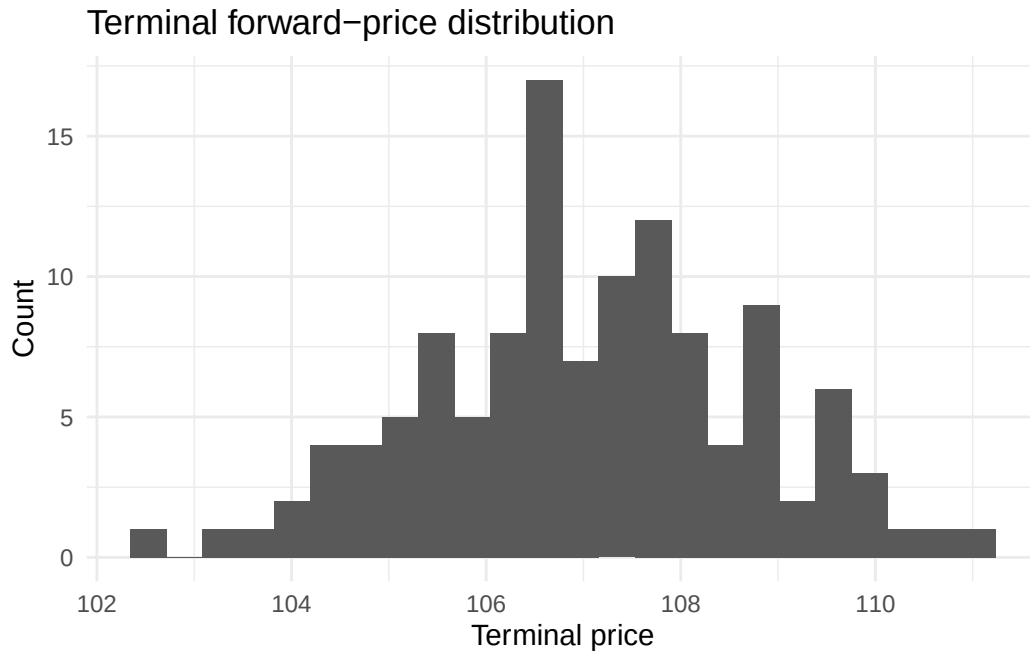
terminal_plot <- ggplot2::ggplot(
  terminal_plot_tbl,
  ggplot2::aes(x = .data$price)
) +
  ggplot2::geom_histogram(
    bins = 24
  ) +
  ggplot2::labs(
    title = "Terminal forward-price distribution",
    x = "Terminal price",
    y = "Count"
  ) +
  ggplot2::theme_minimal()

print(path_plot)

```



```
print(terminal_plot)
```



14.4.3 4.4.3 Black 1

Black process Δt

$$F_{t+\Delta t} = F_t \exp \left[-\frac{1}{2} \sigma^2 \Delta t + \sigma \sqrt{\Delta t} Z \right]$$

Z Ito $-\frac{1}{2} \sigma^2 \Delta t$

QuantLib PathGenerator Gaussian increment

```
n_steps_check <- 3L
seed_check <- 1L

generated_path_tbl <- GaussQuant::generate_path_table_GQL(
  process = black_bundle$process,
  maturity = maturity,
  n_steps = n_steps_check,
  n_paths = 1L,
  seed = seed_check,
```

```

sequence = "pseudo"
)

gaussian_rng_check <- QuantLib::GaussianRandomSequenceGenerator(
  QuantLib::UniformRandomSequenceGenerator(
    n_steps_check,
    QuantLib::UniformRandomGenerator(seed_check)
  )
)

gaussian_increment <- gaussian_rng_check$nextSequence()$value() |>
  as.numeric()

time_grid_check <- generated_path_tbl |>
  dplyr::arrange(.data$step) |>
  dplyr::pull(.data$time)

dt_check <- diff(time_grid_check)

growth_factor <- exp(
  -0.5 * volatility^2 * dt_check +
  volatility * sqrt(dt_check) * gaussian_increment
)

hand_price <- forward_price * c(
  1,
  cumprod(growth_factor)
)

one_path_check_tbl <- generated_path_tbl |>
  dplyr::arrange(.data$step) |>
  dplyr::transmute(
    step = .data$step,
    time = .data$time,
    generated_price = .data$price,
    hand_price = hand_price,
    difference = .data$price - hand_price
  )

GaussQuant::show_tbl_GQH(
  one_path_check_tbl,
  "One-path hand calculation",

```

```

n = 20
)
#> # A tibble: 4 x 5
#>   step      time generated_price hand_price difference
#>   <int>    <dbl>          <dbl>      <dbl>      <dbl>
#> 1     0  0          107.21      107.21      0
#> 2     1 0.032877          106.99      106.99      0
#> 3     2 0.065753          109.82      109.82      0
#> 4     3 0.098630          110.42      110.42 -1.4211e-14

```

PathGenerator Black

14.5 4.5 American put

American

European

$$V_{\text{American}} \geq V_{\text{European}}$$

American put

Tree

- 25 CRR Tree
- Barone–Adesi–Whaley
- Bjerksund–Stensland
- 500 CRR Tree

500 CRR Tree

```

american_put <- GaussQuant::make_american_option_GQL(
  spot = spot_price_bsm,
  strike = strike_price_bsm,
  maturity_date = maturity_date_bsm,
  option_type = "put",
  valuation_date = trade_date_bsm,
  risk_free_rate = risk_free_rate_bsm,
  dividend_yield = dividend_rate_bsm,
  volatility = volatility_bsm,
  day_counter = day_counter,
  calendar = calendar_bsm,
  steps = 500L
)

```

```

)

american_engines <- list(
  crr_25 = QuantLib::BinomialCRRVanillaEngine(
    bsm_bundle$process,
    25L
  ),
  barone_adesi_whaley = QuantLib::BaroneAdesiWhaleyApproximationEngine(
    bsm_bundle$process
  ),
  bjerksund_stensland = QuantLib::BjerksundStenslandApproximationEngine(
    bsm_bundle$process
  ),
  crr_500 = QuantLib::BinomialCRRVanillaEngine(
    bsm_bundle$process,
    500L
  )
)

american_comparison_tbl <- purrr::imap_dfr(
  american_engines,
  function(engine, method) {
    american_put$setPricingEngine(engine)

    tibble::tibble(
      method = method,
      npv = american_put$NPV()
    )
  }
)

crr_500_american_npv <- american_comparison_tbl |>
  dplyr::filter(.data$method == "crr_500") |>
  dplyr::pull(.data$npv)

american_comparison_tbl <- american_comparison_tbl |>
  dplyr::mutate(
    difference_from_crr_500 = .data$npv - crr_500_american_npv
  )

GaussQuant::show_tbl_GQH(
  american_comparison_tbl,

```



```

"American put engine comparison",
  n = 20
)
#> # A tibble: 4 x 3
#>   method                npv difference_from_crr_500
#>   <chr>                <dbl>                <dbl>
#> 1 crr_25                6.1595                0.058390
#> 2 barone_adesi_whaley 6.1100                0.0089221
#> 3 bjerksund_stensland 5.9951               -0.10604
#> 4 crr_500              6.1011                0

```

14.6 4.6 Monte Carlo Longstaff–Schwartz

14.6.1 4.6.1 Sobol

Sobol

Sobol Monte Carlo

Black–Scholes–Merton process 64 Sobol

```

maturity_bsm <- day_counter$yearFraction(
  trade_date_bsm,
  maturity_date_bsm
)

sobol_display_tbl <- GaussQuant::generate_path_table_GQL(
  process = bsm_bundle$process,
  maturity = maturity_bsm,
  n_steps = 12L,
  n_paths = 64L,
  seed = 1L,
  sequence = "sobol"
)

sobol_plot <- ggplot2::ggplot(
  sobol_display_tbl,
  ggplot2::aes(
    x = .data$time,
    y = .data$price,
    group = .data$path_id
  )
)

```

```

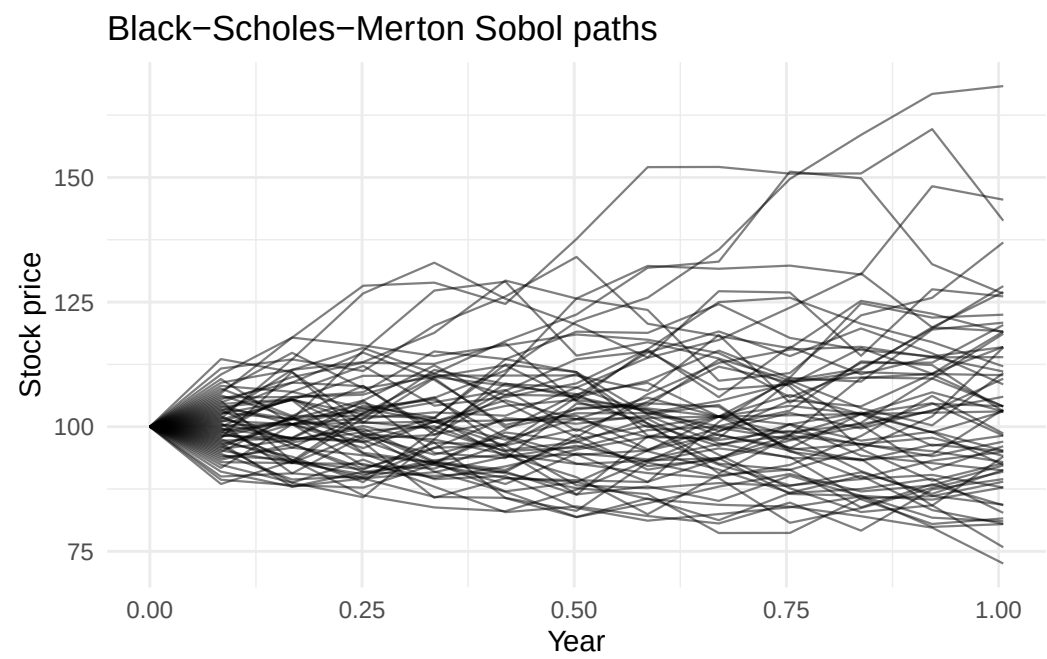
)
) +
  ggplot2::geom_line(
    alpha = 0.5,
    linewidth = 0.4
  ) +
  ggplot2::labs(
    title = "Black-Scholes-Merton Sobol paths",
    x = "Year",
    y = "Stock price"
  ) +
  ggplot2::theme_minimal()

GaussQuant::show_tbl_GQH(
  sobol_display_tbl,
  "Sobol path table",
  n = 20
)

#> # A tibble: 20 x 4
#>   path_id  step    time  price
#>   <int> <int>   <dbl> <dbl>
#> 1     1     0 0      100
#> 2     1     1 0.083790 100.25
#> 3     1     2 0.16758 100.50
#> 4     1     3 0.25137 100.76
#> 5     1     4 0.33516 101.01
#> 6     1     5 0.41895 101.26
#> 7     1     6 0.50274 101.52
#> 8     1     7 0.58653 101.78
#> 9     1     8 0.67032 102.03
#> 10    1     9 0.75411 102.29
#> 11    1    10 0.83790 102.55
#> 12    1    11 0.92169 102.80
#> 13    1    12 1.0055 103.06
#> 14     2     0 0      100
#> 15     2     1 0.083790 104.24
#> 16     2     2 0.16758 100.50
#> 17     2     3 0.25137 104.77
#> 18     2     4 0.33516 101.01
#> 19     2     5 0.41895 105.30
#> 20     2     6 0.50274 101.52

```

```
print(sobol_plot)
```



14.6.2 4.6.2 Longstaff–Schwartz

American
put

$$I(S_t) = \max(K - S_t, 0)$$

Longstaff–Schwartz Cashflow 2

$$\widehat{C}(S_t) = \beta_0 + \beta_1 S_t + \beta_2 S_t^2$$

2,048 Sobol American put 500 CRR Tree

```

lsm_path_tbl <- GaussQuant::generate_path_table_GQL(
  process = bsm_bundle$process,
  maturity = maturity_bsm,
  n_steps = 12L,
  n_paths = 2048L,
  seed = 1L,
  sequence = "sobol"
)

lsm_result <- GaussQuant::lsm_american_put_GQL(
  path_tbl = lsm_path_tbl,
  strike = strike_price_bsm,
  risk_free_rate = risk_free_rate_bsm
)

lsm_npv <- lsm_result$summary |>
  dplyr::filter(.data$metric == "lsm_npv") |>
  dplyr::pull(.data$value)

lsm_comparison_tbl <- tibble::tribble(
  ~method, ~npv,
  "Longstaff-Schwartz Sobol", lsm_npv,
  "CRR Tree 500 steps", crr_500_american_npv
) |>
  dplyr::mutate(
    difference_from_crr_500 = .data$npv - crr_500_american_npv
  )

exercise_summary_tbl <- lsm_result$exercise |>
  dplyr::count(
    .data$exercise_step,
    .data$exercise_time,
    name = "paths"
  ) |>
  dplyr::arrange(.data$exercise_step)

GaussQuant::show_tbl_GQH(
  lsm_result$summary,
  "Longstaff-Schwartz summary",
  n = 20
)
#> # A tibble: 4 x 2

```

```

#>   metric      value
#>   <chr>      <dbl>
#> 1 lsm_npv    6.2925
#> 2 standard_error 0.16653
#> 3 paths      2048
#> 4 steps      12

GaussQuant::show_tbl_GQH(
  lsm_comparison_tbl,
  "Longstaff-Schwartz and CRR comparison",
  n = 20
)
#> # A tibble: 2 x 3
#>   method      npv difference_from_crr_500
#>   <chr>      <dbl>          <dbl>
#> 1 Longstaff-Schwartz Sobol 6.2925          0.19139
#> 2 CRR Tree 500 steps 6.1011          0

GaussQuant::show_tbl_GQH(
  exercise_summary_tbl,
  "Longstaff-Schwartz exercise timing",
  n = 20
)
#> # A tibble: 11 x 3
#>   exercise_step exercise_time paths
#>   <int>      <dbl> <int>
#> 1         2      0.16758    28
#> 2         3      0.25137    52
#> 3         4      0.33516    57
#> 4         5      0.41895   109
#> 5         6      0.50274   107
#> 6         7      0.58653   100
#> 7         8      0.67032    87
#> 8         9      0.75411    38
#> 9        10      0.83790    78
#> 10       11      0.92169    96
#> 11       12      1.0055   1296

GaussQuant::show_tbl_GQH(
  lsm_result$diagnostics,
  "Longstaff-Schwartz diagnostics",
  n = 20
)

```


Tree Monte Carlo	Payoff
Black	Normal Shifted Black

```
chapter04_summary_tbl <- tibble::tribble(
  ~section,          ~main_result,
  "Black-76",        "Black calculator and analytic European pricing",
  "Pseudo-random Monte Carlo", "Discounted terminal payoff and standard error",
  "European put",    "Analytic and CRR Tree comparison",
  "American put",    "Approximation engines and CRR Tree comparison",
  "Sobol",           "Low-discrepancy path generation",
  "Longstaff-Schwartz", "Regression-based early-exercise approximation"
)

GaussQuant::show_tbl_GQH(
  chapter04_summary_tbl,
  "Chapter II-4 summary",
  n = 20
)

#> # A tibble: 6 x 2
#>   section          main_result
#>   <chr>          <chr>
#> 1 Black-76      Black calculator and analytic European pricing
#> 2 Pseudo-random Monte Carlo Discounted terminal payoff and standard error
#> 3 European put  Analytic and CRR Tree comparison
#> 4 American put  Approximation engines and CRR Tree comparison
#> 5 Sobol         Low-discrepancy path generation
#> 6 Longstaff-Schwartz Regression-based early-exercise approximation
```

15

II-5

Norma

Black

Black

Black

$$dF_t = \sigma_B F_t \, dW_t$$

Normal

$$dF_t = \sigma_N \, dW_t$$

$$F_T = F_0 + \sigma_N \sqrt{T} Z$$

Z

Caplet Cap Swaption Nor-

mal

15.1

5.1 Normal

15.1.1

5.1.1

Caplet

Caplet

Numeraire

$[T_1, T_2]$

$L_t(T_1, T_2)$

T_2 -forward measure

$$dL_t(T_1, T_2) = \sigma_N \, dW_t^{T_2}$$

Swaption	Numeraire swap measure
HJM LMM	Normal

15.1.2 5.1.2 Bachelier

Normal

$$F_T \sim N(F_0, \sigma_N^2 T)$$

European call

$$C_N = D(0, T) \left[(F_0 - K)N(d) + \sigma_N \sqrt{T} \phi(d) \right]$$

European put

$$P_N = D(0, T) \left[(K - F_0)N(-d) + \sigma_N \sqrt{T} \phi(d) \right]$$

$$d = \frac{F_0 - K}{\sigma_N \sqrt{T}}$$

$D(0, T)$	$N(\cdot)$	$\phi(\cdot)$
Black	$\log(F_0/K)$	Normal $F_0 - K$

15.1.3 5.1.3 Normal volatility

Black volatility

Normal volatility	Normal volatility 50bp
-------------------	------------------------

$$50 \text{ bp} = \frac{50}{10000} = 0.0050$$

vol = 0.0050	0.5% Black volatility	50bp Normal volatility
Normal		

$$F_T - F_0 = \sigma_N \sqrt{T} Z$$

$$\sigma_N \sqrt{T}$$

```
normal_volatility_bp <- 50
normal_volatility <- normal_volatility_bp / 10000

normal_volatility_tbl <- tibble::tribble(
  ~metric, ~value,
  "normal_volatility_bp", normal_volatility_bp,
  "normal_volatility_decimal", normal_volatility,
  "one_standard_deviation_1y_bp", normal_volatility * 10000
)

GaussQuant::show_tbl_GQH(
  normal_volatility_tbl,
  "Normal volatility units",
  n = 20
)
#> # A tibble: 3 x 2
#>   metric value
#>   <chr>   <dbl>
#> 1 normal_volatility_bp 50
#> 2 normal_volatility_decimal 0.005
#> 3 one_standard_deviation_1y_bp 50
```

15.2 5.2 SOFR

15.2.1 5.2.1

$$\text{Futures Price} = 100 - 100 \times \text{Rate}$$

94.54 5.46%

$$F_0 = \frac{100 - 94.54}{100} = 0.0546$$

94.50 5.50%

$$K = \frac{100 - 94.50}{100} = 0.0550$$

call put Normal option_sign = -1

```
trade_date <- as.Date("2023-09-26")
maturity_date <- as.Date("2023-12-15")

futures_price <- 94.54
strike_futures_price <- 94.50

forward_rate <- (100 - futures_price) / 100
strike_rate <- (100 - strike_futures_price) / 100

normal_volatility <- 50 / 10000
risk_free_rate <- 5.4 / 100

maturity_year_fraction <- as.numeric(
  maturity_date - trade_date
) / 365

maturity_discount_factor <- exp(
  -risk_free_rate * maturity_year_fraction
)

option_sign <- -1

futures_option_input_tbl <- tibble::tribble(
  ~input,          ~value,
  "trade_date",    as.character(trade_date),
  "maturity_date", as.character(maturity_date),
  "futures_price", as.character(futures_price),
  "strike_futures_price", as.character(strike_futures_price),
  "forward_rate",  as.character(forward_rate),
  "strike_rate",   as.character(strike_rate),
  "normal_volatility", as.character(normal_volatility),
  "maturity_year_fraction", as.character(maturity_year_fraction),
  "discount_factor", as.character(maturity_discount_factor)
)

GaussQuant::show_tbl_GQH(
  futures_option_input_tbl,
```

```

    "Normal futures-option assumptions",
    n = 20
  )
#> # A tibble: 9 x 2
#>   input      value
#>   <chr>      <chr>
#> 1 trade_date 2023-09-26
#> 2 maturity_date 2023-12-15
#> 3 futures_price 94.54
#> 4 strike_futures_price 94.5
#> 5 forward_rate 0.05459999999999999
#> 6 strike_rate 0.055
#> 7 normal_volatility 0.005
#> 8 maturity_year_fraction 0.219178082191781
#> 9 discount_factor 0.988234148959797

```

15.2.2 5.2.2 Greeks

Normal Greeks

```

normal_greeks_tbl <- GaussQuant::normal_option_greeks_GQL(
  option_sign = option_sign,
  forward = forward_rate,
  strike = strike_rate,
  vol = normal_volatility,
  maturity = maturity_year_fraction,
  discount_factor = maturity_discount_factor,
  risk_free_rate = risk_free_rate
)

normal_npv <- normal_greeks_tbl |>
  dplyr::filter(.data$metric == "npv") |>
  dplyr::pull(.data$value)

contract_value_multiplier <- 100 * 2500

normal_summary_tbl <- dplyr::bind_rows(
  tibble::tibble(
    metric = c(
      "option_npv_rate_unit",
      "contract_value_multiplier",

```

```

      "delivery_amount"
    ),
    value = c(
      normal_npv,
      contract_value_multiplier,
      normal_npv * contract_value_multiplier
    )
  ),
  normal_greeks_tbl |>
    dplyr::filter(.data$metric != "npv")
)

GaussQuant::show_tbl_GQH(
  normal_summary_tbl,
  "Normal futures-option summary",
  n = 20
)

#> # A tibble: 7 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 option_npv_rate_unit                1.1340e-3
#> 2 contract_value_multiplier          2.5    e+5
#> 3 delivery_amount                    2.8349e+2
#> 4 delta                             -5.6116e-1
#> 5 gamma                             1.6598e+2
#> 6 vega                              1.8190e-1
#> 7 theta                             -2.0135e-3

```

option_npv_rate_unit

100 * 2500

15.3 5.3

15.3.1 5.3.1

Normal

0.20% 0% Normal volatility 60bp

```

negative_rate_example_tbl <- tibble::tribble(
  ~option_type, ~option_sign, ~forward, ~strike,
  "call",      1,             -0.0020,  0.0000,

```

```

"put",          -1,          -0.0020,   0.0000
) |>
dplyr::mutate(
  normal_volatility = 60 / 10000,
  maturity = 1,
  discount_factor = exp(-0.01),
  npv = purrr::pmap_dbl(
    list(
      .data$option_sign,
      .data$forward,
      .data$strike,
      .data$normal_volatility,
      .data$maturity,
      .data$discount_factor
    ),
    function(
      option_sign,
      forward,
      strike,
      normal_volatility,
      maturity,
      discount_factor
    ) {
      GaussQuant::normal_option_price_GQL(
        option_sign = option_sign,
        forward = forward,
        strike = strike,
        vol = normal_volatility,
        maturity = maturity,
        discount_factor = discount_factor
      )
    }
  )
)

GaussQuant::show_tbl_GQH(
  negative_rate_example_tbl,
  "Normal pricing with a negative forward rate",
  n = 20
)
#> # A tibble: 2 x 8
#>   option_type option_sign forward strike normal_volatility maturity

```

```

#>   <chr>           <dbl>   <dbl>   <dbl>           <dbl>   <dbl>
#> 1 call           1 -0.002     0           0.006     1
#> 2 put           -1 -0.002     0           0.006     1
#>   discount_factor   npv
#>           <dbl>   <dbl>
#> 1       0.99005 0.0015102
#> 2       0.99005 0.0034903

```

Normal

15.3.2 5.3.2 Normal

Normal Normal

V_{mkt} σ_N

$$V_N(F_0, K, T, D, \sigma_N) = V_{\text{mkt}}$$

GaussQuant Normal

```

target_npv <- 0.1133 / 100

implied_normal_volatility <- GaussQuant::normal_vol_from_price_GQL(
  option_sign = option_sign,
  strike = strike_rate,
  forward = forward_rate,
  maturity = maturity_year_fraction,
  option_npv = target_npv,
  discount_factor = maturity_discount_factor
)

repriced_target <- GaussQuant::normal_option_price_GQL(
  option_sign = option_sign,
  forward = forward_rate,
  strike = strike_rate,
  vol = implied_normal_volatility,
  maturity = maturity_year_fraction,
  discount_factor = maturity_discount_factor
)

implied_volatility_tbl <- tibble::tribble(

```

```

~metric,                                ~value,
"target_npv",                           target_npv,
"implied_normal_volatility",            implied_normal_volatility,
"implied_normal_volatility_bp",         implied_normal_volatility * 10000,
"repriced_npv",                        repriced_target,
"repricing_difference",                repriced_target - target_npv
)

GaussQuant::show_tbl_GQH(
  implied_volatility_tbl,
  "Implied normal volatility",
  n = 20
)

#> # A tibble: 5 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 target_npv                          0.001133
#> 2 implied_normal_volatility           0.0049948
#> 3 implied_normal_volatility_bp 49.948
#> 4 repriced_npv                       0.001133
#> 5 repricing_difference                0

```

15.3.3 5.3.3

Normal Normal volatility 1bp

```

normal_calculator_function <- GaussQuant::make_normal_calculator_GQL(
  option_sign = option_sign,
  strike = strike_rate,
  maturity = maturity_year_fraction,
  discount_factor = maturity_discount_factor,
  forward = forward_rate,
  vol = normal_volatility
)

normal_calculator_object <- GaussQuant::normal_calculator_GQL(
  option_sign = option_sign,
  strike = strike_rate,
  maturity = maturity_year_fraction,

```



```

    discount_factor = maturity_discount_factor,
    forward = forward_rate,
    vol = normal_volatility
  )

forward_up_1bp <- forward_rate + 1 / 10000
volatility_up_1bp <- normal_volatility + 1 / 10000

maturity_after_one_day <- max(
  maturity_year_fraction - 1 / 365,
  0
)

discount_factor_after_one_day <- exp(
  -risk_free_rate * maturity_after_one_day
)

closure_scenario_tbl <- tibble::tribble(
  ~scenario,      ~npv,
  "base",         normal_calculator_function(),
  "forward_up_1bp", normal_calculator_function(
    forward_new = forward_up_1bp
  ),
  "normal_vol_up_1bp", normal_calculator_function(
    vol_new = volatility_up_1bp
  ),
  "one_day_passed",   normal_calculator_function(
    maturity_new = maturity_after_one_day,
    discount_factor_new = discount_factor_after_one_day
  )
) |>
dplyr::mutate(
  difference_from_base = .data$npv -
    .data$npv[.data$scenario == "base"]
)

class_style_tbl <- tibble::tribble(
  ~scenario,      ~npv,
  "base",         normal_calculator_object$npv(),
  "forward_up_1bp", normal_calculator_object$npv(
    forward_new = forward_up_1bp
  ),

```

```

"normal_vol_up_1bp", normal_calculator_object$npv(
  vol_new = volatility_up_1bp
),
"one_day_passed",    normal_calculator_object$npv(
  maturity_new = maturity_after_one_day,
  discount_factor_new = discount_factor_after_one_day
)
)

GaussQuant::show_tbl_GQH(
  closure_scenario_tbl,
  "Normal-model scenarios",
  n = 20
)
#> # A tibble: 4 x 3
#>   scenario                npv difference_from_base
#>   <chr>                <dbl>                <dbl>
#> 1 base                  0.0011340                0
#> 2 forward_up_1bp        0.0010787       -0.000055284
#> 3 normal_vol_up_1bp     0.0011521        0.000018195
#> 4 one_day_passed        0.0011284       -0.0000055347

GaussQuant::show_tbl_GQH(
  class_style_tbl,
  "Class-like normal calculator",
  n = 20
)
#> # A tibble: 4 x 2
#>   scenario                npv
#>   <chr>                <dbl>
#> 1 base                  0.0011340
#> 2 forward_up_1bp        0.0010787
#> 3 normal_vol_up_1bp     0.0011521
#> 4 one_day_passed        0.0011284

```

call put 1bp Normal volatility

15.4 5.4 Caplet Cap

Caplet call option $[T_{i-1}, T_i]$ F_i K α_i $P(0, T_i)$

Caplet

$$V_i^{\text{caplet}} = N\alpha_i P(0, T_i) [(F_i - K)N(d_i) + \sigma_{N,i}\sqrt{\tau_i}\phi(d_i)]$$

$$d_i = \frac{F_i - K}{\sigma_{N,i}\sqrt{\tau_i}}$$

Cap Caplet

$$V^{\text{cap}} = \sum_i V_i^{\text{caplet}}$$

Caplet Fixing

```
cap_notional <- 10000000
cap_strike <- 0.05
flat_discount_rate <- 0.045

caplet_input_tbl <- tibble::tribble(
  ~caplet, ~fixing_time, ~payment_time, ~accrual_fraction, ~forward_rate, ~normal_vol_bp,
  1L,      0.25,      0.50,      0.25,      0.0510,      80,
  2L,      0.50,      0.75,      0.25,      0.0505,      82,
  3L,      0.75,      1.00,      0.25,      0.0495,      84,
  4L,      1.00,      1.25,      0.25,      0.0485,      86
) |>
dplyr::mutate(
  normal_volatility = .data$normal_vol_bp / 10000,
  discount_factor = exp(
    -flat_discount_rate * .data$payment_time
  ),
  option_value_per_rate_unit = purrr::pmap_dbl(
    list(
      .data$forward_rate,
      .data$normal_volatility,
      .data$fixing_time,
      .data$discount_factor
    ),
    function(
      forward_rate,
      normal_volatility,
```

```

    fixing_time,
    discount_factor
  ) {
    GaussQuant::normal_option_price_GQL(
      option_sign = 1,
      forward = forward_rate,
      strike = cap_strike,
      vol = normal_volatility,
      maturity = fixing_time,
      discount_factor = discount_factor
    )
  }
),
caplet_npv = cap_notional *
  .data$accrual_fraction *
  .data$option_value_per_rate_unit
)

cap_summary_tbl <- tibble::tribble(
  ~metric,      ~value,
  "cap_notional", cap_notional,
  "cap_strike",   cap_strike,
  "cap_npv",      sum(caplet_input_tbl$caplet_npv)
)

GaussQuant::show_tbl_GQH(
  caplet_input_tbl,
  "Bachelier caplet decomposition",
  n = 20
)

#> # A tibble: 4 x 10
#>   caplet fixing_time payment_time accrual_fraction forward_rate normal_vol_bp
#>   <int>      <dbl>      <dbl>          <dbl>          <dbl>          <dbl>
#> 1     1         0.25         0.5            0.25           0.051           80
#> 2     2         0.5         0.75            0.25           0.0505          82
#> 3     3         0.75         1              0.25           0.0495          84
#> 4     4         1         1.25            0.25           0.0485          86
#>   normal_volatility discount_factor option_value_per_rate_unit caplet_npv
#>           <dbl>          <dbl>          <dbl>          <dbl>
#> 1           0.008         0.97775          0.0020976      5244.1
#> 2           0.0082        0.96681          0.0024864      6216.1
#> 3           0.0084        0.95600          0.0025420      6355.0

```

```

#> 4          0.0086          0.94530          0.0025835          6458.7

GaussQuant::show_tbl_GQH(
  cap_summary_tbl,
  "Bachelier cap summary",
  n = 20
)
#> # A tibble: 3 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 cap_notional 10000000
#> 2 cap_strike    0.05
#> 3 cap_npv       24274.

```

Caplet Fixing

15.5 5.5 Swaption

15.5.1 5.5.1

Swaption

$$T_1, \dots, T_n \quad \alpha_i$$

$$A(0) = \sum_{i=1}^n \alpha_i P(0, T_i)$$

$$T_0 \quad T_n$$

$$S_0 = \frac{P(0, T_0) - P(0, T_n)}{A(0)}$$

```

swap_exercise_time <- 1
swap_start_time <- 1
fixed_payment_times <- c(2, 3, 4)
fixed_accrual_fractions <- rep(
  1,
  length(fixed_payment_times)
)

```

```

swap_flat_discount_rate <- 0.035

swap_discount_tbl <- tibble::tibble(
  payment_time = fixed_payment_times,
  accrual_fraction = fixed_accrual_fractions,
  discount_factor = exp(
    -swap_flat_discount_rate * fixed_payment_times
  )
)

swap_annuity <- sum(
  swap_discount_tbl$accrual_fraction *
  swap_discount_tbl$discount_factor
)

swap_start_discount_factor <- exp(
  -swap_flat_discount_rate * swap_start_time
)

swap_end_discount_factor <- exp(
  -swap_flat_discount_rate *
  max(fixed_payment_times)
)

forward_swap_rate <- (
  swap_start_discount_factor -
  swap_end_discount_factor
) / swap_annuity

forward_swap_tbl <- tibble::tribble(
  ~metric, ~value,
  "swap_start_discount_factor", swap_start_discount_factor,
  "swap_end_discount_factor", swap_end_discount_factor,
  "swap_annuity", swap_annuity,
  "forward_swap_rate", forward_swap_rate
)

GaussQuant::show_tbl_GQH(
  swap_discount_tbl,
  "Underlying swap fixed-leg schedule",
  n = 20
)

```

```

#> # A tibble: 3 x 3
#>   payment_time accrual_fraction discount_factor
#>   <dbl>         <dbl>         <dbl>
#> 1         2         1         0.93239
#> 2         3         1         0.90032
#> 3         4         1         0.86936

GaussQuant::show_tbl_GQH(
  forward_swap_tbl,
  "Forward swap rate and annuity",
  n = 20
)
#> # A tibble: 4 x 2
#>   metric          value
#>   <chr>         <dbl>
#> 1 swap_start_discount_factor 0.96561
#> 2 swap_end_discount_factor   0.86936
#> 3 swap_annuity              2.7021
#> 4 forward_swap_rate          0.035620

```

15.5.2 5.5.2 Bachelier Swaption

payer swaption

call option

swap measure

Normal

$$dS_t = \sigma_S dW_t^A$$

payer swaption

$$V_{\text{payer}} = NA(0) \left[(S_0 - K)N(d) + \sigma_S \sqrt{T_e} \phi(d) \right]$$

$$d = \frac{S_0 - K}{\sigma_S \sqrt{T_e}}$$

receiver swaption put option

```

swaption_notional <- 10000000
swaption_strike <- 0.034
swaption_normal_volatility <- 100 / 10000

payer_option_value_per_annuity <- GaussQuant::normal_option_price_GQL(
  option_sign = 1,
  forward = forward_swap_rate,
  strike = swaption_strike,
  vol = swaption_normal_volatility,
  maturity = swap_exercise_time,
  discount_factor = 1
)

receiver_option_value_per_annuity <- GaussQuant::normal_option_price_GQL(
  option_sign = -1,
  forward = forward_swap_rate,
  strike = swaption_strike,
  vol = swaption_normal_volatility,
  maturity = swap_exercise_time,
  discount_factor = 1
)

swaption_tbl <- tibble::tribble(
  ~option_type, ~option_sign, ~npv,
  "payer",      1,            swaption_notional *
                                swap_annuity *
                                payer_option_value_per_annuity,
  "receiver",   -1,            swaption_notional *
                                swap_annuity *
                                receiver_option_value_per_annuity
) |>
dplyr::mutate(
  forward_swap_rate = forward_swap_rate,
  strike = swaption_strike,
  annuity = swap_annuity,
  normal_volatility = swaption_normal_volatility
) |>
dplyr::relocate(
  .data$option_type,
  .data$forward_swap_rate,
  .data$strike,
  .data$annuity,

```



```

    .data$normal_volatility,
    .data$npv
  )

GaussQuant::show_tbl_GQH(
  swaption_tbl,
  "Bachelier swaption summary",
  n = 20
)
#> # A tibble: 2 x 7
#>   option_type forward_swap_rate strike annuity normal_volatility    npv
#>   <chr>          <dbl>    <dbl>    <dbl>          <dbl>    <dbl>
#> 1 payer          0.035620  0.034  2.7021          0.01 131091.
#> 2 receiver       0.035620  0.034  2.7021          0.01  87325.
#>   option_sign
#>   <dbl>
#> 1         1
#> 2        -1

```

$A(0)$ Normal 1

15.6 5.6 Black volatility Normal volatility

Black ATM

$$V_{\text{Black,ATM}} \approx D(0, T) F_0 \sigma_B \sqrt{T} \phi(0)$$

Normal

$$V_{\text{Normal,ATM}} \approx D(0, T) \sigma_N \sqrt{T} \phi(0)$$

ATM

$$\sigma_N \approx F_0 \sigma_B$$

Normal volatility

Black volatility

Normal volatility

```

black_volatility <- 0.20

black_normal_comparison_tbl <- tibble::tribble(
  ~forward_rate, ~black_volatility,
  0.0500,        black_volatility,
  0.0200,        black_volatility,
  0.0050,        black_volatility
) |>
  dplyr::mutate(
    approximate_normal_volatility =
      .data$forward_rate *
      .data$black_volatility,
    approximate_normal_volatility_bp =
      .data$approximate_normal_volatility *
      10000
  )

GaussQuant::show_tbl_GQH(
  black_normal_comparison_tbl,
  "Approximate ATM Black-to-Normal volatility conversion",
  n = 20
)

#> # A tibble: 3 x 4
#>   forward_rate black_volatility approximate_normal_volatility
#>   <dbl>         <dbl>         <dbl>
#> 1      0.05         0.2           0.01
#> 2      0.02         0.2           0.004
#> 3      0.005        0.2           0.001
#>   approximate_normal_volatility_bp
#>   <dbl>
#> 1      100
#> 2       40
#> 3       10

```

ATM Strike ATM

15.7 5.7

Normal

$$dF_t = \sigma_N dW_t$$

Normal volatility		Normal volatility 50bp	0.0050
	Payoff	call	put
Cap	Caplet	Caplet	Swaption
Normal		Black Shifted Black Normal	SABR

```
chapter05_summary_tbl <- tibble::tribble(
  ~section, ~main_result,
  "Normal dynamics", "Additive forward-rate process",
  "Futures option", "Rate conversion, price, and Greeks",
  "Implied normal volatility", "Volatility inversion and repricing",
  "Negative rates", "Pricing without a logarithmic restriction",
  "Cap", "Sum of Bachelier caplets",
  "Swaption", "Forward swap rate times annuity"
)

GaussQuant::show_tbl_GQH(
  chapter05_summary_tbl,
  "Chapter II-5 summary",
  n = 20
)

#> # A tibble: 6 x 2
#>   section main_result
#>   <chr> <chr>
#> 1 Normal dynamics Additive forward-rate process
#> 2 Futures option Rate conversion, price, and Greeks
#> 3 Implied normal volatility Volatility inversion and repricing
#> 4 Negative rates Pricing without a logarithmic restriction
#> 5 Cap Sum of Bachelier caplets
#> 6 Swaption Forward swap rate times annuity

cat(
  "\nChapter II-5 Normal Bachelier model completed successfully.\n"
)

#>
#> Chapter II-5 Normal Bachelier model completed successfully.
```

16 II-6 SABR

Black Normal Strike Strike Strike
SABR Stochastic Alpha Beta Rho

F_t α_t

$dF_t = \alpha_t F_t^\beta dW_t^{(1)}$

$d\alpha_t = \nu \alpha_t dW_t^{(2)}$

$dW_t^{(1)} dW_t^{(2)} = \rho dt$

SABR $\alpha \beta \nu \rho$ 4

- α
- β
- ν
- ρ

$\beta = 1$ $\beta = 0$ Normal 4 β $\alpha \nu \rho$
Black volatility SABR Strike Normal volatility Shifted
SABR

16.1 6.1 SABR

16.1.1 6.1.1 SABR

SABR

16.1.2 6.1.2 Alpha

α α

16.1.3 6.1.3 Beta

β

instantaneous volatility = $\alpha_t F_t^\beta$

$\beta = 1$ Black $\beta = 0$ Normal

16.1.4 6.1.4 Volatility of volatility

ν volatility of volatility vol-of-vol ν
volvol

16.1.5 6.1.5 Rho

ρ ρ Strike

16.1.6 6.1.6 Hagan

SABR Hagan SABR Black volatility Normal volatility
Strike K_i σ_i^{mkt} SABR σ_i^{SABR} RMSE

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\sigma_i^{\text{mkt}} - \sigma_i^{\text{SABR}})^2}$$

β RMSE $\alpha \nu \rho$

16.2 6.2 Black SABR

16.2.1 6.2.1

0.56% 2 $\beta = 0.5$ Black volatility Strike

```

black_market_data <- tibble::tribble(
  ~strike_pct, ~market_vol_pct,
  0.06,        90.78,
  0.31,        46.09,
  0.56,        45.30,
  0.81,        50.17,
  1.06,        53.85,
  1.56,        58.42,
  2.56,        62.72
) |>
dplyr::mutate(
  strike = .data$strike_pct / 100,
  market_vol = .data$market_vol_pct / 100
)

forward_black <- 0.0056
maturity_black <- 2
beta_black <- 0.5

black_model_inputs <- tibble::tribble(
  ~input,      ~value,
  "forward",   forward_black,
  "maturity",  maturity_black,
  "fixed_beta", beta_black
)

initial_parameters_black <- c(
  alpha = 0.1,
  volvol = 0.1,
  rho = 0.1
)

GaussQuant::show_tbl_GQH(
  black_market_data |>
  dplyr::select(
    .data$strike,
    .data$market_vol
  ),
  "Black SABR market data",
  n = 20
)
#> # A tibble: 7 x 2

```

```

#>   strike market_vol
#>   <dbl>      <dbl>
#> 1 0.0006      0.9078
#> 2 0.0031      0.4609
#> 3 0.0056      0.453
#> 4 0.0081      0.5017
#> 5 0.0106      0.5385
#> 6 0.0156      0.5842
#> 7 0.0256      0.6272

GaussQuant::show_tbl_GQH(
  black_model_inputs,
  "Black SABR model inputs",
  n = 20
)
#> # A tibble: 3 x 2
#>   input      value
#>   <chr>      <dbl>
#> 1 forward    0.0056
#> 2 maturity    2
#> 3 fixed_beta 0.5

```

ATM Strike 0.56% 0.56% ATM α ν ρ

16.2.2 6.2.2

Strike QuantLib SABR Black volatility Black volatility RMSE

```

objective_sabr_black <- function(parameters) {
  alpha <- parameters[[1]]
  volvol <- parameters[[2]]
  rho <- parameters[[3]]

  if (
    alpha <= 0 ||
    volvol < 0 ||
    rho <= -0.9999 ||
    rho >= 0.9999
  ) {
    return(1e6)
  }
}

```

```

calculated_volatility <- purrr::map_dbl(
  black_market_data$strike,
  function(strike_value) {
    GaussQuant::safe_sabr_vol_GQL(
      strike = strike_value,
      forward = forward_black,
      maturity = maturity_black,
      alpha = alpha,
      beta = beta_black,
      volvol = volvol,
      rho = rho
    )
  }
)

if (
  any(is.na(calculated_volatility)) ||
  any(!is.finite(calculated_volatility))
) {
  return(1e6)
}

GaussQuant::rmse_GQH(
  actual = black_market_data$market_vol,
  fitted = calculated_volatility
)
}

black_fit <- stats::optim(
  par = initial_parameters_black,
  fn = objective_sabr_black,
  method = "L-BFGS-B",
  lower = c(
    0.0001,
    0,
    -0.9999
  ),
  upper = c(
    Inf,
    Inf,
    0.9999
  )
)

```



```

)

black_parameters <- black_fit$par

black_calculated_volatility <- purrr::map_dbl(
  black_market_data$strike,
  function(strike_value) {
    GaussQuant::safe_sabr_vol_GQL(
      strike = strike_value,
      forward = forward_black,
      maturity = maturity_black,
      alpha = black_parameters[[1]],
      beta = beta_black,
      volvol = black_parameters[[2]],
      rho = black_parameters[[3]]
    )
  }
)

black_calibration_tbl <- tibble::tribble(
  ~parameter,      ~value,
  "alpha",         black_parameters[[1]],
  "beta",          beta_black,
  "volvol",        black_parameters[[2]],
  "rho",           black_parameters[[3]],
  "rmse",          black_fit$value,
  "convergence_code", black_fit$convergence
)

GaussQuant::show_tbl_GQH(
  black_calibration_tbl,
  "Black SABR calibration summary",
  n = 20
)

#> # A tibble: 6 x 2
#>   parameter      value
#>   <chr>         <dbl>
#> 1 alpha        0.030583
#> 2 beta         0.5
#> 3 volvol       0.72848
#> 4 rho          0.44933
#> 5 rmse         0.013881

```

```
#> 6 convergence_code 0
```

convergence_code	0	optim()	RMSE
------------------	---	---------	------

16.2.3 6.2.3

```
black_smile_tbl <- black_market_data |>
  dplyr::transmute(
    strike = .data$strike,
    market_vol = .data$market_vol,
    sabr_vol = black_calculated_volatility,
    residual = .data$market_vol -
      black_calculated_volatility
  )

black_smile_plot_tbl <- black_smile_tbl |>
  dplyr::select(
    .data$strike,
    .data$market_vol,
    .data$sabr_vol
  ) |>
  tidyr::pivot_longer(
    cols = c(
      market_vol,
      sabr_vol
    ),
    names_to = "series",
    values_to = "volatility"
  )

black_smile_plot <- ggplot2::ggplot(
  black_smile_plot_tbl,
  ggplot2::aes(
    x = .data$strike,
    y = .data$volatility,
    group = .data$series,
    linetype = .data$series,
    shape = .data$series
  )
) +
```

```

ggplot2::geom_line(
  linewidth = 0.6
) +
ggplot2::geom_point(
  size = 2.2
) +
ggplot2::scale_x_continuous(
  labels = function(x) sprintf("%.2f%%", 100 * x)
) +
ggplot2::scale_y_continuous(
  labels = function(x) sprintf("%.0f%%", 100 * x)
) +
ggplot2::labs(
  title = "Black SABR calibration",
  subtitle = paste0(
    "alpha=",
    sprintf("%.5f", black_parameters[[1]]),
    ", beta=",
    sprintf("%.3f", beta_black),
    ", volvol=",
    sprintf("%.5f", black_parameters[[2]]),
    ", rho=",
    sprintf("%.5f", black_parameters[[3]])
  ),
  x = "Strike",
  y = "Black volatility",
  linetype = "Series",
  shape = "Series"
) +
ggplot2::theme_minimal()

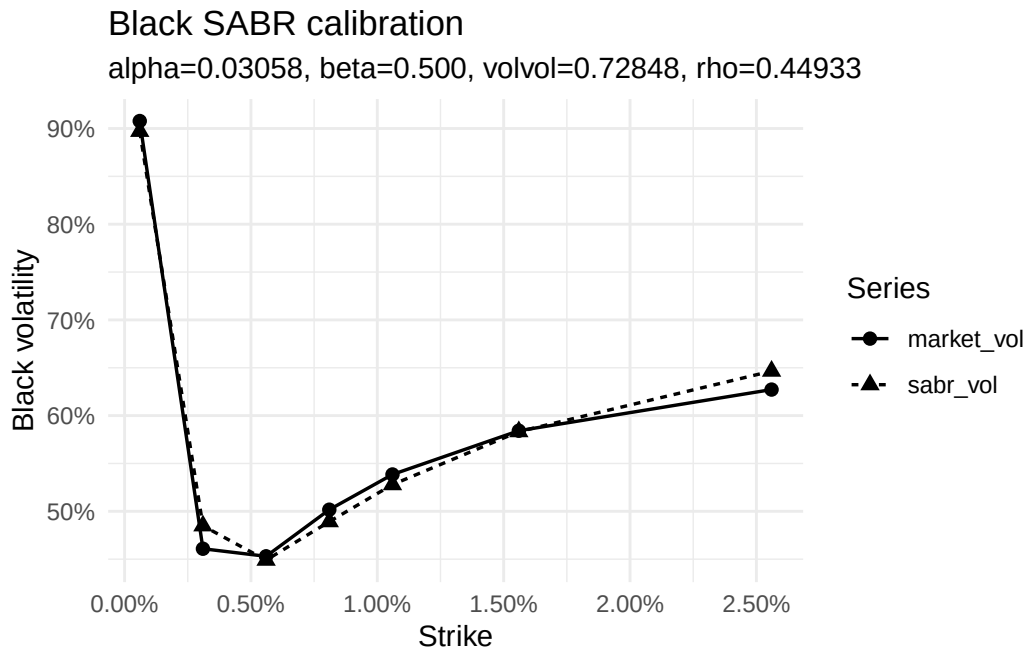
GaussQuant::show_tbl_GQH(
  black_smile_tbl,
  "Black SABR smile table",
  n = 20
)

#> # A tibble: 7 x 4
#>   strike market_vol sabr_vol   residual
#>   <dbl>      <dbl>    <dbl>     <dbl>
#> 1 0.0006      0.9078  0.89705  0.010753
#> 2 0.0031      0.4609  0.48489 -0.023985
#> 3 0.0056      0.453   0.44898  0.0040242

```

```
#> 4 0.0081      0.5017  0.48919  0.012510
#> 5 0.0106      0.5385  0.52789  0.010614
#> 6 0.0156      0.5842  0.58328  0.00092393
#> 7 0.0256      0.6272  0.64648 -0.019277
```

```
print(black_smile_plot)
```



SABR

Strike

16.3 6.3 SABR

```
base_parameter_tbl <- tibble::tribble(
  ~parameter, ~base_value, ~shift,
  "alpha",    black_parameters[[1]], 0.02,
  "beta",     beta_black,           0.20,
  "volvol",   black_parameters[[2]], 0.40,
  "rho",      black_parameters[[3]], 0.44
)
```

```

base_parameters <- stats::setNames(
  base_parameter_tbl$base_value,
  base_parameter_tbl$parameter
)

parameter_shifts <- stats::setNames(
  base_parameter_tbl$shift,
  base_parameter_tbl$parameter
)

sabr_sensitivity_tbl <- purrr::map_dfr(
  names(base_parameters),
  function(parameter_name) {
    parameters_up <- base_parameters
    parameters_down <- base_parameters

    parameters_up[[parameter_name]] <-
      parameters_up[[parameter_name]] +
      parameter_shifts[[parameter_name]]

    parameters_down[[parameter_name]] <-
      parameters_down[[parameter_name]] -
      parameter_shifts[[parameter_name]]

    parameters_up[["alpha"]] <- max(
      parameters_up[["alpha"]],
      1e-6
    )
    parameters_down[["alpha"]] <- max(
      parameters_down[["alpha"]],
      1e-6
    )
    parameters_up[["beta"]] <- min(
      max(parameters_up[["beta"]], 0),
      1
    )
    parameters_down[["beta"]] <- min(
      max(parameters_down[["beta"]], 0),
      1
    )
    parameters_up[["volvol"]] <- max(
      parameters_up[["volvol"]],

```

```

    0
  )
  parameters_down[["volvol"]] <- max(
    parameters_down[["volvol"]],
    0
  )
  parameters_up[["rho"]] <- min(
    max(parameters_up[["rho"]], -0.95),
    0.95
  )
  parameters_down[["rho"]] <- min(
    max(parameters_down[["rho"]], -0.95),
    0.95
  )

  scenario_parameter_tbl <- tibble::tribble(
    ~scenario, ~alpha, ~beta, ~volvol,
    "base",    base_parameters[["alpha"]], base_parameters[["beta"]], base_parameters[
    "up",      parameters_up[["alpha"]],   parameters_up[["beta"]],   parameters_up[["
    "down",    parameters_down[["alpha"]], parameters_down[["beta"]], parameters_down[

  )

  purrr::pmap_dfr(
    scenario_parameter_tbl,
    function(
      scenario,
      alpha,
      beta,
      volvol,
      rho
    ) {
      tibble::tibble(
        strike = black_market_data$strike,
        volatility = purrr::map_dbl(
          black_market_data$strike,
          function(strike_value) {
            GaussQuant::safe_sabr_vol_GQL(
              strike = strike_value,
              forward = forward_black,
              maturity = maturity_black,
              alpha = alpha,
              beta = beta,

```

```

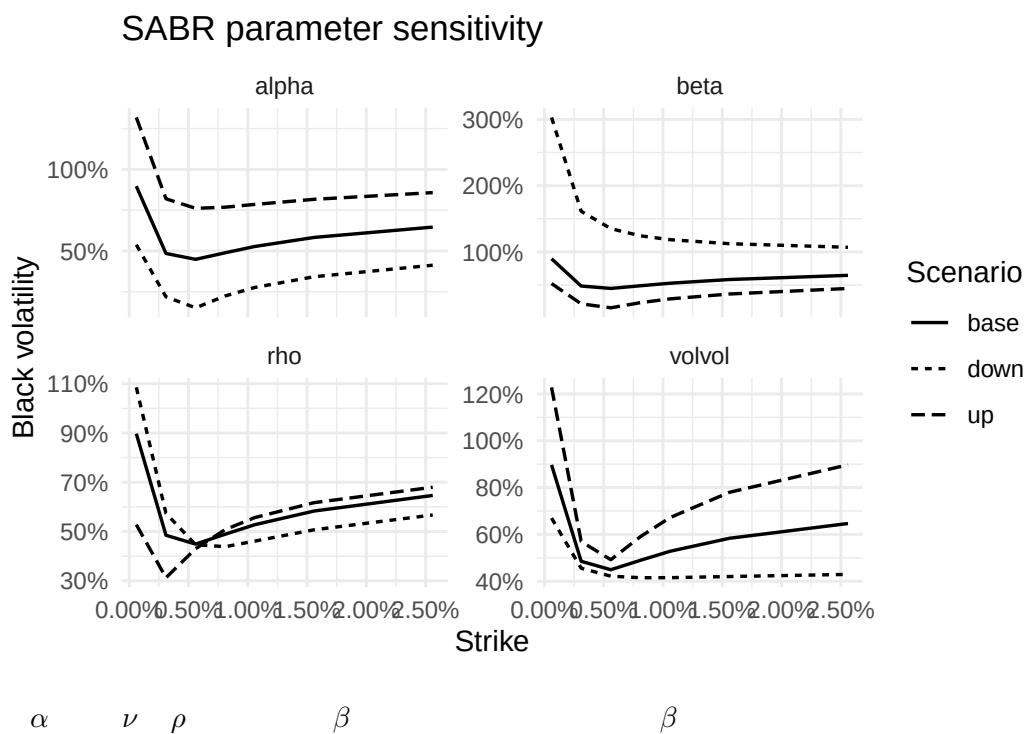
        volvol = volvol,
        rho = rho
    )
}
),
scenario = scenario,
parameter = parameter_name
)
}
)
}
)

sabr_sensitivity_plot <- ggplot2::ggplot(
  sabr_sensitivity_tbl,
  ggplot2::aes(
    x = .data$strike,
    y = .data$volatility,
    group = .data$scenario,
    linetype = .data$scenario
  )
) +
  ggplot2::geom_line(
    linewidth = 0.6
  ) +
  ggplot2::facet_wrap(
    ~parameter,
    scales = "free_y"
  ) +
  ggplot2::scale_x_continuous(
    labels = function(x) sprintf("%.2f%%", 100 * x)
  ) +
  ggplot2::scale_y_continuous(
    labels = function(x) sprintf("%.0f%%", 100 * x)
  ) +
  ggplot2::labs(
    title = "SABR parameter sensitivity",
    x = "Strike",
    y = "Black volatility",
    linetype = "Scenario"
  ) +
  ggplot2::theme_minimal()

```

```
GaussQuant::show_tbl_GQH(
  base_parameter_tbl,
  "SABR sensitivity settings",
  n = 20
)
#> # A tibble: 4 x 3
#>   parameter base_value shift
#>   <chr>         <dbl> <dbl>
#> 1 alpha         0.030583 0.02
#> 2 beta          0.5      0.2
#> 3 volvol        0.72848 0.4
#> 4 rho           0.44933 0.44

print(sabr_sensitivity_plot)
```



16.4 6.4 Shifted SABR

Black SABR

Strike

Strike Shift s

$$\widetilde{F} = F + s$$

$$\widetilde{K} = K + s$$

Shift	SABR					
	Normal volatility	Shift	Hagan	Normal volatility	Shift	Strike
Shift			Strike			

16.5 6.5 Shifted SABR Normal

16.5.1 6.5.1

0.05% 2 $\beta = 0.5$ Shift 2.5% 0.95% Strike

```
normal_market_data <- tibble::tribble(
  ~strike_pct, ~market_vol_bp,
  -0.95,      41.94,
  -0.45,      38.64,
  -0.20,      37.71,
  0.05,       37.80,
  0.30,       39.05,
  0.55,       41.37,
  1.05,       47.81
) |>
dplyr::mutate(
  strike = .data$strike_pct / 100,
  market_vol = .data$market_vol_bp / 10000
)

forward_normal <- 0.0005
maturity_normal <- 2
beta_normal <- 0.5
shift_normal <- 0.025

normal_model_inputs <- tibble::tribble(
  ~input,      ~value,
  "forward",   forward_normal,
  "maturity",  maturity_normal,
```

```

    "fixed_beta",  beta_normal,
    "shift",       shift_normal
  )

  initial_parameters_normal <- c(
    alpha = 0.1,
    volvol = 0.1,
    rho = 0.1
  )

  GaussQuant::show_tbl_GQH(
    normal_market_data |>
      dplyr::select(
        .data$strike,
        .data$market_vol
      ),
    "Shifted SABR normal market data",
    n = 20
  )
#> # A tibble: 7 x 2
#>   strike market_vol
#>   <dbl>     <dbl>
#> 1 -0.0095  0.004194
#> 2 -0.0045  0.003864
#> 3 -0.002   0.003771
#> 4  0.0005  0.00378
#> 5  0.003   0.003905
#> 6  0.0055  0.004137
#> 7  0.0105  0.004781

  GaussQuant::show_tbl_GQH(
    normal_model_inputs,
    "Shifted SABR normal model inputs",
    n = 20
  )
#> # A tibble: 4 x 2
#>   input      value
#>   <chr>     <dbl>
#> 1 forward  0.0005
#> 2 maturity 2
#> 3 fixed_beta 0.5
#> 4 shift    0.025

```

Strike

$$K + s > 0$$

Strike 0.95% Shift 2.5% Shift Strike

16.5.2 6.5.2

Black SABR β α ν ρ Normal volatility RMSE

```
objective_sabr_normal <- function(parameters) {
  alpha <- parameters[[1]]
  volvol <- parameters[[2]]
  rho <- parameters[[3]]

  if (
    alpha <= 0.001 ||
    volvol < 0 ||
    rho <= -0.999 ||
    rho >= 0.999
  ) {
    return(1e6)
  }

  calculated_volatility <- purrr::map_dbl(
    normal_market_data$strike,
    function(strike_value) {
      GaussQuant::shifted_normal_vol_hagan_GQL(
        strike = strike_value,
        forward = forward_normal,
        maturity = maturity_normal,
        beta = beta_normal,
        alpha = alpha,
        volvol = volvol,
        rho = rho,
        shift = shift_normal
      )
    }
  )

  if (
```

```

    any(is.na(calculated_volatility)) ||
    any(!is.finite(calculated_volatility))
  ) {
    return(1e6)
  }

  GaussQuant::rmse_GQH(
    actual = normal_market_data$market_vol,
    fitted = calculated_volatility
  )
}

normal_fit <- stats::optim(
  par = initial_parameters_normal,
  fn = objective_sabr_normal,
  method = "L-BFGS-B",
  lower = c(
    0.001,
    0,
    -0.999
  ),
  upper = c(
    Inf,
    Inf,
    0.999
  )
)

normal_parameters <- normal_fit$par

normal_calculated_volatility <- purrr::map_dbl(
  normal_market_data$strike,
  function(strike_value) {
    GaussQuant::shifted_normal_vol_hagan_GQL(
      strike = strike_value,
      forward = forward_normal,
      maturity = maturity_normal,
      beta = beta_normal,
      alpha = normal_parameters[[1]],
      volvol = normal_parameters[[2]],
      rho = normal_parameters[[3]],
      shift = shift_normal
    )
  }
)

```

```

    )
  }
)

normal_calibration_tbl <- tibble::tribble(
  ~parameter,      ~value,
  "alpha",          normal_parameters[[1]],
  "beta",           beta_normal,
  "volvol",         normal_parameters[[2]],
  "rho",            normal_parameters[[3]],
  "shift",          shift_normal,
  "rmse",           normal_fit$value,
  "convergence_code", normal_fit$convergence
)

GaussQuant::show_tbl_GQH(
  normal_calibration_tbl,
  "Shifted SABR normal calibration summary",
  n = 20
)

#> # A tibble: 7 x 2
#>   parameter      value
#>   <chr>         <dbl>
#> 1 alpha         0.025373
#> 2 beta          0.5
#> 3 volvol        0.099359
#> 4 rho           0.099797
#> 5 shift         0.025
#> 6 rmse          0.00029785
#> 7 convergence_code 0

```

16.5.3 6.5.3 Normal

```

normal_smile_tbl <- normal_market_data |>
  dplyr::transmute(
    strike = .data$strike,
    market_vol = .data$market_vol,
    sabr_vol = normal_calculated_volatility,
    residual = .data$market_vol -
      normal_calculated_volatility
  )

```

```

)

normal_smile_plot_tbl <- normal_smile_tbl |>
  dplyr::select(
    .data$strike,
    .data$market_vol,
    .data$sabr_vol
  ) |>
  tidyr::pivot_longer(
    cols = c(
      market_vol,
      sabr_vol
    ),
    names_to = "series",
    values_to = "volatility"
  )

normal_smile_plot <- ggplot2::ggplot(
  normal_smile_plot_tbl,
  ggplot2::aes(
    x = .data$strike,
    y = .data$volatility,
    group = .data$series,
    linetype = .data$series,
    shape = .data$series
  )
) +
  ggplot2::geom_line(
    linewidth = 0.6
  ) +
  ggplot2::geom_point(
    size = 2.2
  ) +
  ggplot2::scale_x_continuous(
    labels = function(x) sprintf("%.2f%%", 100 * x)
  ) +
  ggplot2::scale_y_continuous(
    labels = function(x) sprintf("%.1f bp", 10000 * x)
  ) +
  ggplot2::labs(
    title = "Shifted SABR normal calibration",
    subtitle = paste0(

```

```

    "alpha=",
    sprintf("%.5f", normal_parameters[[1]]),
    ", beta=",
    sprintf("%.3f", beta_normal),
    ", volvol=",
    sprintf("%.5f", normal_parameters[[2]]),
    ", rho=",
    sprintf("%.5f", normal_parameters[[3]]),
    ", shift=",
    sprintf("%.2f%%", 100 * shift_normal)
  ),
  x = "Strike",
  y = "Normal volatility",
  linetype = "Series",
  shape = "Series"
) +
  ggplot2::theme_minimal()

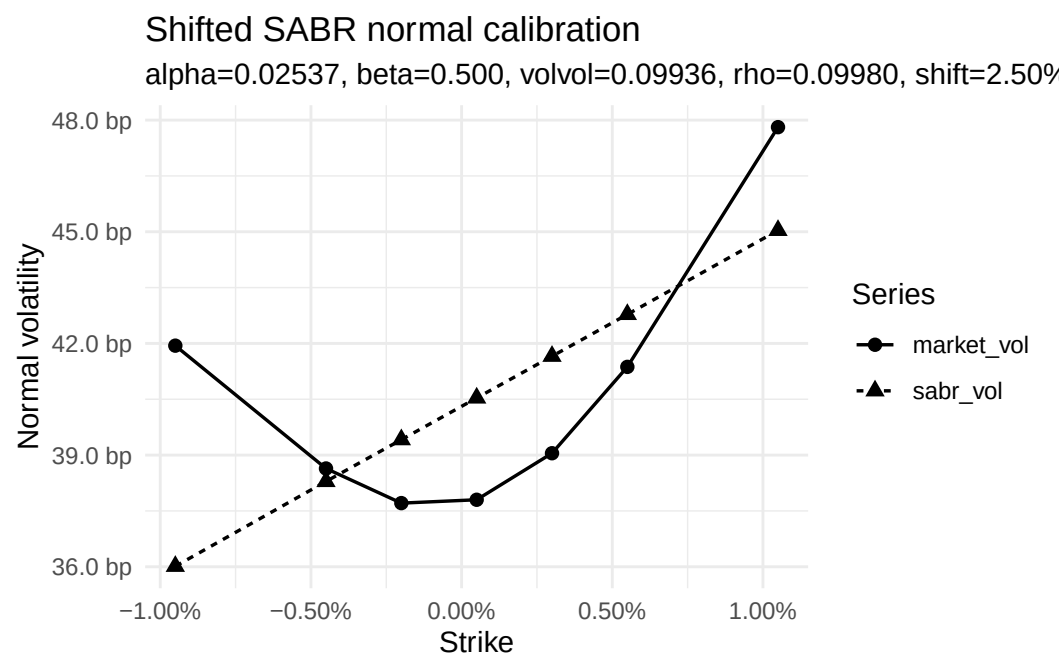
GaussQuant::show_tbl_GQH(
  normal_smile_tbl |>
    dplyr::mutate(
      market_vol_bp = 10000 * .data$market_vol,
      sabr_vol_bp = 10000 * .data$sabr_vol,
      residual_bp = 10000 * .data$residual
    ),
  "Shifted SABR normal smile table",
  n = 20
)

#> # A tibble: 7 x 7
#>   strike market_vol sabr_vol residual market_vol_bp sabr_vol_bp
#>   <dbl>     <dbl>    <dbl>      <dbl>      <dbl>      <dbl>
#> 1 -0.0095  0.004194 0.0036013  0.00059271    41.94    36.013
#> 2 -0.0045  0.003864 0.0038292  0.000034815   38.64    38.292
#> 3 -0.002   0.003771 0.0039414 -0.00017044   37.71    39.414
#> 4  0.0005   0.00378  0.0040535 -0.00027345   37.8     40.535
#> 5  0.003    0.003905 0.0041656 -0.00026056   39.05    41.656
#> 6  0.0055   0.004137 0.0042779 -0.00014094   41.37    42.779
#> 7  0.0105   0.004781 0.0045037  0.00027730   47.81    45.037
#>   residual_bp
#>   <dbl>
#> 1    5.9271
#> 2    0.34815

```

```
#> 3    -1.7044
#> 4    -2.7345
#> 5    -2.6056
#> 6    -1.4094
#> 7     2.7730

print(normal_smile_plot)
```



Black volatility Normal volatility Black SABR RMSE Shifted SABR Normal RM-

SE Black Normal

16.6 6.6

16.6.1 6.6.1 Strike

SABR

Strike Normal volatility


```

outer_strike_input_tbl <- tibble::tribble(
  ~label,      ~strike,
  "lower_wing", -0.0195,
  "upper_wing",  0.0205
)

outer_strike_tbl <- outer_strike_input_tbl |>
  dplyr::mutate(
    shifted_strike = .data$strike +
      shift_normal,
    normal_volatility = purrr::map_dbl(
      .data$strike,
      function(strike_value) {
        GaussQuant::shifted_normal_vol_hagan_GQL(
          strike = strike_value,
          forward = forward_normal,
          maturity = maturity_normal,
          beta = beta_normal,
          alpha = normal_parameters[[1]],
          volvol = normal_parameters[[2]],
          rho = normal_parameters[[3]],
          shift = shift_normal
        )
      }
    ),
    normal_volatility_bp = 10000 *
      .data$normal_volatility
  )

GaussQuant::show_tbl_GQH(
  outer_strike_tbl,
  "Shifted SABR outer-strike check",
  n = 20
)

#> # A tibble: 2 x 5
#>   label      strike shifted_strike normal_volatility normal_volatility_bp
#>   <chr>      <dbl>      <dbl>          <dbl>          <dbl>
#> 1 lower_wing -0.0195      0.0055      0.0030674      30.674
#> 2 upper_wing  0.0205      0.0455      0.0049584      49.584

```

Shift Strike

16.6.2 6.6.2 Black SABR Shifted SABR Normal

```
calibration_comparison_tbl <- tibble::tribble(
  ~model,          ~alpha,          ~beta,          ~volvol,          ~rho,
  "Black SABR",    black_parameters[[1]], beta_black, black_parameters[[2]], black_parameters[[3]],
  "Shifted SABR Normal", normal_parameters[[1]], beta_normal, normal_parameters[[2]], normal_parameters[[3]]
)

GaussQuant::show_tbl_GQH(
  calibration_comparison_tbl,
  "SABR calibration comparison",
  n = 20
)

#> # A tibble: 2 x 8
#>   model          alpha  beta  volvol      rho shift      rmse
#>   <chr>          <dbl> <dbl>   <dbl>   <dbl> <dbl>   <dbl>
#> 1 Black SABR      0.030583  0.5  0.72848  0.44933  0      0.013881
#> 2 Shifted SABR Normal 0.025373  0.5  0.099359  0.099797  0.025  0.00029785
#>   convergence_code
#>           <int>
#> 1               0
#> 2               0
```

Black SABR	Strike	Black volatility	Shifted SABR Normal	Strike Shift	Strike	Nor-
mal volatility						
4 SABR			RMSE			

16.7 6.7

SABR		Strike				
α	ν	ρ	β			
Black SABR		Strike	Black volatility	Normal volatility	Strike	Shift
SABR						Shifted
SABR		Strike		Strike		

```

chapter06_summary_tbl <- tibble::tribble(
  ~section,          ~main_result,
  "Black SABR",      "Calibration to a Black-volatility smile",
  "Parameter sensitivity", "Alpha, beta, vol-of-vol, and rho effects",
  "Shifted SABR Normal", "Normal-volatility smile with negative strikes",
  "Outer strikes",    "Extrapolation outside calibration points"
)

GaussQuant::show_tbl_GQH(
  chapter06_summary_tbl,
  "Chapter II-6 summary",
  n = 20
)

#> # A tibble: 4 x 2
#>   section          main_result
#>   <chr>          <chr>
#> 1 Black SABR      Calibration to a Black-volatility smile
#> 2 Parameter sensitivity Alpha, beta, vol-of-vol, and rho effects
#> 3 Shifted SABR Normal Normal-volatility smile with negative strikes
#> 4 Outer strikes  Extrapolation outside calibration points

cat(
  "\nChapter II-6 SABR model completed successfully.\n"
)

#>
#> Chapter II-6 SABR model completed successfully.

```

17 II-7

Early Termination

Callable Bond

Callable Bond

Bullet Bond

$$V_{\text{callable}} = V_{\text{bullet}} - V_{\text{issuer call}}$$

Receiver Swaption

Bermudan Receiver Swap-

tion

Callable Bond Swaption

$$V_{\text{callable}} \approx V_{\text{bullet}} - V_{\text{Bermudan receiver swaption}}$$

Hull-White 1-factor

Tree

```
knitr::opts_chunk$set(
  collapse = TRUE,
  comment = "#>",
  warning = FALSE,
  message = FALSE
)

suppressPackageStartupMessages({
  library(tidyverse)
  library(ggthemes)
  library(QuantLib)
  library(GaussQuant)
  library(tidyverse)
  library(broom)
})
```

17.1 7.1 Hull–White

17.1.1 7.1.1 Hull–White 1-factor

Hull–White r_t Gaussian

$$dr_t = [\theta(t) - ar_t] dt + \sigma dW_t$$

a σ $\theta(t)$
 a σ

Hull–White $\theta(t)$ x_t Ornstein–Uhlenbeck

$$dx_t = -ax_t dt + \sigma dW_t$$

$$r_t = \phi(t) + x_t$$

Hull–White Gaussian Normal

17.1.2 7.1.2

5 3% 3% Bullet Bond
1 100

```
trade_date <- GaussQuant::date_GQL("2024-04-15")
issue_date <- GaussQuant::date_GQL("2024-04-17")
maturity_date <- GaussQuant::date_GQL("2029-04-17")

GaussQuant::set_eval_date_GQL(trade_date)

coupon_rate <- 0.03
market_yield <- 0.03
settlement_days <- 2L
face_amount <- 100
call_price_clean <- 100
non_call_periods <- 2L
```

```

bond_calendar <- QuantLib::WeekendsOnly()
bond_day_counter <- QuantLib::ActualActual("Bond")
model_day_counter <- QuantLib::Actual365Fixed()
bond_compounding <- QuantLib::Compounding_Compounded_get()
bond_frequency <- QuantLib::Frequency_Semiannual_get()

bond_input_tbl <- tibble::tribble(
  ~input,          ~value,
  "trade_date",    "2024-04-15",
  "issue_date",    "2024-04-17",
  "maturity_date", "2029-04-17",
  "coupon_rate",   as.character(coupon_rate),
  "market_yield",  as.character(market_yield),
  "settlement_days", as.character(settlement_days),
  "face_amount",   as.character(face_amount),
  "call_price",    as.character(call_price_clean),
  "non_call_periods", as.character(non_call_periods)
)

GaussQuant::show_tbl_GQH(
  bond_input_tbl,
  "Callable bond input assumptions",
  n = 20
)

#> # A tibble: 9 x 2
#>   input          value
#>   <chr>         <chr>
#> 1 trade_date    2024-04-15
#> 2 issue_date    2024-04-17
#> 3 maturity_date 2029-04-17
#> 4 coupon_rate   0.03
#> 5 market_yield  0.03
#> 6 settlement_days 2
#> 7 face_amount   100
#> 8 call_price    100
#> 9 non_call_periods 2

```

17.1.3 7.1.3 Bullet Bond

```

bond_schedule <- QuantLib::Schedule(
  issue_date,
  maturity_date,
  GaussQuant::period_months_GQL(6),
  bond_calendar,
  "Unadjusted",
  "Unadjusted",
  "Backward",
  FALSE
)

bond_schedule_tbl <- GaussQuant::schedule_table_GQL(
  bond_schedule
)

bullet_bond <- QuantLib::FixedRateBond(
  settlement_days,
  face_amount,
  bond_schedule,
  c(coupon_rate),
  bond_day_counter
)

bullet_settlement_date <- tryCatch(
  bullet_bond$settlementDate(),
  error = function(e) NULL
)

bullet_clean_price <- GaussQuant::bond_clean_price_from_yield_safe_GQL(
  bond = bullet_bond,
  yield_rate = market_yield,
  day_counter = bond_day_counter,
  compounding = bond_compounding,
  frequency = bond_frequency,
  settlement_date = bullet_settlement_date
)

bullet_summary_tbl <- tibble::tribble(
  ~metric, ~value,
  "coupon_rate", coupon_rate,
  "market_yield", market_yield,
  "clean_price_from_yield", bullet_clean_price,

```

```

    "face_amount",                face_amount
  )

GaussQuant::show_tbl_GQH(
  bond_schedule_tbl,
  "Bullet bond schedule",
  n = 20
)
#> # A tibble: 11 x 2
#>   period schedule_date
#>   <int> <date>
#> 1     1 2024-04-17
#> 2     2 2024-10-17
#> 3     3 2025-04-17
#> 4     4 2025-10-17
#> 5     5 2026-04-17
#> 6     6 2026-10-17
#> 7     7 2027-04-17
#> 8     8 2027-10-17
#> 9     9 2028-04-17
#> 10    10 2028-10-17
#> 11    11 2029-04-17

GaussQuant::show_tbl_GQH(
  bullet_summary_tbl,
  "Bullet bond summary",
  n = 20
)
#> # A tibble: 4 x 2
#>   metric                value
#>   <chr>                 <dbl>
#> 1 coupon_rate           0.03
#> 2 market_yield          0.03
#> 3 clean_price_from_yield 99.999
#> 4 face_amount           100

```

17.1.4 7.1.4 Hull–White

Callable Bond Swaption Hull–White
 3% Callable Bond Tree


```

hull_white_a <- 0.03
hull_white_sigma <- 0.01
tree_steps <- 120L

flat_curve <- GaussQuant::flat_curve_GQL(
  reference_date = trade_date,
  rate = market_yield,
  day_counter = model_day_counter,
  compounding = "Continuous"
)

flat_curve_handle <- QuantLib::YieldTermStructureHandle(
  flat_curve
)

hull_white_model <- GaussQuant::hull_white_model_GQL(
  term_structure = flat_curve_handle,
  a = hull_white_a,
  sigma = hull_white_sigma
)

hull_white_input_tbl <- tibble::tribble(
  ~parameter,      ~value,
  "mean_reversion_a", hull_white_a,
  "sigma",          hull_white_sigma,
  "tree_steps",     tree_steps,
  "flat_curve_rate", market_yield
)

GaussQuant::show_tbl_GQH(
  hull_white_input_tbl,
  "Hull-White model assumptions",
  n = 20
)

#> # A tibble: 4 x 2
#>   parameter      value
#>   <chr>          <dbl>
#> 1 mean_reversion_a 0.03
#> 2 sigma            0.01
#> 3 tree_steps       120
#> 4 flat_curve_rate  0.03

```

17.2 7.2 Callable Bond Tree

17.2.1 7.2.1

2

```
bond_schedule_dates <- GaussQuant::schedule_date_vector_GQL(
  bond_schedule
)

first_call_position <- non_call_periods + 1L

call_dates <- bond_schedule_dates[
  first_call_position:
  (length(bond_schedule_dates) - 1L)
]

call_schedule <- GaussQuant::make_callability_schedule_GQL(
  call_dates = call_dates,
  call_price_clean = call_price_clean
)

call_schedule_tbl <- tibble::tibble(
  call_number = seq_along(call_dates),
  call_date = call_dates,
  call_price = rep(
    call_price_clean,
    length(call_dates)
  )
)

GaussQuant::show_tbl_GQH(
  call_schedule_tbl,
  "Callable bond exercise schedule",
  n = 20
)

#> # A tibble: 8 x 3
#>   call_number call_date  call_price
#>   <int> <date>      <dbl>
#> 1         1 2025-04-17         100
#> 2         2 2025-10-17         100
#> 3         3 2026-04-17         100
```

```
#> 4          4 2026-10-17      100
#> 5          5 2027-04-17      100
#> 6          6 2027-10-17      100
#> 7          7 2028-04-17      100
#> 8          8 2028-10-17      100
```

17.2.2 7.2.2 Callable Bond Tree

Callable Bond Hull–White Tree Pricing Engine

Tree node

```
callable_bond <- QuantLib::CallableFixedRateBond(
  settlement_days,
  face_amount,
  bond_schedule,
  c(coupon_rate),
  bond_day_counter,
  "Unadjusted",
  call_price_clean,
  issue_date,
  call_schedule
)

callable_bond_engine <- QuantLib::TreeCallableFixedRateBondEngine(
  hull_white_model,
  tree_steps
)

callable_bond$setPricingEngine(
  callable_bond_engine
)
#> NULL

callable_clean_price <- GaussQuant::callable_bond_clean_price_safe_GQL(
  callable_bond
)

callable_dirty_price <- GaussQuant::callable_bond_dirty_price_safe_GQL(
  callable_bond
)
```

```

callable_settlement_value <- GaussQuant::callable_bond_settlement_value_safe_GQL(
  callable_bond
)

embedded_call_value <- bullet_clean_price -
  callable_clean_price

callable_summary_tbl <- tibble::tribble(
  ~metric, ~value,
  "bullet_clean_price", bullet_clean_price,
  "callable_clean_price", callable_clean_price,
  "callable_dirty_price", callable_dirty_price,
  "callable_settlement_value", callable_settlement_value,
  "embedded_call_value", embedded_call_value
)

GaussQuant::show_tbl_GQH(
  callable_summary_tbl,
  "Callable bond summary",
  n = 20
)
#> # A tibble: 5 x 2
#>   metric value
#>   <chr>   <dbl>
#> 1 bullet_clean_price 99.999
#> 2 callable_clean_price 98.001
#> 3 callable_dirty_price 98.001
#> 4 callable_settlement_value 98.001
#> 5 embedded_call_value 1.9984

```

Callable Bond Bullet Bond

17.3 7.3 Hull–White

Callable Bond

a

```

callable_sensitivity_input_tbl <- tibble::tribble(
  ~scenario, ~mean_reversion, ~sigma,
  "low_sigma", 0.03, 0.005,

```

```

"base",          0.03,          0.010,
"high_sigma",    0.03,          0.020,
"slow_reversion", 0.01,          0.010,
"fast_reversion", 0.10,          0.010
)

callable_sensitivity_tbl <- purrr::pmap_dfr(
  callable_sensitivity_input_tbl,
  function(
    scenario,
    mean_reversion,
    sigma
  ) {
    model_i <- GaussQuant::hull_white_model_GQL(
      term_structure = flat_curve_handle,
      a = mean_reversion,
      sigma = sigma
    )

    engine_i <- QuantLib::TreeCallableFixedRateBondEngine(
      model_i,
      tree_steps
    )

    callable_bond$setPricingEngine(
      engine_i
    )

    callable_price_i <- GaussQuant::callable_bond_clean_price_safe_GQL(
      callable_bond
    )

    tibble::tibble(
      scenario = scenario,
      mean_reversion = mean_reversion,
      sigma = sigma,
      callable_clean_price = callable_price_i,
      embedded_call_value = bullet_clean_price -
        callable_price_i
    )
  }
)

```

```

callable_bond$setPricingEngine(
  callable_bond_engine
)
#> NULL

GaussQuant::show_tbl_GQH(
  callable_sensitivity_tbl,
  "Callable bond Hull-White sensitivity",
  n = 20
)
#> # A tibble: 5 x 5
#>   scenario      mean_reversion sigma callable_clean_price embedded_call_value
#>   <chr>          <dbl> <dbl>          <dbl>          <dbl>
#> 1 low_sigma      0.03 0.005          98.962          1.0371
#> 2 base           0.03 0.01           98.001          1.9984
#> 3 high_sigma     0.03 0.02           96.090          3.9094
#> 4 slow_reversion 0.01 0.01           97.923          2.0766
#> 5 fast_reversion 0.1  0.01           98.243          1.7566

```

17.4 7.4 Bermudan Receiver Swaption

Receiver Swap

Receiver Swap	Callable Bond	Bermudan Receiver Swaption
---------------	---------------	----------------------------

```

floating_index <- QuantLib::JPYLibor(
  GaussQuant::period_months_GQL(6),
  flat_curve_handle
)

tryCatch(
  floating_index$addFixing(
    trade_date,
    market_yield,
    TRUE
  ),
  error = function(e) NULL
)

```

```

#> NULL

underlying_receiver_swap <- QuantLib::VanillaSwap(
  QuantLib::Swap_Receiver_get(),
  face_amount,
  bond_schedule,
  coupon_rate,
  bond_day_counter,
  bond_schedule,
  floating_index,
  0,
  bond_day_counter
)

bermudan_receiver_engine <- GaussQuant::hull_white_swaption_engine_GQL(
  term_structure = flat_curve_handle,
  a = hull_white_a,
  sigma = hull_white_sigma,
  method = "tree",
  time_steps = tree_steps
)

bermudan_receiver_swaption <- GaussQuant::make_bermudan_swaption_GQL(
  underlying_swap = underlying_receiver_swap,
  exercise_dates = call_dates,
  pricing_engine = bermudan_receiver_engine
)

bermudan_receiver_npv <- GaussQuant::swaption_npv_GQL(
  bermudan_receiver_swaption
)

receiver_swaption_value_per_100 <- bermudan_receiver_npv /
  face_amount *
  100

bullet_minus_receiver_swaption <- bullet_clean_price -
  receiver_swaption_value_per_100

callable_comparison_tbl <- tibble::tribble(
  ~valuation, ~value,
  "bullet_bond_clean_price", bullet_clean_price,

```

```

"callable_bond_clean_price",      callable_clean_price,
"receiver_swaption_value_per_100", receiver_swaption_value_per_100,
"bullet_minus_receiver_swaption", bullet_minus_receiver_swaption,
"pricing_difference",             callable_clean_price -
                                   bullet_minus_receiver_swaption
)

GaussQuant::show_tbl_GQH(
  callable_comparison_tbl,
  "Callable bond and Bermudan receiver swaption comparison",
  n = 20
)

#> # A tibble: 5 x 2
#>   valuation      value
#>   <chr>         <dbl>
#> 1 bullet_bond_clean_price      99.999
#> 2 callable_bond_clean_price     98.001
#> 3 receiver_swaption_value_per_100 1.8868
#> 4 bullet_minus_receiver_swaption 98.113
#> 5 pricing_difference          -0.11163

```

Callable Bond Bermudan Receiver Swaption Hull-White

- Swap
- Clean price Swaption NPV
-
-
- Swap
- Fixing

pricing_difference Callable Bond Receiver Swap-
tion

17.5 7.5 Vasicek Hull-White

17.5.1 7.5.1 Vasicek

Vasicek Gaussian

$$dr_t = a(b - r_t)dt + \sigma dW_t$$

b										
Hull–White	Gaussian	$\theta(t)$								
		<table><tr><th>Vasicek</th><th>Hull–White</th></tr><tr><td>b</td><td></td></tr><tr><td>a, b, σ, r_0</td><td>a, σ</td></tr><tr><td>Gaussian</td><td>Gaussian</td></tr></table>	Vasicek	Hull–White	b		a, b, σ, r_0	a, σ	Gaussian	Gaussian
Vasicek	Hull–White									
b										
a, b, σ, r_0	a, σ									
Gaussian	Gaussian									

Vasicek

Hull–White

17.5.2 7.5.2 Vasicek Hull–White

Vasicek

Hull–White

```
maturity_grid <- seq(
  0.25,
  10,
  by = 0.25
)

vasicek_scenario_tbl <- tibble::tribble(
  ~scenario,      ~short_rate,
  "r0 = 1%",      0.01,
  "r0 = 3%",      0.03,
  "r0 = 5%",      0.05
)

vasicek_curve_tbl <- purrr::pmap_dfr(
  vasicek_scenario_tbl,
  function(
    scenario,
    short_rate
  ) {
    tibble::tibble(
      maturity = maturity_grid,
      zero_rate = purrr::map_dbl(
        maturity_grid,
        function(maturity_value) {
```

```

      GaussQuant::vasicek_zero_rate_GQL(
        time = 0,
        maturity = maturity_value,
        mean_reversion = 0.30,
        sigma = 0.01,
        long_run_rate = 0.03,
        short_rate = short_rate
      )
    }
  ),
  model = "Vasicek",
  scenario = scenario
)
}
)

hull_white_scenario_tbl <- tibble::tribble(
  ~scenario,      ~state_variable,
  "x0 = -0.3%",   -0.003,
  "base curve",   0.000,
  "x0 = 0.3%",    0.003
)

hull_white_curve_tbl <- purrr::pmap_dfr(
  hull_white_scenario_tbl,
  function(
    scenario,
    state_variable
  ) {
    tibble::tibble(
      maturity = maturity_grid,
      zero_rate = purrr::map_dbl(
        maturity_grid,
        function(maturity_value) {
          GaussQuant::hull_white_zero_rate_GQL(
            time = 0,
            maturity = maturity_value,
            curve = flat_curve,
            mean_reversion = 0.30,
            sigma = 0.01,
            state_variable = state_variable
          )
        }
      )
    )
  }
)

```

```

    }
  ),
  model = "Hull-White",
  scenario = scenario
)
}
)

short_rate_curve_tbl <- dplyr::bind_rows(
  vasicek_curve_tbl,
  hull_white_curve_tbl
)

short_rate_curve_plot <- ggplot2::ggplot(
  short_rate_curve_tbl,
  ggplot2::aes(
    x = .data$maturity,
    y = .data$zero_rate,
    group = .data$scenario,
    linetype = .data$scenario
  )
) +
  ggplot2::geom_line(
    linewidth = 0.7
  ) +
  ggplot2::facet_wrap(
    ~model,
    scales = "free_y"
  ) +
  ggplot2::scale_y_continuous(
    labels = function(x) sprintf("%.2f%%", 100 * x)
  ) +
  ggplot2::labs(
    title = "Vasicek and Hull-White zero-rate curves",
    x = "Residual maturity (years)",
    y = "Continuously compounded zero rate",
    linetype = "Scenario"
  ) +
  ggplot2::theme_minimal()

GaussQuant::show_tbl_GQH(
  short_rate_curve_tbl,

```

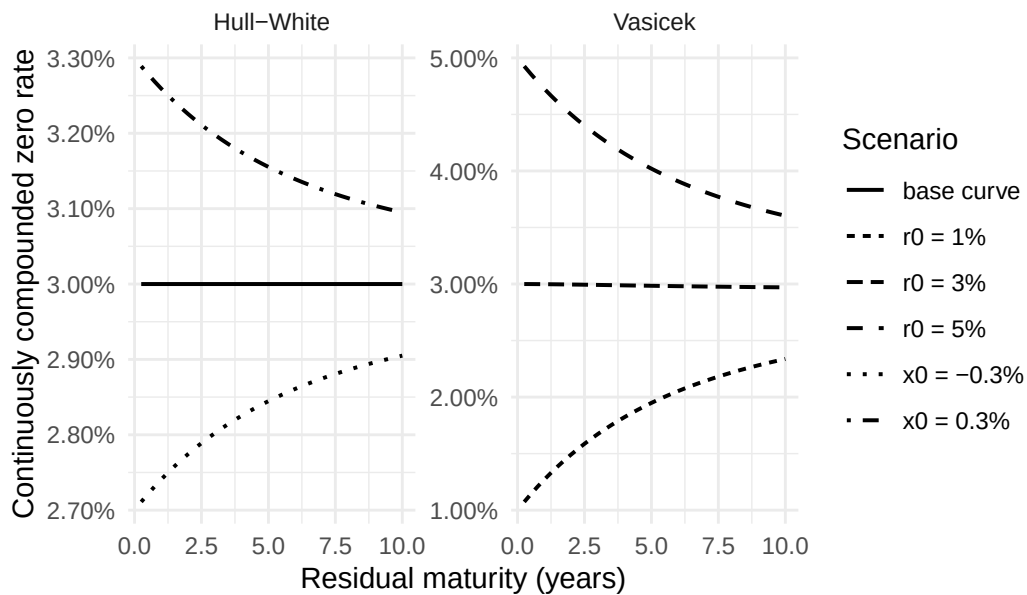
```

"Short-rate model zero-rate curves",
  n = 30
)
#> # A tibble: 30 x 4
#>   maturity zero_rate model  scenario
#>   <dbl>      <dbl> <chr>   <chr>
#> 1     0.25  0.010731 Vasicek r0 = 1%
#> 2     0.5   0.011424 Vasicek r0 = 1%
#> 3     0.75  0.012082 Vasicek r0 = 1%
#> 4     1     0.012708 Vasicek r0 = 1%
#> 5     1.25  0.013302 Vasicek r0 = 1%
#> 6     1.5   0.013867 Vasicek r0 = 1%
#> 7     1.75  0.014405 Vasicek r0 = 1%
#> 8     2     0.014917 Vasicek r0 = 1%
#> 9     2.25  0.015404 Vasicek r0 = 1%
#> 10    2.5   0.015868 Vasicek r0 = 1%
#> # i 20 more rows

print(short_rate_curve_plot)

```

Vasicek and Hull-White zero-rate curves



Hull-White base curve

17.6 7.6

17.6.1 7.6.1 Hull–White

Hull–White a σ Swaption

1 5 1Y×5Y ATM Swaption Normal volatility 36.06bp $a = 0.03$
 σ

a σ Swaption Calibration Helper

17.6.2 7.6.2 1Y×5Y ATM Swap

```
GaussQuant::set_eval_date_GQL(trade_date)

calibration_calendar <- QuantLib::Japan()
calibration_day_counter <- QuantLib::Actual365Fixed()

swaption_expiry_date <- calibration_calendar$advance(
  trade_date,
  1L,
  "Years"
)

swap_effective_date <- calibration_calendar$advance(
  swaption_expiry_date,
  2L,
  "Days"
)

swap_maturity_date <- calibration_calendar$advance(
  swap_effective_date,
  5L,
  "Years"
)

calibration_schedule <- QuantLib::Schedule(
  swap_effective_date,
  swap_maturity_date,
  GaussQuant::period_months_GQL(6),
  calibration_calendar,
```

```

    "ModifiedFollowing",
    "ModifiedFollowing",
    "Backward",
    FALSE
  )

calibration_index <- QuantLib::JPYLibor(
  GaussQuant::period_months_GQL(6),
  flat_curve_handle
)

calibration_annuity <- GaussQuant::swap_annuity_from_schedule_GQL(
  schedule = calibration_schedule,
  curve = flat_curve,
  day_counter = calibration_day_counter
)

calibration_forward_swap_rate <- GaussQuant::forward_swap_rate_from_schedule_GQL(
  schedule = calibration_schedule,
  curve = flat_curve,
  day_counter = calibration_day_counter
)

calibration_swap_tbl <- tibble::tribble(
  ~metric,          ~value,
  "expiry_date",    GaussQuant::iso_GQL(swaption_expiry_date),
  "swap_effective_date", GaussQuant::iso_GQL(swap_effective_date),
  "swap_maturity_date", GaussQuant::iso_GQL(swap_maturity_date),
  "annuity",        as.character(calibration_annuity),
  "forward_swap_rate", as.character(calibration_forward_swap_rate)
)

GaussQuant::show_tbl_GQH(
  GaussQuant::schedule_table_GQL(
    calibration_schedule
  ),
  "1Yx5Y fixed-leg schedule",
  n = 20
)

#> # A tibble: 11 x 2
#>   period schedule_date
#>   <int> <date>
#> 1      1 1 2025-04-17

```

```

#> 2      2 2025-10-17
#> 3      3 2026-04-17
#> 4      4 2026-10-19
#> 5      5 2027-04-19
#> 6      6 2027-10-18
#> 7      7 2028-04-17
#> 8      8 2028-10-17
#> 9      9 2029-04-17
#> 10     10 2029-10-17
#> 11     11 2030-04-17

GaussQuant::show_tbl_GQH(
  calibration_swap_tbl,
  "1Yx5Y forward swap summary",
  n = 20
)
#> # A tibble: 5 x 2
#>   metric      value
#>   <chr>      <chr>
#> 1 expiry_date 2025-04-15
#> 2 swap_effective_date 2025-04-17
#> 3 swap_maturity_date 2030-04-17
#> 4 annuity      4.47364922408499
#> 5 forward_swap_rate 0.0302262727429665

```

17.6.3 7.6.3 One-helper calibration

```

market_normal_volatility <- 36.06 / 10000
fixed_mean_reversion <- 0.03
initial_sigma <- 0.0001

calibration_model <- GaussQuant::hull_white_model_GQL(
  term_structure = flat_curve_handle,
  a = fixed_mean_reversion,
  sigma = initial_sigma
)

calibration_jamshidian_engine <- QuantLib::JamshidianSwaptionEngine(
  calibration_model
)

```

```

swaption_helper <- QuantLib::SwaptionHelper(
  swaption_expiry_date,
  GaussQuant::period_years_GQL(5),
  QuantLib::QuoteHandle(
    QuantLib::SimpleQuote(
      market_normal_volatility
    )
  ),
  calibration_index,
  GaussQuant::period_months_GQL(6),
  calibration_day_counter,
  calibration_day_counter,
  flat_curve_handle,
  QuantLib::BlackCalibrationHelper_RelativePriceError_get(),
  calibration_forward_swap_rate,
  1,
  QuantLib::VolatilityType_Normal_get()
)

swaption_helper$setPricingEngine(
  calibration_jamshidian_engine
)
#> NULL

calibration_helper_vector <- GaussQuant::push_calibration_helpers_GQL(
  list(swaption_helper)
)

calibration_end_criteria <- QuantLib::EndCriteria(
  10000L,
  100L,
  1e-6,
  1e-8,
  1e-8
)

calibration_model$calibrate(
  calibration_helper_vector,
  QuantLib::LevenbergMarquardt(),
  calibration_end_criteria,
  QuantLib::NoConstraint(),
  numeric(),

```



```

    c(
      TRUE,
      FALSE
    )
  )
#> NULL

calibration_parameter_array <- calibration_model$params()

calibrated_parameters <- c(
  as.numeric(calibration_parameter_array[1]),
  as.numeric(calibration_parameter_array[2])
)

calibration_result_tbl <- tibble::tribble(
  ~parameter,      ~value,
  "a_fixed",        calibrated_parameters[[1]],
  "sigma_calibrated", calibrated_parameters[[2]],
  "market_normal_vol", market_normal_volatility
)

GaussQuant::show_tbl_GQH(
  calibration_result_tbl,
  "Hull-White one-helper calibration",
  n = 20
)
#> # A tibble: 3 x 2
#>   parameter      value
#>   <chr>          <dbl>
#> 1 a_fixed        0.03
#> 2 sigma_calibrated 0.0033441
#> 3 market_normal_vol 0.003606
calibration_parameter_array <- calibration_model$params()

calibration_result_tbl <- tibble::tribble(
  ~parameter,      ~value,
  "a_fixed",        calibrated_parameters[[1]],
  "sigma_calibrated", calibrated_parameters[[2]],
  "market_normal_vol", market_normal_volatility
)

GaussQuant::show_tbl_GQH(

```

```

calibration_result_tbl,
"Hull-White one-helper calibration",
n = 20
)
#> # A tibble: 3 x 2
#>   parameter      value
#>   <chr>         <dbl>
#> 1 a_fixed      0.03
#> 2 sigma_calibrated 0.0033441
#> 3 market_normal_vol 0.003606

```

`c(TRUE, FALSE)` a 2 σ

17.6.4 7.6.4 European Swaption Engine

1Y×5Y ATM Swap European Payer Swaption 3

- Jamshidian decomposition
- Hull-White Tree
- Bachelier analytic engine

Jamshidian Tree Hull-White Bachelier Engine Normal volatility

```

calibration_underlying_swap <- QuantLib::VanillaSwap(
  QuantLib::Swap_Payer_get(),
  1,
  calibration_schedule,
  calibration_forward_swap_rate,
  calibration_day_counter,
  calibration_schedule,
  calibration_index,
  0,
  calibration_day_counter
)

calibration_european_swaption <- QuantLib::Swaption(
  calibration_underlying_swap,
  QuantLib::EuropeanExercise(
    swaption_expiry_date
  )
)

```

```

jamshidian_engine <- QuantLib::JamshidianSwaptionEngine(
  calibration_model
)

calibration_european_swaption$setPricingEngine(
  jamshidian_engine
)
#> NULL

npv_jamshidian <- GaussQuant::instrument_npv_safe_GQL(
  calibration_european_swaption
)

tree_engine_500 <- QuantLib::TreeSwaptionEngine(
  calibration_model,
  500L
)

calibration_european_swaption$setPricingEngine(
  tree_engine_500
)
#> NULL

npv_tree_500 <- GaussQuant::instrument_npv_safe_GQL(
  calibration_european_swaption
)

bachelier_engine <- QuantLib::BachelierSwaptionEngine(
  flat_curve_handle,
  QuantLib::QuoteHandle(
    QuantLib::SimpleQuote(
      market_normal_volatility
    )
  )
)

calibration_european_swaption$setPricingEngine(
  bachelier_engine
)
#> NULL

npv_bachelier <- GaussQuant::instrument_npv_safe_GQL(

```

```

    calibration_european_swaption
  )

engine_comparison_tbl <- tibble::tribble(
  ~engine,          ~npv,
  "Jamshidian",     npv_jamshidian,
  "Hull-White tree 500", npv_tree_500,
  "Bachelier",      npv_bachelier
) |>
  dplyr::mutate(
    difference_vs_bachelier = .data$npv -
      npv_bachelier,
    difference_vs_jamshidian = .data$npv -
      npv_jamshidian
  )

GaussQuant::show_tbl_GQH(
  engine_comparison_tbl,
  "1Yx5Y swaption engine comparison",
  n = 20
)
#> # A tibble: 3 x 4
#>   engine          npv difference_vs_bachelier difference_vs_jamshidian
#>   <chr>          <dbl>                <dbl>                <dbl>
#> 1 Jamshidian    0.0055519                -5.8042e-8                0
#> 2 Hull-White tree 500 0.0055480                -3.9537e-6               -3.8956e-6
#> 3 Bachelier     0.0055519                0                    5.8042e-8

```

ATM Swaption	Hull-White	Bachelier	Jamshidian	Tree	Tree
--------------	------------	-----------	------------	------	------

17.6.5 7.6.5 Tree step

Hull-White Tree	node	Tree step	Payoff
European Swaption	Tree step	Jamshidian	

```

tree_step_grid <- c(
  6L,
  12L,
  24L,
  60L,
  120L,

```

```

240L,
500L
)

tree_convergence_tbl <- purrr::map_dfr(
  tree_step_grid,
  function(tree_step) {
    tree_engine_i <- QuantLib::TreeSwaptionEngine(
      calibration_model,
      tree_step
    )

    calibration_european_swaption$setPricingEngine(
      tree_engine_i
    )

    tree_npv_i <- GaussQuant::instrument_npv_safe_GQL(
      calibration_european_swaption
    )

    tibble::tibble(
      tree_steps = tree_step,
      npv = tree_npv_i,
      difference_vs_jamshidian = tree_npv_i -
        npv_jamshidian,
      difference_vs_bachelier = tree_npv_i -
        npv_bachelier
    )
  }
)

tree_convergence_plot <- ggplot2::ggplot(
  tree_convergence_tbl,
  ggplot2::aes(
    x = .data$tree_steps,
    y = .data$npv
  )
) +
  ggplot2::geom_line(
    linewidth = 0.7
  ) +
  ggplot2::geom_point(

```

```

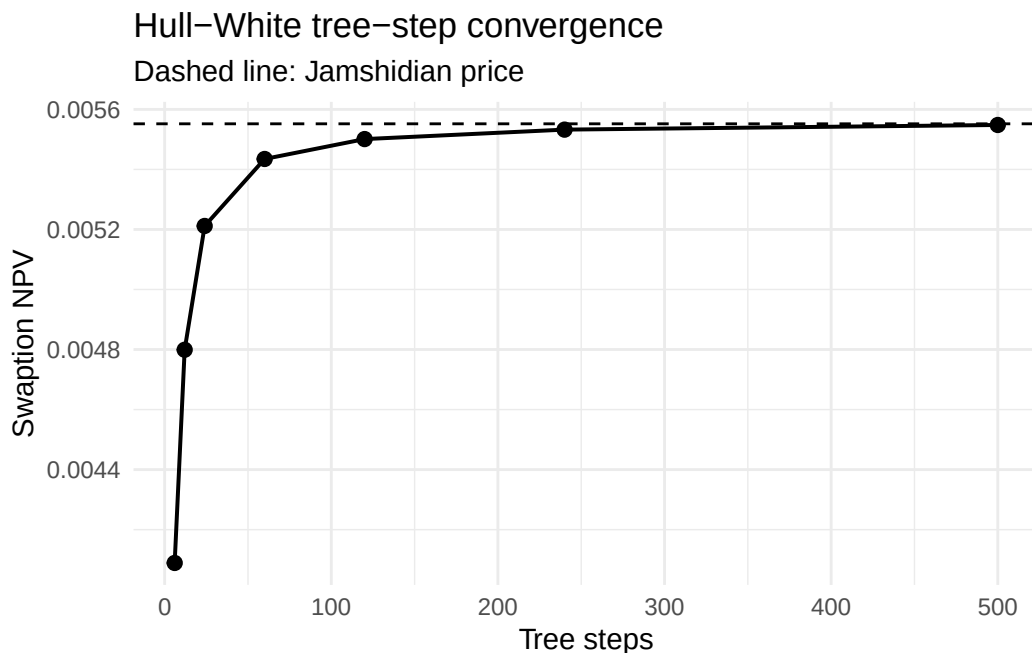
    size = 2.2
  ) +
  ggplot2::geom_hline(
    yintercept = npv_jamshidian,
    linetype = "dashed"
  ) +
  ggplot2::labs(
    title = "Hull-White tree-step convergence",
    subtitle = "Dashed line: Jamshidian price",
    x = "Tree steps",
    y = "Swaption NPV"
  ) +
  ggplot2::theme_minimal()

GaussQuant::show_tbl_GQH(
  tree_convergence_tbl,
  "Hull-White tree convergence",
  n = 20
)

#> # A tibble: 7 x 4
#>   tree_steps      npv difference_vs_jamshidian difference_vs_bachelier
#>   <int>      <dbl>                <dbl>                <dbl>
#> 1         6 0.0040890             -0.0014628             -0.0014629
#> 2        12 0.0047992             -0.00075265            -0.00075270
#> 3        24 0.0052116             -0.00034028            -0.00034034
#> 4        60 0.0054347             -0.00011714            -0.00011720
#> 5       120 0.0055010             -0.000050834           -0.000050892
#> 6       240 0.0055325             -0.000019365           -0.000019423
#> 7       500 0.0055480             -0.0000038956          -0.0000039537

print(tree_convergence_plot)

```



Tree step	Tree	Jamshidian		Bermudan	Swaption	Callable	Bond	Jamshidi-
an	European	Swaption	Tree	Tree				

17.7 7.7

Callable Bond	Bullet Bond		Callable Bond
	Receiver Swaption		Bermudan Receiver Swaption
			Swap
Hull–White	Gaussian		Vasicek
			Hull–White
Hull–White	a	σ	1Y×5Y ATM Swaption
			Normal volatility
			a
			σ
European Swaption	Jamshidian decomposition	Hull–White Tree	Bachelier Engine
step	Tree	Callable Bond	Bermudan Swaption
			Tree

```
chapter07_summary_tbl <- tibble::tribble(
  ~section, ~main_result,
  "Callable Bond", "Bullet bond minus issuer call option",
  "Receiver Swaption", "Interest-rate direction of the issuer call",
  "Vasicek and Hull-White", "Initial-curve fit and mean reversion",
  "Calibration", "Fixed a and calibrated sigma",
  "Tree convergence", "Convergence toward Jamshidian pricing"
)
```

```

GaussQuant::show_tbl_GQH(
  chapter07_summary_tbl,
  "Chapter II-7 summary",
  n = 20
)
#> # A tibble: 5 x 2
#>   section          main_result
#>   <chr>          <chr>
#> 1 Callable Bond      Bullet bond minus issuer call option
#> 2 Receiver Swaption  Interest-rate direction of the issuer call
#> 3 Vasicek and Hull-White Initial-curve fit and mean reversion
#> 4 Calibration        Fixed a and calibrated sigma
#> 5 Tree convergence   Convergence toward Jamshidian pricing

cat(
  "\nChapter II-7 Callable Bond and Hull-White model completed successfully.\n"
)
#>
#> Chapter II-7 Callable Bond and Hull-White model completed successfully.

```


18 II-8 CDS

Credit Default Swap CDS				Premium
CDS				
• Premium Leg • Protection Leg	Spread			
CDS				
		$V_{\text{CDS}} = V_{\text{protection}} + V_{\text{premium}}$		
Premium Leg	Protection Leg			
CDS	Default Probability Curve	Hazard Rate		Premium
Leg Protection Leg				
	Hazard Rate	MidPoint	CDS	CDS2015 ISDA CDS Engine Fair
spread Fair upfront	Accrual rebate			

```
knitr::opts_chunk$set(  
  collapse = TRUE,  
  comment = "#>",  
  warning = FALSE,  
  message = FALSE  
)  
  
suppressPackageStartupMessages({  
  library(tidyverse)  
  library(ggthemes)  
  library(QuantLib)  
  library(GaussQuant)  
  library(tidyverse)  
  library(broom)  
})
```

18.1 8.1 CDS

18.1.1 8.1.1

$\tau \leq t$

$Q(t) = P(\tau > t)$

$0 \leq t$

$P(\tau \leq t) = 1 - Q(t)$

Hazard Rate $\lambda(t)$ t

$$\lambda(t) = \lim_{\Delta t \rightarrow 0} \frac{P(t < \tau \leq t + \Delta t \mid \tau > t)}{\Delta t}$$

Hazard Rate

$Q(t) = \exp(-\lambda t)$

$1 - Q(t) = 1 - \exp(-\lambda t)$

Hazard Rate

Hazard Rate 2% 1

$1 - e^{-0.02}$

2%

18.1.2 8.1.2 Recovery Rate Loss Given Default

Recovery Rate R

Loss Given Default

$$LGD = 1 - R$$

Notional N Protection Leg

$$N(1 - R)$$

Recovery Rate

CDS Spread

18.1.3 8.1.3 CDS Spread Hazard Rate

Premium Hazard Rate Recovery Rate CDS Spread s Hazard Rate λ

$$s \approx (1 - R)\lambda$$

$$\lambda \approx \frac{s}{1 - R}$$

CDS

Accrual on default

Hazard Rate

18.2 8.2 CDS

18.2.1 8.2.1 1 CDS

1 CDS

- 2022 9 19
- 2023 9 20
- Notional 10,000,000
- 10%
- Spread 130bp
- Coupon 100bp
- Recovery Rate 40%

Spread 130bp Coupon 100bp CDS

```
trade_date_flat <- GaussQuant::date_GQL("2022-09-19")
maturity_date_flat <- GaussQuant::date_GQL("2023-09-20")

GaussQuant::set_eval_date_GQL(trade_date_flat)

notional_flat <- 10000000
risk_free_rate_flat <- 0.10
market_spread_flat <- 130 / 10000
contract_coupon_flat <- 100 / 10000
recovery_rate_flat <- 0.40

actual_365 <- QuantLib::Actual365Fixed()
actual_360 <- QuantLib::Actual360()
weekend_calendar <- QuantLib::WeekendsOnly()

flat_cds_input_tbl <- tibble::tribble(
  ~input,      ~value,
  "trade_date", "2022-09-19",
  "maturity_date", "2023-09-20",
  "notional",    as.character(notional_flat),
  "risk_free_rate", as.character(risk_free_rate_flat),
  "market_spread", as.character(market_spread_flat),
  "contract_coupon", as.character(contract_coupon_flat),
  "recovery_rate", as.character(recovery_rate_flat)
)

GaussQuant::show_tbl_GQH(
```

```

flat_cds_input_tbl,
"Flat CDS input assumptions",
n = 20
)
#> # A tibble: 7 x 2
#>   input      value
#>   <chr>      <chr>
#> 1 trade_date 2022-09-19
#> 2 maturity_date 2023-09-20
#> 3 notional    100000000
#> 4 risk_free_rate 0.1
#> 5 market_spread 0.013
#> 6 contract_coupon 0.01
#> 7 recovery_rate 0.4

```

18.2.2 8.2.2 Hazard Rate

Spread 130bp Recovery Rate 40% Hazard Rate

```

hazard_rate_approximation <- market_spread_flat /
  (1 - recovery_rate_flat)

hazard_approximation_tbl <- tibble::tribble(
  ~metric,      ~value,
  "market_spread",      market_spread_flat,
  "recovery_rate",      recovery_rate_flat,
  "loss_given_default", 1 - recovery_rate_flat,
  "hazard_rate_approx",  hazard_rate_approximation
)

probability_time_grid <- seq(
  0,
  5,
  by = 0.25
)

flat_probability_tbl <- tibble::tibble(
  time = probability_time_grid
) |>
  dplyr::mutate(
    survival_probability = exp(

```

```

      -hazard_rate_approximation * .data$time
    ),
    cumulative_default_probability = 1 -
      .data$survival_probability
  )

flat_probability_plot_tbl <- flat_probability_tbl |>
  tidyr::pivot_longer(
    cols = c(
      survival_probability,
      cumulative_default_probability
    ),
    names_to = "probability_type",
    values_to = "probability"
  )

flat_probability_plot <- ggplot2::ggplot(
  flat_probability_plot_tbl,
  ggplot2::aes(
    x = .data$time,
    y = .data$probability,
    linetype = .data$probability_type
  )
) +
  ggplot2::geom_line(
    linewidth = 0.7
  ) +
  ggplot2::scale_y_continuous(
    labels = scales::percent_format(
      accuracy = 0.1
    )
  ) +
  ggplot2::labs(
    title = "Flat-hazard survival and default probabilities",
    x = "Time (years)",
    y = "Probability",
    linetype = "Probability"
  ) +
  ggplot2::theme_minimal()

GaussQuant::show_tbl_GQH(
  hazard_approximation_tbl,

```

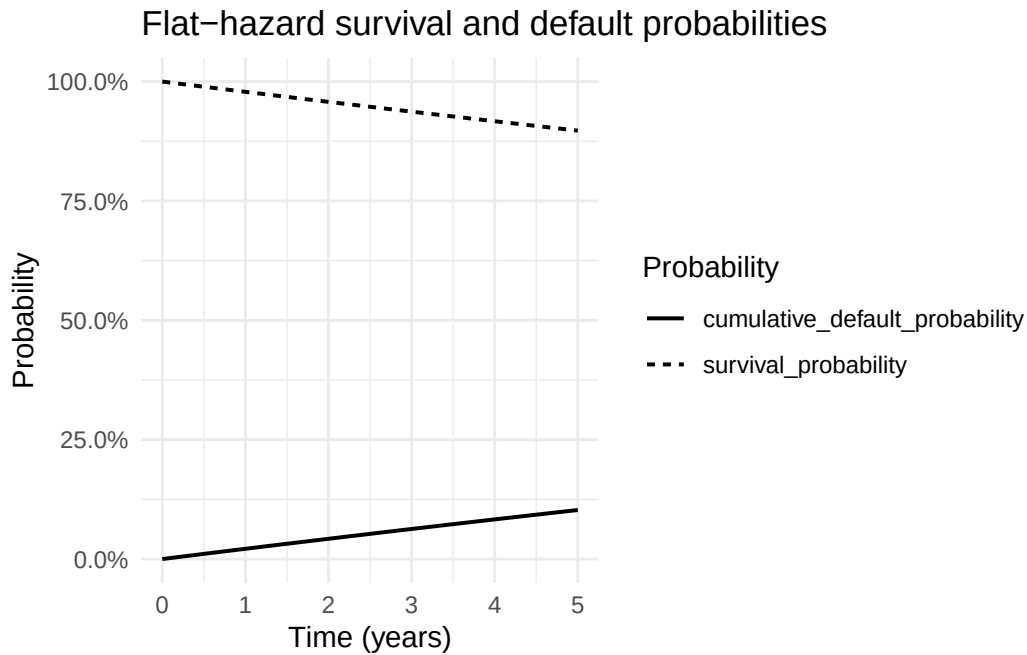
```

"Flat hazard-rate approximation",
  n = 20
)
#> # A tibble: 4 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 market_spread    0.013
#> 2 recovery_rate    0.4
#> 3 loss_given_default 0.6
#> 4 hazard_rate_approx 0.021667

GaussQuant::show_tbl_GQH(
  flat_probability_tbl,
  "Flat-hazard probability curve",
  n = 30
)
#> # A tibble: 21 x 3
#>   time survival_probability cumulative_default_probability
#>   <dbl>          <dbl>          <dbl>
#> 1  0            1            0
#> 2  0.25         0.99460        0.0054020
#> 3  0.5          0.98923        0.010775
#> 4  0.75         0.98388        0.016119
#> 5  1            0.97857        0.021434
#> 6  1.25         0.97328        0.026720
#> 7  1.5          0.96802        0.031978
#> 8  1.75         0.96279        0.037207
#> 9  2            0.95759        0.042408
#> 10 2.25         0.95242        0.047581
#> # i 11 more rows

print(flat_probability_plot)

```



18.2.3 8.2.3 CDS Schedule

Premium

Forward Schedule

```
cds_schedule_flat <- QuantLib::Schedule(
  trade_date_flat + 1L,
  maturity_date_flat,
  GaussQuant::period_months_GQL(3),
  weekend_calendar,
  "Following",
  "Unadjusted",
  "Forward",
  FALSE
)

discount_curve_flat <- GaussQuant::flat_curve_GQL(
  reference_date = trade_date_flat,
  rate = risk_free_rate_flat,
  day_counter = actual_365,
  compounding = "Continuous"
)
```



```

discount_curve_handle_flat <- QuantLib::YieldTermStructureHandle(
  discount_curve_flat
)

hazard_curve_flat <- QuantLib::FlatHazardRate(
  trade_date_flat,
  GaussQuant::quote_handle_GQL(
    hazard_rate_approximation
  ),
  actual_365
)

hazard_curve_handle_flat <- QuantLib::DefaultProbabilityTermStructureHandle(
  hazard_curve_flat
)

flat_curve_check_tbl <- tibble::tribble(
  ~metric, ~value,
  "discount_factor_at_maturity", discount_curve_flat$discount(
    maturity_date_flat
  ),
  "survival_probability_at_maturity", hazard_curve_flat$survivalProbability(
    maturity_date_flat
  ),
  "default_probability_to_maturity", 1 -
    hazard_curve_flat$survivalProbability(
    maturity_date_flat
  ),
  "hazard_rate_at_maturity", hazard_curve_flat$hazardRate(
    maturity_date_flat
  )
)

GaussQuant::show_tbl_GQH(
  GaussQuant::schedule_table_GQL(
    cds_schedule_flat
  ),
  "CDS schedule: flat-curve example",
  n = 20
)

#> # A tibble: 5 x 2
#>   period schedule_date

```

```

#>      <int> <date>
#> 1      1 2022-09-20
#> 2      2 2022-12-20
#> 3      3 2023-03-20
#> 4      4 2023-06-20
#> 5      5 2023-09-20

GaussQuant::show_tbl_GQH(
  flat_curve_check_tbl,
  "Flat discount and hazard-curve check",
  n = 20
)
#> # A tibble: 4 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 discount_factor_at_maturity          0.90459
#> 2 survival_probability_at_maturity 0.97851
#> 3 default_probability_to_maturity 0.021492
#> 4 hazard_rate_at_maturity            0.021667

```

18.2.4 8.2.4 MidPoint CDS Engine

MidPoint Premium

100bp Coupon CDS GaussQuant MidPoint CDS Engine

```

cds_flat <- QuantLib::CreditDefaultSwap(
  GaussQuant::cds_buyer_side_GQL(),
  notional_flat,
  contract_coupon_flat,
  cds_schedule_flat,
  "Following",
  actual_360
)

midpoint_engine_flat <- GaussQuant::cds_midpoint_engine_GQL(
  probability_handle = hazard_curve_handle_flat,
  recovery_rate = recovery_rate_flat,
  discount_handle = discount_curve_handle_flat
)

cds_flat$setPricingEngine(

```

```

midpoint_engine_flat
)
#> NULL

cds_flat_summary_tbl <- GaussQuant::cds_summary_GQL(
  cds_flat
)

cds_flat_leg_check_tbl <- tibble::tribble(
  ~metric,      ~value,
  "coupon_leg_npv",    cds_flat$couponLegNPV(),
  "default_leg_npv",   cds_flat$defaultLegNPV(),
  "sum_of_two_legs",   cds_flat$couponLegNPV() +
                        cds_flat$defaultLegNPV(),
  "reported_npv",     GaussQuant::cds_npv_GQL(
                        cds_flat
                        ),
  "difference",        GaussQuant::cds_npv_GQL(
                        cds_flat
                        ) -
                        (
                          cds_flat$couponLegNPV() +
                          cds_flat$defaultLegNPV()
                        )
)

GaussQuant::show_tbl_GQH(
  cds_flat_summary_tbl,
  "CDS summary: MidPoint engine with flat curves",
  n = 20
)
#> # A tibble: 4 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 npv      28116.
#> 2 fair_spread 0.012983
#> 3 coupon_leg_npv -94246.
#> 4 default_leg_npv 122362.

GaussQuant::show_tbl_GQH(
  cds_flat_leg_check_tbl,
  "CDS two-leg NPV check",

```

```

  n = 20
)
#> # A tibble: 5 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 coupon_leg_npv -94246.
#> 2 default_leg_npv 122362.
#> 3 sum_of_two_legs 28116.
#> 4 reported_npv    28116.
#> 5 difference      0

```

	Premium	Leg	Protection	Leg	Coupon Fair	Spread	CDS
NPV							

18.2.5 8.2.5 CDS

GaussQuant cds_cashflow_table_GQL() Premium

```

cds_cashflow_flat_tbl <- GaussQuant::cds_cashflow_table_GQL(
  cds_schedule = cds_schedule_flat,
  protection_start_date = cds_flat$protectionStartDate(),
  coupon_rate = contract_coupon_flat,
  hazard_curve = hazard_curve_flat,
  discount_curve = discount_curve_flat,
  trade_date = trade_date_flat,
  notional = notional_flat
)

GaussQuant::show_tbl_GQH(
  cds_cashflow_flat_tbl,
  "CDS cash-flow and probability table",
  n = 20
)
#> # A tibble: 5 x 13
#>   payment_date coupon_rate accrual_start accrual_end accrual_days
#>   <chr>          <dbl> <chr>          <chr>          <dbl>
#> 1 2022-09-20      NA    <NA>          2022-09-20      NA
#> 2 2022-12-20      0.01 2022-09-20    2022-12-20      91
#> 3 2023-03-20      0.01 2022-12-20    2023-03-20      90
#> 4 2023-06-20      0.01 2023-03-20    2023-06-20      92
#> 5 2023-09-20      0.01 2023-06-20    2023-09-20      92

```

```

#>   accrual_year_fraction_365 coupon_amount discount_factor survival_probability
#>   <dbl>           <dbl>           <dbl>           <dbl>
#> 1             NA             NA             0.99973           0.99994
#> 2           0.24932         25278.           0.97511           0.99455
#> 3           0.24658         25000           0.95136           0.98925
#> 4           0.25205         25556.           0.92768           0.98387
#> 5           0.25205         25556.           0.90459           0.97851
#>   default_probability midpoint_date midpoint_discount_factor
#>   <dbl> <chr>           <dbl>
#> 1      0      <NA>           NA
#> 2    0.0053869 2022-11-04           0.98748
#> 3    0.0052992 2023-02-03           0.96316
#> 4    0.0053878 2023-05-05           0.93945
#> 5    0.0053584 2023-08-05           0.91606
#>   midpoint_survival_probability
#>   <dbl>
#> 1      NA
#> 2    0.99727
#> 3    0.99190
#> 4    0.98656
#> 5    0.98118

```

18.3 8.3 Premium Leg Protection Leg

18.3.1 8.3.1 Premium Leg

Premium Leg

$$V_{\text{premium}} \approx -Ns \sum_i \alpha_i D(0, t_i) Q(t_i)$$

N Notional s Coupon α_i $D(0, t_i)$ $Q(t_i)$

```

premium_leg_hand_flat <- -sum(
  cds_cashflow_flat_tbl$coupon_amount *
  cds_cashflow_flat_tbl$discount_factor *
  cds_cashflow_flat_tbl$midpoint_survival_probability,
  na.rm = TRUE
)

```

CDS Premium Accrual on default

18.3.2 8.3.2 Protection Leg

$(t_{i-1}, t_i]$

$$\Delta PD_i = Q(t_{i-1}) - Q(t_i)$$

Protection Leg m_i

$$V_{\text{protection}} \approx N(1 - R) \sum_i D(0, m_i) \Delta PD_i$$

```
protection_leg_per_unit_hand_flat <- (
  1 - recovery_rate_flat
) *
sum(
  cds_cashflow_flat_tbl$midpoint_discount_factor *
  cds_cashflow_flat_tbl$default_probability,
  na.rm = TRUE
)

protection_leg_hand_flat <- notional_flat *
  protection_leg_per_unit_hand_flat
```

18.3.3 8.3.3 Risky PV01 Fair Spread

Risky PV01 CDS Spread 1 Premium Leg

$$RPV01 \approx \sum_i \alpha_i D(0, t_i) Q(t_i)$$

Fair Spread Premium Leg Protection Leg Upfront CDS NPV Running Spread

$$s_{\text{fair}} = \frac{V_{\text{protection}}/N}{RPV01}$$

```

risky_pv01_hand_flat <- sum(
  (
    cds_cashflow_flat_tbl$accrual_days /
    360
  ) *
  cds_cashflow_flat_tbl$discount_factor *
  cds_cashflow_flat_tbl$midpoint_survival_probability,
  na.rm = TRUE
)

fair_spread_hand_flat <- protection_leg_per_unit_hand_flat /
  risky_pv01_hand_flat

cds_npv_hand_flat <- premium_leg_hand_flat +
  protection_leg_hand_flat

flat_hand_check_tbl <- tibble::tribble(
  ~metric,          ~hand_value,          ~quantlib_value,
  "coupon_leg_npv", premium_leg_hand_flat,    cds_flat$couponLegNPV(),
  "default_leg_npv", protection_leg_hand_flat, cds_flat$defaultLegNPV(),
  "cds_npv",        cds_npv_hand_flat,    GaussQuant::cds_npv_GQL(
    cds_flat
  ),
  "fair_spread",    fair_spread_hand_flat, GaussQuant::cds_fair_spread_GQL(
    cds_flat
  )
) |>
dplyr::mutate(
  difference = .data$hand_value -
    .data$quantlib_value
)

flat_hand_summary_tbl <- tibble::tribble(
  ~metric,          ~value,
  "risky_pv01",     risky_pv01_hand_flat,
  "protection_leg_per_unit", protection_leg_per_unit_hand_flat,
  "hand_fair_spread", fair_spread_hand_flat,
  "quantlib_fair_spread", GaussQuant::cds_fair_spread_GQL(
    cds_flat
  ),
  "fair_spread_difference", fair_spread_hand_flat -
    GaussQuant::cds_fair_spread_GQL(

```

```

                                cds_flat
                                )
)

GaussQuant::show_tbl_GQH(
  flat_hand_check_tbl,
  "CDS hand calculation and MidPoint engine comparison",
  n = 20
)
#> # A tibble: 4 x 4
#>   metric          hand_value quantlib_value    difference
#>   <chr>          <dbl>          <dbl>          <dbl>
#> 1 coupon_leg_npv -94244.         -94246.         2.0964
#> 2 default_leg_npv 122362.         122362.          0
#> 3 cds_npv         28118.         28116.         2.0964
#> 4 fair_spread      0.012984         0.012983 0.00000028881

GaussQuant::show_tbl_GQH(
  flat_hand_summary_tbl,
  "CDS hand-calculation summary",
  n = 20
)
#> # A tibble: 5 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 risky_pv01      0.94244
#> 2 protection_leg_per_unit 0.012236
#> 3 hand_fair_spread  0.012984
#> 4 quantlib_fair_spread 0.012983
#> 5 fair_spread_difference 0.00000028881

```

MidPoint Engine Accrual on default Protection QuantLib

18.4 8.4 Hazard Rate

18.4.1 8.4.1 Zero-NPV CDS Hazard Rate

Spread Coupon CDS NPV Hazard Rate

Spread 130bp CDS MidPoint Engine NPV Hazard Rate `uniroot()`


```

cds_market_flat <- QuantLib::CreditDefaultSwap(
  GaussQuant::cds_buyer_side_GQL(),
  notional_flat,
  market_spread_flat,
  cds_schedule_flat,
  "Following",
  actual_360
)

cds_market_npv_at_hazard <- function(hazard_rate) {
  engine <- GaussQuant::cds_midpoint_engine_GQL(
    recovery_rate = recovery_rate_flat,
    hazard_rate = hazard_rate,
    discount_rate = risk_free_rate_flat,
    reference_date = trade_date_flat,
    day_counter = "Actual365Fixed"
  )

  cds_market_flat$setPricingEngine(
    engine
  )

  GaussQuant::cds_npv_GQL(
    cds_market_flat
  )
}

hazard_calibration_result <- stats::uniroot(
  cds_market_npv_at_hazard,
  interval = c(
    1e-8,
    0.50
  ),
  tol = 1e-10
)

hazard_rate_calibrated <- hazard_calibration_result$root

calibrated_midpoint_engine <- GaussQuant::cds_midpoint_engine_GQL(
  recovery_rate = recovery_rate_flat,
  hazard_rate = hazard_rate_calibrated,
  discount_rate = risk_free_rate_flat,

```

```

    reference_date = trade_date_flat,
    day_counter = "Actual365Fixed"
)

cds_market_flat$setPricingEngine(
  calibrated_midpoint_engine
)
#> NULL

hazard_calibration_tbl <- tibble::tribble(
  ~metric,          ~value,
  "market_spread",  market_spread_flat,
  "simple_hazard_approximation", hazard_rate_approximation,
  "calibrated_hazard_rate",    hazard_rate_calibrated,
  "zero_npv_check",           GaussQuant::cds_npv_GQL(
                                cds_market_flat
                              ),
  "fair_spread_check",        GaussQuant::cds_fair_spread_GQL(
                                cds_market_flat
                              )
)

GaussQuant::show_tbl_GQH(
  hazard_calibration_tbl,
  "Flat hazard calibration from a zero-NPV CDS",
  n = 20
)
#> # A tibble: 5 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 market_spread                        0.013
#> 2 simple_hazard_approximation 0.021667
#> 3 calibrated_hazard_rate        0.021695
#> 4 zero_npv_check                0.00000036661
#> 5 fair_spread_check             0.013000

```

$$\lambda \approx s/(1 - R)$$

18.4.2 8.4.2 Spread Recovery Rate

Spread Hazard Rate Spread Recovery Rate Spread Haz-

ard Rate

```
credit_sensitivity_input_tbl <- tidyr::crossing(
  market_spread_bp = c(
    50,
    100,
    200
  ),
  recovery_rate = c(
    0.20,
    0.40,
    0.60
  )
)

credit_sensitivity_tbl <- credit_sensitivity_input_tbl |>
  dplyr::mutate(
    market_spread = .data$market_spread_bp /
      10000,
    loss_given_default = 1 -
      .data$recovery_rate,
    hazard_rate_approx = .data$market_spread /
      .data$loss_given_default,
    survival_probability_5y = exp(
      -5 * .data$hazard_rate_approx
    ),
    cumulative_default_probability_5y = 1 -
      .data$survival_probability_5y
  )

GaussQuant::show_tbl_GQH(
  credit_sensitivity_tbl,
  "Spread and recovery-rate sensitivity",
  n = 30
)

#> # A tibble: 9 x 7
#>   market_spread_bp recovery_rate market_spread loss_given_default
#>   <dbl>         <dbl>         <dbl>         <dbl>
#> 1           50         0.2         0.005         0.8
#> 2           50         0.4         0.005         0.6
```

```

#> 3      50      0.6      0.005      0.4
#> 4     100      0.2      0.01      0.8
#> 5     100      0.4      0.01      0.6
#> 6     100      0.6      0.01      0.4
#> 7     200      0.2      0.02      0.8
#> 8     200      0.4      0.02      0.6
#> 9     200      0.6      0.02      0.4
#> hazard_rate_approx survival_probability_5y cumulative_default_probability_5y
#>      <dbl>      <dbl>      <dbl>
#> 1      0.00625      0.96923      0.030767
#> 2      0.0083333      0.95919      0.040811
#> 3      0.0125      0.93941      0.060587
#> 4      0.0125      0.93941      0.060587
#> 5      0.016667      0.92004      0.079956
#> 6      0.025      0.88250      0.11750
#> 7      0.025      0.88250      0.11750
#> 8      0.033333      0.84648      0.15352
#> 9      0.05      0.77880      0.22120

```

Recovery Rate Hazard Rate CDS Spread
bidity Curve

Recovery Rate

Default Proba-

18.5 8.5 CDS2015 ISDA

18.5.1 8.5.1 CDS CDS2015

CDS 5 CDS IMM CDS2015

5 CDS

- 2022 8 19
- Spread 30.426bp
- Coupon 100bp
- Notional 10,000,000
- Recovery Rate 40%
- OIS 0.25%

IL-3 OIS

CDS

OIS

```

trade_date_isda <- GaussQuant::date_GQL(
  "2022-08-19"
)

GaussQuant::set_eval_date_GQL(
  trade_date_isda
)

cds_tenor_isda <- GaussQuant::period_years_GQL(
  5L
)

maturity_date_isda <- QuantLib::cdsMaturity(
  trade_date_isda,
  cds_tenor_isda,
  "CDS2015"
)

market_spread_isda <- 30.426 / 10000
contract_coupon_isda <- 100 / 10000
notional_isda <- 10000000
recovery_rate_isda <- 0.40
ois_discount_rate_isda <- 0.0025

isda_cds_input_tbl <- tibble::tribble(
  ~input,      ~value,
  "trade_date", "2022-08-19",
  "cds_tenor",  "5Y",
  "maturity_date", GaussQuant::iso_GQL(
    maturity_date_isda
  ),
  "market_spread_bp", as.character(
    market_spread_isda *
    10000
  ),
  "contract_coupon_bp", as.character(
    contract_coupon_isda *
    10000
  ),
  "notional", as.character(
    notional_isda
  ),

```

```

"recovery_rate",      as.character(
                        recovery_rate_isda
                      ),
"ois_discount_rate",  as.character(
                        ois_discount_rate_isda
                      )
)

GaussQuant::show_tbl_GQH(
  isda_cds_input_tbl,
  "ISDA CDS input assumptions",
  n = 20
)
#> # A tibble: 8 x 2
#>   input      value
#>   <chr>    <chr>
#> 1 trade_date 2022-08-19
#> 2 cds_tenor  5Y
#> 3 maturity_date 2027-06-20
#> 4 market_spread_bp 30.426
#> 5 contract_coupon_bp 100
#> 6 notional 100000000
#> 7 recovery_rate 0.4
#> 8 ois_discount_rate 0.0025

```

18.5.2 8.5.2 CDS2015 Schedule

CDS2015 Premium

```

cds_calendar_isda <- QuantLib::WeekendsOnly()

cds_schedule_isda <- QuantLib::Schedule(
  trade_date_isda,
  maturity_date_isda,
  GaussQuant::period_months_GQL(3),
  cds_calendar_isda,
  "Following",
  "Unadjusted",
  "CDS2015",
  FALSE
)

```

```

GaussQuant::show_tbl_GQH(
  GaussQuant::schedule_table_GQL(
    cds_schedule_isda
  ),
  "CDS schedule: 5Y CDS2015 example",
  n = 30
)
#> # A tibble: 21 x 2
#>   period schedule_date
#>   <int> <date>
#> 1     1 2022-06-20
#> 2     2 2022-09-20
#> 3     3 2022-12-20
#> 4     4 2023-03-20
#> 5     5 2023-06-20
#> 6     6 2023-09-20
#> 7     7 2023-12-20
#> 8     8 2024-03-20
#> 9     9 2024-06-20
#> 10    10 2024-09-20
#> # i 11 more rows

```

18.5.3 8.5.3 ISDA

```

discount_curve_isda <- GaussQuant::flat_curve_GQL(
  reference_date = trade_date_isda,
  rate = ois_discount_rate_isda,
  day_counter = actual_365,
  compounding = "Continuous"
)

discount_curve_handle_isda <- QuantLib::YieldTermStructureHandle(
  discount_curve_isda
)

isda_discount_check_tbl <- tibble::tribble(
  ~metric, ~value,
  "ois_discount_rate", ois_discount_rate_isda,
  "discount_factor_at_maturity", discount_curve_isda$discount(
    maturity_date_isda
  )
)

```

```

    )

GaussQuant::show_tbl_GQH(
  isda_discount_check_tbl,
  "ISDA discount-curve check",
  n = 20
)
#> # A tibble: 2 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 ois_discount_rate                    0.0025
#> 2 discount_factor_at_maturity 0.98798

```

18.5.4 8.5.4 ISDA Hazard Rate

Spread 30.426bp Running Spread CDS ISDA Engine NPV Hazard Rate

```

market_cds_isda <- GaussQuant::make_cds_GQL(
  maturity_date = maturity_date_isda,
  running_spread = market_spread_isda,
  notional = notional_isda,
  side = "buyer",
  trade_date = trade_date_isda,
  coupon_tenor = "3M",
  day_counter = "Actual360",
  date_generation_rule = "CDS2015"
)

isda_market_npv_at_hazard <- function(hazard_rate) {
  probability_handle <- GaussQuant::flat_hazard_rate_GQL(
    hazard_rate = hazard_rate,
    reference_date = trade_date_isda,
    day_counter = "Actual365Fixed"
  )

  engine <- GaussQuant::cds_isda_engine_GQL(
    hazard_curve_handle = probability_handle,
    recovery_rate = recovery_rate_isda,
    discount_curve_handle = discount_curve_handle_isda
  )
}

```



```

market_cds_isda$setPricingEngine(
  engine
)

GaussQuant::cds_npv_GQL(
  market_cds_isda
)
}

isda_hazard_calibration_result <- stats::uniroot(
  isda_market_npv_at_hazard,
  interval = c(
    1e-8,
    0.50
  ),
  tol = 1e-10
)

hazard_rate_isda <- isda_hazard_calibration_result$root

hazard_curve_isda <- QuantLib::FlatHazardRate(
  trade_date_isda,
  GaussQuant::quote_handle_GQL(
    hazard_rate_isda
  ),
  actual_365
)

hazard_curve_handle_isda <- QuantLib::DefaultProbabilityTermStructureHandle(
  hazard_curve_isda
)

isda_hazard_check_tbl <- tibble::tribble(
  ~metric, ~value,
  "market_spread", market_spread_isda,
  "recovery_rate", recovery_rate_isda,
  "implied_hazard_rate", hazard_rate_isda,
  "simple_hazard_approximation", market_spread_isda /
    (
      1 -
      recovery_rate_isda
    ),

```

```

"survival_probability_at_maturity",
hazard_curve_isda$survivalProbability(
  maturity_date_isda
),
"default_probability_to_maturity",
1 -
  hazard_curve_isda$survivalProbability(
    maturity_date_isda
  )
)

GaussQuant::show_tbl_GQH(
  isda_hazard_check_tbl,
  "ISDA implied hazard-rate check",
  n = 20
)

#> # A tibble: 6 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 market_spread                        0.0030426
#> 2 recovery_rate                        0.4
#> 3 implied_hazard_rate                  0.0051398
#> 4 simple_hazard_approximation          0.005071
#> 5 survival_probability_at_maturity    0.97544
#> 6 default_probability_to_maturity    0.024562

```

Spread	Recovery	Rate	ISDA Engine	NPV	Hazard Rate	CDS	Accrual on default
--------	----------	------	-------------	-----	-------------	-----	--------------------

18.6 8.6 ISDA Engine

18.6.1 8.6.1 ISDA CDS Engine

Coupon	Coupon 100bp	5 CDS			
Spread 30.426bp	Coupon 100bp		Premium	Running Coupon	N-PV

```

isda_engine <- GaussQuant::cds_isda_engine_GQL(
  hazard_curve_handle = hazard_curve_handle_isda,
  recovery_rate = recovery_rate_isda,

```

```

    discount_curve_handle = discount_curve_handle_isda
  )

  cds_isda <- GaussQuant::make_cds_GQL(
    maturity_date = maturity_date_isda,
    running_spread = contract_coupon_isda,
    notional = notional_isda,
    side = "buyer",
    trade_date = trade_date_isda,
    coupon_tenor = "3M",
    day_counter = "Actual360",
    date_generation_rule = "CDS2015",
    pricing_engine = isda_engine
  )

  cds_isda_summary_base_tbl <- GaussQuant::cds_summary_GQL(
    cds_isda
  )

  GaussQuant::show_tbl_GQH(
    cds_isda_summary_base_tbl,
    "ISDA CDS base summary",
    n = 20
  )
#> # A tibble: 4 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 npv      -3.3496e+5
#> 2 fair_spread  3.0426e-3
#> 3 coupon_leg_npv -4.9839e+5
#> 4 default_leg_npv  1.4649e+5

```

18.6.2 8.6.2 Accrual rebate NPV

CDS	Coupon	Coupon	Accrual rebate
	NPV		

$$V_{\text{CDS}} = V_{\text{coupon}} + V_{\text{default}} + V_{\text{accrual rebate}}$$

```

coupon_leg_npv_isda <- cds_isda$couponLegNPV()
default_leg_npv_isda <- cds_isda$defaultLegNPV()

accrual_rebate_npv_isda <- tryCatch(
  as.numeric(
    cds_isda$accrualRebateNPV()
  ),
  error = function(e) NA_real_
)

three_component_sum_isda <- coupon_leg_npv_isda +
  default_leg_npv_isda +
  accrual_rebate_npv_isda

isda_leg_check_tbl <- tibble::tribble(
  ~metric,          ~value,
  "coupon_leg_npv", coupon_leg_npv_isda,
  "default_leg_npv", default_leg_npv_isda,
  "accrual_rebate_npv", accrual_rebate_npv_isda,
  "three_component_sum", three_component_sum_isda,
  "reported_npv",     GaussQuant::cds_npv_GQL(
    cds_isda
  ),
  "difference",       GaussQuant::cds_npv_GQL(
    cds_isda
  ) -
    three_component_sum_isda
)

GaussQuant::show_tbl_GQH(
  isda_leg_check_tbl,
  "ISDA CDS NPV decomposition",
  n = 20
)

#> # A tibble: 6 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 coupon_leg_npv -498392.
#> 2 default_leg_npv 146485.
#> 3 accrual_rebate_npv 16944.
#> 4 three_component_sum -334963.
#> 5 reported_npv -334963.

```

```
#> 6 difference 0
```

18.6.3 8.6.3 Fair Spread Fair Upfront

Fair Spread	Upfront	CDS NPV	Running Spread
Fair Upfront	Coupon	Coupon	Notional

```
fair_spread_isda <- GaussQuant::cds_fair_spread_GQL(
  cds_isda
)

fair_upfront_isda <- tryCatch(
  as.numeric(
    cds_isda$fairUpfront()
  ),
  error = function(e) NA_real_
)

fair_upfront_amount_isda <- fair_upfront_isda *
  notional_isda

isda_cds_summary_tbl <- tibble::tribble(
  ~metric,          ~value,
  "coupon_leg_npv", coupon_leg_npv_isda,
  "default_leg_npv", default_leg_npv_isda,
  "accrual_rebate_npv", accrual_rebate_npv_isda,
  "cds_npv",         GaussQuant::cds_npv_GQL(
    cds_isda
  ),
  "market_spread",  market_spread_isda,
  "contract_coupon", contract_coupon_isda,
  "fair_spread",    fair_spread_isda,
  "fair_upfront",   fair_upfront_isda,
  "fair_upfront_amount", fair_upfront_amount_isda,
  "hazard_rate",    hazard_rate_isda
)

GaussQuant::show_tbl_GQH(
  isda_cds_summary_tbl,
  "CDS summary: ISDA engine",
  n = 30
)
```

```
)
#> # A tibble: 10 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 coupon_leg_npv -4.9839e+5
#> 2 default_leg_npv 1.4649e+5
#> 3 accrual_rebate_npv 1.6944e+4
#> 4 cds_npv -3.3496e+5
#> 5 market_spread 3.0426e-3
#> 6 contract_coupon 1 e-2
#> 7 fair_spread 3.0426e-3
#> 8 fair_upfront -3.3497e-2
#> 9 fair_upfront_amount -3.3497e+5
#> 10 hazard_rate 5.1398e-3
```

	Coupon	Spread		Premium		Upfront
Fair	Upfront	QuantLib	Buyer	Seller		CDS NPV Coupon Fair Spread-
	Upfront					Upfront

18.6.4 8.6.4 ISDA

CDS2015 Schedule ISDA Hazard Curve Premium

```
cds_cashflow_isda_tbl <- GaussQuant::cds_cashflow_table_GQL(
  cds_schedule = cds_schedule_isda,
  protection_start_date = cds_isda$protectionStartDate(),
  coupon_rate = contract_coupon_isda,
  hazard_curve = hazard_curve_isda,
  discount_curve = discount_curve_isda,
  trade_date = trade_date_isda,
  notional = notional_isda
)

GaussQuant::show_tbl_GQH(
  cds_cashflow_isda_tbl,
  "CDS cash-flow table: CDS2015 example",
  n = 30
)

#> # A tibble: 21 x 13
#>   payment_date coupon_rate accrual_start accrual_end accrual_days
```

```

#>      <chr>                <dbl> <chr>                <chr>                <dbl>
#> 1 2022-08-19              NA      <NA>                2022-08-19              NA
#> 2 2022-09-20              0.01 2022-06-20          2022-09-20              92
#> 3 2022-12-20              0.01 2022-09-20          2022-12-20              91
#> 4 2023-03-20              0.01 2022-12-20          2023-03-20              90
#> 5 2023-06-20              0.01 2023-03-20          2023-06-20              92
#> 6 2023-09-20              0.01 2023-06-20          2023-09-20              92
#> 7 2023-12-20              0.01 2023-09-20          2023-12-20              91
#> 8 2024-03-20              0.01 2023-12-20          2024-03-20              91
#> 9 2024-06-20              0.01 2024-03-20          2024-06-20              92
#> 10 2024-09-20             0.01 2024-06-20          2024-09-20              92
#>      accrual_year_fraction_365 coupon_amount discount_factor survival_probability
#>      <dbl>                <dbl>                <dbl>                <dbl>
#> 1              NA              NA              1              1
#> 2              0.25205          25556.          0.99978          0.99955
#> 3              0.24932          25278.          0.99916          0.99827
#> 4              0.24658          25000          0.99854          0.99701
#> 5              0.25205          25556.          0.99791          0.99571
#> 6              0.25205          25556.          0.99728          0.99443
#> 7              0.24932          25278.          0.99666          0.99315
#> 8              0.24932          25278.          0.99604          0.99188
#> 9              0.25205          25556.          0.99541          0.99060
#> 10             0.25205          25556.          0.99479          0.98931
#>      default_probability midpoint_date midpoint_discount_factor
#>      <dbl> <chr>                <dbl>
#> 1              0      <NA>                NA
#> 2      0.00045051 2022-09-04          0.99989
#> 3      0.0012800 2022-11-04          0.99947
#> 4      0.0012644 2023-02-03          0.99885
#> 5      0.0012908 2023-05-05          0.99823
#> 6      0.0012891 2023-08-05          0.99760
#> 7      0.0012735 2023-11-04          0.99698
#> 8      0.0012718 2024-02-03          0.99636
#> 9      0.0012842 2024-05-05          0.99573
#> 10     0.0012825 2024-08-05          0.99510
#>      midpoint_survival_probability
#>      <dbl>
#> 1              NA
#> 2              0.99977
#> 3              0.99892
#> 4              0.99764
#> 5              0.99636

```

```
#> 6 0.99507
#> 7 0.99380
#> 8 0.99252
#> 9 0.99124
#> 10 0.98995
#> # i 11 more rows
```

CDS MidPoint ISDA Engine Accrual on default

18.6.5 8.6.5 MidPoint ISDA Engine

ISDA Premium Leg Protection Leg RPV01 Fair Spread

```
premium_leg_hand_isda <- -sum(
  cds_cashflow_isda_tbl$coupon_amount *
  cds_cashflow_isda_tbl$discount_factor *
  cds_cashflow_isda_tbl$midpoint_survival_probability,
  na.rm = TRUE
)

protection_leg_per_unit_hand_isda <- (
  1 -
  recovery_rate_isda
) *
sum(
  cds_cashflow_isda_tbl$midpoint_discount_factor *
  cds_cashflow_isda_tbl$default_probability,
  na.rm = TRUE
)

protection_leg_hand_isda <- notional_isda *
  protection_leg_per_unit_hand_isda

risky_pv01_hand_isda <- sum(
  (
    cds_cashflow_isda_tbl$accrual_days /
    360
  ) *
  cds_cashflow_isda_tbl$discount_factor *
  cds_cashflow_isda_tbl$midpoint_survival_probability,
  na.rm = TRUE
)
```



```

)

fair_spread_hand_isda <- protection_leg_per_unit_hand_isda /
  risky_pv01_hand_isda

cds_npv_hand_isda <- premium_leg_hand_isda +
  protection_leg_hand_isda +
  accrual_rebate_npv_isda

isda_hand_check_tbl <- tibble::tribble(
  ~metric,          ~hand_value,          ~isda_value,
  "coupon_leg_npv", premium_leg_hand_isda,    coupon_leg_npv_isda,
  "default_leg_npv", protection_leg_hand_isda, default_leg_npv_isda,
  "cds_npv",        cds_npv_hand_isda,    GaussQuant::cds_npv_GQL(
                                     cds_isda
                                     ),
  "fair_spread",    fair_spread_hand_isda, fair_spread_isda
) |>
dplyr::mutate(
  difference = .data$hand_value -
    .data$isda_value
)

GaussQuant::show_tbl_GQH(
  isda_hand_check_tbl,
  "MidPoint-style approximation and ISDA comparison",
  n = 20
)

#> # A tibble: 4 x 4
#>   metric          hand_value isda_value  difference
#>   <chr>          <dbl>      <dbl>      <dbl>
#> 1 coupon_leg_npv -4.9811e+5 -4.9839e+5 280.16
#> 2 default_leg_npv 1.4649e+5 1.4649e+5 0.19448
#> 3 cds_npv        -3.3468e+5 -3.3496e+5 280.35
#> 4 fair_spread     2.9408e-3 3.0426e-3 -0.00010178

```

18.6.6 8.6.6 MidPoint Engine ISDA Engine

Hazard Curve Recovery Rate Coupon MidPoint Engine ISDA Engine

```

midpoint_engine_isda_inputs <- GaussQuant::cds_midpoint_engine_GQL(
  probability_handle = hazard_curve_handle_isda,
  recovery_rate = recovery_rate_isda,
  discount_handle = discount_curve_handle_isda
)

cds_midpoint_standard <- GaussQuant::make_cds_GQL(
  maturity_date = maturity_date_isda,
  running_spread = contract_coupon_isda,
  notional = notional_isda,
  side = "buyer",
  trade_date = trade_date_isda,
  coupon_tenor = "3M",
  day_counter = "Actual360",
  date_generation_rule = "CDS2015",
  pricing_engine = midpoint_engine_isda_inputs
)

midpoint_standard_summary_tbl <- GaussQuant::cds_summary_GQL(
  cds_midpoint_standard
) |>
  dplyr::mutate(
    engine = "MidPoint"
  )

isda_standard_summary_tbl <- GaussQuant::cds_summary_GQL(
  cds_isda
) |>
  dplyr::mutate(
    engine = "ISDA"
  )

engine_comparison_tbl <- dplyr::bind_rows(
  midpoint_standard_summary_tbl,
  isda_standard_summary_tbl
) |>
  dplyr::relocate(
    .data$engine
  ) |>
  tidyr::pivot_wider(
    names_from = engine,
    values_from = value
  )

```

```
) |>
dplyr::mutate(
  difference_isda_minus_midpoint = .data$ISDA -
    .data$MidPoint
)

GaussQuant::show_tbl_GQH(
  engine_comparison_tbl,
  "MidPoint and ISDA CDS engine comparison",
  n = 20
)

#> # A tibble: 4 x 4
#>   metric      MidPoint      ISDA difference_isda_minus_midpoint
#>   <chr>          <dbl>      <dbl>                  <dbl>
#> 1 npv          -3.3495e+5 -3.3496e+5          -1.1669e+1
#> 2 fair_spread    3.0427e-3  3.0426e-3          -7.6555e-8
#> 3 coupon_leg_npv -4.9838e+5 -4.9839e+5          -1.1474e+1
#> 4 default_leg_npv 1.4649e+5  1.4649e+5          -1.9448e-1
```

MidPoint Premium ISDA Engine Leg NPV Fair
Spread

18.7 8.7

CDS Premium Leg Protection Leg Premium
Hazard Rate Hazard Rate

$Q(t) = e^{-\lambda t}$

Recovery Rate R CDS Spread Hazard Rate

$\lambda \approx \frac{s}{1 - R}$

 Hazard Rate Premium Accrual on default CDS
Premium Leg Spread Risky PV01 Fair Spread Protection Leg Notional Risky
PV01
MidPoint Engine CDS ISDA Engine CDS2015 Accrual rebate Accrual
on default

Coupon Spread Running Coupon

Fair Upfront

```
chapter08_summary_tbl <- tibble::tribble(
  ~section,                ~main_result,
  "Hazard Rate",           "Survival and cumulative default probabilities",
  "MidPoint CDS",          "Premium and protection leg decomposition",
  "Hazard calibration",    "Zero-NPV market CDS",
  "CDS2015",               "Standard maturity and coupon schedule",
  "ISDA CDS",              "Fair spread, upfront, and accrual rebate",
  "Engine comparison",     "MidPoint approximation versus ISDA pricing"
)

GaussQuant::show_tbl_GQH(
  chapter08_summary_tbl,
  "Chapter II-8 summary",
  n = 20
)

#> # A tibble: 6 x 2
#>   section                main_result
#>   <chr>                  <chr>
#> 1 Hazard Rate           Survival and cumulative default probabilities
#> 2 MidPoint CDS          Premium and protection leg decomposition
#> 3 Hazard calibration    Zero-NPV market CDS
#> 4 CDS2015               Standard maturity and coupon schedule
#> 5 ISDA CDS              Fair spread, upfront, and accrual rebate
#> 6 Engine comparison     MidPoint approximation versus ISDA pricing

cat(
  "\nChapter II-8 Credit risk and CDS completed successfully.\n"
)

#>
#> Chapter II-8 Credit risk and CDS completed successfully.
```

19

II-9

PRDC

C

Swap Option Callable Bond CDS

II

PRD-

PRDC Power Reverse Dual Currency Note

Coupon

Coupon rate

$$C(S_t) = \min \left[C_{\max}, \max \left(a \frac{S_t}{S_0} - b, C_{\min} \right) \right]$$

- S_t Coupon
- S_0 Coupon
- a
- b
- $C_{\min}$ Coupon Floor
- $C_{\max}$ Coupon Cap

Coupon rate

Floor Cap

PRDC Coupon FX Spot

Domestic Leg

Foreign Redemption Leg

FX volatility

Garman–Kohlhagen

Coupon

FX

Coupon

Coupon

PRDC

Squares Monte Carlo

Callable PRDC

Callable PRDC

Least-

19.1 9.1 PRDC

19.1.1 9.1.1

2015 9 3	2041 9 3	Coupon	PRDC	1	Coupon	FX Coupon
2023 4 11		Intro Coupon		GaussQuant	PRDC	Intro Coupon
		Monte Carlo		3,000	1,000	

```
valuation_date_prdc <- as.Date("2023-04-11")
effective_date_prdc <- as.Date("2015-09-03")
termination_date_prdc <- as.Date("2041-09-03")
intro_coupon_end_date_prdc <- as.Date("2016-09-03")

notional_prdc <- 300000000
intro_coupon_rate_prdc <- 0.022
coupon_frequency_months_prdc <- 6L

domestic_rate_prdc <- 0.01
foreign_rate_prdc <- 0.03
discount_rate_prdc <- 0.005

fx_spot_prdc <- 133.2681
fx_volatility_prdc <- 0.10

coupon_fx_base_prdc <- 120
coupon_leverage_prdc <- 0.122
coupon_subtraction_prdc <- 0.10
coupon_floor_prdc <- 0
coupon_cap_prdc <- 0.022

number_of_paths_prdc <- 3000L
sensitivity_paths_prdc <- 1000L
random_seed_prdc <- 123L

GaussQuant::set_eval_date_GQL(
  valuation_date_prdc
)

prdc_input_tbl <- tibble::tribble(
  ~input, ~value,
  "valuation_date", as.character(valuation_date_prdc),
```

```

"effective_date", as.character(effective_date_prdc),
"termination_date", as.character(termination_date_prdc),
"intro_coupon_end_date", as.character(intro_coupon_end_date_prdc),
"notional_jpy", as.character(notional_prdc),
"intro_coupon_rate", as.character(intro_coupon_rate_prdc),
"coupon_frequency_months", as.character(coupon_frequency_months_prdc),
"domestic_rate", as.character(domestic_rate_prdc),
"foreign_rate", as.character(foreign_rate_prdc),
"discount_rate", as.character(discount_rate_prdc),
"fx_spot", as.character(fx_spot_prdc),
"fx_volatility", as.character(fx_volatility_prdc),
"coupon_fx_base", as.character(coupon_fx_base_prdc),
"coupon_leverage", as.character(coupon_leverage_prdc),
"coupon_subtraction", as.character(coupon_subtraction_prdc),
"coupon_floor", as.character(coupon_floor_prdc),
"coupon_cap", as.character(coupon_cap_prdc),
"valuation_paths", as.character(number_of_paths_prdc),
"sensitivity_paths", as.character(sensitivity_paths_prdc),
"random_seed", as.character(random_seed_prdc)
)

GaussQuant::show_tbl_GQH(
  prdc_input_tbl,
  "PRDC product and model assumptions",
  n = 30
)

#> # A tibble: 20 x 2
#>   input                value
#>   <chr>                <chr>
#> 1 valuation_date      2023-04-11
#> 2 effective_date      2015-09-03
#> 3 termination_date    2041-09-03
#> 4 intro_coupon_end_date 2016-09-03
#> 5 notional_jpy        300000000
#> 6 intro_coupon_rate    0.022
#> 7 coupon_frequency_months 6
#> 8 domestic_rate        0.01
#> 9 foreign_rate         0.03
#> 10 discount_rate       0.005
#> 11 fx_spot            133.2681
#> 12 fx_volatility       0.1
#> 13 coupon_fx_base      120

```

```
#> 14 coupon_leverage      0.122
#> 15 coupon_subtraction    0.1
#> 16 coupon_floor         0
#> 17 coupon_cap           0.022
#> 18 valuation_paths      3000
#> 19 sensitivity_paths     1000
#> 20 random_seed          123
```

19.1.2 9.1.2 Coupon Schedule

Coupon Schedule Intro Coupon Schedule

```
valuation_date_ql_prdc <- GaussQuant::date_GQL(
  valuation_date_prdc
)

effective_date_ql_prdc <- GaussQuant::date_GQL(
  effective_date_prdc
)

termination_date_ql_prdc <- GaussQuant::date_GQL(
  termination_date_prdc
)

intro_coupon_end_date_ql_prdc <- GaussQuant::date_GQL(
  intro_coupon_end_date_prdc
)

coupon_schedule_prdc <- QuantLib::Schedule(
  effective_date_ql_prdc,
  termination_date_ql_prdc,
  GaussQuant::period_months_GQL(
    coupon_frequency_months_prdc
  ),
  QuantLib::TARGET(),
  "ModifiedFollowing",
  "ModifiedFollowing",
  "Backward",
  FALSE
)
```



```

intro_coupon_schedule_prdc <- QuantLib::Schedule(
  effective_date_ql_prdc,
  intro_coupon_end_date_ql_prdc,
  GaussQuant::period_months_GQL(
    coupon_frequency_months_prdc
  ),
  QuantLib::TARGET(),
  "ModifiedFollowing",
  "ModifiedFollowing",
  "Backward",
  FALSE
)

coupon_schedule_tbl <- GaussQuant::schedule_table_GQL(
  coupon_schedule_prdc
)

intro_coupon_dates_prdc <- GaussQuant::schedule_date_vector_GQL(
  intro_coupon_schedule_prdc
)

remaining_intro_coupon_dates_prdc <- intro_coupon_dates_prdc[
  intro_coupon_dates_prdc >
  valuation_date_prdc
]

intro_coupon_status_tbl <- tibble::tribble(
  ~metric, ~value,
  "intro_coupon_rate", intro_coupon_rate_prdc,
  "total_intro_schedule_dates", length(intro_coupon_dates_prdc),
  "remaining_intro_coupon_dates", length(remaining_intro_coupon_dates_prdc),
  "has_remaining_intro_coupon", as.numeric(
    length(
      remaining_intro_coupon_dates_prdc
    ) > 0
  )
)

GaussQuant::show_tbl_GQH(
  coupon_schedule_tbl,
  "PRDC full coupon schedule",
  n = 60
)

```

```

)
#> # A tibble: 53 x 2
#>   period schedule_date
#>   <int> <date>
#> 1     1 2015-09-03
#> 2     2 2016-03-03
#> 3     3 2016-09-05
#> 4     4 2017-03-03
#> 5     5 2017-09-04
#> 6     6 2018-03-05
#> 7     7 2018-09-03
#> 8     8 2019-03-04
#> 9     9 2019-09-03
#> 10    10 2020-03-03
#> # i 43 more rows

GaussQuant::show_tbl_GQH(
  intro_coupon_status_tbl,
  "PRDC introductory coupon status",
  n = 20
)
#> # A tibble: 4 x 2
#>   metric value
#>   <chr>   <dbl>
#> 1 intro_coupon_rate 0.022
#> 2 total_intro_schedule_dates 3
#> 3 remaining_intro_coupon_dates 0
#> 4 has_remaining_intro_coupon 0

```

19.1.3 9.1.3 Domestic Foreign Discount Curve

FX Quote 1

S_t = JPY per unit of foreign currency

- Domestic Curve JPY
- Foreign Curve

Garman–Kohlhagen

Domestic Foreign

$$\frac{dS_t}{S_t} = (r_d - r_f)dt + \sigma_{FX}dW_t$$

Domestic 1% Foreign 3%

FX 2%

Coupon

0.5%

Domestic OIS Curve Collateral

```
fx_day_counter_prdc <- QuantLib::Actual360()

domestic_curve_prdc <- GaussQuant::flat_curve_GQL(
  reference_date = valuation_date_ql_prdc,
  rate = domestic_rate_prdc,
  day_counter = fx_day_counter_prdc,
  compounding = "Continuous"
)

foreign_curve_prdc <- GaussQuant::flat_curve_GQL(
  reference_date = valuation_date_ql_prdc,
  rate = foreign_rate_prdc,
  day_counter = fx_day_counter_prdc,
  compounding = "Continuous"
)

discount_curve_prdc <- GaussQuant::flat_curve_GQL(
  reference_date = valuation_date_ql_prdc,
  rate = discount_rate_prdc,
  day_counter = fx_day_counter_prdc,
  compounding = "Continuous"
)

domestic_curve_handle_prdc <- QuantLib::YieldTermStructureHandle(
  domestic_curve_prdc
)

foreign_curve_handle_prdc <- QuantLib::YieldTermStructureHandle(
  foreign_curve_prdc
)

discount_curve_handle_prdc <- QuantLib::YieldTermStructureHandle(
```

```

    discount_curve_prdc
  )

curve_summary_tbl <- tibble::tribble(
  ~curve, ~flat_rate,
  "Domestic FX curve", domestic_rate_prdc,
  "Foreign FX curve", foreign_rate_prdc,
  "PRDC discount curve", discount_rate_prdc,
  "FX risk-neutral drift", domestic_rate_prdc -
    foreign_rate_prdc
)

GaussQuant::show_tbl_GQH(
  curve_summary_tbl,
  "PRDC flat-curve assumptions",
  n = 20
)

#> # A tibble: 4 x 2
#>   curve                flat_rate
#>   <chr>                <dbl>
#> 1 Domestic FX curve      0.01
#> 2 Foreign FX curve      0.03
#> 3 PRDC discount curve   0.005
#> 4 FX risk-neutral drift -0.02

```

19.2 9.2 FX Coupon Payoff

19.2.1 9.2.1 FX Process

FX volatility 10% Garman–Kohlhagen Process

```

fx_volatility_curve_prdc <- QuantLib::BlackConstantVol(
  valuation_date_ql_prdc,
  QuantLib::NullCalendar(),
  GaussQuant::quote_handle_GQL(
    fx_volatility_prdc
  ),
  fx_day_counter_prdc
)

fx_volatility_handle_prdc <- QuantLib::BlackVolTermStructureHandle(

```

```

    fx_volatility_curve_prdc
  )

fx_process_prdc <- GaussQuant::fx_process_GQL(
  spot = fx_spot_prdc,
  foreign_curve_handle = foreign_curve_handle_prdc,
  domestic_curve_handle = domestic_curve_handle_prdc,
  volatility_handle = fx_volatility_handle_prdc
)

fx_market_summary_tbl <- tibble::tribble(
  ~metric, ~value,
  "fx_spot", fx_spot_prdc,
  "fx_volatility", fx_volatility_prdc,
  "domestic_rate", domestic_rate_prdc,
  "foreign_rate", foreign_rate_prdc,
  "risk_neutral_drift", domestic_rate_prdc -
    foreign_rate_prdc
)

GaussQuant::show_tbl_GQH(
  fx_market_summary_tbl,
  "PRDC FX market assumptions",
  n = 20
)

#> # A tibble: 5 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 fx_spot    133.27
#> 2 fx_volatility  0.1
#> 3 domestic_rate  0.01
#> 4 foreign_rate  0.03
#> 5 risk_neutral_drift -0.02

```

19.2.2 9.2.2 Coupon Payoff

FX Coupon

$$C(S) = \min \left[2.2\%, \max \left(12.2\% \frac{S}{120} - 10\%, 0 \right) \right]$$

Floor FX

$$S_{\text{floor}} = S_0 \frac{b + C_{\min}}{a}$$

Cap FX

$$S_{\text{cap}} = S_0 \frac{b + C_{\max}}{a}$$

```
fx_floor_threshold_prdc <- coupon_fx_base_prdc *  
  (  
    coupon_subtraction_prdc +  
    coupon_floor_prdc  
  ) /  
  coupon_leverage_prdc  
  
fx_cap_threshold_prdc <- coupon_fx_base_prdc *  
  (  
    coupon_subtraction_prdc +  
    coupon_cap_prdc  
  ) /  
  coupon_leverage_prdc  
  
fx_payoff_grid_tbl <- tibble::tibble(  
  fx_rate = seq(  
    70,  
    180,  
    by = 2  
  )  
) |>  
  dplyr::mutate(  
    uncapped_coupon = coupon_leverage_prdc *  
      .data$fx_rate /  
      coupon_fx_base_prdc -  
      coupon_subtraction_prdc,  
    coupon_rate = GaussQuant::prdc_coupon_GQL(  
      fx_rate = .data$fx_rate,  
      fx_base = coupon_fx_base_prdc,  
      leverage = coupon_leverage_prdc,  
      subtraction = coupon_subtraction_prdc,
```

```

      floor_rate = coupon_floor_prdc,
      cap_rate = coupon_cap_prdc
    )
  )

coupon_threshold_tbl <- tibble::tribble(
  ~threshold, ~fx_rate,
  "floor_threshold", fx_floor_threshold_prdc,
  "cap_threshold", fx_cap_threshold_prdc,
  "current_spot", fx_spot_prdc
)

prdc_payoff_plot <- ggplot2::ggplot(
  fx_payoff_grid_tbl,
  ggplot2::aes(
    x = .data$fx_rate,
    y = .data$coupon_rate
  )
) +
  ggplot2::geom_line(
    linewidth = 0.8
  ) +
  ggplot2::geom_vline(
    xintercept = c(
      fx_floor_threshold_prdc,
      fx_cap_threshold_prdc
    ),
    linetype = "dashed"
  ) +
  ggplot2::scale_y_continuous(
    labels = scales::percent_format(
      accuracy = 0.1
    )
  ) +
  ggplot2::labs(
    title = "PRDC coupon payoff",
    subtitle = "Floored and capped FX-linked coupon",
    x = "FX rate",
    y = "Coupon rate"
  ) +
  ggplot2::theme_minimal()

```

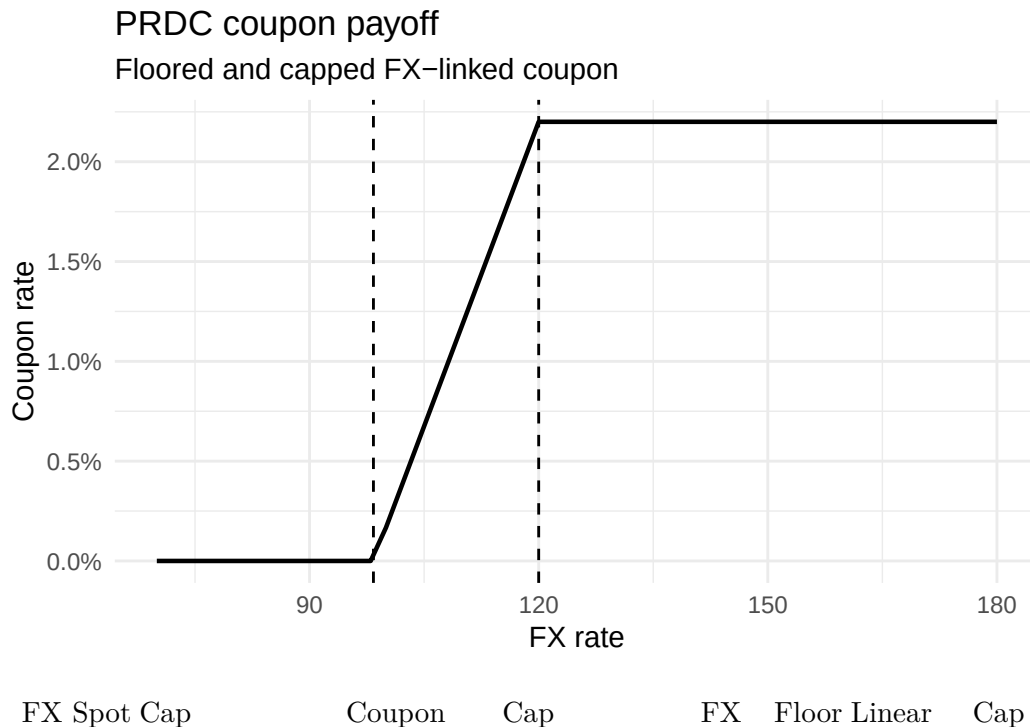
```

GaussQuant::show_tbl_GQH(
  coupon_threshold_tbl,
  "PRDC coupon FX thresholds",
  n = 20
)
#> # A tibble: 3 x 2
#>   threshold      fx_rate
#>   <chr>          <dbl>
#> 1 floor_threshold 98.361
#> 2 cap_threshold  120
#> 3 current_spot   133.27

GaussQuant::show_tbl_GQH(
  fx_payoff_grid_tbl,
  "PRDC coupon payoff table",
  n = 60
)
#> # A tibble: 56 x 3
#>   fx_rate uncapped_coupon coupon_rate
#>   <dbl>      <dbl>      <dbl>
#> 1    70      -0.028833      0
#> 2    72      -0.026800      0
#> 3    74      -0.024767      0
#> 4    76      -0.022733      0
#> 5    78      -0.0207      0
#> 6    80      -0.018667      0
#> 7    82      -0.016633      0
#> 8    84      -0.014600      0
#> 9    86      -0.012567      0
#> 10   88      -0.010533      0
#> # i 46 more rows

print(prdc_payoff_plot)

```

19.3 9.3 Monte Carlo PRDC

19.3.1 9.3.1 Coupon FX

PRDC Coupon GaussQuant `fx_path_generator_GQL()` Coupon

```
fx_path_generator_prdc <- GaussQuant::fx_path_generator_GQL(
  valuation_date = valuation_date_prdc,
  coupon_schedule = coupon_schedule_prdc,
  day_counter = fx_day_counter_prdc,
  process = fx_process_prdc
)

time_grid_tbl <- tibble::tibble(
  coupon_number = seq_along(
    fx_path_generator_prdc$remaining_coupon_dates
  ),
  coupon_date = fx_path_generator_prdc$remaining_coupon_dates,
  time_from_valuation = fx_path_generator_prdc$coupon_times,
  time_step = fx_path_generator_prdc$time_steps
)
```

```

)

path_generator_summary_tbl <- tibble::tribble(
  ~metric, ~value,
  "remaining_coupon_dates", length(
    fx_path_generator_prdc$remaining_coupon_dates
  ),
  "number_of_steps", fx_path_generator_prdc$number_of_steps,
  "first_coupon_time", min(
    fx_path_generator_prdc$coupon_times
  ),
  "final_coupon_time", max(
    fx_path_generator_prdc$coupon_times
  )
)

GaussQuant::show_tbl_GQH(
  path_generator_summary_tbl,
  "PRDC FX path-generator summary",
  n = 20
)

#> # A tibble: 4 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 remaining_coupon_dates 37
#> 2 number_of_steps      37
#> 3 first_coupon_time     0.40556
#> 4 final_coupon_time     18.667

GaussQuant::show_tbl_GQH(
  time_grid_tbl,
  "PRDC remaining coupon-date time grid",
  n = 60
)

#> # A tibble: 37 x 4
#>   coupon_number coupon_date time_from_valuation time_step
#>   <int> <date>          <dbl>      <dbl>
#> 1         1 2023-09-04      0.40556    0.40556
#> 2         2 2024-03-04      0.91111    0.50556
#> 3         3 2024-09-03      1.4194     0.50833
#> 4         4 2025-03-03      1.9222     0.50278
#> 5         5 2025-09-03      2.4333     0.51111

```

```
#> 6          6 2026-03-03          2.9361    0.50278
#> 7          7 2026-09-03          3.4472    0.51111
#> 8          8 2027-03-03          3.95      0.50278
#> 9          9 2027-09-03          4.4611    0.51111
#> 10         10 2028-03-03          4.9667    0.50556
#> # i 27 more rows
```

QuantLib StochasticProcess1D\$evolve()
t Path Generator Coupon

Brownian increment

GaussQuant

19.3.2 9.3.2 FX

Monte Carlo 20 FX FX Coupon

```
sample_path_count_prdc <- 20L

sample_fx_path_matrix_prdc <- GaussQuant::prdc_path_matrix_GQL(
  path_generator = fx_path_generator_prdc,
  n_paths = sample_path_count_prdc,
  seed = random_seed_prdc
)

sample_fx_path_tbl <- purrr::map_dfr(
  seq_len(
    ncol(sample_fx_path_matrix_prdc)
  ),
  function(path_id) {
    path_values <- sample_fx_path_matrix_prdc[
      ,
      path_id
    ]

    tibble::tibble(
      path_id = path_id,
      coupon_number = seq_along(
        path_values
      ),
      coupon_date = fx_path_generator_prdc$remaining_coupon_dates,
      time = fx_path_generator_prdc$coupon_times,
      fx_rate = path_values,
      coupon_rate = GaussQuant::prdc_coupon_GQL(
```

```

    fx_rate = path_values,
    fx_base = coupon_fx_base_prdc,
    leverage = coupon_leverage_prdc,
    subtraction = coupon_subtraction_prdc,
    floor_rate = coupon_floor_prdc,
    cap_rate = coupon_cap_prdc
  )
)
}
)

sample_fx_path_plot <- ggplot2::ggplot(
  sample_fx_path_tbl,
  ggplot2::aes(
    x = .data$time,
    y = .data$fx_rate,
    group = .data$path_id
  )
) +
  ggplot2::geom_line(
    linewidth = 0.5,
    alpha = 0.7
  ) +
  ggplot2::labs(
    title = "Sample FX paths for PRDC valuation",
    x = "Time from valuation date (years)",
    y = "FX rate"
  ) +
  ggplot2::theme_minimal()

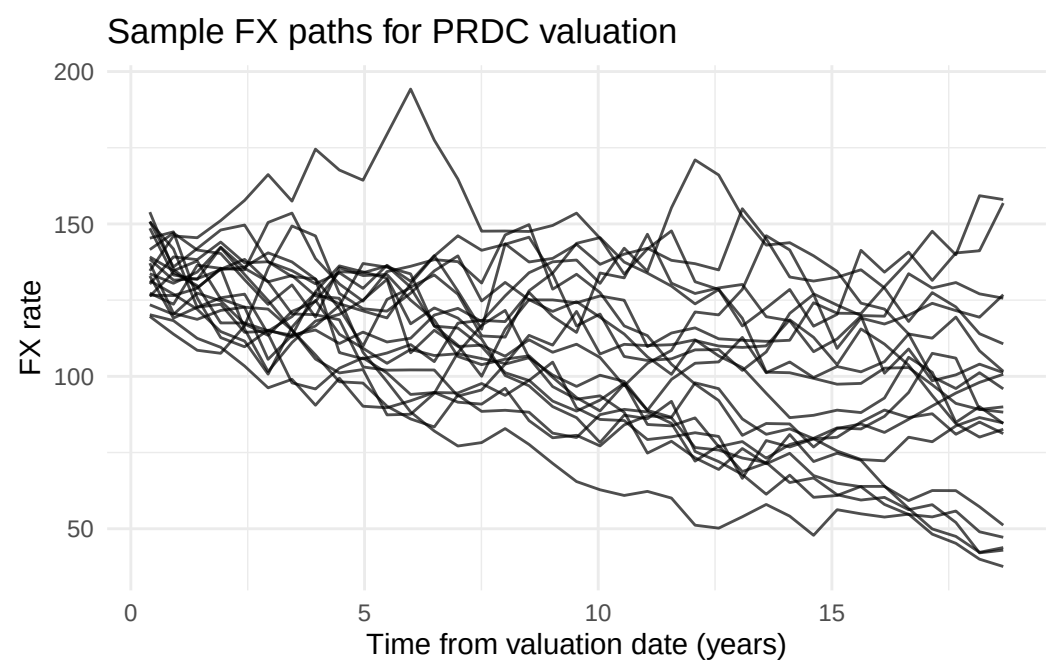
GaussQuant::show_tbl_GQH(
  sample_fx_path_tbl,
  "Sample PRDC FX and coupon paths",
  n = 40
)

#> # A tibble: 40 x 6
#>   path_id coupon_number coupon_date    time fx_rate coupon_rate
#>   <int>      <int> <date>      <dbl>   <dbl>      <dbl>
#> 1         1          1 2023-09-04  0.40556 127.30      0.022
#> 2         1          2 2024-03-04  0.91111 123.66      0.022
#> 3         1          3 2024-09-03  1.4194 136.45      0.022
#> 4         1          4 2025-03-03  1.9222 135.42      0.022

```

```
#> 5      1      5 2025-09-03  2.4333  134.94    0.022
#> 6      1      6 2026-03-03  2.9361  150.49    0.022
#> 7      1      7 2026-09-03  3.4472  153.56    0.022
#> 8      1      8 2027-03-03  3.95    138.63    0.022
#> 9      1      9 2027-09-03  4.4611  130.31    0.022
#> 10     1     10 2028-03-03  4.9667  124.66    0.022
#> # i 30 more rows

print(sample_fx_path_plot)
```



19.3.3 9.3.3 PRDC

Coupon Coupon rate Monte Carlo

$$E[C(S_{t_i})] \approx \frac{1}{M} \sum_{j=1}^M C(S_{t_i}^{(j)})$$

M $S_{t_i}^{(j)}$ j Coupon t_i FX Rate
Coupon cash flow

$$CF_i = N_0 \alpha_i E[C(S_{t_i})]$$

PRDC

$$PV_{\text{PRDC}} = \sum_i D(0, t_i) CF_i + D(0, T) N_0$$

```
prdc_result <- GaussQuant::prdc_npv_GQL(
  valuation_date = valuation_date_prdc,
  coupon_schedule = coupon_schedule_prdc,
  process = fx_process_prdc,
  discount_curve = discount_curve_prdc,
  notional = notional_prdc,
  fx_base = coupon_fx_base_prdc,
  leverage = coupon_leverage_prdc,
  subtraction = coupon_subtraction_prdc,
  floor_rate = coupon_floor_prdc,
  cap_rate = coupon_cap_prdc,
  day_counter = fx_day_counter_prdc,
  intro_coupon_dates = intro_coupon_dates_prdc,
  intro_coupon_rate = intro_coupon_rate_prdc,
  n_paths = number_of_paths_prdc,
  seed = random_seed_prdc,
  redemption_amount = notional_prdc,
  return_details = TRUE
)

fx_path_matrix_prdc <- prdc_result$path_matrix
expected_coupon_tbl <- prdc_result$expected_coupon
prdc_cashflow_tbl <- prdc_result$cashflows

coupon_leg_present_value_prdc <- prdc_result$summary |>
  dplyr::filter(
    .data$metric == "coupon_leg_npv"
  ) |>
  dplyr::pull(
    value
  )

redemption_present_value_prdc <- prdc_result$summary |>
```

```

dplyr::filter(
  .data$metric == "redemption_leg_npv"
) |>
dplyr::pull(
  value
)

prdc_present_value_jpy <- prdc_result$npv

prdc_valuation_summary_tbl <- tibble::tribble(
  ~metric, ~value,
  "coupon_leg_pv_jpy", coupon_leg_present_value_prdc,
  "redemption_leg_pv_jpy", redemption_present_value_prdc,
  "total_pv_jpy", prdc_present_value_jpy,
  "pv_as_percent_of_notional", prdc_present_value_jpy /
    notional_prdc,
  "number_of_paths", number_of_paths_prdc,
  "number_of_coupon_steps", nrow(
    fx_path_matrix_prdc
  )
)

GaussQuant::show_tbl_GQH(
  prdc_valuation_summary_tbl,
  "PRDC Monte Carlo valuation summary",
  n = 20
)

#> # A tibble: 6 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 coupon_leg_pv_jpy 6.1290e7
#> 2 redemption_leg_pv_jpy 2.7327e8
#> 3 total_pv_jpy      3.3456e8
#> 4 pv_as_percent_of_notional 1.1152e0
#> 5 number_of_paths      3      e3
#> 6 number_of_coupon_steps 3.7    e1

```

Redemption Leg FX FX Spot FX volatility Coupon Leg

19.3.4 9.3.4 Coupon

```
expected_coupon_display_tbl <- expected_coupon_tbl |>
  dplyr::select(
    coupon_number,
    coupon_date,
    time,
    expected_fx,
    expected_coupon_rate,
    is_intro_coupon,
    final_coupon_rate
  )

prdc_cashflow_display_tbl <- prdc_cashflow_tbl |>
  dplyr::select(
    coupon_number,
    accrual_start,
    accrual_end,
    final_coupon_rate,
    accrual_fraction,
    coupon_amount,
    redemption_amount,
    total_amount,
    discount_factor,
    total_present_value
  )

GaussQuant::show_tbl_GQH(
  expected_coupon_display_tbl,
  "PRDC expected FX and coupon rates",
  n = 60
)

#> # A tibble: 37 x 7
#>   coupon_number coupon_date    time expected_fx expected_coupon_rate
#>   <int> <date>      <dbl>      <dbl>          <dbl>
#> 1         1 2023-09-04  0.40556      132.03        0.021771
#> 2         2 2024-03-04  0.91111      130.88        0.020779
#> 3         3 2024-09-03  1.4194       129.55        0.019616
#> 4         4 2025-03-03  1.9222       128.28        0.018543
#> 5         5 2025-09-03  2.4333       127.03        0.017649
#> 6         6 2026-03-03  2.9361       125.71        0.016803
#> 7         7 2026-09-03  3.4472       124.48        0.016096
```



```

#> 8      8 2027-03-03 3.95      123.49      0.015349
#> 9      9 2027-09-03 4.4611    122.11      0.014650
#> 10     10 2028-03-03 4.9667    120.84      0.014013
#>   is_intro_coupon final_coupon_rate
#>   <lgl>           <dbl>
#> 1 FALSE           0.021771
#> 2 FALSE           0.020779
#> 3 FALSE           0.019616
#> 4 FALSE           0.018543
#> 5 FALSE           0.017649
#> 6 FALSE           0.016803
#> 7 FALSE           0.016096
#> 8 FALSE           0.015349
#> 9 FALSE           0.014650
#> 10 FALSE          0.014013
#> # i 27 more rows

GaussQuant::show_tbl_GQH(
  prdc_cashflow_display_tbl,
  "PRDC expected cash-flow table",
  n = 60
)
#> # A tibble: 37 x 10
#>   coupon_number accrual_start accrual_end final_coupon_rate accrual_fraction
#>   <int> <date>      <date>      <dbl>      <dbl>
#> 1      1 2023-03-03 2023-09-04      0.021771    0.51389
#> 2      2 2023-09-04 2024-03-04      0.020779    0.50556
#> 3      3 2024-03-04 2024-09-03      0.019616    0.50833
#> 4      4 2024-09-03 2025-03-03      0.018543    0.50278
#> 5      5 2025-03-03 2025-09-03      0.017649    0.51111
#> 6      6 2025-09-03 2026-03-03      0.016803    0.50278
#> 7      7 2026-03-03 2026-09-03      0.016096    0.51111
#> 8      8 2026-09-03 2027-03-03      0.015349    0.50278
#> 9      9 2027-03-03 2027-09-03      0.014650    0.51111
#> 10     10 2027-09-03 2028-03-03      0.014013    0.50556
#>   coupon_amount redemption_amount total_amount discount_factor
#>   <dbl>          <dbl>      <dbl>      <dbl>
#> 1 3356427.      0      3356427.    0.99797
#> 2 3151496.      0      3151496.    0.99545
#> 3 2991377.      0      2991377.    0.99293
#> 4 2796944.      0      2796944.    0.99043
#> 5 2706124.      0      2706124.    0.98791

```

```

#> 6      2534415.      0      2534415.      0.98543
#> 7      2468101.      0      2468101.      0.98291
#> 8      2315073.      0      2315073.      0.98044
#> 9      2246348.      0      2246348.      0.97794
#> 10     2125268.      0      2125268.      0.97547
#>      total_present_value
#>      <dbl>
#> 1      3349627.
#> 2      3137172.
#> 3      2970222.
#> 4      2770191.
#> 5      2673399.
#> 6      2497480.
#> 7      2425925.
#> 8      2269799.
#> 9      2196797.
#> 10     2073141.
#> # i 27 more rows

```

	FX	Domestic	Foreign		Foreign	Domestic		FX Rate	
Cap	Linear	Floor							-

19.4 9.4 Coupon

19.4.1 9.4.1 Coupon

```

expected_coupon_plot <- ggplot2::ggplot(
  expected_coupon_tbl,
  ggplot2::aes(
    x = .data$time,
    y = .data$final_coupon_rate
  )
) +
  ggplot2::geom_line(
    linewidth = 0.8
  ) +
  ggplot2::geom_point(
    size = 1.8
  ) +

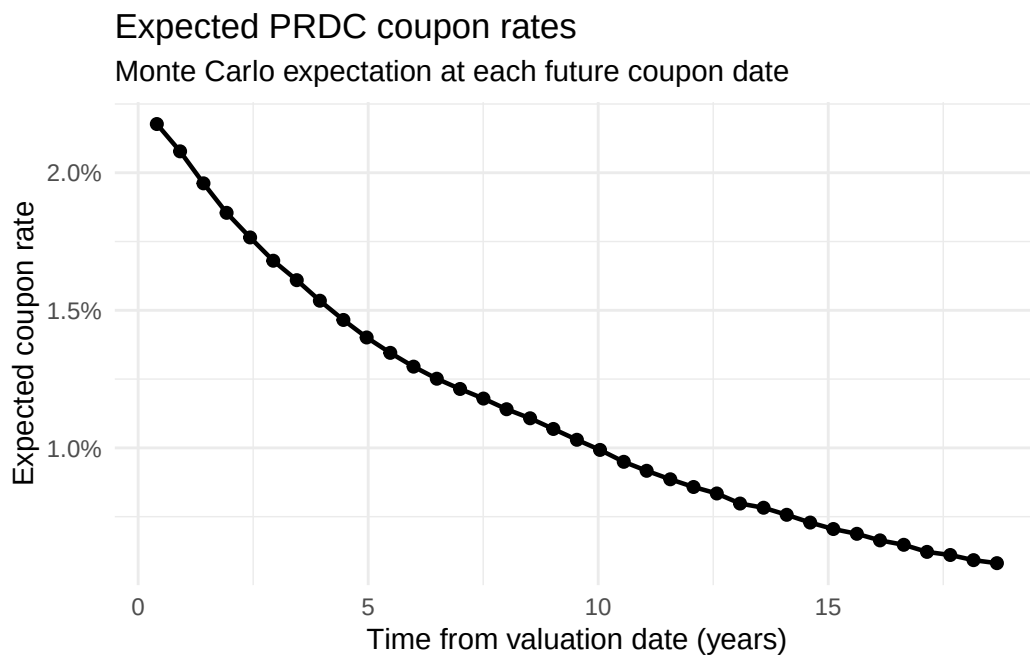
```

```

ggplot2::scale_y_continuous(
  labels = scales::percent_format(
    accuracy = 0.1
  )
) +
ggplot2::labs(
  title = "Expected PRDC coupon rates",
  subtitle = "Monte Carlo expectation at each future coupon date",
  x = "Time from valuation date (years)",
  y = "Expected coupon rate"
) +
ggplot2::theme_minimal()

print(expected_coupon_plot)

```



Coupon FX Coupon

$$E[C(S_t)] \neq C(E[S_t])$$

Cap Floor Payoff FX

19.4.2 9.4.2 Coupon Cap Floor

Coupon Coupon Floor Cap

```
coupon_cap_floor_probability_tbl <- GaussQuant::prdc_cap_floor_probability_GQL(
  path_matrix = fx_path_matrix_prdc,
  fx_base = coupon_fx_base_prdc,
  leverage = coupon_leverage_prdc,
  subtraction = coupon_subtraction_prdc,
  floor_rate = coupon_floor_prdc,
  cap_rate = coupon_cap_prdc,
  coupon_dates = fx_path_generator_prdc$remaining_coupon_dates,
  coupon_times = fx_path_generator_prdc$coupon_times
)

GaussQuant::show_tbl_GQH(
  coupon_cap_floor_probability_tbl,
  "PRDC coupon floor and cap probabilities",
  n = 60
)

#> # A tibble: 37 x 6
#>   coupon_number floor_probability cap_probability interior_probability
#>   <int>          <dbl>          <dbl>          <dbl>
#> 1         1         0         0.93133         0.068667
#> 2         2      0.00066667      0.80267         0.19667
#> 3         3      0.013667      0.72167         0.26467
#> 4         4      0.034667      0.65333         0.312
#> 5         5      0.063333      0.615         0.32167
#> 6         6      0.091667      0.58133         0.327
#> 7         7      0.11833      0.54567         0.336
#> 8         8      0.14967      0.523         0.32733
#> 9         9      0.17533      0.493         0.33167
#> 10        10      0.205      0.46533         0.32967
#>   coupon_date    time
#>   <date>      <dbl>
#> 1 2023-09-04  0.40556
#> 2 2024-03-04  0.91111
#> 3 2024-09-03  1.4194
#> 4 2025-03-03  1.9222
#> 5 2025-09-03  2.4333
#> 6 2026-03-03  2.9361
#> 7 2026-09-03  3.4472
#> 8 2027-03-03  3.95
```

```
#> 9 2027-09-03 4.4611
#> 10 2028-03-03 4.9667
#> # i 27 more rows
```

Cap Linear Floor Coupon Spot Cap Coupon Cap

19.4.3 9.4.3 Coupon

Coupon Coupon rate

```
coupon_path_matrix_prdc <- GaussQuant::prdc_coupon_GQL(
  fx_rate = fx_path_matrix_prdc,
  fx_base = coupon_fx_base_prdc,
  leverage = coupon_leverage_prdc,
  subtraction = coupon_subtraction_prdc,
  floor_rate = coupon_floor_prdc,
  cap_rate = coupon_cap_prdc
)

number_of_coupon_steps_prdc <- nrow(
  coupon_path_matrix_prdc
)

distribution_coupon_ids <- unique(
  pmax(
    1L,
    pmin(
      number_of_coupon_steps_prdc,
      c(
        1L,
        round(
          number_of_coupon_steps_prdc /
            2
        ),
        number_of_coupon_steps_prdc
      )
    )
  )
)

coupon_distribution_tbl <- purrr::map_dfr(
```

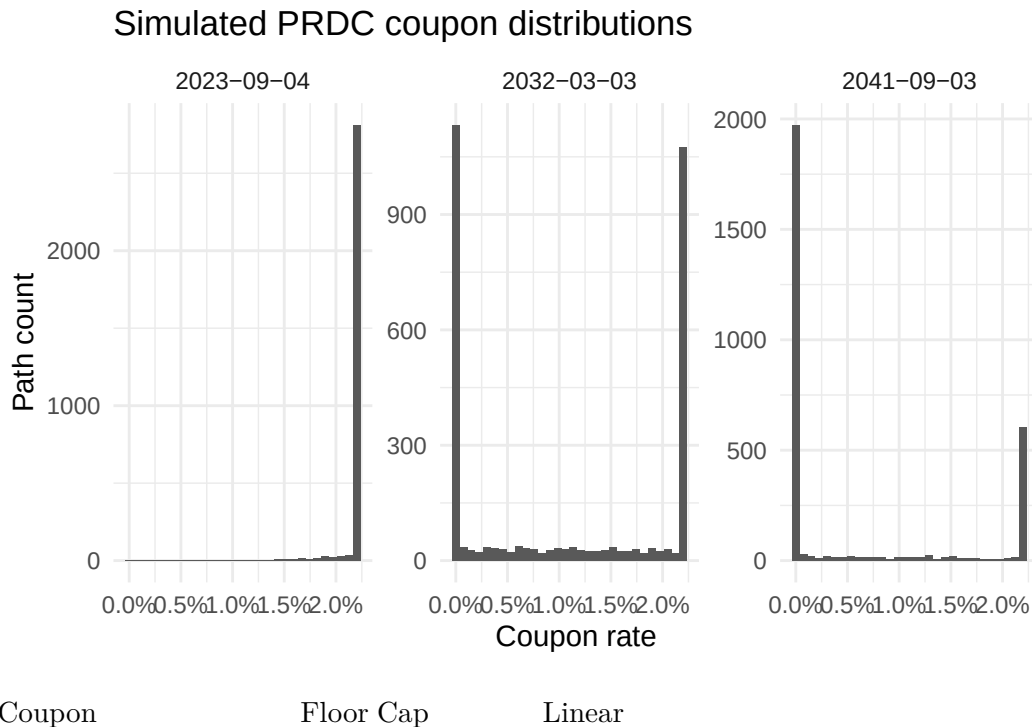
```

distribution_coupon_ids,
function(coupon_id) {
  tibble::tibble(
    coupon_number = coupon_id,
    coupon_date = as.character(
      fx_path_generator_prdc$remaining_coupon_dates[
        coupon_id
      ]
    ),
    coupon_rate = coupon_path_matrix_prdc[
      coupon_id,
    ]
  )
}
)

coupon_distribution_plot <- ggplot2::ggplot(
  coupon_distribution_tbl,
  ggplot2::aes(
    x = .data$coupon_rate
  )
) +
  ggplot2::geom_histogram(
    bins = 30,
    linewidth = 0.3
  ) +
  ggplot2::facet_wrap(
    ~coupon_date,
    scales = "free_y"
  ) +
  ggplot2::scale_x_continuous(
    labels = scales::percent_format(
      accuracy = 0.1
    )
  ) +
  ggplot2::labs(
    title = "Simulated PRDC coupon distributions",
    x = "Coupon rate",
    y = "Path count"
  ) +
  ggplot2::theme_minimal()

```

```
print(coupon_distribution_plot)
```



19.5 9.5 FX Monte Carlo

19.5.1 9.5.1 FX Spot

PRDC Coupon FX Spot Spot
Seed Monte Carlo Common Random Numbers

```
spot_scenario_input_tbl <- tibble::tribble(
  ~scenario, ~fx_spot,
  "spot_minus_10", fx_spot_prdc * 0.90,
  "spot_minus_5", fx_spot_prdc * 0.95,
  "base", fx_spot_prdc,
  "spot_plus_5", fx_spot_prdc * 1.05,
  "spot_plus_10", fx_spot_prdc * 1.10
)
```

```

spot_sensitivity_tbl <- GaussQuant::prdc_spot_sensitivity_GQL(
  spot_scenarios = spot_scenario_input_tbl,
  valuation_date = valuation_date_prdc,
  coupon_schedule = coupon_schedule_prdc,
  foreign_curve_handle = foreign_curve_handle_prdc,
  domestic_curve_handle = domestic_curve_handle_prdc,
  volatility_handle = fx_volatility_handle_prdc,
  discount_curve = discount_curve_prdc,
  notional = notional_prdc,
  fx_base = coupon_fx_base_prdc,
  leverage = coupon_leverage_prdc,
  subtraction = coupon_subtraction_prdc,
  floor_rate = coupon_floor_prdc,
  cap_rate = coupon_cap_prdc,
  day_counter = fx_day_counter_prdc,
  intro_coupon_dates = intro_coupon_dates_prdc,
  intro_coupon_rate = intro_coupon_rate_prdc,
  n_paths = sensitivity_paths_prdc,
  seed = random_seed_prdc,
  redemption_amount = notional_prdc,
  base_scenario = "base"
)

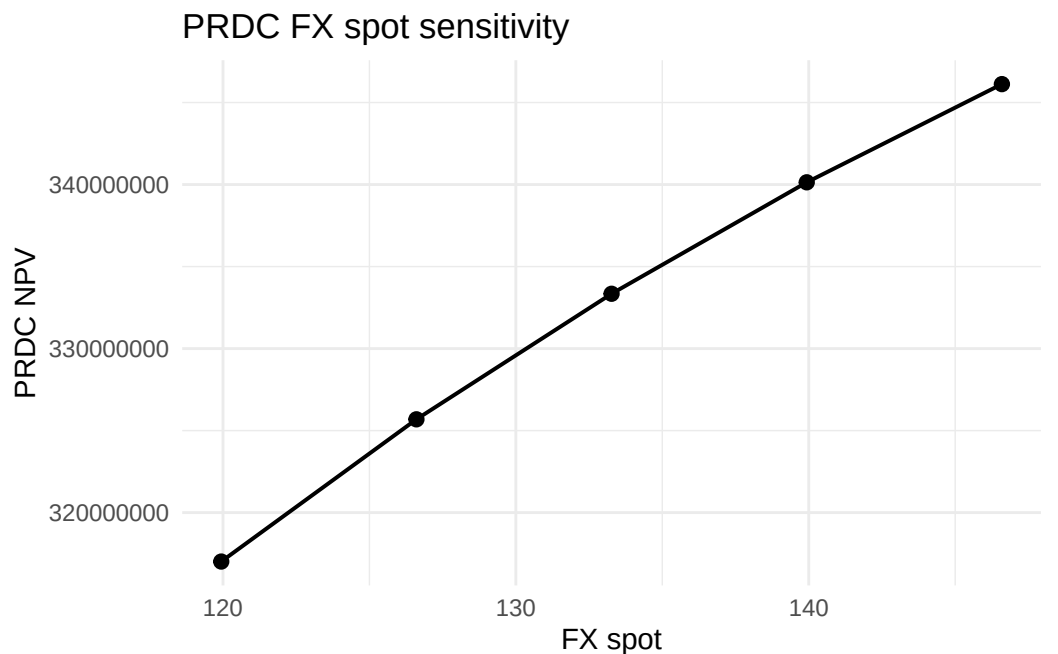
spot_sensitivity_plot <- ggplot2::ggplot(
  spot_sensitivity_tbl,
  ggplot2::aes(
    x = .data$fx_spot,
    y = .data$prdc_npv
  )
) +
  ggplot2::geom_line(
    linewidth = 0.7
  ) +
  ggplot2::geom_point(
    size = 2.2
  ) +
  ggplot2::labs(
    title = "PRDC FX spot sensitivity",
    x = "FX spot",
    y = "PRDC NPV"
  ) +
  ggplot2::theme_minimal()

```



```
GaussQuant::show_tbl_GQH(
  spot_sensitivity_tbl,
  "PRDC FX spot sensitivity",
  n = 20
)
#> # A tibble: 5 x 4
#>   scenario      fx_spot  prdc_npv npv_difference_from_base
#>   <chr>         <dbl>    <dbl>         <dbl>
#> 1 spot_minus_10  119.94 317016976.         -16325124.
#> 2 spot_minus_5   126.60 325687168.         -7654931.
#> 3 base           133.27 333342099.             0
#> 4 spot_plus_5    139.93 340136016.          6793916.
#> 5 spot_plus_10   146.59 346121967.         12779868.

print(spot_sensitivity_plot)
```



Spot Coupon Cap Spot Floor Coupon PRDC Spot

19.5.2 9.5.2 FX Volatility

PRDC Coupon Floor Cap Payoff FX volatility FX Floor Linear -
Cap

```

volatility_scenario_input_tbl <- tibble::tribble(
  ~scenario, ~fx_volatility,
  "vol_5pct", 0.05,
  "vol_7_5pct", 0.075,
  "base", fx_volatility_prdc,
  "vol_12_5pct", 0.125,
  "vol_15pct", 0.15
)

volatility_sensitivity_tbl <- GaussQuant::prdc_volatility_sensitivity_GQL(
  volatility_scenarios = volatility_scenario_input_tbl,
  valuation_date = valuation_date_prdc,
  coupon_schedule = coupon_schedule_prdc,
  spot = fx_spot_prdc,
  foreign_curve_handle = foreign_curve_handle_prdc,
  domestic_curve_handle = domestic_curve_handle_prdc,
  discount_curve = discount_curve_prdc,
  notional = notional_prdc,
  fx_base = coupon_fx_base_prdc,
  leverage = coupon_leverage_prdc,
  subtraction = coupon_subtraction_prdc,
  floor_rate = coupon_floor_prdc,
  cap_rate = coupon_cap_prdc,
  day_counter = fx_day_counter_prdc,
  calendar = QuantLib::NullCalendar(),
  intro_coupon_dates = intro_coupon_dates_prdc,
  intro_coupon_rate = intro_coupon_rate_prdc,
  n_paths = sensitivity_paths_prdc,
  seed = random_seed_prdc,
  redemption_amount = notional_prdc,
  base_scenario = "base"
)

volatility_sensitivity_plot <- ggplot2::ggplot(
  volatility_sensitivity_tbl,
  ggplot2::aes(
    x = .data$fx_volatility,
    y = .data$prdc_npv
  )
) +
  ggplot2::geom_line(
    linewidth = 0.7
  )

```

```

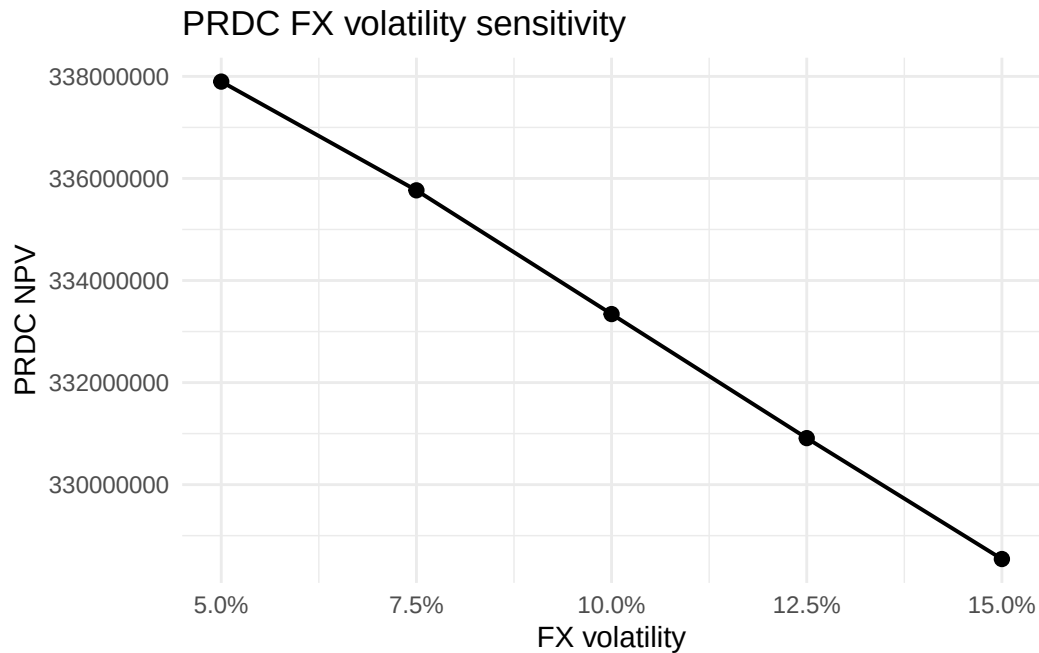
) +
ggplot2::geom_point(
  size = 2.2
) +
ggplot2::scale_x_continuous(
  labels = scales::percent_format(
    accuracy = 0.1
  )
) +
ggplot2::labs(
  title = "PRDC FX volatility sensitivity",
  x = "FX volatility",
  y = "PRDC NPV"
) +
ggplot2::theme_minimal()

GaussQuant::show_tbl_GQH(
  volatility_sensitivity_tbl,
  "PRDC FX volatility sensitivity",
  n = 20
)

#> # A tibble: 5 x 4
#>   scenario    fx_volatility  prdc_npv npv_difference_from_base
#>   <chr>          <dbl>      <dbl>          <dbl>
#> 1 vol_5pct      0.05  337897654.      4555555.
#> 2 vol_7_5pct    0.075 335766964.      2424865.
#> 3 base          0.1   333342099.         0
#> 4 vol_12_5pct   0.125 330911366.     -2430733.
#> 5 vol_15pct     0.15  328542072.     -4800028.

print(volatility_sensitivity_plot)

```



Option Volatility PRDC Coupon Cap Payoff Floor Spot Volatil-

19.5.3 9.5.3 Monte Carlo

Monte Carlo Coupon Redemption

$$\hat{\sigma}_{PV} \quad M$$

$$SE = \frac{\hat{\sigma}_{PV}}{\sqrt{M}}$$

```
monte_carlo_error_result <- GaussQuant::prdc_monte_carlo_error_GQL(
  path_matrix = fx_path_matrix_prdc,
  cashflow_tbl = prdc_cashflow_tbl,
  notional = notional_prdc,
  fx_base = coupon_fx_base_prdc,
  leverage = coupon_leverage_prdc,
  subtraction = coupon_subtraction_prdc,
  floor_rate = coupon_floor_prdc,
  cap_rate = coupon_cap_prdc,
```

```

    intro_coupon_dates = intro_coupon_dates_prdc,
    intro_coupon_rate = intro_coupon_rate_prdc,
    confidence_level = 0.95
)

path_present_value_tbl <- monte_carlo_error_result$path_present_values
monte_carlo_error_tbl <- monte_carlo_error_result$summary

GaussQuant::show_tbl_GQH(
  monte_carlo_error_tbl,
  "PRDC Monte Carlo error estimate",
  n = 20
)
#> # A tibble: 7 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 number_of_paths                      3000
#> 2 mean_present_value                  334557294.
#> 3 present_value_sd                   36143296.
#> 4 standard_error                      659883.
#> 5 lower_confidence_bound             333263946.
#> 6 upper_confidence_bound             335850641.
#> 7 confidence_level                    0.95

```

4 Antithetic Variates Control Variates Quasi-Monte Carlo

19.6 9.6 Part II

19.6.1 9.6.1 PRDC

PRDC Coupon Monte Carlo

19.6.2 9.6.2

Domestic Foreign Discount Curve PRDC Domestic Foreign FX For-
ward Coupon

19.6.3 9.6.3 FX volatility

Black volatility FX Option Strike Volatility Surface PRDC Smile Skew Volatility term structure

19.6.4 9.6.4

PRDC Domestic Foreign FX FX Cross-currency

19.6.5 9.6.5

PRDC Coupon Callable

Call value Continuation value

Continuation value Closed-form Least-Squares Monte Carlo Backward Induction

Callable PRDC Coupon Leg Redemption Leg Callable PRDC

$$V_{\text{callable PRDC}} = V_{\text{noncallable PRDC}} - V_{\text{issuer call}}$$

19.6.6 9.6.6 Part II

II

```
part2_chapter_summary_tbl <- tibble::tribble(
  ~chapter, ~subject, ~main_method,
  "II-1", "Bonds and market conventions", "Yield, price, duration, and convexity",
  "II-2", "Treasury futures and CTD", "Basis, carry, implied repo, and futures BPV",
  "II-3", "OIS and multi-curve valuation", "Discount and forecast curve separation",
  "II-4", "Black model", "Lognormal forward-option pricing",
  "II-5", "Normal model", "Bachelier pricing for zero and negative rates",
  "II-6", "SABR model", "Volatility-smile calibration",
  "II-7", "Callable Bond and Hull-White", "Short-rate tree and early exercise",
  "II-8", "Credit risk and CDS", "Hazard curve, MidPoint, and ISDA valuation",
  "II-9", "Exotic derivatives and PRDC", "FX-linked coupon Monte Carlo valuation"
)
```

```

GaussQuant::show_tbl_GQH(
  part2_chapter_summary_tbl,
  "Part II chapter summary",
  n = 20
)
#> # A tibble: 9 x 3
#>   chapter subject
#>   <chr>    <chr>
#> 1 II-1     Bonds and market conventions
#> 2 II-2     Treasury futures and CTD
#> 3 II-3     OIS and multi-curve valuation
#> 4 II-4     Black model
#> 5 II-5     Normal model
#> 6 II-6     SABR model
#> 7 II-7     Callable Bond and Hull-White
#> 8 II-8     Credit risk and CDS
#> 9 II-9     Exotic derivatives and PRDC
#>   main_method
#>   <chr>
#> 1 Yield, price, duration, and convexity
#> 2 Basis, carry, implied repo, and futures BPV
#> 3 Discount and forecast curve separation
#> 4 Lognormal forward-option pricing
#> 5 Bachelier pricing for zero and negative rates
#> 6 Volatility-smile calibration
#> 7 Short-rate tree and early exercise
#> 8 Hazard curve, MidPoint, and ISDA valuation
#> 9 FX-linked coupon Monte Carlo valuation

```

OIS Option Volatility Smile

Part II

- Schedule
- Curve
-
- Cash flow Payoff
- Pricing Engine
- NPV Risk Calibration
-

GaussQuant QuantLib SWIG

LagrangeFinance

19.7 9.7

PRDC FX Rate Coupon

Coupon rate

$$C(S) = \min \left[C_{\max}, \max \left(a \frac{S}{S_0} - b, C_{\min} \right) \right]$$

Floor Cap Payoff

Garman–Kohlhagen FX Rate Domestic Foreign

$$\frac{dS_t}{S_t} = (r_d - r_f)dt + \sigma_{\text{FX}}dW_t$$

Coupon FX Monte Carlo Coupon Coupon Coupon Leg Redemption Leg PRDC
NPV

FX Spot FX volatility Floor Cap Monte Carlo NPV

Callable PRDC Callable PRDC Domestic Foreign FX Volatility Surface

```
chapter09_summary_tbl <- tibble::tribble(
  ~section, ~main_result,
  "Coupon payoff", "Floored and capped FX-linked coupon",
  "FX process", "Garman-Kohlhagen risk-neutral dynamics",
  "Monte Carlo valuation", "Coupon leg and redemption leg NPV",
  "Spot sensitivity", "Nonlinear response caused by cap and floor",
  "Volatility sensitivity", "Distributional effect on coupon regions",
  "Monte Carlo error", "Standard error and confidence interval",
  "Model boundary", "Non-callable deterministic-rate PRDC"
)

GaussQuant::show_tbl_GQH(
  chapter09_summary_tbl,
  "Chapter II-9 summary",
  n = 20
)

#> # A tibble: 7 x 2
#>   section                                main_result
#>   <chr>                                <chr>
#> 1 Coupon payoff                        Floored and capped FX-linked coupon
#> 2 FX process                          Garman-Kohlhagen risk-neutral dynamics
```



```

#> 3 Monte Carlo valuation   Coupon leg and redemption leg NPV
#> 4 Spot sensitivity        Nonlinear response caused by cap and floor
#> 5 Volatility sensitivity  Distributional effect on coupon regions
#> 6 Monte Carlo error       Standard error and confidence interval
#> 7 Model boundary         Non-callable deterministic-rate PRDC

cat(
  "\nChapter II-9 Exotic derivatives and PRDC completed successfully.\n"
)
#>
#> Chapter II-9 Exotic derivatives and PRDC completed successfully.

```

Part III

III

20 III-1

I

II

Arrow

tidyverse

future

↓
↓
map() / future_map()
↓
↓
Arrow / Parquet

20.1 1.1

Trade Data
Market Data
Calibration Data
↓
Pricing Function
↓
Risk Table
↓
Report

```
result <- pricing_function(
  trade_data = trade_data,
  market_data = market_data,
  calibration_data = calibration_data
)
```

CDS PRDC

20.1.1

```
calculate_present_value_LFL <- function(cashflow, discount_rate, maturity) {
  tibble(
    cashflow = cashflow,
    discount_rate = discount_rate,
    maturity = maturity,
    present_value = cashflow / (1 + discount_rate)^maturity
  )
}
```

tibble

20.2 1.2

tibble

```
tbl_calculation <- tribble(
  ~scenario_ID, ~cashflow, ~discount_rate, ~maturity,
  1L, 100, 0.010, 1,
  2L, 100, 0.015, 2,
  3L, 100, 0.020, 3,
  4L, 100, 0.025, 4,
  5L, 100, 0.030, 5,
  6L, 100, 0.035, 6
)
```

```
show_table_LFL(tbl_calculation)
#> # A tibble: 6 x 4
#>   scenario_ID cashflow discount_rate maturity
#>   <int>      <dbl>      <dbl>      <dbl>
#> 1         1      100        0.01         1
#> 2         2      100        0.015        2
#> 3         3      100        0.02         3
#> 4         4      100        0.025        4
#> 5         5      100        0.03         5
#> 6         6      100        0.035        6
```

`purrr::pmap()`

```
tbl_sequential_result <- tbl_calculation |>
  mutate(
    result = purrr::pmap(
      list(cashflow, discount_rate, maturity),
      calculate_present_value_LFL
    )
  ) |>
  select(scenario_ID, result) |>
  tidyr::unnest(result)

show_table_LFL(tbl_sequential_result)
#> # A tibble: 6 x 5
#>   scenario_ID cashflow discount_rate maturity present_value
#>   <int>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1         1      100        0.01         1      99.010
#> 2         2      100        0.015        2      97.066
#> 3         3      100        0.02         3      94.232
#> 4         4      100        0.025        4      90.595
#> 5         5      100        0.03         5      86.261
#> 6         6      100        0.035        6      81.350
```

`calculate_present_value_LFL()`

```
Data
+
Function
↓
Result
```

20.3 1.3 future

```
future      plan()
  Windows   multisession
```

```
available_cores <- future::availableCores()
workers <- max(1L, min(2L, available_cores - 1L))

future::plan(
  future::multisession,
  workers = workers
)

tbl_future_setting <- tibble(
  available_cores = available_cores,
  workers = workers
)

show_table_LFL(tbl_future_setting)
#> # A tibble: 1 x 2
#>   available_cores workers
#>       <int>     <int>
#> 1         8         2
```

```
purrr::pmap() furrr::future_pmap()
```

```
tbl_parallel_result <- tbl_calculation |>
  mutate(
    result = furrr::future_pmap(
      list(cashflow, discount_rate, maturity),
      calculate_present_value_LFL,
      .options = furrr::furrr_options(seed = TRUE)
    )
  ) |>
  select(scenario_ID, result) |>
  tidyr::unnest(result)

show_table_LFL(tbl_parallel_result)
```

```
#> # A tibble: 6 x 5
#>   scenario_ID cashflow discount_rate maturity present_value
#>   <int>      <dbl>      <dbl>    <dbl>      <dbl>
#> 1         1      100        0.01         1      99.010
#> 2         2      100        0.015        2      97.066
#> 3         3      100        0.02         3      94.232
#> 4         4      100        0.025        4      90.595
#> 5         5      100        0.03         5      86.261
#> 6         6      100        0.035        6      81.350
```

```
tbl_result_comparison <- tbl_sequential_result |>
  select(
    scenario_ID,
    sequential_present_value = present_value
  ) |>
  left_join(
    tbl_parallel_result |>
      select(
        scenario_ID,
        parallel_present_value = present_value
      ),
    by = "scenario_ID"
  ) |>
  mutate(
    difference = parallel_present_value - sequential_present_value
  )
```

```
show_table_LFL(tbl_result_comparison)
#> # A tibble: 6 x 4
#>   scenario_ID sequential_present_value parallel_present_value difference
#>   <int>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1         1      99.010      99.010          0
#> 2         2      97.066      97.066          0
#> 3         3      94.232      94.232          0
#> 4         4      90.595      90.595          0
#> 5         5      86.261      86.261          0
#> 6         6      81.350      81.350          0
```

```

stopifnot(
  isTRUE(
    all.equal(
      tbl_result_comparison$sequential_present_value,
      tbl_result_comparison$parallel_present_value,
      tolerance = 1e-12
    )
  )
)

```

```

future::plan(future::sequential)

```

20.4 1.4 Arrow

I Parquet III Parquet

```

read_parquet()
  ↓
  ↓
map() / future_map()
  ↓
  ↓
write_parquet()

```

```

tbl_parallel_output <- tbl_calculation |>
  left_join(
    tbl_parallel_result |>
      select(scenario_ID, present_value),
    by = "scenario_ID"
  ) |>
  arrange(scenario_ID)

show_table_LFL(tbl_parallel_output)
#> # A tibble: 6 x 5

```



```
#>   scenario_ID cashflow discount_rate maturity present_value
#>       <int>    <dbl>      <dbl>    <dbl>      <dbl>
#> 1         1      100        0.01         1      99.010
#> 2         2      100        0.015        2      97.066
#> 3         3      100        0.02         3      94.232
#> 4         4      100        0.025        4      90.595
#> 5         5      100        0.03         5      86.261
#> 6         6      100        0.035        6      81.350
```

```
parallel_output_path <- write_derived_parquet_file_LFL(
  tbl_parallel_output,
  file.path(
    "distributed",
    "parallel_calculation_example.parquet"
  )
)
```

```
parallel_output_path
```

```
#> [1] "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/distributed/parallel_calculation"
```

```
tbl_parallel_output_check <- read_derived_parquet_file_LFL(
  file.path(
    "distributed",
    "parallel_calculation_example.parquet"
  )
)
```

```
tbl_output_verification <- tibble(
  expected_rows = nrow(tbl_parallel_output),
  actual_rows = nrow(tbl_parallel_output_check),
  same_columns = identical(
    names(tbl_parallel_output),
    names(tbl_parallel_output_check)
  )
)
```

```
show_table_LFL(tbl_output_verification)
```

```
#> # A tibble: 1 x 3
```

```
#>   expected_rows actual_rows same_columns
```

```
#>           <int>           <int> <lgl>
#> 1             6             6 TRUE
```

```
stopifnot(
  nrow(tbl_parallel_output_check) == nrow(tbl_parallel_output),
  identical(
    names(tbl_parallel_output_check),
    names(tbl_parallel_output)
  )
)
```

20.5 1.5

```
calculate_task_LFL <- function(task_ID) {
  Sys.sleep(0.10)

  tibble(
    task_ID = task_ID,
    result = sqrt(task_ID)
  )
}

vec_task_ID <- seq_len(12L)
```

```
sequential_time <- system.time(
  sequential_benchmark <- purrr::map(
    vec_task_ID,
    calculate_task_LFL
  )
)[["elapsed"]]
```

```
future::plan(
  future::multisession,
  workers = workers
)
```

```
parallel_time <- system.time(
  parallel_benchmark <- furrr::future_map(
    vec_task_ID,
    calculate_task_LFL,
    .options = furrr::furrr_options(seed = TRUE)
  )
)["elapsed"]

future::plan(future::sequential)
```

```
tbl_benchmark <- tribble(
  ~method, ~elapsed_seconds,
  "purrr::map", sequential_time,
  "furrr::future_map", parallel_time
) |>
  mutate(
    relative_to_sequential = elapsed_seconds / sequential_time
  )

show_table_LFL(tbl_benchmark)
#> # A tibble: 2 x 3
#>   method                elapsed_seconds relative_to_sequential
#>   <chr>                  <dbl>                <dbl>
#> 1 purrr::map              1.36                  1
#> 2 furrr::future_map      1.3100                0.96324
```

```
plot_benchmark <- tbl_benchmark |>
  ggplot(
    aes(
      x = method,
      y = elapsed_seconds,
      fill = method
    )
  ) +
  geom_col(show.legend = FALSE) +
  labs(
    x = NULL,
    y = "Elapsed Seconds",
    title = "Sequential and Parallel Processing"
  )

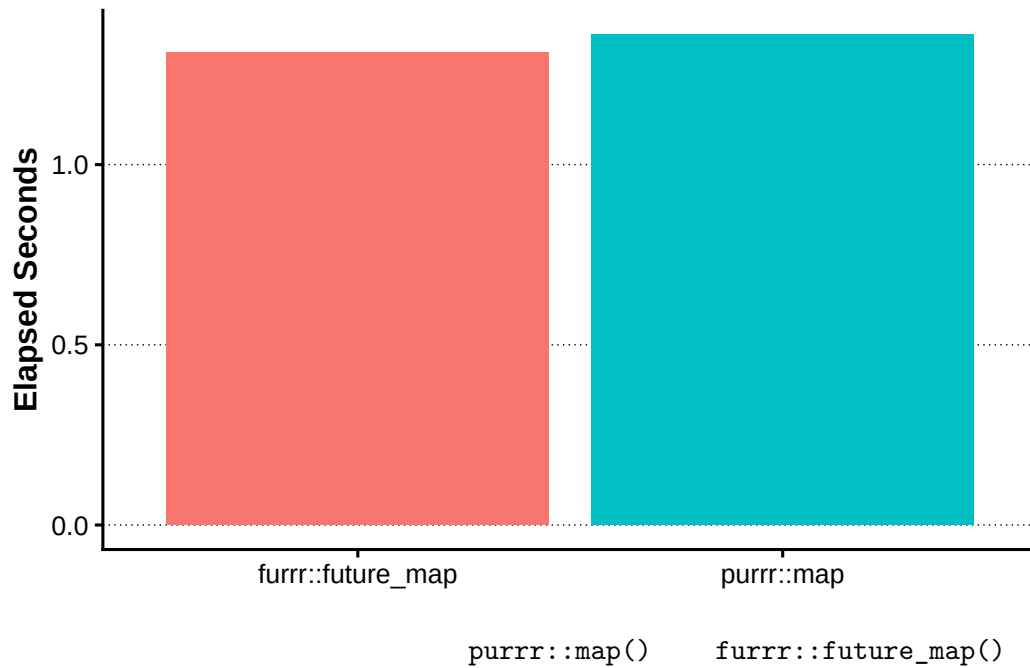
show_plot_LFL(
```

```

add_lagrange_theme_LFL(
  plot_benchmark
)
)

```

Sequential and Parallel Processing



Arrow Parquet

```

1 PC
  ↓
future / furr
  ↓
  ↓
gRPC
  ↓
Slurm

```

21 III-2

III-1 `purrr::map()` `furrr::future_map()`

III-1

↓
↓
↓
↓
↓

21.1 2.1

Computer
Worker 1
Worker 2
Worker 3
Worker 4

```
Master
  Computer 1
  Computer 2
  Computer 3
  Computer 4
```

```
Master
```

```
Worker
```

```
tbl_task <- tribble(
  ~task_ID, ~input_value, ~worker_group,
  1L, 10, "A",
  2L, 20, "A",
  3L, 30, "B",
  4L, 40, "B",
  5L, 50, "C",
  6L, 60, "C"
)

show_table_LFL(tbl_task)
#> # A tibble: 6 x 3
#>   task_ID input_value worker_group
#>   <int>      <dbl> <chr>
#> 1     1         10 A
#> 2     2         20 A
#> 3     3         30 B
#> 4     4         40 B
#> 5     5         50 C
#> 6     6         60 C
```

```

calculate_task_result_LFL <- function(
  task_ID,
  input_value,
  worker_group
) {
  tibble(
    task_ID = task_ID,
    worker_group = worker_group,
    input_value = input_value,
    output_value = sqrt(input_value)
  )
}

```

III-1

21.2 2.2

```

Data
↓
Serialize
↓
Remote Function
↓
Deserialize
↓
Result

```

```

result <- calculate_task_result_LFL(
  task_ID = 1L,
  input_value = 10,
  worker_group = "A"
)

```

```

result <- remote_calculate_task(
  task_ID = 1L,
  input_value = 10,
  worker_group = "A"
)

```

```

tbl_remote_request <- tbl_task |>
  transmute(
    request_ID = paste0(
      "REQ-",
      stringr::str_pad(
        task_ID,
        width = 3,
        pad = "0"
      )
    ),
    task_ID,
    payload = purrr::pmap(
      list(
        task_ID,
        input_value,
        worker_group
      ),
      \(task_ID, input_value, worker_group) {
        list(
          task_ID = task_ID,
          input_value = input_value,
          worker_group = worker_group
        )
      }
    )
  )

show_table_LFL(
  tbl_remote_request |>
    select(
      request_ID,
      task_ID
    )

```



```

    )
  )
#> # A tibble: 6 x 2
#>   request_ID task_ID
#>   <chr>      <int>
#> 1 REQ-001      1
#> 2 REQ-002      2
#> 3 REQ-003      3
#> 4 REQ-004      4
#> 5 REQ-005      5
#> 6 REQ-006      6

```

```

remote_calculate_task_LFL <- function(payload) {
  calculate_task_result_LFL(
    task_ID = payload$task_ID,
    input_value = payload$input_value,
    worker_group = payload$worker_group
  )
}

```

```

tbl_remote_result <- tbl_remote_request |>
  mutate(
    response = purrr::map(
      payload,
      remote_calculate_task_LFL
    )
  ) |>
  select(
    request_ID,
    response
  ) |>
  tidyr::unnest(response)

show_table_LFL(tbl_remote_result)
#> # A tibble: 6 x 5
#>   request_ID task_ID worker_group input_value output_value
#>   <chr>      <int> <chr>          <dbl>      <dbl>

```

#> 1	REQ-001	1	A	10	3.1623
#> 2	REQ-002	2	A	20	4.4721
#> 3	REQ-003	3	B	30	5.4772
#> 4	REQ-004	4	B	40	6.3246
#> 5	REQ-005	5	C	50	7.0711
#> 6	REQ-006	6	C	60	7.7460

gRPC

21.3 2.3

```

Task
↓
Job Queue
↓
Scheduler
↓
Worker
↓
Result

```

```

tbl_job <- tbl_task |>
  transmute(
    job_ID = paste0(
      "JOB-",
      stringr::str_pad(
        task_ID,
        width = 3,
        pad = "0"
      )
    ),
    task_ID,
    status = "queued",
    worker_group,
    input_value
  )

```

)

```
show_table_LFL(tbl_job)
```

```
#> # A tibble: 6 x 5
```

```
#>   job_ID task_ID status worker_group input_value
#>   <chr>   <int> <chr>   <chr>          <dbl>
#> 1 JOB-001     1 queued A             10
#> 2 JOB-002     2 queued A             20
#> 3 JOB-003     3 queued B             30
#> 4 JOB-004     4 queued B             40
#> 5 JOB-005     5 queued C             50
#> 6 JOB-006     6 queued C             60
```

queued

↓

running

↓

completed

failed

```
tbl_job_status <- tribble(
  ~job_ID, ~status, ~sequence,
  "JOB-001", "queued", 1L,
  "JOB-001", "running", 2L,
  "JOB-001", "completed", 3L,
  "JOB-002", "queued", 1L,
  "JOB-002", "running", 2L,
  "JOB-002", "failed", 3L
)
```

```
show_table_LFL(tbl_job_status)
```

```
#> # A tibble: 6 x 3
```

```
#>   job_ID status      sequence
#>   <chr>   <chr>         <int>
#> 1 JOB-001 queued         1
#> 2 JOB-001 running        2
#> 3 JOB-001 completed       3
#> 4 JOB-002 queued         1
```

```
#> 5 JOB-002 running      2
#> 6 JOB-002 failed      3
```

Slurm

```
sbatch
```

```
squeue
```

```
srun
```

R Slurm

```
future::plan(
  future.batchtools::batchtools_slurm
)
```

Slurm Linux

21.4 2.4

```
Market Data
Calibration Data
```

```
Trade Data
Scenario Data
```

```
tbl_market_data <- tribble(
  ~market_data_ID, ~discount_rate, ~fx_spot,
  "MKT-001", 0.020, 150,
  "MKT-002", 0.025, 155
)

tbl_calibration_data <- tribble(
  ~calibration_ID, ~volatility, ~correlation,
  "CAL-001", 0.15, 0.20,
  "CAL-002", 0.20, 0.40
)

tbl_trade_data <- tribble(
  ~trade_ID, ~notional, ~maturity,
  "TRD-001", 1000000, 3,
  "TRD-002", 2000000, 5
)

tbl_scenario_data <- tidyr::crossing(
  trade_ID = tbl_trade_data$trade_ID,
  market_data_ID =
    tbl_market_data$market_data_ID,
  calibration_ID =
    tbl_calibration_data$calibration_ID
) |>
  mutate(
    scenario_ID = row_number(),
    .before = 1
  )

show_table_LFL(tbl_scenario_data)
#> # A tibble: 8 x 4
#>   scenario_ID trade_ID market_data_ID calibration_ID
#>       <int> <chr>      <chr>          <chr>
#> 1         1 TRD-001 MKT-001      CAL-001
#> 2         2 TRD-001 MKT-001      CAL-002
#> 3         3 TRD-001 MKT-002      CAL-001
#> 4         4 TRD-001 MKT-002      CAL-002
#> 5         5 TRD-002 MKT-001      CAL-001
#> 6         6 TRD-002 MKT-001      CAL-002
#> 7         7 TRD-002 MKT-002      CAL-001
#> 8         8 TRD-002 MKT-002      CAL-002
```

```
tbl_distributed_input <- tbl_scenario_data |>
  left_join(
    tbl_trade_data,
    by = "trade_ID"
  ) |>
  left_join(
    tbl_market_data,
    by = "market_data_ID"
  ) |>
  left_join(
    tbl_calibration_data,
    by = "calibration_ID"
  )

show_table_LFL(tbl_distributed_input)
#> # A tibble: 8 x 10
#>   scenario_ID trade_ID market_data_ID calibration_ID notional maturity
#>   <int> <chr>      <chr>          <chr>          <dbl>    <dbl>
#> 1         1 TRD-001 MKT-001          CAL-001      1000000      3
#> 2         2 TRD-001 MKT-001          CAL-002      1000000      3
#> 3         3 TRD-001 MKT-002          CAL-001      1000000      3
#> 4         4 TRD-001 MKT-002          CAL-002      1000000      3
#> 5         5 TRD-002 MKT-001          CAL-001      2000000      5
#> 6         6 TRD-002 MKT-001          CAL-002      2000000      5
#> 7         7 TRD-002 MKT-002          CAL-001      2000000      5
#> 8         8 TRD-002 MKT-002          CAL-002      2000000      5
#>   discount_rate fx_spot volatility correlation
#>   <dbl>    <dbl>      <dbl>      <dbl>
#> 1      0.02      150      0.15      0.2
#> 2      0.02      150      0.2       0.4
#> 3     0.025     155      0.15      0.2
#> 4     0.025     155      0.2       0.4
#> 5      0.02      150      0.15      0.2
#> 6      0.02      150      0.2       0.4
#> 7     0.025     155      0.15      0.2
#> 8     0.025     155      0.2       0.4
```

Arrow Parquet

```
trade_data.parquet
market_data.parquet
calibration_data.parquet
scenario_data.parquet
```

```
tbl_distributed_files <- list(
  trade_data = tbl_trade_data,
  market_data = tbl_market_data,
  calibration_data =
    tbl_calibration_data,
  scenario_data = tbl_scenario_data
)
```

```
vec_distributed_paths <- tbl_distributed_files |>
  purrr::imap_chr(
    \(data, name) {
      write_derived_parquet_file_LFL(
        data,
        file.path(
          "distributed",
          paste0(name, ".parquet")
        )
      )
    }
  )
```

```
vec_distributed_paths
```

```
#>                                                                 trade_data
#>      "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/distributed/trade_data.parquet"
#>                                                                 market_data
#>      "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/distributed/market_data.parquet"
#>                                                                 calibration_data
#>      "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/distributed/calibration_data.parquet"
#>                                                                 scenario_data
#>      "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/distributed/scenario_data.parquet"
```

21.5 2.5

III-1 future furr

```
Data
↓
Remote Function
↓
Job Queue
↓
Worker
↓
Result
```

```
Trade Data
Market Data
Calibration Data
Scenario Data
↓
Calculation Function
↓
Risk Table
```

II

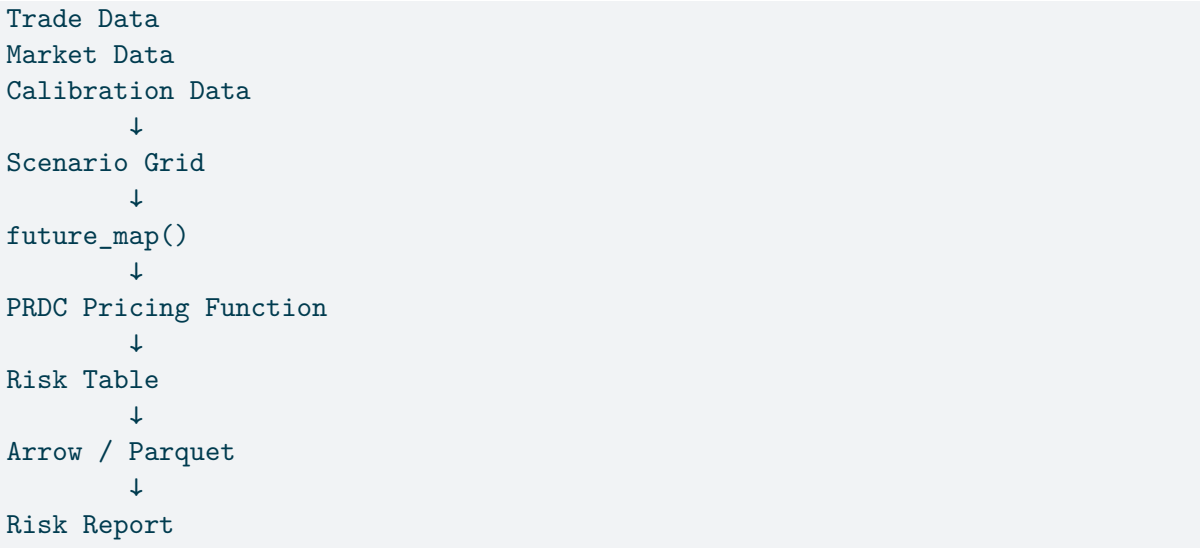
22 III-3 PRDC

III-1

map() future_map()

III-2

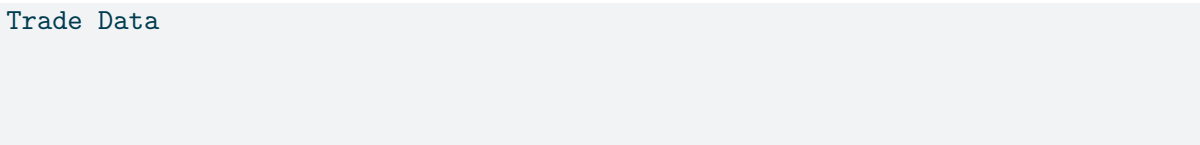
II-9 PRDC



PRDC

II

22.1 3.1



Coupon
Market Data
FX Spot
Domestic Rate
Foreign Rate
Discount Rate
Calibration Data
FX Volatility
Correlation
Model Parameters
Calculation Data
Path Count
Random Seed
Worker Setting

II-9	PRDC	Domestic	Foreign	FX Spot	FX Volatility	Garman–Kohlhagen	
	PRDC	Domestic	Foreign	FX		PRDC	FX
	Spot	FX	Volatility				

22.1.1

```
tbl_prdc_trade <- tribble(
  ~trade_ID,
  ~valuation_date,
  ~effective_date,
  ~termination_date,
  ~intro_coupon_end_date,
  ~notional,
  ~intro_coupon_rate,
  ~coupon_frequency_months,
  ~coupon_fx_base,
  ~coupon_leverage,
  ~coupon_subtraction,
  ~coupon_floor,
  ~coupon_cap,
  "PRDC-001",
```

```

as.Date("2023-04-11"),
as.Date("2015-09-03"),
as.Date("2041-09-03"),
as.Date("2016-09-03"),
300000000,
0.022,
6L,
120,
0.122,
0.10,
0,
0.022
)

show_table_LFL(tbl_prdc_trade)
#> # A tibble: 1 x 13
#>   trade_ID valuation_date effective_date termination_date intro_coupon_end_date
#>   <chr>      <date>         <date>         <date>         <date>
#> 1 PRDC-001 2023-04-11      2015-09-03      2041-09-03      2016-09-03
#>   notional intro_coupon_rate coupon_frequency_months coupon_fx_base
#>   <dbl>      <dbl>                <int>         <dbl>
#> 1 300000000      0.022                6            120
#>   coupon_leverage coupon_subtraction coupon_floor coupon_cap
#>   <dbl>          <dbl>         <dbl>    <dbl>
#> 1      0.122        0.1          0      0.022

```

22.1.2

```

tbl_prdc_market <- tribble(
  ~market_data_ID,
  ~fx_spot,
  ~domestic_rate,
  ~foreign_rate,
  ~discount_rate,
  "MKT-BASE",
  133.2681,
  0.01,
  0.03,
  0.005
)

```

```

tbl_prdc_calibration <- tribble(
  ~calibration_ID,
  ~fx_volatility,
  ~correlation,
  ~correlation_active,
  "CAL-BASE",
  0.10,
  0.00,
  FALSE
)

tbl_prdc_calculation <- tribble(
  ~calculation_ID,
  ~number_of_paths,
  ~random_seed,
  "CALC-BASE",
  500L,
  123L
)

show_table_LFL(tbl_prdc_market)
#> # A tibble: 1 x 5
#>   market_data_ID fx_spot domestic_rate foreign_rate discount_rate
#>   <chr>          <dbl>         <dbl>         <dbl>         <dbl>
#> 1 MKT-BASE      133.27          0.01          0.03          0.005
show_table_LFL(tbl_prdc_calibration)
#> # A tibble: 1 x 4
#>   calibration_ID fx_volatility correlation correlation_active
#>   <chr>          <dbl>         <dbl> <lgl>
#> 1 CAL-BASE      0.1          0 FALSE
show_table_LFL(tbl_prdc_calculation)
#> # A tibble: 1 x 3
#>   calculation_ID number_of_paths random_seed
#>   <chr>          <int>         <int>
#> 1 CALC-BASE      500          123

```

HTML

500

22.2 3.2 PRDC

II-9 PRDC Schedule Curve FX Process

```
price_prdc_scenario_LFL <- function(  
  trade,  
  market,  
  calibration,  
  calculation  
) {  
  valuation_date <- trade$valuation_date[[1]]  
  effective_date <- trade$effective_date[[1]]  
  termination_date <- trade$termination_date[[1]]  
  intro_coupon_end_date <-  
    trade$intro_coupon_end_date[[1]]  
  
  GaussQuant::set_eval_date_GQL(  
    valuation_date  
  )  
  
  valuation_date_ql <-  
    GaussQuant::date_GQL(  
      valuation_date  
    )  
  
  effective_date_ql <-  
    GaussQuant::date_GQL(  
      effective_date  
    )  
  
  termination_date_ql <-  
    GaussQuant::date_GQL(  
      termination_date  
    )  
  
  intro_coupon_end_date_ql <-  
    GaussQuant::date_GQL(  
      intro_coupon_end_date  
    )  
  
  coupon_schedule <- QuantLib::Schedule(  
    effective_date_ql,
```

```

    termination_date_ql,
    GaussQuant::period_months_GQL(
      trade$coupon_frequency_months[[1]]
    ),
    QuantLib::TARGET(),
    "ModifiedFollowing",
    "ModifiedFollowing",
    "Backward",
    FALSE
  )

intro_coupon_schedule <- QuantLib::Schedule(
  effective_date_ql,
  intro_coupon_end_date_ql,
  GaussQuant::period_months_GQL(
    trade$coupon_frequency_months[[1]]
  ),
  QuantLib::TARGET(),
  "ModifiedFollowing",
  "ModifiedFollowing",
  "Backward",
  FALSE
)

intro_coupon_dates <-
  GaussQuant::schedule_date_vector_GQL(
    intro_coupon_schedule
  )

day_counter <- QuantLib::Actual360()

domestic_curve <-
  GaussQuant::flat_curve_GQL(
    reference_date =
      valuation_date_ql,
    rate =
      market$domestic_rate[[1]],
    day_counter =
      day_counter,
    compounding =
      "Continuous"
  )

```

```

foreign_curve <-
  GaussQuant::flat_curve_GQL(
    reference_date =
      valuation_date_ql,
    rate =
      market$foreign_rate[[1]],
    day_counter =
      day_counter,
    compounding =
      "Continuous"
  )

discount_curve <-
  GaussQuant::flat_curve_GQL(
    reference_date =
      valuation_date_ql,
    rate =
      market$discount_rate[[1]],
    day_counter =
      day_counter,
    compounding =
      "Continuous"
  )

domestic_curve_handle <-
  QuantLib::YieldTermStructureHandle(
    domestic_curve
  )

foreign_curve_handle <-
  QuantLib::YieldTermStructureHandle(
    foreign_curve
  )

volatility_curve <-
  QuantLib::BlackConstantVol(
    valuation_date_ql,
    QuantLib::NullCalendar(),
    GaussQuant::quote_handle_GQL(
      calibration$fx_volatility[[1]]
    ),
    day_counter
  )

```

```

)

volatility_handle <-
  QuantLib::BlackVolTermStructureHandle(
    volatility_curve
  )

fx_process <-
  GaussQuant::fx_process_GQL(
    spot =
      market$fx_spot[[1]],
    foreign_curve_handle =
      foreign_curve_handle,
    domestic_curve_handle =
      domestic_curve_handle,
    volatility_handle =
      volatility_handle
  )

result <- GaussQuant::prdc_npv_GQL(
  valuation_date =
    valuation_date,
  coupon_schedule =
    coupon_schedule,
  process =
    fx_process,
  discount_curve =
    discount_curve,
  notional =
    trade$notional[[1]],
  fx_base =
    trade$coupon_fx_base[[1]],
  leverage =
    trade$coupon_leverage[[1]],
  subtraction =
    trade$coupon_subtraction[[1]],
  floor_rate =
    trade$coupon_floor[[1]],
  cap_rate =
    trade$coupon_cap[[1]],
  day_counter =
    day_counter,

```



```

    intro_coupon_dates =
      intro_coupon_dates,
    intro_coupon_rate =
      trade$intro_coupon_rate[[1]],
    n_paths =
      calculation$number_of_paths[[1]],
    seed =
      calculation$random_seed[[1]],
    redemption_amount =
      trade$notional[[1]],
    return_details =
      FALSE
  )

  tibble(
    trade_ID =
      trade$trade_ID[[1]],
    fx_spot =
      market$fx_spot[[1]],
    fx_volatility =
      calibration$fx_volatility[[1]],
    correlation =
      calibration$correlation[[1]],
    correlation_active =
      calibration$correlation_active[[1]],
    number_of_paths =
      calculation$number_of_paths[[1]],
    random_seed =
      calculation$random_seed[[1]],
    prdc_npv =
      as.numeric(result)
  )
}

```

tibble

22.2.1

```

tbl_prdc_base_result <-
  price_prdc_scenario_LFL(

```

```

trade =
  tbl_prdc_trade,
market =
  tbl_prdc_market,
calibration =
  tbl_prdc_calibration,
calculation =
  tbl_prdc_calculation
)

show_table_LFL(tbl_prdc_base_result)
#> # A tibble: 1 x 8
#>   trade_ID fx_spot fx_volatility correlation correlation_active number_of_paths
#>   <chr>      <dbl>      <dbl>      <dbl> <lgl>                <int>
#> 1 PRDC-001 133.27          0.1          0 FALSE                500
#>   random_seed prdc_npv
#>   <int>      <dbl>
#> 1      123 334081804.

```

22.3 3.3

FX Spot FX Volatility

```

base_fx_spot <-
  tbl_prdc_market$fx_spot[[1]]

tbl_prdc_scenario <- tidyr::crossing(
  fx_spot = base_fx_spot *
    c(0.90, 0.95, 1.00, 1.05, 1.10),
  fx_volatility =
    c(0.075, 0.10, 0.125),
  correlation =
    c(-0.50, 0.00, 0.50)
) |>
mutate(
  scenario_ID = row_number(),
  correlation_active = FALSE,
  .before = 1
)

```

```

show_table_LFL(
  tbl_prdc_scenario,
  n = Inf
)
#> # A tibble: 45 x 5
#>   scenario_ID correlation_active fx_spot fx_volatility correlation
#>   <int> <lgl>                <dbl>      <dbl>      <dbl>
#> 1         1 FALSE             119.94      0.075     -0.5
#> 2         2 FALSE             119.94      0.075      0
#> 3         3 FALSE             119.94      0.075      0.5
#> 4         4 FALSE             119.94      0.1      -0.5
#> 5         5 FALSE             119.94      0.1       0
#> 6         6 FALSE             119.94      0.1      0.5
#> 7         7 FALSE             119.94      0.125    -0.5
#> 8         8 FALSE             119.94      0.125     0
#> 9         9 FALSE             119.94      0.125      0.5
#> 10        10 FALSE             126.60      0.075    -0.5
#> 11        11 FALSE             126.60      0.075     0
#> 12        12 FALSE             126.60      0.075      0.5
#> 13        13 FALSE             126.60      0.1     -0.5
#> 14        14 FALSE             126.60      0.1       0
#> 15        15 FALSE             126.60      0.1      0.5
#> 16        16 FALSE             126.60      0.125    -0.5
#> 17        17 FALSE             126.60      0.125     0
#> 18        18 FALSE             126.60      0.125      0.5
#> 19        19 FALSE             133.27      0.075    -0.5
#> 20        20 FALSE             133.27      0.075     0
#> 21        21 FALSE             133.27      0.075      0.5
#> 22        22 FALSE             133.27      0.1     -0.5
#> 23        23 FALSE             133.27      0.1       0
#> 24        24 FALSE             133.27      0.1      0.5
#> 25        25 FALSE             133.27      0.125    -0.5
#> 26        26 FALSE             133.27      0.125     0
#> 27        27 FALSE             133.27      0.125      0.5
#> 28        28 FALSE             139.93      0.075    -0.5
#> 29        29 FALSE             139.93      0.075     0
#> 30        30 FALSE             139.93      0.075      0.5
#> 31        31 FALSE             139.93      0.1     -0.5
#> 32        32 FALSE             139.93      0.1       0
#> 33        33 FALSE             139.93      0.1      0.5
#> 34        34 FALSE             139.93      0.125    -0.5
#> 35        35 FALSE             139.93      0.125     0

```

#> 36	36 FALSE	139.93	0.125	0.5
#> 37	37 FALSE	146.59	0.075	-0.5
#> 38	38 FALSE	146.59	0.075	0
#> 39	39 FALSE	146.59	0.075	0.5
#> 40	40 FALSE	146.59	0.1	-0.5
#> 41	41 FALSE	146.59	0.1	0
#> 42	42 FALSE	146.59	0.1	0.5
#> 43	43 FALSE	146.59	0.125	-0.5
#> 44	44 FALSE	146.59	0.125	0
#> 45	45 FALSE	146.59	0.125	0.5

II-9

Spot Volatility

Domestic Foreign FX

22.3.1

```
tbl_prdc_scenario_input <-
  tbl_prdc_scenario |>
  mutate(
    market = purrr::map2(
      scenario_ID,
      fx_spot,
      \(scenario_ID,
        scenario_fx_spot) {
        tbl_prdc_market |>
          mutate(
            market_data_ID = paste0(
              "MKT-",
              stringr::str_pad(
                scenario_ID,
                width = 3,
                pad = "0"
              )
            ),
            fx_spot = scenario_fx_spot
          )
        }
      ),
    calibration = purrr::map2(
      fx_volatility,
```

```

correlation,
\ (scenario_volatility,
  scenario_correlation) {
tbl_prdc_calibration |>
  mutate(
    calibration_ID = paste0(
      "CAL-V",
      stringr::str_replace_all(
        format(
          scenario_volatility,
          trim = TRUE
        ),
        "\\.",
        "_"
      ),
      "-R",
      stringr::str_replace_all(
        format(
          scenario_correlation,
          trim = TRUE
        ),
        "-",
        "M",
        "\\." = "_"
      )
    ),
    fx_volatility =
      scenario_volatility,
    correlation =
      scenario_correlation,
    correlation_active =
      FALSE
  )
}
)
)

```

scenario_ID

ID

```

tbl_prdc_scenario_input <-
tbl_prdc_scenario |>

```

```

mutate(
  market = purrr::map2(
    scenario_ID,
    fx_spot,
    \(scenario_ID,
      scenario_fx_spot) {
      tbl_prdc_market |>
        mutate(
          market_data_ID =
            paste0(
              "MKT-",
              stringr::str_pad(
                scenario_ID,
                width = 3,
                pad = "0"
              )
            ),
          fx_spot =
            scenario_fx_spot
        )
      }
    ),
  calibration = purrr::map2(
    fx_volatility,
    correlation,
    \(scenario_volatility,
      scenario_correlation) {
      tbl_prdc_calibration |>
        mutate(
          fx_volatility =
            scenario_volatility,
          correlation =
            scenario_correlation,
          correlation_active =
            FALSE
        )
      }
    )
  )
)

```

22.3.2 future_map()

```
available_cores <-  
  future::availableCores()  
  
workers <- max(  
  1L,  
  min(  
    2L,  
    available_cores - 1L  
  )  
)  
  
future::plan(  
  future::multisession,  
  workers = workers  
)  
  
tbl_prdc_scenario_result <-  
  tbl_prdc_scenario_input |>  
  mutate(  
    result = furrr::future_map2(  
      market,  
      calibration,  
      \(scenario_market,  
        scenario_calibration) {  
        price_prdc_scenario_LFL(  
          trade =  
            tbl_prdc_trade,  
          market =  
            scenario_market,  
          calibration =  
            scenario_calibration,  
          calculation =  
            tbl_prdc_calculation  
        )  
      },  
    .options =  
      furrr::furrr_options(  
        seed = TRUE  
      )  
    )  
  )
```

```

) |>
select(
  scenario_ID,
  result
) |>
tidyr::unnest(result) |>
arrange(scenario_ID)

future::plan(
  future::sequential
)

show_table_LFL(
  tbl_prdc_scenario_result,
  n = Inf
)
#> # A tibble: 45 x 9
#>   scenario_ID trade_ID fx_spot fx_volatility correlation correlation_active
#>   <int> <chr>      <dbl>      <dbl>      <dbl> <lgl>
#> 1         1 PRDC-001 119.94      0.075     -0.5 FALSE
#> 2         2 PRDC-001 119.94      0.075      0 FALSE
#> 3         3 PRDC-001 119.94      0.075     0.5 FALSE
#> 4         4 PRDC-001 119.94      0.1     -0.5 FALSE
#> 5         5 PRDC-001 119.94      0.1      0 FALSE
#> 6         6 PRDC-001 119.94      0.1     0.5 FALSE
#> 7         7 PRDC-001 119.94      0.125    -0.5 FALSE
#> 8         8 PRDC-001 119.94      0.125      0 FALSE
#> 9         9 PRDC-001 119.94      0.125     0.5 FALSE
#> 10        10 PRDC-001 126.60      0.075    -0.5 FALSE
#> 11        11 PRDC-001 126.60      0.075      0 FALSE
#> 12        12 PRDC-001 126.60      0.075     0.5 FALSE
#> 13        13 PRDC-001 126.60      0.1     -0.5 FALSE
#> 14        14 PRDC-001 126.60      0.1      0 FALSE
#> 15        15 PRDC-001 126.60      0.1     0.5 FALSE
#> 16        16 PRDC-001 126.60      0.125    -0.5 FALSE
#> 17        17 PRDC-001 126.60      0.125      0 FALSE
#> 18        18 PRDC-001 126.60      0.125     0.5 FALSE
#> 19        19 PRDC-001 133.27      0.075    -0.5 FALSE
#> 20        20 PRDC-001 133.27      0.075      0 FALSE
#> 21        21 PRDC-001 133.27      0.075     0.5 FALSE
#> 22        22 PRDC-001 133.27      0.1     -0.5 FALSE
#> 23        23 PRDC-001 133.27      0.1      0 FALSE

```



```

#> 24      24 PRDC-001 133.27      0.1      0.5 FALSE
#> 25      25 PRDC-001 133.27      0.125    -0.5 FALSE
#> 26      26 PRDC-001 133.27      0.125      0  FALSE
#> 27      27 PRDC-001 133.27      0.125      0.5 FALSE
#> 28      28 PRDC-001 139.93      0.075    -0.5 FALSE
#> 29      29 PRDC-001 139.93      0.075      0  FALSE
#> 30      30 PRDC-001 139.93      0.075      0.5 FALSE
#> 31      31 PRDC-001 139.93      0.1      -0.5 FALSE
#> 32      32 PRDC-001 139.93      0.1      0  FALSE
#> 33      33 PRDC-001 139.93      0.1      0.5 FALSE
#> 34      34 PRDC-001 139.93      0.125    -0.5 FALSE
#> 35      35 PRDC-001 139.93      0.125      0  FALSE
#> 36      36 PRDC-001 139.93      0.125      0.5 FALSE
#> 37      37 PRDC-001 146.59      0.075    -0.5 FALSE
#> 38      38 PRDC-001 146.59      0.075      0  FALSE
#> 39      39 PRDC-001 146.59      0.075      0.5 FALSE
#> 40      40 PRDC-001 146.59      0.1      -0.5 FALSE
#> 41      41 PRDC-001 146.59      0.1      0  FALSE
#> 42      42 PRDC-001 146.59      0.1      0.5 FALSE
#> 43      43 PRDC-001 146.59      0.125    -0.5 FALSE
#> 44      44 PRDC-001 146.59      0.125      0  FALSE
#> 45      45 PRDC-001 146.59      0.125      0.5 FALSE
#>      number_of_paths random_seed  prdc_npv
#>      <int>          <int>      <dbl>
#> 1          500          123 316522073.
#> 2          500          123 316522073.
#> 3          500          123 316522073.
#> 4          500          123 317768256.
#> 5          500          123 317768256.
#> 6          500          123 317768256.
#> 7          500          123 317896344.
#> 8          500          123 317896344.
#> 9          500          123 317896344.
#> 10         500          123 327308562.
#> 11         500          123 327308562.
#> 12         500          123 327308562.
#> 13         500          123 326499769.
#> 14         500          123 326499769.
#> 15         500          123 326499769.
#> 16         500          123 325186860.
#> 17         500          123 325186860.
#> 18         500          123 325186860.

```

```

#> 19          500      123 336768079.
#> 20          500      123 336768079.
#> 21          500      123 336768079.
#> 22          500      123 334264600.
#> 23          500      123 334264600.
#> 24          500      123 334264600.
#> 25          500      123 331768191.
#> 26          500      123 331768191.
#> 27          500      123 331768191.
#> 28          500      123 344915395.
#> 29          500      123 344915395.
#> 30          500      123 344915395.
#> 31          500      123 341153967.
#> 32          500      123 341153967.
#> 33          500      123 341153967.
#> 34          500      123 337694221.
#> 35          500      123 337694221.
#> 36          500      123 337694221.
#> 37          500      123 351928229.
#> 38          500      123 351928229.
#> 39          500      123 351928229.
#> 40          500      123 347228273.
#> 41          500      123 347228273.
#> 42          500      123 347228273.
#> 43          500      123 343006994.
#> 44          500      123 343006994.
#> 45          500      123 343006994.

```

`map()` `future_map()`

```

Scenario Data
      +
Pricing Function
      ↓
future_map()
      ↓
Risk Table

```

22.4 3.4

Spot Volatility 0

```
tbl_prdc_base_scenario <-
tbl_prdc_scenario_result |>
filter(
  dplyr::near(
    fx_spot,
    tbl_prdc_market$fx_spot[[1]]
  ),
  dplyr::near(
    fx_volatility,
    tbl_prdc_calibration$
      fx_volatility[[1]]
  ),
  dplyr::near(
    correlation,
    0
  )
) |>
slice_head(n = 1)

base_prdc_npv <-
tbl_prdc_base_scenario$
  prdc_npv[[1]]
```

```
tbl_prdc_risk <- tbl_prdc_scenario_result |>
mutate(
  base_prdc_npv =
    base_prdc_npv,
  npv_change =
    prdc_npv -
    base_prdc_npv,
  npv_change_percent =
    npv_change /
    abs(base_prdc_npv),
  spot_change_percent =
    fx_spot /
    tbl_prdc_market$
      fx_spot[[1]] -
    1,
  volatility_change =
    fx_volatility -
```

```

tbl_prdc_calibration$
  fx_volatility[[1]]
)

show_table_LFL(
  tbl_prdc_risk,
  n = Inf
)
#> # A tibble: 45 x 14
#>   scenario_ID trade_ID fx_spot fx_volatility correlation correlation_active
#>   <int> <chr>      <dbl>      <dbl>      <dbl> <lgl>
#> 1         1 PRDC-001  119.94      0.075     -0.5 FALSE
#> 2         2 PRDC-001  119.94      0.075      0 FALSE
#> 3         3 PRDC-001  119.94      0.075      0.5 FALSE
#> 4         4 PRDC-001  119.94      0.1      -0.5 FALSE
#> 5         5 PRDC-001  119.94      0.1       0 FALSE
#> 6         6 PRDC-001  119.94      0.1      0.5 FALSE
#> 7         7 PRDC-001  119.94      0.125    -0.5 FALSE
#> 8         8 PRDC-001  119.94      0.125     0 FALSE
#> 9         9 PRDC-001  119.94      0.125     0.5 FALSE
#> 10        10 PRDC-001  126.60      0.075    -0.5 FALSE
#> 11        11 PRDC-001  126.60      0.075     0 FALSE
#> 12        12 PRDC-001  126.60      0.075     0.5 FALSE
#> 13        13 PRDC-001  126.60      0.1     -0.5 FALSE
#> 14        14 PRDC-001  126.60      0.1       0 FALSE
#> 15        15 PRDC-001  126.60      0.1      0.5 FALSE
#> 16        16 PRDC-001  126.60      0.125    -0.5 FALSE
#> 17        17 PRDC-001  126.60      0.125     0 FALSE
#> 18        18 PRDC-001  126.60      0.125     0.5 FALSE
#> 19        19 PRDC-001  133.27      0.075    -0.5 FALSE
#> 20        20 PRDC-001  133.27      0.075     0 FALSE
#> 21        21 PRDC-001  133.27      0.075     0.5 FALSE
#> 22        22 PRDC-001  133.27      0.1     -0.5 FALSE
#> 23        23 PRDC-001  133.27      0.1       0 FALSE
#> 24        24 PRDC-001  133.27      0.1      0.5 FALSE
#> 25        25 PRDC-001  133.27      0.125    -0.5 FALSE
#> 26        26 PRDC-001  133.27      0.125     0 FALSE
#> 27        27 PRDC-001  133.27      0.125     0.5 FALSE
#> 28        28 PRDC-001  139.93      0.075    -0.5 FALSE
#> 29        29 PRDC-001  139.93      0.075     0 FALSE
#> 30        30 PRDC-001  139.93      0.075     0.5 FALSE
#> 31        31 PRDC-001  139.93      0.1     -0.5 FALSE

```

```

#> 32      32 PRDC-001 139.93      0.1      0 FALSE
#> 33      33 PRDC-001 139.93      0.1      0.5 FALSE
#> 34      34 PRDC-001 139.93      0.125    -0.5 FALSE
#> 35      35 PRDC-001 139.93      0.125      0 FALSE
#> 36      36 PRDC-001 139.93      0.125      0.5 FALSE
#> 37      37 PRDC-001 146.59      0.075    -0.5 FALSE
#> 38      38 PRDC-001 146.59      0.075      0 FALSE
#> 39      39 PRDC-001 146.59      0.075      0.5 FALSE
#> 40      40 PRDC-001 146.59      0.1      -0.5 FALSE
#> 41      41 PRDC-001 146.59      0.1      0 FALSE
#> 42      42 PRDC-001 146.59      0.1      0.5 FALSE
#> 43      43 PRDC-001 146.59      0.125    -0.5 FALSE
#> 44      44 PRDC-001 146.59      0.125      0 FALSE
#> 45      45 PRDC-001 146.59      0.125      0.5 FALSE
#>      number_of_paths random_seed  prdc_npv base_prdc_npv npv_change
#>      <int>          <int>      <dbl>      <dbl>      <dbl>
#> 1          500          123 316522073.    334264600. -17742527.
#> 2          500          123 316522073.    334264600. -17742527.
#> 3          500          123 316522073.    334264600. -17742527.
#> 4          500          123 317768256.    334264600. -16496344.
#> 5          500          123 317768256.    334264600. -16496344.
#> 6          500          123 317768256.    334264600. -16496344.
#> 7          500          123 317896344.    334264600. -16368257.
#> 8          500          123 317896344.    334264600. -16368257.
#> 9          500          123 317896344.    334264600. -16368257.
#> 10         500          123 327308562.    334264600. -6956039.
#> 11         500          123 327308562.    334264600. -6956039.
#> 12         500          123 327308562.    334264600. -6956039.
#> 13         500          123 326499769.    334264600. -7764831.
#> 14         500          123 326499769.    334264600. -7764831.
#> 15         500          123 326499769.    334264600. -7764831.
#> 16         500          123 325186860.    334264600. -9077740.
#> 17         500          123 325186860.    334264600. -9077740.
#> 18         500          123 325186860.    334264600. -9077740.
#> 19         500          123 336768079.    334264600. 2503478.
#> 20         500          123 336768079.    334264600. 2503478.
#> 21         500          123 336768079.    334264600. 2503478.
#> 22         500          123 334264600.    334264600. 0
#> 23         500          123 334264600.    334264600. 0
#> 24         500          123 334264600.    334264600. 0
#> 25         500          123 331768191.    334264600. -2496410.
#> 26         500          123 331768191.    334264600. -2496410.

```

```

#> 27          500          123 331768191.    334264600.   -2496410.
#> 28          500          123 344915395.    334264600.    10650795.
#> 29          500          123 344915395.    334264600.    10650795.
#> 30          500          123 344915395.    334264600.    10650795.
#> 31          500          123 341153967.    334264600.    6889367.
#> 32          500          123 341153967.    334264600.    6889367.
#> 33          500          123 341153967.    334264600.    6889367.
#> 34          500          123 337694221.    334264600.    3429620.
#> 35          500          123 337694221.    334264600.    3429620.
#> 36          500          123 337694221.    334264600.    3429620.
#> 37          500          123 351928229.    334264600.    17663629.
#> 38          500          123 351928229.    334264600.    17663629.
#> 39          500          123 351928229.    334264600.    17663629.
#> 40          500          123 347228273.    334264600.    12963673.
#> 41          500          123 347228273.    334264600.    12963673.
#> 42          500          123 347228273.    334264600.    12963673.
#> 43          500          123 343006994.    334264600.     8742394.
#> 44          500          123 343006994.    334264600.     8742394.
#> 45          500          123 343006994.    334264600.     8742394.
#>      npv_change_percent spot_change_percent volatility_change
#>      <dbl>              <dbl>              <dbl>
#> 1      -0.053079          -0.1              -0.025
#> 2      -0.053079          -0.1              -0.025
#> 3      -0.053079          -0.1              -0.025
#> 4      -0.049351          -0.1               0
#> 5      -0.049351          -0.1               0
#> 6      -0.049351          -0.1               0
#> 7      -0.048968          -0.1              0.025
#> 8      -0.048968          -0.1              0.025
#> 9      -0.048968          -0.1              0.025
#> 10     -0.020810          -0.050000         -0.025
#> 11     -0.020810          -0.050000         -0.025
#> 12     -0.020810          -0.050000         -0.025
#> 13     -0.023230          -0.050000          0
#> 14     -0.023230          -0.050000          0
#> 15     -0.023230          -0.050000          0
#> 16     -0.027157          -0.050000         0.025
#> 17     -0.027157          -0.050000         0.025
#> 18     -0.027157          -0.050000         0.025
#> 19       0.0074895          0              -0.025
#> 20       0.0074895          0              -0.025
#> 21       0.0074895          0              -0.025

```

```

#> 22      0      0      0
#> 23      0      0      0
#> 24      0      0      0
#> 25 -0.0074684      0      0.025
#> 26 -0.0074684      0      0.025
#> 27 -0.0074684      0      0.025
#> 28  0.031863      0.050000     -0.025
#> 29  0.031863      0.050000     -0.025
#> 30  0.031863      0.050000     -0.025
#> 31  0.020611      0.050000      0
#> 32  0.020611      0.050000      0
#> 33  0.020611      0.050000      0
#> 34  0.010260      0.050000      0.025
#> 35  0.010260      0.050000      0.025
#> 36  0.010260      0.050000      0.025
#> 37  0.052843      0.10000     -0.025
#> 38  0.052843      0.10000     -0.025
#> 39  0.052843      0.10000     -0.025
#> 40  0.038783      0.10000      0
#> 41  0.038783      0.10000      0
#> 42  0.038783      0.10000      0
#> 43  0.026154      0.10000      0.025
#> 44  0.026154      0.10000      0.025
#> 45  0.026154      0.10000      0.025

```

22.4.1 FX Spot

Volatility 0

```

tbl_prdc_spot_risk <- tbl_prdc_risk |>
  filter(
    dplyr::near(
      fx_volatility,
      tbl_prdc_calibration$
        fx_volatility[[1]]
    ),
    dplyr::near(
      correlation,
      0
    )
  ) |>

```

```

arrange(fx_spot)

show_table_LFL(tbl_prdc_spot_risk)
#> # A tibble: 5 x 14
#>   scenario_ID trade_ID fx_spot fx_volatility correlation correlation_active
#>       <int> <chr>      <dbl>      <dbl>      <dbl> <lgl>
#> 1         5 PRDC-001  119.94        0.1          0 FALSE
#> 2        14 PRDC-001  126.60        0.1          0 FALSE
#> 3        23 PRDC-001  133.27        0.1          0 FALSE
#> 4        32 PRDC-001  139.93        0.1          0 FALSE
#> 5        41 PRDC-001  146.59        0.1          0 FALSE
#>   number_of_paths random_seed prdc_npv base_prdc_npv npv_change
#>       <int>      <int>      <dbl>      <dbl>      <dbl>
#> 1         500         123 317768256.    334264600. -16496344.
#> 2         500         123 326499769.    334264600.  -7764831.
#> 3         500         123 334264600.    334264600.      0
#> 4         500         123 341153967.    334264600.   6889367.
#> 5         500         123 347228273.    334264600.  12963673.
#>   npv_change_percent spot_change_percent volatility_change
#>       <dbl>      <dbl>      <dbl>
#> 1   -0.049351      -0.1          0
#> 2   -0.023230     -0.050000      0
#> 3      0           0          0
#> 4    0.020611     0.050000      0
#> 5    0.038783     0.10000      0

```

```

plot_prdc_spot_risk <-
  tbl_prdc_spot_risk |>
  ggplot(
    aes(
      x = fx_spot,
      y = prdc_npv
    )
  ) +
  geom_line(
    linewidth = 0.8
  ) +
  geom_point(
    size = 2
  ) +
  labs(
    x = "FX Spot",

```

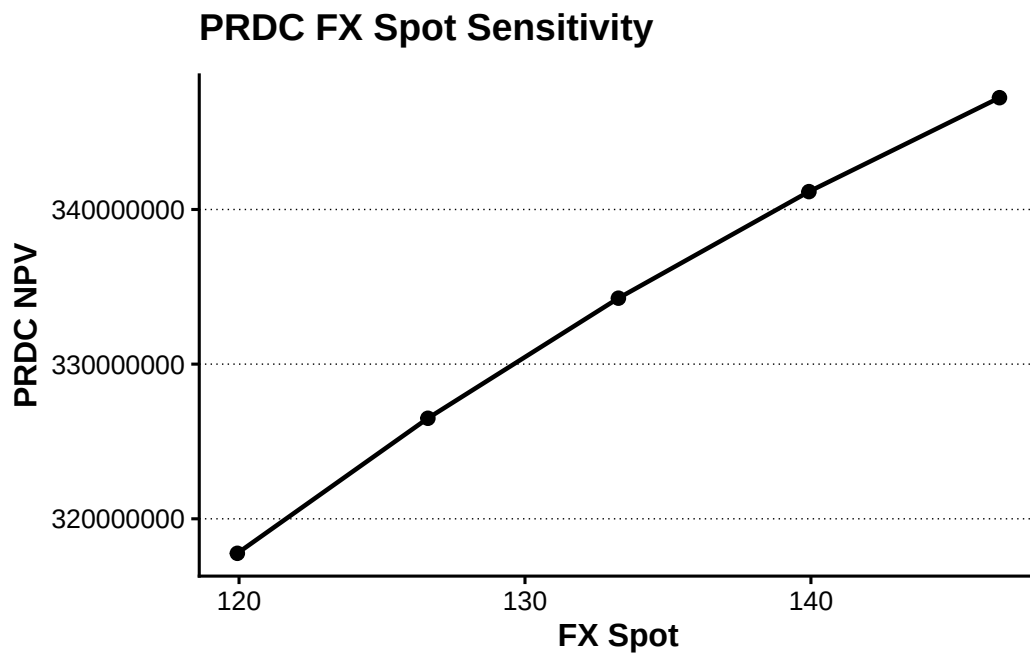


```

    y = "PRDC NPV",
    title = "PRDC FX Spot Sensitivity"
  )

show_plot_LFL(
  add_lagrange_theme_LFL(
    plot_prdc_spot_risk
  )
)

```



22.4.2 FX Volatility

```

tbl_prdc_volatility_risk <-
  tbl_prdc_risk |>
  filter(
    dplyr::near(
      fx_spot,
      tbl_prdc_market$fx_spot[[1]]
    ),
    dplyr::near(

```

```

    correlation,
    0
  )
) |>
arrange(fx_volatility)

show_table_LFL(
  tbl_prdc_volatility_risk
)
#> # A tibble: 3 x 14
#>   scenario_ID trade_ID fx_spot fx_volatility correlation correlation_active
#>   <int> <chr>      <dbl>      <dbl>      <dbl> <lgl>
#> 1      20 PRDC-001  133.27      0.075        0 FALSE
#> 2      23 PRDC-001  133.27      0.1          0 FALSE
#> 3      26 PRDC-001  133.27      0.125        0 FALSE
#>   number_of_paths random_seed prdc_npv base_prdc_npv npv_change
#>   <int>      <int>      <dbl>      <dbl>      <dbl>
#> 1      500      123 336768079.    334264600.    2503478.
#> 2      500      123 334264600.    334264600.         0
#> 3      500      123 331768191.    334264600.   -2496410.
#>   npv_change_percent spot_change_percent volatility_change
#>   <dbl>      <dbl>      <dbl>      <dbl>
#> 1    0.0074895      0          0        -0.025
#> 2      0          0          0         0
#> 3   -0.0074684      0          0         0.025

```

22.4.3

```

tbl_prdc_correlation_scenario <-
  tbl_prdc_risk |>
  filter(
    dplyr::near(
      fx_spot,
      tbl_prdc_market$fx_spot[[1]]
    ),
    dplyr::near(
      fx_volatility,
      tbl_prdc_calibration$
        fx_volatility[[1]]
    )
  )

```

```

) |>
select(
  scenario_ID,
  correlation,
  correlation_active,
  prdc_npv,
  npv_change
) |>
arrange(correlation)

show_table_LFL(
  tbl_prdc_correlation_scenario
)
#> # A tibble: 3 x 5
#>   scenario_ID correlation correlation_active prdc_npv npv_change
#>   <int>      <dbl> <lgl>                <dbl>      <dbl>
#> 1         22    -0.5 FALSE                334264600.      0
#> 2         23      0 FALSE                334264600.      0
#> 3         24    0.5 FALSE                334264600.      0

```

correlation_active FALSE PRDC

price_prdc_scenario_LFL() Trade Data Market Data Calibration Data Sce-

nario Grid future_map() Risk Table

22.5 3.5 Arrow

Parquet

```

tbl_prdc_output <- list(
  trade_data =
    tbl_prdc_trade,
  market_data =
    tbl_prdc_market,
  calibration_data =
    tbl_prdc_calibration,
  scenario_grid =
    tbl_prdc_scenario,
  scenario_results =
    tbl_prdc_scenario_result,
  risk_table =

```

```

    tbl_prdc_risk
  )

vec_prdc_output_files <- c(
  trade_data =
    file.path(
      "risk_system",
      "prdc_trade_data.parquet"
    ),
  market_data =
    file.path(
      "risk_system",
      "prdc_market_data.parquet"
    ),
  calibration_data =
    file.path(
      "risk_system",
      "prdc_calibration_data.parquet"
    ),
  scenario_grid =
    file.path(
      "risk_system",
      "prdc_scenario_grid.parquet"
    ),
  scenario_results =
    file.path(
      "risk_system",
      "prdc_scenario_results.parquet"
    ),
  risk_table =
    file.path(
      "risk_system",
      "prdc_risk_table.parquet"
    )
)

vec_prdc_output_paths <-
  tbl_prdc_output |>
  purrr::imap_chr(
    \(data, name) {
      write_derived_parquet_file_LFL(
        data,

```

```

        vec_prdc_output_files[[name]]
    )
}
)

vec_prdc_output_paths
#>
#>      "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/risk_system/prdc_trade_data.p
#>
#>      "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/risk_system/prdc_market_data.p
#>
#>      "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/risk_system/prdc_calibration
#> "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/risk_system/prdc_calibration_data.p
#>
#>      "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/risk_system/prdc_scenario_grid.p
#>
#>      "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/risk_system/prdc_scenario_results.p
#>
#>      "C:/AnalyticFin/Projects/LagrangeFinance/data/derived/risk_system/prdc_risk_table.p

```

```

tbl_prdc_risk_check <-
  read_derived_parquet_file_LFL(
    file.path(
      "risk_system",
      "prdc_risk_table.parquet"
    )
  )

tbl_prdc_output_check <- tibble(
  expected_rows =
    nrow(tbl_prdc_risk),
  actual_rows =
    nrow(tbl_prdc_risk_check),
  same_columns =
    identical(
      names(tbl_prdc_risk),
      names(tbl_prdc_risk_check)
    )
)

show_table_LFL(

```

```
tbl_prdc_output_check
)
#> # A tibble: 1 x 3
#>   expected_rows actual_rows same_columns
#>       <int>       <int> <lgl>
#> 1         45         45  TRUE
```

```
stopifnot(
  nrow(tbl_prdc_risk_check) ==
    nrow(tbl_prdc_risk),
  identical(
    names(tbl_prdc_risk_check),
    names(tbl_prdc_risk)
  )
)
```

II-9 PRDC

```
Trade Data
Market Data
Calibration Data
  ↓
Scenario Grid
  ↓
future_map()
  ↓
PRDC Pricing
  ↓
Risk Table
  ↓
Parquet
  ↓
Report
```

PRDC	OIS Swap CDS
I	II III

Parquet